SQL & Database Design

A Complete Guide

Master SQL Queries, Database Design, and Performance -- With PostgreSQL

2025 Edition • 34 Chapters

Table of Contents

Chapter 1 — What Are Databases
Chapter 2 — The Relational Model
Chapter 3 — Installing PostgreSQL
Chapter 4 — Creating Databases and Tables
Chapter 5 — Data Types
Chapter 6 — Inserting Data
Chapter 7 — SELECT Queries
Chapter 8 — WHERE Filtering
Chapter 9 — Sorting and Limiting
Chapter 10 — Aggregate Functions
Chapter 11 — GROUP BY and HAVING
Chapter 12 — Introduction to Joins
Chapter 13 — Inner Join
Chapter 14 — Left and Right Join
Chapter 15 — Full, Cross, and Self Join
Chapter 16 — Subqueries
Chapter 17 — Set Operations
Chapter 18 — String and Date Functions
Chapter 19 — CASE Expressions
Chapter 20 — Views
Chapter 21 — Indexes
Chapter 22 — Normalization
Chapter 23 — ER Diagrams
Chapter 24 — Constraints and Keys
Chapter 25 — Transactions
Chapter 26 — Window Functions
Chapter 27 — CTEs and Recursive Queries

Chapter 28 — Stored Procedures
Chapter 29 — Triggers
Chapter 30 — Performance Tuning
Chapter 31 — Database Security
Chapter 32 — Project: E-Commerce Database
Chapter 33 — Project: Analytics
Chapter 34 — Glossary

Chapter 1: What Are Databases?

What You Will Learn

By the end of this chapter, you will understand:

- What a database is and why it exists
- Why spreadsheets are not enough for serious data
- The different types of databases (relational, NoSQL, graph)
- What an RDBMS is and which popular ones exist
- What SQL is and why every developer needs it
- The four basic operations you can do with data (CRUD)
- Real-world examples of databases in action

Why This Chapter Matters

Every application you use daily sits on top of a database. When you log into Instagram, check your bank balance, or order food online, a database is doing the heavy lifting behind the scenes. Understanding databases is not optional for anyone working with technology. It is foundational.

This chapter gives you the big picture before we dive into the details. Think of it as looking at a map before starting a road trip. You need to know where you are going before you start driving.

What Is a Database?

A database is an organized collection of data that can be easily accessed, managed, and updated.

That definition is correct, but let us make it real.

The Filing Cabinet Analogy

Imagine you run a small school. You need to keep track of students, teachers, courses, and grades. In the old days, you would use a metal filing cabinet.

```
+-----------------------------------------------------+
|                   FILING CABINET                    |
|                                                     |
|  +------------+  +------------+  +------------+    |
|  |  Drawer 1  |  |  Drawer 2  |  |  Drawer 3  |    |
|  |  STUDENTS  |  |  TEACHERS  |  |  COURSES   |    |
|  +------------+  +------------+  +------------+    |
|                                                     |
|  Each drawer has folders.                           |
|  Each folder has one person or course.              |
|  Each folder has papers with details inside.        |
+-----------------------------------------------------+
```

- The filing cabinet is the database.
- Each drawer is a table (a collection of related data).
- Each folder is a row (one specific record, like one student).
- The papers inside are columns (individual pieces of information, like name, age, email).

A database works exactly like this, but it is digital, lightning fast, and can hold millions of records without breaking a sweat.

A Simple Example

Here is what a "students" table might look like inside a database:

```
+----+-----------+----------+-----+------------------+
| id | first_name| last_name| age | email            |
+----+-----------+----------+-----+------------------+
|  1 | Alice     | Johnson  |  20 | alice@email.com  |
|  2 | Bob       | Smith    |  22 | bob@email.com    |
|  3 | Charlie   | Brown    |  19 | charlie@email.com|
+----+-----------+----------+-----+------------------+
```

Each row is one student. Each column is one piece of information about that student. This is structured, organized, and easy to search.

Why Not Just Use a Spreadsheet?

Great question. If a database looks like a table, why not just use Excel or Google Sheets?

Spreadsheets are fantastic for small, simple tasks. But they fall apart when things get serious. Here is a comparison:

```
+-------------------------+-------------------+--------------------+
| Feature                 | Spreadsheet       | Database           |
+-------------------------+-------------------+--------------------+
| Max rows                | ~1 million        | Billions+          |
| Multiple users at once  | Limited           | Thousands          |
| Data validation         | Manual            | Automatic (rules)  |
| Relationships between   | Copy-paste        | Built-in links     |
|   data                  |                   |   (foreign keys)   |
| Security                | File password     | User roles,        |
|                         |                   |   permissions      |
| Speed with large data   | Slow, crashes     | Fast, optimized    |
| Backup and recovery     | Manual            | Automatic          |
| Duplicate prevention    | None              | Built-in           |
+-------------------------+-------------------+--------------------+
```

When Spreadsheets Break Down

Imagine you run an online store with 50,000 products and 200,000 customers. You need to:

1. Track every order (who bought what, when, for how much).
2. Make sure two people do not buy the last item at the same time.
3. Let 50 employees access the data simultaneously.
4. Generate reports instantly.
5. Keep customer credit card data secure.

A spreadsheet cannot do any of this reliably. A database can.

The Key Difference

A spreadsheet is a tool for one person to organize small amounts of data.

A database is a tool for many people (or applications) to organize large amounts of data with rules, security, and speed.

Types of Databases

Not all databases are the same. There are several types, each designed for different situations.

1. Relational Databases (RDBMS)

This is the most common type and the focus of this book.

How it works: Data is stored in tables with rows and columns. Tables can be linked (related) to each other using keys.

Think of it as: A collection of interconnected spreadsheets with strict rules.

```
+------------------+         +------------------+
|     students     |         |     courses      |
+------------------+         +------------------+
| id (PK)          |         | id (PK)          |
| first_name       |         | course_name      |
| last_name        |         | credits          |
| email            |         +------------------+
+------------------+
        |
        |  (linked by student_id)
        v
+------------------+
|   enrollments    |
+------------------+
| id (PK)          |
| student_id (FK)  |----> points to students.id
| course_id (FK)   |----> points to courses.id
| grade            |
+------------------+
```

PK means Primary Key (a unique identifier, like a student ID number). FK means Foreign Key (a reference to another table's primary key).

Examples: PostgreSQL, MySQL, SQLite, Oracle, Microsoft SQL Server.

Best for: Most applications. Banking, e-commerce, healthcare, education, and more.

2. NoSQL Databases

NoSQL stands for "Not Only SQL." These databases do not use the traditional table structure.

How it works: Data is stored in flexible formats like documents, key-value pairs, or wide columns.

Think of it as: A folder where you can throw in any kind of document without following a strict template.

```
// A document in a NoSQL database (like MongoDB)
{
    "name": "Alice Johnson",
    "age": 20,
    "courses": ["Math 101", "History 202"],
    "address": {
        "street": "123 Main St",
        "city": "Springfield"
    }
}
```

Examples: MongoDB (documents), Redis (key-value), Cassandra (wide column).

Best for: Applications that need flexibility, like social media feeds, real-time analytics, or when your data structure changes frequently.

3. Graph Databases

How it works: Data is stored as nodes (things) and edges (relationships between things).

Think of it as: A social network map where people are dots and friendships are lines connecting them.

```
    (Alice)---[FRIENDS_WITH]--->(Bob)
       |                         |
       |                         |
  [ENROLLED_IN]           [ENROLLED_IN]
       |                         |
       v                         v
   (Math 101)              (History 202)
```

Examples: Neo4j, Amazon Neptune.

Best for: Social networks, recommendation engines, fraud detection, or any data where relationships are the most important part.

Quick Comparison

```
+------------------+-------------------+---------------------------+
| Type             | Structure         | Best For                  |
+------------------+-------------------+---------------------------+
| Relational       | Tables with rows  | Most applications,        |
```

```
|  (RDBMS)          |  and columns      |  structured data          |
+-------------------+-------------------+---------------------------+
| NoSQL             | Documents, key-   | Flexible data, real-time  |
|                   |  value, columns   |  apps, big data           |
+-------------------+-------------------+---------------------------+
| Graph             | Nodes and edges   | Relationship-heavy data,  |
|                   |                   |  social networks          |
+-------------------+-------------------+---------------------------+
```

In this book, we focus entirely on relational databases using PostgreSQL. Relational databases power the vast majority of applications, and the skills you learn here transfer to any RDBMS.

What Is an RDBMS?

RDBMS stands for Relational Database Management System. Let us break that down:

- Relational: Data is organized in tables that can relate to each other.
- Database: An organized collection of data.
- Management System: The software that lets you create, read, update, and delete data.

An RDBMS is the software you install on your computer (or server) that manages your databases. You do not interact with the data files directly. Instead, you talk to the RDBMS using SQL, and it handles everything for you.

```
+----------+         +-----------+         +----------+
|   You    | ------> |   RDBMS   | ------> |   Data   |
| (write   |  SQL    | (software |  reads/ |  (files  |
|  SQL)    |         |  engine)  |  writes |  on disk)|
+----------+         +-----------+         +----------+
```

Popular RDBMS Options

```
+-------------------+--------+--------+---------+-------------------+
| RDBMS             | Cost   | Speed  | Scale   | Common Use        |
+-------------------+--------+--------+---------+-------------------+
| PostgreSQL        | Free   | Fast   | Large   | Web apps, analytics|
| MySQL             | Free   | Fast   | Large   | Web apps, CMS      |
| SQLite            | Free   | Fast   | Small   | Mobile apps, local |
| Oracle            | Paid   | Fast   | Huge    | Enterprise         |
| SQL Server        | Paid   | Fast   | Huge    | Enterprise (.NET)  |
```

```
+-----------------+--------+--------+---------+-----------------+
```

Why PostgreSQL?

This book uses PostgreSQL (often called "Postgres") for all examples. Here is why:

1. It is free and open source. You will never pay a dime.
2. It is powerful. It supports advanced features that other free databases do not.
3. It is popular. Companies like Apple, Instagram, Spotify, and Netflix use it.
4. It follows the SQL standard closely. What you learn in PostgreSQL works almost everywhere.
5. It has an amazing community. Help is always available.

> How to pronounce it: "post-GRESS-Q-L" or just "post-GRESS." Both are correct.

What Is SQL?

SQL stands for Structured Query Language. It is pronounced "S-Q-L" (each letter separately) or "sequel" (like a movie sequel). Both are correct.

SQL is the language you use to talk to a relational database. You write SQL commands, and the database responds.

Think of it as: SQL is to databases what English is to people. It is how you communicate.

```
+----------+                    +-----------+
|   You    |  "Show me all     |  Database |
|          |   students who    |           |
|          |   are older       |           |
|          |   than 20"        |           |
|          | ----------------> |           |
|          |                   |           |
|          |  Here are the     |           |
|          |  results:         |           |
|          |  Bob (22)         |           |
|          | <---------------- |           |
+----------+                    +-----------+
```

In SQL, that request looks like this:

```sql
SELECT first_name, age
FROM students
WHERE age > 20;
```

And the database responds:

```
 first_name | age
------------+-----
 Bob        |  22
(1 row)
```

Let us break down that SQL statement line by line:

- `SELECT first_name, age` means "I want to see the first_name and age columns."
- `FROM students` means "Look in the students table."
- `WHERE age > 20` means "But only show me rows where age is greater than 20."
- The semicolon (`;`) marks the end of the command, like a period at the end of a sentence.

SQL Is Universal

The beautiful thing about SQL is that it works across almost all relational databases. Learn SQL once, and you can use PostgreSQL, MySQL, SQLite, Oracle, or SQL Server. The syntax might have tiny differences between them, but the core is the same.

CRUD: The Four Things You Do with Data

Every database operation falls into one of four categories. Together, they are called CRUD:

```
+----------+-------------+-------------------------------+
| Letter   | Operation   | What It Does                  |
+----------+-------------+-------------------------------+
| C        | CREATE      | Add new data                  |
| R        | READ        | Retrieve existing data        |
| U        | UPDATE      | Change existing data          |
| D        | DELETE      | Remove data                   |
+----------+-------------+-------------------------------+
```

Here is what each looks like in SQL:

Create (INSERT)

Add a new student to the database:

```sql
INSERT INTO students (first_name, last_name, age, email)
VALUES ('Diana', 'Prince', 21, 'diana@email.com');
```

Line-by-line:

- `INSERT INTO students` means "I want to add a new row to the students table."
- `(first_name, last_name, age, email)` lists the columns we are providing data for.
- `VALUES ('Diana', 'Prince', 21, 'diana@email.com')` provides the actual data.

Read (SELECT)

Get all students from the database:

```sql
SELECT * FROM students;
```

```
 id | first_name | last_name | age | email
----+------------+-----------+-----+-------------------
  1 | Alice      | Johnson   |  20 | alice@email.com
  2 | Bob        | Smith     |  22 | bob@email.com
  3 | Charlie    | Brown     |  19 | charlie@email.com
  4 | Diana      | Prince    |  21 | diana@email.com
(4 rows)
```

Line-by-line:

- `SELECT *` means "I want to see all columns." The asterisk (`*`) is a wildcard that means "everything."
- `FROM students` means "Look in the students table."

Update (UPDATE)

Change Bob's age to 23:

```sql
UPDATE students
SET age = 23
WHERE first_name = 'Bob';
```

Line-by-line:

- `UPDATE students` means "I want to change data in the students table."
- `SET age = 23` means "Change the age column to 23."
- `WHERE first_name = 'Bob'` means "But only for the row where first_name is Bob."

> Warning: If you forget the `WHERE` clause, every row in the table gets updated. This is a very common and dangerous mistake.

Delete (DELETE)

Remove Charlie from the database:

```sql
DELETE FROM students
WHERE first_name = 'Charlie';
```

Line-by-line:

- `DELETE FROM students` means "I want to remove rows from the students table."
- `WHERE first_name = 'Charlie'` means "But only the row where first_name is Charlie."

> Warning: If you forget the `WHERE` clause, every row in the table gets deleted. Always double-check your `DELETE` statements.

Real-World Database Examples

Databases are everywhere. Here are some examples to show you just how important they are.

Banking

When you check your bank balance, a database is involved.

```
+-------------------+          +-------------------+
|    customers      |          |   transactions    |
+-------------------+          +-------------------+
| id                |          | id                |
| name              |          | customer_id (FK)  |
| account_number    |          | amount            |
| balance           |          | type (debit/      |
| created_at        |          |   credit)         |
+-------------------+          | timestamp         |
                               +-------------------+
```

The bank needs to:

- Store millions of customer accounts.
- Record every transaction (deposit, withdrawal, transfer).
- Make sure money does not disappear or get duplicated.
- Process thousands of transactions per second.
- Keep data secure and private.

Only a database can handle all of this.

Hospitals

When you visit a doctor, databases track your medical history.

```
+-------------------+          +-------------------+
|    patients       |          |   appointments    |
+-------------------+          +-------------------+
| id                |          | id                |
| name              |          | patient_id (FK)   |
| date_of_birth     |          | doctor_id (FK)    |
| blood_type        |          | date_time         |
| allergies         |          | reason            |
+-------------------+          +-------------------+
          |
          v
+-------------------+
|  prescriptions    |
+-------------------+
| id                |
| patient_id (FK)   |
| medicine_name     |
| dosage            |
| start_date        |
+-------------------+
```

The hospital needs to:

- Track patient records, allergies, and medications.
- Schedule appointments across hundreds of doctors.
- Make sure the wrong medicine is never prescribed.
- Keep everything private and compliant with regulations.

E-Commerce (Online Shopping)

When you buy something on Amazon, multiple databases work together.

```
+------------------+     +------------------+     +------------------+
|    customers     |     |     orders       |     |    products      |
+------------------+     +------------------+     +------------------+
| id               |     | id               |     | id               |
| name             |     | customer_id (FK) |     | name             |
| email            |     | order_date       |     | price            |
| shipping_address |     | total_amount     |     | stock_quantity   |
+------------------+     | status           |     | category         |
                         +------------------+     +------------------+
                                  |
                                  v
                         +------------------+
                         |   order_items    |
                         +------------------+
                         | id               |
                         | order_id (FK)    |
                         | product_id (FK)  |
                         | quantity         |
                         | price_at_purchase|
                         +------------------+
```

The store needs to:

- Show product details to millions of visitors.
- Handle thousands of simultaneous purchases.
- Update stock in real time so you cannot buy an item that is out of stock.
- Track orders from placement to delivery.
- Generate sales reports.

Social Media

When you scroll through your feed, databases are working overtime.

```
+------------------+     +------------------+     +------------------+
|     users        |     |      posts       |     |    comments      |
+------------------+     +------------------+     +------------------+
| id               |     | id               |     | id               |
| username         |     | user_id (FK)     |     | post_id (FK)     |
| email            |     | content          |     | user_id (FK)     |
| profile_photo    |     | image_url        |     | text             |
| join_date        |     | created_at       |     | created_at       |
+------------------+     | likes_count      |     +------------------+
                         +------------------+
```

Education

When your school posts grades online, a database stores them.

```
+------------------+     +------------------+     +------------------+
|    students      |     |   enrollments    |     |    courses       |
+------------------+     +------------------+     +------------------+
| id               |     | id               |     | id               |
| first_name       |     | student_id (FK)  |     | course_name      |
| last_name        |     | course_id (FK)   |     | instructor       |
| email            |     | semester         |     | credits          |
| gpa              |     | grade            |     | department       |
+------------------+     +------------------+     +------------------+
```

Putting It All Together

Let us recap the big picture with one diagram:

```
+----------------------------------------------------------------+
|                    THE DATABASE WORLD                           |
|                                                                |
|   You write SQL -----> RDBMS (PostgreSQL) -----> Data on disk  |
|                                                                |
|   SQL lets you:                                                |
|     C - Create data   (INSERT)                                 |
|     R - Read data     (SELECT)                                 |
|     U - Update data   (UPDATE)                                 |
|     D - Delete data   (DELETE)                                 |
|                                                                |
|   Data is organized in:                                        |
|     - Databases (the filing cabinet)                           |
|     - Tables (the drawers)                                     |
|     - Rows (the folders)                                       |
|     - Columns (the papers inside)                              |
|                                                                |
|   Tables can be connected using keys:                          |
|     - Primary Key (unique ID for each row)                     |
|     - Foreign Key (link to another table)                      |
+----------------------------------------------------------------+
```

Common Mistakes

1. Thinking databases and spreadsheets are the same thing. They serve different purposes. Spreadsheets are for small, personal data. Databases are for large-scale, multi-user applications.

2. Thinking SQL is a programming language like Python or Java. SQL is a query language. You use it to ask questions and give instructions to a database. You do not build applications with SQL alone.
3. Confusing a database with a database management system. A database is the data. An RDBMS (like PostgreSQL) is the software that manages the data.
4. Thinking you need to choose between SQL and NoSQL. Many real-world applications use both. They solve different problems.
5. Being afraid of SQL. SQL is one of the most approachable languages in all of technology. If you can write an English sentence, you can learn SQL.

Best Practices

1. Start with relational databases. They cover the vast majority of use cases, and the skills transfer everywhere.
2. Learn SQL before learning any specific database tool. SQL is the universal skill. Tools change; SQL stays.
3. Always think about your data structure before writing code. Planning your tables and relationships up front saves enormous time later.
4. Practice with real-world examples. Do not just read about databases. Build them. Break them. Fix them.
5. Use PostgreSQL for learning. It is free, powerful, standards-compliant, and widely used in industry.

Quick Summary

A database is an organized collection of data managed by software called an RDBMS. Relational databases store data in tables with rows and columns. Tables are linked using primary keys and foreign keys. SQL is the language used to interact with relational databases. Every data operation falls into one of four CRUD categories: Create, Read, Update, or Delete. Databases power virtually every application you use, from banking to social media.

Key Points

- A database is an organized, digital collection of data.
- A table holds related data in rows (records) and columns (fields).
- An RDBMS is the software that manages relational databases.
- SQL is the language you use to communicate with an RDBMS.
- CRUD stands for Create, Read, Update, Delete.
- PostgreSQL is a free, powerful, and widely-used RDBMS.
- Relational databases use primary keys and foreign keys to connect tables.
- Databases handle things spreadsheets cannot: scale, security, concurrency, and data integrity.

Practice Questions

1. In the filing cabinet analogy, what does each drawer represent in database terms? What about each folder?
2. Name three things a database can do that a spreadsheet cannot do well.
3. What does CRUD stand for? Write the SQL keyword associated with each letter.
4. What is the difference between a database and a database management system (RDBMS)?
5. You are building an app to track books in a library. What tables might you need? What columns would each table have?

Exercises

Exercise 1: Identify the Database

Think about three applications you use every day (for example, a music streaming service, a ride-sharing app, or an online store). For each application, list:

- At least two tables the database probably has.
- At least three columns for each table.
- How the tables might be connected.

Exercise 2: Spreadsheet vs Database

You run a small bakery with 10 employees and 50 regular customers. Right now, you track orders in a spreadsheet. Write down three specific problems you would face if the bakery grows to 500 employees and 50,000 customers. Explain how a database would solve each problem.

Exercise 3: Match the Database Type

For each scenario below, decide whether a relational database, NoSQL database, or graph database would be the best fit. Explain your reasoning.

1. An airline reservation system that tracks flights, passengers, and bookings.
2. A social network that needs to find "friends of friends" quickly.
3. A mobile app that stores user preferences as flexible, nested data.

What Is Next?

Now that you understand what databases are and why they matter, it is time to go deeper. In Chapter 2, you will learn about the relational model in detail. You will understand exactly how tables, rows, columns, primary keys, and foreign keys work together to create a powerful, organized system for your data. This is the foundation that everything else in this book builds on.

Chapter 2: The Relational Model

What You Will Learn

By the end of this chapter, you will understand:

- What tables, rows, and columns really are
- What a primary key is and why every table needs one
- What a foreign key is and how it connects tables
- The three types of relationships: one-to-one, one-to-many, and many-to-many
- What a schema is
- How to read and draw ASCII table diagrams
- A complete Students/Courses example that ties everything together

Why This Chapter Matters

The relational model is the foundation of every relational database. It is the blueprint. If you skip this chapter, everything that follows will feel confusing. If you understand this chapter well, everything that follows will feel logical.

Think of it this way: before you build a house, you need to understand walls, floors, doors, and how rooms connect. The relational model teaches you the "architecture" of databases.

Tables: The Core Building Block

A table is where your data lives. Every piece of data in a relational database is stored in a table.

Think of it as: A spreadsheet with strict rules.

Here is a table called `students`:

```
+----+------------+-----------+-----+-------------------+
| id | first_name | last_name | age | email             |
+----+------------+-----------+-----+-------------------+
|  1 | Alice      | Johnson   |  20 | alice@email.com   |
|  2 | Bob        | Smith     |  22 | bob@email.com     |
|  3 | Charlie    | Brown     |  19 | charlie@email.com |
|  4 | Diana      | Prince    |  21 | diana@email.com   |
+----+------------+-----------+-----+-------------------+
```

A table has a name (like `students`). You choose the name when you create the table. The name should describe what kind of data the table holds.

Rules That Make Tables Different from Spreadsheets

1. Every column has a fixed data type. The `age` column only accepts numbers. The `email` column only accepts text. You cannot put a name in the age column.
2. Every column has a name. No unnamed columns allowed.
3. Every row follows the same structure. You cannot have one row with 5 columns and another with 3 columns.
4. There is a way to uniquely identify each row. This is called a primary key (more on this shortly).

Rows: Individual Records

A row (also called a record or tuple) represents one specific item in the table.

In the `students` table above, each row is one student:

```
Row 1: Alice Johnson, age 20, alice@email.com
Row 2: Bob Smith, age 22, bob@email.com
Row 3: Charlie Brown, age 19, charlie@email.com
Row 4: Diana Prince, age 21, diana@email.com
```

Think of it as: One folder in your filing cabinet. The folder for Alice contains all of Alice's information.

When you add a new student to the database, you are adding a new row. When you remove a student, you are removing a row.

Columns: Attributes of Your Data

A column (also called a field or attribute) represents one specific piece of information.

In the `students` table:

```
+----+------------+-----------+-----+-------------------+
| id | first_name | last_name | age | email             |
+----+------------+-----------+-----+-------------------+
  ^       ^            ^        ^         ^
  |       |            |        |         |
  |       |            |        |         +-- Column 5: email address
  |       |            |        +------------ Column 4: student's age
  |       |            +--------------------- Column 3: last name
  |       +----------------------------------- Column 2: first name
  +------------------------------------------ Column 1: unique ID
```

Think of it as: The labels on a form. First Name: , Last Name: , Age: , Email: .

Every column has:

- A name (like `first_name`).
- A data type (like text, number, or date). We will cover data types in detail in Chapter 5.
- Rules (like "this column cannot be empty"). These are called constraints, and we will cover them in Chapter 4.

Primary Keys: The Unique Identifier

A primary key is a column (or combination of columns) that uniquely identifies each row in a table. No two rows can have the same primary key value.

Think of it as: Your government-issued ID card. No two people have the same ID number. Even if two people share the same name, their ID numbers are different.

```
+----+------------+-----------+
| id | first_name | last_name |
+----+------------+-----------+
|  1 | Alice      | Johnson   |  <-- id = 1 (unique)
|  2 | Bob        | Smith     |  <-- id = 2 (unique)
```

```
|  3 | Alice      | Williams  |  <-- id = 3 (unique, even though
|    |            |           |      first_name is also "Alice")
+----+------------+-----------+
  ^
  |
  Primary Key (PK): Every value is unique, never NULL
```

Rules of Primary Keys

1. Every value must be unique. No duplicates.
2. No value can be NULL (empty). Every row must have a primary key value.
3. The value should never change. Once assigned, it stays forever.
4. Every table should have one. A table without a primary key is like a filing cabinet without labels. It works, but it is a mess.

Common Primary Key Patterns

The most common approach is to use an auto-incrementing integer:

```sql
CREATE TABLE students (
    id SERIAL PRIMARY KEY,
    first_name VARCHAR(50),
    last_name VARCHAR(50)
);
```

Line-by-line:

- `id SERIAL PRIMARY KEY` creates a column called `id` that automatically generates a unique number for each new row. `SERIAL` is a PostgreSQL feature that auto-increments (1, 2, 3, 4, ...).
- `first_name VARCHAR(50)` creates a text column that can hold up to 50 characters.
- `last_name VARCHAR(50)` creates another text column.

When you insert data, you do not need to provide the `id`. PostgreSQL generates it for you:

```sql
INSERT INTO students (first_name, last_name) VALUES ('Alice', 'Johnson');
INSERT INTO students (first_name, last_name) VALUES ('Bob', 'Smith');
```

```
 id | first_name | last_name
----+------------+-----------
  1 | Alice      | Johnson
  2 | Bob        | Smith
```

```
(2 rows)
```

Why Not Use a Name as the Primary Key?

You might think, "Why not just use the student's name as the primary key?" Here is why:

1. Names are not unique. There can be two students named "Alice Johnson."
2. Names change. People get married, change their names, or fix typos.
3. Names are long. Numbers are faster for the database to search and compare.

Always use a numeric ID or UUID as your primary key.

Foreign Keys: Links Between Tables

A foreign key is a column in one table that refers to the primary key of another table. It creates a link between the two tables.

Think of it as: A pointer or reference. It says, "This row is connected to that row in another table."

Let us say we have two tables: `students` and `courses`. We want to track which students are enrolled in which courses. We create a third table called `enrollments`:

```
+-------------------+         +-------------------+
|     students      |         |     courses       |
+-------------------+         +-------------------+
| id (PK)      = 1 |         | id (PK)      = 10|
| first_name        |         | course_name       |
| last_name         |         | credits           |
+-------------------+         +-------------------+
        ^                             ^
        |                             |
        |    +-------------------+    |
        |    |   enrollments     |    |
        |    +-------------------+    |
        +----| student_id (FK)   |    |
             | course_id (FK)    |----+
             | grade             |
             +-------------------+
```

In the `enrollments` table:

- `student_id` is a foreign key that points to `students.id`.

- `course_id` is a foreign key that points to `courses.id`.

Here is what the data looks like:

students table:

```
+----+------------+-----------+
| id | first_name | last_name |
+----+------------+-----------+
|  1 | Alice      | Johnson   |
|  2 | Bob        | Smith     |
+----+------------+-----------+
```

courses table:

```
+----+------------------+---------+
| id | course_name      | credits |
+----+------------------+---------+
| 10 | Mathematics 101  |       3 |
| 20 | History 202      |       4 |
| 30 | Computer Science |       3 |
+----+------------------+---------+
```

enrollments table:

```
+----+------------+-----------+-------+
| id | student_id | course_id | grade |
+----+------------+-----------+-------+
|  1 |          1 |        10 | A     |
|  2 |          1 |        20 | B+    |
|  3 |          2 |        10 | A-    |
|  4 |          2 |        30 | B     |
+----+------------+-----------+-------+
```

Reading the enrollments table:

- Row 1: Student 1 (Alice) is enrolled in Course 10 (Mathematics 101) and got an A.
- Row 2: Student 1 (Alice) is enrolled in Course 20 (History 202) and got a B+.
- Row 3: Student 2 (Bob) is enrolled in Course 10 (Mathematics 101) and got an A-.
- Row 4: Student 2 (Bob) is enrolled in Course 30 (Computer Science) and got a B.

Why Foreign Keys Matter

Foreign keys enforce referential integrity. That is a fancy way of saying:

> "You cannot reference something that does not exist."

For example, if you try to insert an enrollment for student_id = 99, but there is no student with id = 99, the database will reject it with an error. This prevents orphaned data (data that points to nothing).

```sql
-- This would FAIL because student_id 99 does not exist:
INSERT INTO enrollments (student_id, course_id, grade)
VALUES (99, 10, 'A');

-- ERROR: insert or update on table "enrollments" violates
-- foreign key constraint
```

Relationships: How Tables Connect

Tables in a relational database are connected through relationships. There are three types.

One-to-One (1:1)

Each row in Table A is linked to exactly one row in Table B, and vice versa.

Real-world example: Each person has exactly one passport, and each passport belongs to exactly one person.

```
+-------------------+         +-------------------+
|     persons       |         |    passports      |
+-------------------+         +-------------------+
| id (PK)           |----+    | id (PK)           |
| name              |    |    | person_id (FK)    |-----> persons.id
| date_of_birth     |    |    | passport_number   |       (UNIQUE)
+-------------------+    |    | expiry_date       |
                         |    +-------------------+
                         |
                         +--> One person = One passport
```

The key detail: `person_id` in the passports table has a UNIQUE constraint, meaning no two passports can have the same `person_id`. This enforces the one-to-one relationship.

```sql
CREATE TABLE persons (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100),
    date_of_birth DATE
```

```
);

CREATE TABLE passports (
    id SERIAL PRIMARY KEY,
    person_id INTEGER UNIQUE REFERENCES persons(id),
    passport_number VARCHAR(20),
    expiry_date DATE
);
```

Line-by-line for the passports table:

- `id SERIAL PRIMARY KEY` creates an auto-incrementing unique identifier.
- `person_id INTEGER UNIQUE REFERENCES persons(id)` creates a foreign key to the persons table. `UNIQUE` ensures each person can have only one passport. `REFERENCES persons(id)` means this column points to the `id` column in the `persons` table.
- `passport_number VARCHAR(20)` stores the passport number as text up to 20 characters.
- `expiry_date DATE` stores the expiration date.

One-to-Many (1:N)

Each row in Table A can be linked to many rows in Table B, but each row in Table B is linked to only one row in Table A.

This is the most common relationship type.

Real-world example: One teacher teaches many courses, but each course has only one teacher.

```
+------------------+         +------------------+
|    teachers      |         |     courses      |
+------------------+         +------------------+
| id (PK)          |----+    | id (PK)          |
| name             |    |    | course_name      |
| department       |    +---<| teacher_id (FK)  |
+------------------+         | credits          |
                              +------------------+

One teacher -----> Many courses
```

The arrow `---<` means "one to many." One teacher points to many courses.

```
teachers table:
+----+------------------+------------+
| id | name             | department |
+----+------------------+------------+
|  1 | Prof. Smith      | Math       |
```

```
|  2 | Prof. Garcia     | History    |
+----+------------------+------------+

courses table:
+----+------------------+------------+---------+
| id | course_name      | teacher_id | credits |
+----+------------------+------------+---------+
| 10 | Algebra          |          1 |       3 |
| 20 | Calculus         |          1 |       4 |
| 30 | World History    |          2 |       3 |
+----+------------------+------------+---------+
```

Reading this data:

- Prof. Smith (id=1) teaches Algebra and Calculus (two courses).
- Prof. Garcia (id=2) teaches World History (one course).
- Each course has exactly one teacher.

Many-to-Many (M:N)

Each row in Table A can be linked to many rows in Table B, and each row in Table B can also be linked to many rows in Table A.

Real-world example: Students and courses. One student can enroll in many courses, and one course can have many students.

```
+------------------+     +------------------+     +------------------+
|    students      |     |   enrollments    |     |     courses      |
+------------------+     +------------------+     +------------------+
| id (PK)          |--+  | id (PK)          |  +--| id (PK)          |
| first_name       |  +->| student_id (FK)  |  |  | course_name      |
| last_name        |     | course_id (FK)   |<-+  | credits          |
+------------------+     | grade            |     +------------------+
                          +------------------+

                          This middle table is
                          called a "junction table"
                          or "bridge table"
```

Important: You cannot directly represent a many-to-many relationship in a relational database. You always need a junction table (also called a bridge table, linking table, or join table) in the middle.

The junction table has foreign keys pointing to both of the related tables. Each row in the junction table represents one specific link between the two sides.

```sql
CREATE TABLE students (
    id SERIAL PRIMARY KEY,
    first_name VARCHAR(50),
    last_name VARCHAR(50)
);

CREATE TABLE courses (
    id SERIAL PRIMARY KEY,
    course_name VARCHAR(100),
    credits INTEGER
);

CREATE TABLE enrollments (
    id SERIAL PRIMARY KEY,
    student_id INTEGER REFERENCES students(id),
    course_id INTEGER REFERENCES courses(id),
    grade VARCHAR(2)
);
```

Line-by-line for the enrollments table:

- `id SERIAL PRIMARY KEY` gives each enrollment its own unique ID.
- `student_id INTEGER REFERENCES students(id)` is a foreign key linking to the students table.
- `course_id INTEGER REFERENCES courses(id)` is a foreign key linking to the courses table.
- `grade VARCHAR(2)` stores the grade as text (like "A", "B+", "C").

Relationship Summary

```
+------------------+---------------------+----------------------------+
| Type             | Example             | How It Works               |
+------------------+---------------------+----------------------------+
| One-to-One (1:1) | Person - Passport   | FK with UNIQUE constraint  |
| One-to-Many (1:N)| Teacher - Courses   | FK in the "many" table     |
| Many-to-Many     | Students - Courses  | Junction table with two    |
|  (M:N)           |                     |   foreign keys             |
+------------------+---------------------+----------------------------+
```

What Is a Schema?

A schema is the overall structure or design of your database. It defines:

- What tables exist

- What columns each table has
- What data type each column uses
- What relationships exist between tables
- What rules (constraints) apply

Think of it as: The blueprint of a house. The blueprint tells you how many rooms there are, how big each room is, and where the doors connect them. The schema is the blueprint of your database.

Here is a schema for a simple school database:

```
+==================================================================+
|                     SCHOOL DATABASE SCHEMA                       |
+==================================================================+
|                                                                  |
|  +------------------+                                            |
|  |    students      |                                            |
|  +------------------+                                            |
|  | id        SERIAL | (PK)                                      |
|  | first_name VARCHAR(50)                                        |
|  | last_name  VARCHAR(50)                                        |
|  | email      VARCHAR(100)                                       |
|  | age        INTEGER                                            |
|  +------------------+                                            |
|           |                                                      |
|           | (one-to-many: one student, many enrollments)          |
|           v                                                      |
|  +------------------+                                            |
|  |   enrollments    |                                            |
|  +------------------+                                            |
|  | id         SERIAL | (PK)                                     |
|  | student_id INTEGER | (FK -> students.id)                     |
|  | course_id  INTEGER | (FK -> courses.id)                      |
|  | grade      VARCHAR(2)                                         |
|  | enrolled_date DATE                                            |
|  +------------------+                                            |
|           ^                                                      |
|           | (one-to-many: one course, many enrollments)           |
|           |                                                      |
|  +------------------+                                            |
|  |    courses       |                                            |
|  +------------------+                                            |
|  | id          SERIAL | (PK)                                    |
|  | course_name VARCHAR(100)                                      |
|  | credits     INTEGER                                           |
|  | department  VARCHAR(50)                                       |
|  +------------------+                                            |
|                                                                  |
+==================================================================+
```

You do not need to memorize schema diagrams. You need to be able to read them and eventually design your own. We will practice this throughout the book.

Complete Example: A School Database

Let us walk through a complete example step by step. We will build a small school database with students, courses, and enrollments.

Step 1: Identify the Entities

An entity is a thing you want to store data about. In a school:

- Students
- Courses
- Teachers

Step 2: Identify the Attributes

Attributes are the details about each entity:

```
Student: first_name, last_name, email, age, enrollment_date
Course: course_name, credits, department
Teacher: name, email, department
```

Step 3: Identify the Relationships

- A student can enroll in many courses. A course can have many students. This is a many-to-many relationship. We need a junction table: `enrollments`.
- A teacher can teach many courses, but each course has one teacher. This is a one-to-many relationship.

Step 4: Design the Tables

```sql
-- Teachers table
CREATE TABLE teachers (
    id SERIAL PRIMARY KEY,
```

```
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    department VARCHAR(50)
);

-- Courses table (has a FK to teachers)
CREATE TABLE courses (
    id SERIAL PRIMARY KEY,
    course_name VARCHAR(100) NOT NULL,
    credits INTEGER NOT NULL,
    department VARCHAR(50),
    teacher_id INTEGER REFERENCES teachers(id)
);

-- Students table
CREATE TABLE students (
    id SERIAL PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    age INTEGER,
    enrollment_date DATE DEFAULT CURRENT_DATE
);

-- Enrollments junction table (many-to-many between students and courses)
CREATE TABLE enrollments (
    id SERIAL PRIMARY KEY,
    student_id INTEGER REFERENCES students(id),
    course_id INTEGER REFERENCES courses(id),
    grade VARCHAR(2),
    enrolled_date DATE DEFAULT CURRENT_DATE
);
```

Step 5: Add Some Data

```sql
-- Insert teachers
INSERT INTO teachers (name, email, department)
VALUES
    ('Prof. Smith', 'smith@school.edu', 'Mathematics'),
    ('Prof. Garcia', 'garcia@school.edu', 'History'),
    ('Prof. Lee', 'lee@school.edu', 'Computer Science');

-- Insert courses
INSERT INTO courses (course_name, credits, department, teacher_id)
VALUES
    ('Algebra', 3, 'Mathematics', 1),
    ('Calculus', 4, 'Mathematics', 1),
    ('World History', 3, 'History', 2),
    ('Intro to Programming', 3, 'Computer Science', 3);
```

```
-- Insert students
INSERT INTO students (first_name, last_name, email, age)
VALUES
    ('Alice', 'Johnson', 'alice@email.com', 20),
    ('Bob', 'Smith', 'bob@email.com', 22),
    ('Charlie', 'Brown', 'charlie@email.com', 19),
    ('Diana', 'Prince', 'diana@email.com', 21);

-- Insert enrollments
INSERT INTO enrollments (student_id, course_id, grade)
VALUES
    (1, 1, 'A'),     -- Alice in Algebra
    (1, 3, 'B+'),    -- Alice in World History
    (2, 1, 'A-'),    -- Bob in Algebra
    (2, 4, 'B'),     -- Bob in Intro to Programming
    (3, 2, 'B+'),    -- Charlie in Calculus
    (3, 3, 'A'),     -- Charlie in World History
    (4, 1, 'A'),     -- Diana in Algebra
    (4, 4, 'A-');    -- Diana in Intro to Programming
```

Step 6: Query the Data

Now we can ask the database questions using SQL. Here is a query that shows each student's name and the courses they are enrolled in:

```sql
SELECT
    s.first_name,
    s.last_name,
    c.course_name,
    e.grade
FROM enrollments e
JOIN students s ON e.student_id = s.id
JOIN courses c ON e.course_id = c.id
ORDER BY s.last_name, c.course_name;
```

```
 first_name | last_name | course_name          | grade
------------+-----------+----------------------+-------
 Charlie    | Brown     | Calculus             | B+
 Charlie    | Brown     | World History        | A
 Alice      | Johnson   | Algebra              | A
 Alice      | Johnson   | World History        | B+
 Diana      | Prince    | Algebra              | A
 Diana      | Prince    | Intro to Programming | A-
 Bob        | Smith     | Algebra              | A-
 Bob        | Smith     | Intro to Programming | B
(8 rows)
```

Do not worry if the JOIN syntax looks unfamiliar. We will cover it in detail in Chapters 12 through 15. The point here is to see how tables connect and how you can combine data from multiple tables.

Reading ASCII Table Diagrams

Throughout this book and in many online resources, you will see ASCII diagrams that show table structures and relationships. Here is how to read them:

Table Structure Diagram

```
+-------------------+
|    table_name     |     <-- Table name at the top
+-------------------+
| column1 (PK)      |     <-- PK means Primary Key
| column2 (FK)      |     <-- FK means Foreign Key
| column3           |     <-- Regular column
| column4           |     <-- Regular column
+-------------------+
```

Relationship Diagram

```
+----------+         +----------+
| Table A  |----+    | Table B  |
+----------+    |    +----------+
| id (PK)  |    +--->| a_id(FK) |
| name     |         | data     |
+----------+         +----------+

The arrow shows the direction of the foreign key.
Table B has a foreign key (a_id) that points to Table A's id.
```

Common Symbols

```
+----------+----------+-----------------------------------+
| Symbol   | Meaning  | Example                           |
+----------+----------+-----------------------------------+
| PK       | Primary  | id (PK) - unique identifier       |
|          |   Key    |                                   |
```

```
| FK        | Foreign   | student_id (FK) - reference to    |
|           |   Key     |   another table                   |
| --->      | Points    | Table B points to Table A         |
|           |   to      |                                   |
| ---<      | One to    | One teacher, many courses          |
|           |   many    |                                   |
| >---<     | Many to   | Many students, many courses        |
|           |   many    |   (needs junction table)          |
+-----------+-----------+-----------------------------------+
```

Common Mistakes

1. Forgetting to add a primary key. Every table needs a way to uniquely identify each row. Always add a primary key.
2. Using meaningful data as a primary key. Do not use email, phone number, or name as a primary key. These can change. Use a numeric ID or UUID.
3. Not understanding many-to-many relationships. You cannot directly link two tables in a many-to-many relationship. You always need a junction table in the middle.
4. Putting all data in one big table. Beginners often try to cram everything into a single table. This leads to duplicated data and maintenance nightmares. Use separate tables and connect them with foreign keys.
5. Confusing primary keys and foreign keys. A primary key identifies a row in its own table. A foreign key references a row in another table. They work together, but they are different things.

Best Practices

1. Always use a SERIAL or BIGSERIAL integer as your primary key. It is simple, fast, and reliable. Name it `id`.
2. Name your tables in lowercase with underscores. Use `student_courses`, not `StudentCourses` or `STUDENT-COURSES`. This is the PostgreSQL convention.
3. Name foreign key columns clearly. Use `student_id` for a foreign key that references the `students` table. The pattern is `referenced_table_singular_id`.
4. Draw your tables on paper or a whiteboard before writing SQL. Even a rough sketch helps you spot problems early.

5. Start simple. Design two or three tables first. Get them working. Then add more complexity.
6. Use plural names for tables. A table holds many students, so call it `students`, not `student`.

Quick Summary

The relational model organizes data into tables with rows and columns. Each table has a primary key that uniquely identifies every row. Foreign keys create links between tables, allowing you to represent relationships. There are three relationship types: one-to-one, one-to-many (the most common), and many-to-many (which requires a junction table). A schema is the complete blueprint of your database structure.

Key Points

- A table stores related data in rows and columns.
- A row (record) is one entry in a table.
- A column (field) is one attribute of the data.
- A primary key uniquely identifies each row. It must be unique and never NULL.
- A foreign key is a reference from one table to another table's primary key.
- One-to-one: Each row in Table A links to exactly one row in Table B.
- One-to-many: One row in Table A links to many rows in Table B.
- Many-to-many: Rows in Table A and Table B link to each other through a junction table.
- A schema is the blueprint that describes all tables, columns, types, and relationships.

Practice Questions

1. What is the difference between a row and a column? Give a real-world example of each.
2. Why should you not use a person's name as a primary key? Give at least two reasons.
3. A library has books and authors. One book can have many authors, and one author can write many books. What type of relationship is this? What tables would you need?

4. Look at the enrollments table in our school example. What would happen if you tried to insert a row with `student_id = 99` when no student with id 99 exists?
5. In a one-to-many relationship between teachers and courses, which table contains the foreign key? Why?

Exercises

Exercise 1: Design a Database for a Bookstore

A bookstore needs to track:

- Books (title, price, publication year, number of pages)
- Authors (name, country, birth year)
- Customers (name, email, phone)
- Orders (which customer bought which books, when, for how much)

Draw the tables with their columns. Identify the primary keys, foreign keys, and relationships. Use ASCII diagrams.

Exercise 2: Identify Relationships

For each pair below, identify the relationship type (one-to-one, one-to-many, or many-to-many):

1. Country and Capital City
2. Doctor and Patients
3. Movie and Actors
4. Car and License Plate
5. Blog Post and Comments

Exercise 3: Read the Schema

Given this schema, answer the questions below:

```
+-------------------+     +-------------------+     +-------------------+
|    employees      |     | project_members   |     |    projects       |
+-------------------+     +-------------------+     +-------------------+
| id (PK)           |---->| employee_id (FK)  |     | id (PK)           |
| name              |     | project_id (FK)   |<----| project_name      |
```

```
| email            |     | role             |     | start_date       |
| department       |     +------------------+     | budget           |
+------------------+                               +------------------+
```

1. What type of relationship exists between employees and projects?
2. What is the junction table?
3. Can an employee work on multiple projects? Can a project have multiple employees?
4. What extra information does the junction table store besides the foreign keys?

What Is Next?

You now understand the theory behind relational databases. In Chapter 3, you will get your hands dirty by installing PostgreSQL on your computer. You will set up the database software, connect to it, and run your first commands. Theory is important, but nothing beats actually doing it.

Chapter 3: Installing PostgreSQL

What You Will Learn

By the end of this chapter, you will have:

- Understood why PostgreSQL is the best choice for learning SQL
- Installed PostgreSQL on your computer (Mac, Windows, or Linux)
- Set up pgAdmin 4, a graphical tool for managing databases
- Connected to PostgreSQL using the psql command line
- Created your first database
- Learned essential psql commands: `\l`, `\dt`, `\d`, and more
- Troubleshot common installation errors

Why This Chapter Matters

You have learned the theory. Now it is time to install the tools and get your hands dirty. You cannot learn SQL by just reading about it. You need to type commands, see results, make mistakes, and fix them. This chapter sets up your workspace so you can follow along with every example in the rest of the book.

Why PostgreSQL?

Before we install anything, let us briefly revisit why we are using PostgreSQL.

```
+----------------------------+-----------------------------------+
| Reason                     | Details                           |
+----------------------------+-----------------------------------+
```

```
| Free and open source        | No license fees, ever             |
| Standards-compliant         | Follows the SQL standard closely  |
| Feature-rich                | JSON, arrays, full-text search,   |
|                             |   window functions, CTEs, more    |
| Reliable                    | Used by Apple, Instagram, Spotify |
| Great community             | Excellent documentation and help  |
| Good for learning           | What you learn transfers to other |
|                             |   databases                       |
| Runs everywhere             | Mac, Windows, Linux, cloud        |
+-----------------------------+-----------------------------------+
```

PostgreSQL has been in active development since 1996. It is battle-tested, well-documented, and trusted by organizations of all sizes.

Installing PostgreSQL on Mac

There are several ways to install PostgreSQL on a Mac. We will use Homebrew, which is the most popular package manager for macOS.

Step 1: Install Homebrew (if you do not have it)

Open the Terminal app (you can find it in Applications > Utilities > Terminal, or search for "Terminal" using Spotlight with Command + Space).

Check if Homebrew is already installed:

```bash
brew --version
```

If you see a version number (like `Homebrew 4.x.x`), you are good. Skip to Step 2.

If you see "command not found," install Homebrew by paste this command into your terminal:

```bash
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/inst...
```

Follow the on-screen instructions. You may need to enter your Mac password.

Step 2: Install PostgreSQL

Run this command in your terminal:

```bash
brew install postgresql@16
```

This installs PostgreSQL version 16 (the latest stable version as of this writing). The download and installation may take a few minutes.

Step 3: Start the PostgreSQL Service

After installation, start the PostgreSQL service so it runs in the background:

```bash
brew services start postgresql@16
```

You should see output like:

```
==> Successfully started `postgresql@16`
```

To verify it is running:

```bash
brew services list
```

You should see `postgresql@16` with a status of `started`.

Step 4: Add PostgreSQL to Your PATH

You may need to add PostgreSQL to your PATH so your terminal can find the `psql` command. Run:

```bash
echo 'export PATH="/opt/homebrew/opt/postgresql@16/bin:$PATH"' >> ~/.zshrc
source ~/.zshrc
```

> Note: If you use bash instead of zsh, replace `~/.zshrc` with `~/.bash_profile`.

Step 5: Verify the Installation

```bash
psql --version
```

You should see something like:

```
psql (PostgreSQL) 16.x
```

Step 6: Connect to PostgreSQL

```bash
psql postgres
```

You should see the PostgreSQL prompt:

```
postgres=#
```

Congratulations! You are connected to PostgreSQL. Type `\q` to quit for now. We will come back shortly.

Installing PostgreSQL on Windows

Step 1: Download the Installer

1. Go to the official PostgreSQL download page: https://www.postgresql.org/download/windows/
2. Click on "Download the installer" (this takes you to the EnterpriseDB page).
3. Download the latest version for Windows (PostgreSQL 16.x or newer).

Step 2: Run the Installer

1. Double-click the downloaded `.exe` file.
2. Click Next through the welcome screen.
3. Installation Directory: Keep the default (usually `C:\Program Files\PostgreSQL\16`). Click Next.
4. Select Components: Make sure all boxes are checked:
    - PostgreSQL Server
    - pgAdmin 4
    - Stack Builder
    - Command Line Tools
5. Data Directory: Keep the default. Click Next.

6. Password: Enter a password for the `postgres` superuser. Write this password down! You will need it. Click Next.
7. Port: Keep the default port 5432. Click Next.
8. Locale: Keep the default. Click Next.
9. Click Next and then Finish to complete the installation.

```
+-----------------------------------------------------------------+
|                    IMPORTANT: REMEMBER YOUR PASSWORD            |
|                                                                 |
|  The password you set during installation is for the "postgres"  |
|  superuser account. You will need it every time you connect.    |
|                                                                 |
|  If you forget it, you will need to reset it, which is a pain.  |
|  Write it somewhere safe!                                       |
+-----------------------------------------------------------------+
```

Step 3: Verify the Installation

1. Open the Start Menu.
2. Search for "SQL Shell (psql)" and open it.
3. Press Enter for each prompt to accept the defaults (Server, Database, Port, Username).
4. Enter the password you set during installation.

You should see:

```
postgres=#
```

Step 4: Add psql to Your PATH (Optional but Recommended)

To use `psql` from any command prompt or PowerShell window:

1. Open System Properties (search for "Environment Variables" in the Start Menu).
2. Click Environment Variables.
3. Under System variables, find `Path` and click Edit.
4. Click New and add: `C:\Program Files\PostgreSQL\16\bin`
5. Click OK on all windows.
6. Open a new Command Prompt or PowerShell and test:

```bash
psql --version
```

Installing PostgreSQL on Linux (Ubuntu/Debian)

Step 1: Update Your Package List

Open a terminal and run:

```bash
sudo apt update
```

Step 2: Install PostgreSQL

```bash
sudo apt install postgresql postgresql-contrib
```

The `postgresql-contrib` package includes useful additional tools and features.

Step 3: Start and Enable the Service

```bash
sudo systemctl start postgresql
sudo systemctl enable postgresql
```

The `enable` command makes sure PostgreSQL starts automatically when your computer boots.

Step 4: Verify the Installation

```bash
psql --version
```

You should see the version number.

Step 5: Connect to PostgreSQL

On Linux, PostgreSQL creates a system user called `postgres`. Switch to that user:

```bash
sudo -u postgres psql
```

You should see:

```
postgres=#
```

Step 6: Set a Password for the postgres User

While connected to psql, set a password:

```sql
ALTER USER postgres PASSWORD 'your_secure_password';
```

Replace `your_secure_password` with an actual password. Remember this password.

Type `\q` to exit.

Setting Up pgAdmin 4

pgAdmin 4 is a free graphical tool for managing PostgreSQL databases. It gives you a visual interface instead of typing commands in a terminal. It is useful for browsing tables, running queries, and visualizing your data.

Installing pgAdmin 4

On Mac:

```bash
brew install --cask pgadmin4
```

Or download from https://www.pgadmin.org/download/pgadmin-4-macos/

On Windows:

If you installed PostgreSQL using the EnterpriseDB installer, pgAdmin 4 was included. Find it in your Start Menu.

If not, download from https://www.pgadmin.org/download/pgadmin-4-windows/

On Linux:

```bash
sudo apt install pgadmin4-desktop
```

Or follow the official instructions at https://www.pgadmin.org/download/pgadmin-4-apt/

First-Time Setup

1. Open pgAdmin 4.
2. It will ask you to set a master password. This is for pgAdmin itself, not PostgreSQL. Choose something you will remember.
3. In the left sidebar, click Servers.
4. Right-click Servers and select Register > Server.
5. In the General tab, give it a name like "Local PostgreSQL."
6. In the Connection tab:
    - Host: `localhost`
    - Port: `5432`
    - Maintenance database: `postgres`
    - Username: `postgres`
    - Password: the password you set during PostgreSQL installation
7. Click Save.

You should now see your PostgreSQL server in the sidebar. You can expand it to see databases, tables, and more.

```
+-- Servers
    +-- Local PostgreSQL
        +-- Databases
        |   +-- postgres (default database)
        +-- Login/Group Roles
        +-- Tablespaces
```

The psql Command Line

While pgAdmin 4 is nice for visual exploration, psql is the command-line tool that professionals use daily. It is fast, powerful, and available on every system where PostgreSQL is installed.

Connecting to PostgreSQL with psql

The basic connection command is:

```bash
psql -U postgres -d postgres
```

Flags explained:

- `-U postgres` means connect as the user "postgres."
- `-d postgres` means connect to the database named "postgres."

On Mac (with Homebrew), you can often just type:

```bash
psql postgres
```

On Linux:

```bash
sudo -u postgres psql
```

The psql Prompt

Once connected, you see a prompt like this:

```
postgres=#
```

- `postgres` is the name of the database you are connected to.
- `=#` means you are a superuser (administrator). Regular users see `=>`.

Creating Your First Database

Let us create a database called `school`. Connect to psql and run:

```sql
CREATE DATABASE school;
```

```
CREATE DATABASE
```

That one line created a new, empty database called `school`. Now connect to it:

```bash
\c school
```

```
You are now connected to database "school" as user "postgres".
school=#
```

Notice the prompt changed from `postgres=#` to `school=#`. This tells you which database you are working in.

Creating a Simple Table

Let us create a `students` table inside the `school` database:

```sql
CREATE TABLE students (
    id SERIAL PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    email VARCHAR(100) UNIQUE,
    age INTEGER
);
```

```
CREATE TABLE
```

Line-by-line:

- `CREATE TABLE students (` starts the table definition.
- `id SERIAL PRIMARY KEY` creates an auto-incrementing integer as the primary key.
- `first_name VARCHAR(50) NOT NULL` creates a text column that cannot be empty.
- `last_name VARCHAR(50) NOT NULL` same as above, for the last name.
- `email VARCHAR(100) UNIQUE` creates a text column where every value must be different.
- `age INTEGER` creates a whole number column.
- `)` ends the table definition.

Inserting Some Data

```sql
INSERT INTO students (first_name, last_name, email, age)
VALUES ('Alice', 'Johnson', 'alice@email.com', 20);

INSERT INTO students (first_name, last_name, email, age)
VALUES ('Bob', 'Smith', 'bob@email.com', 22);
```

```
INSERT 0 1
INSERT 0 1
```

Querying the Data

```sql
SELECT * FROM students;
```

```
 id | first_name | last_name |      email      | age
----+------------+-----------+-----------------+-----
  1 | Alice      | Johnson   | alice@email.com |  20
  2 | Bob        | Smith     | bob@email.com   |  22
(2 rows)
```

You just created a database, created a table, inserted data, and queried it. That is the full cycle.

Essential psql Commands

psql has many built-in commands that start with a backslash (`\`). These are not SQL. They are psql-specific shortcuts.

\l — List All Databases

```
\l
```

```
                                   List of databases
   Name    |  Owner   | Encoding |   Collate   |    Ctype    |   Access privileges
-----------+----------+----------+-------------+-------------+-----------------------
 postgres  | postgres | UTF8     | en_US.UTF-8 | en_US.UTF-8 |
 school    | postgres | UTF8     | en_US.UTF-8 | en_US.UTF-8 |
 template0 | postgres | UTF8     | en_US.UTF-8 | en_US.UTF-8 | =c/postgres          +
           |          |          |             |             | postgres=CTc/postgres
 template1 | postgres | UTF8     | en_US.UTF-8 | en_US.UTF-8 | =c/postgres          +
           |          |          |             |             | postgres=CTc/postgres
(4 rows)
```

You can see the `school` database we just created, along with the default databases (`postgres`, `template0`, `template1`).

\c — Connect to a Database

```
\c school
```

```
You are now connected to database "school" as user "postgres".
```

\dt — List All Tables in the Current Database

```
\dt
```

```
          List of relations
 Schema |   Name   | Type  |  Owner
--------+----------+-------+----------
 public | students | table | postgres
(1 row)
```

\d — Describe a Table's Structure

```
\d students
```

```
                                        Table "public.students"
   Column   |          Type          | Collation | Nullable |               Default
------------+------------------------+-----------+----------+-------------------------...
 id         | integer                |           | not null | nextval('students_id_seq'...
 first_name | character varying(50)  |           | not null |
 last_name  | character varying(50)  |           | not null |
 email      | character varying(100) |           |          |
 age        | integer                |           |          |
Indexes:
    "students_pkey" PRIMARY KEY, btree (id)
    "students_email_key" UNIQUE CONSTRAINT, btree (email)
```

This shows you every column, its data type, whether it can be NULL, and its default value. It also shows indexes and constraints.

Other Useful psql Commands

```
+-----------+-------------------------------------------+
| Command   | What It Does                              |
+-----------+-------------------------------------------+
| \l        | List all databases                        |
| \c dbname | Connect to a database                     |
| \dt       | List tables in current database            |
| \d table  | Describe a table (columns, types, etc.)   |
| \du       | List all users/roles                      |
| \di       | List all indexes                          |
| \dn       | List all schemas                          |
| \df       | List all functions                        |
| \timing   | Toggle query execution time display       |
| \x        | Toggle expanded output (vertical display) |
| \q        | Quit psql                                 |
| \?        | Show all psql commands                    |
| \h        | Show SQL command help                     |
| \h SELECT | Show help for the SELECT command           |
+-----------+-------------------------------------------+
```

Multiline Commands

In psql, a command does not run until you type a semicolon (`;`) and press Enter. This means you can split commands across multiple lines:

```
school=# SELECT
school-#   first_name,
school-#   last_name
school-# FROM students;
```

Notice the prompt changes from `=#` to `-#` when you are in the middle of a command. The `-#` tells you, "I am waiting for more input."

If you forget the semicolon:

```
school=# SELECT * FROM students
school-#
```

The database is waiting. Just type `;` and press Enter:

```
school-# ;
```

Common Errors and Fixes

Error: "psql: command not found"

Cause: PostgreSQL's `bin` directory is not in your PATH.

Fix (Mac):

```bash
echo 'export PATH="/opt/homebrew/opt/postgresql@16/bin:$PATH"' >> ~/.zshrc
source ~/.zshrc
```

Fix (Windows): Add `C:\Program Files\PostgreSQL\16\bin` to your system PATH (see the Windows installation section).

Fix (Linux): PostgreSQL is usually in the PATH by default. If not:

```bash
sudo apt install postgresql-client
```

Error: "connection refused" or "could not connect to server"

Cause: The PostgreSQL service is not running.

Fix (Mac):

```bash
brew services start postgresql@16
```

Fix (Windows): Open the Services app (search for "Services" in Start Menu), find "postgresql-x64-16", and click Start.

Fix (Linux):

```bash
sudo systemctl start postgresql
```

Error: "password authentication failed for user postgres"

Cause: Wrong password.

Fix: If you forgot your password, you need to reset it. The process varies by operating system.

On Mac (Homebrew): Homebrew installations often do not require a password. Try:

```bash
psql postgres
```

On Linux: Edit the pg_hba.conf file to temporarily allow passwordless access:

```bash
sudo -u postgres psql
ALTER USER postgres PASSWORD 'new_password';
\q
```

Error: "database does not exist"

Cause: You are trying to connect to a database that has not been created yet.

Fix: Connect to the default `postgres` database first, then create your database:

```bash
psql -U postgres -d postgres
```

```sql
CREATE DATABASE school;
```

Error: "role does not exist"

Cause: The username you are trying to connect with does not exist in PostgreSQL.

Fix: Create the role:

```bash
sudo -u postgres psql
```

```sql
CREATE ROLE your_username WITH LOGIN PASSWORD 'your_password';
```

Error: "permission denied"

Cause: Your user does not have the necessary privileges.

Fix: Grant privileges:

```sql
GRANT ALL PRIVILEGES ON DATABASE school TO your_username;
```

Verifying Everything Works

Let us run through a quick checklist to make sure your setup is complete.

```
+----+----------------------------------------+------------------+
| #  | Check                                  | Command          |
+----+----------------------------------------+------------------+
| 1  | PostgreSQL is installed                | psql --version   |
| 2  | PostgreSQL service is running          | (platform-specific)|
| 3  | You can connect to psql                | psql postgres    |
| 4  | You can list databases                 | \l               |
| 5  | You can create a database              | CREATE DATABASE  |
| 6  | You can connect to your database       | \c school        |
| 7  | You can create a table                 | CREATE TABLE     |
| 8  | You can insert data                    | INSERT INTO      |
| 9  | You can query data                     | SELECT * FROM    |
| 10 | pgAdmin 4 connects to your server      | (visual check)   |
+----+----------------------------------------+------------------+
```

If all ten checks pass, you are ready for the rest of this book.

Common Mistakes

1. Not starting the PostgreSQL service. Installing PostgreSQL does not automatically start it on every system. Make sure the service is running before trying to connect.
2. Forgetting the semicolon. Every SQL command in psql must end with a semicolon (`;`). If your command does not seem to run, you probably forgot the semicolon.
3. Confusing psql commands and SQL commands. Commands that start with a backslash (like `\l`, `\dt`) are psql shortcuts. They do not end with a semicolon. SQL commands (like `SELECT`, `CREATE TABLE`) must end with a semicolon.
4. Forgetting the postgres password. Write it down during installation. You will need it many times.
5. Trying to run psql before adding it to PATH. If `psql` is not found, you need to add the PostgreSQL bin directory to your system PATH.

Best Practices

1. Use psql for learning. It is the fastest way to experiment. Type a command, see the result immediately.
2. Use pgAdmin 4 for exploring. When you want to browse tables visually or see table structures at a glance, pgAdmin is helpful.
3. Keep your PostgreSQL service running. On Mac, use `brew services start postgresql@16` so it starts automatically. On Linux, use `systemctl enable postgresql`.
4. Create a separate database for each project. Do not dump everything into the default `postgres` database. Create a `school` database for school examples, a `bookstore` database for bookstore examples, and so on.
5. Use `\d table_name` frequently. It is the fastest way to remind yourself what columns a table has and what types they use.

Quick Summary

PostgreSQL is free, powerful, and the best RDBMS for learning SQL. You can install it on Mac (using Homebrew), Windows (using the EnterpriseDB installer), or Linux (using apt). pgAdmin 4 provides a graphical interface, while psql provides a fast command-line interface. Essential psql commands include `\l` (list databases), `\dt` (list tables), `\d` (describe a table), and `\c` (connect to a database). Every SQL command must end with a semicolon.

Key Points

- Install PostgreSQL using Homebrew (Mac), the EnterpriseDB installer (Windows), or apt (Linux).
- psql is the command-line client. pgAdmin 4 is the graphical client.
- The default superuser is called `postgres`.
- Use `\l` to list databases, `\dt` to list tables, `\d table_name` to describe a table.
- Use `\c database_name` to switch to a different database.
- Every SQL statement must end with a semicolon (`;`).
- psql backslash commands (like `\l`) do not need a semicolon.
- Keep the PostgreSQL service running while you work.

- Always remember your `postgres` user password.

Practice Questions

1. What is the difference between psql and pgAdmin 4? When would you use each?
2. You type `SELECT * FROM students` in psql and nothing happens. What did you probably forget?
3. What psql command shows you the columns, data types, and constraints of a table called `courses`?
4. What is the default port number for PostgreSQL?
5. You get an error "connection refused" when trying to run psql. What is the most likely cause and how do you fix it?

Exercises

Exercise 1: Install and Verify

Follow the installation instructions for your operating system. Verify your installation by:

1. Checking the PostgreSQL version with `psql --version`.
2. Connecting to the default database with `psql postgres`.
3. Running `\l` to list all databases.
4. Creating a test database called `my_first_db`.
5. Connecting to it with `\c my_first_db`.
6. Running `\dt` to confirm it has no tables yet.

Exercise 2: Create a Practice Database

1. Create a database called `practice`.
2. Connect to it.
3. Create a table called `movies` with these columns:
    - `id` (auto-incrementing primary key)
    - `title` (text, up to 200 characters, required)
    - `release_year` (integer)

    - `rating` (numeric with one decimal place)
4. Insert three movies of your choice.
5. Run `SELECT * FROM movies;` to see your data.
6. Run `\d movies` to see the table structure.

Exercise 3: Explore psql

Connect to your `practice` database and try each of these commands. Write down what each one shows you:

1. `\l`
2. `\dt`
3. `\d movies`
4. `\du`
5. `\timing`
6. `SELECT * FROM movies;` (after turning on timing)
7. `\x` then `SELECT * FROM movies;` (what changed?)

What Is Next?

Now that PostgreSQL is installed and running, you are ready to start building real databases. In Chapter 4, you will learn how to create databases and tables using SQL. You will master the `CREATE TABLE` command, learn about column constraints like NOT NULL and UNIQUE, and build complete, well-structured tables from scratch.

Chapter 4: Creating Databases and Tables

What You Will Learn

By the end of this chapter, you will know how to:

- Create and drop databases with `CREATE DATABASE` and `DROP DATABASE`
- Create tables with `CREATE TABLE` and define columns
- Use column constraints: NOT NULL, UNIQUE, DEFAULT, and CHECK
- Set primary keys
- Get a preview of data types (covered fully in Chapter 5)
- Drop tables with `DROP TABLE`
- Modify existing tables with `ALTER TABLE` (add, drop, rename columns)
- Use `IF NOT EXISTS` and `IF EXISTS` to avoid errors
- Build a complete students table from scratch

Why This Chapter Matters

Creating databases and tables is the first real thing you do as a database developer. Everything else (inserting data, querying, joining tables) depends on having well-designed tables. If your tables are poorly designed, you will struggle with every operation that follows. This chapter teaches you how to lay a solid foundation.

CREATE DATABASE

A database is a container that holds all your tables, data, and other objects. Before you can create tables, you need a database.

Basic Syntax

```sql
CREATE DATABASE database_name;
```

Example

```sql
CREATE DATABASE school;
```

```
CREATE DATABASE
```

That is it. One line, and you have a new database.

Naming Rules for Databases

```
+-----------------------------------+-----------------------------------+
| Rule                              | Example                           |
+-----------------------------------+-----------------------------------+
| Use lowercase letters             | school (not School or SCHOOL)     |
| Use underscores for spaces        | online_store (not online store)   |
| Start with a letter               | my_db (not 1_db)                 |
| Keep it short and descriptive     | school, inventory, blog            |
| Avoid special characters          | No @, #, -, or spaces             |
+-----------------------------------+-----------------------------------+
```

Connecting to Your Database

After creating a database, you need to connect to it before you can create tables inside it:

```
\c school
```

```
You are now connected to database "school" as user "postgres".
school=#
```

DROP DATABASE

To delete a database and all its data permanently:

```sql
DROP DATABASE database_name;
```

Example

```sql
DROP DATABASE school;
```

```
DROP DATABASE
```

> Warning: This is permanent and irreversible. All tables and all data inside the database are gone forever. There is no undo. There is no recycle bin.

Using IF EXISTS

If you try to drop a database that does not exist, PostgreSQL throws an error:

```sql
DROP DATABASE school;
-- ERROR: database "school" does not exist
```

To avoid this error, use `IF EXISTS`:

```sql
DROP DATABASE IF EXISTS school;
```

```
DROP DATABASE
```

If the database exists, it gets dropped. If it does not exist, PostgreSQL does nothing and moves on without an error.

> Important: You cannot drop a database while you are connected to it. Connect to a different database (like `postgres`) first, then drop the one you want to remove.

```sql
\c postgres
DROP DATABASE school;
```

CREATE TABLE

This is where the real work begins. The `CREATE TABLE` command defines a new table with its columns, data types, and rules.

Basic Syntax

```sql
CREATE TABLE table_name (
    column1_name data_type constraints,
    column2_name data_type constraints,
    column3_name data_type constraints
);
```

A Simple Example

```sql
CREATE TABLE students (
    id SERIAL PRIMARY KEY,
    first_name VARCHAR(50),
    last_name VARCHAR(50),
    age INTEGER
);
```

```
CREATE TABLE
```

Let us break this down piece by piece:

```
CREATE TABLE students (          -- Start defining a table called "students"
    id SERIAL PRIMARY KEY,       -- Column: id
                                 --   SERIAL = auto-incrementing integer (1, 2, 3, ...)
                                 --   PRIMARY KEY = unique identifier for each row
    first_name VARCHAR(50),      -- Column: first_name
                                 --   VARCHAR(50) = text up to 50 characters
    last_name VARCHAR(50),       -- Column: last_name
                                 --   VARCHAR(50) = text up to 50 characters
    age INTEGER                  -- Column: age
                                 --   INTEGER = whole number
);                               -- End of table definition
```

Verifying the Table

After creating the table, verify it exists:

```
\dt
```

```
          List of relations
 Schema |   Name   | Type  |  Owner
--------+----------+-------+----------
 public | students | table | postgres
(1 row)
```

See the table structure:

```
\d students
```

```
                                      Table "public.students"
   Column   |          Type          | Collation | Nullable |                Default
------------+------------------------+-----------+----------+-------------------------...
 id         | integer                |           | not null | nextval('students_id_seq'...
 first_name | character varying(50)  |           |          |
 last_name  | character varying(50)  |           |          |
 age        | integer                |           |          |
Indexes:
    "students_pkey" PRIMARY KEY, btree (id)
```

Data Types Preview

Every column needs a data type that tells PostgreSQL what kind of data it can hold. Here is a quick preview of the most common types. We will cover all data types in detail in Chapter 5.

```
+------------------+----------------------------+---------------------+
| Data Type        | What It Stores             | Example             |
+------------------+----------------------------+---------------------+
| INTEGER          | Whole numbers              | 42, -7, 0           |
| SERIAL           | Auto-incrementing integer  | 1, 2, 3, 4, ...     |
| VARCHAR(n)       | Text up to n characters    | 'Alice', 'Hello'    |
| TEXT             | Text of any length         | Long descriptions    |
| BOOLEAN          | True or false              | TRUE, FALSE          |
| DATE             | Calendar date              | '2024-01-15'         |
```

```
| TIMESTAMP        | Date and time               | '2024-01-15 14:30:00'|
| NUMERIC(p,s)     | Exact decimal number        | 19.99, 1000.50       |
+------------------+-----------------------------+---------------------+
```

Think of data types as labels on containers:

- An INTEGER column is like a container labeled "whole numbers only."
- A VARCHAR(50) column is like a container labeled "text, max 50 characters."
- A BOOLEAN column is like a container labeled "yes or no only."

If you try to put the wrong type of data into a column, PostgreSQL rejects it:

```sql
-- This will FAIL because 'twenty' is text, not a number:
INSERT INTO students (first_name, last_name, age)
VALUES ('Alice', 'Johnson', 'twenty');

-- ERROR: invalid input syntax for type integer: "twenty"
```

Column Constraints

Constraints are rules you put on columns to control what data is allowed. They protect your data quality.

Think of constraints as bouncers at a nightclub. They check every piece of data that tries to enter and reject anything that does not meet the rules.

NOT NULL

The `NOT NULL` constraint means the column cannot be empty. Every row must have a value for this column.

```sql
CREATE TABLE students (
    id SERIAL PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    email VARCHAR(100),
    age INTEGER
);
```

Here, `first_name` and `last_name` are required. `email` and `age` are optional (they can be NULL).

What happens if you try to insert without a required field:

```sql
INSERT INTO students (last_name, age)
VALUES ('Johnson', 20);
```

```
ERROR: null value in column "first_name" of relation "students"
violates not-null constraint
```

PostgreSQL stops you from inserting incomplete data. This is a good thing.

UNIQUE

The `UNIQUE` constraint means no two rows can have the same value in this column.

```sql
CREATE TABLE students (
    id SERIAL PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    email VARCHAR(100) UNIQUE,
    age INTEGER
);
```

Now each student must have a different email address:

```sql
INSERT INTO students (first_name, last_name, email, age)
VALUES ('Alice', 'Johnson', 'alice@email.com', 20);
-- Works fine

INSERT INTO students (first_name, last_name, email, age)
VALUES ('Bob', 'Smith', 'alice@email.com', 22);
-- ERROR: duplicate key value violates unique constraint "students_email_key"
```

> Note: NULL values are treated as unique. Two rows can both have NULL in a UNIQUE column. This is because NULL means "unknown," and two unknowns are not considered equal.

DEFAULT

The `DEFAULT` constraint provides a value automatically when you do not specify one.

```sql
CREATE TABLE students (
    id SERIAL PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    email VARCHAR(100) UNIQUE,
    age INTEGER,
    enrollment_date DATE DEFAULT CURRENT_DATE,
    is_active BOOLEAN DEFAULT TRUE
);
```

Now when you insert a student without specifying `enrollment_date` or `is_active`:

```sql
INSERT INTO students (first_name, last_name, email, age)
VALUES ('Alice', 'Johnson', 'alice@email.com', 20);
```

```sql
SELECT * FROM students;
```

```
 id | first_name | last_name |      email      | age | enrollment_date | is_active
----+------------+-----------+-----------------+-----+-----------------+-----------
  1 | Alice      | Johnson   | alice@email.com |  20 | 2024-06-15      | t
(1 row)
```

The `enrollment_date` was automatically set to today's date, and `is_active` was automatically set to TRUE (shown as `t` in psql).

CHECK

The `CHECK` constraint lets you define a custom rule. The data must pass this check to be inserted.

```sql
CREATE TABLE students (
    id SERIAL PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    email VARCHAR(100) UNIQUE,
    age INTEGER CHECK (age >= 0 AND age <= 150),
    gpa NUMERIC(3,2) CHECK (gpa >= 0.00 AND gpa <= 4.00)
);
```

The `age` column only accepts values between 0 and 150. The `gpa` column only accepts values between 0.00 and 4.00.

```sql
INSERT INTO students (first_name, last_name, age, gpa)
VALUES ('Alice', 'Johnson', -5, 3.80);
-- ERROR: new row for relation "students" violates check constraint
-- "students_age_check"

INSERT INTO students (first_name, last_name, age, gpa)
VALUES ('Alice', 'Johnson', 20, 5.00);
-- ERROR: new row for relation "students" violates check constraint
-- "students_gpa_check"
```

Combining Constraints

You can use multiple constraints on the same column:

```sql
CREATE TABLE products (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL UNIQUE,
    price NUMERIC(10,2) NOT NULL CHECK (price > 0),
    stock INTEGER NOT NULL DEFAULT 0 CHECK (stock >= 0)
);
```

This table enforces:

- Every product must have a name (`NOT NULL`), and no two products can have the same name (`UNIQUE`).
- Every product must have a price (`NOT NULL`), and the price must be positive (`CHECK (price > 0)`).
- Stock defaults to 0 (`DEFAULT 0`), must be provided or defaulted (`NOT NULL`), and cannot be negative (`CHECK (stock >= 0)`).

Constraints Summary

```
+------------+---------------------------------------+----------------------+
| Constraint | Rule                                  | Example              |
+------------+---------------------------------------+----------------------+
| NOT NULL   | Column cannot be empty                | first_name NOT NULL  |
| UNIQUE     | No duplicate values allowed           | email UNIQUE         |
| DEFAULT    | Automatic value if none provided      | status DEFAULT 'new' |
| CHECK      | Custom validation rule                | age CHECK(age >= 0)  |
| PRIMARY KEY| Unique + NOT NULL (identifies the row)| id PRIMARY KEY       |
+------------+---------------------------------------+----------------------+
```

PRIMARY KEY

We covered primary keys in Chapter 2, but let us see the SQL syntax.

Method 1: Inline (Column Level)

```sql
CREATE TABLE students (
    id SERIAL PRIMARY KEY,
    first_name VARCHAR(50)
);
```

Method 2: Table Level

```sql
CREATE TABLE students (
    id SERIAL,
    first_name VARCHAR(50),
    PRIMARY KEY (id)
);
```

Both methods do the same thing. Method 1 is more common for single-column primary keys. Method 2 is useful when you need a composite primary key (a primary key made of multiple columns):

```sql
CREATE TABLE enrollments (
    student_id INTEGER REFERENCES students(id),
    course_id INTEGER REFERENCES courses(id),
    grade VARCHAR(2),
    PRIMARY KEY (student_id, course_id)
);
```

Here, the combination of `student_id` and `course_id` is the primary key. This means a student can enroll in a course only once (no duplicate enrollments).

DROP TABLE

To delete a table and all its data:

```sql
DROP TABLE students;
```

```
DROP TABLE
```

> Warning: Like `DROP DATABASE`, this is permanent. All data in the table is lost.

Using IF EXISTS

```sql
DROP TABLE IF EXISTS students;
```

This drops the table if it exists. If it does not exist, PostgreSQL does nothing instead of throwing an error.

Dropping Multiple Tables

You can drop several tables in one command:

```sql
DROP TABLE IF EXISTS enrollments, students, courses;
```

> Order matters! If `enrollments` has foreign keys pointing to `students` and `courses`, you must drop `enrollments` first. Or use `CASCADE`:

```sql
DROP TABLE IF EXISTS students CASCADE;
```

`CASCADE` automatically drops anything that depends on the table (like foreign key constraints in other tables). Use this carefully.

ALTER TABLE

After creating a table, you may need to change it. The `ALTER TABLE` command lets you modify an existing table without dropping and recreating it.

Adding a Column

```sql
ALTER TABLE students ADD COLUMN phone VARCHAR(20);
```

```
ALTER TABLE
```

Let us verify:

```
\d students
```

The new `phone` column appears at the end of the column list.

Adding a Column with Constraints

```sql
ALTER TABLE students ADD COLUMN is_active BOOLEAN DEFAULT TRUE NOT NULL;
```

This adds a column called `is_active` that defaults to TRUE and cannot be NULL.

Dropping a Column

```sql
ALTER TABLE students DROP COLUMN phone;
```

```
ALTER TABLE
```

> Warning: Dropping a column deletes all data in that column for every row. This cannot be undone.

Renaming a Column

```sql
ALTER TABLE students RENAME COLUMN first_name TO given_name;
```

```
ALTER TABLE
```

This changes the column name from `first_name` to `given_name`. The data stays the same.

Renaming a Table

```sql
ALTER TABLE students RENAME TO pupils;
```

```
ALTER TABLE
```

Now the table is called `pupils` instead of `students`.

Changing a Column's Data Type

```sql
ALTER TABLE students ALTER COLUMN age TYPE SMALLINT;
```

This changes the `age` column from INTEGER to SMALLINT (a smaller number type). PostgreSQL will try to convert existing data to the new type.

Adding a Constraint to an Existing Column

```sql
ALTER TABLE students ALTER COLUMN email SET NOT NULL;
```

This makes the `email` column required (NOT NULL) after the table has already been created.

```sql
ALTER TABLE students ADD CONSTRAINT email_unique UNIQUE (email);
```

This adds a UNIQUE constraint to the `email` column.

Dropping a Constraint

```sql
ALTER TABLE students DROP CONSTRAINT email_unique;
```

ALTER TABLE Summary

```
+-------------------------------+-----------------------------------------+
| Command                       | What It Does                            |
+-------------------------------+-----------------------------------------+
| ADD COLUMN name TYPE          | Add a new column                        |
| DROP COLUMN name              | Remove a column and its data            |
| RENAME COLUMN old TO new      | Change a column's name                  |
| RENAME TO new_table_name      | Change the table's name                 |
```

```
| ALTER COLUMN name TYPE new    | Change a column's data type              |
| ALTER COLUMN name SET NOT NULL| Make a column required                   |
| ALTER COLUMN name DROP NOT NULL| Make a column optional                  |
| ALTER COLUMN name SET DEFAULT | Set a default value                      |
| ADD CONSTRAINT name ...       | Add a new constraint                     |
| DROP CONSTRAINT name          | Remove a constraint                      |
+-------------------------------+------------------------------------------+
```

IF NOT EXISTS

When you create a table that already exists, PostgreSQL throws an error:

```sql
CREATE TABLE students (id SERIAL PRIMARY KEY);
-- ERROR: relation "students" already exists
```

To avoid this, use `IF NOT EXISTS`:

```sql
CREATE TABLE IF NOT EXISTS students (
    id SERIAL PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    email VARCHAR(100) UNIQUE,
    age INTEGER
);
```

```
NOTICE: relation "students" already exists, skipping
CREATE TABLE
```

If the table exists, PostgreSQL skips the command and gives you a notice. If the table does not exist, it creates it.

This is especially useful in scripts that you might run multiple times.

Complete Example: Building a School Database

Let us put everything together and create a complete school database from scratch. This example uses everything we have learned in this chapter.

```sql
-- ============================================
-- Step 1: Create the database
-- ============================================

CREATE DATABASE school;

-- Connect to it (in psql, use \c school)
```

```
\c school
```

```sql
-- ============================================
-- Step 2: Create the teachers table
-- ============================================

CREATE TABLE IF NOT EXISTS teachers (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    department VARCHAR(50) NOT NULL,
    hire_date DATE DEFAULT CURRENT_DATE
);
```

Line-by-line:

- `id SERIAL PRIMARY KEY` auto-incrementing unique identifier.
- `name VARCHAR(100) NOT NULL` teacher's name, required, up to 100 characters.
- `email VARCHAR(100) UNIQUE NOT NULL` teacher's email, must be unique, required.
- `department VARCHAR(50) NOT NULL` department name, required.
- `hire_date DATE DEFAULT CURRENT_DATE` automatically set to today when not specified.

```sql
-- ============================================
-- Step 3: Create the courses table
-- ============================================

CREATE TABLE IF NOT EXISTS courses (
    id SERIAL PRIMARY KEY,
    course_name VARCHAR(100) NOT NULL,
    course_code VARCHAR(10) UNIQUE NOT NULL,
    credits INTEGER NOT NULL CHECK (credits >= 1 AND credits <= 6),
    department VARCHAR(50) NOT NULL,
    teacher_id INTEGER REFERENCES teachers(id)
);
```

Line-by-line:

- `course_code VARCHAR(10) UNIQUE NOT NULL` a short unique code like "MATH101."

- `credits INTEGER NOT NULL CHECK (credits >= 1 AND credits <= 6)` credits must be between 1 and 6.
- `teacher_id INTEGER REFERENCES teachers(id)` foreign key linking to the teachers table.

```sql
-- ============================================
-- Step 4: Create the students table
-- ============================================

CREATE TABLE IF NOT EXISTS students (
    id SERIAL PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    date_of_birth DATE,
    enrollment_date DATE DEFAULT CURRENT_DATE,
    gpa NUMERIC(3,2) DEFAULT 0.00 CHECK (gpa >= 0.00 AND gpa <= 4.00),
    is_active BOOLEAN DEFAULT TRUE
);
```

Line-by-line:

- `date_of_birth DATE` optional field for the student's birthday.
- `enrollment_date DATE DEFAULT CURRENT_DATE` auto-set to today.
- `gpa NUMERIC(3,2) DEFAULT 0.00 CHECK (gpa >= 0.00 AND gpa <= 4.00)` GPA starts at 0.00 and must stay between 0.00 and 4.00. `NUMERIC(3,2)` means 3 total digits, 2 after the decimal point (e.g., 3.85).
- `is_active BOOLEAN DEFAULT TRUE` defaults to TRUE for new students.

```sql
-- ============================================
-- Step 5: Create the enrollments junction table
-- ============================================

CREATE TABLE IF NOT EXISTS enrollments (
    id SERIAL PRIMARY KEY,
    student_id INTEGER NOT NULL REFERENCES students(id),
    course_id INTEGER NOT NULL REFERENCES courses(id),
    enrolled_date DATE DEFAULT CURRENT_DATE,
    grade VARCHAR(2),
    UNIQUE (student_id, course_id)
);
```

Line-by-line:

- `student_id INTEGER NOT NULL REFERENCES students(id)` required foreign key to students.
- `course_id INTEGER NOT NULL REFERENCES courses(id)` required foreign key to courses.

- `UNIQUE (student_id, course_id)` prevents a student from enrolling in the same course twice.

The Complete Schema

```
+=================================================================+
|                    SCHOOL DATABASE SCHEMA                       |
+=================================================================+
|                                                                 |
|  +------------------+                                           |
|  |    teachers      |                                           |
|  +------------------+                                           |
|  | id (PK, SERIAL)  |                                           |
|  | name (NOT NULL)   |                                           |
|  | email (UNIQUE)    |                                           |
|  | department        |                                           |
|  | hire_date (DEFAULT)|                                           |
|  +------------------+                                           |
|           |                                                     |
|           | one-to-many                                         |
|           v                                                     |
|  +------------------+                                           |
|  |    courses       |                                           |
|  +------------------+                                           |
|  | id (PK, SERIAL)  |                                           |
|  | course_name       |                                           |
|  | course_code       |                                           |
|  | credits (CHECK)   |                                           |
|  | department        |                                           |
|  | teacher_id (FK)   |---> teachers.id                           |
|  +------------------+                                           |
|           ^                                                     |
|           | one-to-many                                         |
|           |                                                     |
|  +------------------+                                           |
|  |   enrollments    |                                           |
|  +------------------+                                           |
|  | id (PK, SERIAL)  |                                           |
|  | student_id (FK)   |---> students.id                           |
|  | course_id (FK)    |---> courses.id                            |
|  | enrolled_date      |                                           |
|  | grade             |                                           |
|  +------------------+                                           |
|           |                                                     |
|           | one-to-many                                         |
|           v                                                     |
|  +------------------+                                           |
|  |    students      |                                           |
|  +------------------+                                           |
|  | id (PK, SERIAL)  |                                           |
|  | first_name        |                                           |
|  | last_name         |                                           |
|  | email (UNIQUE)    |                                           |
```

```
|  | date_of_birth     |                                          |
|  | enrollment_date   |                                          |
|  | gpa (CHECK)       |                                          |
|  | is_active (DEFAULT)|                                         |
|  +------------------+                                         |
|                                                               |
+===============================================================+
```

Verify Everything

```
\dt
```

```
            List of relations
 Schema |    Name     | Type  |  Owner
--------+-------------+-------+----------
 public | courses     | table | postgres
 public | enrollments | table | postgres
 public | students    | table | postgres
 public | teachers    | table | postgres
(4 rows)
```

```
\d students
```

```
                                          Table "public.students"
     Column      |          Type          | Collation | Nullable |                Default
-----------------+------------------------+-----------+----------+----------------------...
 id              | integer                |           | not null | nextval('students_id...
 first_name      | character varying(50)  |           | not null |
 last_name       | character varying(50)  |           | not null |
 email           | character varying(100) |           | not null |
 date_of_birth   | date                   |           |          |
 enrollment_date | date                   |           |          | CURRENT_DATE
 gpa             | numeric(3,2)           |           |          | 0.00
 is_active       | boolean                |           |          | true
Indexes:
    "students_pkey" PRIMARY KEY, btree (id)
    "students_email_key" UNIQUE CONSTRAINT, btree (email)
Check constraints:
    "students_gpa_check" CHECK (gpa >= 0.00 AND gpa <= 4.00)
```

Everything is in place. In Chapter 6, we will populate these tables with data.

Common Mistakes

1. Forgetting the semicolon. Every SQL command must end with `;`. If your command seems stuck, you probably forgot it.
2. Creating tables in the wrong order. If Table B has a foreign key pointing to Table A, you must create Table A first. Otherwise, PostgreSQL does not know what you are referencing.
3. Using reserved words as column names. Words like `user`, `order`, `table`, `select` are reserved. If you must use them, put them in double quotes: `"user"`, `"order"`. Better yet, choose different names: `app_user`, `customer_order`.
4. Forgetting NOT NULL on important columns. If a column should always have a value (like a person's name), add NOT NULL. Otherwise, you will end up with incomplete data.
5. Not using IF NOT EXISTS. When writing scripts that might run multiple times, always use `CREATE TABLE IF NOT EXISTS` to avoid errors.
6. Dropping tables with CASCADE without thinking. `DROP TABLE ... CASCADE` removes the table and everything that depends on it. This can have unexpected consequences.

Best Practices

1. Always start with a primary key. Every table should have an `id SERIAL PRIMARY KEY` column (or equivalent).
2. Use NOT NULL generously. Require data for columns that should always have values. It is easier to remove a NOT NULL constraint later than to fix dirty data.
3. Use meaningful names. `students` is better than `tbl1`. `enrollment_date` is better than `ed`.
4. Use snake_case for everything. `first_name`, not `firstName` or `First_Name`. This is the PostgreSQL convention.
5. Add CHECK constraints for numeric ranges. If age must be between 0 and 150, say so. Let the database enforce it.
6. Use DEFAULT values where appropriate. `enrollment_date DATE DEFAULT CURRENT_DATE` saves you from having to specify the date every time.
7. Plan your schema on paper first. Draw the tables, columns, and relationships before writing any SQL.

Quick Summary

`CREATE DATABASE` makes a new database. `CREATE TABLE` defines a new table with columns, data types, and constraints. Constraints like NOT NULL, UNIQUE, DEFAULT, and CHECK protect your data quality. `DROP TABLE` removes a table permanently. `ALTER TABLE` lets you modify existing tables by adding, removing, or renaming columns. `IF NOT EXISTS` prevents errors when creating objects that might already exist.

Key Points

- `CREATE DATABASE name;` creates a new database.
- `CREATE TABLE name (columns);` creates a new table.
- NOT NULL ensures a column always has a value.
- UNIQUE prevents duplicate values in a column.
- DEFAULT provides an automatic value when none is specified.
- CHECK enforces a custom rule on values.
- PRIMARY KEY is a combination of UNIQUE and NOT NULL. It identifies each row.
- `DROP TABLE name;` permanently deletes a table.
- `ALTER TABLE` modifies an existing table (add/drop/rename columns).
- `IF NOT EXISTS` prevents "already exists" errors.
- Always create referenced tables before tables that reference them.

Practice Questions

1. What is the difference between `NOT NULL` and `DEFAULT`? Can you use both on the same column?
2. You try to create a table called `orders`, but you get an error saying the name is reserved. How would you solve this?
3. What happens if you run `DROP TABLE students;` and the `enrollments` table has a foreign key pointing to `students`?
4. You have a table with a `price` column. Write a CHECK constraint that ensures the price is always greater than zero.

5. What is the difference between `DROP TABLE students;` and `DROP TABLE IF EXISTS students;`?

Exercises

Exercise 1: Create a Library Database

Create a database called `library` with the following tables:

1. authors table with columns: id (auto-increment PK), name (required), country (optional), birth_year (integer).
2. books table with columns: id (auto-increment PK), title (required, unique), isbn (required, unique), publication_year (integer), price (must be positive), author_id (foreign key to authors).

Write the complete SQL statements. Verify your work with `\dt` and `\d table_name`.

Exercise 2: Modify the Table

Starting from the `students` table in the complete example:

1. Add a column called `phone` (VARCHAR(20)).
2. Add a column called `major` (VARCHAR(50)) with a default value of 'Undeclared'.
3. Rename the `email` column to `school_email`.
4. Drop the `date_of_birth` column.

Write the ALTER TABLE statements for each change.

Exercise 3: Design Constraints

For each scenario below, write the column definition with appropriate constraints:

1. A `temperature` column that stores readings between -100 and 100 degrees.
2. A `status` column that defaults to 'pending' and cannot be empty.
3. A `username` column that is required and must be unique.
4. An `account_balance` column that defaults to 0.00 and cannot be negative.
5. A `created_at` column that defaults to the current date and time.

What Is Next?

Now that you can create databases and tables, you need to know what types of data your columns can hold. In Chapter 5, you will do a deep dive into PostgreSQL data types. You will learn about integers, decimals, text, booleans, dates, UUIDs, JSON, arrays, and how to choose the right type for every situation.

Chapter 5: Data Types in PostgreSQL

What You Will Learn

By the end of this chapter, you will understand:

- Integer types: INTEGER, BIGINT, and SMALLINT
- Auto-incrementing types: SERIAL and BIGSERIAL
- Exact decimal types: NUMERIC and DECIMAL (and why you must use them for money)
- Text types: VARCHAR(n), TEXT, and CHAR(n)
- Boolean type: TRUE and FALSE
- Date and time types: DATE, TIME, TIMESTAMP, and INTERVAL
- UUID: universally unique identifiers
- JSON and JSONB: storing structured data inside a column
- ARRAY: storing lists inside a column
- How to choose the right data type for every situation
- Common mistakes like using FLOAT for money or VARCHAR when TEXT works fine

Why This Chapter Matters

Choosing the right data type is one of the most important decisions you make when designing a database. The wrong data type can cause:

- Lost data (storing a large number in a column too small to hold it).
- Wrong calculations (using FLOAT for money and getting rounding errors).
- Wasted space (using BIGINT when SMALLINT is enough).
- Slow queries (using TEXT when VARCHAR would be faster for indexing).

Getting data types right from the start saves you from painful problems later.

Integer Types: Whole Numbers

Integers are whole numbers without decimal points. PostgreSQL provides three sizes:

```
+------------+----------------------------+-------------------------------+
| Type       | Range                      | Storage Size                  |
+------------+----------------------------+-------------------------------+
| SMALLINT   | -32,768 to 32,767          | 2 bytes                       |
| INTEGER    | -2,147,483,648 to          | 4 bytes                       |
|            |  2,147,483,647             |                               |
| BIGINT     | -9,223,372,036,854,775,808 | 8 bytes                       |
|            |  to 9,223,372,036,854,     |                               |
|            |  775,807                   |                               |
+------------+----------------------------+-------------------------------+
```

Think of it as choosing a container size:

- SMALLINT is a small box. It holds numbers up to about 32 thousand.
- INTEGER is a medium box. It holds numbers up to about 2 billion.
- BIGINT is a large box. It holds numbers up to about 9 quintillion.

When to Use Each

```
+------------+-------------------------------------------------+
| Type       | Use When                                        |
+------------+-------------------------------------------------+
| SMALLINT   | Values will never exceed 32,767                 |
|            | Examples: age, quantity of items in a cart,     |
|            |   number of pages in a book                     |
+------------+-------------------------------------------------+
| INTEGER    | The default choice for most situations          |
|            | Examples: user IDs, product IDs, counts         |
+------------+-------------------------------------------------+
| BIGINT     | Values could exceed 2 billion                   |
|            | Examples: row counts in huge tables,            |
|            |   social media post IDs, financial cents        |
+------------+-------------------------------------------------+
```

Example

```sql
CREATE TABLE products (
    id INTEGER PRIMARY KEY,
    name VARCHAR(100),
    stock_quantity SMALLINT,
    total_views BIGINT
);

INSERT INTO products (id, name, stock_quantity, total_views)
VALUES (1, 'Widget', 500, 15000000000);
```

```sql
SELECT * FROM products;
```

```
 id |  name  | stock_quantity | total_views
----+--------+----------------+-------------
  1 | Widget |            500 | 15000000000
(1 row)
```

What Happens When You Exceed the Limit

```sql
-- SMALLINT max is 32,767
INSERT INTO products (id, name, stock_quantity, total_views)
VALUES (2, 'Gadget', 50000, 100);

-- ERROR: smallint out of range
```

PostgreSQL stops you from inserting a value that does not fit. This is protection against data loss.

SERIAL and BIGSERIAL: Auto-Incrementing IDs

SERIAL and BIGSERIAL are not true data types. They are shortcuts that create an auto-incrementing integer column. Each new row automatically gets the next number in the sequence.

```
+------------+---------------------------+-----------+
| Type       | Underlying Type           | Max Value |
+------------+---------------------------+-----------+
| SERIAL     | INTEGER                   | ~2 billion|
| BIGSERIAL  | BIGINT                    | ~9 quintillion|
```

```
+------------+---------------------------+-----------+
```

Think of it as: A ticket machine at a deli counter. Each customer gets the next number automatically. Nobody has to manually assign numbers.

Example

```sql
CREATE TABLE students (
    id SERIAL PRIMARY KEY,
    first_name VARCHAR(50),
    last_name VARCHAR(50)
);

-- Notice: we do NOT provide the id value
INSERT INTO students (first_name, last_name) VALUES ('Alice', 'Johnson');
INSERT INTO students (first_name, last_name) VALUES ('Bob', 'Smith');
INSERT INTO students (first_name, last_name) VALUES ('Charlie', 'Brown');
```

```sql
SELECT * FROM students;
```

```
 id | first_name | last_name
----+------------+-----------
  1 | Alice      | Johnson
  2 | Bob        | Smith
  3 | Charlie    | Brown
(3 rows)
```

The `id` values (1, 2, 3) were assigned automatically.

SERIAL vs BIGSERIAL

Use `SERIAL` for most tables. Use `BIGSERIAL` only when you expect the table to have more than 2 billion rows. Social media platforms, logging systems, and IoT data might need BIGSERIAL. A school or small business application will be fine with SERIAL.

What Happens If You Delete a Row?

If you delete Bob (id=2) and insert a new student, the new student gets id=4, not id=2. Serial values never reuse numbers:

```sql
DELETE FROM students WHERE id = 2;
INSERT INTO students (first_name, last_name) VALUES ('Diana', 'Prince');
```

```sql
SELECT * FROM students;
```

```
 id | first_name | last_name
----+------------+-----------
  1 | Alice      | Johnson
  3 | Charlie    | Brown
  4 | Diana      | Prince
(3 rows)
```

The gap at id=2 is normal and expected. Do not worry about gaps in SERIAL values.

NUMERIC and DECIMAL: Exact Decimal Numbers

NUMERIC and DECIMAL are identical in PostgreSQL. They store exact decimal numbers with a specified precision.

```
NUMERIC(precision, scale)
```

- Precision is the total number of digits (before and after the decimal point).
- Scale is the number of digits after the decimal point.

```
+---------------------+-------------------------------------+
| Definition          | What It Accepts                     |
+---------------------+-------------------------------------+
| NUMERIC(5, 2)       | Up to 5 total digits, 2 after       |
|                     | the decimal: 999.99 max             |
|                     | Examples: 123.45, 99.99, 0.50       |
+---------------------+-------------------------------------+
| NUMERIC(10, 2)      | Up to 10 digits, 2 after decimal    |
|                     | 99,999,999.99 max                   |
|                     | Good for prices and money           |
+---------------------+-------------------------------------+
| NUMERIC(3, 2)       | Up to 3 digits, 2 after decimal     |
|                     | 9.99 max                            |
|                     | Good for GPA (0.00 to 4.00)         |
+---------------------+-------------------------------------+
```

Why NUMERIC for Money? (The FLOAT Trap)

This is critically important. Never use FLOAT or REAL for money. Here is why:

```sql
-- FLOAT is not exact!
SELECT 0.1::FLOAT + 0.2::FLOAT AS float_result;
```

```
    float_result
--------------------
 0.30000000000000004
```

0.1 + 0.2 does not equal 0.3 in floating-point arithmetic. It equals 0.30000000000000004. This is not a PostgreSQL bug. It is how all computers handle floating-point numbers.

Now compare with NUMERIC:

```sql
-- NUMERIC is exact!
SELECT 0.1::NUMERIC + 0.2::NUMERIC AS numeric_result;
```

```
 numeric_result
----------------
            0.3
```

If you store money as FLOAT, you will eventually have transactions that are off by a fraction of a cent. Over thousands of transactions, these errors add up.

Example

```sql
CREATE TABLE invoices (
    id SERIAL PRIMARY KEY,
    customer_name VARCHAR(100),
    subtotal NUMERIC(10, 2) NOT NULL,
    tax NUMERIC(10, 2) NOT NULL,
    total NUMERIC(10, 2) NOT NULL
);

INSERT INTO invoices (customer_name, subtotal, tax, total)
VALUES ('Alice', 99.99, 8.00, 107.99);
```

```sql
SELECT * FROM invoices;
```

```
 id | customer_name | subtotal | tax  |  total
----+---------------+----------+------+--------
  1 | Alice         |    99.99 | 8.00 | 107.99
(1 row)
```

Text Types: Storing Words and Sentences

PostgreSQL offers three main text types:

```
+----------------+-----------------------------------+---------------------------+
| Type           | Description                       | Max Length                |
+----------------+-----------------------------------+---------------------------+
| CHAR(n)        | Fixed-length text, padded with    | Exactly n characters      |
|                |   spaces if shorter               |                           |
+----------------+-----------------------------------+---------------------------+
| VARCHAR(n)     | Variable-length text with a       | Up to n characters        |
|                |   maximum length                  |                           |
+----------------+-----------------------------------+---------------------------+
| TEXT           | Variable-length text with no      | Unlimited (up to ~1 GB)   |
|                |   length limit                    |                           |
+----------------+-----------------------------------+---------------------------+
```

CHAR(n): Fixed-Length Text

`CHAR(n)` always stores exactly `n` characters. If your text is shorter, it pads the rest with spaces.

```sql
CREATE TABLE codes (
    country_code CHAR(2),
    state_code CHAR(3)
);

INSERT INTO codes VALUES ('US', 'CA');
INSERT INTO codes VALUES ('UK', 'LN');
```

`CHAR(2)` is useful for data that is always exactly 2 characters (like country codes: US, UK, CA).

When to use: Only when every value has the same fixed length. This is rare.

VARCHAR(n): Variable-Length Text with a Limit

`VARCHAR(n)` stores text up to `n` characters. If your text is shorter, it does not pad with spaces.

```sql
CREATE TABLE users (
    username VARCHAR(30),
    email VARCHAR(100)
);

INSERT INTO users VALUES ('alice_j', 'alice.johnson@example.com');
```

`VARCHAR(30)` means the username can be anywhere from 0 to 30 characters.

When to use: When you want to enforce a maximum length. Good for usernames, emails, phone numbers, and other fields with natural limits.

TEXT: Unlimited Text

`TEXT` stores text of any length with no specified limit.

```sql
CREATE TABLE blog_posts (
    id SERIAL PRIMARY KEY,
    title VARCHAR(200),
    content TEXT
);
```

When to use: When you do not need a length limit. Good for descriptions, comments, blog content, and notes.

VARCHAR vs TEXT: The Great Debate

In PostgreSQL, `VARCHAR` without a length limit (just `VARCHAR`) and `TEXT` are identical in performance. There is no speed difference.

```
+---------------------------+-----------------------------------------+
| Use This                  | When                                    |
+---------------------------+-----------------------------------------+
| VARCHAR(n) with a limit   | You want the database to enforce a      |
|                           |   maximum length (e.g., username max 30)|
+---------------------------+-----------------------------------------+
| TEXT                      | You do not need a length limit           |
|                           | (e.g., blog post content, comments)    |
+---------------------------+-----------------------------------------+
| CHAR(n)                   | Every value is exactly n characters      |
|                           | (e.g., country codes, zip codes)       |
+---------------------------+-----------------------------------------+
```

> Common Mistake: Using `VARCHAR(255)` everywhere "just in case." This is a habit from MySQL, where there was a performance difference. In PostgreSQL, there is no reason to

do this. Use `TEXT` if you do not need a limit, or use `VARCHAR(n)` with a meaningful limit.

BOOLEAN: True or False

The BOOLEAN type stores true/false values.

```
+---------------------------+-----------------------------------+
| Valid TRUE Values          | Valid FALSE Values                |
+---------------------------+-----------------------------------+
| TRUE, 't', 'true',         | FALSE, 'f', 'false',              |
| 'yes', 'y', '1', 'on'     | 'no', 'n', '0', 'off'            |
+---------------------------+-----------------------------------+
```

In psql output, TRUE is displayed as `t` and FALSE as `f`.

Example

```sql
CREATE TABLE tasks (
    id SERIAL PRIMARY KEY,
    title VARCHAR(100) NOT NULL,
    is_completed BOOLEAN DEFAULT FALSE,
    is_urgent BOOLEAN DEFAULT FALSE
);

INSERT INTO tasks (title, is_completed, is_urgent)
VALUES
    ('Write report', FALSE, TRUE),
    ('Buy groceries', FALSE, FALSE),
    ('Submit homework', TRUE, FALSE);
```

```sql
SELECT * FROM tasks;
```

```
 id |      title      | is_completed | is_urgent
----+-----------------+--------------+-----------
  1 | Write report    | f            | t
  2 | Buy groceries   | f            | f
  3 | Submit homework | t            | f
(3 rows)
```

Filtering by BOOLEAN

```sql
-- Find all incomplete tasks
SELECT * FROM tasks WHERE is_completed = FALSE;

-- Shorter way (same result):
SELECT * FROM tasks WHERE NOT is_completed;
```

```
 id |     title     | is_completed | is_urgent
----+---------------+--------------+-----------
  1 | Write report  | f            | t
  2 | Buy groceries | f            | f
(2 rows)
```

Date and Time Types

PostgreSQL has rich support for dates and times.

```
+-------------+----------------------------+---------------------------+
| Type        | What It Stores             | Example                   |
+-------------+----------------------------+---------------------------+
| DATE        | Calendar date only         | '2024-06-15'              |
| TIME        | Time of day only           | '14:30:00'                |
| TIMESTAMP   | Date and time combined     | '2024-06-15 14:30:00'     |
| TIMESTAMPTZ | Date, time, and timezone   | '2024-06-15 14:30:00-05'  |
| INTERVAL    | A duration of time         | '3 days', '2 hours'       |
+-------------+----------------------------+---------------------------+
```

DATE

Stores a calendar date in the format `YYYY-MM-DD`.

```sql
CREATE TABLE events (
    id SERIAL PRIMARY KEY,
    event_name VARCHAR(100),
    event_date DATE
);

INSERT INTO events (event_name, event_date)
VALUES ('Conference', '2024-09-15');
```

```sql
SELECT * FROM events;
```

```
 id | event_name | event_date
----+------------+------------
  1 | Conference | 2024-09-15
(1 row)
```

TIME

Stores a time of day without a date.

```sql
ALTER TABLE events ADD COLUMN start_time TIME;

UPDATE events SET start_time = '09:00:00' WHERE id = 1;
```

```sql
SELECT * FROM events;
```

```
 id | event_name | event_date | start_time
----+------------+------------+------------
  1 | Conference | 2024-09-15 | 09:00:00
(1 row)
```

TIMESTAMP

Stores both date and time together.

```sql
CREATE TABLE logs (
    id SERIAL PRIMARY KEY,
    message TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

INSERT INTO logs (message) VALUES ('User logged in');
INSERT INTO logs (message) VALUES ('Page viewed');
```

```sql
SELECT * FROM logs;
```

```
 id |    message     |         created_at
----+----------------+----------------------------
  1 | User logged in | 2024-06-15 14:32:15.123456
  2 | Page viewed    | 2024-06-15 14:32:16.789012
```

```
(2 rows)
```

> Best Practice: Use `TIMESTAMPTZ` (timestamp with time zone) instead of plain `TIMESTAMP` when your application has users in different time zones. `TIMESTAMPTZ` stores the time zone information, so a time entered in New York is correctly displayed in Tokyo.

INTERVAL

An INTERVAL represents a duration of time, not a specific point in time.

```sql
SELECT CURRENT_DATE + INTERVAL '30 days' AS thirty_days_from_now;
```

```
 thirty_days_from_now
----------------------
 2024-07-15
(1 row)
```

```sql
SELECT CURRENT_TIMESTAMP + INTERVAL '2 hours 30 minutes' AS later;
```

```
            later
----------------------------
 2024-06-15 17:02:15.123456
(1 row)
```

Intervals are great for calculating due dates, expiration dates, and time differences.

Useful Date Functions

```
+-------------------------------+-------------------------------+
| Function                      | What It Returns               |
+-------------------------------+-------------------------------+
| CURRENT_DATE                  | Today's date                  |
| CURRENT_TIME                  | Current time                  |
| CURRENT_TIMESTAMP             | Current date and time         |
| NOW()                         | Same as CURRENT_TIMESTAMP     |
| AGE(date)                     | Time elapsed since that date  |
| EXTRACT(YEAR FROM date)       | The year from a date          |
| DATE_TRUNC('month', date)     | Truncate to the first of      |
|                               |   the month                   |
+-------------------------------+-------------------------------+
```

Example with Date Functions

```sql
SELECT
    CURRENT_DATE AS today,
    CURRENT_DATE + INTERVAL '7 days' AS next_week,
    EXTRACT(YEAR FROM CURRENT_DATE) AS current_year,
    EXTRACT(MONTH FROM CURRENT_DATE) AS current_month;
```

```
   today    | next_week  | current_year | current_month
------------+------------+--------------+---------------
 2024-06-15 | 2024-06-22 |         2024 |             6
(1 row)
```

UUID: Universally Unique Identifiers

A UUID (Universally Unique Identifier) is a 128-bit number used as a unique identifier. It looks like this:

```
550e8400-e29b-41d4-a716-446655440000
```

Think of it as: A super-long, random ID that is virtually guaranteed to be unique across all computers everywhere, forever.

Why Use UUID Instead of SERIAL?

```
+------------------+-----------------------------------+
| SERIAL           | UUID                              |
+------------------+-----------------------------------+
| Simple: 1, 2, 3  | Complex: 550e8400-e29b-...        |
| Predictable      | Random, unpredictable             |
| Unique per table | Unique globally                   |
| Smaller (4 bytes)| Larger (16 bytes)                 |
| Easy to guess    | Impossible to guess               |
+------------------+-----------------------------------+
```

Use UUID when:

- You need IDs that are unique across multiple databases or systems.
- You do not want people to guess IDs (security).

- You are building distributed systems.

Example

```sql
-- Enable the UUID extension (one time per database)
CREATE EXTENSION IF NOT EXISTS "uuid-ossp";

CREATE TABLE api_keys (
    id UUID DEFAULT uuid_generate_v4() PRIMARY KEY,
    key_name VARCHAR(100),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

INSERT INTO api_keys (key_name) VALUES ('Production Key');
INSERT INTO api_keys (key_name) VALUES ('Development Key');
```

```sql
SELECT * FROM api_keys;
```

```
                  id                   |    key_name     |         created_at
---------------------------------------+-----------------+----------------------------
 a1b2c3d4-e5f6-7890-abcd-ef1234567890 | Production Key  | 2024-06-15 14:35:00.000000
 f9e8d7c6-b5a4-3210-fedc-ba9876543210 | Development Key | 2024-06-15 14:35:01.000000
(2 rows)
```

> Note: PostgreSQL 13 and later also supports `gen_random_uuid()` without needing the extension.

JSON and JSONB: Structured Data in a Column

JSON (JavaScript Object Notation) is a format for storing structured data. PostgreSQL lets you store JSON directly in a column.

There are two types:

```
+--------+-----------------------------------------------+
| Type   | Description                                   |
+--------+-----------------------------------------------+
| JSON   | Stores JSON as plain text. Preserves          |
|        |   formatting and duplicate keys.              |
+--------+-----------------------------------------------+
```

```
| JSONB  | Stores JSON in a binary format. Faster       |
|        |   for queries, supports indexing.             |
+--------+-----------------------------------------------+
```

Always use JSONB. It is faster for reading and supports indexing. The only reason to use JSON is if you need to preserve the exact formatting of the input (which is rare).

Example

```sql
CREATE TABLE user_profiles (
    id SERIAL PRIMARY KEY,
    username VARCHAR(50),
    preferences JSONB
);

INSERT INTO user_profiles (username, preferences)
VALUES (
    'alice',
    '{"theme": "dark", "language": "en", "notifications": {"email": true, "sms": false}}'
);

INSERT INTO user_profiles (username, preferences)
VALUES (
    'bob',
    '{"theme": "light", "language": "es", "notifications": {"email": false, "sms": true}}'
);
```

```sql
SELECT * FROM user_profiles;
```

```
 id | username |                                      preferences
----+----------+-------------------------------------------------------------------------...
  1 | alice    | {"theme": "dark", "language": "en", "notifications": {"email": true, "...
  2 | bob      | {"theme": "light", "language": "es", "notifications": {"email": false,...
(2 rows)
```

Querying JSON Data

You can query inside JSON columns using the `->` and `->>` operators:

```sql
-- Get the theme (as JSON)
SELECT username, preferences -> 'theme' AS theme
FROM user_profiles;
```

```
 username | theme
----------+--------
 alice    | "dark"
 bob      | "light"
(2 rows)
```

```sql
-- Get the theme (as text, without quotes)
SELECT username, preferences ->> 'theme' AS theme
FROM user_profiles;
```

```
 username | theme
----------+-------
 alice    | dark
 bob      | light
(2 rows)
```

The difference: `->` returns JSON (with quotes). `->>` returns plain text (without quotes).

When to Use JSONB

- Storing user preferences or settings.
- Storing data with a flexible structure that varies from row to row.
- Storing API responses.
- Not for core business data. If every row has the same fields, use regular columns instead.

ARRAY: Storing Lists in a Column

PostgreSQL lets you store arrays (ordered lists) in a column. This is a feature most other databases do not have.

Example

```sql
CREATE TABLE students (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100),
    hobbies TEXT[],
    scores INTEGER[]
```

```
);

INSERT INTO students (name, hobbies, scores)
VALUES ('Alice', ARRAY['reading', 'swimming', 'coding'], ARRAY[95, 88, 92]);

INSERT INTO students (name, hobbies, scores)
VALUES ('Bob', ARRAY['gaming', 'cooking'], ARRAY[78, 85, 90, 88]);
```

```sql
SELECT * FROM students;
```

```
 id | name  |         hobbies          |    scores
----+-------+--------------------------+---------------
  1 | Alice | {reading,swimming,coding}| {95,88,92}
  2 | Bob   | {gaming,cooking}         | {78,85,90,88}
(2 rows)
```

Accessing Array Elements

Arrays in PostgreSQL are 1-indexed (they start counting at 1, not 0):

```sql
-- Get the first hobby
SELECT name, hobbies[1] AS first_hobby FROM students;
```

```
 name  | first_hobby
-------+-------------
 Alice | reading
 Bob   | gaming
(2 rows)
```

Searching Inside Arrays

```sql
-- Find students who have 'coding' as a hobby
SELECT name FROM students WHERE 'coding' = ANY(hobbies);
```

```
 name
-------
 Alice
(1 row)
```

When to Use Arrays

- Storing tags or categories.
- Storing a short list of values that belong to one record.
- Not for large or complex lists. If you need to query or filter by individual items often, use a separate table instead.

Choosing the Right Data Type

Here is a decision guide to help you choose the right type for any column:

```
+---------------------------+-------------------+----------------------------+
| What You Are Storing      | Recommended Type  | Notes                      |
+---------------------------+-------------------+----------------------------+
| Primary key (auto)        | SERIAL            | BIGSERIAL for huge tables  |
| Primary key (manual)      | UUID              | For distributed systems    |
| Person's age              | SMALLINT          | Range: 0-150               |
| Quantity of items         | INTEGER           | Or SMALLINT for small qty  |
| Money / prices            | NUMERIC(10,2)     | NEVER use FLOAT for money  |
| GPA / rating              | NUMERIC(3,2)      | Small precise decimal      |
| Short text (name, email)  | VARCHAR(n)        | Set a meaningful limit     |
| Long text (description)   | TEXT              | No need for a limit        |
| Fixed-length code         | CHAR(n)           | Country code, zip code     |
| Yes/No flag               | BOOLEAN           | DEFAULT FALSE or TRUE      |
| Calendar date             | DATE              | YYYY-MM-DD                 |
| Time of day               | TIME              | HH:MM:SS                   |
| Date + time               | TIMESTAMPTZ       | Always with time zone      |
| Duration                  | INTERVAL          | '3 days', '2 hours'        |
| Globally unique ID        | UUID              | uuid_generate_v4()         |
| Flexible/nested data      | JSONB             | Preferences, metadata      |
| Short list of values      | ARRAY             | Tags, scores               |
+---------------------------+-------------------+----------------------------+
```

The Golden Rules

1. Use INTEGER for most whole numbers. Only use SMALLINT or BIGINT when you have a specific reason.
2. Use NUMERIC for money. Always. Never FLOAT, never REAL, never DOUBLE PRECISION.
3. Use TEXT for text without a size limit. Use VARCHAR(n) only when you need to enforce a maximum length.

4. Use TIMESTAMPTZ, not TIMESTAMP. Always store time zones. You will thank yourself later.
5. Use BOOLEAN for yes/no values. Do not use INTEGER (0/1) or VARCHAR ('yes'/'no').

Common Mistakes

1. Using FLOAT or REAL for money. This leads to rounding errors. Always use `NUMERIC(10,2)` or similar for financial data.

```sql
-- BAD: Will cause rounding errors
price FLOAT

-- GOOD: Exact decimal
price NUMERIC(10,2)
```

2. Using VARCHAR(255) for everything. This is a MySQL habit. In PostgreSQL, VARCHAR without a limit and TEXT are identical in performance. Use TEXT if you do not need a limit.

```sql
-- Unnecessary in PostgreSQL
description VARCHAR(255)

-- Better: use TEXT if no limit needed
description TEXT

-- Better: use VARCHAR with a meaningful limit
username VARCHAR(30)
```

3. Using CHAR when VARCHAR would be better. CHAR pads with spaces, which can cause unexpected behavior when comparing strings. Only use CHAR for truly fixed-length data like country codes.
4. Using TIMESTAMP instead of TIMESTAMPTZ. If your application has users in different time zones, plain TIMESTAMP loses time zone information and leads to bugs.
5. Storing dates as strings. If you store a date as VARCHAR ('June 15, 2024'), you cannot sort by date, calculate differences, or use date functions. Always use the DATE type.

```sql
-- BAD: Cannot sort or compare properly
event_date VARCHAR(20)

-- GOOD: Full date functionality
event_date DATE
```

6. Using BIGINT when INTEGER is enough. BIGINT uses twice the storage (8 bytes vs 4 bytes). For a table with millions of rows, this adds up. Use INTEGER unless you genuinely need numbers larger than 2 billion.

Best Practices

1. Match the type to the data. Ages are SMALLINT. Prices are NUMERIC. Names are VARCHAR with a limit. Descriptions are TEXT. Do not overthink it, but do not use the same type for everything.
2. Use NUMERIC for any financial calculation. There are no exceptions. Money must be exact.
3. Set meaningful VARCHAR limits. If an email address can be at most 254 characters (per the email standard), use VARCHAR(254). If a name can be at most 100 characters, use VARCHAR(100).
4. Use BOOLEAN instead of tricks. Do not use an INTEGER column with 0 and 1 to represent true/false. PostgreSQL has a proper BOOLEAN type. Use it.
5. Prefer TIMESTAMPTZ for timestamps. Always include time zone information unless you have a very specific reason not to.
6. Use JSONB sparingly. It is powerful, but overusing it defeats the purpose of a relational database. Use regular columns for core data and JSONB for truly flexible metadata.
7. Use SERIAL for primary keys in most tables. It is simple, efficient, and sufficient for the vast majority of applications.

Quick Summary

PostgreSQL offers a rich set of data types. INTEGER, SMALLINT, and BIGINT store whole numbers of different sizes. SERIAL auto-increments for primary keys. NUMERIC provides exact decimal storage and must be used for money. VARCHAR(n) enforces a maximum text length, while TEXT has no limit. BOOLEAN stores true/false values. DATE, TIME, TIMESTAMP, and INTERVAL handle dates and durations. UUID provides globally unique identifiers. JSONB stores flexible structured data, and ARRAY stores ordered lists. Choosing the right type prevents data loss, rounding errors, and wasted space.

Key Points

- INTEGER is the default for whole numbers. Use SMALLINT for small ranges and BIGINT for very large numbers.
- SERIAL auto-increments and is perfect for primary keys.
- NUMERIC(p,s) stores exact decimals. Always use it for money.
- VARCHAR(n) limits text length. TEXT has no limit. Both perform the same in PostgreSQL.
- CHAR(n) is fixed-length and pads with spaces. Use it rarely.
- BOOLEAN stores TRUE or FALSE.
- DATE stores a date. TIMESTAMPTZ stores date, time, and timezone.
- INTERVAL represents a duration like '3 days' or '2 hours'.
- UUID provides globally unique identifiers.
- JSONB stores flexible structured data. Use it for metadata, not core data.
- ARRAY stores lists. Use separate tables for large or frequently queried lists.
- Never use FLOAT for money. This cannot be stressed enough.

Practice Questions

1. You are storing product prices that can range from $0.01 to $99,999.99. What data type and parameters would you use?
2. What is the difference between VARCHAR(100) and TEXT in PostgreSQL? When would you use each?
3. Why is it dangerous to use FLOAT for financial calculations? Give an example of what could go wrong.
4. What is the difference between TIMESTAMP and TIMESTAMPTZ? When should you use each?
5. You have a column that stores a student's grade letter (A, B, C, D, F). Would you use CHAR(1), VARCHAR(2), or TEXT? Why?

Exercises

Exercise 1: Design a Product Catalog

Create a table called `products` with appropriate data types for:

- A unique auto-incrementing ID
- Product name (max 150 characters, required)
- Description (no length limit)
- Price (exact, up to $999,999.99)
- Stock quantity (will never exceed 10,000)
- Is available (yes/no, default yes)
- Weight in kilograms (up to 9999.99 kg)
- Release date
- Tags (a list of text tags)
- Created timestamp (auto-set)

Write the complete `CREATE TABLE` statement.

Exercise 2: Fix the Bad Schema

The following table has several data type mistakes. Identify each mistake and write the corrected version:

```sql
CREATE TABLE bank_accounts (
    id VARCHAR(10) PRIMARY KEY,
    account_holder CHAR(255),
    balance FLOAT,
    is_active INTEGER,
    opened_date VARCHAR(20),
    account_type BOOLEAN
);
```

Exercise 3: JSON Exploration

Create a table called `app_settings` with an id, app_name (VARCHAR), and config (JSONB) column. Insert two rows with different JSON structures in the config column. Then write a SELECT query that extracts a specific value from the JSON.

What Is Next?

Now that you understand data types, it is time to fill your tables with data. In Chapter 6, you will learn how to insert data using INSERT INTO, insert multiple rows at once, use the

RETURNING clause, handle conflicts with ON CONFLICT (upsert), and bulk load data with COPY. Your empty tables are about to come alive.

Chapter 6: Inserting Data

What You Will Learn

By the end of this chapter, you will know how to:

- Insert a single row with `INSERT INTO`
- Insert multiple rows in one statement
- Insert data with a column list (and why you should always use one)
- Use DEFAULT values during insertion
- Get back the inserted data with `RETURNING` (a PostgreSQL feature)
- Copy data from one table to another with `INSERT ... SELECT`
- Handle duplicate conflicts with `ON CONFLICT` (upsert)
- Bulk load large amounts of data with `COPY`
- Understand the difference between NULL and an empty string
- Build a complete school database with sample data

Why This Chapter Matters

A database without data is like a library without books. You have built the shelves (tables) and labeled them (columns and types). Now it is time to fill them up. Knowing how to insert data efficiently and correctly is a fundamental skill. Every application that saves user input, logs events, or stores records uses INSERT statements constantly.

INSERT INTO: Adding a Single Row

The `INSERT INTO` command adds a new row to a table.

Basic Syntax

```sql
INSERT INTO table_name (column1, column2, column3)
VALUES (value1, value2, value3);
```

Example

Let us start with a simple `students` table:

```sql
CREATE TABLE students (
    id SERIAL PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    email VARCHAR(100) UNIQUE,
    age INTEGER,
    enrollment_date DATE DEFAULT CURRENT_DATE
);
```

Now insert one student:

```sql
INSERT INTO students (first_name, last_name, email, age)
VALUES ('Alice', 'Johnson', 'alice@email.com', 20);
```

```
INSERT 0 1
```

Line-by-line:

- `INSERT INTO students` tells PostgreSQL which table to add data to.
- `(first_name, last_name, email, age)` lists the columns we are providing values for. We did not list `id` (because SERIAL handles it automatically) or `enrollment_date` (because it has a DEFAULT).
- `VALUES ('Alice', 'Johnson', 'alice@email.com', 20)` provides the actual data. The values must match the columns in order.
- The response `INSERT 0 1` means: 0 is the OID (an internal identifier, usually 0) and 1 is the number of rows inserted.

Let us verify:

```sql
SELECT * FROM students;
```

```
 id | first_name | last_name |      email      | age | enrollment_date
----+------------+-----------+-----------------+-----+-----------------
  1 | Alice      | Johnson   | alice@email.com |  20 | 2024-06-15
(1 row)
```

Notice that `id` was automatically set to 1 (by SERIAL) and `enrollment_date` was automatically set to today's date (by DEFAULT CURRENT_DATE).

Inserting Multiple Rows

You can insert several rows in a single statement by separating each set of values with a comma. This is faster than running multiple INSERT statements.

Syntax

```sql
INSERT INTO table_name (column1, column2, column3)
VALUES
    (value1a, value2a, value3a),
    (value1b, value2b, value3b),
    (value1c, value2c, value3c);
```

Example

```sql
INSERT INTO students (first_name, last_name, email, age)
VALUES
    ('Bob', 'Smith', 'bob@email.com', 22),
    ('Charlie', 'Brown', 'charlie@email.com', 19),
    ('Diana', 'Prince', 'diana@email.com', 21),
    ('Eve', 'Davis', 'eve@email.com', 23);
```

```
INSERT 0 4
```

The response `INSERT 0 4` confirms that 4 rows were inserted.

```sql
SELECT * FROM students;
```

```
 id | first_name | last_name |       email       | age | enrollment_date
----+------------+-----------+-------------------+-----+-----------------
  1 | Alice      | Johnson   | alice@email.com   |  20 | 2024-06-15
  2 | Bob        | Smith     | bob@email.com     |  22 | 2024-06-15
  3 | Charlie    | Brown     | charlie@email.com |  19 | 2024-06-15
  4 | Diana      | Prince    | diana@email.com   |  21 | 2024-06-15
  5 | Eve        | Davis     | eve@email.com     |  23 | 2024-06-15
(5 rows)
```

Why Multi-Row INSERT Is Better

```
+---------------------------+--------------------------+
| Approach                  | Performance              |
+---------------------------+--------------------------+
| 5 separate INSERT         | 5 round trips to the     |
|   statements              |   database               |
+---------------------------+--------------------------+
| 1 INSERT with 5 rows      | 1 round trip to the      |
|   in VALUES               |   database               |
+---------------------------+--------------------------+
```

When inserting 5 rows, the difference is negligible. When inserting 10,000 rows, multi-row INSERT can be 10 to 50 times faster than individual inserts.

INSERT with Column List

You should always specify the column list in your INSERT statements.

With Column List (Recommended)

```sql
INSERT INTO students (first_name, last_name, email, age)
VALUES ('Frank', 'Wilson', 'frank@email.com', 24);
```

Without Column List (Not Recommended)

```sql
INSERT INTO students
VALUES (7, 'Frank', 'Wilson', 'frank@email.com', 24, '2024-06-15');
```

This works, but it is fragile and dangerous. Here is why:

1. You must provide values for ALL columns, including auto-generated ones like `id`.
2. The order must match exactly. If someone adds a column to the table, your INSERT breaks.
3. It is hard to read. Without column names, you cannot tell what each value represents.

```
+-----------------------------------+-----------------------------------+
| Without Column List               | With Column List                  |
+-----------------------------------+-----------------------------------+
| VALUES (7, 'Frank', 'Wilson',     | (first_name, last_name, email,    |
|   'frank@email.com', 24,         |   age)                            |
|   '2024-06-15')                   | VALUES ('Frank', 'Wilson',        |
|                                   |   'frank@email.com', 24)          |
| What is 7? What is 24?           | Clear: first_name is 'Frank',     |
| Hard to understand.               |   age is 24.                      |
+-----------------------------------+-----------------------------------+
```

Rule of thumb: Always use a column list. It makes your code clear, robust, and maintainable.

INSERT with DEFAULT

When a column has a DEFAULT value, you can let PostgreSQL use it in several ways.

Method 1: Omit the Column

Simply leave the column out of your column list:

```sql
INSERT INTO students (first_name, last_name, email, age)
VALUES ('Grace', 'Lee', 'grace@email.com', 20);
```

Since `enrollment_date` is not in the column list, PostgreSQL uses its DEFAULT (CURRENT_DATE).

Method 2: Use the DEFAULT Keyword

You can explicitly say "use the default":

```sql
INSERT INTO students (first_name, last_name, email, age, enrollment_date)
VALUES ('Grace', 'Lee', 'grace@email.com', 20, DEFAULT);
```

This does the same thing as Method 1 but is more explicit. Use this when you want to make it clear that you intentionally want the default value.

Method 3: DEFAULT VALUES (All Defaults)

If every column has either a DEFAULT or is nullable, you can insert a row with all defaults:

```sql
CREATE TABLE counters (
    id SERIAL PRIMARY KEY,
    count INTEGER DEFAULT 0,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

INSERT INTO counters DEFAULT VALUES;
```

```sql
SELECT * FROM counters;
```

```
 id | count |         created_at
----+-------+----------------------------
  1 |     0 | 2024-06-15 14:45:00.000000
(1 row)
```

Every column used its default value. This is useful for tables where you want to create a record and fill in details later.

RETURNING: Get Back What You Inserted

The `RETURNING` clause is a powerful PostgreSQL feature that returns the data you just inserted. This is incredibly useful when you need the auto-generated `id` of a new row.

Syntax

```sql
INSERT INTO table_name (columns)
VALUES (values)
RETURNING column1, column2, ...;
```

Example: Get the New ID

```sql
INSERT INTO students (first_name, last_name, email, age)
VALUES ('Henry', 'Taylor', 'henry@email.com', 21)
RETURNING id;
```

```
 id
----
  8
(1 row)
```

Instead of the usual `INSERT 0 1` response, you get back the id of the newly inserted row. This is essential in applications where you need the new record's ID immediately (for example, to insert related data in another table).

Example: Return Multiple Columns

```sql
INSERT INTO students (first_name, last_name, email, age)
VALUES ('Ivy', 'Clark', 'ivy@email.com', 22)
RETURNING id, first_name, last_name, enrollment_date;
```

```
 id | first_name | last_name | enrollment_date
----+------------+-----------+-----------------
  9 | Ivy        | Clark     | 2024-06-15
(1 row)
```

Example: Return Everything

```sql
INSERT INTO students (first_name, last_name, email, age)
VALUES ('Jack', 'White', 'jack@email.com', 19)
RETURNING *;
```

```
 id | first_name | last_name |     email      | age | enrollment_date
----+------------+-----------+----------------+-----+-----------------
 10 | Jack       | White     | jack@email.com |  19 | 2024-06-15
(1 row)
```

`RETURNING *` returns all columns of the inserted row, including auto-generated ones.

RETURNING with Multi-Row INSERT

```sql
INSERT INTO students (first_name, last_name, email, age)
VALUES
    ('Kate', 'Adams', 'kate@email.com', 20),
    ('Leo', 'Brown', 'leo@email.com', 23)
RETURNING id, first_name, email;
```

```
 id | first_name |     email
----+------------+----------------
 11 | Kate       | kate@email.com
 12 | Leo        | leo@email.com
(2 rows)
```

> Note: RETURNING is a PostgreSQL extension. It is not available in all databases (MySQL does not have it, for example). But it is one of the features that makes PostgreSQL so great.

INSERT from SELECT: Copying Data Between Tables

You can insert data into a table by selecting it from another table (or even the same table).

Syntax

```sql
INSERT INTO target_table (column1, column2)
SELECT column1, column2
FROM source_table
WHERE condition;
```

Example: Archive Old Students

Let us say we want to move graduated students to an archive table:

```sql
-- Create the archive table with the same structure
CREATE TABLE students_archive (
    id INTEGER,
    first_name VARCHAR(50),
    last_name VARCHAR(50),
    email VARCHAR(100),
    age INTEGER,
    enrollment_date DATE,
    archived_date DATE DEFAULT CURRENT_DATE
);

-- Copy students older than 22 to the archive
INSERT INTO students_archive (id, first_name, last_name, email, age, enrollment_date)
SELECT id, first_name, last_name, email, age, enrollment_date
FROM students
WHERE age > 22;
```

```
INSERT 0 2
```

```sql
SELECT * FROM students_archive;
```

```
 id | first_name | last_name |     email      | age | enrollment_date | archived_date
----+------------+-----------+----------------+-----+-----------------+---------------
  5 | Eve        | Davis     | eve@email.com  |  23 | 2024-06-15      | 2024-06-15
 12 | Leo        | Brown     | leo@email.com  |  23 | 2024-06-15      | 2024-06-15
(2 rows)
```

Line-by-line:

- `INSERT INTO students_archive (id, first_name, ...)` specifies the target table and columns.
- `SELECT id, first_name, ... FROM students` selects the data from the source table.
- `WHERE age > 22` filters to only include students older than 22.
- The `archived_date` was not in the column list, so it used its DEFAULT (CURRENT_DATE).

Example: Create a Summary Table

```sql
CREATE TABLE department_counts (
    department VARCHAR(50),
    student_count INTEGER
);
```

```
INSERT INTO department_counts (department, student_count)
SELECT department, COUNT(*)
FROM courses
GROUP BY department;
```

This creates a summary by counting courses per department and storing the results in a new table.

ON CONFLICT: Upsert (Insert or Update)

Upsert means "insert if new, update if already exists." PostgreSQL handles this with `ON CONFLICT`.

Think of it as: A guest list at a hotel. If the guest is new, check them in (INSERT). If they are already checked in, update their room information (UPDATE).

Syntax

```sql
INSERT INTO table_name (columns)
VALUES (values)
ON CONFLICT (conflict_column)
DO UPDATE SET column = value;
```

Example: Insert or Update

Let us create a table where each student can have only one profile:

```sql
CREATE TABLE student_profiles (
    student_email VARCHAR(100) PRIMARY KEY,
    nickname VARCHAR(50),
    bio TEXT,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Insert a new profile:

```sql
INSERT INTO student_profiles (student_email, nickname, bio)
VALUES ('alice@email.com', 'Ali', 'Math enthusiast')
ON CONFLICT (student_email)
DO UPDATE SET
    nickname = EXCLUDED.nickname,
    bio = EXCLUDED.bio,
    updated_at = CURRENT_TIMESTAMP;
```

```
INSERT 0 1
```

Line-by-line:

- `INSERT INTO student_profiles ... VALUES (...)` tries to insert a new row.
- `ON CONFLICT (student_email)` if a row with this email already exists...
- `DO UPDATE SET` ...then update these columns instead.
- `EXCLUDED.nickname` refers to the value that was attempted to be inserted. In this case, 'Ali'.
- `updated_at = CURRENT_TIMESTAMP` also updates the timestamp.

Now run the same command again with a different nickname:

```sql
INSERT INTO student_profiles (student_email, nickname, bio)
VALUES ('alice@email.com', 'Alice J', 'Math and science enthusiast')
ON CONFLICT (student_email)
DO UPDATE SET
    nickname = EXCLUDED.nickname,
    bio = EXCLUDED.bio,
    updated_at = CURRENT_TIMESTAMP;
```

```sql
SELECT * FROM student_profiles;
```

```
  student_email  | nickname |             bio              |         updated_at
-----------------+----------+------------------------------+----------------------------
 alice@email.com | Alice J  | Math and science enthusiast | 2024-06-15 15:10:00.000000
(1 row)
```

The row was updated, not duplicated. The nickname changed from 'Ali' to 'Alice J' and the bio was updated.

ON CONFLICT DO NOTHING

If you just want to skip the insert when a conflict occurs (without updating):

```sql
INSERT INTO student_profiles (student_email, nickname, bio)
VALUES ('alice@email.com', 'Ignored', 'This will be ignored')
ON CONFLICT (student_email)
DO NOTHING;
```

```
INSERT 0 0
```

The response `INSERT 0 0` means zero rows were inserted. The existing row was not touched.

When to Use ON CONFLICT

- Importing data that might have duplicates.
- Synchronizing data from an external source.
- Caching or storing "last known" values.
- Any situation where you want "insert if new, update if exists" behavior.

COPY: Bulk Loading Data

For loading large amounts of data (thousands or millions of rows), `COPY` is much faster than INSERT. It reads data directly from a file.

Loading from a CSV File

First, create a CSV file called `students.csv`:

```
first_name,last_name,email,age
Alice,Johnson,alice@school.com,20
Bob,Smith,bob@school.com,22
Charlie,Brown,charlie@school.com,19
Diana,Prince,diana@school.com,21
```

Then load it into PostgreSQL:

```sql
COPY students (first_name, last_name, email, age)
FROM '/path/to/students.csv'
DELIMITER ','
CSV HEADER;
```

Line-by-line:

- `COPY students (first_name, last_name, email, age)` specifies the table and columns.
- `FROM '/path/to/students.csv'` points to the file on disk. This must be an absolute path.
- `DELIMITER ','` says the columns are separated by commas.
- `CSV HEADER` tells PostgreSQL that the first line contains column names, not data.

The \copy Command (For Client-Side Files)

The `COPY` command runs on the server and needs the file to be on the server's filesystem. If you are working locally and want to load a file from your own computer, use `\copy` (with a backslash) instead:

```
\copy students (first_name, last_name, email, age) FROM '/path/to/students.csv' DELIMIT...
```

`\copy` is a psql command (not SQL), so it accesses files on the client machine. It has the same syntax as COPY but runs client-side.

Exporting Data to a CSV File

You can also export data from a table to a CSV file:

```sql
COPY students TO '/path/to/export.csv' DELIMITER ',' CSV HEADER;
```

Or using psql's `\copy`:

```
\copy students TO '/path/to/export.csv' DELIMITER ',' CSV HEADER;
```

Performance Comparison

```
+---------------------------+-----------------------------------+
| Method                    | Speed for 100,000 Rows            |
+---------------------------+-----------------------------------+
| Individual INSERT         | ~30-60 seconds                    |
|   statements              |                                   |
+---------------------------+-----------------------------------+
| Multi-row INSERT          | ~5-10 seconds                     |
|   (batches of 1000)       |                                   |
+---------------------------+-----------------------------------+
| COPY from CSV             | ~1-2 seconds                      |
```

```
+--------------------------+-----------------------------------+
```

For large data loads, COPY is the clear winner.

NULL vs Empty String

This is a concept that trips up many beginners. NULL and an empty string ('') are not the same thing.

```
+-------------------+-----------------------------------------+
| Value             | Meaning                                 |
+-------------------+-----------------------------------------+
| NULL              | "Unknown" or "Not applicable"           |
|                   | There is no value.                      |
+-------------------+-----------------------------------------+
| '' (empty string) | "Known to be empty"                     |
|                   | There IS a value, and it is empty text. |
+-------------------+-----------------------------------------+
```

Think of it this way:

- NULL is like a blank space on a form. The person did not fill it in. You do not know the answer.
- An empty string is like a person who wrote nothing in the box on purpose. The answer is intentionally empty.

Example

```sql
CREATE TABLE contacts (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    phone VARCHAR(20),
    notes TEXT
);

INSERT INTO contacts (name, phone, notes) VALUES ('Alice', '555-1234', 'Friend');
INSERT INTO contacts (name, phone, notes) VALUES ('Bob', NULL, 'No phone');
INSERT INTO contacts (name, phone, notes) VALUES ('Charlie', '', 'Has phone but unknown');
```

```sql
SELECT * FROM contacts;
```

```
 id |  name   |  phone   |         notes
----+---------+----------+-----------------------
  1 | Alice   | 555-1234 | Friend
  2 | Bob     |          | No phone
  3 | Charlie |          | Has phone but unknown
(2 rows)
```

They look the same in the output, but they are different:

```sql
-- Find contacts where phone is NULL
SELECT name FROM contacts WHERE phone IS NULL;
```

```
 name
------
 Bob
(1 row)
```

```sql
-- Find contacts where phone is an empty string
SELECT name FROM contacts WHERE phone = '';
```

```
  name
---------
 Charlie
(1 row)
```

Important NULL Rules

1. NULL is not equal to anything, not even another NULL.

```sql
   SELECT NULL = NULL;  -- Returns NULL, not TRUE
```

2. Use IS NULL and IS NOT NULL to check for NULL.

```sql
   -- Correct:
   WHERE phone IS NULL

   -- Wrong (will not work as expected):
   WHERE phone = NULL
```

3. NULL in calculations produces NULL.

```sql
   SELECT 5 + NULL;  -- Returns NULL
   SELECT NULL || 'hello';  -- Returns NULL (|| is string concatenation)
```

Complete Example: Populating a School Database

Let us put everything together. We will create a school database and fill it with realistic sample data.

Step 1: Create the Tables

```sql
-- Create a fresh database
CREATE DATABASE school_complete;
```

```
\c school_complete
```

```sql
-- Teachers
CREATE TABLE teachers (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    department VARCHAR(50) NOT NULL,
    hire_date DATE DEFAULT CURRENT_DATE
);

-- Courses
CREATE TABLE courses (
    id SERIAL PRIMARY KEY,
    course_name VARCHAR(100) NOT NULL,
    course_code VARCHAR(10) UNIQUE NOT NULL,
    credits INTEGER NOT NULL CHECK (credits >= 1 AND credits <= 6),
    department VARCHAR(50) NOT NULL,
    teacher_id INTEGER REFERENCES teachers(id),
    max_students INTEGER DEFAULT 30
);

-- Students
CREATE TABLE students (
    id SERIAL PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
```

```
    date_of_birth DATE,
    enrollment_date DATE DEFAULT CURRENT_DATE,
    gpa NUMERIC(3,2) DEFAULT 0.00 CHECK (gpa >= 0.00 AND gpa <= 4.00),
    is_active BOOLEAN DEFAULT TRUE
);

-- Enrollments (junction table)
CREATE TABLE enrollments (
    id SERIAL PRIMARY KEY,
    student_id INTEGER NOT NULL REFERENCES students(id),
    course_id INTEGER NOT NULL REFERENCES courses(id),
    enrolled_date DATE DEFAULT CURRENT_DATE,
    grade VARCHAR(2),
    UNIQUE (student_id, course_id)
);
```

Step 2: Insert Teachers

```sql
INSERT INTO teachers (name, email, department, hire_date)
VALUES
    ('Dr. Sarah Chen', 'chen@school.edu', 'Mathematics', '2015-08-15'),
    ('Prof. James Miller', 'miller@school.edu', 'Computer Science', '2018-01-10'),
    ('Dr. Maria Garcia', 'garcia@school.edu', 'History', '2012-09-01'),
    ('Prof. David Kim', 'kim@school.edu', 'Physics', '2020-03-20'),
    ('Dr. Emily Brown', 'brown@school.edu', 'English', '2016-06-01')
RETURNING id, name, department;
```

```
 id |        name        |    department
----+--------------------+------------------
  1 | Dr. Sarah Chen     | Mathematics
  2 | Prof. James Miller | Computer Science
  3 | Dr. Maria Garcia   | History
  4 | Prof. David Kim    | Physics
  5 | Dr. Emily Brown    | English
(5 rows)
```

Step 3: Insert Courses

```sql
INSERT INTO courses (course_name, course_code, credits, department, teacher_id, max_stu...
VALUES
    ('Calculus I', 'MATH101', 4, 'Mathematics', 1, 35),
    ('Linear Algebra', 'MATH201', 3, 'Mathematics', 1, 30),
    ('Intro to Programming', 'CS101', 3, 'Computer Science', 2, 40),
    ('Data Structures', 'CS201', 4, 'Computer Science', 2, 30),
```

```
    ('World History', 'HIST101', 3, 'History', 3, 50),
    ('American History', 'HIST201', 3, 'History', 3, 40),
    ('Physics I', 'PHYS101', 4, 'Physics', 4, 35),
    ('Creative Writing', 'ENG101', 3, 'English', 5, 25),
    ('English Literature', 'ENG201', 3, 'English', 5, 30)
RETURNING id, course_name, course_code;
```

```
 id |     course_name      | course_code
----+----------------------+-------------
  1 | Calculus I            | MATH101
  2 | Linear Algebra        | MATH201
  3 | Intro to Programming  | CS101
  4 | Data Structures       | CS201
  5 | World History         | HIST101
  6 | American History      | HIST201
  7 | Physics I             | PHYS101
  8 | Creative Writing      | ENG101
  9 | English Literature    | ENG201
(9 rows)
```

Step 4: Insert Students

```sql
INSERT INTO students (first_name, last_name, email, date_of_birth, gpa)
VALUES
    ('Alice', 'Johnson', 'alice.j@student.edu', '2004-03-15', 3.85),
    ('Bob', 'Smith', 'bob.s@student.edu', '2003-07-22', 3.20),
    ('Charlie', 'Brown', 'charlie.b@student.edu', '2004-11-08', 2.95),
    ('Diana', 'Prince', 'diana.p@student.edu', '2003-01-30', 3.90),
    ('Eve', 'Davis', 'eve.d@student.edu', '2004-05-12', 3.50),
    ('Frank', 'Wilson', 'frank.w@student.edu', '2003-09-25', 2.80),
    ('Grace', 'Lee', 'grace.l@student.edu', '2004-02-14', 3.75),
    ('Henry', 'Taylor', 'henry.t@student.edu', '2003-12-01', 3.10),
    ('Ivy', 'Clark', 'ivy.c@student.edu', '2004-08-19', 3.60),
    ('Jack', 'White', 'jack.w@student.edu', '2003-04-05', 2.70)
RETURNING id, first_name, last_name, gpa;
```

```
 id | first_name | last_name | gpa
----+------------+-----------+------
  1 | Alice      | Johnson   | 3.85
  2 | Bob        | Smith     | 3.20
  3 | Charlie    | Brown     | 2.95
  4 | Diana      | Prince    | 3.90
  5 | Eve        | Davis     | 3.50
  6 | Frank      | Wilson    | 2.80
  7 | Grace      | Lee       | 3.75
  8 | Henry      | Taylor    | 3.10
  9 | Ivy        | Clark     | 3.60
 10 | Jack       | White     | 2.70
```

```
(10 rows)
```

Step 5: Insert Enrollments

```sql
INSERT INTO enrollments (student_id, course_id, grade)
VALUES
    -- Alice: Calculus, Data Structures, World History
    (1, 1, 'A'),
    (1, 4, 'A-'),
    (1, 5, 'B+'),

    -- Bob: Intro to Programming, Physics, Creative Writing
    (2, 3, 'B'),
    (2, 7, 'B+'),
    (2, 8, 'A'),

    -- Charlie: Calculus, World History, American History
    (3, 1, 'C+'),
    (3, 5, 'B'),
    (3, 6, 'B-'),

    -- Diana: Linear Algebra, Data Structures, English Literature
    (4, 2, 'A'),
    (4, 4, 'A'),
    (4, 9, 'A-'),

    -- Eve: Intro to Programming, Physics, Creative Writing
    (5, 3, 'A-'),
    (5, 7, 'B+'),
    (5, 8, 'A'),

    -- Frank: Calculus, World History
    (6, 1, 'C'),
    (6, 5, 'C+'),

    -- Grace: Linear Algebra, Data Structures, English Literature
    (7, 2, 'A-'),
    (7, 4, 'B+'),
    (7, 9, 'A'),

    -- Henry: Intro to Programming, American History, Physics
    (8, 3, 'B-'),
    (8, 6, 'B'),
    (8, 7, 'C+'),

    -- Ivy: Calculus, Creative Writing, English Literature
    (9, 1, 'B+'),
    (9, 8, 'A'),
    (9, 9, 'A-'),
```

```
    -- Jack: World History, American History
    (10, 5, 'C'),
    (10, 6, 'C-')
RETURNING id, student_id, course_id, grade;
```

```
 id | student_id | course_id | grade
----+------------+-----------+-------
  1 |          1 |         1 | A
  2 |          1 |         4 | A-
  3 |          1 |         5 | B+
  4 |          2 |         3 | B
  5 |          2 |         7 | B+
  6 |          2 |         8 | A
  7 |          3 |         1 | C+
  8 |          3 |         5 | B
  9 |          3 |         6 | B-
 10 |          4 |         2 | A
 11 |          4 |         4 | A
 12 |          4 |         9 | A-
 13 |          5 |         3 | A-
 14 |          5 |         7 | B+
 15 |          5 |         8 | A
 16 |          6 |         1 | C
 17 |          6 |         5 | C+
 18 |          7 |         2 | A-
 19 |          7 |         4 | B+
 20 |          7 |         9 | A
 21 |          8 |         3 | B-
 22 |          8 |         6 | B
 23 |          8 |         7 | C+
 24 |          9 |         1 | B+
 25 |          9 |         8 | A
 26 |          9 |         9 | A-
 27 |         10 |         5 | C
 28 |         10 |         6 | C-
(28 rows)
```

Step 6: Verify the Data

Let us run a few queries to make sure everything looks right:

```sql
-- How many students?
SELECT COUNT(*) AS total_students FROM students;
```

```
 total_students
----------------
             10
(1 row)
```

```sql
-- How many courses per department?
SELECT department, COUNT(*) AS course_count
FROM courses
GROUP BY department
ORDER BY course_count DESC;
```

```
    department    | course_count
------------------+--------------
 Mathematics      |            2
 Computer Science |            2
 History          |            2
 English          |            2
 Physics          |            1
(5 rows)
```

```sql
-- Which students are in Calculus I?
SELECT s.first_name, s.last_name, e.grade
FROM enrollments e
JOIN students s ON e.student_id = s.id
JOIN courses c ON e.course_id = c.id
WHERE c.course_name = 'Calculus I'
ORDER BY s.last_name;
```

```
 first_name | last_name | grade
------------+-----------+-------
 Charlie    | Brown     | C+
 Ivy        | Clark     | B+
 Alice      | Johnson   | A
 Frank      | Wilson    | C
(4 rows)
```

The school database is now fully populated and ready for use in future chapters.

Common Mistakes

1. Forgetting the column list in INSERT. Always specify which columns you are inserting into. It makes your code readable and resistant to table changes.
2. Mixing up the order of values. If your column list says `(first_name, last_name)` but your values say `('Smith', 'Bob')`, you will get Bob's name backwards. Always double-check the order.

3. Inserting into columns with foreign keys that do not exist. If you try to insert `teacher_id = 99` but there is no teacher with id 99, PostgreSQL will reject the insert. Insert parent records first.
4. Confusing NULL and empty string. NULL means "unknown." An empty string means "intentionally blank." Use NULL for missing data and empty strings only when emptiness is a valid value.
5. Not using RETURNING. When you need the auto-generated ID of a new row, use `RETURNING id` instead of running a separate SELECT query. It is faster and safer.
6. Using individual INSERTs for bulk data. When inserting more than a few rows, use multi-row INSERT or COPY. Individual INSERTs are slow for large datasets.

Best Practices

1. Always use a column list. Never rely on the default column order.
2. Use multi-row INSERT for batches. It is faster and more readable than multiple individual INSERTs.
3. Use RETURNING when you need the result. Especially useful for getting auto-generated IDs.
4. Use ON CONFLICT for upserts. It handles the "insert or update" pattern cleanly in a single statement.
5. Use COPY for large data loads. It is orders of magnitude faster than INSERT for thousands or millions of rows.
6. Insert parent records before child records. If courses reference teachers, insert teachers first.
7. Use transactions for related inserts. When inserting related data (like a student and their enrollments), wrap everything in a transaction to ensure consistency. We will cover transactions in a later chapter.
8. Validate your data after insertion. Run a quick SELECT to make sure the data looks right, especially after bulk inserts.

Quick Summary

INSERT INTO adds rows to a table. Always specify a column list for clarity and safety. Use multi-row INSERT to add several rows at once. The RETURNING clause gives you back the

inserted data, including auto-generated IDs. INSERT ... SELECT copies data between tables. ON CONFLICT handles upserts (insert or update on duplicate). COPY loads data from files and is the fastest method for bulk imports. NULL means "unknown" and is different from an empty string.

Key Points

- `INSERT INTO table (columns) VALUES (values);` adds a new row.
- Always include a column list in your INSERT statements.
- Use multi-row INSERT for efficiency: `VALUES (row1), (row2), (row3);`
- DEFAULT lets PostgreSQL fill in a value automatically.
- RETURNING gives back the inserted data (great for getting auto-generated IDs).
- INSERT ... SELECT copies data from one table to another.
- ON CONFLICT DO UPDATE performs an upsert (insert or update).
- ON CONFLICT DO NOTHING silently skips duplicate inserts.
- COPY loads data from CSV files and is much faster than INSERT for large datasets.
- NULL means "unknown." Empty string means "intentionally empty." They are different.
- Insert parent records (teachers, courses) before child records (enrollments).

Practice Questions

1. What is the difference between `INSERT INTO students VALUES (...)` and `INSERT INTO students (first_name, last_name) VALUES (...)`? Which is better and why?
2. You insert a student and need to know the auto-generated ID immediately. How do you do this in PostgreSQL without running a separate query?
3. Explain what `ON CONFLICT DO NOTHING` does. Give a scenario where it would be useful.
4. What is the difference between NULL and an empty string (')? Write a query that finds all rows where the `phone` column is NULL.
5. You need to load 500,000 rows of product data from a CSV file into a PostgreSQL table. What command would you use, and why?

Exercises

Exercise 1: Build a Bookstore Database

Create a `bookstore` database with these tables and populate them with data:

1. authors table: id, name, country (at least 5 authors).
2. books table: id, title, isbn, price, publication_year, author_id (at least 10 books).
3. customers table: id, name, email, joined_date (at least 5 customers).

Use multi-row INSERT for efficiency. Use RETURNING to verify the IDs.

Exercise 2: Practice ON CONFLICT

1. Create a `user_settings` table with columns: user_email (PRIMARY KEY), theme (VARCHAR), font_size (INTEGER).
2. Insert a row for 'alice@email.com' with theme 'dark' and font_size 14.
3. Write an ON CONFLICT statement that updates the theme to 'light' if a row with the same email already exists.
4. Run the ON CONFLICT statement and verify the theme changed.

Exercise 3: NULL vs Empty String

1. Create a `survey_responses` table with: id (SERIAL PK), respondent_name (VARCHAR), answer (TEXT).
2. Insert three rows: one with a real answer, one with NULL, and one with an empty string.
3. Write a query that finds only the rows where the answer is NULL.
4. Write a query that finds only the rows where the answer is an empty string.
5. Write a query that finds rows where the answer is either NULL or an empty string.

What Is Next?

Your tables are now filled with data. It is time to start asking questions. In Chapter 7, you will learn how to query data with SELECT. You will master selecting specific columns, using aliases, removing duplicates, and combining SELECT with arithmetic expressions. The real power of SQL is in reading data, and that is where we are headed next.

Chapter 7: SELECT Queries — Retrieving Data from Your Database

What You Will Learn

In this chapter, you will learn how to:

- Retrieve data from a table using the SELECT statement
- Choose between selecting all columns and selecting specific columns
- Create column aliases with the AS keyword
- Perform arithmetic calculations directly in your queries
- Combine text columns using concatenation
- Remove duplicate rows with DISTINCT
- Write expressions in your SELECT list

Why This Chapter Matters

Imagine you have a filing cabinet full of employee records. Every time your boss asks a question — "Who works in Marketing?" or "What is the total salary budget?" — you need to pull out the right folders and find the right information.

The SELECT statement is your way of asking questions to the database. It is the single most important SQL command you will ever learn. Almost every interaction with a database starts with SELECT. Whether you are building a website, generating a report, or analyzing data, SELECT is the tool you reach for first.

Learning to write good SELECT queries is like learning to ask good questions. The more precise your question, the more useful the answer.

Setting Up Our Practice Table

Before we start writing queries, let us create a table we will use throughout this chapter and the next several chapters. Think of this as setting up a spreadsheet with employee information.

```sql
CREATE TABLE employees (
    id          SERIAL PRIMARY KEY,
    name        VARCHAR(100) NOT NULL,
    department  VARCHAR(50)  NOT NULL,
    salary      NUMERIC(10,2) NOT NULL,
    hire_date   DATE         NOT NULL
);
```

Now let us fill it with some sample data:

```sql
INSERT INTO employees (name, department, salary, hire_date) VALUES
    ('Alice Johnson',   'Engineering',  85000.00, '2020-03-15'),
    ('Bob Smith',       'Marketing',    62000.00, '2019-07-22'),
    ('Carol Williams',  'Engineering',  92000.00, '2018-11-01'),
    ('David Brown',     'Sales',        58000.00, '2021-01-10'),
    ('Eva Martinez',    'Marketing',    65000.00, '2020-09-05'),
    ('Frank Lee',       'Engineering',  78000.00, '2022-02-14'),
    ('Grace Kim',       'Sales',        61000.00, '2019-04-30'),
    ('Henry Wilson',    'HR',           55000.00, '2021-06-18'),
    ('Ivy Chen',        'HR',           59000.00, '2020-12-01'),
    ('Jack Davis',      'Sales',        63000.00, '2023-03-20');
```

Our table now looks like this:

```
+----+------------------+-------------+-----------+------------+
| id | name             | department  | salary    | hire_date  |
+----+------------------+-------------+-----------+------------+
|  1 | Alice Johnson    | Engineering | 85000.00  | 2020-03-15 |
|  2 | Bob Smith        | Marketing   | 62000.00  | 2019-07-22 |
|  3 | Carol Williams   | Engineering | 92000.00  | 2018-11-01 |
|  4 | David Brown      | Sales       | 58000.00  | 2021-01-10 |
|  5 | Eva Martinez     | Marketing   | 65000.00  | 2020-09-05 |
|  6 | Frank Lee        | Engineering | 78000.00  | 2022-02-14 |
|  7 | Grace Kim        | Sales       | 61000.00  | 2019-04-30 |
|  8 | Henry Wilson     | HR          | 55000.00  | 2021-06-18 |
|  9 | Ivy Chen         | HR          | 59000.00  | 2020-12-01 |
| 10 | Jack Davis       | Sales       | 63000.00  | 2023-03-20 |
+----+------------------+-------------+-----------+------------+
```

SELECT * — Getting Everything

The simplest way to retrieve data is to ask for everything:

```sql
SELECT * FROM employees;
```

Line-by-line breakdown:

- `SELECT` — This keyword tells PostgreSQL you want to retrieve data.
- `*` — The asterisk means "all columns." It is a wildcard, like saying "give me everything."
- `FROM employees` — This tells PostgreSQL which table to look in.

Result:

```
+----+------------------+-------------+-----------+------------+
| id | name             | department  | salary    | hire_date  |
+----+------------------+-------------+-----------+------------+
|  1 | Alice Johnson    | Engineering | 85000.00  | 2020-03-15 |
|  2 | Bob Smith        | Marketing   | 62000.00  | 2019-07-22 |
|  3 | Carol Williams   | Engineering | 92000.00  | 2018-11-01 |
|  4 | David Brown      | Sales       | 58000.00  | 2021-01-10 |
|  5 | Eva Martinez     | Marketing   | 65000.00  | 2020-09-05 |
|  6 | Frank Lee        | Engineering | 78000.00  | 2022-02-14 |
|  7 | Grace Kim        | Sales       | 61000.00  | 2019-04-30 |
|  8 | Henry Wilson     | HR          | 55000.00  | 2021-06-18 |
|  9 | Ivy Chen         | HR          | 59000.00  | 2020-12-01 |
| 10 | Jack Davis       | Sales       | 63000.00  | 2023-03-20 |
+----+------------------+-------------+-----------+------------+
(10 rows)
```

Think of `SELECT *` like telling a librarian, "Show me every book on that shelf." You get everything, whether you need it or not.

When SELECT * Is Okay

- Exploring a new table — You just found a table and want to see what is in it.
- Small tables — The table has only a few columns and rows.
- Quick debugging — You are troubleshooting a problem and need to see all the data.

When SELECT * Is a Bad Idea

- Large tables — If a table has 50 columns and millions of rows, `SELECT *` wastes time and memory.

- Production applications — Your web app only needs the employee name and department, but `SELECT *` fetches salary, hire date, and everything else too.
- When the table might change — If someone adds a new column later, your `SELECT *` query suddenly returns extra data your application was not expecting.

Selecting Specific Columns

Instead of grabbing everything, you can ask for exactly what you need:

```sql
SELECT name, department FROM employees;
```

Line-by-line breakdown:

- `SELECT name, department` — We list the exact columns we want, separated by commas.
- `FROM employees` — The table to look in.

Result:

```
+------------------+-------------+
| name             | department  |
+------------------+-------------+
| Alice Johnson    | Engineering |
| Bob Smith        | Marketing   |
| Carol Williams   | Engineering |
| David Brown      | Sales       |
| Eva Martinez     | Marketing   |
| Frank Lee        | Engineering |
| Grace Kim        | Sales       |
| Henry Wilson     | HR          |
| Ivy Chen         | HR          |
| Jack Davis       | Sales       |
+------------------+-------------+
(10 rows)
```

This is like telling the librarian, "I only need the title and author of each book." You get exactly what you asked for, nothing more.

You can list columns in any order:

```sql
SELECT department, name, salary FROM employees;
```

Result:

```
+-------------+------------------+-----------+
| department  | name             | salary    |
+-------------+------------------+-----------+
| Engineering | Alice Johnson    | 85000.00  |
| Marketing   | Bob Smith        | 62000.00  |
| Engineering | Carol Williams   | 92000.00  |
| Sales       | David Brown      | 58000.00  |
| Marketing   | Eva Martinez     | 65000.00  |
| Engineering | Frank Lee        | 78000.00  |
| Sales       | Grace Kim        | 61000.00  |
| HR          | Henry Wilson     | 55000.00  |
| HR          | Ivy Chen         | 59000.00  |
| Sales       | Jack Davis       | 63000.00  |
+-------------+------------------+-----------+
(10 rows)
```

Notice that the columns appear in the order you listed them, not the order they were created in the table.

Column Aliases with AS

Sometimes column names in the database are not very readable. Maybe a column is called `dept` or `emp_nm` or something technical. Aliases let you rename columns in the output without changing anything in the database.

```sql
SELECT
    name        AS employee_name,
    department  AS dept,
    salary      AS annual_salary
FROM employees;
```

Line-by-line breakdown:

- `name AS employee_name` — Show the `name` column, but label it as `employee_name` in the results.
- `department AS dept` — Show `department` but call it `dept` in the output.
- `salary AS annual_salary` — Show `salary` labeled as `annual_salary`.

Result:

```
+------------------+-------------+---------------+
| employee_name    | dept        | annual_salary |
```

```
+------------------+-------------+---------------+
| Alice Johnson    | Engineering | 85000.00      |
| Bob Smith        | Marketing   | 62000.00      |
| Carol Williams   | Engineering | 92000.00      |
| David Brown      | Sales       | 58000.00      |
| Eva Martinez     | Marketing   | 65000.00      |
| Frank Lee        | Engineering | 78000.00      |
| Grace Kim        | Sales       | 61000.00      |
| Henry Wilson     | HR          | 55000.00      |
| Ivy Chen         | HR          | 59000.00      |
| Jack Davis       | Sales       | 63000.00      |
+------------------+-------------+---------------+
(10 rows)
```

Think of aliases like name tags at a conference. Your real name might be "Robert," but your name tag says "Bob." The alias does not change who you are — it just changes how you are introduced.

Aliases with Spaces

If you want an alias with spaces in it, use double quotes:

```sql
SELECT
    name        AS "Employee Name",
    salary      AS "Annual Salary"
FROM employees;
```

Result:

```
+------------------+---------------+
| Employee Name    | Annual Salary |
+------------------+---------------+
| Alice Johnson    | 85000.00      |
| Bob Smith        | 62000.00      |
| Carol Williams   | 92000.00      |
| ...              | ...           |
+------------------+---------------+
```

> Note: The `AS` keyword is actually optional in PostgreSQL. You can write `SELECT name employee_name` instead of `SELECT name AS employee_name`. However, using `AS` makes your code much easier to read, so always include it.

Column Arithmetic — Calculating in Your Queries

SQL can do math right inside your queries. You do not need to pull data into a spreadsheet to calculate things.

Basic Arithmetic Operators

```
+----------+----------------+
| Operator | Meaning        |
+----------+----------------+
| +        | Addition       |
| -        | Subtraction    |
| *        | Multiplication |
| /        | Division       |
| %        | Modulo         |
+----------+----------------+
```

Monthly Salary

```sql
SELECT
    name,
    salary                  AS annual_salary,
    salary / 12             AS monthly_salary
FROM employees;
```

Line-by-line breakdown:

- `salary / 12` — PostgreSQL divides each employee's salary by 12 and shows the result.
- `AS monthly_salary` — We give the calculated column a meaningful name.

Result:

```
+------------------+---------------+------------------+
| name             | annual_salary | monthly_salary   |
+------------------+---------------+------------------+
| Alice Johnson    | 85000.00      | 7083.333333      |
| Bob Smith        | 62000.00      | 5166.666667      |
| Carol Williams   | 92000.00      | 7666.666667      |
| David Brown      | 58000.00      | 4833.333333      |
| Eva Martinez     | 65000.00      | 5416.666667      |
| Frank Lee        | 78000.00      | 6500.000000      |
| Grace Kim        | 61000.00      | 5083.333333      |
| Henry Wilson     | 55000.00      | 4583.333333      |
| Ivy Chen         | 59000.00      | 4916.666667      |
| Jack Davis       | 63000.00      | 5250.000000      |
```

```
+------------------+---------------+------------------+
(10 rows)
```

Salary After a 10% Raise

```sql
SELECT
    name,
    salary              AS current_salary,
    salary * 1.10       AS salary_after_raise,
    salary * 0.10       AS raise_amount
FROM employees;
```

Result:

```
+------------------+----------------+---------------------+--------------+
| name             | current_salary | salary_after_raise  | raise_amount |
+------------------+----------------+---------------------+--------------+
| Alice Johnson    | 85000.00       | 93500.0000          | 8500.0000    |
| Bob Smith        | 62000.00       | 68200.0000          | 6200.0000    |
| Carol Williams   | 92000.00       | 101200.0000         | 9200.0000    |
| David Brown      | 58000.00       | 63800.0000          | 5800.0000    |
| Eva Martinez     | 65000.00       | 71500.0000          | 6500.0000    |
| Frank Lee        | 78000.00       | 85800.0000          | 7800.0000    |
| Grace Kim        | 61000.00       | 67100.0000          | 6100.0000    |
| Henry Wilson     | 55000.00       | 60500.0000          | 5500.0000    |
| Ivy Chen         | 59000.00       | 64900.0000          | 5900.0000    |
| Jack Davis       | 63000.00       | 69300.0000          | 6300.0000    |
+------------------+----------------+---------------------+--------------+
(10 rows)
```

> Important: These calculations do NOT change the data in your table. They only affect what you see in the results. The actual salary values remain the same. Think of it like looking at a price tag through a magnifying glass — the price does not actually change.

Rounding Results

Those long decimal numbers are not pretty. Use the ROUND function to clean them up:

```sql
SELECT
    name,
    salary                          AS annual_salary,
    ROUND(salary / 12, 2)           AS monthly_salary
FROM employees;
```

Result:

```
+------------------+---------------+----------------+
| name             | annual_salary | monthly_salary |
+------------------+---------------+----------------+
| Alice Johnson    | 85000.00      | 7083.33        |
| Bob Smith        | 62000.00      | 5166.67        |
| Carol Williams   | 92000.00      | 7666.67        |
| David Brown      | 58000.00      | 4833.33        |
| Eva Martinez     | 65000.00      | 5416.67        |
| Frank Lee        | 78000.00      | 6500.00        |
| Grace Kim        | 61000.00      | 5083.33        |
| Henry Wilson     | 55000.00      | 4583.33        |
| Ivy Chen         | 59000.00      | 4916.67        |
| Jack Davis       | 63000.00      | 5250.00        |
+------------------+---------------+----------------+
(10 rows)
```

The `ROUND(value, 2)` function rounds the number to 2 decimal places.

String Concatenation with ||

In PostgreSQL, the `||` operator joins (concatenates) text strings together. Think of it like gluing words together.

```sql
SELECT
    name || ' works in ' || department AS employee_info
FROM employees;
```

Line-by-line breakdown:

- `name` — The employee's name (text).
- `||` — The concatenation operator. It glues the next piece of text onto the previous one.
- `' works in '` — A literal string (notice the spaces inside the quotes).
- `||` — Glue again.
- `department` — The department name.
- `AS employee_info` — Give the resulting column a name.

Result:

```
+-----------------------------------------+
| employee_info                           |
+-----------------------------------------+
| Alice Johnson works in Engineering      |
| Bob Smith works in Marketing            |
| Carol Williams works in Engineering     |
| David Brown works in Sales              |
| Eva Martinez works in Marketing         |
| Frank Lee works in Engineering          |
| Grace Kim works in Sales                |
| Henry Wilson works in HR                |
| Ivy Chen works in HR                    |
| Jack Davis works in Sales               |
+-----------------------------------------+
(10 rows)
```

Building Formatted Output

You can build complex strings:

```sql
SELECT
    'Employee #' || id || ': ' || name AS employee_label
FROM employees;
```

Result:

```
+-----------------------------------+
| employee_label                    |
+-----------------------------------+
| Employee #1: Alice Johnson        |
| Employee #2: Bob Smith            |
| Employee #3: Carol Williams       |
| Employee #4: David Brown          |
| Employee #5: Eva Martinez         |
| Employee #6: Frank Lee            |
| Employee #7: Grace Kim            |
| Employee #8: Henry Wilson         |
| Employee #9: Ivy Chen             |
| Employee #10: Jack Davis          |
+-----------------------------------+
(10 rows)
```

Notice that PostgreSQL automatically converts the `id` (a number) to text when you concatenate it with strings.

DISTINCT — Removing Duplicates

Sometimes you want to know what unique values exist in a column. DISTINCT removes duplicate rows from your results.

Without DISTINCT

```sql
SELECT department FROM employees;
```

Result:

```
+--------------+
| department   |
+--------------+
| Engineering  |
| Marketing    |
| Engineering  |
| Sales        |
| Marketing    |
| Engineering  |
| Sales        |
| HR           |
| HR           |
| Sales        |
+--------------+
(10 rows)
```

You see "Engineering" three times, "Sales" three times, and so on.

With DISTINCT

```sql
SELECT DISTINCT department FROM employees;
```

Result:

```
+--------------+
| department   |
+--------------+
| Engineering  |
| HR           |
| Marketing    |
| Sales        |
+--------------+
```

```
(4 rows)
```

Now each department appears only once. Think of DISTINCT like removing duplicate cards from a deck — if you have three Kings of Hearts, DISTINCT keeps just one.

DISTINCT with Multiple Columns

When you use DISTINCT with multiple columns, it removes rows where the combination of all listed columns is the same:

```sql
SELECT DISTINCT department, salary FROM employees;
```

Result:

```
+-------------+-----------+
| department  | salary    |
+-------------+-----------+
| Engineering | 78000.00  |
| Engineering | 85000.00  |
| Engineering | 92000.00  |
| HR          | 55000.00  |
| HR          | 59000.00  |
| Marketing   | 62000.00  |
| Marketing   | 65000.00  |
| Sales       | 58000.00  |
| Sales       | 61000.00  |
| Sales       | 63000.00  |
+-------------+-----------+
(10 rows)
```

All 10 rows appear because every combination of department and salary is unique in our data.

Expressions in SELECT

You can use SELECT without a FROM clause to evaluate expressions, almost like a calculator:

```sql
SELECT 2 + 3 AS result;
```

Result:

```
+--------+
| result |
+--------+
|      5 |
+--------+
```

```sql
SELECT
    100 * 0.15      AS fifteen_percent,
    'Hello' || ' ' || 'World' AS greeting,
    CURRENT_DATE    AS today;
```

Result:

```
+-----------------+-------------+------------+
| fifteen_percent | greeting    | today      |
+-----------------+-------------+------------+
|           15.00 | Hello World | 2024-01-15 |
+-----------------+-------------+------------+
```

This is handy for quick calculations or testing expressions before using them in bigger queries.

How SELECT Works — A Mental Model

Here is how to think about what happens when you run a SELECT query:

```
                    +-------------------+
                    |   employees       |
                    |   (full table)    |
                    +--------+----------+
                             |
                    Step 1: FROM
                    "Open the filing cabinet"
                             |
                             v
                    +-------------------+
                    |  All rows from    |
                    |  employees table  |
                    +--------+----------+
                             |
                    Step 2: SELECT
                    "Pick the columns
```

```
                      you want to see"
                              |
                              v
                    +-------------------+
                    |  Only the columns |
                    |  you asked for    |
                    +-------------------+
```

Even though you write SELECT first, PostgreSQL actually processes FROM first (to know where to look), then applies SELECT (to know what to show). We will add more steps to this mental model in later chapters.

Common Mistakes

Mistake 1: Forgetting the FROM Clause

```sql
-- Wrong: PostgreSQL does not know which table to look in
SELECT name, salary;

-- ERROR: column "name" does not exist

-- Right: Always specify the table
SELECT name, salary FROM employees;
```

Mistake 2: Misspelling Column Names

```sql
-- Wrong: The column is called "name", not "nme"
SELECT nme, department FROM employees;

-- ERROR: column "nme" does not exist

-- Right: Check your spelling
SELECT name, department FROM employees;
```

Mistake 3: Using Commas Incorrectly

```sql
-- Wrong: Extra comma after the last column
SELECT name, department, FROM employees;

-- ERROR: syntax error at or near "FROM"

-- Right: No trailing comma
SELECT name, department FROM employees;
```

Mistake 4: Confusing || with +

```sql
-- Wrong in PostgreSQL: + is for math, not string joining
SELECT name + department FROM employees;

-- ERROR: operator is not unique

-- Right: Use || for concatenation
SELECT name || ' - ' || department FROM employees;
```

Best Practices

1. Select only the columns you need. Avoid `SELECT *` in production code. It wastes resources and can break your application when table structures change.
2. Always use aliases for calculated columns. Without an alias, PostgreSQL generates an ugly name like `?column?` for expressions.
3. Use meaningful alias names. `AS x` tells you nothing. `AS monthly_salary` is clear and self-documenting.
4. Format your SQL for readability. Put each column on its own line for complex queries:

```sql
SELECT
    name,
    department,
    salary,
    ROUND(salary / 12, 2) AS monthly_salary
FROM employees;
```

5. Use DISTINCT sparingly. If you find yourself using DISTINCT to fix duplicate results, the real problem might be in your query logic (especially once you learn JOINs).

Quick Summary

```
+-------------------+-----------------------------------------+
| Concept           | Example                                 |
+-------------------+-----------------------------------------+
| All columns       | SELECT * FROM employees;                |
| Specific columns  | SELECT name, salary FROM employees;     |
| Alias             | SELECT name AS employee_name ...         |
| Arithmetic        | SELECT salary * 1.10 AS new_salary ...  |
| Concatenation     | SELECT name || ' - ' || department ...  |
| Remove duplicates | SELECT DISTINCT department ...           |
| Expression only   | SELECT 2 + 3 AS result;                 |
+-------------------+-----------------------------------------+
```

Key Points

- `SELECT *` retrieves all columns. Use it for exploration, not production code.
- List specific column names to get only the data you need.
- The `AS` keyword creates aliases that rename columns in the output without changing the table.
- You can perform arithmetic (`+`, `-`, `*`, `/`) directly in SELECT.
- The `||` operator concatenates (joins) strings together in PostgreSQL.
- `DISTINCT` removes duplicate rows from results.
- Calculations in SELECT do not modify the actual data in the table.
- Always give calculated columns a meaningful alias.

Practice Questions

1. Write a query that selects only the `name` and `hire_date` columns from the employees table.
2. What is wrong with this query?

```sql
SELECT name salary FROM employees;
```

3. Write a query that shows each employee's name and their daily salary (assume 260 working days per year). Round the result to 2 decimal places and give it an appropriate alias.
4. Write a query that shows the unique departments in the employees table. How many rows does the result have?
5. What will this query produce?

```sql
SELECT DISTINCT department, name FROM employees;
```

Will it return fewer rows than `SELECT department, name FROM employees`? Why or why not?

Exercises

Exercise 1: Employee Directory

Write a query that produces output like this:

```
+-------------------------------------------+-------------+
| employee_listing                          | department  |
+-------------------------------------------+-------------+
| #1 - Alice Johnson                        | Engineering |
| #2 - Bob Smith                            | Marketing   |
| ...                                       | ...         |
+-------------------------------------------+-------------+
```

Use concatenation to build the `employee_listing` column.

Exercise 2: Compensation Report

Write a query that shows each employee's:

- Name
- Annual salary
- Monthly salary (rounded to 2 decimal places)
- Weekly salary (assume 52 weeks, rounded to 2 decimal places)
- Salary after a 15% raise (rounded to 2 decimal places)

Give all columns clear, readable aliases.

Exercise 3: Data Exploration

Imagine you just discovered the employees table and need to answer these questions:

1. How many rows does the table have? (Hint: `SELECT * FROM employees` and count, or peek ahead to Chapter 10 for a better way.)
2. What departments exist?
3. What is the salary range? (Find the lowest and highest by scanning the output.)

Write the queries you would use to explore this table.

What Is Next?

Now that you know how to retrieve and format data, the next chapter will teach you how to filter that data. Instead of getting all 10 employees every time, you will learn to ask questions like "Show me only the engineers" or "Who earns more than 70,000?" The WHERE clause is what turns SQL from a simple data viewer into a powerful query language.

Chapter 8: WHERE Clause — Filtering Your Data

What You Will Learn

In this chapter, you will learn how to:

- Filter rows using the WHERE clause
- Use comparison operators (=, <>, <, >, <=, >=)
- Combine conditions with AND, OR, and NOT
- Check for values within a list using IN
- Find values within a range using BETWEEN
- Search for text patterns with LIKE and ILIKE
- Handle missing data with IS NULL and IS NOT NULL
- Understand operator precedence when combining conditions

Why This Chapter Matters

In the previous chapter, you learned how to retrieve data — but you got all of it every time. Imagine going to a library and asking, "Show me every book you have." That is not very helpful when you are looking for mystery novels published after 2020.

The WHERE clause is your filter. It lets you say, "Only show me the rows that match these conditions." In the real world, nearly every query you write will have a WHERE clause. Whether you are looking up a customer by email, finding overdue invoices, or checking which products are low on stock, WHERE is the tool that narrows millions of rows down to exactly what you need.

Without WHERE, SQL would be like a search engine that always returns every web page on the internet.

Our Practice Table

We will continue using the employees table from Chapter 7:

```
+----+------------------+-------------+-----------+------------+
| id | name             | department  | salary    | hire_date  |
+----+------------------+-------------+-----------+------------+
|  1 | Alice Johnson    | Engineering | 85000.00  | 2020-03-15 |
|  2 | Bob Smith        | Marketing   | 62000.00  | 2019-07-22 |
|  3 | Carol Williams   | Engineering | 92000.00  | 2018-11-01 |
|  4 | David Brown      | Sales       | 58000.00  | 2021-01-10 |
|  5 | Eva Martinez     | Marketing   | 65000.00  | 2020-09-05 |
|  6 | Frank Lee        | Engineering | 78000.00  | 2022-02-14 |
|  7 | Grace Kim        | Sales       | 61000.00  | 2019-04-30 |
|  8 | Henry Wilson     | HR          | 55000.00  | 2021-06-18 |
|  9 | Ivy Chen         | HR          | 59000.00  | 2020-12-01 |
| 10 | Jack Davis       | Sales       | 63000.00  | 2023-03-20 |
+----+------------------+-------------+-----------+------------+
```

The WHERE Clause — Basic Syntax

The WHERE clause goes after FROM and before any ORDER BY:

```sql
SELECT columns
FROM table_name
WHERE condition;
```

Think of WHERE like a bouncer at a club. Every row in the table walks up to the door. The WHERE clause checks each row: "Do you meet the condition?" If yes, the row gets through. If no, it is turned away.

```
                 +-------------------+
                 |   employees       |
                 |   (all 10 rows)   |
                 +--------+----------+
                          |
                 WHERE department = 'Sales'
                 "Does this row match?"
                          |
```

```
                  +----+----+
                  |         |
                Yes         No
                  |         |
                  v         v
            +----------+ (discarded)
            | Matching |
            |   rows   |
            +----------+
```

Comparison Operators

These operators compare a column value to something:

```
+----------+------------------------+-----------------------------------+
| Operator | Meaning                | Example                           |
+----------+------------------------+-----------------------------------+
| =        | Equal to               | department = 'Sales'              |
| <> or != | Not equal to           | department <> 'HR'                |
| <        | Less than              | salary < 60000                    |
| >        | Greater than           | salary > 80000                    |
| <=       | Less than or equal     | salary <= 65000                   |
| >=       | Greater than or equal  | salary >= 70000                   |
+----------+------------------------+-----------------------------------+
```

Equal To (=)

```sql
SELECT name, department, salary
FROM employees
WHERE department = 'Engineering';
```

Line-by-line breakdown:

- `SELECT name, department, salary` — The columns we want to see.
- `FROM employees` — The table to search.
- `WHERE department = 'Engineering'` — Only include rows where the department column contains exactly 'Engineering'.

Result:

```
+------------------+-------------+-----------+
| name             | department  | salary    |
+------------------+-------------+-----------+
| Alice Johnson    | Engineering | 85000.00  |
| Carol Williams   | Engineering | 92000.00  |
| Frank Lee        | Engineering | 78000.00  |
+------------------+-------------+-----------+
(3 rows)
```

> Important: Text values in SQL must be wrapped in single quotes: `'Engineering'`, not `"Engineering"`. Double quotes are for column names and aliases in PostgreSQL.

Not Equal To (<>)

```sql
SELECT name, department
FROM employees
WHERE department <> 'Sales';
```

Result:

```
+------------------+-------------+
| name             | department  |
+------------------+-------------+
| Alice Johnson    | Engineering |
| Bob Smith        | Marketing   |
| Carol Williams   | Engineering |
| Eva Martinez     | Marketing   |
| Frank Lee        | Engineering |
| Henry Wilson     | HR          |
| Ivy Chen         | HR          |
+------------------+-------------+
(7 rows)
```

Everyone except the Sales department. You can also use `!=` instead of `<>` — they mean the same thing. The `<>` syntax is the SQL standard, while `!=` is more common in programming languages.

Greater Than (>) and Less Than (<)

```sql
SELECT name, salary
FROM employees
WHERE salary > 80000;
```

Result:

```
+------------------+-----------+
| name             | salary    |
+------------------+-----------+
| Alice Johnson    | 85000.00  |
| Carol Williams   | 92000.00  |
+------------------+-----------+
(2 rows)
```

```sql
SELECT name, salary
FROM employees
WHERE salary < 60000;
```

Result:

```
+------------------+-----------+
| name             | salary    |
+------------------+-----------+
| David Brown      | 58000.00  |
| Henry Wilson     | 55000.00  |
| Ivy Chen         | 59000.00  |
+------------------+-----------+
(3 rows)
```

Comparing Dates

Dates work with comparison operators too. Think of dates as a timeline — "less than" means earlier, "greater than" means later:

```sql
SELECT name, hire_date
FROM employees
WHERE hire_date >= '2021-01-01';
```

Result:

```
+------------------+------------+
| name             | hire_date  |
+------------------+------------+
| David Brown      | 2021-01-10 |
| Frank Lee        | 2022-02-14 |
| Henry Wilson     | 2021-06-18 |
| Jack Davis       | 2023-03-20 |
+------------------+------------+
(4 rows)
```

Employees hired on or after January 1, 2021.

Combining Conditions: AND, OR, NOT

Real-world questions often have multiple parts: "Who works in Engineering AND earns more than 80,000?" The logical operators AND, OR, and NOT let you combine conditions.

AND — Both Conditions Must Be True

```sql
SELECT name, department, salary
FROM employees
WHERE department = 'Engineering'
  AND salary > 80000;
```

Line-by-line breakdown:

- `WHERE department = 'Engineering'` — First condition: must be in Engineering.
- `AND salary > 80000` — Second condition: must also earn more than 80,000.
- Both conditions must be true for a row to appear in the results.

Result:

```
+------------------+-------------+-----------+
| name             | department  | salary    |
+------------------+-------------+-----------+
| Alice Johnson    | Engineering | 85000.00  |
| Carol Williams   | Engineering | 92000.00  |
+------------------+-------------+-----------+
(2 rows)
```

Frank Lee is in Engineering but earns 78,000, so he does not pass the salary check.

Think of AND like a checklist at airport security. You need a valid ID AND a boarding pass. If you have only one, you do not get through.

OR — At Least One Condition Must Be True

```sql
SELECT name, department, salary
FROM employees
```

```
WHERE department = 'Sales'
   OR department = 'Marketing';
```

Result:

```
+------------------+-------------+-----------+
| name             | department  | salary    |
+------------------+-------------+-----------+
| Bob Smith        | Marketing   | 62000.00  |
| David Brown      | Sales       | 58000.00  |
| Eva Martinez     | Marketing   | 65000.00  |
| Grace Kim        | Sales       | 61000.00  |
| Jack Davis       | Sales       | 63000.00  |
+------------------+-------------+-----------+
(5 rows)
```

A row only needs to match one of the conditions. Think of OR like a restaurant menu: "Would you like coffee OR tea?" Either one works.

NOT — Reverse a Condition

```sql
SELECT name, department
FROM employees
WHERE NOT department = 'Engineering';
```

Result:

```
+------------------+-------------+
| name             | department  |
+------------------+-------------+
| Bob Smith        | Marketing   |
| David Brown      | Sales       |
| Eva Martinez     | Marketing   |
| Grace Kim        | Sales       |
| Henry Wilson     | HR          |
| Ivy Chen         | HR          |
| Jack Davis       | Sales       |
+------------------+-------------+
(7 rows)
```

NOT flips the condition. "Not Engineering" means everyone who is NOT in Engineering. This gives the same result as `department <> 'Engineering'`, but NOT becomes more powerful when combined with other operators like IN and BETWEEN.

IN — Checking Against a List

When you want to check if a value matches any item in a list, use IN instead of writing multiple OR conditions:

```sql
-- The long way with OR
SELECT name, department
FROM employees
WHERE department = 'Sales'
   OR department = 'Marketing'
   OR department = 'HR';

-- The short way with IN
SELECT name, department
FROM employees
WHERE department IN ('Sales', 'Marketing', 'HR');
```

Both queries produce the same result:

```
+------------------+-------------+
| name             | department  |
+------------------+-------------+
| Bob Smith        | Marketing   |
| David Brown      | Sales       |
| Eva Martinez     | Marketing   |
| Grace Kim        | Sales       |
| Henry Wilson     | HR          |
| Ivy Chen         | HR          |
| Jack Davis       | Sales       |
+------------------+-------------+
(7 rows)
```

IN is cleaner and easier to read, especially with long lists. Think of it like a guest list at a party: "Is your name IN the list? Come on in."

NOT IN — Excluding a List

```sql
SELECT name, department
FROM employees
WHERE department NOT IN ('Engineering', 'HR');
```

Result:

```
+------------------+-------------+
| name             | department  |
+------------------+-------------+
| Bob Smith        | Marketing   |
| David Brown      | Sales       |
| Eva Martinez     | Marketing   |
| Grace Kim        | Sales       |
| Jack Davis       | Sales       |
+------------------+-------------+
(5 rows)
```

BETWEEN — Range Checking

BETWEEN checks if a value falls within a range (inclusive on both ends):

```sql
SELECT name, salary
FROM employees
WHERE salary BETWEEN 60000 AND 75000;
```

Line-by-line breakdown:

- `BETWEEN 60000 AND 75000` — This includes 60,000 and 75,000 themselves. It is the same as writing `salary >= 60000 AND salary <= 75000`.

Result:

```
+------------------+-----------+
| name             | salary    |
+------------------+-----------+
| Bob Smith        | 62000.00  |
| Eva Martinez     | 65000.00  |
| Grace Kim        | 61000.00  |
| Jack Davis       | 63000.00  |
+------------------+-----------+
(4 rows)
```

BETWEEN with Dates

```sql
SELECT name, hire_date
FROM employees
WHERE hire_date BETWEEN '2020-01-01' AND '2021-12-31';
```

Result:

```
+------------------+------------+
| name             | hire_date  |
+------------------+------------+
| Alice Johnson    | 2020-03-15 |
| David Brown      | 2021-01-10 |
| Eva Martinez     | 2020-09-05 |
| Henry Wilson     | 2021-06-18 |
| Ivy Chen         | 2020-12-01 |
+------------------+------------+
(5 rows)
```

Employees hired during 2020 or 2021.

NOT BETWEEN

```sql
SELECT name, salary
FROM employees
WHERE salary NOT BETWEEN 60000 AND 80000;
```

Result:

```
+------------------+-----------+
| name             | salary    |
+------------------+-----------+
| Carol Williams   | 92000.00  |
| Alice Johnson    | 85000.00  |
| David Brown      | 58000.00  |
| Henry Wilson     | 55000.00  |
| Ivy Chen         | 59000.00  |
+------------------+-----------+
(5 rows)
```

LIKE and ILIKE — Pattern Matching

LIKE lets you search for text patterns. It uses two special wildcard characters:

```
+-----------+-------------------------------------------+
| Wildcard  | Meaning                                   |
+-----------+-------------------------------------------+
```

```
| %         | Matches any sequence of characters        |
|           | (including zero characters)                |
| _         | Matches exactly one character              |
+-----------+--------------------------------------------+
```

The % Wildcard — Any Number of Characters

```sql
-- Names that start with 'A'
SELECT name FROM employees
WHERE name LIKE 'A%';
```

Result:

```
+------------------+
| name             |
+------------------+
| Alice Johnson    |
+------------------+
(1 row)
```

```sql
-- Names that end with 'son'
SELECT name FROM employees
WHERE name LIKE '%son';
```

Result:

```
+------------------+
| name             |
+------------------+
| Alice Johnson    |
| Henry Wilson     |
+------------------+
(2 rows)
```

```sql
-- Names that contain 'ar'
SELECT name FROM employees
WHERE name LIKE '%ar%';
```

Result:

```
+------------------+
| name             |
+------------------+
| Carol Williams   |
| Eva Martinez     |
+------------------+
(2 rows)
```

Think of `%` like a wildcard in card games — it can stand for anything.

The _ Wildcard — Exactly One Character

```sql
-- Names where the second character is 'v'
SELECT name FROM employees
WHERE name LIKE '_v%';
```

Result:

```
+------------------+
| name             |
+------------------+
| Eva Martinez     |
| Ivy Chen         |
+------------------+
(2 rows)
```

The `_` matches exactly one character, then `v` must be the second character, then `%` matches the rest.

ILIKE — Case-Insensitive Pattern Matching

LIKE is case-sensitive. ILIKE is the PostgreSQL extension that ignores case:

```sql
-- LIKE is case-sensitive (this finds nothing because 'engineering' != 'Engineering')
SELECT name, department FROM employees
WHERE department LIKE 'engineering';
```

Result:

```
(0 rows)
```

```sql
-- ILIKE ignores case (this works!)
SELECT name, department FROM employees
WHERE department ILIKE 'engineering';
```

Result:

```
+------------------+-------------+
| name             | department  |
+------------------+-------------+
| Alice Johnson    | Engineering |
| Carol Williams   | Engineering |
| Frank Lee        | Engineering |
+------------------+-------------+
(3 rows)
```

> Note: ILIKE is a PostgreSQL-specific feature. In other databases like MySQL, LIKE is case-insensitive by default. In SQL Server, you would use `LIKE` with a case-insensitive collation.

Common LIKE Patterns

```
+------------------+----------------------------------------+
| Pattern          | Matches                                |
+------------------+----------------------------------------+
| 'A%'             | Starts with 'A'                        |
| '%son'           | Ends with 'son'                        |
| '%art%'          | Contains 'art' anywhere                |
| '_a%'            | Second character is 'a'                |
| '____'           | Exactly 4 characters long              |
| 'A%n'            | Starts with 'A' and ends with 'n'      |
+------------------+----------------------------------------+
```

IS NULL and IS NOT NULL

In SQL, NULL means "unknown" or "no value." It is not zero, not an empty string — it is the absence of a value. Think of it like a blank space on a form that was never filled in.

Let us add an employee with a missing department to demonstrate:

```sql
INSERT INTO employees (name, department, salary, hire_date)
VALUES ('Karen Null', 'Temp', 45000.00, '2024-01-15');

UPDATE employees SET department = NULL WHERE name = 'Karen Null';
```

Now here is the critical rule: you cannot use = or <> to check for NULL. You must use IS NULL or IS NOT NULL.

```sql
-- WRONG: This will NOT find Karen
SELECT name FROM employees
WHERE department = NULL;
-- Returns: (0 rows)  <-- Nothing! Even though Karen has NULL department

-- RIGHT: Use IS NULL
SELECT name, department FROM employees
WHERE department IS NULL;
```

Result:

```
+------------------+-------------+
| name             | department  |
+------------------+-------------+
| Karen Null       | NULL        |
+------------------+-------------+
(1 row)
```

Why does `= NULL` not work? Because NULL means "unknown." Is an unknown value equal to another unknown value? SQL says "I do not know" — which is not true, so the row is excluded. It is like asking, "Is the monster under the bed the same as the monster in the closet?" If you have never seen either one, you cannot say yes.

IS NOT NULL — Finding Rows That Have Values

```sql
SELECT name, department FROM employees
WHERE department IS NOT NULL;
```

This returns all employees who have a department assigned (all 10 original employees, excluding Karen).

Let us clean up our test data:

```sql
DELETE FROM employees WHERE name = 'Karen Null';
```

Operator Precedence — Order of Operations

When you combine AND, OR, and NOT, SQL follows a specific order, just like math follows "multiplication before addition":

```
+----------+--------------------------+
| Priority | Operator                 |
+----------+--------------------------+
| 1 (high) | NOT                      |
| 2        | AND                      |
| 3 (low)  | OR                       |
+----------+--------------------------+
```

This means AND is evaluated before OR. This can cause surprising results:

The Trap

```sql
-- What does this return?
SELECT name, department, salary
FROM employees
WHERE department = 'Sales'
   OR department = 'Marketing'
  AND salary > 63000;
```

You might expect: "Sales or Marketing employees who earn more than 63,000."

But because AND is evaluated first, PostgreSQL reads it as:

```
department = 'Sales'
OR
(department = 'Marketing' AND salary > 63000)
```

Result:

```
+------------------+-------------+-----------+
| name             | department  | salary    |
+------------------+-------------+-----------+
| David Brown      | Sales       | 58000.00  |
| Eva Martinez     | Marketing   | 65000.00  |
| Grace Kim        | Sales       | 61000.00  |
| Jack Davis       | Sales       | 63000.00  |
```

```
+------------------+-------------+-----------+
(4 rows)
```

ALL Sales employees appear (regardless of salary), plus only Marketing employees earning more than 63,000. David Brown from Sales at 58,000 got through even though you might not have intended that.

The Fix — Use Parentheses

Always use parentheses to make your intentions clear:

```sql
-- "Sales or Marketing employees" who earn more than 63,000
SELECT name, department, salary
FROM employees
WHERE (department = 'Sales' OR department = 'Marketing')
  AND salary > 63000;
```

Result:

```
+------------------+-------------+-----------+
| name             | department  | salary    |
+------------------+-------------+-----------+
| Eva Martinez     | Marketing   | 65000.00  |
+------------------+-------------+-----------+
(1 row)
```

Now the parentheses ensure OR is evaluated first, then AND. Only Eva qualifies — she is in Marketing and earns more than 63,000.

Combining Multiple Conditions — Practical Examples

Example 1: Complex Employee Search

"Find engineers hired after 2019 who earn between 75,000 and 90,000."

```sql
SELECT name, department, salary, hire_date
FROM employees
WHERE department = 'Engineering'
  AND hire_date > '2019-12-31'
  AND salary BETWEEN 75000 AND 90000;
```

Result:

```
+------------------+-------------+-----------+------------+
| name             | department  | salary    | hire_date  |
+------------------+-------------+-----------+------------+
| Alice Johnson    | Engineering | 85000.00  | 2020-03-15 |
| Frank Lee        | Engineering | 78000.00  | 2022-02-14 |
+------------------+-------------+-----------+------------+
(2 rows)
```

Example 2: Name Search

"Find employees whose last name starts with 'W' or 'D'."

```sql
SELECT name, department
FROM employees
WHERE name LIKE '%W%'
  AND (name LIKE '% W%' OR name LIKE '% D%');
```

A simpler approach:

```sql
SELECT name, department
FROM employees
WHERE name LIKE '% W%'
   OR name LIKE '% D%';
```

Result:

```
+------------------+-------------+
| name             | department  |
+------------------+-------------+
| Carol Williams   | Engineering |
| Henry Wilson     | HR          |
| David Brown      | Sales       |
| Jack Davis       | Sales       |
+------------------+-------------+
(4 rows)
```

Example 3: Exclusion Query

"Find everyone who is NOT in Engineering or HR and earns at least 60,000."

```sql
SELECT name, department, salary
FROM employees
WHERE department NOT IN ('Engineering', 'HR')
  AND salary >= 60000;
```

Result:

```
+------------------+-------------+-----------+
| name             | department  | salary    |
+------------------+-------------+-----------+
| Bob Smith        | Marketing   | 62000.00  |
| Eva Martinez     | Marketing   | 65000.00  |
| Grace Kim        | Sales       | 61000.00  |
| Jack Davis       | Sales       | 63000.00  |
+------------------+-------------+-----------+
(4 rows)
```

Updated Mental Model

Here is how SELECT works now with WHERE added:

```
                +-------------------+
                |   employees       |
                |   (all 10 rows)   |
                +--------+----------+
                         |
                Step 1: FROM
                "Open the filing cabinet"
                         |
                         v
                +-------------------+
                |  All rows from    |
                |  employees table  |
                +--------+----------+
                         |
                Step 2: WHERE
                "Check each row against
                 the conditions"
                         |
                         v
```

```
                +-------------------+
                |  Only rows that   |
                |  pass the filter  |
                +--------+----------+
                         |
                Step 3: SELECT
                "Pick the columns
                 you want to see"
                         |
                         v
                +-------------------+
                |  Final results    |
                +-------------------+
```

Common Mistakes

Mistake 1: Using = Instead of IS NULL

```sql
-- Wrong: This will always return 0 rows
SELECT * FROM employees WHERE department = NULL;

-- Right: Use IS NULL
SELECT * FROM employees WHERE department IS NULL;
```

Mistake 2: Forgetting Quotes Around Text

```sql
-- Wrong: PostgreSQL thinks Sales is a column name
SELECT * FROM employees WHERE department = Sales;
-- ERROR: column "sales" does not exist

-- Right: Text values need single quotes
SELECT * FROM employees WHERE department = 'Sales';
```

Mistake 3: Ignoring Operator Precedence

```sql
-- Probably wrong: AND binds tighter than OR
SELECT * FROM employees
WHERE department = 'Sales' OR department = 'HR' AND salary > 60000;
```

```
-- Right: Use parentheses to be explicit
SELECT * FROM employees
WHERE (department = 'Sales' OR department = 'HR') AND salary > 60000;
```

Mistake 4: Using LIKE When = Would Do

```sql
-- Works but unnecessary — no wildcards used
SELECT * FROM employees WHERE name LIKE 'Alice Johnson';

-- Better: Use = for exact matches (faster too)
SELECT * FROM employees WHERE name = 'Alice Johnson';
```

Mistake 5: Case Sensitivity with LIKE

```sql
-- Returns nothing because LIKE is case-sensitive
SELECT * FROM employees WHERE department LIKE 'sales';

-- Use ILIKE for case-insensitive matching
SELECT * FROM employees WHERE department ILIKE 'sales';
```

Best Practices

1. Always use parentheses when mixing AND and OR. Even if you know the precedence rules, parentheses make your code readable for others (and your future self).
2. Use IN instead of multiple ORs. `WHERE x IN (1, 2, 3)` is cleaner than `WHERE x = 1 OR x = 2 OR x = 3`.
3. Use BETWEEN for range checks. It is more readable than `x >= 10 AND x <= 20`.
4. Use IS NULL, never = NULL. This is the most common beginner mistake with SQL.
5. Use ILIKE when case does not matter. If users might type "sales", "Sales", or "SALES", use ILIKE to catch all variations.
6. Put the most selective condition first. While PostgreSQL's optimizer usually handles this, writing your most restrictive filter first makes the query easier to read.

Quick Summary

```
+-------------------+-------------------------------------------+
| Concept           | Example                                   |
+-------------------+-------------------------------------------+
| Equal             | WHERE name = 'Alice Johnson'              |
| Not equal         | WHERE department <> 'HR'                  |
| Greater than      | WHERE salary > 70000                      |
| Less than         | WHERE salary < 60000                      |
| AND               | WHERE dept = 'Sales' AND salary > 60000   |
| OR                | WHERE dept = 'Sales' OR dept = 'HR'       |
| NOT               | WHERE NOT department = 'Sales'            |
| IN                | WHERE dept IN ('Sales', 'HR')             |
| BETWEEN           | WHERE salary BETWEEN 50000 AND 80000      |
| LIKE              | WHERE name LIKE 'A%'                      |
| ILIKE             | WHERE name ILIKE 'alice%'                 |
| IS NULL           | WHERE department IS NULL                  |
| IS NOT NULL       | WHERE department IS NOT NULL              |
+-------------------+-------------------------------------------+
```

Key Points

- WHERE filters rows before they appear in results. It goes after FROM.
- Use `=` for exact matches and `<>` (or `!=`) for "not equal."
- AND requires all conditions to be true. OR requires at least one.
- IN checks if a value is in a list — cleaner than multiple ORs.
- BETWEEN checks inclusive ranges and works with numbers, dates, and text.
- LIKE uses `%` (any characters) and _ (one character) for pattern matching.
- ILIKE is the case-insensitive version of LIKE (PostgreSQL only).
- Always use IS NULL and IS NOT NULL to check for missing values, never `= NULL`.
- AND has higher precedence than OR. Use parentheses to be safe.

Practice Questions

1. Write a query to find all employees in the HR department.
2. Write a query to find employees who earn between 60,000 and 80,000 (inclusive).
3. What is wrong with this query?

```sql
SELECT * FROM employees WHERE salary = NULL;
```

4. Write a query to find employees whose names contain the letter 'a' (case-insensitive).
5. Explain what this query returns and why:

```sql
SELECT name, department, salary
FROM employees
WHERE department = 'Engineering'
   OR salary > 80000
  AND hire_date > '2020-01-01';
```

Exercises

Exercise 1: Employee Search Tool

Write queries to answer each of these questions:

1. Who works in Sales or Marketing?
2. Which engineers were hired before 2021?
3. Who earns less than the average of all salaries (you can calculate the average mentally or just pick a reasonable number for now)?

Exercise 2: Pattern Matching Detective

Using LIKE and ILIKE, write queries to find:

1. All employees whose first name starts with a vowel (A, E, I, O, U).
2. All employees whose last name is exactly 3 characters long.
3. All employees whose name contains "an" anywhere.

Exercise 3: Complex Conditions

Write a single query that finds employees who meet ALL of these criteria:

- Not in the Engineering department
- Hired in 2020 or later
- Earning between 55,000 and 65,000

What Is Next?

You can now filter data, but the results come back in whatever order PostgreSQL feels like. In the next chapter, you will learn how to sort your results with ORDER BY and limit how many rows you get back with LIMIT. Together with WHERE, these tools let you ask precise questions like "Show me the top 3 highest-paid engineers hired after 2020."

Chapter 9: Sorting and Limiting Results

What You Will Learn

In this chapter, you will learn how to:

- Sort results in ascending and descending order with ORDER BY
- Sort by multiple columns
- Control where NULL values appear with NULLS FIRST and NULLS LAST
- Limit the number of rows returned with LIMIT
- Skip rows with OFFSET for pagination
- Use FETCH FIRST as an alternative to LIMIT
- Combine WHERE, ORDER BY, and LIMIT for powerful top-N queries

Why This Chapter Matters

Imagine you search for "best restaurants near me" and Google gives you 10,000 results in random order. Not very helpful, right? You want them sorted by rating and you only want to see the top 10.

That is exactly what ORDER BY and LIMIT do for your database queries. ORDER BY puts your results in a logical order — alphabetically, by salary, by date, whatever makes sense. LIMIT controls how many rows you see. Together, they turn a flood of data into a neatly organized, manageable result.

These are the tools you will use every time you build a paginated list (like search results on a website), a leaderboard (top sellers, highest scores), or a report sorted by any criterion.

Our Practice Table

We continue with the employees table from previous chapters:

```
+----+------------------+-------------+-----------+------------+
| id | name             | department  | salary    | hire_date  |
+----+------------------+-------------+-----------+------------+
|  1 | Alice Johnson    | Engineering | 85000.00  | 2020-03-15 |
|  2 | Bob Smith        | Marketing   | 62000.00  | 2019-07-22 |
|  3 | Carol Williams   | Engineering | 92000.00  | 2018-11-01 |
|  4 | David Brown      | Sales       | 58000.00  | 2021-01-10 |
|  5 | Eva Martinez     | Marketing   | 65000.00  | 2020-09-05 |
|  6 | Frank Lee        | Engineering | 78000.00  | 2022-02-14 |
|  7 | Grace Kim        | Sales       | 61000.00  | 2019-04-30 |
|  8 | Henry Wilson     | HR          | 55000.00  | 2021-06-18 |
|  9 | Ivy Chen         | HR          | 59000.00  | 2020-12-01 |
| 10 | Jack Davis       | Sales       | 63000.00  | 2023-03-20 |
+----+------------------+-------------+-----------+------------+
```

ORDER BY — Sorting Results

Without ORDER BY, PostgreSQL returns rows in no guaranteed order. It might return them in the order they were inserted, or it might not. You should never rely on the default order.

Ascending Order (Default)

```sql
SELECT name, salary
FROM employees
ORDER BY salary;
```

Line-by-line breakdown:

- `SELECT name, salary` — The columns we want.
- `FROM employees` — The table.
- `ORDER BY salary` — Sort the results by the salary column. The default direction is ascending (smallest to largest).

Result:

```
+------------------+-----------+
| name             | salary    |
+------------------+-----------+
| Henry Wilson     | 55000.00  |
| David Brown      | 58000.00  |
| Ivy Chen         | 59000.00  |
| Grace Kim        | 61000.00  |
| Bob Smith        | 62000.00  |
| Jack Davis       | 63000.00  |
| Eva Martinez     | 65000.00  |
| Frank Lee        | 78000.00  |
| Alice Johnson    | 85000.00  |
| Carol Williams   | 92000.00  |
+------------------+-----------+
(10 rows)
```

You can also write `ORDER BY salary ASC` to be explicit. ASC stands for "ascending" — from low to high, from A to Z, from oldest to newest.

Descending Order

```sql
SELECT name, salary
FROM employees
ORDER BY salary DESC;
```

Line-by-line breakdown:

- `ORDER BY salary DESC` — DESC stands for "descending" — from high to low, from Z to A, from newest to oldest.

Result:

```
+------------------+-----------+
| name             | salary    |
+------------------+-----------+
| Carol Williams   | 92000.00  |
| Alice Johnson    | 85000.00  |
| Frank Lee        | 78000.00  |
| Eva Martinez     | 65000.00  |
| Jack Davis       | 63000.00  |
| Bob Smith        | 62000.00  |
| Grace Kim        | 61000.00  |
| Ivy Chen         | 59000.00  |
| David Brown      | 58000.00  |
| Henry Wilson     | 55000.00  |
+------------------+-----------+
(10 rows)
```

Think of ASC/DESC like an elevator. ASC goes up (1, 2, 3...). DESC goes down (10, 9, 8...).

Sorting Text Alphabetically

```sql
SELECT name, department
FROM employees
ORDER BY name;
```

Result:

```
+------------------+-------------+
| name             | department  |
+------------------+-------------+
| Alice Johnson    | Engineering |
| Bob Smith        | Marketing   |
| Carol Williams   | Engineering |
| David Brown      | Sales       |
| Eva Martinez     | Marketing   |
| Frank Lee        | Engineering |
| Grace Kim        | Sales       |
| Henry Wilson     | HR          |
| Ivy Chen         | HR          |
| Jack Davis       | Sales       |
+------------------+-------------+
(10 rows)
```

Text is sorted alphabetically. Uppercase and lowercase sorting depends on your database's collation settings, but in most PostgreSQL setups, the sort is case-insensitive alphabetical.

Sorting by Date

```sql
SELECT name, hire_date
FROM employees
ORDER BY hire_date;
```

Result:

```
+------------------+------------+
| name             | hire_date  |
+------------------+------------+
| Carol Williams   | 2018-11-01 |
| Grace Kim        | 2019-04-30 |
| Bob Smith        | 2019-07-22 |
| Alice Johnson    | 2020-03-15 |
| Eva Martinez     | 2020-09-05 |
```

```
| Ivy Chen          | 2020-12-01 |
| David Brown       | 2021-01-10 |
| Henry Wilson      | 2021-06-18 |
| Frank Lee         | 2022-02-14 |
| Jack Davis        | 2023-03-20 |
+-------------------+------------+
(10 rows)
```

Earliest hire date first. Use `DESC` to see the most recent hires first.

Sorting by Multiple Columns

What happens when two employees have the same value in the sort column? You can add a second (or third) column to break ties.

```sql
SELECT name, department, salary
FROM employees
ORDER BY department, salary DESC;
```

Line-by-line breakdown:

- `ORDER BY department` — First, sort by department alphabetically (ascending).
- `, salary DESC` — Within each department, sort by salary from highest to lowest.

Result:

```
+-------------------+-------------+-----------+
| name              | department  | salary    |
+-------------------+-------------+-----------+
| Carol Williams    | Engineering | 92000.00  |
| Alice Johnson     | Engineering | 85000.00  |
| Frank Lee         | Engineering | 78000.00  |
| Ivy Chen          | HR          | 59000.00  |
| Henry Wilson      | HR          | 55000.00  |
| Eva Martinez      | Marketing   | 65000.00  |
| Bob Smith         | Marketing   | 62000.00  |
| Jack Davis        | Sales       | 63000.00  |
| Grace Kim         | Sales       | 61000.00  |
| David Brown       | Sales       | 58000.00  |
+-------------------+-------------+-----------+
(10 rows)
```

Notice: departments are in alphabetical order (Engineering, HR, Marketing, Sales), and within each department, salaries go from highest to lowest.

Think of it like sorting a deck of cards: first sort by suit (Hearts, Diamonds, Clubs, Spades), then within each suit, sort by value (King, Queen, Jack, 10...).

```
Sort by department (ascending)
|
|   Then by salary (descending) within each department
|   |
v   v
Engineering  92000  (Carol)
Engineering  85000  (Alice)
Engineering  78000  (Frank)
HR           59000  (Ivy)
HR           55000  (Henry)
Marketing    65000  (Eva)
Marketing    62000  (Bob)
Sales        63000  (Jack)
Sales        61000  (Grace)
Sales        58000  (David)
```

Each column in the ORDER BY can have its own ASC or DESC direction.

NULLS FIRST and NULLS LAST

When a column contains NULL values, where should they appear in the sorted results? PostgreSQL gives you control:

- By default, NULLs come last in ascending order.
- By default, NULLs come first in descending order.

You can override this:

```sql
-- NULLs at the beginning
SELECT name, department
FROM employees
ORDER BY department NULLS FIRST;

-- NULLs at the end (explicit)
SELECT name, department
FROM employees
ORDER BY department NULLS LAST;
```

This is especially useful in real applications where missing data is common. For example, if you are sorting customers by their last purchase date, you might want customers who have never purchased (NULL date) to appear at the end:

```sql
SELECT customer_name, last_purchase_date
FROM customers
ORDER BY last_purchase_date DESC NULLS LAST;
```

This way, active customers appear first, and customers with no purchase history appear at the bottom.

LIMIT — Controlling How Many Rows You Get

LIMIT restricts the number of rows returned:

```sql
SELECT name, salary
FROM employees
ORDER BY salary DESC
LIMIT 3;
```

Line-by-line breakdown:

- `ORDER BY salary DESC` — Sort by salary, highest first.
- `LIMIT 3` — Only return the first 3 rows.

Result:

```
+------------------+-----------+
| name             | salary    |
+------------------+-----------+
| Carol Williams   | 92000.00  |
| Alice Johnson    | 85000.00  |
| Frank Lee        | 78000.00  |
+------------------+-----------+
(3 rows)
```

The top 3 earners. This is a classic "top-N query."

> Important: LIMIT without ORDER BY gives you an unpredictable set of rows. Always use ORDER BY with LIMIT to get meaningful results.

LIMIT for Quick Peeks

When exploring a large table, use LIMIT to get a quick sample without waiting for millions of rows:

```sql
SELECT * FROM employees LIMIT 5;
```

This is much faster than `SELECT *` on a table with millions of rows.

OFFSET — Skipping Rows

OFFSET tells PostgreSQL to skip a certain number of rows before starting to return results:

```sql
SELECT name, salary
FROM employees
ORDER BY salary DESC
LIMIT 3 OFFSET 3;
```

Line-by-line breakdown:

- `ORDER BY salary DESC` — Sort by salary, highest first.
- `LIMIT 3` — Return 3 rows.
- `OFFSET 3` — But first, skip the first 3 rows.

Result:

```
+------------------+-----------+
| name             | salary    |
+------------------+-----------+
| Eva Martinez     | 65000.00  |
| Jack Davis       | 63000.00  |
| Bob Smith        | 62000.00  |
+------------------+-----------+
(3 rows)
```

We skipped the top 3 (Carol, Alice, Frank) and got the next 3 (Eva, Jack, Bob).

Think of it like pages in a book. LIMIT is how many items per page. OFFSET is how many items to skip:

```
All employees sorted by salary (high to low):

OFFSET 0  --> Carol Williams   92000  --|
            Alice Johnson    85000  --|-- LIMIT 3 (Page 1)
            Frank Lee        78000  --|
OFFSET 3  --> Eva Martinez     65000  --|
            Jack Davis       63000  --|-- LIMIT 3 (Page 2)
            Bob Smith        62000  --|
OFFSET 6  --> Grace Kim        61000  --|
            Ivy Chen         59000  --|-- LIMIT 3 (Page 3)
            David Brown      58000  --|
OFFSET 9  --> Henry Wilson     55000  --|-- LIMIT 3 (Page 4, partial)
```

The Pagination Pattern

One of the most common uses of LIMIT and OFFSET is building paginated results — like search results on Google where you click "Next" to see more.

The formula is:

```
OFFSET = (page_number - 1) * items_per_page
```

```sql
-- Page 1: Show employees 1-3
SELECT name, department, salary
FROM employees
ORDER BY name
LIMIT 3 OFFSET 0;

-- Page 2: Show employees 4-6
SELECT name, department, salary
FROM employees
ORDER BY name
LIMIT 3 OFFSET 3;

-- Page 3: Show employees 7-9
SELECT name, department, salary
FROM employees
ORDER BY name
LIMIT 3 OFFSET 6;

-- Page 4: Show employees 10+
SELECT name, department, salary
FROM employees
ORDER BY name
LIMIT 3 OFFSET 9;
```

Page 1 Result:

```
+------------------+-------------+-----------+
| name             | department  | salary    |
+------------------+-------------+-----------+
| Alice Johnson    | Engineering | 85000.00  |
| Bob Smith        | Marketing   | 62000.00  |
| Carol Williams   | Engineering | 92000.00  |
+------------------+-------------+-----------+
(3 rows)
```

Page 2 Result:

```
+------------------+-------------+-----------+
| name             | department  | salary    |
+------------------+-------------+-----------+
| David Brown      | Sales       | 58000.00  |
| Eva Martinez     | Marketing   | 65000.00  |
| Frank Lee        | Engineering | 78000.00  |
+------------------+-------------+-----------+
(3 rows)
```

Performance Warning: OFFSET can be slow on large tables. To get page 100 with 10 items per page, PostgreSQL must read and discard the first 990 rows. For large datasets, consider cursor-based pagination (using WHERE id > last_seen_id) instead. We will cover this in later chapters.

FETCH FIRST — The SQL Standard Alternative

LIMIT is widely used but is technically a PostgreSQL (and MySQL) extension. The SQL standard way to limit rows is FETCH FIRST:

```sql
-- Using LIMIT (PostgreSQL style)
SELECT name, salary
FROM employees
ORDER BY salary DESC
LIMIT 5;

-- Using FETCH FIRST (SQL standard)
SELECT name, salary
FROM employees
```

```
ORDER BY salary DESC
FETCH FIRST 5 ROWS ONLY;
```

Both produce identical results:

```
+------------------+-----------+
| name             | salary    |
+------------------+-----------+
| Carol Williams   | 92000.00  |
| Alice Johnson    | 85000.00  |
| Frank Lee        | 78000.00  |
| Eva Martinez     | 65000.00  |
| Jack Davis       | 63000.00  |
+------------------+-----------+
(5 rows)
```

You can also write `FETCH FIRST 1 ROW ONLY` (singular) for a single row. PostgreSQL accepts both ROW and ROWS.

FETCH FIRST with OFFSET

```sql
SELECT name, salary
FROM employees
ORDER BY salary DESC
OFFSET 3 ROWS
FETCH FIRST 3 ROWS ONLY;
```

This skips 3 rows and returns the next 3, just like `LIMIT 3 OFFSET 3`.

> Which should you use? In PostgreSQL, most people use LIMIT because it is shorter. If you care about SQL portability across different databases, use FETCH FIRST.

Combining WHERE + ORDER BY + LIMIT

The real power comes from combining all three:

Top 3 Highest-Paid Non-Engineering Employees

```sql
SELECT name, department, salary
FROM employees
WHERE department <> 'Engineering'
ORDER BY salary DESC
LIMIT 3;
```

Line-by-line breakdown:

- `WHERE department <> 'Engineering'` — Filter out engineers.
- `ORDER BY salary DESC` — Sort remaining employees by salary, highest first.
- `LIMIT 3` — Take only the top 3.

Result:

```
+------------------+-------------+-----------+
| name             | department  | salary    |
+------------------+-------------+-----------+
| Eva Martinez     | Marketing   | 65000.00  |
| Jack Davis       | Sales       | 63000.00  |
| Bob Smith        | Marketing   | 62000.00  |
+------------------+-------------+-----------+
(3 rows)
```

Most Recent Hire in Each Department (Simple Version)

```sql
-- Most recently hired person in Sales
SELECT name, hire_date
FROM employees
WHERE department = 'Sales'
ORDER BY hire_date DESC
LIMIT 1;
```

Result:

```
+------------------+------------+
| name             | hire_date  |
+------------------+------------+
| Jack Davis       | 2023-03-20 |
+------------------+------------+
(1 row)
```

The 5 Lowest-Paid Employees Hired Since 2020

```sql
SELECT name, department, salary, hire_date
FROM employees
WHERE hire_date >= '2020-01-01'
ORDER BY salary ASC
LIMIT 5;
```

Result:

```
+------------------+-------------+-----------+------------+
| name             | department  | salary    | hire_date  |
+------------------+-------------+-----------+------------+
| Henry Wilson     | HR          | 55000.00  | 2021-06-18 |
| David Brown      | Sales       | 58000.00  | 2021-01-10 |
| Ivy Chen         | HR          | 59000.00  | 2020-12-01 |
| Jack Davis       | Sales       | 63000.00  | 2023-03-20 |
| Eva Martinez     | Marketing   | 65000.00  | 2020-09-05 |
+------------------+-------------+-----------+------------+
(5 rows)
```

The Order of Clauses

SQL requires clauses in a specific order:

```sql
SELECT columns          -- 1. What to show
FROM table              -- 2. Where to look
WHERE conditions        -- 3. Which rows to include
ORDER BY columns        -- 4. How to sort
LIMIT n OFFSET m;       -- 5. How many to return
```

You cannot rearrange these. ORDER BY must come after WHERE. LIMIT must come after ORDER BY.

Execution Order vs. Writing Order

Remember, the order PostgreSQL executes the clauses is different from the order you write them:

```
  Writing Order:          Execution Order:
  1. SELECT               1. FROM        (find the table)
  2. FROM                 2. WHERE       (filter rows)
  3. WHERE                3. SELECT      (pick columns)
```

```
  4. ORDER BY             4. ORDER BY    (sort results)
  5. LIMIT/OFFSET         5. LIMIT/OFFSET (cut to size)
```

This is why you can use column aliases in ORDER BY but not in WHERE:

```sql
-- This WORKS: ORDER BY can use aliases
SELECT name, salary * 12 AS annual_compensation
FROM employees
ORDER BY annual_compensation DESC;

-- This FAILS: WHERE cannot use aliases
SELECT name, salary * 12 AS annual_compensation
FROM employees
WHERE annual_compensation > 700000;
-- ERROR: column "annual_compensation" does not exist

-- This is the fix: repeat the expression in WHERE
SELECT name, salary * 12 AS annual_compensation
FROM employees
WHERE salary * 12 > 700000;
```

Top-N Queries — A Common Pattern

Top-N queries answer questions like "What are the top 5 most expensive products?" or "Who are the 3 newest employees?" They follow this pattern:

```sql
SELECT columns
FROM table
WHERE filters          -- optional
ORDER BY column DESC   -- or ASC, depending on what "top" means
LIMIT N;
```

The Single Highest-Paid Employee

```sql
SELECT name, salary
FROM employees
ORDER BY salary DESC
LIMIT 1;
```

Result:

```
+------------------+-----------+
| name             | salary    |
+------------------+-----------+
| Carol Williams   | 92000.00  |
+------------------+-----------+
(1 row)
```

The 3 Oldest Employees (by Hire Date)

```sql
SELECT name, hire_date, department
FROM employees
ORDER BY hire_date ASC
LIMIT 3;
```

Result:

```
+------------------+------------+-------------+
| name             | hire_date  | department  |
+------------------+------------+-------------+
| Carol Williams   | 2018-11-01 | Engineering |
| Grace Kim        | 2019-04-30 | Sales       |
| Bob Smith        | 2019-07-22 | Marketing   |
+------------------+------------+-------------+
(3 rows)
```

Bottom-N (The Lowest)

Just flip ASC and DESC:

```sql
-- 3 lowest-paid employees
SELECT name, salary
FROM employees
ORDER BY salary ASC
LIMIT 3;
```

Result:

```
+------------------+-----------+
| name             | salary    |
+------------------+-----------+
| Henry Wilson     | 55000.00  |
| David Brown      | 58000.00  |
| Ivy Chen         | 59000.00  |
```

```
+------------------+-----------+
(3 rows)
```

Common Mistakes

Mistake 1: Using LIMIT Without ORDER BY

```sql
-- Bad: Which 5 employees will you get? It is unpredictable!
SELECT name FROM employees LIMIT 5;

-- Good: Now you reliably get the first 5 alphabetically
SELECT name FROM employees ORDER BY name LIMIT 5;
```

Mistake 2: Wrong OFFSET Calculation for Pagination

```sql
-- Wrong: Page 2 with 5 items per page
-- OFFSET 5 is correct, not OFFSET 2
SELECT * FROM employees ORDER BY id LIMIT 5 OFFSET 2;  -- Wrong!
SELECT * FROM employees ORDER BY id LIMIT 5 OFFSET 5;  -- Right!

-- Formula: OFFSET = (page_number - 1) * items_per_page
-- Page 1: OFFSET 0
-- Page 2: OFFSET 5
-- Page 3: OFFSET 10
```

Mistake 3: Sorting by a Column Not in SELECT

This actually works in PostgreSQL — you can sort by a column even if it is not in your SELECT list:

```sql
-- This is valid! Sort by salary even though it is not displayed
SELECT name, department
FROM employees
ORDER BY salary DESC;
```

It is not a mistake, but it can be confusing to readers of your code. Consider including the sort column in your SELECT.

Mistake 4: Expecting Consistent Order Without ORDER BY

```sql
-- These two queries might return rows in different orders!
SELECT * FROM employees;  -- run now
SELECT * FROM employees;  -- run later, after updates

-- Always use ORDER BY if order matters to you
```

Best Practices

1. Always use ORDER BY when order matters. Never assume a default order. Tables are unordered sets.
2. Always pair LIMIT with ORDER BY. Without ORDER BY, LIMIT gives you arbitrary rows.
3. Be cautious with large OFFSET values. Paginating to page 1000 (OFFSET 9990) is slow. Consider keyset pagination for large datasets.
4. Use LIMIT for data exploration. When exploring a table for the first time, `SELECT * FROM big_table LIMIT 10` is much safer than `SELECT * FROM big_table`.
5. Include a tiebreaker column in ORDER BY. If two employees have the same salary, their order is unpredictable. Add a second column (like `id` or `name`) to ensure consistent ordering:

```sql
SELECT name, salary
FROM employees
ORDER BY salary DESC, name ASC;
```

Quick Summary

```
+-------------------+-------------------------------------------------+
| Concept           | Example                                         |
+-------------------+-------------------------------------------------+
| Sort ascending    | ORDER BY salary ASC                             |
| Sort descending   | ORDER BY salary DESC                            |
| Multiple sorts    | ORDER BY department, salary DESC                |
| NULLs position    | ORDER BY col NULLS FIRST                        |
| Limit rows        | LIMIT 10                                        |
```

```
| Skip rows         | LIMIT 10 OFFSET 20                              |
| SQL standard      | FETCH FIRST 10 ROWS ONLY                        |
| Top-N query       | ORDER BY salary DESC LIMIT 3                    |
| Pagination        | LIMIT size OFFSET (page - 1) * size              |
+-------------------+-------------------------------------------------+
```

Key Points

- ORDER BY sorts results. ASC (ascending) is the default; use DESC for descending.
- You can sort by multiple columns. The first column is the primary sort, the second breaks ties.
- NULLS FIRST and NULLS LAST control where NULL values appear in sorted results.
- LIMIT restricts the number of rows returned. Always pair it with ORDER BY.
- OFFSET skips rows before returning results. Useful for pagination.
- FETCH FIRST is the SQL standard equivalent of LIMIT.
- The pagination formula is: OFFSET = (page_number - 1) * items_per_page.
- Clauses must appear in order: SELECT, FROM, WHERE, ORDER BY, LIMIT/OFFSET.
- Column aliases can be used in ORDER BY but not in WHERE.

Practice Questions

1. Write a query to list all employees sorted by hire date, with the most recent hire first.
2. Write a query to find the 3 lowest-paid employees. Show their name, department, and salary.
3. How would you write a pagination query to show page 4 of results, with 3 employees per page, sorted by name?
4. What is wrong with this query?

```sql
SELECT name FROM employees LIMIT 5 ORDER BY name;
```

5. Explain the difference between these two queries:

```sql
-- Query A
SELECT name, salary FROM employees ORDER BY salary DESC LIMIT 1;

-- Query B
SELECT name, salary FROM employees ORDER BY salary ASC LIMIT 1;
```

Exercises

Exercise 1: Top Earners Report

Write queries to answer:

1. Who is the highest-paid employee in each department? (Write a separate query for each department using WHERE and LIMIT 1.)
2. List all employees sorted by department (A-Z), then by salary within each department (highest first).

Exercise 2: Pagination System

Imagine you are building a web page that shows 4 employees per page, sorted alphabetically by name. Write the queries for pages 1, 2, and 3.

Exercise 3: Combined Filtering and Sorting

Write a single query that:

1. Excludes employees from the HR department
2. Only includes employees earning more than 60,000
3. Sorts the results by hire date (newest first)
4. Returns only the top 3 results

What Is Next?

You can now filter, sort, and paginate your data. But what if you want to know the total salary of all employees, or the average salary per department? In the next chapter, you will learn aggregate functions — tools like COUNT, SUM, AVG, MIN, and MAX that let you calculate

summaries from your data instead of looking at individual rows.

Chapter 10: Aggregate Functions — Summarizing Your Data

What You Will Learn

In this chapter, you will learn how to:

- Count rows with COUNT(*), COUNT(column), and COUNT(DISTINCT)
- Add up values with SUM
- Calculate averages with AVG
- Find the smallest and largest values with MIN and MAX
- Combine multiple aggregate functions in a single query
- Understand how NULL values affect aggregates
- Use ROUND to clean up averages
- Write practical summary queries

Why This Chapter Matters

So far, every query you have written returns individual rows — one row per employee, one row per record. But what if your boss asks, "How many employees do we have?" or "What is our total salary budget?" You do not want a list of 10 (or 10,000) rows. You want a single number.

Aggregate functions collapse many rows into a single summary value. They are like the "total" row at the bottom of a spreadsheet. Instead of looking at every individual sale, you can instantly see the total revenue, the average order size, or the number of customers.

If SELECT is like reading a book word by word, aggregate functions are like reading the summary on the back cover. They give you the big picture.

Our Practice Table

We continue with the employees table:

```
+----+------------------+-------------+-----------+------------+
| id | name             | department  | salary    | hire_date  |
+----+------------------+-------------+-----------+------------+
|  1 | Alice Johnson    | Engineering | 85000.00  | 2020-03-15 |
|  2 | Bob Smith        | Marketing   | 62000.00  | 2019-07-22 |
|  3 | Carol Williams   | Engineering | 92000.00  | 2018-11-01 |
|  4 | David Brown      | Sales       | 58000.00  | 2021-01-10 |
|  5 | Eva Martinez     | Marketing   | 65000.00  | 2020-09-05 |
|  6 | Frank Lee        | Engineering | 78000.00  | 2022-02-14 |
|  7 | Grace Kim        | Sales       | 61000.00  | 2019-04-30 |
|  8 | Henry Wilson     | HR          | 55000.00  | 2021-06-18 |
|  9 | Ivy Chen         | HR          | 59000.00  | 2020-12-01 |
| 10 | Jack Davis       | Sales       | 63000.00  | 2023-03-20 |
+----+------------------+-------------+-----------+------------+
```

COUNT — How Many Rows?

COUNT is the simplest aggregate. It counts rows.

COUNT(*) — Count All Rows

```sql
SELECT COUNT(*) AS total_employees
FROM employees;
```

Line-by-line breakdown:

- `COUNT(*)` — Count every row in the table, regardless of what is in the columns.
- `AS total_employees` — Give the result a meaningful name.

Result:

```
+-----------------+
| total_employees |
+-----------------+
|              10 |
```

```
+-----------------+
(1 row)
```

Notice: instead of 10 individual rows, we get a single row with a single number. That is what aggregate functions do — they collapse many rows into one summary.

Think of COUNT(*) like a headcount at a meeting. You do not care who is there or what they do. You just want to know how many people are in the room.

COUNT(*) with WHERE

You can combine COUNT with WHERE to count only matching rows:

```sql
SELECT COUNT(*) AS engineering_count
FROM employees
WHERE department = 'Engineering';
```

Result:

```
+-------------------+
| engineering_count |
+-------------------+
|                 3 |
+-------------------+
(1 row)
```

Three engineers. The WHERE clause filters first, then COUNT counts what is left.

COUNT(column) — Count Non-NULL Values

COUNT(column_name) counts only the rows where that column is NOT NULL:

```sql
-- First, let us see what COUNT(column) does differently
-- Add a test row with a NULL salary for demonstration
-- (we will remove it after)

-- For now, on our current data with no NULLs:
SELECT
    COUNT(*)       AS total_rows,
    COUNT(salary)  AS rows_with_salary
FROM employees;
```

Result:

```
+------------+------------------+
| total_rows | rows_with_salary |
+------------+------------------+
|         10 |               10 |
+------------+------------------+
```

Since none of our salary values are NULL, both counts are 10. The difference matters when you have NULL values:

```
  COUNT(*)        -->  Counts ALL rows (including those with NULLs)
  COUNT(column)   -->  Counts only rows where column IS NOT NULL
```

Think of it this way: COUNT(*) asks "How many people showed up?" while COUNT(email) asks "How many people left their email address?"

COUNT(DISTINCT column) — Count Unique Values

```sql
SELECT COUNT(DISTINCT department) AS department_count
FROM employees;
```

Result:

```
+------------------+
| department_count |
+------------------+
|                4 |
+------------------+
(1 row)
```

There are 4 unique departments (Engineering, HR, Marketing, Sales), even though we have 10 employees. COUNT(DISTINCT) eliminates duplicates before counting.

```sql
-- Compare:
SELECT
    COUNT(department)          AS total_dept_values,
    COUNT(DISTINCT department) AS unique_departments
FROM employees;
```

Result:

```
+-------------------+--------------------+
| total_dept_values | unique_departments |
+-------------------+--------------------+
|                10 |                  4 |
+-------------------+--------------------+
```

SUM — Adding Up Values

SUM adds up all the values in a numeric column:

```sql
SELECT SUM(salary) AS total_salary_budget
FROM employees;
```

Result:

```
+----------------------+
| total_salary_budget |
+----------------------+
|           678000.00 |
+----------------------+
(1 row)
```

The company spends $678,000 on salaries. Think of SUM like the total on a receipt — it adds up all the individual items.

SUM with WHERE

```sql
SELECT SUM(salary) AS engineering_budget
FROM employees
WHERE department = 'Engineering';
```

Result:

```
+---------------------+
| engineering_budget |
+---------------------+
|          255000.00 |
+---------------------+
(1 row)
```

The Engineering department costs $255,000 in salaries (85,000 + 92,000 + 78,000).

> Note: SUM only works with numeric columns. You cannot SUM text or dates.

AVG — The Average

AVG calculates the arithmetic mean (add everything up, then divide by the count):

```sql
SELECT AVG(salary) AS average_salary
FROM employees;
```

Result:

```
+----------------------+
| average_salary       |
+----------------------+
| 67800.000000000000   |
+----------------------+
(1 row)
```

That is a lot of decimal places. Let us clean it up with ROUND:

```sql
SELECT ROUND(AVG(salary), 2) AS average_salary
FROM employees;
```

Result:

```
+----------------+
| average_salary |
+----------------+
|       67800.00 |
+----------------+
(1 row)
```

The ROUND function takes two arguments: the number to round and the number of decimal places.

Average for a Specific Group

```sql
SELECT ROUND(AVG(salary), 2) AS avg_sales_salary
FROM employees
WHERE department = 'Sales';
```

Result:

```
+------------------+
| avg_sales_salary |
+------------------+
|         60666.67 |
+------------------+
(1 row)
```

The average salary in Sales is $60,666.67 (58,000 + 61,000 + 63,000 = 182,000 / 3).

MIN and MAX — Finding Extremes

MIN finds the smallest value. MAX finds the largest.

```sql
SELECT
    MIN(salary) AS lowest_salary,
    MAX(salary) AS highest_salary
FROM employees;
```

Result:

```
+---------------+----------------+
| lowest_salary | highest_salary |
+---------------+----------------+
|      55000.00 |       92000.00 |
+---------------+----------------+
(1 row)
```

MIN and MAX with Dates

MIN and MAX work with dates too. MIN gives the earliest date, MAX gives the latest:

```sql
SELECT
    MIN(hire_date) AS earliest_hire,
    MAX(hire_date) AS latest_hire
FROM employees;
```

Result:

```
+---------------+-------------+
| earliest_hire | latest_hire |
+---------------+-------------+
| 2018-11-01    | 2023-03-20  |
+---------------+-------------+
(1 row)
```

Carol Williams was the first hired (2018), and Jack Davis was the most recent (2023).

MIN and MAX with Text

For text columns, MIN gives the first alphabetically and MAX gives the last:

```sql
SELECT
    MIN(name) AS first_alphabetically,
    MAX(name) AS last_alphabetically
FROM employees;
```

Result:

```
+----------------------+---------------------+
| first_alphabetically | last_alphabetically |
+----------------------+---------------------+
| Alice Johnson        | Jack Davis          |
+----------------------+---------------------+
```

Combining Multiple Aggregates

You can use multiple aggregate functions in a single query:

```sql
SELECT
    COUNT(*)              AS total_employees,
    SUM(salary)           AS total_salary,
    ROUND(AVG(salary), 2) AS average_salary,
    MIN(salary)           AS lowest_salary,
    MAX(salary)           AS highest_salary,
    MAX(salary) - MIN(salary) AS salary_range
FROM employees;
```

Result:

```
+-----------+--------------+----------------+---------------+----------------+---------...
| total_    | total_salary | average_salary | lowest_salary | highest_salary | salary_r...
| employees |              |                |               |                |         ...
+-----------+--------------+----------------+---------------+----------------+---------...
|        10 |    678000.00 |       67800.00 |      55000.00 |       92000.00 |     3700...
+-----------+--------------+----------------+---------------+----------------+---------...
(1 row)
```

One query gives you a complete salary summary. This is like getting the executive summary of a 100-page report in a single glance.

Department-Specific Summary

```sql
SELECT
    COUNT(*)              AS headcount,
    ROUND(AVG(salary), 2) AS avg_salary,
    MIN(salary)           AS min_salary,
    MAX(salary)           AS max_salary
FROM employees
WHERE department = 'Engineering';
```

Result:

```
+-----------+------------+------------+------------+
| headcount | avg_salary | min_salary | max_salary |
+-----------+------------+------------+------------+
|         3 |   85000.00 |   78000.00 |   92000.00 |
+-----------+------------+------------+------------+
(1 row)
```

How NULL Values Affect Aggregates

NULLs are a common source of confusion with aggregates. Here is how each function handles them:

```
+----------+--------------------------------------------+
| Function | How it handles NULL                         |
+----------+--------------------------------------------+
| COUNT(*) | Counts ALL rows, including those with NULLs |
| COUNT(x) | Ignores rows where x IS NULL                |
| SUM(x)   | Ignores NULL values                         |
| AVG(x)   | Ignores NULL values (affects the divisor!)  |
| MIN(x)   | Ignores NULL values                         |
| MAX(x)   | Ignores NULL values                         |
+----------+--------------------------------------------+
```

The most important thing to understand is how AVG handles NULLs:

```
  Example with 5 values: 10, 20, NULL, 30, NULL

  AVG ignores NULLs:
  AVG = (10 + 20 + 30) / 3 = 20.0

  NOT:
  AVG = (10 + 20 + 0 + 30 + 0) / 5 = 12.0

  NULL is ignored entirely — it does not count as zero,
  and it does not increase the divisor.
```

This is important because it means AVG gives you the average of known values, not the average of all possible values. In most cases this is what you want, but be aware of it.

SUM on All NULLs

If every value is NULL, SUM returns NULL (not zero):

```sql
SELECT SUM(NULL::numeric) AS result;
```

Result:

```
+--------+
| result |
+--------+
|  NULL  |
+--------+
```

Use COALESCE to convert NULL to zero if needed:

```sql
SELECT COALESCE(SUM(NULL::numeric), 0) AS result;
```

Result:

```
+--------+
| result |
+--------+
|      0 |
+--------+
```

Practical Queries

What Percentage of the Budget Goes to Each Department?

```sql
SELECT
    ROUND(
        SUM(CASE WHEN department = 'Engineering' THEN salary ELSE 0 END)
        / SUM(salary) * 100, 1
    ) AS engineering_pct,
    ROUND(
        SUM(CASE WHEN department = 'Sales' THEN salary ELSE 0 END)
        / SUM(salary) * 100, 1
    ) AS sales_pct,
    ROUND(
        SUM(CASE WHEN department = 'Marketing' THEN salary ELSE 0 END)
        / SUM(salary) * 100, 1
    ) AS marketing_pct,
    ROUND(
        SUM(CASE WHEN department = 'HR' THEN salary ELSE 0 END)
        / SUM(salary) * 100, 1
    ) AS hr_pct
FROM employees;
```

Result:

```
+----------------+-----------+---------------+--------+
| engineering_pct| sales_pct | marketing_pct | hr_pct |
+----------------+-----------+---------------+--------+
|           37.6 |      26.8 |          18.7 |   16.8 |
+----------------+-----------+---------------+--------+
```

(Do not worry about CASE — we will cover it in Chapter 19. The point here is that aggregate functions can be used in creative ways.)

How Many Employees Earn Above Average?

```sql
SELECT COUNT(*) AS above_average_count
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

Result:

```
+---------------------+
| above_average_count |
+---------------------+
|                   4 |
+---------------------+
(1 row)
```

Four employees earn above the average salary of $67,800. This uses a subquery (a query inside a query), which we will cover in detail in Chapter 16.

Salary Summary by Salary Range

```sql
SELECT
    COUNT(CASE WHEN salary < 60000 THEN 1 END)                  AS under_60k,
    COUNT(CASE WHEN salary BETWEEN 60000 AND 79999 THEN 1 END)  AS between_60k_80k,
    COUNT(CASE WHEN salary >= 80000 THEN 1 END)                 AS over_80k
FROM employees;
```

Result:

```
+-----------+-----------------+----------+
| under_60k | between_60k_80k | over_80k |
+-----------+-----------------+----------+
|         3 |               5 |        2 |
+-----------+-----------------+----------+
```

How Aggregates Fit in the Query Flow

Here is our updated mental model of how PostgreSQL processes a query:

```
+-------------------+
|   employees       |
|   (all 10 rows)   |
+--------+----------+
         |
Step 1: FROM
         |
         v
+-------------------+
|  All rows         |
+--------+----------+
         |
Step 2: WHERE
(filter individual rows)
         |
         v
+-------------------+
|  Filtered rows    |
+--------+----------+
         |
Step 3: SELECT + Aggregates
(collapse rows into
 summary values)
         |
         v
+-------------------+
|  Single summary   |
|  row              |
+-------------------+
```

> Key insight: WHERE runs BEFORE aggregates. This means you cannot use WHERE to filter on the result of an aggregate. For that, you need HAVING, which we will learn in the next chapter.

Common Mistakes

Mistake 1: Mixing Aggregates with Regular Columns

```sql
-- Wrong: What name should PostgreSQL show with the MAX salary?
SELECT name, MAX(salary) FROM employees;
-- ERROR: column "employees.name" must appear in the GROUP BY clause
```

```
-- Right: If you want the name of the highest-paid person, use a subquery or ORDER BY
SELECT name, salary
FROM employees
ORDER BY salary DESC
LIMIT 1;
```

This is one of the most common SQL mistakes. When you use an aggregate like MAX(salary), PostgreSQL collapses all rows into one. But which `name` should it pick? It does not know, so it gives you an error.

Mistake 2: Using COUNT(column) When You Mean COUNT(*)

```sql
-- These are different!
SELECT COUNT(*)          AS all_rows;       -- Counts every row
SELECT COUNT(department) AS dept_values;    -- Counts non-NULL departments only
```

If you want to count total rows, use `COUNT(*)`. Use `COUNT(column)` only when you specifically want to count non-NULL values.

Mistake 3: Forgetting That AVG Ignores NULLs

```sql
-- If you have: 100, 200, NULL, 400
-- AVG = (100 + 200 + 400) / 3 = 233.33
-- NOT:  (100 + 200 + 0 + 400) / 4 = 175.00
```

Mistake 4: Trying to Filter Aggregates with WHERE

```sql
-- Wrong: WHERE runs before aggregates
SELECT department, AVG(salary)
FROM employees
WHERE AVG(salary) > 60000   -- ERROR!
GROUP BY department;

-- Right: Use HAVING (covered in the next chapter)
SELECT department, AVG(salary)
FROM employees
GROUP BY department
HAVING AVG(salary) > 60000;
```

Best Practices

1. Always alias your aggregates. `SELECT COUNT(*) FROM employees` gives you a column named `count`. `SELECT COUNT(*) AS total_employees` is self-documenting.
2. Use ROUND with AVG. Averages often produce many decimal places. ROUND(AVG(col), 2) keeps results clean.
3. Use COUNT(*) for total row counts. It is the fastest and clearest way to count rows.
4. Be aware of NULL behavior. Know that SUM, AVG, MIN, and MAX all ignore NULLs. Use COALESCE if you need to treat NULL as zero.
5. Do not mix aggregates with non-aggregated columns unless you use GROUP BY (next chapter).

Quick Summary

```
+-----------------+---------------------------------------------+
| Function        | What It Does                                |
+-----------------+---------------------------------------------+
| COUNT(*)        | Counts all rows                             |
| COUNT(column)   | Counts non-NULL values in a column          |
| COUNT(DISTINCT) | Counts unique non-NULL values               |
| SUM(column)     | Adds up all values                          |
| AVG(column)     | Calculates the average (ignores NULLs)      |
| MIN(column)     | Finds the smallest value                    |
| MAX(column)     | Finds the largest value                     |
| ROUND(value, n) | Rounds to n decimal places                  |
+-----------------+---------------------------------------------+
```

Key Points

- Aggregate functions collapse many rows into a single summary value.
- COUNT(*) counts all rows. COUNT(column) counts non-NULL values only.
- COUNT(DISTINCT column) counts unique values.
- SUM and AVG only work with numeric columns. MIN and MAX work with numbers, text, and dates.
- All aggregate functions except COUNT(*) ignore NULL values.

- AVG ignores NULLs in both the sum and the count (which can be surprising).
- You cannot mix aggregate and non-aggregate columns in SELECT without GROUP BY.
- WHERE filters rows BEFORE aggregation. To filter after aggregation, use HAVING (next chapter).
- Always use aliases and ROUND to make aggregate results readable.

Practice Questions

1. Write a query to find the total number of employees and the total salary budget.
2. What is the difference between `COUNT(*)` and `COUNT(department)`? When would they give different results?
3. Write a query to find the average salary of employees hired in 2020 or later. Round to 2 decimal places.
4. If a column has values 10, 20, NULL, and 40, what does AVG return? What does SUM return? What does COUNT(*) return? What does COUNT(column) return?
5. Write a query that shows the lowest salary, highest salary, and the difference between them (the salary range).

Exercises

Exercise 1: Company Dashboard

Write a single query that produces a "company dashboard" showing:

- Total number of employees
- Total salary budget
- Average salary (rounded to 2 decimal places)
- Lowest salary
- Highest salary
- Number of departments

Exercise 2: Department Comparison

Write separate queries to find the average salary for each department (Engineering, Sales, Marketing, HR). Which department has the highest average? Which has the lowest?

(Hint: In the next chapter, you will learn how to do this in a single query with GROUP BY.)

Exercise 3: Tenure Analysis

Write queries to answer:

1. When was the first employee hired? (Earliest hire date)
2. When was the most recent hire? (Latest hire date)
3. How many employees were hired in 2020?

What Is Next?

Right now, to compare departments, you need to write a separate query for each one. That is tedious and does not scale. In the next chapter, you will learn GROUP BY and HAVING — tools that let you calculate aggregates for each group in a single query. Instead of asking "What is the average salary in Engineering?" then "What is the average salary in Sales?" you will be able to ask "What is the average salary in EACH department?" with one query.

Chapter 11: GROUP BY and HAVING — Aggregating by Category

What You Will Learn

In this chapter, you will learn how to:

- Group rows by one or more columns using GROUP BY
- Use aggregate functions (COUNT, SUM, AVG, MIN, MAX) with GROUP BY
- Group by multiple columns at once
- Filter groups using HAVING (vs. WHERE which filters rows)
- Combine GROUP BY with ORDER BY for sorted summaries
- Avoid the most common GROUP BY mistake (non-aggregated columns in SELECT)
- Write practical group-based queries

Why This Chapter Matters

In the previous chapter, you learned aggregate functions. But you could only summarize the entire table at once, or write separate queries for each department. That is like having a calculator that only shows one total — you would have to manually run it once for each category.

GROUP BY is the tool that lets you say, "Calculate the average salary FOR EACH department" in a single query. It takes a pile of mixed data and organizes it into groups, then runs your aggregate functions on each group separately.

Think of it like sorting a box of mixed LEGO bricks by color, then counting how many bricks you have of each color. Without GROUP BY, you can only count the total number of bricks. With GROUP BY, you get a breakdown: 15 red, 22 blue, 8 green, and so on.

GROUP BY is essential for any kind of reporting: sales by region, orders by month, employees by department, revenue by product category. If you have ever seen a summary table or a bar chart, there was probably a GROUP BY query behind it.

Our Practice Table

We continue with the employees table:

```
+----+------------------+-------------+-----------+------------+
| id | name             | department  | salary    | hire_date  |
+----+------------------+-------------+-----------+------------+
|  1 | Alice Johnson    | Engineering | 85000.00  | 2020-03-15 |
|  2 | Bob Smith        | Marketing   | 62000.00  | 2019-07-22 |
|  3 | Carol Williams   | Engineering | 92000.00  | 2018-11-01 |
|  4 | David Brown      | Sales       | 58000.00  | 2021-01-10 |
|  5 | Eva Martinez     | Marketing   | 65000.00  | 2020-09-05 |
|  6 | Frank Lee        | Engineering | 78000.00  | 2022-02-14 |
|  7 | Grace Kim        | Sales       | 61000.00  | 2019-04-30 |
|  8 | Henry Wilson     | HR          | 55000.00  | 2021-06-18 |
|  9 | Ivy Chen         | HR          | 59000.00  | 2020-12-01 |
| 10 | Jack Davis       | Sales       | 63000.00  | 2023-03-20 |
+----+------------------+-------------+-----------+------------+
```

GROUP BY — The Concept

GROUP BY takes all the rows and puts them into groups based on a column's value. Then, any aggregate functions in your SELECT run once per group instead of once for the entire table.

Visualizing Groups

Here is what happens when you GROUP BY department:

```
Before GROUP BY (all mixed together):
+------------------+-------------+-----------+
| Alice Johnson    | Engineering | 85000.00  |
| Bob Smith        | Marketing   | 62000.00  |
| Carol Williams   | Engineering | 92000.00  |
```

```
| David Brown      | Sales       | 58000.00  |
| Eva Martinez     | Marketing   | 65000.00  |
| Frank Lee        | Engineering | 78000.00  |
| Grace Kim        | Sales       | 61000.00  |
| Henry Wilson     | HR          | 55000.00  |
| Ivy Chen         | HR          | 59000.00  |
| Jack Davis       | Sales       | 63000.00  |
+------------------+-------------+-----------+

After GROUP BY department (sorted into buckets):

+----- Engineering -----+   +----- HR -----+
| Alice    | 85000.00   |   | Henry | 55000 |
| Carol    | 92000.00   |   | Ivy   | 59000 |
| Frank    | 78000.00   |   +-------+-------+
+----------+------------+
                             +----- Marketing -----+
+----- Sales -----+         | Bob   | 62000         |
| David | 58000   |         | Eva   | 65000         |
| Grace | 61000   |         +------+---------------+
| Jack  | 63000   |
+-------+---------+

Then aggregates run on EACH bucket:
Engineering: COUNT = 3, AVG = 85000
HR:          COUNT = 2, AVG = 57000
Marketing:   COUNT = 2, AVG = 63500
Sales:       COUNT = 3, AVG = 60666.67
```

GROUP BY with COUNT

The simplest GROUP BY query — count how many employees are in each department:

```sql
SELECT
    department,
    COUNT(*) AS employee_count
FROM employees
GROUP BY department;
```

Line-by-line breakdown:

- `SELECT department` — Show the department name (this is the column we are grouping by).
- `COUNT(*) AS employee_count` — Count the rows in each group.
- `FROM employees` — The table.

- `GROUP BY department` — Create groups based on unique department values.

Result:

```
+-------------+----------------+
| department  | employee_count |
+-------------+----------------+
| Engineering |              3 |
| HR          |              2 |
| Marketing   |              2 |
| Sales       |              3 |
+-------------+----------------+
(4 rows)
```

One row per group, not one row per employee. The result has 4 rows because there are 4 unique departments.

GROUP BY with Other Aggregates

Average Salary by Department

```sql
SELECT
    department,
    ROUND(AVG(salary), 2) AS avg_salary
FROM employees
GROUP BY department;
```

Result:

```
+-------------+------------+
| department  | avg_salary |
+-------------+------------+
| Engineering |   85000.00 |
| HR          |   57000.00 |
| Marketing   |   63500.00 |
| Sales       |   60666.67 |
+-------------+------------+
(4 rows)
```

In one query, you can see that Engineering has the highest average salary and HR has the lowest.

Total Salary Budget by Department

```sql
SELECT
    department,
    SUM(salary) AS total_budget,
    COUNT(*)    AS headcount
FROM employees
GROUP BY department;
```

Result:

```
+-------------+--------------+-----------+
| department  | total_budget | headcount |
+-------------+--------------+-----------+
| Engineering |    255000.00 |         3 |
| HR          |    114000.00 |         2 |
| Marketing   |    127000.00 |         2 |
| Sales       |    182000.00 |         3 |
+-------------+--------------+-----------+
(4 rows)
```

Complete Department Summary

You can use multiple aggregates with GROUP BY:

```sql
SELECT
    department,
    COUNT(*)              AS headcount,
    SUM(salary)           AS total_salary,
    ROUND(AVG(salary), 2) AS avg_salary,
    MIN(salary)           AS min_salary,
    MAX(salary)           AS max_salary
FROM employees
GROUP BY department;
```

Result:

```
+-------------+-----------+--------------+------------+------------+------------+
| department  | headcount | total_salary | avg_salary | min_salary | max_salary |
+-------------+-----------+--------------+------------+------------+------------+
| Engineering |         3 |    255000.00 |   85000.00 |   78000.00 |   92000.00 |
| HR          |         2 |    114000.00 |   57000.00 |   55000.00 |   59000.00 |
| Marketing   |         2 |    127000.00 |   63500.00 |   62000.00 |   65000.00 |
| Sales       |         3 |    182000.00 |   60666.67 |   58000.00 |   63000.00 |
+-------------+-----------+--------------+------------+------------+------------+
(4 rows)
```

This is a complete departmental salary report in a single query. This is the power of GROUP BY.

GROUP BY with Multiple Columns

You can group by more than one column. Each unique combination of values becomes its own group.

Let us add a second table to make this more interesting:

```sql
CREATE TABLE sales (
    id          SERIAL PRIMARY KEY,
    product     VARCHAR(50),
    category    VARCHAR(50),
    amount      NUMERIC(10,2),
    sale_date   DATE
);

INSERT INTO sales (product, category, amount, sale_date) VALUES
    ('Laptop',     'Electronics', 1200.00, '2024-01-15'),
    ('Mouse',      'Electronics',   25.00, '2024-01-15'),
    ('Laptop',     'Electronics', 1200.00, '2024-01-20'),
    ('Desk Chair', 'Furniture',    350.00, '2024-01-18'),
    ('Desk',       'Furniture',    500.00, '2024-01-22'),
    ('Mouse',      'Electronics',   25.00, '2024-02-01'),
    ('Keyboard',   'Electronics',   75.00, '2024-02-05'),
    ('Bookshelf',  'Furniture',    200.00, '2024-02-10'),
    ('Laptop',     'Electronics', 1200.00, '2024-02-15'),
    ('Desk Chair', 'Furniture',    350.00, '2024-02-20');
```

Sales by Category

```sql
SELECT
    category,
    COUNT(*)    AS total_sales,
    SUM(amount) AS total_revenue
FROM sales
GROUP BY category;
```

Result:

```
+-------------+-------------+---------------+
| category    | total_sales | total_revenue |
+-------------+-------------+---------------+
| Electronics |           6 |       3725.00 |
| Furniture   |           4 |       1400.00 |
+-------------+-------------+---------------+
(2 rows)
```

Sales by Category and Product

```sql
SELECT
    category,
    product,
    COUNT(*)    AS times_sold,
    SUM(amount) AS total_revenue
FROM sales
GROUP BY category, product
ORDER BY category, total_revenue DESC;
```

Result:

```
+-------------+------------+------------+---------------+
| category    | product    | times_sold | total_revenue |
+-------------+------------+------------+---------------+
| Electronics | Laptop     |          3 |       3600.00 |
| Electronics | Keyboard   |          1 |         75.00 |
| Electronics | Mouse      |          2 |         50.00 |
| Furniture   | Desk       |          1 |        500.00 |
| Furniture   | Desk Chair |          2 |        700.00 |
| Furniture   | Bookshelf  |          1 |        200.00 |
+-------------+------------+------------+---------------+
(6 rows)
```

With two GROUP BY columns, each unique combination of category + product becomes its own group. Laptops in Electronics is one group, Mice in Electronics is another, and so on.

GROUP BY + ORDER BY

GROUP BY produces groups but does not guarantee their order. Add ORDER BY to sort the grouped results:

```sql
SELECT
    department,
    ROUND(AVG(salary), 2) AS avg_salary
FROM employees
GROUP BY department
ORDER BY avg_salary DESC;
```

Result:

```
+-------------+------------+
| department  | avg_salary |
+-------------+------------+
| Engineering |   85000.00 |
| Marketing   |   63500.00 |
| Sales       |   60666.67 |
| HR          |   57000.00 |
+-------------+------------+
(4 rows)
```

Now the departments are sorted by average salary, highest first. You can quickly see that Engineering pays the most on average.

HAVING — Filtering Groups

WHERE filters individual rows BEFORE grouping. But what if you want to filter AFTER grouping? For example, "Show me only departments that have more than 2 employees."

That is what HAVING does. It filters groups, not rows.

WHERE vs. HAVING

```
WHERE:   Filters individual rows  BEFORE they are grouped
HAVING:  Filters groups           AFTER  they are formed
```

```
+-------------------+
|   All rows        |
+--------+----------+
         |
WHERE (filter rows)
         |
         v
```

```
+-------------------+
|  Filtered rows    |
+--------+----------+
         |
GROUP BY (create groups)
         |
         v
+-------------------+
|  Groups with      |
|  aggregates       |
+--------+----------+
         |
HAVING (filter groups)
         |
         v
+-------------------+
|  Filtered groups  |
+-------------------+
```

Example: Departments with More Than 2 Employees

```sql
SELECT
    department,
    COUNT(*) AS employee_count
FROM employees
GROUP BY department
HAVING COUNT(*) > 2;
```

Line-by-line breakdown:

- `GROUP BY department` — Create groups by department.
- `HAVING COUNT(*) > 2` — After grouping, keep only groups where the count is greater than 2.

Result:

```
+-------------+----------------+
| department  | employee_count |
+-------------+----------------+
| Engineering |              3 |
| Sales       |              3 |
+-------------+----------------+
(2 rows)
```

HR (2 employees) and Marketing (2 employees) are excluded because they do not have more than 2 employees.

Example: Departments with High Average Salary

```sql
SELECT
    department,
    ROUND(AVG(salary), 2) AS avg_salary
FROM employees
GROUP BY department
HAVING AVG(salary) > 60000
ORDER BY avg_salary DESC;
```

Result:

```
+-------------+------------+
| department  | avg_salary |
+-------------+------------+
| Engineering |   85000.00 |
| Marketing   |   63500.00 |
| Sales       |   60666.67 |
+-------------+------------+
(3 rows)
```

HR is excluded because its average salary ($57,000) is below $60,000.

Combining WHERE and HAVING

You can use both WHERE and HAVING in the same query. WHERE filters rows first, then GROUP BY groups the remaining rows, then HAVING filters the groups:

```sql
SELECT
    department,
    COUNT(*)              AS headcount,
    ROUND(AVG(salary), 2) AS avg_salary
FROM employees
WHERE hire_date >= '2020-01-01'
GROUP BY department
HAVING COUNT(*) >= 2
ORDER BY avg_salary DESC;
```

Line-by-line breakdown:

- `WHERE hire_date >= '2020-01-01'` — First, keep only employees hired in 2020 or later.
- `GROUP BY department` — Group the remaining employees by department.
- `HAVING COUNT(*) >= 2` — Keep only groups with 2 or more employees.

Step-by-step execution:

```
  Step 1: WHERE hire_date >= '2020-01-01'
  Keeps: Alice, David, Eva, Frank, Henry, Ivy, Jack  (7 rows)
  Removes: Bob (2019), Carol (2018), Grace (2019)

  Step 2: GROUP BY department
  Engineering: Alice, Frank         (2 employees)
  HR:          Henry, Ivy           (2 employees)
  Marketing:   Eva                  (1 employee)
  Sales:       David, Jack          (2 employees)

  Step 3: HAVING COUNT(*) >= 2
  Keeps: Engineering (2), HR (2), Sales (2)
  Removes: Marketing (1)
```

Result:

```
+-------------+-----------+------------+
| department  | headcount | avg_salary |
+-------------+-----------+------------+
| Engineering |         2 |   81500.00 |
| Sales       |         2 |   60500.00 |
| HR          |         2 |   57000.00 |
+-------------+-----------+------------+
(3 rows)
```

The Critical Mistake: Non-Aggregated Columns in SELECT

This is the most common GROUP BY error. Every column in your SELECT must either:

1. Be in the GROUP BY clause, OR
2. Be inside an aggregate function (COUNT, SUM, AVG, etc.)

The Error

```sql
-- WRONG: "name" is not in GROUP BY and not inside an aggregate
SELECT
    department,
    name,           -- Which name? There are multiple per department!
    AVG(salary)
FROM employees
GROUP BY department;
```

```
ERROR:  column "employees.name" must appear in the GROUP BY clause
        or be used in an aggregate function
```

Why This Happens

When you GROUP BY department, PostgreSQL collapses the Engineering group from 3 rows (Alice, Carol, Frank) into 1 row. If you also SELECT name, which of those 3 names should it show? PostgreSQL does not know and refuses to guess.

```
  Engineering group:
  +------------------+-----------+
  | Alice Johnson    | 85000.00  |   Which name should
  | Carol Williams   | 92000.00  |   appear in the result?
  | Frank Lee        | 78000.00  |   PostgreSQL says: ERROR!
  +------------------+-----------+

  The average is clear: 85000.00
  But the name is ambiguous.
```

How to Fix It

Option 1: Remove the non-aggregated column:

```sql
SELECT
    department,
    ROUND(AVG(salary), 2) AS avg_salary
FROM employees
GROUP BY department;
```

Option 2: Add the column to GROUP BY (changes the grouping):

```sql
SELECT
    department,
    name,
    salary
FROM employees
GROUP BY department, name, salary;
```

But this gives you one row per employee again, defeating the purpose of grouping.

Option 3: Use an aggregate function on the column:

```sql
SELECT
    department,
    MIN(name)             AS first_name_alphabetically,
    ROUND(AVG(salary), 2) AS avg_salary
FROM employees
GROUP BY department;
```

Result:

```
+-------------+---------------------------+------------+
| department  | first_name_alphabetically  | avg_salary |
+-------------+---------------------------+------------+
| Engineering | Alice Johnson              |   85000.00 |
| HR          | Henry Wilson               |   57000.00 |
| Marketing   | Bob Smith                  |   63500.00 |
| Sales       | David Brown                |   60666.67 |
+-------------+---------------------------+------------+
```

Practical Examples

Example 1: Hiring Trends by Year

```sql
SELECT
    EXTRACT(YEAR FROM hire_date) AS hire_year,
    COUNT(*)                     AS hires
FROM employees
GROUP BY EXTRACT(YEAR FROM hire_date)
ORDER BY hire_year;
```

Result:

```
+-----------+-------+
| hire_year | hires |
+-----------+-------+
|      2018 |     1 |
|      2019 |     2 |
|      2020 |     3 |
|      2021 |     2 |
|      2022 |     1 |
|      2023 |     1 |
+-----------+-------+
(6 rows)
```

2020 was the busiest hiring year with 3 new employees.

Example 2: Sales Report by Category

```sql
SELECT
    category,
    COUNT(*)              AS transactions,
    SUM(amount)           AS total_revenue,
    ROUND(AVG(amount), 2) AS avg_transaction,
    MIN(amount)           AS smallest_sale,
    MAX(amount)           AS largest_sale
FROM sales
GROUP BY category
ORDER BY total_revenue DESC;
```

Result:

```
+-------------+--------------+---------------+-----------------+---------------+-------...
| category    | transactions | total_revenue | avg_transaction | smallest_sale | larges...
+-------------+--------------+---------------+-----------------+---------------+-------...
| Electronics |            6 |       3725.00 |          620.83 |         25.00 |      1...
| Furniture   |            4 |       1400.00 |          350.00 |        200.00 |       ...
+-------------+--------------+---------------+-----------------+---------------+-------...
(2 rows)
```

Example 3: Finding Departments That Need Attention

"Show me departments where the salary range (max - min) is greater than 10,000."

```sql
SELECT
    department,
    MIN(salary)              AS min_salary,
    MAX(salary)              AS max_salary,
    MAX(salary) - MIN(salary) AS salary_spread
FROM employees
GROUP BY department
HAVING MAX(salary) - MIN(salary) > 10000
ORDER BY salary_spread DESC;
```

Result:

```
+-------------+------------+------------+---------------+
| department  | min_salary | max_salary | salary_spread |
+-------------+------------+------------+---------------+
| Engineering |   78000.00 |   92000.00 |      14000.00 |
```

```
+-------------+------------+------------+---------------+
(1 row)
```

Only Engineering has a salary spread greater than $10,000. This might indicate salary inequality worth investigating.

The Complete Query Execution Order

Here is the full picture of how PostgreSQL processes a query with GROUP BY and HAVING:

```
  Writing Order:           Execution Order:
  1. SELECT                1. FROM         (find the table)
  2. FROM                  2. WHERE        (filter individual rows)
  3. WHERE                 3. GROUP BY     (form groups)
  4. GROUP BY              4. HAVING       (filter groups)
  5. HAVING                5. SELECT       (pick columns & compute)
  6. ORDER BY              6. ORDER BY     (sort results)
  7. LIMIT/OFFSET          7. LIMIT/OFFSET (cut to size)
```

This is why:

- WHERE cannot use aggregate functions (it runs before GROUP BY)
- HAVING can use aggregate functions (it runs after GROUP BY)
- ORDER BY can use aliases (it runs after SELECT)

Common Mistakes

Mistake 1: Using WHERE Instead of HAVING for Aggregate Conditions

```sql
-- Wrong: WHERE runs before GROUP BY, so it cannot filter aggregates
SELECT department, COUNT(*)
FROM employees
WHERE COUNT(*) > 2        -- ERROR!
GROUP BY department;
```

```
-- Right: Use HAVING for aggregate conditions
SELECT department, COUNT(*)
FROM employees
GROUP BY department
HAVING COUNT(*) > 2;
```

Mistake 2: Non-Aggregated Column in SELECT

```sql
-- Wrong: salary is not in GROUP BY and not in an aggregate
SELECT department, salary, AVG(salary)
FROM employees
GROUP BY department;

-- Right: Either add salary to GROUP BY or use it in an aggregate
SELECT department, AVG(salary) AS avg_salary
FROM employees
GROUP BY department;
```

Mistake 3: Using HAVING When WHERE Would Work

```sql
-- Works but inefficient: HAVING filters after grouping
SELECT department, COUNT(*) AS cnt
FROM employees
GROUP BY department
HAVING department = 'Sales';

-- Better: WHERE filters before grouping (fewer rows to process)
SELECT department, COUNT(*) AS cnt
FROM employees
WHERE department = 'Sales'
GROUP BY department;
```

Use WHERE for conditions on individual row values. Use HAVING only for conditions on aggregate results.

Mistake 4: Forgetting GROUP BY with Aggregates

```sql
-- Wrong: department is not aggregated and GROUP BY is missing
SELECT department, AVG(salary)
FROM employees;
-- ERROR!
```

```
-- Right: Add GROUP BY
SELECT department, AVG(salary)
FROM employees
GROUP BY department;
```

Best Practices

1. Use WHERE for row-level filters, HAVING for group-level filters. WHERE is more efficient because it reduces the number of rows before grouping.
2. Always pair GROUP BY with aggregates. A GROUP BY without any aggregate function in SELECT is technically valid but usually means something is wrong.
3. Include the GROUP BY column in SELECT. While not required, it makes results readable:

```sql
-- Confusing: What do these numbers refer to?
SELECT COUNT(*), AVG(salary) FROM employees GROUP BY department;

-- Clear: Now we know which department each row represents
SELECT department, COUNT(*), AVG(salary) FROM employees GROUP BY department;
```

4. Add ORDER BY for predictable results. GROUP BY does not guarantee any particular order.
5. Use aliases for aggregate columns. `COUNT(*)` as a column header is not very informative. `employee_count` is much better.

Quick Summary

```
+---------------------+-------------------------------------------------+
| Concept             | Example                                         |
+---------------------+-------------------------------------------------+
| Group by 1 column   | GROUP BY department                             |
| Group by 2 columns  | GROUP BY category, product                      |
| Count per group     | SELECT dept, COUNT(*) ... GROUP BY dept          |
| Average per group   | SELECT dept, AVG(salary) ... GROUP BY dept        |
| Filter groups       | HAVING COUNT(*) > 2                             |
| WHERE + HAVING      | WHERE salary > 50000 ... HAVING AVG(salary) > 60k|
```

```
| Sorted groups       | GROUP BY dept ORDER BY AVG(salary) DESC           |
+---------------------+---------------------------------------------------+
```

Key Points

- GROUP BY divides rows into groups based on column values, then aggregates run on each group.
- Every non-aggregated column in SELECT must appear in the GROUP BY clause.
- You can GROUP BY multiple columns — each unique combination becomes a group.
- WHERE filters individual rows BEFORE grouping. HAVING filters groups AFTER grouping.
- Use WHERE for row-level conditions (department, salary, date). Use HAVING for aggregate conditions (COUNT, AVG, SUM).
- GROUP BY does not guarantee order — add ORDER BY if you need sorted results.
- GROUP BY + aggregate functions are the foundation of all summary reports.

Practice Questions

1. Write a query to count how many employees are in each department. Sort by count, highest first.
2. Write a query to find the average salary for each department. Only show departments where the average salary is above $60,000.
3. What is wrong with this query?

```sql
SELECT department, name, COUNT(*)
FROM employees
GROUP BY department;
```

4. Explain the difference between these two queries:

```sql
-- Query A
SELECT department, COUNT(*)
FROM employees
WHERE salary > 60000
```

```
GROUP BY department;

-- Query B
SELECT department, COUNT(*)
FROM employees
GROUP BY department
HAVING COUNT(*) > 2;
```

5. Can you use both WHERE and HAVING in the same query? If yes, write a query that uses both.

Exercises

Exercise 1: Department Budget Report

Write a single query that shows for each department:

- Department name
- Number of employees
- Total salary budget
- Average salary (rounded to 2 decimal places)
- Highest salary

Sort the results by total salary budget, highest first.

Exercise 2: Sales Analysis

Using the sales table created in this chapter, write queries to answer:

1. How many transactions occurred for each product?
2. What is the total revenue per category?
3. Which products generated more than $500 in total revenue?

Exercise 3: Selective Grouping

Write a query that:

1. Excludes employees hired before 2020 (use WHERE)
2. Groups the remaining employees by department
3. Only shows departments with 2 or more recent hires (use HAVING)
4. Shows the department name, count of recent hires, and their average salary

5. Sorts by average salary descending

What Is Next?

So far, all your queries have worked with a single table. But real databases spread data across multiple tables. Employees are in one table, departments in another, and projects in a third. In the next chapter, you will learn about JOINs — the technique for combining data from multiple tables into a single result. Joins are one of the most powerful features of relational databases and the key reason why the relational model has dominated data storage for decades.

Chapter 12: Introduction to Joins — Combining Data from Multiple Tables

What You Will Learn

In this chapter, you will learn how to:

- Understand why data is split across multiple tables
- Explain what a join does conceptually
- Set up two related tables connected by a foreign key
- Understand the different types of joins at a high level
- Visualize join types with Venn diagrams
- Write basic join syntax
- Avoid the Cartesian product trap

Why This Chapter Matters

Imagine you are organizing a wedding. You have one list of guests and another list of table assignments. To create the seating chart, you need to combine these two lists — matching each guest with their table. That matching process is exactly what a join does in SQL.

Until now, all your queries have worked with a single table. But real databases almost never work that way. A well-designed database splits data into separate tables to avoid duplication. Customer information goes in a customers table. Orders go in an orders table. Products go in a products table. This design keeps data organized and consistent (we will explore why in the normalization chapter later).

The trade-off is that when you need to see a customer along with their orders, you need to combine the tables. Joins are how you do that. They are the bridge between separate tables,

and they are arguably the single most important concept in relational databases.

If SELECT is the most-used SQL command, joins are the most-used technique within SELECT. Once you understand joins, you understand the heart of relational databases.

Why Data Is Split Across Tables

Let us start with a bad design to understand why splitting data is necessary.

The Bad Way — One Big Table

Imagine storing everything in one table:

```
+----+------------------+-------------+-----------+------------+--------------+--------...
| id | employee_name    | department  | salary    | hire_date  | dept_budget  | dept_lo...
+----+------------------+-------------+-----------+------------+--------------+--------...
|  1 | Alice Johnson    | Engineering | 85000.00  | 2020-03-15 | 500000.00    | Buildin...
|  2 | Bob Smith        | Marketing   | 62000.00  | 2019-07-22 | 200000.00    | Buildin...
|  3 | Carol Williams   | Engineering | 92000.00  | 2018-11-01 | 500000.00    | Buildin...
|  4 | David Brown      | Sales       | 58000.00  | 2021-01-10 | 300000.00    | Buildin...
|  5 | Eva Martinez     | Marketing   | 65000.00  | 2020-09-05 | 200000.00    | Buildin...
|  6 | Frank Lee        | Engineering | 78000.00  | 2022-02-14 | 500000.00    | Buildin...
+----+------------------+-------------+-----------+------------+--------------+--------...
```

See the problem? "500000.00" and "Building A, Fl 3" are repeated for every Engineering employee. This causes three issues:

1. Wasted storage — The same information is stored over and over.
2. Update anomalies — If Engineering moves to a new building, you need to update every Engineering row. Miss one, and your data is inconsistent.
3. Deletion anomalies — If you delete the last Marketing employee, you lose the Marketing department's budget and location information too.

The Good Way — Two Separate Tables

Split the data into two tables, each storing one type of thing:

departments table (information about departments):

```
+---------+-------------+-----------+------------------+
| dept_id | dept_name   | budget    | location         |
```

```
+---------+-------------+-----------+------------------+
|       1 | Engineering | 500000.00 | Building A, Fl 3 |
|       2 | Marketing   | 200000.00 | Building B, Fl 1 |
|       3 | Sales       | 300000.00 | Building A, Fl 1 |
|       4 | HR          | 150000.00 | Building B, Fl 2 |
+---------+-------------+-----------+------------------+
```

employees table (information about employees):

```
+----+------------------+---------+-----------+------------+
| id | name             | dept_id | salary    | hire_date  |
+----+------------------+---------+-----------+------------+
|  1 | Alice Johnson    |       1 | 85000.00  | 2020-03-15 |
|  2 | Bob Smith        |       2 | 62000.00  | 2019-07-22 |
|  3 | Carol Williams   |       1 | 92000.00  | 2018-11-01 |
|  4 | David Brown      |       3 | 58000.00  | 2021-01-10 |
|  5 | Eva Martinez     |       2 | 65000.00  | 2020-09-05 |
|  6 | Frank Lee        |       1 | 78000.00  | 2022-02-14 |
+----+------------------+---------+-----------+------------+
```

Now "Building A, Fl 3" is stored only once. If Engineering moves, you update one row in the departments table and every employee query automatically reflects the change.

The `dept_id` column in the employees table is what connects the two tables. It is a foreign key — a pointer that says "this employee belongs to that department."

Setting Up Our Practice Tables

Let us create these two tables so we can practice joining them:

```sql
-- Drop the old employees table and start fresh
DROP TABLE IF EXISTS employees;

-- Create departments table
CREATE TABLE departments (
    dept_id   SERIAL PRIMARY KEY,
    dept_name VARCHAR(50) NOT NULL,
    budget    NUMERIC(12,2),
    location  VARCHAR(100)
);

-- Create employees table with a foreign key to departments
CREATE TABLE employees (
```

```
    id         SERIAL PRIMARY KEY,
    name       VARCHAR(100) NOT NULL,
    dept_id    INTEGER REFERENCES departments(dept_id),
    salary     NUMERIC(10,2) NOT NULL,
    hire_date  DATE NOT NULL
);
```

Now insert the data:

```sql
INSERT INTO departments (dept_name, budget, location) VALUES
    ('Engineering', 500000.00, 'Building A, Floor 3'),
    ('Marketing',   200000.00, 'Building B, Floor 1'),
    ('Sales',       300000.00, 'Building A, Floor 1'),
    ('HR',          150000.00, 'Building B, Floor 2'),
    ('Research',    400000.00, 'Building C, Floor 1');

INSERT INTO employees (name, dept_id, salary, hire_date) VALUES
    ('Alice Johnson',   1, 85000.00, '2020-03-15'),
    ('Bob Smith',       2, 62000.00, '2019-07-22'),
    ('Carol Williams',  1, 92000.00, '2018-11-01'),
    ('David Brown',     3, 58000.00, '2021-01-10'),
    ('Eva Martinez',    2, 65000.00, '2020-09-05'),
    ('Frank Lee',       1, 78000.00, '2022-02-14'),
    ('Grace Kim',       3, 61000.00, '2019-04-30'),
    ('Henry Wilson',    4, 55000.00, '2021-06-18'),
    ('Ivy Chen',        4, 59000.00, '2020-12-01'),
    ('Jack Davis',      3, 63000.00, '2023-03-20'),
    ('Karen Lopez',  NULL, 52000.00, '2024-01-08');
```

Notice two important things:

1. The Research department (dept_id = 5) has no employees assigned to it yet.
2. Karen Lopez has `dept_id = NULL` — she has been hired but not yet assigned to a department.

These edge cases will help us understand the difference between join types.

The Foreign Key Connection

The connection between our two tables works through matching values:

```
  departments table                 employees table
  +-------+-------------+          +----+----------------+-------+
  |dept_id| dept_name   |          | id | name           |dept_id|
```

```
+-------+-------------+          +----+----------------+-------+
|   1   | Engineering |<----+----|  1 | Alice Johnson  |   1   |
|       |             |     +----|  3 | Carol Williams |   1   |
|       |             |     +----|  6 | Frank Lee      |   1   |
+-------+-------------+          +----+----------------+-------+
|   2   | Marketing   |<----+----|  2 | Bob Smith      |   2   |
|       |             |     +----|  5 | Eva Martinez   |   2   |
+-------+-------------+          +----+----------------+-------+
|   3   | Sales       |<----+----|  4 | David Brown    |   3   |
|       |             |     +----|  7 | Grace Kim      |   3   |
|       |             |     +----|  10| Jack Davis     |   3   |
+-------+-------------+          +----+----------------+-------+
|   4   | HR          |<----+----|  8 | Henry Wilson   |   4   |
|       |             |     +----|  9 | Ivy Chen       |   4   |
+-------+-------------+          +----+----------------+-------+
|   5   | Research    |<--- (no employees)
+-------+-------------+          +----+----------------+-------+
                                 | 11 | Karen Lopez    | NULL  |
                                 +----+----------------+-------+
                                       (no department)
```

- `dept_id` in the departments table is the primary key — a unique identifier for each department.
- `dept_id` in the employees table is the foreign key — it references the primary key in departments.
- The arrows show which employees belong to which department.

What a Join Does

A join combines rows from two tables based on a related column. You specify the condition that connects them (usually a matching key), and PostgreSQL pairs up the matching rows.

```
departments              employees
+-------+--------+       +----+--------+-------+
|dept_id|dept_name|       | id | name   |dept_id|
+-------+--------+       +----+--------+-------+
|   1   | Eng    |       |  1 | Alice  |   1   |
|   2   | Mkt    |       |  2 | Bob    |   2   |
+-------+--------+       +----+--------+-------+

                JOIN ON
         departments.dept_id = employees.dept_id

                    |
                    v
```

```
+-------+--------+----+--------+-------+
|dept_id|dept_name| id | name   |dept_id|
+-------+--------+----+--------+-------+
|   1   | Eng    |  1 | Alice  |   1   |  <-- dept_id 1 matches
|   2   | Mkt    |  2 | Bob    |   2   |  <-- dept_id 2 matches
+-------+--------+----+--------+-------+
```

The join condition `departments.dept_id = employees.dept_id` tells PostgreSQL: "For each employee, find the department where the dept_id values match, and combine those rows."

Think of it like a matchmaking service. Two lists of people. The join condition is what they have in common (like matching interest in hiking). The result is pairs of people who share that connection.

Types of Joins — The Big Picture

There are several types of joins, each handling unmatched rows differently. Here is an overview using Venn diagrams:

INNER JOIN — Only Matches

```
        +-------------+-------------+
       /              \             \
      /    Departments   \  Employees  \
     /      only          \   only      \
    |                      |             |
    |         +-----------+|             |
    |         |  MATCHING ||             |
    |         |   ROWS   ||             |
    |         +-----------+|             |
    |                      |             |
     \                    /             /
      \                  /             /
       \                /             /
        +-------------+-------------+
                      ^
                      |
            INNER JOIN returns
            only the overlap
```

INNER JOIN returns only rows that have a match in BOTH tables. Departments with no employees and employees with no department are excluded.

LEFT JOIN — All Left + Matches

```
        +-------------+-------------+
       /              \            \
      / +-------------+ \            \
     /  | ALL LEFT    |  \            \
    |   | TABLE ROWS  |   |            |
    |   |  +---------+|   |            |
    |   |  |MATCHING ||   |            |
    |   |  | ROWS    ||   |            |
    |   |  +---------+|   |            |
    |   +-------------+   |            |
     \                    /            /
      \                  /            /
       +-------------+-------------+
              ^
              |
      LEFT JOIN returns
      everything shaded
```

LEFT JOIN returns ALL rows from the left table. If a left-table row has no match in the right table, the right-table columns are filled with NULL.

RIGHT JOIN — Matches + All Right

```
        +-------------+-------------+
       /              \            \
      /                \  +--------+ \
     /                  \ | ALL    |  \
    |                    || RIGHT  |   |
    |         +----------+| TABLE  |   |
    |         |  MATCHING || ROWS   |   |
    |         |   ROWS   ||        |   |
    |         +----------+|        |   |
    |                    |+--------+   |
     \                  /            /
      \                /            /
       +-------------+-------------+
                     ^
                     |
                RIGHT JOIN returns
                everything shaded
```

RIGHT JOIN is the mirror of LEFT JOIN. It returns ALL rows from the right table.

FULL OUTER JOIN — Everything

```
         +-------------+-------------+
        / +---------------------------+\
       /  | ALL ROWS FROM BOTH TABLES | \
      /   |                           |  \
     |    |         +-----------+     |   |
     |    |         |  MATCHING |     |   |
     |    |         |   ROWS    |     |   |
     |    |         +-----------+     |   |
     |    |                           |   |
      \   +---------------------------+  /
       \                               /
        +-------------+-------------+
                      ^
                      |
             FULL OUTER JOIN returns
               everything
```

FULL OUTER JOIN returns ALL rows from BOTH tables. Unmatched rows on either side get NULLs.

Quick Comparison Table

```
+-----------------+-----------------------------------------------------+
| Join Type       | Returns                                             |
+-----------------+-----------------------------------------------------+
| INNER JOIN      | Only rows with matches in both tables               |
| LEFT JOIN       | All rows from left table + matches from right       |
| RIGHT JOIN      | All rows from right table + matches from left       |
| FULL OUTER JOIN | All rows from both tables                           |
| CROSS JOIN      | Every possible combination (Cartesian product)      |
+-----------------+-----------------------------------------------------+
```

We will dive deeply into each join type in the next two chapters. For now, let us learn the basic syntax using INNER JOIN.

Basic Join Syntax

Here is how to write a simple INNER JOIN:

```sql
SELECT
    employees.name,
    departments.dept_name,
```

```
    employees.salary
FROM employees
INNER JOIN departments
    ON employees.dept_id = departments.dept_id;
```

Line-by-line breakdown:

- `SELECT employees.name, departments.dept_name, employees.salary` — We prefix columns with table names to be clear about which table each column comes from.
- `FROM employees` — Start with the employees table.
- `INNER JOIN departments` — Combine it with the departments table.
- `ON employees.dept_id = departments.dept_id` — The join condition: match rows where the dept_id values are equal.

Result:

```
+------------------+-------------+-----------+
| name             | dept_name   | salary    |
+------------------+-------------+-----------+
| Alice Johnson    | Engineering | 85000.00  |
| Bob Smith        | Marketing   | 62000.00  |
| Carol Williams   | Engineering | 92000.00  |
| David Brown      | Sales       | 58000.00  |
| Eva Martinez     | Marketing   | 65000.00  |
| Frank Lee        | Engineering | 78000.00  |
| Grace Kim        | Sales       | 61000.00  |
| Henry Wilson     | HR          | 55000.00  |
| Ivy Chen         | HR          | 59000.00  |
| Jack Davis       | Sales       | 63000.00  |
+------------------+-------------+-----------+
(10 rows)
```

Notice: Karen Lopez is missing (she has no department — NULL dept_id) and the Research department is missing (it has no employees). INNER JOIN only returns matches.

Using Table Aliases for Shorter Queries

Typing `employees.name` and `departments.dept_name` gets tedious. Table aliases make it shorter:

```sql
SELECT
    e.name,
    d.dept_name,
    e.salary
FROM employees AS e
```

```
INNER JOIN departments AS d
    ON e.dept_id = d.dept_id;
```

`e` is short for `employees`, `d` is short for `departments`. The result is identical — aliases are just nicknames.

You can even drop the `AS` keyword:

```sql
SELECT e.name, d.dept_name, e.salary
FROM employees e
INNER JOIN departments d ON e.dept_id = d.dept_id;
```

Using Joins with WHERE, ORDER BY, and GROUP BY

Joins work seamlessly with everything you have learned so far:

```sql
-- Employees in Engineering, sorted by salary
SELECT e.name, d.dept_name, e.salary
FROM employees e
INNER JOIN departments d ON e.dept_id = d.dept_id
WHERE d.dept_name = 'Engineering'
ORDER BY e.salary DESC;
```

Result:

```
+------------------+-------------+-----------+
| name             | dept_name   | salary    |
+------------------+-------------+-----------+
| Carol Williams   | Engineering | 92000.00  |
| Alice Johnson    | Engineering | 85000.00  |
| Frank Lee        | Engineering | 78000.00  |
+------------------+-------------+-----------+
(3 rows)
```

```sql
-- Employee count and average salary per department
SELECT
    d.dept_name,
    COUNT(*)              AS headcount,
    ROUND(AVG(e.salary), 2) AS avg_salary
FROM employees e
INNER JOIN departments d ON e.dept_id = d.dept_id
GROUP BY d.dept_name
ORDER BY avg_salary DESC;
```

Result:

```
+-------------+-----------+------------+
| dept_name   | headcount | avg_salary |
+-------------+-----------+------------+
| Engineering |         3 |   85000.00 |
| Marketing   |         2 |   63500.00 |
| Sales       |         3 |   60666.67 |
| HR          |         2 |   57000.00 |
+-------------+-----------+------------+
(4 rows)
```

The Cartesian Product Warning

What happens if you forget the join condition? You get a Cartesian product (also called a CROSS JOIN) — every row from one table is combined with every row from the other table.

```sql
-- DANGER: No join condition!
SELECT e.name, d.dept_name
FROM employees e, departments d;
```

If employees has 11 rows and departments has 5 rows, you get 11 x 5 = 55 rows. Each employee is paired with EVERY department, which is almost never what you want.

```
  Without join condition (Cartesian product):

  Alice   + Engineering  ]
  Alice   + Marketing    ]-- Alice paired with ALL 5 depts
  Alice   + Sales        ]
  Alice   + HR           ]
  Alice   + Research     ]
  Bob     + Engineering  ]
  Bob     + Marketing    ]-- Bob paired with ALL 5 depts
  Bob     + Sales        ]
  Bob     + HR           ]
  Bob     + Research     ]
  ...                       (55 total rows!)
```

This is one of the most common join mistakes. With small tables it produces confusing results. With large tables, it can crash your database or fill up memory. Imagine two tables with 10,000 rows each — a Cartesian product would produce 100,000,000 rows.

How to Avoid It

Always use the explicit JOIN syntax with an ON condition:

```sql
-- Good: Explicit join with condition
SELECT e.name, d.dept_name
FROM employees e
INNER JOIN departments d ON e.dept_id = d.dept_id;

-- Bad: Implicit join (easy to forget the WHERE)
SELECT e.name, d.dept_name
FROM employees e, departments d
WHERE e.dept_id = d.dept_id;
```

Both produce the same result, but the explicit `JOIN ... ON` syntax makes it impossible to accidentally forget the join condition.

> Note: There are rare cases where a Cartesian product is intentional (such as generating all possible combinations). We will cover CROSS JOIN in a later chapter.

How Joins Fit in the Query Flow

```
+-------------------+    +-------------------+
|   employees       |    |   departments     |
+--------+----------+    +--------+----------+
         |                        |
Step 1: FROM + JOIN
"Combine the tables using
 the ON condition"
         |
         v
+-------------------+
|  Combined rows    |
|  (matched pairs)  |
+--------+----------+
         |
Step 2: WHERE
"Filter individual rows"
         |
Step 3: GROUP BY
"Form groups"
         |
Step 4: HAVING
"Filter groups"
         |
```

```
                Step 5: SELECT
                "Pick columns"
                         |
                Step 6: ORDER BY
                "Sort results"
                         |
                Step 7: LIMIT/OFFSET
                "Cut to size"
                         |
                         v
                +-------------------+
                |  Final results    |
                +-------------------+
```

The join happens first, before WHERE, GROUP BY, or anything else. PostgreSQL combines the tables into one big result set, then processes everything else on that combined data.

A Preview of What Is Next

Here is a quick preview of how the different join types handle our test data:

```sql
-- INNER JOIN: Only matched rows (10 rows)
-- Missing: Karen (no dept), Research dept (no employees)
SELECT e.name, d.dept_name
FROM employees e
INNER JOIN departments d ON e.dept_id = d.dept_id;

-- LEFT JOIN: All employees + their dept (11 rows)
-- Karen shows up with NULL dept_name
SELECT e.name, d.dept_name
FROM employees e
LEFT JOIN departments d ON e.dept_id = d.dept_id;

-- RIGHT JOIN: All departments + their employees (11 rows)
-- Research shows up with NULL employee name
SELECT e.name, d.dept_name
FROM employees e
RIGHT JOIN departments d ON e.dept_id = d.dept_id;

-- FULL OUTER JOIN: Everything (12 rows)
-- Both Karen and Research show up
SELECT e.name, d.dept_name
FROM employees e
FULL OUTER JOIN departments d ON e.dept_id = d.dept_id;
```

We will explore each of these in detail in the next two chapters.

Common Mistakes

Mistake 1: Forgetting the Join Condition

```sql
-- Wrong: Creates a Cartesian product (every combination)
SELECT e.name, d.dept_name
FROM employees e
INNER JOIN departments d;
-- ERROR (or unexpected results)

-- Right: Always specify ON
SELECT e.name, d.dept_name
FROM employees e
INNER JOIN departments d ON e.dept_id = d.dept_id;
```

Mistake 2: Ambiguous Column Names

```sql
-- Wrong: Both tables have a column that could be "dept_id"
SELECT name, dept_id, dept_name
FROM employees e
INNER JOIN departments d ON e.dept_id = d.dept_id;
-- ERROR: column reference "dept_id" is ambiguous

-- Right: Prefix with the table name or alias
SELECT e.name, e.dept_id, d.dept_name
FROM employees e
INNER JOIN departments d ON e.dept_id = d.dept_id;
```

When both tables have a column with the same name, you must specify which table you mean by using the table name or alias prefix.

Mistake 3: Joining on the Wrong Column

```sql
-- Wrong: Joining employee id with department id (different things!)
SELECT e.name, d.dept_name
FROM employees e
INNER JOIN departments d ON e.id = d.dept_id;
-- Returns wrong results: employee 1 matched with dept 1, etc.
```

```
-- Right: Join on the foreign key relationship
SELECT e.name, d.dept_name
FROM employees e
INNER JOIN departments d ON e.dept_id = d.dept_id;
```

Always join on the column that represents the actual relationship (the foreign key to the primary key).

Mistake 4: Using INNER JOIN When You Want All Rows

```sql
-- If you want ALL employees including those without a department:
-- Wrong: INNER JOIN excludes Karen (NULL dept_id)
SELECT e.name, d.dept_name
FROM employees e
INNER JOIN departments d ON e.dept_id = d.dept_id;
-- Karen is missing!

-- Right: Use LEFT JOIN
SELECT e.name, d.dept_name
FROM employees e
LEFT JOIN departments d ON e.dept_id = d.dept_id;
-- Karen appears with NULL dept_name
```

Best Practices

1. Always use explicit JOIN syntax. Write `INNER JOIN ... ON` instead of listing tables with commas. It is clearer and prevents accidental Cartesian products.
2. Use table aliases. They keep queries short and readable, especially when joining many tables.
3. Prefix all column names with table aliases in join queries. Even if a column name is unique, prefixing makes it clear which table it comes from.
4. Choose the right join type. Think about what should happen with unmatched rows before writing your query. Need all employees, even unassigned ones? Use LEFT JOIN.
5. Start with INNER JOIN when in doubt. It is the most common join and the safest default. Switch to LEFT/RIGHT/FULL only when you need unmatched rows.
6. Verify your results. After writing a join, check the row count. If you expect 10 rows but get 50, you might have a bad join condition causing duplicates.

Quick Summary

```
+--------------------+-----------------------------------------------------+
| Concept            | Description                                         |
+--------------------+-----------------------------------------------------+
| Why split tables?  | Avoid duplication, prevent update anomalies         |
| Foreign key        | Column in one table that references another table   |
| INNER JOIN         | Returns only matching rows from both tables         |
| LEFT JOIN          | All left rows + matches from right (NULLs if none)  |
| RIGHT JOIN         | All right rows + matches from left (NULLs if none)  |
| FULL OUTER JOIN    | All rows from both tables                           |
| CROSS JOIN         | Every combination (Cartesian product)               |
| Join condition     | ON table1.col = table2.col                          |
| Table alias        | FROM employees AS e (or just FROM employees e)      |
+--------------------+-----------------------------------------------------+
```

Key Points

- Real databases split data across multiple tables to avoid duplication and inconsistency.
- A foreign key in one table references the primary key of another table, creating a relationship.
- A join combines rows from two tables based on a matching condition (the ON clause).
- INNER JOIN returns only rows that have matches in both tables.
- LEFT JOIN returns all rows from the left table, filling in NULLs for unmatched right-side columns.
- Always use explicit `JOIN ... ON` syntax to prevent accidental Cartesian products.
- Table aliases (e for employees, d for departments) keep join queries readable.
- Prefix column names with table aliases to avoid ambiguity.
- The join is processed before WHERE, GROUP BY, HAVING, and ORDER BY.

Practice Questions

1. In our setup, why does Karen Lopez not appear in an INNER JOIN result between employees and departments?
2. What is a Cartesian product, and how do you accidentally create one?

3. Write a query that shows each employee's name, their department name, and their department's location. Use INNER JOIN.
4. How many rows would a Cartesian product produce between a table with 100 rows and a table with 50 rows?
5. Which join type would you use if you want to see ALL departments, including those with no employees?

Exercises

Exercise 1: Employee Directory

Write a query that produces a complete employee directory showing:

- Employee name
- Department name
- Salary
- Department location

Sort by department name, then by employee name within each department.

Exercise 2: Department Report with Joins

Write a query that shows each department's name, location, number of employees, and total salary. Sort by total salary descending. (Hint: use INNER JOIN with GROUP BY.)

Exercise 3: Exploring Join Behavior

Run each of these queries and compare the results:

1. INNER JOIN between employees and departments
2. Count the rows in the INNER JOIN result
3. How many employees are in the employees table total?
4. Why are the counts different? Which employees or departments are missing from the INNER JOIN result?

What Is Next?

Now that you understand what joins are and why they matter, the next chapter dives deep into INNER JOIN — the most common join type. You will practice writing INNER JOIN queries with multiple conditions, joining more than two tables, and solving real-world problems that require combining data from different sources.

Chapter 13: INNER JOIN — Finding Matching Rows Across Tables

What You Will Learn

- How INNER JOIN works and why it only returns matching rows
- How to use the ON clause to specify join conditions
- How to use table aliases to write shorter, cleaner queries
- How to join tables using foreign keys
- How to join three or more tables in a single query
- How to combine JOIN with WHERE, ORDER BY, and LIMIT
- How to solve real-world problems: employees with departments, orders with customers and products

Why This Chapter Matters

In the previous chapter, you learned what joins are and why relational databases split data across multiple tables. Now it is time to learn the most common join type: INNER JOIN.

Think of INNER JOIN like a matchmaking service. You have two guest lists for a party. INNER JOIN finds the people who appear on both lists. If someone is only on one list, they do not show up in the results.

In real-world applications, you use INNER JOIN constantly. Every time you display an order with the customer name, show an employee with their department, or list products with their categories, you are using INNER JOIN.

Setting Up Our Practice Tables

Let us create some tables we will use throughout this chapter.

```sql
CREATE TABLE departments (
    department_id   SERIAL PRIMARY KEY,
    department_name VARCHAR(50) NOT NULL,
    location        VARCHAR(50)
);

CREATE TABLE employees (
    employee_id   SERIAL PRIMARY KEY,
    first_name    VARCHAR(50) NOT NULL,
    last_name     VARCHAR(50) NOT NULL,
    email         VARCHAR(100),
    salary        NUMERIC(10, 2),
    department_id INTEGER REFERENCES departments(department_id)
);

CREATE TABLE customers (
    customer_id  SERIAL PRIMARY KEY,
    first_name   VARCHAR(50) NOT NULL,
    last_name    VARCHAR(50) NOT NULL,
    email        VARCHAR(100)
);

CREATE TABLE products (
    product_id   SERIAL PRIMARY KEY,
    product_name VARCHAR(100) NOT NULL,
    price        NUMERIC(10, 2),
    category     VARCHAR(50)
);

CREATE TABLE orders (
    order_id    SERIAL PRIMARY KEY,
    customer_id INTEGER REFERENCES customers(customer_id),
    product_id  INTEGER REFERENCES products(product_id),
    quantity    INTEGER,
    order_date  DATE
);
```

Now let us insert some data.

```sql
INSERT INTO departments (department_name, location) VALUES
('Engineering', 'Building A'),
('Marketing',   'Building B'),
('Sales',       'Building C'),
('HR',          'Building D');

INSERT INTO employees (first_name, last_name, email, salary, department_id) VALUES
```

```
('Alice',   'Johnson',  'alice@company.com',   85000, 1),
('Bob',     'Smith',    'bob@company.com',     72000, 1),
('Charlie', 'Brown',    'charlie@company.com', 65000, 2),
('Diana',   'Lee',      'diana@company.com',   78000, 3),
('Eve',     'Davis',    'eve@company.com',     90000, 1),
('Frank',   'Wilson',   'frank@company.com',   60000, NULL);

INSERT INTO customers (first_name, last_name, email) VALUES
('Grace',  'Taylor',   'grace@email.com'),
('Henry',  'Martinez', 'henry@email.com'),
('Ivy',    'Anderson', 'ivy@email.com'),
('Jack',   'Thomas',   'jack@email.com');

INSERT INTO products (product_name, price, category) VALUES
('Laptop',    999.99, 'Electronics'),
('Keyboard',   49.99, 'Electronics'),
('Desk Chair', 299.99, 'Furniture'),
('Monitor',   399.99, 'Electronics');

INSERT INTO orders (customer_id, product_id, quantity, order_date) VALUES
(1, 1, 1, '2024-01-15'),
(1, 2, 2, '2024-01-15'),
(2, 3, 1, '2024-02-20'),
(3, 1, 1, '2024-03-10'),
(3, 4, 1, '2024-03-10');
```

How INNER JOIN Works

INNER JOIN combines rows from two tables where the join condition is true. If a row in one table does not have a matching row in the other table, it is excluded from the results.

The Basic Syntax

```sql
SELECT columns
FROM table_a
INNER JOIN table_b
    ON table_a.column = table_b.column;
```

Think of it step by step:

1. Start with `table_a` (the "left" table)
2. For each row in `table_a`, look at every row in `table_b`
3. If the ON condition is true, combine those two rows into one result row
4. If no match is found, skip that row entirely

A Visual Way to Think About It

Imagine two circles in a Venn diagram. INNER JOIN returns only the overlapping area — the rows that exist in both tables.

```
    Table A           Table B
  +---------+     +---------+
  |         |     |         |
  |    +----+-----+----+    |
  |    |  INNER JOIN   |    |
  |    |  (matches)    |    |
  |    +----+-----+----+    |
  |         |     |         |
  +---------+     +---------+

  Only the middle section is returned.
```

Your First INNER JOIN

Let us find all employees along with their department names.

```sql
SELECT employees.first_name,
       employees.last_name,
       departments.department_name
FROM employees
INNER JOIN departments
    ON employees.department_id = departments.department_id;
```

Result:

```
+------------+-----------+-----------------+
| first_name | last_name | department_name |
+------------+-----------+-----------------+
| Alice      | Johnson   | Engineering     |
| Bob        | Smith     | Engineering     |
| Charlie    | Brown     | Marketing       |
| Diana      | Lee       | Sales           |
| Eve        | Davis     | Engineering     |
+------------+-----------+-----------------+
(5 rows)
```

Line-by-Line Explanation

```
SELECT employees.first_name,       -- Get first_name from employees table
       employees.last_name,        -- Get last_name from employees table
       departments.department_name -- Get department_name from departments table
FROM employees                     -- Start with the employees table
INNER JOIN departments             -- Combine with the departments table
    ON employees.department_id     -- Where the department_id in employees
     = departments.department_id;  -- matches department_id in departments
```

Notice: Frank Wilson does not appear in the results. His `department_id` is NULL, so there is no matching row in the departments table. The HR department also does not appear because no employee belongs to it. INNER JOIN only returns matches.

Table Aliases — Writing Cleaner Queries

Typing full table names like `employees.first_name` gets tedious. Table aliases let you give tables short nicknames.

```sql
SELECT e.first_name,
       e.last_name,
       d.department_name
FROM employees AS e
INNER JOIN departments AS d
    ON e.department_id = d.department_id;
```

The `AS` keyword is optional. This works exactly the same:

```sql
SELECT e.first_name,
       e.last_name,
       d.department_name
FROM employees e
INNER JOIN departments d
    ON e.department_id = d.department_id;
```

Result:

```
+------------+-----------+-----------------+
| first_name | last_name | department_name |
+------------+-----------+-----------------+
| Alice      | Johnson   | Engineering     |
| Bob        | Smith     | Engineering     |
| Charlie    | Brown     | Marketing       |
```

```
| Diana      | Lee       | Sales           |
| Eve        | Davis     | Engineering     |
+------------+-----------+-----------------+
(5 rows)
```

Common alias conventions:

- `employees` becomes `e`
- `departments` becomes `d`
- `customers` becomes `c`
- `orders` becomes `o`
- `products` becomes `p`

Think of it like a nickname. Your friend "Alexander" might go by "Alex." The table `employees` goes by `e`.

Joining on Foreign Keys

The ON clause almost always matches a foreign key in one table with the primary key in another. This is the glue that holds relational databases together.

```
employees table                    departments table
+-------------+---------------+    +---------------+-----------------+
| employee_id | department_id |    | department_id | department_name |
+-------------+---------------+    +---------------+-----------------+
| 1           | 1          ---+--->| 1             | Engineering     |
| 2           | 1          ---+--->| 1             | Engineering     |
| 3           | 2          ---+--->| 2             | Marketing       |
| 4           | 3          ---+--->| 3             | Sales           |
| 5           | 1          ---+--->| 1             | Engineering     |
| 6           | NULL (no match)|  | 4             | HR              |
+-------------+---------------+    +---------------+-----------------+

The arrows show FK -> PK relationships.
INNER JOIN follows these arrows to combine rows.
```

The foreign key (`employees.department_id`) points to the primary key (`departments.department_id`). INNER JOIN follows these references to combine the data.

JOIN with WHERE — Filtering Joined Results

You can add a WHERE clause after a JOIN to filter the combined results.

Find all Engineering employees:

```sql
SELECT e.first_name,
       e.last_name,
       d.department_name
FROM employees e
INNER JOIN departments d
    ON e.department_id = d.department_id
WHERE d.department_name = 'Engineering';
```

Result:

```
+------------+-----------+-----------------+
| first_name | last_name | department_name |
+------------+-----------+-----------------+
| Alice      | Johnson   | Engineering     |
| Bob        | Smith     | Engineering     |
| Eve        | Davis     | Engineering     |
+------------+-----------+-----------------+
(3 rows)
```

Line-by-Line Explanation

```
SELECT e.first_name,              -- Columns we want
       e.last_name,
       d.department_name
FROM employees e                  -- Start with employees
INNER JOIN departments d          -- Combine with departments
    ON e.department_id            -- Match on department_id
     = d.department_id
WHERE d.department_name           -- Then filter: only keep rows
    = 'Engineering';              -- where department is Engineering
```

The order of operations matters:

1. First, the JOIN combines the tables
2. Then, the WHERE filters the combined results

JOIN with ORDER BY and LIMIT

Find the top 3 highest-paid employees and their departments:

```sql
SELECT e.first_name,
       e.last_name,
       e.salary,
       d.department_name
FROM employees e
INNER JOIN departments d
    ON e.department_id = d.department_id
ORDER BY e.salary DESC
LIMIT 3;
```

Result:

```
+------------+-----------+----------+-----------------+
| first_name | last_name | salary   | department_name |
+------------+-----------+----------+-----------------+
| Eve        | Davis     | 90000.00 | Engineering     |
| Alice      | Johnson   | 85000.00 | Engineering     |
| Diana      | Lee       | 78000.00 | Sales           |
+------------+-----------+----------+-----------------+
(3 rows)
```

The execution order is: FROM -> JOIN -> WHERE -> ORDER BY -> LIMIT. Think of it as a pipeline: first you gather all the data, then you sort it, then you chop off the extras.

Multi-Table Joins — Joining Three or More Tables

Real applications often need data from three, four, or even more tables. You simply chain multiple JOIN clauses together.

Orders with Customer Names and Product Names

```sql
SELECT o.order_id,
       c.first_name || ' ' || c.last_name AS customer_name,
       p.product_name,
       o.quantity,
       p.price,
```

```
       (o.quantity * p.price) AS total,
       o.order_date
FROM orders o
INNER JOIN customers c
    ON o.customer_id = c.customer_id
INNER JOIN products p
    ON o.product_id = p.product_id;
```

Result:

```
+----------+----------------+--------------+----------+--------+---------+------------+
| order_id | customer_name  | product_name | quantity | price  | total   | order_date |
+----------+----------------+--------------+----------+--------+---------+------------+
|        1 | Grace Taylor   | Laptop       |        1 | 999.99 |  999.99 | 2024-01-15 |
|        2 | Grace Taylor   | Keyboard     |        2 |  49.99 |   99.98 | 2024-01-15 |
|        3 | Henry Martinez | Desk Chair   |        1 | 299.99 |  299.99 | 2024-02-20 |
|        4 | Ivy Anderson   | Laptop       |        1 | 999.99 |  999.99 | 2024-03-10 |
|        5 | Ivy Anderson   | Monitor      |        1 | 399.99 |  399.99 | 2024-03-10 |
+----------+----------------+--------------+----------+--------+---------+------------+
(5 rows)
```

Line-by-Line Explanation

```
SELECT o.order_id,                                      -- Order ID from orders
       c.first_name || ' ' || c.last_name               -- Combine first + last name
           AS customer_name,                             -- Call it customer_name
       p.product_name,                                   -- Product name from products
       o.quantity,                                       -- Quantity from orders
       p.price,                                          -- Price from products
       (o.quantity * p.price) AS total,                  -- Calculate total cost
       o.order_date                                      -- Order date from orders
FROM orders o                                            -- Start with orders
INNER JOIN customers c                                   -- Join to customers
    ON o.customer_id = c.customer_id                     -- Match on customer_id
INNER JOIN products p                                    -- Also join to products
    ON o.product_id = p.product_id;                      -- Match on product_id
```

How Multi-Table Joins Work — Step by Step

```
Step 1: Start with orders
+----------+-------------+------------+----------+
| order_id | customer_id | product_id | quantity |
+----------+-------------+------------+----------+
|        1 |           1 |          1 |        1 |
+----------+-------------+------------+----------+
```

```
Step 2: Join with customers (match customer_id)
+----------+-------------+------------+----------+----------------+
| order_id | customer_id | product_id | quantity | customer_name  |
+----------+-------------+------------+----------+----------------+
|        1 |           1 |          1 |        1 | Grace Taylor   |
+----------+-------------+------------+----------+----------------+

Step 3: Join with products (match product_id)
+----------+----------------+--------------+----------+--------+
| order_id | customer_name  | product_name | quantity | price  |
+----------+----------------+--------------+----------+--------+
|        1 | Grace Taylor   | Laptop       |        1 | 999.99 |
+----------+----------------+--------------+----------+--------+
```

Each JOIN adds more columns from another table. Think of it like adding more pieces to a puzzle — each JOIN snaps in another piece of information.

Practical Example: Employee Directory

Let us build a full employee directory with department info, filtering, and sorting.

```sql
SELECT e.employee_id,
       e.first_name || ' ' || e.last_name AS full_name,
       e.email,
       d.department_name,
       d.location,
       e.salary
FROM employees e
INNER JOIN departments d
    ON e.department_id = d.department_id
WHERE e.salary > 70000
ORDER BY d.department_name, e.last_name;
```

Result:

```
+-------------+--------------+---------------------+-----------------+------------+----...
| employee_id | full_name    | email               | department_name | location   | sal...
+-------------+--------------+---------------------+-----------------+------------+----...
|           1 | Alice Johnson| alice@company.com   | Engineering     | Building A | 850...
|           5 | Eve Davis    | eve@company.com     | Engineering     | Building A | 900...
|           2 | Bob Smith    | bob@company.com     | Engineering     | Building A | 720...
|           4 | Diana Lee    | diana@company.com   | Sales           | Building C | 780...
+-------------+--------------+---------------------+-----------------+------------+----...
```

```
(4 rows)
```

Practical Example: Order Summary Report

Create a report showing orders with all related information.

```sql
SELECT o.order_id,
       o.order_date,
       c.first_name || ' ' || c.last_name AS customer,
       c.email AS customer_email,
       p.product_name,
       p.category,
       o.quantity,
       p.price AS unit_price,
       (o.quantity * p.price) AS line_total
FROM orders o
INNER JOIN customers c ON o.customer_id = c.customer_id
INNER JOIN products p  ON o.product_id  = p.product_id
WHERE o.order_date >= '2024-02-01'
ORDER BY o.order_date, o.order_id;
```

Result:

```
+----------+------------+----------------+------------------+--------------+-----------...
| order_id | order_date | customer       | customer_email   | product_name | category  ...
+----------+------------+----------------+------------------+--------------+-----------...
|        3 | 2024-02-20 | Henry Martinez | henry@email.com  | Desk Chair   | Furniture ...
|        4 | 2024-03-10 | Ivy Anderson   | ivy@email.com    | Laptop       | Electronic...
|        5 | 2024-03-10 | Ivy Anderson   | ivy@email.com    | Monitor      | Electronic...
+----------+------------+----------------+------------------+--------------+-----------...
(3 rows)
```

JOIN vs. Comma Syntax (Old Style — Avoid It)

You might see older SQL that uses commas instead of JOIN:

```sql
-- Old style (avoid this)
SELECT e.first_name, d.department_name
FROM employees e, departments d
WHERE e.department_id = d.department_id;

-- Modern style (use this)
SELECT e.first_name, d.department_name
FROM employees e
INNER JOIN departments d
    ON e.department_id = d.department_id;
```

Both produce the same result, but the modern JOIN syntax is:

- Clearer — the join condition is separate from the filter condition
- Safer — if you forget the WHERE clause with comma syntax, you get a cartesian product (every row combined with every other row)
- Standard — supported by all modern databases

Common Mistakes

Mistake 1: Forgetting the ON Clause

```sql
-- WRONG: This will cause an error or unexpected results
SELECT e.first_name, d.department_name
FROM employees e
INNER JOIN departments d;
-- Missing ON clause!
```

Fix: Always include the ON clause to specify how the tables relate.

```sql
-- CORRECT
SELECT e.first_name, d.department_name
FROM employees e
INNER JOIN departments d
    ON e.department_id = d.department_id;
```

Mistake 2: Ambiguous Column Names

```sql
-- WRONG: department_id exists in both tables
SELECT first_name, department_id
FROM employees e
INNER JOIN departments d
    ON e.department_id = d.department_id;
-- ERROR: column reference "department_id" is ambiguous
```

Fix: Always prefix the column with the table alias.

```sql
-- CORRECT
SELECT e.first_name, e.department_id
FROM employees e
INNER JOIN departments d
    ON e.department_id = d.department_id;
```

Mistake 3: Expecting Non-Matching Rows to Appear

```sql
-- This will NOT show Frank (department_id is NULL)
-- This will NOT show HR (no employees)
SELECT e.first_name, d.department_name
FROM employees e
INNER JOIN departments d
    ON e.department_id = d.department_id;
```

Fix: If you need non-matching rows, use LEFT JOIN or RIGHT JOIN (next chapter).

Mistake 4: Wrong Join Column

```sql
-- WRONG: Joining on the wrong columns
SELECT e.first_name, d.department_name
FROM employees e
INNER JOIN departments d
    ON e.employee_id = d.department_id;
-- This joins employee_id to department_id — nonsense!
```

Fix: Make sure you join on the foreign key and the primary key it references.

Best Practices

1. Always use table aliases. They make queries shorter and easier to read. Use meaningful single letters (e for employees, d for departments).
2. Always qualify column names. Write `e.first_name` instead of just `first_name`, even if the column name is unique. It makes the query self-documenting.
3. Use the modern JOIN syntax. Write `INNER JOIN ... ON` instead of comma-separated tables with WHERE. It is clearer and less error-prone.
4. Join on foreign key to primary key. This is the standard pattern. If you find yourself joining on other columns, double-check your logic.
5. Put each JOIN on its own line. When you have multiple joins, put each one on a separate line for readability.
6. Think about what gets excluded. INNER JOIN drops non-matching rows. Always ask yourself: "Is it okay if some rows are missing from the results?"

Quick Summary

Concept	Description
INNER JOIN	Returns only rows that have matches in both tables
ON clause	Specifies the condition for matching rows
Table alias	A short name for a table (e.g., `employees e`)
Foreign key join	Matching FK in one table to PK in another
Multi-table join	Chaining multiple JOIN clauses in one query
JOIN + WHERE	First join, then filter the combined results

Key Points

- INNER JOIN returns only matching rows. If there is no match, the row is excluded.
- The ON clause defines how to match rows between tables, usually using a foreign key.
- Table aliases (`FROM employees e`) make your SQL shorter and more readable.
- You can chain multiple INNER JOINs to combine three or more tables.
- WHERE, ORDER BY, and LIMIT work with JOINs just like with single tables.

- INNER JOIN is the default join type. Writing `JOIN` without `INNER` works the same way.
- If you need rows that do NOT have matches, INNER JOIN is the wrong choice. Use LEFT JOIN instead.

Practice Questions

Question 1: What is the difference between the ON clause and the WHERE clause in a JOIN query?

<details> <summary>Answer</summary>

The ON clause specifies how to match rows between two tables (the join condition). The WHERE clause filters the results after the join has been performed. For INNER JOIN, you can technically put conditions in either place and get the same result, but best practice is to use ON for the join condition and WHERE for filtering.

</details>

Question 2: Why does Frank Wilson not appear in the results when we INNER JOIN employees with departments?

<details> <summary>Answer</summary>

Frank's `department_id` is NULL. INNER JOIN only returns rows where the ON condition finds a match. Since NULL does not equal any value (not even another NULL), there is no matching department for Frank, so his row is excluded.

</details>

Question 3: Write a query to find all orders placed by "Grace Taylor" and show the product names.

<details> <summary>Answer</summary>

```sql
SELECT o.order_id,
       p.product_name,
       o.quantity,
       o.order_date
FROM orders o
INNER JOIN customers c ON o.customer_id = c.customer_id
INNER JOIN products p  ON o.product_id  = p.product_id
WHERE c.first_name = 'Grace'
  AND c.last_name  = 'Taylor';
```

</details>

Question 4: What happens if you write JOIN instead of INNER JOIN?

<details> <summary>Answer</summary>

Nothing different. `JOIN` is shorthand for `INNER JOIN`. They produce exactly the same result. Most developers use `INNER JOIN` for clarity, but `JOIN` alone is perfectly valid.

</details>

Question 5: Can you join a table to itself? What would that be called?

<details> <summary>Answer</summary>

Yes, you can. It is called a self join. You would give the same table two different aliases. We will cover this in Chapter 15.

</details>

Exercises

Exercise 1: Employee Department Report

Write a query that shows each employee's full name (first and last combined), their email, department name, and location. Sort the results by department name, then by last name.

<details> <summary>Solution</summary>

```sql
SELECT e.first_name || ' ' || e.last_name AS full_name,
       e.email,
       d.department_name,
       d.location
FROM employees e
INNER JOIN departments d
    ON e.department_id = d.department_id
ORDER BY d.department_name, e.last_name;
```

Expected output:

```
+---------------+---------------------+-----------------+------------+
| full_name     | email               | department_name | location   |
+---------------+---------------------+-----------------+------------+
| Eve Davis     | eve@company.com     | Engineering     | Building A |
```

```
| Alice Johnson | alice@company.com   | Engineering     | Building A |
| Bob Smith     | bob@company.com     | Engineering     | Building A |
| Charlie Brown | charlie@company.com | Marketing       | Building B |
| Diana Lee     | diana@company.com   | Sales           | Building C |
+---------------+---------------------+-----------------+------------+
```

</details>

Exercise 2: Customer Order Summary

Write a query that shows each customer's name, the products they ordered, quantities, and the total cost for each line item. Only show orders from March 2024 or later. Sort by order date.

<details> <summary>Solution</summary>

```sql
SELECT c.first_name || ' ' || c.last_name AS customer_name,
       p.product_name,
       o.quantity,
       p.price,
       (o.quantity * p.price) AS total_cost,
       o.order_date
FROM orders o
INNER JOIN customers c ON o.customer_id = c.customer_id
INNER JOIN products p  ON o.product_id  = p.product_id
WHERE o.order_date >= '2024-03-01'
ORDER BY o.order_date;
```

Expected output:

```
+---------------+--------------+----------+--------+------------+------------+
| customer_name | product_name | quantity | price  | total_cost | order_date |
+---------------+--------------+----------+--------+------------+------------+
| Ivy Anderson  | Laptop       |        1 | 999.99 |     999.99 | 2024-03-10 |
| Ivy Anderson  | Monitor      |        1 | 399.99 |     399.99 | 2024-03-10 |
+---------------+--------------+----------+--------+------------+------------+
```

</details>

Exercise 3: Department Salary Report

Write a query that shows each department's name and the number of employees in it, along with the average salary. Only show departments that have more than one employee. Sort by average salary descending.

<details> <summary>Solution</summary>

```sql
SELECT d.department_name,
       COUNT(e.employee_id) AS employee_count,
       ROUND(AVG(e.salary), 2) AS avg_salary
FROM employees e
INNER JOIN departments d
    ON e.department_id = d.department_id
GROUP BY d.department_name
HAVING COUNT(e.employee_id) > 1
ORDER BY avg_salary DESC;
```

Expected output:

```
+-----------------+----------------+------------+
| department_name | employee_count | avg_salary |
+-----------------+----------------+------------+
| Engineering     |              3 |   82333.33 |
+-----------------+----------------+------------+
```

</details>

What Is Next?

INNER JOIN is powerful, but it has a limitation: it drops rows that do not have matches. What if you want to see ALL employees, even those without a department? Or ALL departments, even those with no employees?

In the next chapter, you will learn about LEFT JOIN and RIGHT JOIN — join types that keep non-matching rows and fill in NULL for the missing data. This is essential for finding gaps in your data, like customers who have never placed an order.

Chapter 14: LEFT JOIN and RIGHT JOIN — Keeping Non-Matching Rows

What You Will Learn

- How LEFT JOIN works and why it keeps all rows from the left table
- How RIGHT JOIN works and why it keeps all rows from the right table
- What happens to non-matching rows (they get NULLs)
- How to find rows WITHOUT matches using WHERE ... IS NULL
- Practical patterns: customers without orders, departments without employees
- Why LEFT JOIN and RIGHT JOIN are interchangeable (just swap the table order)

Why This Chapter Matters

In the previous chapter, you learned that INNER JOIN drops any row that does not have a match. But sometimes the missing data IS the important data.

Think about a classroom. INNER JOIN is like calling roll and only listing students who are present. But what if you need to know who is absent? That is what LEFT JOIN is for — it keeps everyone from your list, even if they did not show up.

In real applications, LEFT JOIN is one of the most frequently used join types. Finding customers who have not placed orders, employees without assigned projects, or products that have never been sold — all of these require LEFT JOIN.

Quick Refresher: Our Practice Tables

We will continue using the same tables from Chapter 13. Here is a quick reminder of the data:

```
employees:                                  departments:
+----+---------+-----------+--------+      +----+-----------------+------------+
| id | first   | last      | dep_id |      | id | department_name | location   |
+----+---------+-----------+--------+      +----+-----------------+------------+
|  1 | Alice   | Johnson   |      1 |      |  1 | Engineering     | Building A |
|  2 | Bob     | Smith     |      1 |      |  2 | Marketing       | Building B |
|  3 | Charlie | Brown     |      2 |      |  3 | Sales           | Building C |
|  4 | Diana   | Lee       |      3 |      |  4 | HR              | Building D |
|  5 | Eve     | Davis     |      1 |      +----+-----------------+------------+
|  6 | Frank   | Wilson    | NULL   |
+----+---------+-----------+--------+

customers:                        orders:
+----+-------+-----------+        +----+------+------+-----+------------+
| id | first | last      |        | id | c_id | p_id | qty | order_date |
+----+-------+-----------+        +----+------+------+-----+------------+
|  1 | Grace | Taylor    |        |  1 |    1 |    1 |   1 | 2024-01-15 |
|  2 | Henry | Martinez  |        |  2 |    1 |    2 |   2 | 2024-01-15 |
|  3 | Ivy   | Anderson  |        |  3 |    2 |    3 |   1 | 2024-02-20 |
|  4 | Jack  | Thomas    |        |  4 |    3 |    1 |   1 | 2024-03-10 |
+----+-------+-----------+        |  5 |    3 |    4 |   1 | 2024-03-10 |
                                   +----+------+------+-----+------------+

Notice: Jack Thomas (customer 4) has NO orders.
Notice: Frank Wilson (employee 6) has NO department.
Notice: HR (department 4) has NO employees.
```

How LEFT JOIN Works

LEFT JOIN returns all rows from the left table (the one after FROM), plus any matching rows from the right table (the one after JOIN). If there is no match, the columns from the right table are filled with NULL.

The Syntax

```sql
SELECT columns
FROM left_table
LEFT JOIN right_table
    ON left_table.column = right_table.column;
```

Visual Representation

```
    Left Table        Right Table
  +-----------+     +-----------+
  |           |     |           |
  |  +--------+-----+----+     |
  |  | matches|     |    |     |
  |  +--------+-----+----+     |
  |  kept     |     |  dropped |
  |  (NULLs)  |     |          |
  +-----------+     +-----------+

  Everything in the left circle is returned.
  The right side only contributes matching rows.
```

Your First LEFT JOIN

Let us find all employees and their departments — including employees without a department.

```sql
SELECT e.first_name,
       e.last_name,
       d.department_name
FROM employees e
LEFT JOIN departments d
    ON e.department_id = d.department_id;
```

Result:

```
+------------+-----------+-----------------+
| first_name | last_name | department_name |
+------------+-----------+-----------------+
| Alice      | Johnson   | Engineering     |
| Bob        | Smith     | Engineering     |
| Charlie    | Brown     | Marketing       |
| Diana      | Lee       | Sales           |
| Eve        | Davis     | Engineering     |
| Frank      | Wilson    | NULL            |
+------------+-----------+-----------------+
(6 rows)
```

Line-by-Line Explanation

```
SELECT e.first_name,          -- Get first_name from employees
       e.last_name,           -- Get last_name from employees
       d.department_name      -- Get department_name from departments
FROM employees e              -- Start with ALL employees (left table)
LEFT JOIN departments d       -- Try to match with departments (right table)
    ON e.department_id        -- Where department_id in employees
     = d.department_id;       -- matches department_id in departments
```

Compare with INNER JOIN:

Employee	INNER JOIN	LEFT JOIN
Alice	Shows	Shows
Bob	Shows	Shows
Charlie	Shows	Shows
Diana	Shows	Shows
Eve	Shows	Shows
Frank	Missing	Shows (NULL department)

Frank appears in LEFT JOIN but not in INNER JOIN. His department columns are filled with NULL because there is no matching department.

How RIGHT JOIN Works

RIGHT JOIN is the mirror image of LEFT JOIN. It returns all rows from the right table (the one after JOIN), plus any matching rows from the left table. If there is no match, the columns from the left table are filled with NULL.

The Syntax

```sql
SELECT columns
FROM left_table
RIGHT JOIN right_table
    ON left_table.column = right_table.column;
```

Visual Representation

```
    Left Table        Right Table
  +-----------+     +-----------+
  |           |     |           |
  |    +------+-----+------+   |
  |    | matches     |     |   |
  |    +------+-----+------+   |
  |  dropped  |     |  kept    |
  |           |     |  (NULLs) |
  +-----------+     +-----------+

  Everything in the right circle is returned.
  The left side only contributes matching rows.
```

RIGHT JOIN Example

Find all departments and their employees — including departments with no employees.

```sql
SELECT e.first_name,
       e.last_name,
       d.department_name
FROM employees e
RIGHT JOIN departments d
    ON e.department_id = d.department_id;
```

Result:

```
+------------+-----------+-----------------+
| first_name | last_name | department_name |
+------------+-----------+-----------------+
| Alice      | Johnson   | Engineering     |
| Bob        | Smith     | Engineering     |
| Eve        | Davis     | Engineering     |
| Charlie    | Brown     | Marketing       |
| Diana      | Lee       | Sales           |
| NULL       | NULL      | HR              |
+------------+-----------+-----------------+
(6 rows)
```

Notice that the HR department appears even though no employee belongs to it. The employee columns (first_name, last_name) are NULL for that row.

LEFT JOIN vs RIGHT JOIN — They Are Interchangeable

Here is an important insight: any RIGHT JOIN can be rewritten as a LEFT JOIN by swapping the table order.

These two queries produce the exact same result:

```sql
-- Using RIGHT JOIN
SELECT e.first_name, d.department_name
FROM employees e
RIGHT JOIN departments d
    ON e.department_id = d.department_id;

-- Same result using LEFT JOIN (tables swapped)
SELECT e.first_name, d.department_name
FROM departments d
LEFT JOIN employees e
    ON d.department_id = e.department_id;
```

Both return all departments, including those without employees.

In practice, most developers use LEFT JOIN exclusively and simply adjust which table comes first. This keeps your queries consistent and easier to read.

```
RIGHT JOIN:  A RIGHT JOIN B  =  B LEFT JOIN A

Just swap the table order and change RIGHT to LEFT.
```

NULL for Non-Matching Rows

When LEFT JOIN (or RIGHT JOIN) finds no match, it fills in NULL for every column from the unmatched table. This is important to understand because NULL behaves differently from regular values.

```sql
SELECT e.first_name,
       e.last_name,
       d.department_name,
       d.location
```

```
FROM employees e
LEFT JOIN departments d
    ON e.department_id = d.department_id;
```

Result:

```
+------------+-----------+-----------------+------------+
| first_name | last_name | department_name | location   |
+------------+-----------+-----------------+------------+
| Alice      | Johnson   | Engineering     | Building A |
| Bob        | Smith     | Engineering     | Building A |
| Charlie    | Brown     | Marketing       | Building B |
| Diana      | Lee       | Sales           | Building C |
| Eve        | Davis     | Engineering     | Building A |
| Frank      | Wilson    | NULL            | NULL       |
+------------+-----------+-----------------+------------+
(6 rows)
```

For Frank, both department_name and location are NULL. Every column from the right table becomes NULL when there is no match.

Finding Rows WITHOUT Matches — The Anti-Join Pattern

One of the most powerful uses of LEFT JOIN is finding rows that do not have a match. This is called an anti-join pattern.

The trick: Use LEFT JOIN, then filter for rows where the right table's key IS NULL.

Pattern: Find Customers Without Orders

```sql
SELECT c.first_name,
       c.last_name,
       c.email
FROM customers c
LEFT JOIN orders o
    ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

Result:

```
+------------+-----------+------------------+
| first_name | last_name | email            |
+------------+-----------+------------------+
| Jack       | Thomas    | jack@email.com   |
+------------+-----------+------------------+
(1 row)
```

Line-by-Line Explanation

```
SELECT c.first_name,           -- Customer first name
       c.last_name,            -- Customer last name
       c.email                 -- Customer email
FROM customers c               -- Start with ALL customers
LEFT JOIN orders o             -- Try to match with orders
    ON c.customer_id           -- Match on customer_id
     = o.customer_id
WHERE o.order_id IS NULL;      -- Keep ONLY rows where no order exists
```

How It Works — Step by Step

```
Step 1: LEFT JOIN (all customers, matched orders)

+-------+---------+----------+
| c_id  | c_name  | o_id     |
+-------+---------+----------+
|     1 | Grace   |        1 |   <- has order
|     1 | Grace   |        2 |   <- has order
|     2 | Henry   |        3 |   <- has order
|     3 | Ivy     |        4 |   <- has order
|     3 | Ivy     |        5 |   <- has order
|     4 | Jack    |     NULL |   <- NO order (NULL from LEFT JOIN)
+-------+---------+----------+

Step 2: WHERE o.order_id IS NULL (filter to unmatched)

+-------+---------+----------+
| c_id  | c_name  | o_id     |
+-------+---------+----------+
|     4 | Jack    |     NULL |   <- Only Jack remains
+-------+---------+----------+
```

Pattern: Find Departments Without Employees

```sql
SELECT d.department_name,
       d.location
FROM departments d
LEFT JOIN employees e
    ON d.department_id = e.department_id
WHERE e.employee_id IS NULL;
```

Result:

```
+-----------------+------------+
| department_name | location   |
+-----------------+------------+
| HR              | Building D |
+-----------------+------------+
(1 row)
```

The HR department has no employees, so it shows up when we look for unmatched rows.

Pattern: Find Employees Without a Department

```sql
SELECT e.first_name,
       e.last_name
FROM employees e
LEFT JOIN departments d
    ON e.department_id = d.department_id
WHERE d.department_id IS NULL;
```

Result:

```
+------------+-----------+
| first_name | last_name |
+------------+-----------+
| Frank      | Wilson    |
+------------+-----------+
(1 row)
```

The Anti-Join Pattern Explained Visually

```
Full LEFT JOIN result:
+------------------+-------------------+
```

```
| Left table rows  | Right table match |
+------------------+-------------------+
| Row A            | Match found       |  <- WHERE right.id IS NOT NULL
| Row B            | Match found       |  <- WHERE right.id IS NOT NULL
| Row C            | NULL (no match)   |  <- WHERE right.id IS NULL <<<
| Row D            | Match found       |  <- WHERE right.id IS NOT NULL
| Row E            | NULL (no match)   |  <- WHERE right.id IS NULL <<<
+------------------+-------------------+

Anti-join (WHERE right.id IS NULL) returns ONLY:
+------------------+-------------------+
| Row C            | NULL (no match)   |
| Row E            | NULL (no match)   |
+------------------+-------------------+

These are the "orphaned" rows — left table rows with no match.
```

LEFT JOIN with Multiple Tables

You can chain LEFT JOINs just like INNER JOINs.

```sql
SELECT c.first_name || ' ' || c.last_name AS customer_name,
       o.order_id,
       p.product_name,
       o.quantity,
       o.order_date
FROM customers c
LEFT JOIN orders o   ON c.customer_id = o.customer_id
LEFT JOIN products p ON o.product_id  = p.product_id
ORDER BY c.last_name, o.order_date;
```

Result:

```
+----------------+----------+--------------+----------+------------+
| customer_name  | order_id | product_name | quantity | order_date |
+----------------+----------+--------------+----------+------------+
| Ivy Anderson   |        4 | Laptop       |        1 | 2024-03-10 |
| Ivy Anderson   |        5 | Monitor      |        1 | 2024-03-10 |
| Henry Martinez |        3 | Desk Chair   |        1 | 2024-02-20 |
| Grace Taylor   |        1 | Laptop       |        1 | 2024-01-15 |
| Grace Taylor   |        2 | Keyboard     |        2 | 2024-01-15 |
| Jack Thomas    |     NULL | NULL         |     NULL | NULL       |
+----------------+----------+--------------+----------+------------+
(6 rows)
```

Jack Thomas appears with NULLs for all order and product columns because he has no orders.

Practical Example: Customer Activity Report

This report shows every customer with their order count and total spending, including customers who have not ordered anything.

```sql
SELECT c.first_name || ' ' || c.last_name AS customer_name,
       c.email,
       COUNT(o.order_id) AS total_orders,
       COALESCE(SUM(o.quantity * p.price), 0) AS total_spent
FROM customers c
LEFT JOIN orders o   ON c.customer_id = o.customer_id
LEFT JOIN products p ON o.product_id  = p.product_id
GROUP BY c.customer_id, c.first_name, c.last_name, c.email
ORDER BY total_spent DESC;
```

Result:

```
+----------------+------------------+--------------+-------------+
| customer_name  | email            | total_orders | total_spent |
+----------------+------------------+--------------+-------------+
| Ivy Anderson   | ivy@email.com    |            2 |     1399.98 |
| Grace Taylor   | grace@email.com  |            2 |     1099.97 |
| Henry Martinez | henry@email.com  |            1 |      299.99 |
| Jack Thomas    | jack@email.com   |            0 |        0.00 |
+----------------+------------------+--------------+-------------+
(4 rows)
```

Line-by-Line Explanation

```
SELECT c.first_name || ' ' || c.last_name   -- Combine into full name
           AS customer_name,
       c.email,                               -- Customer email
       COUNT(o.order_id) AS total_orders,     -- Count orders (NULL not counted)
       COALESCE(                              -- If the sum is NULL...
           SUM(o.quantity * p.price), 0       -- ...replace with 0
       ) AS total_spent
FROM customers c                              -- Start with ALL customers
LEFT JOIN orders o                            -- Match with orders
    ON c.customer_id = o.customer_id
```

```
LEFT JOIN products p                          -- Match with products
    ON o.product_id  = p.product_id
GROUP BY c.customer_id, c.first_name,         -- Group by customer
         c.last_name, c.email
ORDER BY total_spent DESC;                    -- Highest spenders first
```

Key detail: `COUNT(o.order_id)` counts only non-NULL values. Since Jack has no orders, his order_id is NULL and the count is 0. We use `COALESCE` to turn a NULL sum into 0.

Practical Example: Department Staffing Overview

```sql
SELECT d.department_name,
       d.location,
       COUNT(e.employee_id) AS employee_count,
       COALESCE(ROUND(AVG(e.salary), 2), 0) AS avg_salary
FROM departments d
LEFT JOIN employees e
    ON d.department_id = e.department_id
GROUP BY d.department_id, d.department_name, d.location
ORDER BY employee_count DESC;
```

Result:

```
+-----------------+------------+----------------+------------+
| department_name | location   | employee_count | avg_salary |
+-----------------+------------+----------------+------------+
| Engineering     | Building A |              3 |   82333.33 |
| Marketing       | Building B |              1 |   65000.00 |
| Sales           | Building C |              1 |   78000.00 |
| HR              | Building D |              0 |       0.00 |
+-----------------+------------+----------------+------------+
(4 rows)
```

HR shows up with 0 employees and 0 average salary — exactly the kind of insight you need for staffing decisions.

Common Mistakes

Mistake 1: Using WHERE Instead of AND for Join Conditions

```sql
-- WRONG: This turns LEFT JOIN into INNER JOIN
SELECT c.first_name, o.order_id
FROM customers c
LEFT JOIN orders o
    ON c.customer_id = o.customer_id
WHERE o.order_date >= '2024-02-01';
-- Jack disappears! His order_date is NULL, and NULL >= '2024-02-01' is not true.
```

Fix: If you want to filter the right table but keep all left rows, put the condition in the ON clause.

```sql
-- CORRECT: Jack still appears with NULLs
SELECT c.first_name, o.order_id, o.order_date
FROM customers c
LEFT JOIN orders o
    ON c.customer_id = o.customer_id
   AND o.order_date >= '2024-02-01';
```

This is one of the trickiest aspects of LEFT JOIN. Conditions in WHERE filter after the join, potentially eliminating rows that LEFT JOIN was supposed to keep. Conditions in ON filter during the join, keeping all left rows regardless.

Mistake 2: Using = Instead of IS NULL to Check for NULLs

```sql
-- WRONG: This returns no rows
SELECT c.first_name
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_id = NULL;
-- NULL = NULL is not true! It evaluates to NULL.
```

Fix: Always use IS NULL.

```sql
-- CORRECT
SELECT c.first_name
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

Mistake 3: Checking the Wrong Column for IS NULL

```sql
-- WRONG: Checking a column that could legitimately be NULL
SELECT c.first_name
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE o.quantity IS NULL;
-- This finds both unmatched rows AND rows where quantity happens to be NULL
```

Fix: Always check the primary key or a NOT NULL column from the right table.

```sql
-- CORRECT: Check the primary key
SELECT c.first_name
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

Best Practices

1. Prefer LEFT JOIN over RIGHT JOIN. Most developers exclusively use LEFT JOIN and put the "must keep all rows" table first (after FROM). This is a widely accepted convention.
2. Use COALESCE for NULL values in aggregations. When LEFT JOIN produces NULLs, use `COALESCE(value, default)` to replace them with meaningful defaults.
3. Check the primary key for IS NULL in anti-joins. When finding unmatched rows, check the right table's primary key (or any NOT NULL column) for IS NULL. This avoids false positives.
4. Be careful with WHERE on LEFT JOINed tables. A WHERE clause on the right table can accidentally turn your LEFT JOIN into an INNER JOIN. Put the condition in the ON clause if you want to preserve all left rows.
5. Comment your intent. Add a comment like `-- Find customers WITHOUT orders` so future readers understand why you are using the anti-join pattern.

Quick Summary

Concept	Description

LEFT JOIN	All rows from left table, matching from right, NULLs for no match
RIGHT JOIN	All rows from right table, matching from left, NULLs for no match
Anti-join	LEFT JOIN + WHERE right.pk IS NULL to find unmatched rows
LEFT vs RIGHT	Interchangeable — just swap table order
WHERE vs ON	WHERE filters after join (can remove NULLs); ON filters during join
COALESCE	Replaces NULL with a default value

Key Points

- LEFT JOIN keeps all rows from the left table, even if there is no match in the right table.
- Non-matching rows get NULL for every column from the right table.
- The anti-join pattern (LEFT JOIN + WHERE IS NULL) is a powerful way to find orphaned or missing data.
- RIGHT JOIN is the mirror of LEFT JOIN. In practice, use LEFT JOIN and swap table order instead.
- Be careful with WHERE clauses on the right table — they can accidentally turn a LEFT JOIN into an INNER JOIN.
- Use COALESCE to replace NULLs with default values in your results.
- Always check the primary key (or a NOT NULL column) when using the IS NULL anti-join pattern.

Practice Questions

Question 1: What is the difference between INNER JOIN and LEFT JOIN?

<details> <summary>Answer</summary>

INNER JOIN returns only rows that have matches in both tables. LEFT JOIN returns all rows from the left table, plus matching rows from the right table. If there is no match, the right

table's columns are filled with NULL.

</details>

Question 2: Why does this query not find customers without orders?

```sql
SELECT c.first_name
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_id = NULL;
```

<details> <summary>Answer</summary>

Because you cannot compare with NULL using `=`. The expression `NULL = NULL` evaluates to NULL (not true), so no rows are returned. You must use `WHERE o.order_id IS NULL` instead.

</details>

Question 3: How can you rewrite a RIGHT JOIN as a LEFT JOIN?

<details> <summary>Answer</summary>

Swap the table order. `A RIGHT JOIN B ON ...` is the same as `B LEFT JOIN A ON ...`. Most developers prefer LEFT JOIN and adjust which table comes first.

</details>

Question 4: Why might a WHERE clause on the right table be problematic with LEFT JOIN?

<details> <summary>Answer</summary>

A WHERE clause filters results after the join. If the right table's columns are NULL (for non-matching rows), the WHERE condition will filter them out, effectively turning the LEFT JOIN into an INNER JOIN. To avoid this, put the condition in the ON clause instead.

</details>

Question 5: What is the anti-join pattern and when would you use it?

<details> <summary>Answer</summary>

The anti-join pattern is: LEFT JOIN followed by WHERE right_table.primary_key IS NULL. It finds rows in the left table that have no match in the right table. Use it to find missing relationships: customers without orders, departments without employees, products never sold, etc.

</details>

Exercises

Exercise 1: Products Never Ordered

Write a query to find all products that have never been ordered. Show the product name, price, and category.

<details> <summary>Solution</summary>

```sql
SELECT p.product_name,
       p.price,
       p.category
FROM products p
LEFT JOIN orders o
    ON p.product_id = o.product_id
WHERE o.order_id IS NULL;
```

Expected output (no products are missing in our sample data, so let us verify):

```
+--------------+-------+----------+
| product_name | price | category |
+--------------+-------+----------+
(0 rows)
```

All products have been ordered in our sample data. If we had a product with no orders, it would appear here.

</details>

Exercise 2: Complete Employee Directory

Write a query that shows ALL employees (including those without a department) and ALL departments (including those without employees). Use two separate queries — one LEFT JOIN for all employees and one LEFT JOIN for all departments.

<details> <summary>Solution</summary>

```sql
-- All employees with their departments (including Frank with no department)
SELECT e.first_name || ' ' || e.last_name AS employee_name,
       COALESCE(d.department_name, 'Unassigned') AS department
FROM employees e
LEFT JOIN departments d
    ON e.department_id = d.department_id
```

```
ORDER BY e.last_name;

-- All departments with employee count (including HR with 0)
SELECT d.department_name,
       COUNT(e.employee_id) AS employee_count
FROM departments d
LEFT JOIN employees e
    ON d.department_id = e.department_id
GROUP BY d.department_id, d.department_name
ORDER BY d.department_name;
```

Expected output for first query:

```
+---------------+-------------+
| employee_name | department  |
+---------------+-------------+
| Charlie Brown | Marketing   |
| Eve Davis     | Engineering |
| Alice Johnson | Engineering |
| Diana Lee     | Sales       |
| Bob Smith     | Engineering |
| Frank Wilson  | Unassigned  |
+---------------+-------------+
```

Expected output for second query:

```
+-----------------+----------------+
| department_name | employee_count |
+-----------------+----------------+
| Engineering     |              3 |
| HR              |              0 |
| Marketing       |              1 |
| Sales           |              1 |
+-----------------+----------------+
```

</details>

Exercise 3: Customer Order Gaps

Write a query that identifies customers who have NOT placed any orders in 2024. Show their name and email. (Hint: You need to check if the customer has no orders at all OR no orders in 2024.)

<details> <summary>Solution</summary>

```sql
SELECT c.first_name || ' ' || c.last_name AS customer_name,
       c.email
FROM customers c
LEFT JOIN orders o
    ON c.customer_id = o.customer_id
   AND o.order_date >= '2024-01-01'
   AND o.order_date < '2025-01-01'
WHERE o.order_id IS NULL;
```

Expected output:

```
+---------------+------------------+
| customer_name | email            |
+---------------+------------------+
| Jack Thomas   | jack@email.com   |
+---------------+------------------+
```

Note: The date filter is in the ON clause, not the WHERE clause. This ensures that customers without any orders still appear. If we put it in WHERE, it would turn into an INNER JOIN.

</details>

What Is Next?

You now know how to use INNER JOIN (matching rows only) and LEFT/RIGHT JOIN (keep all rows from one side). But what if you want to keep all rows from both sides? Or combine every row with every other row? Or join a table to itself?

In the next chapter, you will learn about FULL OUTER JOIN, CROSS JOIN, and SELF JOIN — three specialized join types that handle these less common but important scenarios.

Chapter 15: FULL OUTER JOIN, CROSS JOIN, and SELF JOIN

What You Will Learn

- How FULL OUTER JOIN keeps all rows from both tables, filling NULLs on both sides
- How CROSS JOIN creates every possible combination of rows (cartesian product)
- How SELF JOIN lets a table join to itself
- When to use CROSS JOIN in practice (size and color combinations, scheduling)
- When to use SELF JOIN in practice (employee-manager, finding duplicates)
- Why you should avoid NATURAL JOIN

Why This Chapter Matters

You have learned INNER JOIN (only matches) and LEFT/RIGHT JOIN (all from one side). This chapter covers three more specialized join types that solve problems the others cannot.

Think of these as specialty tools in a toolbox. You do not use them every day, but when you need them, nothing else will do:

- FULL OUTER JOIN is like comparing two guest lists and seeing everyone — attendees at both events, attendees at only the first event, and attendees at only the second event.
- CROSS JOIN is like a menu generator — if you have 3 sizes and 4 colors, it produces all 12 combinations.
- SELF JOIN is like an organizational chart — each employee row points to another employee row (their manager).

Setting Up Additional Practice Data

We will continue using the tables from previous chapters and add a few more for this chapter.

```sql
CREATE TABLE sizes (
    size_id   SERIAL PRIMARY KEY,
    size_name VARCHAR(20) NOT NULL
);

CREATE TABLE colors (
    color_id   SERIAL PRIMARY KEY,
    color_name VARCHAR(20) NOT NULL
);

INSERT INTO sizes (size_name) VALUES
('Small'), ('Medium'), ('Large');

INSERT INTO colors (color_name) VALUES
('Red'), ('Blue'), ('Green'), ('Black');
```

Let us also create a table where employees have managers (other employees).

```sql
CREATE TABLE staff (
    staff_id   SERIAL PRIMARY KEY,
    name       VARCHAR(50) NOT NULL,
    role       VARCHAR(50),
    manager_id INTEGER REFERENCES staff(staff_id)
);

INSERT INTO staff (name, role, manager_id) VALUES
('Sarah',   'CEO',              NULL),
('Tom',     'VP Engineering',   1),
('Lisa',    'VP Marketing',     1),
('Mike',    'Senior Developer', 2),
('Anna',    'Developer',        2),
('Jake',    'Marketing Lead',   3),
('Rachel',  'Designer',         3);
```

The staff table hierarchy looks like this:

```
Sarah (CEO)
├── Tom (VP Engineering)
│   ├── Mike (Senior Developer)
│   └── Anna (Developer)
└── Lisa (VP Marketing)
    ├── Jake (Marketing Lead)
    └── Rachel (Designer)
```

FULL OUTER JOIN — All Rows from Both Tables

FULL OUTER JOIN returns everything: all rows from the left table AND all rows from the right table. Where there is a match, it combines the rows. Where there is no match on either side, it fills in NULLs.

The Syntax

```sql
SELECT columns
FROM table_a
FULL OUTER JOIN table_b
    ON table_a.column = table_b.column;
```

Visual Representation

```
    Table A           Table B
  +-----------+     +-----------+
  |           |     |           |
  |  +--------+-----+--------+ |
  |  | matches      |        | |
  |  +--------+-----+--------+ |
  |  kept     |     |   kept   |
  |  (NULLs   |     |  (NULLs  |
  |  on right)|     | on left) |
  +-----------+     +-----------+

  EVERYTHING is returned. The entire Venn diagram.
```

FULL OUTER JOIN Example

```sql
SELECT e.first_name,
       e.last_name,
       d.department_name
FROM employees e
FULL OUTER JOIN departments d
    ON e.department_id = d.department_id;
```

Result:

```
+------------+-----------+-----------------+
| first_name | last_name | department_name |
+------------+-----------+-----------------+
| Alice      | Johnson   | Engineering     |
| Bob        | Smith     | Engineering     |
| Eve        | Davis     | Engineering     |
| Charlie    | Brown     | Marketing       |
| Diana      | Lee       | Sales           |
| Frank      | Wilson    | NULL            |
| NULL       | NULL      | HR              |
+------------+-----------+-----------------+
(7 rows)
```

Line-by-Line Explanation

```
SELECT e.first_name,          -- Employee first name (NULL if no employee)
       e.last_name,           -- Employee last name (NULL if no employee)
       d.department_name      -- Department name (NULL if no department)
FROM employees e              -- All employees
FULL OUTER JOIN departments d -- Combined with all departments
    ON e.department_id        -- Where department IDs match
     = d.department_id;
```

Compare all join types:

```
+--------+--------+------------+-----------+-----------+
| emp    | dept   | INNER JOIN | LEFT JOIN | FULL JOIN |
+--------+--------+------------+-----------+-----------+
| Alice  | Eng    | Yes        | Yes       | Yes       |
| Bob    | Eng    | Yes        | Yes       | Yes       |
| Eve    | Eng    | Yes        | Yes       | Yes       |
| Charlie| Mktg   | Yes        | Yes       | Yes       |
| Diana  | Sales  | Yes        | Yes       | Yes       |
| Frank  | (none) | NO         | Yes       | Yes       |
| (none) | HR     | NO         | NO        | Yes       |
+--------+--------+------------+-----------+-----------+
```

FULL OUTER JOIN catches both sides: Frank (no department) AND HR (no employees).

When to Use FULL OUTER JOIN

FULL OUTER JOIN is less common than INNER or LEFT JOIN, but it is useful when you need to find mismatches on both sides:

- Reconciling two data sources (find records in A not in B AND records in B not in A)
- Auditing relationships (find all orphaned records on both sides)

- Data migration validation (ensure both old and new tables have the same records)

```sql
-- Find all mismatches: employees without departments AND departments without employees
SELECT e.first_name,
       e.last_name,
       d.department_name,
       CASE
           WHEN e.employee_id IS NULL THEN 'Department has no employees'
           WHEN d.department_id IS NULL THEN 'Employee has no department'
           ELSE 'Matched'
       END AS status
FROM employees e
FULL OUTER JOIN departments d
    ON e.department_id = d.department_id
WHERE e.employee_id IS NULL
   OR d.department_id IS NULL;
```

Result:

```
+------------+-----------+-----------------+-----------------------------+
| first_name | last_name | department_name | status                      |
+------------+-----------+-----------------+-----------------------------+
| Frank      | Wilson    | NULL            | Employee has no department   |
| NULL       | NULL      | HR              | Department has no employees  |
+------------+-----------+-----------------+-----------------------------+
(2 rows)
```

CROSS JOIN — Every Possible Combination

CROSS JOIN combines every row from the first table with every row from the second table. If table A has 3 rows and table B has 4 rows, the result has 3 x 4 = 12 rows.

This is called a cartesian product. There is no ON clause because you are not matching anything — you want every combination.

The Syntax

```sql
SELECT columns
FROM table_a
CROSS JOIN table_b;
```

CROSS JOIN Example: Size and Color Combinations

```sql
SELECT s.size_name,
       c.color_name
FROM sizes s
CROSS JOIN colors c
ORDER BY s.size_id, c.color_id;
```

Result:

```
+-----------+------------+
| size_name | color_name |
+-----------+------------+
| Small     | Red        |
| Small     | Blue       |
| Small     | Green      |
| Small     | Black      |
| Medium    | Red        |
| Medium    | Blue       |
| Medium    | Green      |
| Medium    | Black      |
| Large     | Red        |
| Large     | Blue       |
| Large     | Green      |
| Large     | Black      |
+-----------+------------+
(12 rows)
```

3 sizes x 4 colors = 12 combinations. Every size paired with every color.

How CROSS JOIN Works Visually

```
Sizes:     Colors:
+--------+ +--------+
| Small  | | Red    |    Small-Red, Small-Blue, Small-Green, Small-Black
| Medium | | Blue   |    Medium-Red, Medium-Blue, Medium-Green, Medium-Black
| Large  | | Green  |    Large-Red, Large-Blue, Large-Green, Large-Black
+--------+ | Black  |
           +--------+

3 rows  x  4 rows  =  12 result rows
```

Practical Use: Product Variant Generator

```sql
SELECT s.size_name,
       c.color_name,
       'SKU-' || UPPER(LEFT(s.size_name, 1))
              || '-' || UPPER(LEFT(c.color_name, 3)) AS sku
FROM sizes s
CROSS JOIN colors c
ORDER BY s.size_name, c.color_name;
```

Result:

```
+-----------+------------+---------+
| size_name | color_name | sku     |
+-----------+------------+---------+
| Large     | Black      | SKU-L-BLA |
| Large     | Blue       | SKU-L-BLU |
| Large     | Green      | SKU-L-GRE |
| Large     | Red        | SKU-L-RED |
| Medium    | Black      | SKU-M-BLA |
| Medium    | Blue       | SKU-M-BLU |
| Medium    | Green      | SKU-M-GRE |
| Medium    | Red        | SKU-M-RED |
| Small     | Black      | SKU-S-BLA |
| Small     | Blue       | SKU-S-BLU |
| Small     | Green      | SKU-S-GRE |
| Small     | Red        | SKU-S-RED |
+-----------+------------+---------+
(12 rows)
```

Warning: CROSS JOIN Can Produce Huge Results

```
Table A: 1,000 rows
Table B: 1,000 rows
CROSS JOIN result: 1,000,000 rows!

Table A: 10,000 rows
Table B: 10,000 rows
CROSS JOIN result: 100,000,000 rows!
```

Always be aware of the table sizes before using CROSS JOIN. A cartesian product of two large tables can crash your database or consume all available memory.

SELF JOIN — A Table Joins Itself

A SELF JOIN is when a table is joined to itself. This sounds strange at first, but it is incredibly useful for hierarchical data (like org charts) or finding duplicates.

You must use different aliases for the same table, so the database can tell the two "copies" apart.

The Syntax

```sql
SELECT columns
FROM table_name alias_a
INNER JOIN table_name alias_b
    ON alias_a.column = alias_b.column;
```

Example: Employee-Manager Relationship

```sql
SELECT e.name AS employee,
       e.role AS employee_role,
       m.name AS manager,
       m.role AS manager_role
FROM staff e
LEFT JOIN staff m
    ON e.manager_id = m.staff_id;
```

Result:

```
+----------+-------------------+---------+-----------------+
| employee | employee_role     | manager | manager_role    |
+----------+-------------------+---------+-----------------+
| Sarah    | CEO               | NULL    | NULL            |
| Tom      | VP Engineering    | Sarah   | CEO             |
| Lisa     | VP Marketing      | Sarah   | CEO             |
| Mike     | Senior Developer  | Tom     | VP Engineering  |
| Anna     | Developer         | Tom     | VP Engineering  |
| Jake     | Marketing Lead    | Lisa    | VP Marketing    |
| Rachel   | Designer          | Lisa    | VP Marketing    |
+----------+-------------------+---------+-----------------+
(7 rows)
```

Line-by-Line Explanation

```
SELECT e.name AS employee,       -- The employee's name
       e.role AS employee_role,  -- The employee's role
```

```
       m.name AS manager,        -- The manager's name
       m.role AS manager_role    -- The manager's role
FROM staff e                     -- Treat staff as "employees" (alias e)
LEFT JOIN staff m                -- Join the SAME table as "managers" (alias m)
    ON e.manager_id              -- Where the employee's manager_id
     = m.staff_id;               -- matches the manager's staff_id
```

We use LEFT JOIN (not INNER JOIN) so Sarah (the CEO) still appears even though she has no manager (manager_id is NULL).

How Self Join Works Visually

```
staff table (as "employees"):        staff table (as "managers"):
+----+--------+------------+        +----+--------+
| id | name   | manager_id |        | id | name   |
+----+--------+------------+        +----+--------+
|  1 | Sarah  | NULL       |        |  1 | Sarah  |
|  2 | Tom    | 1       ---+------->|  1 | Sarah  |
|  3 | Lisa   | 1       ---+------->|  1 | Sarah  |
|  4 | Mike   | 2       ---+------->|  2 | Tom    |
|  5 | Anna   | 2       ---+------->|  2 | Tom    |
|  6 | Jake   | 3       ---+------->|  3 | Lisa   |
|  7 | Rachel | 3       ---+------->|  3 | Lisa   |
+----+--------+------------+        +----+--------+

Same table, two aliases, different roles in the query.
```

Example: Finding Employees Who Share the Same Manager

```sql
SELECT a.name AS employee_1,
       b.name AS employee_2,
       m.name AS shared_manager
FROM staff a
INNER JOIN staff b
    ON a.manager_id = b.manager_id
   AND a.staff_id < b.staff_id
INNER JOIN staff m
    ON a.manager_id = m.staff_id;
```

Result:

```
+------------+------------+----------------+
| employee_1 | employee_2 | shared_manager |
+------------+------------+----------------+
| Tom        | Lisa       | Sarah          |
```

```
| Mike       | Anna       | Tom            |
| Jake       | Rachel     | Lisa           |
+------------+------------+----------------+
(3 rows)
```

The trick `a.staff_id < b.staff_id` prevents duplicates. Without it, you would get "Tom-Lisa" AND "Lisa-Tom" as separate rows.

Example: Finding Duplicate Emails

Self join can find rows that share the same value in a column.

```sql
-- Using our employees table
SELECT a.first_name AS emp_1,
       b.first_name AS emp_2,
       a.email
FROM employees a
INNER JOIN employees b
    ON a.email = b.email
   AND a.employee_id < b.employee_id;
```

If any two employees share the same email address, this query will find them. In our sample data there are no duplicates, so the result would be empty. But in a real database with thousands of records, this is a valuable data quality check.

NATURAL JOIN — Know It, But Avoid It

NATURAL JOIN automatically matches columns that share the same name between two tables. It requires no ON clause.

```sql
-- NATURAL JOIN (avoid this)
SELECT e.first_name, d.department_name
FROM employees e
NATURAL JOIN departments d;
```

This is equivalent to:

```sql
-- Explicit JOIN (use this instead)
SELECT e.first_name, d.department_name
FROM employees e
INNER JOIN departments d
    ON e.department_id = d.department_id;
```

Why You Should Avoid NATURAL JOIN

1. Fragile. If someone adds a column with the same name to both tables, the join condition changes silently. Your query could break or return wrong results without any error message.
2. Implicit. You cannot tell what columns are being matched just by reading the query. You have to know the table schemas.
3. Dangerous. Two tables might share column names that are not meant to be joined on (like `name`, `status`, or `created_at`).

```
Scenario: Both tables have a "name" column

-- NATURAL JOIN will match on BOTH department_id AND name
-- This is almost certainly wrong!

Before: JOIN ON department_id           (correct)
After:  JOIN ON department_id AND name  (wrong!)
```

Rule of thumb: Always write explicit JOIN conditions with ON. It is a few more characters to type, but it makes your intent crystal clear and protects against schema changes.

Comparison: All Join Types at a Glance

```
+-------------------+-----------------------------------------+
| Join Type         | What It Returns                         |
+-------------------+-----------------------------------------+
| INNER JOIN        | Only matching rows                      |
| LEFT JOIN         | All from left + matching from right     |
| RIGHT JOIN        | All from right + matching from left     |
| FULL OUTER JOIN   | All from both sides                     |
| CROSS JOIN        | Every combination (no ON clause)        |
| SELF JOIN         | Table joined to itself (any join type)  |
| NATURAL JOIN      | Auto-matches same-named columns (avoid) |
+-------------------+-----------------------------------------+
```

Visual summary:

```
INNER:       LEFT:        RIGHT:       FULL OUTER:    CROSS:
  +--+         +----+       +----+       +------+      AxB
 /    \       /  +--+\     /+--+  \     / +----+ \    (all
| A&&B |     |A |A&B| |   | |A&B|B |   |A |A&&B| B|   combos)
 \    /       \  +--+/     \+--+  /     \ +----+ /
  +--+         +----+       +----+       +------+
```

Common Mistakes

Mistake 1: Accidental CROSS JOIN

```sql
-- WRONG: Forgot the ON clause — this creates a CROSS JOIN
SELECT e.first_name, d.department_name
FROM employees e, departments d;
-- Returns 6 x 4 = 24 rows (every combination)!
```

Fix: Always use explicit JOIN syntax with an ON clause.

Mistake 2: SELF JOIN Without Proper Aliases

```sql
-- WRONG: Ambiguous table references
SELECT name, manager_id
FROM staff
INNER JOIN staff ON manager_id = staff_id;
-- ERROR: table name "staff" specified more than once
```

Fix: Give each instance a unique alias.

```sql
-- CORRECT
SELECT e.name, m.name AS manager
FROM staff e
INNER JOIN staff m ON e.manager_id = m.staff_id;
```

Mistake 3: Forgetting the Duplicate Prevention in Self Joins

```sql
-- WRONG: Gets duplicates (Tom-Lisa AND Lisa-Tom)
SELECT a.name, b.name
FROM staff a
INNER JOIN staff b
    ON a.manager_id = b.manager_id;
-- Also includes Tom-Tom and Lisa-Lisa (self-pairs)!
```

Fix: Use `a.id < b.id` to get each pair only once and exclude self-pairs.

```sql
-- CORRECT
SELECT a.name, b.name
FROM staff a
INNER JOIN staff b
    ON a.manager_id = b.manager_id
   AND a.staff_id < b.staff_id;
```

Mistake 4: Using CROSS JOIN on Large Tables

```sql
-- DANGEROUS: Could produce millions of rows
SELECT *
FROM customers
CROSS JOIN orders;
-- With 10,000 customers and 50,000 orders = 500,000,000 rows!
```

Fix: Only use CROSS JOIN on small reference tables (sizes, colors, categories). Check row counts first.

Best Practices

1. Use FULL OUTER JOIN for reconciliation tasks. It is perfect for comparing two data sets and finding mismatches on both sides.
2. Be cautious with CROSS JOIN. Always check the row counts of both tables first. Use it only with small reference tables (under a few hundred rows).
3. Use meaningful aliases in self joins. Instead of `a` and `b`, use `e` (employee) and `m` (manager) to make the query self-documenting.
4. Use `a.id < b.id` in self joins when finding pairs to prevent duplicates and self-matches.
5. Never use NATURAL JOIN in production code. Always write explicit ON conditions.

6. Comment your self joins. They can be confusing to read. Add a comment explaining the relationship (e.g., `-- e = employee, m = their manager`).

Quick Summary

Concept	Description
FULL OUTER JOIN	Returns all rows from both tables, NULLs on both sides
CROSS JOIN	Every row from A combined with every row from B (cartesian product)
SELF JOIN	A table joined to itself using different aliases
NATURAL JOIN	Auto-matches same-named columns (avoid it)
`a.id < b.id`	Prevents duplicate pairs in self joins
Cartesian product	M rows x N rows = M*N result rows

Key Points

- FULL OUTER JOIN returns all rows from both tables. Use it when you need to find mismatches on both sides of a relationship.
- CROSS JOIN produces a cartesian product — every combination. It has no ON clause. Use it for generating combinations (sizes, colors, schedules).
- SELF JOIN lets a table reference itself. It requires two different aliases for the same table. Common use cases are hierarchies (employee-manager) and finding duplicates.
- NATURAL JOIN automatically matches same-named columns. Avoid it — it is implicit, fragile, and dangerous when schemas change.
- Always use `a.id < b.id` in self joins when finding pairs to eliminate duplicates and self-matches.
- CROSS JOIN on large tables can produce millions of rows. Always check table sizes first.

Practice Questions

Question 1: When would you choose FULL OUTER JOIN over LEFT JOIN?

<details> <summary>Answer</summary>

Use FULL OUTER JOIN when you need to find mismatches on both sides. LEFT JOIN only shows unmatched rows from the left table. FULL OUTER JOIN shows unmatched rows from both the left AND right tables. This is useful for data reconciliation and auditing.

</details>

Question 2: If table A has 5 rows and table B has 3 rows, how many rows does CROSS JOIN produce?

<details> <summary>Answer</summary>

5 x 3 = 15 rows. CROSS JOIN produces the cartesian product — every row from A paired with every row from B.

</details>

Question 3: Why do self joins require two different aliases for the same table?

<details> <summary>Answer</summary>

The database needs a way to distinguish between the two "copies" of the table. Without aliases, column references would be ambiguous — the database would not know which instance of the table you mean. Each alias represents a different logical role (e.g., `e` for employee, `m` for manager).

</details>

Question 4: Why is NATURAL JOIN considered dangerous?

<details> <summary>Answer</summary>

NATURAL JOIN automatically matches ALL columns with the same name. If someone adds a new column to one of the tables that happens to share a name with a column in the other table, the join condition silently changes. This can cause wrong results without any error message. Explicit ON conditions prevent this.

</details>

Question 5: How do you prevent duplicate pairs in a self join?

<details> <summary>Answer</summary>

Add the condition `AND a.id < b.id` to the ON clause. This ensures each pair appears only once (A-B but not B-A) and also prevents self-pairs (A-A). Using `<` instead of `<>` or `!=` is the key — `<>` would still give you both A-B and B-A.

</details>

Exercises

Exercise 1: Data Reconciliation

Write a query using FULL OUTER JOIN to find all employees without departments AND all departments without employees. Show the employee name (or "No employee"), the department name (or "No department"), and a status column indicating the issue.

<details> <summary>Solution</summary>

```sql
SELECT COALESCE(e.first_name || ' ' || e.last_name, 'No employee')
           AS person,
       COALESCE(d.department_name, 'No department')
           AS department,
       CASE
           WHEN e.employee_id IS NULL THEN 'Empty department'
           WHEN d.department_id IS NULL THEN 'Unassigned employee'
           ELSE 'OK'
       END AS status
FROM employees e
FULL OUTER JOIN departments d
    ON e.department_id = d.department_id;
```

Expected output:

```
+---------------+-----------------+---------------------+
| person        | department      | status              |
+---------------+-----------------+---------------------+
| Alice Johnson | Engineering     | OK                  |
| Bob Smith     | Engineering     | OK                  |
| Eve Davis     | Engineering     | OK                  |
| Charlie Brown | Marketing       | OK                  |
| Diana Lee     | Sales           | OK                  |
| Frank Wilson  | No department   | Unassigned employee |
| No employee   | HR              | Empty department    |
+---------------+-----------------+---------------------+
```

</details>

Exercise 2: Product Combinations

Using the sizes and colors tables, write a query that generates a product catalog with a formatted product description and a price based on the size (Small = 19.99, Medium = 24.99, Large = 29.99).

<details> <summary>Solution</summary>

```sql
SELECT s.size_name || ' ' || c.color_name || ' T-Shirt' AS product,
       CASE s.size_name
           WHEN 'Small'  THEN 19.99
           WHEN 'Medium' THEN 24.99
           WHEN 'Large'  THEN 29.99
       END AS price
FROM sizes s
CROSS JOIN colors c
ORDER BY price, c.color_name;
```

Expected output:

```
+---------------------------+-------+
| product                   | price |
+---------------------------+-------+
| Small Black T-Shirt       | 19.99 |
| Small Blue T-Shirt        | 19.99 |
| Small Green T-Shirt       | 19.99 |
| Small Red T-Shirt         | 19.99 |
| Medium Black T-Shirt      | 24.99 |
| Medium Blue T-Shirt       | 24.99 |
| Medium Green T-Shirt      | 24.99 |
| Medium Red T-Shirt        | 24.99 |
| Large Black T-Shirt       | 29.99 |
| Large Blue T-Shirt        | 29.99 |
| Large Green T-Shirt       | 29.99 |
| Large Red T-Shirt         | 29.99 |
+---------------------------+-------+
```

</details>

Exercise 3: Organization Chart

Using the staff table, write a query that shows each person, their role, their manager's name, and their manager's manager's name (grandmanager). Use self joins. Include everyone, even the CEO who has no manager.

<details> <summary>Solution</summary>

```sql
SELECT e.name AS employee,
       e.role,
       m.name AS manager,
       gm.name AS grandmanager
FROM staff e
LEFT JOIN staff m  ON e.manager_id = m.staff_id
LEFT JOIN staff gm ON m.manager_id = gm.staff_id
ORDER BY e.staff_id;
```

Expected output:

```
+----------+-------------------+---------+--------------+
| employee | role              | manager | grandmanager |
+----------+-------------------+---------+--------------+
| Sarah    | CEO               | NULL    | NULL         |
| Tom      | VP Engineering    | Sarah   | NULL         |
| Lisa     | VP Marketing      | Sarah   | NULL         |
| Mike     | Senior Developer  | Tom     | Sarah        |
| Anna     | Developer         | Tom     | Sarah        |
| Jake     | Marketing Lead    | Lisa    | Sarah        |
| Rachel   | Designer          | Lisa    | Sarah        |
+----------+-------------------+---------+--------------+
```

</details>

What Is Next?

You now know all the major join types. But what if you need to use the result of one query as the input to another? What if you want to find employees who earn more than the average salary, or departments that have at least one employee?

In the next chapter, you will learn about subqueries — queries nested inside other queries. Subqueries let you break complex problems into smaller, manageable steps and are one of the most powerful features in SQL.

Chapter 16: Subqueries — Queries Inside Queries

What You Will Learn

- What a subquery is and how it works
- How to use subqueries in the WHERE clause (filtering with dynamic values)
- How to use subqueries in the FROM clause (derived tables)
- How to use subqueries in the SELECT clause (scalar subqueries)
- How correlated subqueries work (referencing the outer query)
- How to use EXISTS and NOT EXISTS
- How to use IN with subqueries
- When to use subqueries vs JOINs

Why This Chapter Matters

Imagine you are asked: "Show me all employees who earn more than the average salary." You know how to calculate the average salary. You know how to filter with WHERE. But how do you combine these into one query?

That is exactly what subqueries do. A subquery is a query nested inside another query. Think of it like a Russian nesting doll — a small query lives inside a bigger query, and the bigger query uses the result of the smaller one.

Subqueries let you solve problems that are too complex for a single simple query. They are one of the most powerful tools in SQL, and once you understand them, you can break down almost any complex question into manageable pieces.

What Is a Subquery?

A subquery is a SELECT statement written inside parentheses, placed within another SQL statement. The inner query runs first, and its result is used by the outer query.

```sql
SELECT column
FROM table
WHERE column = (SELECT something FROM another_table);
--              ^^^
```

```
| Eve        | Davis     | 90000.00 |
+------------+-----------+----------+
(3 rows)
```

Line-by-Line Explanation

```
SELECT first_name,                          -- Columns to display
       last_name,
       salary
FROM employees                              -- From the employees table
WHERE salary > (                            -- Filter: salary must be greater than...
    SELECT AVG(salary) FROM employees       -- ...the average salary (75000)
);
```

How it works step by step:

```
Step 1: Inner query runs first
SELECT AVG(salary) FROM employees
Result: 75000.00

Step 2: Outer query uses that result
SELECT first_name, last_name, salary
FROM employees
WHERE salary > 75000.00
```

The inner query is replaced by its result, and then the outer query runs as a normal query.

The Subquery Must Return the Right Shape

When using `=`, `>`, `<`, etc., the subquery must return exactly one value (one row, one column). This is called a scalar subquery.

```sql
-- CORRECT: Returns one value
WHERE salary > (SELECT AVG(salary) FROM employees)

-- CORRECT: Returns one value
WHERE salary = (SELECT MAX(salary) FROM employees)

-- WRONG: Returns multiple values — use IN instead
WHERE salary = (SELECT salary FROM employees WHERE department_id = 1)
-- ERROR: more than one row returned by a subquery used as an expression
```

IN with Subqueries — Matching Against a List

When a subquery returns multiple rows (but one column), use `IN` instead of `=`.

Find Employees in Departments Located in Building A

```sql
SELECT first_name,
       last_name
FROM employees
WHERE department_id IN (
    SELECT department_id
    FROM departments
    WHERE location = 'Building A'
);
```

Result:

```
+------------+-----------+
| first_name | last_name |
+------------+-----------+
| Alice      | Johnson   |
| Bob        | Smith     |
| Eve        | Davis     |
+------------+-----------+
(3 rows)
```

Line-by-Line Explanation

```
SELECT first_name, last_name          -- Columns to show
FROM employees                        -- From employees table
WHERE department_id IN (              -- department_id must be in the list...
    SELECT department_id              -- ...of department IDs
    FROM departments                  -- ...from departments table
    WHERE location = 'Building A'    -- ...that are in Building A
);
```

Step by step:

```
Step 1: Inner query
SELECT department_id FROM departments WHERE location = 'Building A'
Result: (1)    -- Only Engineering is in Building A

Step 2: Outer query
SELECT first_name, last_name FROM employees
```

```
WHERE department_id IN (1)
```

You can also use `NOT IN` to find the opposite — employees NOT in those departments.

```sql
SELECT first_name, last_name
FROM employees
WHERE department_id NOT IN (
    SELECT department_id
    FROM departments
    WHERE location = 'Building A'
);
```

Result:

```
+------------+-----------+
| first_name | last_name |
+------------+-----------+
| Charlie    | Brown     |
| Diana      | Lee       |
+------------+-----------+
(2 rows)
```

> Warning about NOT IN and NULLs: If the subquery returns any NULL values, `NOT IN` will return no rows at all! This is because `value NOT IN (1, 2, NULL)` is equivalent to `value != 1 AND value != 2 AND value != NULL`, and comparing anything to NULL yields NULL (not true). Use `NOT EXISTS` instead when NULLs are possible.

Subquery in FROM — Derived Tables

You can use a subquery in the FROM clause to create a temporary table (called a derived table) that the outer query can select from.

Department Salary Statistics

```sql
SELECT dept_stats.department_name,
       dept_stats.emp_count,
       dept_stats.avg_salary
FROM (
```

```
    SELECT d.department_name,
           COUNT(e.employee_id) AS emp_count,
           ROUND(AVG(e.salary), 2) AS avg_salary
    FROM departments d
    INNER JOIN employees e
        ON d.department_id = e.department_id
    GROUP BY d.department_name
) AS dept_stats
WHERE dept_stats.avg_salary > 70000;
```

Result:

```
+-----------------+-----------+------------+
| department_name | emp_count | avg_salary |
+-----------------+-----------+------------+
| Engineering     |         3 |   82333.33 |
| Sales           |         1 |   78000.00 |
+-----------------+-----------+------------+
(2 rows)
```

Line-by-Line Explanation

```
SELECT dept_stats.department_name,     -- Columns from the derived table
       dept_stats.emp_count,
       dept_stats.avg_salary
FROM (                                 -- Start of subquery (derived table)
    SELECT d.department_name,          -- Department name
           COUNT(e.employee_id)        -- Count employees
               AS emp_count,
           ROUND(AVG(e.salary), 2)     -- Average salary, rounded
               AS avg_salary
    FROM departments d
    INNER JOIN employees e
        ON d.department_id = e.department_id
    GROUP BY d.department_name
) AS dept_stats                        -- Give the derived table an alias
WHERE dept_stats.avg_salary > 70000;   -- Filter using derived table columns
```

The derived table is like a virtual table:

```
Step 1: Inner query produces a "table"

dept_stats:
+-----------------+-----------+------------+
| department_name | emp_count | avg_salary |
+-----------------+-----------+------------+
| Engineering     |         3 |   82333.33 |
```

```
| Marketing       |         1 |   65000.00 |
| Sales           |         1 |   78000.00 |
+-----------------+-----------+------------+

Step 2: Outer query filters this "table"
WHERE avg_salary > 70000
```

> Note: A derived table MUST have an alias. You cannot write `FROM (SELECT ...) WHERE ...` — you must write `FROM (SELECT ...) AS some_name WHERE ...`.

Subquery in SELECT — Scalar Subqueries

You can place a subquery in the SELECT clause to add a calculated column. The subquery must return exactly one value (scalar).

Employees with Company Average for Comparison

```sql
SELECT first_name,
       last_name,
       salary,
       (SELECT ROUND(AVG(salary), 2) FROM employees) AS company_avg,
       salary - (SELECT ROUND(AVG(salary), 2) FROM employees) AS diff_from_avg
FROM employees
WHERE department_id IS NOT NULL
ORDER BY salary DESC;
```

Result:

```
+------------+-----------+----------+-------------+---------------+
| first_name | last_name | salary   | company_avg | diff_from_avg |
+------------+-----------+----------+-------------+---------------+
| Eve        | Davis     | 90000.00 |    75000.00 |      15000.00 |
| Alice      | Johnson   | 85000.00 |    75000.00 |      10000.00 |
| Diana      | Lee       | 78000.00 |    75000.00 |       3000.00 |
| Bob        | Smith     | 72000.00 |    75000.00 |      -3000.00 |
| Charlie    | Brown     | 65000.00 |    75000.00 |     -10000.00 |
+------------+-----------+----------+-------------+---------------+
(5 rows)
```

The scalar subquery `(SELECT ROUND(AVG(salary), 2) FROM employees)` runs once and returns the same value for every row. Each employee's row now shows how their salary compares to

the company average.

Correlated Subqueries — Referencing the Outer Query

A correlated subquery is different from a regular subquery. It references a column from the outer query, which means it runs once for each row in the outer query.

Employees Who Earn More Than Their Department's Average

```sql
SELECT e.first_name,
       e.last_name,
       e.salary,
       e.department_id
FROM employees e
WHERE e.salary > (
    SELECT AVG(e2.salary)
    FROM employees e2
    WHERE e2.department_id = e.department_id
);
```

Result:

```
+------------+-----------+----------+---------------+
| first_name | last_name | salary   | department_id |
+------------+-----------+----------+---------------+
| Alice      | Johnson   | 85000.00 |             1 |
| Eve        | Davis     | 90000.00 |             1 |
+------------+-----------+----------+---------------+
(2 rows)
```

Line-by-Line Explanation

```
SELECT e.first_name,                    -- Employee info
       e.last_name,
       e.salary,
       e.department_id
FROM employees e                        -- Outer query: each employee
WHERE e.salary > (                      -- Salary must exceed...
    SELECT AVG(e2.salary)               -- ...the average salary
```

```
    FROM employees e2                    -- ...of employees
    WHERE e2.department_id               -- ...in the SAME department
        = e.department_id                -- (references outer query!)
);
```

How it works — step by step for each row:

```
For Alice (dept 1, salary 85000):
  Inner query: AVG salary where dept = 1 → (85000+72000+90000)/3 = 82333
  85000 > 82333? YES → include Alice

For Bob (dept 1, salary 72000):
  Inner query: AVG salary where dept = 1 → 82333
  72000 > 82333? NO → exclude Bob

For Charlie (dept 2, salary 65000):
  Inner query: AVG salary where dept = 2 → 65000
  65000 > 65000? NO → exclude Charlie (not greater than, equal to)

For Diana (dept 3, salary 78000):
  Inner query: AVG salary where dept = 3 → 78000
  78000 > 78000? NO → exclude Diana

For Eve (dept 1, salary 90000):
  Inner query: AVG salary where dept = 1 → 82333
  90000 > 82333? YES → include Eve
```

The key difference: `e.department_id` in the inner query comes from the outer query. This link makes the subquery "correlated" — it depends on the current row of the outer query.

EXISTS and NOT EXISTS

`EXISTS` checks whether a subquery returns any rows at all. It does not care about the actual values — only whether the result set is empty or not.

Find Customers Who Have Placed At Least One Order

```sql
SELECT c.first_name,
       c.last_name
FROM customers c
WHERE EXISTS (
    SELECT 1
```

```
    FROM orders o
    WHERE o.customer_id = c.customer_id
);
```

Result:

```
+------------+-----------+
| first_name | last_name |
+------------+-----------+
| Grace      | Taylor    |
| Henry      | Martinez  |
| Ivy        | Anderson  |
+------------+-----------+
(3 rows)
```

Line-by-Line Explanation

```
SELECT c.first_name,                -- Customer info
       c.last_name
FROM customers c                    -- For each customer...
WHERE EXISTS (                      -- ...check if there exists...
    SELECT 1                        -- ...any row at all (1 is arbitrary)
    FROM orders o                   -- ...in the orders table
    WHERE o.customer_id             -- ...where the customer_id
        = c.customer_id             -- ...matches this customer
);
```

> Note: `SELECT 1` is conventional in EXISTS subqueries. Since EXISTS only checks if rows exist (not what they contain), the actual selected value does not matter. `SELECT *`, `SELECT 42`, or `SELECT 'hello'` would all work the same way.

Find Customers Who Have NEVER Placed an Order

```sql
SELECT c.first_name,
       c.last_name,
       c.email
FROM customers c
WHERE NOT EXISTS (
    SELECT 1
    FROM orders o
    WHERE o.customer_id = c.customer_id
);
```

Result:

```
+------------+-----------+------------------+
| first_name | last_name | email            |
+------------+-----------+------------------+
| Jack       | Thomas    | jack@email.com   |
+------------+-----------+------------------+
(1 row)
```

`NOT EXISTS` is the opposite — it returns rows from the outer query where the subquery finds no matches. This is similar to the anti-join pattern (LEFT JOIN + IS NULL) from Chapter 14.

EXISTS vs IN — Which to Use?

Both can accomplish similar tasks:

```sql
-- Using IN
SELECT first_name FROM customers
WHERE customer_id IN (SELECT customer_id FROM orders);

-- Using EXISTS
SELECT first_name FROM customers c
WHERE EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id);
```

Key differences:

Feature	IN	EXISTS
NULL handling	NOT IN fails with NULLs	NOT EXISTS handles NULLs correctly
Performance	Good for small subquery results	Good for large tables with indexes
Readability	Simpler for simple cases	Better for complex conditions

Rule of thumb: Use `NOT EXISTS` instead of `NOT IN` when the subquery might return NULLs. For other cases, choose whichever is more readable.

Subquery vs JOIN — When to Use Each

Many subqueries can be rewritten as JOINs, and vice versa. Here are the same problem solved both ways.

Problem: Find Employees in the Engineering Department

Using a subquery:

```sql
SELECT first_name, last_name
FROM employees
WHERE department_id = (
    SELECT department_id
    FROM departments
    WHERE department_name = 'Engineering'
);
```

Using a JOIN:

```sql
SELECT e.first_name, e.last_name
FROM employees e
INNER JOIN departments d
    ON e.department_id = d.department_id
WHERE d.department_name = 'Engineering';
```

Both return the same result. So when should you use which?

When to Use Subqueries

```
+-----------------------------------------------+
| USE A SUBQUERY WHEN:                          |
+-----------------------------------------------+
| 1. You need a single value for comparison     |
|    WHERE salary > (SELECT AVG(salary) ...)    |
|                                               |
| 2. You need to check existence                |
|    WHERE EXISTS (SELECT ...)                  |
|                                               |
| 3. The logic is step-by-step                  |
|    "First find X, then use X to find Y"       |
|                                               |
| 4. You do NOT need columns from the subquery  |
|    in your final result                       |
+-----------------------------------------------+
```

When to Use JOINs

```
+-----------------------------------------------+
| USE A JOIN WHEN:                              |
```

```
+-----------------------------------------------+
| 1. You need columns from BOTH tables          |
|    SELECT e.name, d.department_name           |
|                                               |
| 2. You need to combine multiple tables        |
|    Three or more tables are easier with JOINs |
|                                               |
| 3. Performance matters on large tables         |
|    JOINs are often faster than correlated     |
|    subqueries                                 |
|                                               |
| 4. You want aggregations across joined data   |
|    GROUP BY with joined tables                |
+-----------------------------------------------+
```

Quick Comparison

Aspect	Subquery	JOIN
Readability	Step-by-step logic	All at once
Columns from inner table	Not available (unless in FROM)	Available
Performance	Correlated can be slow	Usually optimized well
NULL safety	EXISTS handles NULLs	LEFT JOIN handles NULLs
Aggregation comparison	Natural (WHERE > AVG)	Requires derived table

---

Practical Example: Complete Order Analysis

Let us combine several subquery techniques in one practical scenario.

```sql
-- Find orders where the total exceeds the average order total
SELECT o.order_id,
       c.first_name || ' ' || c.last_name AS customer,
       p.product_name,
       (o.quantity * p.price) AS order_total
FROM orders o
INNER JOIN customers c ON o.customer_id = c.customer_id
INNER JOIN products p  ON o.product_id  = p.product_id
WHERE (o.quantity * p.price) > (
```

```
    SELECT AVG(o2.quantity * p2.price)
    FROM orders o2
    INNER JOIN products p2 ON o2.product_id = p2.product_id
)
ORDER BY order_total DESC;
```

Result:

```
+----------+----------------+--------------+-------------+
| order_id | customer       | product_name | order_total |
+----------+----------------+--------------+-------------+
|        1 | Grace Taylor   | Laptop       |      999.99 |
|        4 | Ivy Anderson   | Laptop       |      999.99 |
|        5 | Ivy Anderson   | Monitor      |      399.99 |
+----------+----------------+--------------+-------------+
(3 rows)
```

The average order total is about 559.99. Only orders exceeding that amount appear.

Nesting Subqueries

You can nest subqueries inside other subqueries, but keep it reasonable. Deeply nested subqueries become hard to read.

```sql
-- Find employees in departments that have above-average employee counts
SELECT first_name, last_name
FROM employees
WHERE department_id IN (
    SELECT department_id
    FROM employees
    WHERE department_id IS NOT NULL
    GROUP BY department_id
    HAVING COUNT(*) > (
        SELECT AVG(dept_count)
        FROM (
            SELECT COUNT(*) AS dept_count
            FROM employees
            WHERE department_id IS NOT NULL
            GROUP BY department_id
        ) AS counts
    )
);
```

Result:

```
+------------+-----------+
| first_name | last_name |
+------------+-----------+
| Alice      | Johnson   |
| Bob        | Smith     |
| Eve        | Davis     |
+------------+-----------+
(3 rows)
```

These are the Engineering department employees. Engineering has 3 employees, which is above the average department size of about 1.67.

> Tip: If your subqueries are nested more than 2-3 levels deep, consider using CTEs (Common Table Expressions) instead, which you will learn in a later chapter. CTEs make complex queries much more readable.

Common Mistakes

Mistake 1: Subquery Returns Multiple Rows with = Operator

```sql
-- WRONG: Subquery returns multiple values
SELECT first_name FROM employees
WHERE department_id = (SELECT department_id FROM departments);
-- ERROR: more than one row returned by a subquery
```

Fix: Use `IN` for multiple values, or add a filter to return one value.

```sql
-- CORRECT: Use IN for multiple values
SELECT first_name FROM employees
WHERE department_id IN (SELECT department_id FROM departments);
```

Mistake 2: Forgetting the Alias for Derived Tables

```sql
-- WRONG: No alias for the subquery in FROM
SELECT * FROM (SELECT department_id, COUNT(*) FROM employees GROUP BY department_id);
-- ERROR: subquery in FROM must have an alias
```

Fix: Always give derived tables an alias.

```sql
-- CORRECT
SELECT * FROM (
    SELECT department_id, COUNT(*) AS emp_count
    FROM employees
    GROUP BY department_id
) AS dept_counts;
```

Mistake 3: NOT IN with NULLs

```sql
-- WRONG: If the subquery returns any NULLs, NO rows are returned
SELECT first_name FROM customers
WHERE customer_id NOT IN (SELECT customer_id FROM orders);
-- Works only if customer_id in orders is never NULL
```

Fix: Use NOT EXISTS, which handles NULLs correctly.

```sql
-- CORRECT: NOT EXISTS handles NULLs safely
SELECT c.first_name FROM customers c
WHERE NOT EXISTS (
    SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id
);
```

Mistake 4: Using Correlated Subqueries When a Join Would Be Faster

```sql
-- SLOW: Correlated subquery runs once per row
SELECT e.first_name,
       (SELECT d.department_name FROM departments d
        WHERE d.department_id = e.department_id) AS dept
FROM employees e;

-- FASTER: Simple JOIN
SELECT e.first_name, d.department_name
FROM employees e
LEFT JOIN departments d ON e.department_id = d.department_id;
```

Fix: When you need columns from another table, use a JOIN instead of a correlated scalar subquery.

Best Practices

1. Start simple. Write and test the inner query by itself before nesting it. Make sure it returns the right shape (one value, one column, or a table).
2. Use EXISTS/NOT EXISTS over IN/NOT IN when NULLs are possible. EXISTS handles NULLs correctly and often performs better on large tables.
3. Prefer JOINs when you need columns from both tables. Subqueries in WHERE are great for filtering, but if you need data from the subquery table in your results, a JOIN is cleaner.
4. Limit nesting depth. If you have more than 2-3 levels of nesting, refactor using CTEs (covered in a later chapter).
5. Always alias derived tables. PostgreSQL requires it, and it makes your code self-documenting.
6. Test correlated subqueries carefully. They run once per outer row, which can be slow on large tables. Check performance and consider JOINs as an alternative.

Quick Summary

Concept	Description
Subquery in WHERE	Filter using a dynamically calculated value
Subquery in FROM	Create a temporary "derived table"
Subquery in SELECT	Add a calculated column (must be scalar)
Correlated subquery	References the outer query; runs per row
EXISTS / NOT EXISTS	Checks if any rows exist (or do not)
IN with subquery	Matches against a list of values from a subquery
Scalar subquery	Returns exactly one row and one column

Key Points

- A subquery is a query nested inside another query, enclosed in parentheses.
- Subqueries in WHERE are used for dynamic filtering (e.g., above average, in a specific set).
- Subqueries in FROM create derived tables that must have an alias.
- Subqueries in SELECT add scalar calculated columns (one value per row).
- Correlated subqueries reference the outer query and run once per outer row. They can be slow on large data sets.
- EXISTS checks for row existence. NOT EXISTS is safer than NOT IN when NULLs are involved.
- JOINs are preferred when you need columns from both tables or need better performance. Subqueries are preferred for step-by-step logic and aggregate comparisons.

Practice Questions

Question 1: What is the difference between a regular subquery and a correlated subquery?

<details> <summary>Answer</summary>

A regular subquery runs once and its result is used by the outer query. A correlated subquery references a column from the outer query, so it runs once for each row in the outer query. Correlated subqueries are more flexible but can be slower.

</details>

Question 2: Why is `NOT IN` dangerous when the subquery might return NULL values?

<details> <summary>Answer</summary>

If the subquery returns any NULL values, `NOT IN` returns no rows at all. This is because `value != NULL` evaluates to NULL (not true or false), and `NOT IN` requires all comparisons to be true. Use `NOT EXISTS` instead, which handles NULLs correctly.

</details>

Question 3: When would you use a subquery in the FROM clause instead of a regular query?

<details> <summary>Answer</summary>

Use a subquery in FROM (derived table) when you need to filter on aggregated values that you cannot use directly in WHERE (because WHERE runs before GROUP BY). For example, finding departments where the average salary exceeds a threshold. The inner query groups

and aggregates, the outer query filters.

</details>

Question 4: Write a query to find the product with the highest price using a subquery.

<details> <summary>Answer</summary>

```sql
SELECT product_name, price
FROM products
WHERE price = (SELECT MAX(price) FROM products);
```

</details>

Question 5: Can you always replace a subquery with a JOIN?

<details> <summary>Answer</summary>

Not always. Subqueries that calculate aggregate values for comparison (like "above average") do not have a direct JOIN equivalent. You would need a JOIN with a derived table or CTE. However, many subqueries (especially IN subqueries and correlated subqueries that fetch columns) can be rewritten as JOINs.

</details>

Exercises

Exercise 1: Above-Average Products

Write a query that shows all products whose price is above the average product price. Display the product name, price, and how much above the average they are.

<details> <summary>Solution</summary>

```sql
SELECT product_name,
       price,
       price - (SELECT ROUND(AVG(price), 2) FROM products) AS above_avg_by
FROM products
WHERE price > (SELECT AVG(price) FROM products)
ORDER BY price DESC;
```

Expected output:

```
+--------------+--------+-------------+
| product_name | price  | above_avg_by|
+--------------+--------+-------------+
| Laptop       | 999.99 |      562.50 |
| Monitor      | 399.99 |      -37.50 |
+--------------+--------+-------------+
```

(Average price is 437.49)

</details>

Exercise 2: Customers with Multiple Orders

Write a query using EXISTS to find customers who have placed more than one order. Show their name and email.

<details> <summary>Solution</summary>

```sql
SELECT c.first_name || ' ' || c.last_name AS customer_name,
       c.email
FROM customers c
WHERE EXISTS (
    SELECT 1
    FROM orders o
    WHERE o.customer_id = c.customer_id
    GROUP BY o.customer_id
    HAVING COUNT(*) > 1
);
```

Expected output:

```
+----------------+------------------+
| customer_name  | email            |
+----------------+------------------+
| Grace Taylor   | grace@email.com  |
| Ivy Anderson   | ivy@email.com    |
+----------------+------------------+
```

Alternative approach using IN:

```sql
SELECT first_name || ' ' || last_name AS customer_name, email
FROM customers
WHERE customer_id IN (
    SELECT customer_id
    FROM orders
```

```
    GROUP BY customer_id
    HAVING COUNT(*) > 1
);
```

</details>

Exercise 3: Department Salary Comparison

Write a query that shows each employee, their salary, their department's average salary, and whether they earn above or below the department average. Use a correlated subquery.

<details> <summary>Solution</summary>

```sql
SELECT e.first_name,
       e.last_name,
       e.salary,
       (SELECT ROUND(AVG(e2.salary), 2)
        FROM employees e2
        WHERE e2.department_id = e.department_id) AS dept_avg,
       CASE
           WHEN e.salary > (SELECT AVG(e2.salary)
                            FROM employees e2
                            WHERE e2.department_id = e.department_id)
               THEN 'Above average'
           WHEN e.salary < (SELECT AVG(e2.salary)
                            FROM employees e2
                            WHERE e2.department_id = e.department_id)
               THEN 'Below average'
           ELSE 'At average'
       END AS comparison
FROM employees e
WHERE e.department_id IS NOT NULL
ORDER BY e.department_id, e.salary DESC;
```

Expected output:

```
+------------+-----------+----------+----------+---------------+
| first_name | last_name | salary   | dept_avg | comparison    |
+------------+-----------+----------+----------+---------------+
| Eve        | Davis     | 90000.00 | 82333.33 | Above average |
| Alice      | Johnson   | 85000.00 | 82333.33 | Above average |
| Bob        | Smith     | 72000.00 | 82333.33 | Below average |
| Charlie    | Brown     | 65000.00 | 65000.00 | At average    |
| Diana      | Lee       | 78000.00 | 78000.00 | At average    |
+------------+-----------+----------+----------+---------------+
```

</details>

What Is Next?

Now that you can nest queries inside other queries, the next chapter introduces another way to combine results from multiple queries: set operations. UNION, INTERSECT, and EXCEPT let you combine, find common elements, or subtract the results of two or more queries. Think of them as mathematical operations for result sets.

Chapter 17: Set Operations — UNION, INTERSECT, and EXCEPT

What You Will Learn

- How UNION combines results from two queries and removes duplicates
- How UNION ALL combines results and keeps duplicates
- How INTERSECT finds rows that appear in both queries
- How EXCEPT finds rows in the first query that are not in the second
- The rules for using set operations (column count, compatible types)
- How to use ORDER BY with set operations
- Practical examples: combining customer lists, finding shared data, reporting

Why This Chapter Matters

Imagine you have two spreadsheets of customer names — one from your online store and one from your physical store. You might want to:

- Combine both lists into one (UNION)
- Find customers who appear on both lists (INTERSECT)
- Find customers on the first list who are NOT on the second (EXCEPT)

These are set operations, and they work just like the set theory you may have learned in math class. In SQL, set operations let you combine the results of two or more SELECT queries into a single result set.

While JOINs combine tables horizontally (adding columns), set operations combine queries vertically (adding rows).

```
JOIN (horizontal):              SET OPERATIONS (vertical):
+------+------+                +------+
| T1   | T2   |                | Q1   |
| cols | cols |                | rows |
+------+------+                +------+
                                | Q2   |
                                | rows |
                                +------+
```

Setting Up Practice Data

Let us create two customer tables to simulate data from different sources.

```sql
CREATE TABLE online_customers (
    customer_id SERIAL PRIMARY KEY,
    first_name  VARCHAR(50) NOT NULL,
    last_name   VARCHAR(50) NOT NULL,
    email       VARCHAR(100),
    city        VARCHAR(50)
);

CREATE TABLE store_customers (
    customer_id SERIAL PRIMARY KEY,
    first_name  VARCHAR(50) NOT NULL,
    last_name   VARCHAR(50) NOT NULL,
    email       VARCHAR(100),
    city        VARCHAR(50)
);

INSERT INTO online_customers (first_name, last_name, email, city) VALUES
('Alice',   'Johnson', 'alice@email.com',   'New York'),
('Bob',     'Smith',   'bob@email.com',     'Chicago'),
('Charlie', 'Brown',   'charlie@email.com', 'Boston'),
('Diana',   'Lee',     'diana@email.com',   'New York'),
('Eve',     'Davis',   'eve@email.com',     'Seattle');

INSERT INTO store_customers (first_name, last_name, email, city) VALUES
('Charlie', 'Brown',   'charlie@email.com', 'Boston'),
('Frank',   'Wilson',  'frank@email.com',   'Denver'),
('Grace',   'Taylor',  'grace@email.com',   'Chicago'),
('Diana',   'Lee',     'diana@email.com',   'New York'),
('Henry',   'Martinez','henry@email.com',   'Miami');
```

Here is what the data looks like:

```
online_customers:                store_customers:
+---------+---------+--------+   +---------+---------+--------+
| first   | last    | city   |   | first   | last    | city   |
+---------+---------+--------+   +---------+---------+--------+
| Alice   | Johnson | NY     |   | Charlie | Brown   | Boston |
| Bob     | Smith   | Chicago|   | Frank   | Wilson  | Denver |
| Charlie | Brown   | Boston |   | Grace   | Taylor  | Chicago|
| Diana   | Lee     | NY     |   | Diana   | Lee     | NY     |
| Eve     | Davis   | Seattle|   | Henry   | Martinez| Miami  |
+---------+---------+--------+   +---------+---------+--------+

Shared: Charlie Brown and Diana Lee appear in BOTH tables.
```

UNION — Combine and Remove Duplicates

UNION takes the results of two SELECT queries and combines them into one result set, removing duplicate rows.

The Syntax

```sql
SELECT columns FROM table_1
UNION
SELECT columns FROM table_2;
```

Example: All Unique Customers

```sql
SELECT first_name, last_name, email, city
FROM online_customers
UNION
SELECT first_name, last_name, email, city
FROM store_customers;
```

Result:

```
+-----------+-----------+---------------------+----------+
| first_name| last_name | email               | city     |
+-----------+-----------+---------------------+----------+
| Alice     | Johnson   | alice@email.com     | New York |
| Bob       | Smith     | bob@email.com       | Chicago  |
| Charlie   | Brown     | charlie@email.com   | Boston   |
```

```
| Diana     | Lee       | diana@email.com     | New York |
| Eve       | Davis     | eve@email.com       | Seattle  |
| Frank     | Wilson    | frank@email.com     | Denver   |
| Grace     | Taylor    | grace@email.com     | Chicago  |
| Henry     | Martinez  | henry@email.com     | Miami    |
+-----------+-----------+---------------------+----------+
(8 rows)
```

We started with 5 + 5 = 10 rows, but got only 8. Charlie Brown and Diana Lee appeared in both tables, so UNION removed the duplicates.

Visual Representation

```
Online:  {Alice, Bob, Charlie, Diana, Eve}
Store:   {Charlie, Frank, Grace, Diana, Henry}

UNION:   {Alice, Bob, Charlie, Diana, Eve, Frank, Grace, Henry}
          (duplicates removed — Charlie and Diana appear once)

Think of it as merging two stacks of cards and
removing any cards that appear twice.
```

UNION ALL — Combine and Keep Duplicates

UNION ALL works like UNION, but it keeps all rows, including duplicates.

```sql
SELECT first_name, last_name, email, city
FROM online_customers
UNION ALL
SELECT first_name, last_name, email, city
FROM store_customers;
```

Result:

```
+-----------+-----------+---------------------+----------+
| first_name| last_name | email               | city     |
+-----------+-----------+---------------------+----------+
| Alice     | Johnson   | alice@email.com     | New York |
| Bob       | Smith     | bob@email.com       | Chicago  |
| Charlie   | Brown     | charlie@email.com   | Boston   |
| Diana     | Lee       | diana@email.com     | New York |
```

```
| Eve       | Davis     | eve@email.com       | Seattle  |
| Charlie   | Brown     | charlie@email.com   | Boston   |
| Frank     | Wilson    | frank@email.com     | Denver   |
| Grace     | Taylor    | grace@email.com     | Chicago  |
| Diana     | Lee       | diana@email.com     | New York |
| Henry     | Martinez  | henry@email.com     | Miami    |
+-----------+-----------+---------------------+----------+
(10 rows)
```

All 10 rows are returned. Charlie and Diana each appear twice.

UNION vs UNION ALL

```
UNION:     Removes duplicates (slower — must compare all rows)
UNION ALL: Keeps duplicates   (faster — no comparison needed)
```

When to use which:

Use UNION when...	Use UNION ALL when...
You need unique results	You want all rows, including duplicates
Combining overlapping data sources	Combining data you know has no overlap
Building a master list	Aggregating (COUNT, SUM) across sources

Performance tip: UNION ALL is faster than UNION because it does not need to check for and remove duplicates. If you know there are no duplicates, always use UNION ALL.

INTERSECT — Find Common Rows

INTERSECT returns only rows that appear in both query results.

Example: Customers Who Shop Both Online and In-Store

```sql
SELECT first_name, last_name, email
FROM online_customers
INTERSECT
```

```
SELECT first_name, last_name, email
FROM store_customers;
```

Result:

```
+-----------+-----------+---------------------+
| first_name| last_name | email               |
+-----------+-----------+---------------------+
| Charlie   | Brown     | charlie@email.com   |
| Diana     | Lee       | diana@email.com     |
+-----------+-----------+---------------------+
(2 rows)
```

Only Charlie and Diana appear in both tables.

Visual Representation

```
Online:    {Alice, Bob, Charlie, Diana, Eve}
Store:     {Charlie, Frank, Grace, Diana, Henry}

INTERSECT: {Charlie, Diana}
            (only the overlap)

Like a Venn diagram — INTERSECT returns
only the middle section.

  Online          Store
+---------+     +---------+
|         |     |         |
| Alice   +-----+ Frank   |
| Bob     |Charl| Grace   |
| Eve     |Diana| Henry   |
|         +-----+         |
+---------+     +---------+
```

EXCEPT — Find Rows in First but Not Second

EXCEPT returns rows from the first query that do not appear in the second query. Think of it as subtraction.

Example: Online-Only Customers

```sql
SELECT first_name, last_name, email
FROM online_customers
EXCEPT
SELECT first_name, last_name, email
FROM store_customers;
```

Result:

```
+-----------+-----------+---------------------+
| first_name| last_name | email               |
+-----------+-----------+---------------------+
| Alice     | Johnson   | alice@email.com     |
| Bob       | Smith     | bob@email.com       |
| Eve       | Davis     | eve@email.com       |
+-----------+-----------+---------------------+
(3 rows)
```

These customers shop online but have never visited the physical store.

Example: Store-Only Customers (Reverse the Order)

```sql
SELECT first_name, last_name, email
FROM store_customers
EXCEPT
SELECT first_name, last_name, email
FROM online_customers;
```

Result:

```
+-----------+-----------+---------------------+
| first_name| last_name | email               |
+-----------+-----------+---------------------+
| Frank     | Wilson    | frank@email.com     |
| Grace     | Taylor    | grace@email.com     |
| Henry     | Martinez  | henry@email.com     |
+-----------+-----------+---------------------+
(3 rows)
```

Order matters with EXCEPT. `A EXCEPT B` gives rows in A but not B. `B EXCEPT A` gives rows in B but not A. These are different results.

Visual Representation

```
Online EXCEPT Store:              Store EXCEPT Online:

  Online          Store             Online          Store
+---------+     +---------+      +---------+     +---------+
|#########|     |         |      |         |     |#########|
|# Alice #|     |         |      |         |     |# Frank #|
|# Bob   #|     |         |      |         |     |# Grace #|
|# Eve   #|     |         |      |         |     |# Henry #|
|#########|     |         |      |         |     |#########|
+---------+     +---------+      +---------+     +---------+

The shaded area (###) is the result.
```

Rules for Set Operations

All set operations follow these rules:

Rule 1: Same Number of Columns

Both queries must select the same number of columns.

```sql
-- WRONG: First query has 3 columns, second has 2
SELECT first_name, last_name, email FROM online_customers
UNION
SELECT first_name, last_name FROM store_customers;
-- ERROR: each UNION query must have the same number of columns
```

Fix: Make sure both queries have the same number of columns.

```sql
-- CORRECT
SELECT first_name, last_name, email FROM online_customers
UNION
SELECT first_name, last_name, email FROM store_customers;
```

Rule 2: Compatible Data Types

The columns in matching positions must have compatible data types.

```sql
-- WRONG: Can't UNION a VARCHAR with an INTEGER
SELECT first_name FROM online_customers
UNION
SELECT customer_id FROM store_customers;
-- ERROR: UNION types varchar and integer cannot be matched
```

Fix: Make sure corresponding columns have compatible types.

Rule 3: Column Names Come from the First Query

The column names in the result are determined by the first query.

```sql
SELECT first_name AS name, city AS location FROM online_customers
UNION
SELECT first_name AS customer, city AS town FROM store_customers;
-- Result columns will be "name" and "location" (from the first query)
```

Summary of Rules

```
+-----------------------------------------------+
| SET OPERATION RULES                           |
+-----------------------------------------------+
| 1. Same number of columns in both queries     |
| 2. Corresponding columns have compatible types |
| 3. Column names come from the first query     |
| 4. ORDER BY goes at the very end (once)       |
+-----------------------------------------------+
```

ORDER BY with Set Operations

You can add ORDER BY to a set operation, but it goes at the very end and applies to the entire combined result.

```sql
SELECT first_name, last_name, city
FROM online_customers
UNION
SELECT first_name, last_name, city
FROM store_customers
ORDER BY city, last_name;
```

Result:

```
+-----------+-----------+----------+
| first_name| last_name | city     |
+-----------+-----------+----------+
| Charlie   | Brown     | Boston   |
| Bob       | Smith     | Chicago  |
| Grace     | Taylor    | Chicago  |
| Frank     | Wilson    | Denver   |
| Henry     | Martinez  | Miami    |
| Alice     | Johnson   | New York |
| Diana     | Lee       | New York |
| Eve       | Davis     | Seattle  |
+-----------+-----------+----------+
(8 rows)
```

Important: You cannot put ORDER BY inside individual queries of a set operation (unless you wrap the query in a subquery).

```sql
-- WRONG
SELECT first_name FROM online_customers ORDER BY first_name
UNION
SELECT first_name FROM store_customers;
-- ERROR: ORDER BY within individual UNION queries is not allowed

-- CORRECT: ORDER BY at the end
SELECT first_name FROM online_customers
UNION
SELECT first_name FROM store_customers
ORDER BY first_name;
```

Adding a Source Column

When combining data from different sources, it is often helpful to add a column that identifies where each row came from.

```sql
SELECT first_name, last_name, city, 'Online' AS source
FROM online_customers
UNION ALL
SELECT first_name, last_name, city, 'Store' AS source
FROM store_customers
ORDER BY last_name;
```

Result:

```
+-----------+-----------+----------+--------+
| first_name| last_name | city     | source |
+-----------+-----------+----------+--------+
| Charlie   | Brown     | Boston   | Online |
| Charlie   | Brown     | Boston   | Store  |
| Eve       | Davis     | Seattle  | Online |
| Alice     | Johnson   | New York | Online |
| Diana     | Lee       | New York | Online |
| Diana     | Lee       | New York | Store  |
| Henry     | Martinez  | Miami    | Store  |
| Bob       | Smith     | Chicago  | Online |
| Grace     | Taylor    | Chicago  | Store  |
| Frank     | Wilson    | Denver   | Store  |
+-----------+-----------+----------+--------+
(10 rows)
```

Notice we used UNION ALL here, not UNION. If we used UNION, it would consider Charlie's two rows different (because the source column differs) and keep both anyway. But UNION ALL is faster since no duplicate checking is needed.

Practical Example: Combining Different Query Results

Set operations do not have to combine data from different tables. You can combine different queries from the same table.

Employees by Salary Category

```sql
SELECT first_name, last_name, salary, 'High' AS category
FROM employees
WHERE salary >= 80000

UNION ALL

SELECT first_name, last_name, salary, 'Medium' AS category
FROM employees
WHERE salary >= 65000 AND salary < 80000

UNION ALL

SELECT first_name, last_name, salary, 'Low' AS category
FROM employees
```

```
WHERE salary < 65000

ORDER BY salary DESC;
```

Result:

```
+------------+-----------+----------+----------+
| first_name | last_name | salary   | category |
+------------+-----------+----------+----------+
| Eve        | Davis     | 90000.00 | High     |
| Alice      | Johnson   | 85000.00 | High     |
| Diana      | Lee       | 78000.00 | Medium   |
| Bob        | Smith     | 72000.00 | Medium   |
| Charlie    | Brown     | 65000.00 | Medium   |
| Frank      | Wilson    | 60000.00 | Low      |
+------------+-----------+----------+----------+
(6 rows)
```

> Note: This specific example could be done more cleanly with a CASE expression, but it illustrates how UNION ALL can combine categorized subsets.

Practical Example: Finding Cities Served by Both Channels

```sql
-- Cities with both online and store customers
SELECT city FROM online_customers
INTERSECT
SELECT city FROM store_customers;
```

Result:

```
+----------+
| city     |
+----------+
| Boston   |
| Chicago  |
| New York |
+----------+
(3 rows)
```

These cities have customers in both channels.

```sql
-- Cities with online customers but NO store customers
SELECT city FROM online_customers
EXCEPT
SELECT city FROM store_customers;
```

Result:

```
+----------+
| city     |
+----------+
| Seattle  |
+----------+
(1 row)
```

Seattle only has online customers.

Chaining Multiple Set Operations

You can chain more than two queries with set operations.

```sql
-- All cities from all customer sources (employees too)
SELECT city FROM online_customers
UNION
SELECT city FROM store_customers
UNION
SELECT location AS city FROM departments
ORDER BY city;
```

Result:

```
+------------+
| city       |
+------------+
| Boston     |
| Building A |
| Building B |
| Building C |
| Building D |
| Chicago    |
| Denver     |
```

```
| Miami      |
| New York   |
| Seattle    |
+------------+
(10 rows)
```

When chaining, operations are evaluated left to right. You can use parentheses to control the order:

```sql
-- Parentheses control evaluation order
(SELECT city FROM online_customers
 INTERSECT
 SELECT city FROM store_customers)
EXCEPT
SELECT 'Boston';
```

This first finds cities in both tables (Boston, Chicago, New York), then removes Boston.

Common Mistakes

Mistake 1: Different Number of Columns

```sql
-- WRONG
SELECT first_name, last_name, email FROM online_customers
UNION
SELECT first_name, last_name FROM store_customers;
-- ERROR: each UNION query must have the same number of columns
```

Fix: Add NULL or a placeholder for missing columns.

```sql
-- CORRECT
SELECT first_name, last_name, email FROM online_customers
UNION
SELECT first_name, last_name, NULL AS email FROM store_customers;
```

Mistake 2: ORDER BY Inside a UNION

```sql
-- WRONG
SELECT first_name FROM online_customers ORDER BY first_name
UNION
SELECT first_name FROM store_customers;
```

Fix: Put ORDER BY at the very end.

```sql
-- CORRECT
SELECT first_name FROM online_customers
UNION
SELECT first_name FROM store_customers
ORDER BY first_name;
```

Mistake 3: Using UNION When You Mean UNION ALL

```sql
-- WRONG: UNION removes duplicates, hiding the true count
SELECT customer_id, product_id FROM orders
UNION
SELECT customer_id, product_id FROM orders;
-- Duplicates removed — you lose information!
```

Fix: Use UNION ALL when you want to keep all rows.

Mistake 4: Assuming EXCEPT Is Commutative

```sql
-- These are NOT the same!
SELECT city FROM online_customers EXCEPT SELECT city FROM store_customers;
-- vs
SELECT city FROM store_customers EXCEPT SELECT city FROM online_customers;
```

Fix: Remember that EXCEPT is directional. `A EXCEPT B` is different from `B EXCEPT A`.

Best Practices

1. Use UNION ALL instead of UNION when you know there are no duplicates or when you want to keep duplicates. It is significantly faster on large data sets.
2. Add a source column when combining data from different tables. This makes it easy to trace where each row came from.

3. Use the same column aliases in both queries for clarity, even though only the first query's names matter.
4. Put ORDER BY at the end of the entire set operation, not inside individual queries.
5. Use INTERSECT for "both" questions and EXCEPT for "only in one" questions. They are more readable than equivalent subqueries or JOINs for these specific patterns.
6. Be explicit about what you want. UNION removes duplicates and UNION ALL does not. Choose deliberately rather than defaulting to one.

Quick Summary

Operation	What It Returns
UNION	All rows from both queries, duplicates removed
UNION ALL	All rows from both queries, duplicates kept
INTERSECT	Only rows that appear in both queries
EXCEPT	Rows in the first query that are NOT in the second

Key Points

- UNION combines results and removes duplicates. UNION ALL keeps all rows.
- INTERSECT finds rows common to both queries (the overlap).
- EXCEPT finds rows in the first query that do not appear in the second (the difference). Order matters.
- All set operations require the same number of columns with compatible data types.
- Column names in the result come from the first query.
- ORDER BY goes at the very end and applies to the entire combined result.
- UNION ALL is faster than UNION because it skips duplicate checking.
- Set operations combine results vertically (more rows), while JOINs combine horizontally (more columns).

Practice Questions

Question 1: What is the difference between UNION and UNION ALL?

<details> <summary>Answer</summary>

UNION combines results from two queries and removes duplicate rows. UNION ALL combines results and keeps all rows, including duplicates. UNION ALL is faster because it does not need to check for duplicates.

</details>

Question 2: If Query A returns 100 rows and Query B returns 80 rows, what is the maximum number of rows UNION can return? What about UNION ALL?

<details> <summary>Answer</summary>

UNION: Maximum 180 rows (if there are no duplicates between the two queries). Minimum 100 rows (if all 80 rows of B are duplicates of rows in A).

UNION ALL: Always exactly 180 rows (100 + 80), regardless of duplicates.

</details>

Question 3: Is `A EXCEPT B` the same as `B EXCEPT A`?

<details> <summary>Answer</summary>

No. `A EXCEPT B` returns rows that are in A but NOT in B. `B EXCEPT A` returns rows that are in B but NOT in A. EXCEPT is not commutative — the order of the queries matters.

</details>

Question 4: Can you use set operations with queries from the same table?

<details> <summary>Answer</summary>

Yes. Set operations work with any two SELECT queries, regardless of whether they come from the same table or different tables. The only requirement is that both queries return the same number of columns with compatible types.

</details>

Question 5: Why might you add a literal string column (like `'Online'` or `'Store'`) when using UNION ALL?

<details> <summary>Answer</summary>

To identify which source each row came from. When you combine data from multiple tables or queries, a source column helps you trace the origin of each row. This is especially useful for

reporting and debugging.

</details>

Exercises

Exercise 1: Master Customer List

Create a deduplicated master customer list from both online_customers and store_customers. Include all columns and sort by last name. How many unique customers are there?

<details> <summary>Solution</summary>

```sql
SELECT first_name, last_name, email, city
FROM online_customers
UNION
SELECT first_name, last_name, email, city
FROM store_customers
ORDER BY last_name;
```

Expected output (8 unique customers):

```
+-----------+-----------+---------------------+----------+
| first_name| last_name | email               | city     |
+-----------+-----------+---------------------+----------+
| Charlie   | Brown     | charlie@email.com   | Boston   |
| Eve       | Davis     | eve@email.com       | Seattle  |
| Alice     | Johnson   | alice@email.com     | New York |
| Diana     | Lee       | diana@email.com     | New York |
| Henry     | Martinez  | henry@email.com     | Miami    |
| Bob       | Smith     | bob@email.com       | Chicago  |
| Grace     | Taylor    | grace@email.com     | Chicago  |
| Frank     | Wilson    | frank@email.com     | Denver   |
+-----------+-----------+---------------------+----------+
```

</details>

Exercise 2: Exclusive Customers Report

Write two queries: (1) customers who ONLY shop online and (2) customers who ONLY shop in-store. Add a source column to each and combine them into one result using UNION ALL.

<details> <summary>Solution</summary>

```sql
SELECT first_name, last_name, 'Online Only' AS channel
FROM (
    SELECT first_name, last_name, email
    FROM online_customers
    EXCEPT
    SELECT first_name, last_name, email
    FROM store_customers
) AS online_only

UNION ALL

SELECT first_name, last_name, 'Store Only' AS channel
FROM (
    SELECT first_name, last_name, email
    FROM store_customers
    EXCEPT
    SELECT first_name, last_name, email
    FROM online_customers
) AS store_only

ORDER BY channel, last_name;
```

Expected output:

```
+-----------+-----------+-------------+
| first_name| last_name | channel     |
+-----------+-----------+-------------+
| Eve       | Davis     | Online Only |
| Alice     | Johnson   | Online Only |
| Bob       | Smith     | Online Only |
| Henry     | Martinez  | Store Only  |
| Grace     | Taylor    | Store Only  |
| Frank     | Wilson    | Store Only  |
+-----------+-----------+-------------+
```

</details>

Exercise 3: City Analysis

Write queries to answer these three questions:

1. Which cities have customers in BOTH channels?
2. Which cities have ONLY online customers?
3. Which cities have ONLY store customers?

<details> <summary>Solution</summary>

```sql
-- 1. Cities with customers in BOTH channels
SELECT city, 'Both channels' AS status
FROM online_customers
INTERSECT
SELECT city, 'Both channels'
FROM store_customers;

-- 2. Cities with ONLY online customers
SELECT city FROM online_customers
EXCEPT
SELECT city FROM store_customers;

-- 3. Cities with ONLY store customers
SELECT city FROM store_customers
EXCEPT
SELECT city FROM online_customers;
```

Expected outputs:

```
-- Query 1: Both channels
+----------+
| city     |
+----------+
| Boston   |
| Chicago  |
| New York |
+----------+

-- Query 2: Online only
+----------+
| city     |
+----------+
| Seattle  |
+----------+

-- Query 3: Store only
+----------+
| city     |
+----------+
| Denver   |
| Miami    |
+----------+
```

</details>

What Is Next?

You now know how to combine, intersect, and subtract query results. In the next chapter, we switch gears to work with string and date functions — the tools you need to manipulate text data (formatting names, extracting substrings, searching within strings) and date data (calculating ages, extracting parts of dates, formatting timestamps). These are the functions you will use in almost every real-world query.

Chapter 18: String and Date Functions

What You Will Learn

String functions:

- UPPER, LOWER — change letter case
- LENGTH — count characters
- TRIM, LTRIM, RTRIM — remove whitespace
- SUBSTRING — extract part of a string
- REPLACE — swap text within a string
- CONCAT and || — join strings together
- LEFT, RIGHT — extract from start or end
- POSITION — find where a substring occurs
- SPLIT_PART — split strings by a delimiter
- REGEXP_MATCHES — pattern matching with regular expressions

Date functions:

- CURRENT_DATE, CURRENT_TIMESTAMP — get the current date and time
- EXTRACT — pull out year, month, day, etc.
- DATE_TRUNC — truncate to a specific precision
- AGE — calculate the difference between dates
- Interval arithmetic — add and subtract time periods
- TO_CHAR — format dates as strings
- Casting strings to dates

Why This Chapter Matters

Raw data in a database is rarely in the exact format you need. Customer names might be in all caps. Dates might need to be displayed in a specific format. You might need to extract the domain from an email address or calculate someone's age from their birth date.

String and date functions are the formatting and calculation tools of SQL. Think of them as the equivalent of text functions in a spreadsheet — UPPER, LOWER, LEFT, RIGHT, and date calculations. You will use these functions in almost every real-world query.

Setting Up Practice Data

```sql
CREATE TABLE contacts (
    contact_id SERIAL PRIMARY KEY,
    full_name  VARCHAR(100),
    email      VARCHAR(100),
    phone      VARCHAR(20),
    birth_date DATE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

INSERT INTO contacts (full_name, email, phone, birth_date, created_at) VALUES
('  Alice Johnson  ', 'ALICE@EXAMPLE.COM',   '555-123-4567', '1990-03-15', '2023-01-15 ...
('bob smith',         'bob@example.com',     '555.987.6543', '1985-07-22', '2023-03-20 ...
('Charlie Brown',     'charlie@GMAIL.COM',   '(555) 246-8101','1992-11-08', '2023-06-10...
('DIANA LEE',         'diana@example.com',   '555 369 2580', '1988-01-30', '2024-01-05 ...
('Eve Davis-Wilson',  'eve.davis@work.co.uk','555-111-2222', '1995-12-25', '2024-02-14 ...
```

String Functions

UPPER and LOWER — Change Letter Case

```sql
SELECT full_name,
       UPPER(full_name) AS upper_name,
       LOWER(full_name) AS lower_name
FROM contacts;
```

Result:

```
+--------------------+--------------------+--------------------+
| full_name          | upper_name         | lower_name         |
+--------------------+--------------------+--------------------+
|   Alice Johnson    |   ALICE JOHNSON    |   alice johnson    |
| bob smith          | BOB SMITH          | bob smith          |
| Charlie Brown      | CHARLIE BROWN      | charlie brown      |
| DIANA LEE          | DIANA LEE          | diana lee          |
| Eve Davis-Wilson   | EVE DAVIS-WILSON   | eve davis-wilson   |
+--------------------+--------------------+--------------------+
(5 rows)
```

Common use: Standardizing data for comparison.

```sql
-- Case-insensitive search
SELECT full_name FROM contacts
WHERE LOWER(full_name) LIKE '%alice%';
```

INITCAP — Title Case

```sql
SELECT full_name,
       INITCAP(full_name) AS title_case
FROM contacts;
```

Result:

```
+--------------------+--------------------+
| full_name          | title_case         |
+--------------------+--------------------+
|   Alice Johnson    |   Alice Johnson    |
| bob smith          | Bob Smith          |
| Charlie Brown      | Charlie Brown      |
| DIANA LEE          | Diana Lee          |
| Eve Davis-Wilson   | Eve Davis-Wilson   |
+--------------------+--------------------+
(5 rows)
```

INITCAP capitalizes the first letter of each word and lowercases the rest.

LENGTH — Count Characters

```sql
SELECT full_name,
       LENGTH(full_name) AS name_length,
       LENGTH(TRIM(full_name)) AS trimmed_length
FROM contacts;
```

Result:

```
+--------------------+-------------+----------------+
| full_name          | name_length | trimmed_length |
+--------------------+-------------+----------------+
|   Alice Johnson    |          18 |             14 |
| bob smith          |           9 |              9 |
| Charlie Brown      |          13 |             13 |
| DIANA LEE          |           9 |              9 |
| Eve Davis-Wilson   |          16 |             16 |
+--------------------+-------------+----------------+
(5 rows)
```

Notice that Alice Johnson's name has leading and trailing spaces, making it 18 characters instead of 14.

TRIM, LTRIM, RTRIM — Remove Whitespace

```sql
SELECT full_name,
       TRIM(full_name) AS trimmed,
       LTRIM(full_name) AS left_trimmed,
       RTRIM(full_name) AS right_trimmed
FROM contacts
WHERE full_name LIKE ' %' OR full_name LIKE '% ';
```

Result:

```
+--------------------+----------------+------------------+-------------------+
| full_name          | trimmed        | left_trimmed     | right_trimmed     |
+--------------------+----------------+------------------+-------------------+
|   Alice Johnson    | Alice Johnson  | Alice Johnson    |   Alice Johnson   |
+--------------------+----------------+------------------+-------------------+
(1 row)
```

- `TRIM` removes spaces from both sides
- `LTRIM` removes spaces from the left only
- `RTRIM` removes spaces from the right only

You can also trim specific characters:

```sql
SELECT TRIM(BOTH '-' FROM '--hello--') AS result;
-- Result: hello
```

SUBSTRING — Extract Part of a String

SUBSTRING extracts a portion of a string by position and length.

```sql
-- SUBSTRING(string FROM start FOR length)
-- or: SUBSTRING(string, start, length)

SELECT email,
       SUBSTRING(email FROM 1 FOR 5) AS first_five,
       SUBSTRING(email, 1, POSITION('@' IN email) - 1) AS username
FROM contacts;
```

Result:

```
+----------------------+------------+-----------+
| email                | first_five | username  |
+----------------------+------------+-----------+
| ALICE@EXAMPLE.COM    | ALICE      | ALICE     |
| bob@example.com      | bob@e      | bob       |
| charlie@GMAIL.COM    | charl      | charlie   |
| diana@example.com    | diana      | diana     |
| eve.davis@work.co.uk | eve.d      | eve.davis |
+----------------------+------------+-----------+
(5 rows)
```

Line-by-Line Explanation

```
SUBSTRING(email FROM 1 FOR 5)                   -- Characters 1 through 5
SUBSTRING(email, 1, POSITION('@' IN email) - 1) -- From char 1 to the char
                                                 -- before the @ symbol
```

REPLACE — Swap Text

```sql
SELECT phone,
       REPLACE(phone, '-', '') AS no_dashes,
       REPLACE(REPLACE(REPLACE(phone, '-', ''), '.', ''), ' ', '') AS digits_only
FROM contacts;
```

Result:

```
+----------------+-------------+-------------+
| phone          | no_dashes   | digits_only |
+----------------+-------------+-------------+
| 555-123-4567   | 5551234567  | 5551234567  |
| 555.987.6543   | 555.987.6543| 5559876543  |
| (555) 246-8101 | (555) 2468101| (555)2468101|
| 555 369 2580   | 555 369 2580| 5553692580  |
| 555-111-2222   | 5551112222  | 5551112222  |
+----------------+-------------+-------------+
(5 rows)
```

REPLACE is case-sensitive and replaces all occurrences of the search string.

CONCAT and || — Join Strings Together

PostgreSQL offers two ways to concatenate strings:

```sql
SELECT CONCAT('Hello', ' ', 'World') AS using_concat,
       'Hello' || ' ' || 'World' AS using_pipes;
```

Result:

```
+--------------+-------------+
| using_concat | using_pipes |
+--------------+-------------+
| Hello World  | Hello World |
+--------------+-------------+
```

Key difference: `||` returns NULL if any part is NULL. `CONCAT` treats NULL as an empty string.

```sql
SELECT CONCAT('Hello', NULL, 'World') AS concat_result,
       'Hello' || NULL || 'World' AS pipe_result;
```

Result:

```
+---------------+-------------+
| concat_result | pipe_result |
+---------------+-------------+
| HelloWorld    | NULL        |
+---------------+-------------+
```

Use `CONCAT` when NULLs are possible and you do not want the entire result to become NULL.

LEFT and RIGHT — Extract from Start or End

```sql
SELECT email,
       LEFT(email, 3) AS first_three,
       RIGHT(email, 3) AS last_three
FROM contacts;
```

Result:

```
+----------------------+-------------+------------+
| email                | first_three | last_three |
+----------------------+-------------+------------+
| ALICE@EXAMPLE.COM    | ALI         | COM        |
| bob@example.com      | bob         | com        |
| charlie@GMAIL.COM    | cha         | COM        |
| diana@example.com    | dia         | com        |
| eve.davis@work.co.uk | eve         | .uk        |
+----------------------+-------------+------------+
(5 rows)
```

POSITION — Find Where a Substring Occurs

POSITION returns the index of the first occurrence of a substring. Returns 0 if not found.

```sql
SELECT email,
       POSITION('@' IN email) AS at_position,
       POSITION('.' IN email) AS dot_position
FROM contacts;
```

Result:

```
+----------------------+-------------+--------------+
| email                | at_position | dot_position |
+----------------------+-------------+--------------+
| ALICE@EXAMPLE.COM    |           6 |           14 |
| bob@example.com      |           4 |            12|
| charlie@GMAIL.COM    |           8 |           14 |
| diana@example.com    |           6 |           14 |
| eve.davis@work.co.uk |          10 |            4 |
+----------------------+-------------+--------------+
(5 rows)
```

Practical: Extract Domain from Email

```sql
SELECT email,
       SUBSTRING(email FROM POSITION('@' IN email) + 1) AS domain
FROM contacts;
```

Result:

```
+----------------------+--------------+
| email                | domain       |
+----------------------+--------------+
| ALICE@EXAMPLE.COM    | EXAMPLE.COM  |
| bob@example.com      | example.com  |
| charlie@GMAIL.COM    | GMAIL.COM    |
| diana@example.com    | example.com  |
| eve.davis@work.co.uk | work.co.uk   |
+----------------------+--------------+
(5 rows)
```

SPLIT_PART — Split Strings by a Delimiter

SPLIT_PART splits a string by a delimiter and returns the Nth part.

```sql
-- SPLIT_PART(string, delimiter, position)

SELECT email,
       SPLIT_PART(email, '@', 1) AS username,
       SPLIT_PART(email, '@', 2) AS domain
FROM contacts;
```

Result:

```
+----------------------+-----------+--------------+
| email                | username  | domain       |
+----------------------+-----------+--------------+
| ALICE@EXAMPLE.COM    | ALICE     | EXAMPLE.COM  |
| bob@example.com      | bob       | example.com  |
| charlie@GMAIL.COM    | charlie   | GMAIL.COM    |
| diana@example.com    | diana     | example.com  |
| eve.davis@work.co.uk | eve.davis | work.co.uk   |
+----------------------+-----------+--------------+
(5 rows)
```

SPLIT_PART is often more readable than SUBSTRING + POSITION for splitting strings.

```sql
-- Splitting a phone number
SELECT phone,
       SPLIT_PART(phone, '-', 1) AS area_code,
       SPLIT_PART(phone, '-', 2) AS prefix,
       SPLIT_PART(phone, '-', 3) AS line_number
FROM contacts
WHERE phone LIKE '%-%-%';
```

Result:

```
+--------------+-----------+--------+-------------+
| phone        | area_code | prefix | line_number |
+--------------+-----------+--------+-------------+
| 555-123-4567 | 555       | 123    | 4567        |
| 555-111-2222 | 555       | 111    | 2222        |
+--------------+-----------+--------+-------------+
(2 rows)
```

REGEXP_MATCHES — Pattern Matching with Regular Expressions

For complex pattern matching, PostgreSQL supports regular expressions.

```sql
-- Extract numbers from phone numbers
SELECT phone,
       (REGEXP_MATCHES(phone, '\d+', 'g')) AS number_parts
FROM contacts;
```

Result:

```
+----------------+--------------+
| phone          | number_parts |
+----------------+--------------+
| 555-123-4567   | {555}        |
| 555-123-4567   | {123}        |
| 555-123-4567   | {4567}       |
| 555.987.6543   | {555}        |
| ...            | ...          |
+----------------+--------------+
```

The `'g'` flag means "global" — find all matches, not just the first.

A simpler approach for checking patterns is the `~` operator:

```sql
-- Find emails that end with .com (case-insensitive)
SELECT email
FROM contacts
WHERE email ~* '\.com$';
```

Result:

```
+----------------------+
| email                |
+----------------------+
| ALICE@EXAMPLE.COM    |
| bob@example.com      |
| charlie@GMAIL.COM    |
| diana@example.com    |
+----------------------+
(4 rows)
```

- `~` is case-sensitive regex match
- `~*` is case-insensitive regex match
- `!~` is case-sensitive regex NOT match
- `!~*` is case-insensitive regex NOT match

String Functions Quick Reference

```
+---------------+-------------------------------+--------------------+
| Function      | What It Does                  | Example            |
+---------------+-------------------------------+--------------------+
| UPPER(s)      | All uppercase                 | 'hello' -> 'HELLO' |
```

```
| LOWER(s)      | All lowercase                  | 'HELLO' -> 'hello' |
| INITCAP(s)    | Title Case                     | 'bob smith'->'Bob Smith'|
| LENGTH(s)     | Character count                | 'hello' -> 5       |
| TRIM(s)       | Remove leading/trailing spaces | ' hi ' -> 'hi'    |
| SUBSTRING     | Extract part of string         | 'hello',2,3->'ell' |
| REPLACE(s,a,b)| Replace a with b               | 'hello','l','r'->'herro'|
| CONCAT(a,b)   | Join strings (NULL-safe)       | 'a','b' -> 'ab'    |
| a || b        | Join strings (NULL=NULL)       | 'a'||'b' -> 'ab'   |
| LEFT(s,n)     | First n characters             | 'hello',3 -> 'hel' |
| RIGHT(s,n)    | Last n characters              | 'hello',3 -> 'llo' |
| POSITION(a IN b)| Find substring position      | '@' IN 'a@b' -> 2  |
| SPLIT_PART    | Split by delimiter, get Nth    | 'a-b-c','-',2->'b' |
| REGEXP_MATCHES| Regex pattern matching         | Find complex patterns|
+---------------+--------------------------------+--------------------+
```

Date Functions

CURRENT_DATE and CURRENT_TIMESTAMP

```sql
SELECT CURRENT_DATE      AS today,
       CURRENT_TIMESTAMP AS right_now,
       NOW()             AS also_right_now;
```

Result:

```
+------------+----------------------------+----------------------------+
| today      | right_now                  | also_right_now             |
+------------+----------------------------+----------------------------+
| 2024-06-15 | 2024-06-15 14:30:45.123456 | 2024-06-15 14:30:45.123456 |
+------------+----------------------------+----------------------------+
```

- `CURRENT_DATE` returns just the date (no time)
- `CURRENT_TIMESTAMP` and `NOW()` return date + time with microseconds

> Note: These return the current date and time when the query runs. Your results will differ from the examples shown here.

EXTRACT — Pull Out Parts of a Date

EXTRACT lets you get specific parts (year, month, day, hour, etc.) from a date or timestamp.

```sql
SELECT full_name,
       birth_date,
       EXTRACT(YEAR  FROM birth_date) AS birth_year,
       EXTRACT(MONTH FROM birth_date) AS birth_month,
       EXTRACT(DAY   FROM birth_date) AS birth_day,
       EXTRACT(DOW   FROM birth_date) AS day_of_week
FROM contacts;
```

Result:

```
+--------------------+------------+------------+-------------+-----------+-------------+
| full_name          | birth_date | birth_year | birth_month | birth_day | day_of_week |
+--------------------+------------+------------+-------------+-----------+-------------+
|   Alice Johnson    | 1990-03-15 |       1990 |           3 |        15 |           4 |
| bob smith          | 1985-07-22 |       1985 |           7 |        22 |           1 |
| Charlie Brown      | 1992-11-08 |       1992 |          11 |         8 |           0 |
| DIANA LEE          | 1988-01-30 |       1988 |           1 |        30 |           6 |
| Eve Davis-Wilson   | 1995-12-25 |       1995 |          12 |        25 |           1 |
+--------------------+------------+------------+-------------+-----------+-------------+
(5 rows)
```

Day of week values (DOW):

```
0 = Sunday
1 = Monday
2 = Tuesday
3 = Wednesday
4 = Thursday
5 = Friday
6 = Saturday
```

Practical: Group by Month

```sql
SELECT EXTRACT(MONTH FROM created_at) AS month,
       COUNT(*) AS signups
FROM contacts
GROUP BY EXTRACT(MONTH FROM created_at)
ORDER BY month;
```

Result:

```
+-------+---------+
| month | signups |
+-------+---------+
```

```
|     1 |       2 |
|     2 |       1 |
|     3 |       1 |
|     6 |       1 |
+-------+---------+
(4 rows)
```

DATE_TRUNC — Truncate to a Specific Precision

DATE_TRUNC rounds a date/timestamp down to the specified precision.

```sql
SELECT created_at,
       DATE_TRUNC('year',  created_at) AS trunc_year,
       DATE_TRUNC('month', created_at) AS trunc_month,
       DATE_TRUNC('day',   created_at) AS trunc_day
FROM contacts;
```

Result:

```
+---------------------+---------------------+---------------------+---------------------+
| created_at          | trunc_year          | trunc_month         | trunc_day           |
+---------------------+---------------------+---------------------+---------------------+
| 2023-01-15 09:30:00 | 2023-01-01 00:00:00 | 2023-01-01 00:00:00 | 2023-01-15 00:00:00 |
| 2023-03-20 14:15:00 | 2023-01-01 00:00:00 | 2023-03-01 00:00:00 | 2023-03-20 00:00:00 |
| 2023-06-10 11:00:00 | 2023-01-01 00:00:00 | 2023-06-01 00:00:00 | 2023-06-10 00:00:00 |
| 2024-01-05 16:45:00 | 2024-01-01 00:00:00 | 2024-01-01 00:00:00 | 2024-01-05 00:00:00 |
| 2024-02-14 08:00:00 | 2024-01-01 00:00:00 | 2024-02-01 00:00:00 | 2024-02-14 00:00:00 |
+---------------------+---------------------+---------------------+---------------------+
(5 rows)
```

DATE_TRUNC is extremely useful for grouping by time periods:

```sql
-- Count signups per month
SELECT DATE_TRUNC('month', created_at) AS month,
       COUNT(*) AS signups
FROM contacts
GROUP BY DATE_TRUNC('month', created_at)
ORDER BY month;
```

Result:

```
+---------------------+---------+
| month               | signups |
```

```
+---------------------+---------+
| 2023-01-01 00:00:00 |       1 |
| 2023-03-01 00:00:00 |       1 |
| 2023-06-01 00:00:00 |       1 |
| 2024-01-01 00:00:00 |       1 |
| 2024-02-01 00:00:00 |       1 |
+---------------------+---------+
(5 rows)
```

AGE — Calculate the Difference Between Dates

AGE returns the difference between two dates as an interval.

```sql
SELECT full_name,
       birth_date,
       AGE(birth_date) AS age
FROM contacts;
```

Result:

```
+--------------------+------------+---------------------------+
| full_name          | birth_date | age                       |
+--------------------+------------+---------------------------+
|   Alice Johnson    | 1990-03-15 | 34 years 3 mons 0 days    |
| bob smith          | 1985-07-22 | 38 years 10 mons 23 days  |
| Charlie Brown      | 1992-11-08 | 31 years 7 mons 7 days    |
| DIANA LEE          | 1988-01-30 | 36 years 4 mons 15 days   |
| Eve Davis-Wilson   | 1995-12-25 | 28 years 5 mons 20 days   |
+--------------------+------------+---------------------------+
(5 rows)
```

> Note: Your results will differ because AGE calculates from the current date.

You can also calculate the age between two specific dates:

```sql
SELECT AGE('2024-06-15', '2023-01-15') AS time_between;
-- Result: 1 year 5 mons 0 days
```

Extract Just the Years from AGE

```sql
SELECT TRIM(full_name) AS name,
       birth_date,
       EXTRACT(YEAR FROM AGE(birth_date)) AS age_years
FROM contacts;
```

Result:

```
+------------------+------------+-----------+
| name             | birth_date | age_years |
+------------------+------------+-----------+
| Alice Johnson    | 1990-03-15 |        34 |
| bob smith        | 1985-07-22 |        38 |
| Charlie Brown    | 1992-11-08 |        31 |
| DIANA LEE        | 1988-01-30 |        36 |
| Eve Davis-Wilson | 1995-12-25 |        28 |
+------------------+------------+-----------+
(5 rows)
```

Interval Arithmetic — Add and Subtract Time Periods

In PostgreSQL, you can add or subtract intervals from dates and timestamps.

```sql
SELECT CURRENT_DATE AS today,
       CURRENT_DATE + INTERVAL '7 days' AS next_week,
       CURRENT_DATE - INTERVAL '1 month' AS last_month,
       CURRENT_DATE + INTERVAL '1 year' AS next_year;
```

Result:

```
+------------+------------+------------+------------+
| today      | next_week  | last_month | next_year  |
+------------+------------+------------+------------+
| 2024-06-15 | 2024-06-22 | 2024-05-15 | 2025-06-15 |
+------------+------------+------------+------------+
```

Practical: Find Records from the Last 30 Days

```sql
SELECT TRIM(full_name) AS name,
       created_at
FROM contacts
WHERE created_at >= CURRENT_TIMESTAMP - INTERVAL '30 days';
```

Practical: Find Upcoming Birthdays (Next 30 Days)

```sql
SELECT TRIM(full_name) AS name,
       birth_date,
       EXTRACT(MONTH FROM birth_date) AS birth_month,
       EXTRACT(DAY FROM birth_date) AS birth_day
FROM contacts
WHERE (EXTRACT(MONTH FROM birth_date), EXTRACT(DAY FROM birth_date))
   IN (
       SELECT EXTRACT(MONTH FROM d), EXTRACT(DAY FROM d)
       FROM generate_series(
           CURRENT_DATE,
           CURRENT_DATE + INTERVAL '30 days',
           INTERVAL '1 day'
       ) AS d
   );
```

Date Formatting with TO_CHAR

TO_CHAR converts a date or timestamp to a formatted string.

```sql
SELECT birth_date,
       TO_CHAR(birth_date, 'MM/DD/YYYY')    AS us_format,
       TO_CHAR(birth_date, 'DD-Mon-YYYY')   AS readable,
       TO_CHAR(birth_date, 'Day, Month DD, YYYY') AS full_format,
       TO_CHAR(birth_date, 'YYYY-MM-DD')    AS iso_format
FROM contacts;
```

Result:

```
+------------+------------+-------------+----------------------------+------------+
| birth_date | us_format  | readable    | full_format                | iso_format |
+------------+------------+-------------+----------------------------+------------+
| 1990-03-15 | 03/15/1990 | 15-Mar-1990 | Thursday , March     15, 1990| 1990-03-15|
| 1985-07-22 | 07/22/1985 | 22-Jul-1985 | Monday   , July      22, 1985| 1985-07-22|
| 1992-11-08 | 11/08/1992 | 08-Nov-1992 | Sunday   , November  08, 1992| 1992-11-08|
| 1988-01-30 | 01/30/1988 | 30-Jan-1988 | Saturday , January   30, 1988| 1988-01-30|
| 1995-12-25 | 12/25/1995 | 25-Dec-1995 | Monday   , December  25, 1995| 1995-12-25|
+------------+------------+-------------+----------------------------+------------+
(5 rows)
```

Common TO_CHAR Format Codes

```
+--------+---------------------------+------------------+
| Code   | Meaning                   | Example          |
+--------+---------------------------+------------------+
| YYYY   | 4-digit year              | 2024             |
| YY     | 2-digit year              | 24               |
| MM     | Month number (01-12)      | 06               |
| Mon    | Abbreviated month name    | Jun              |
| Month  | Full month name           | June             |
| DD     | Day of month (01-31)      | 15               |
| Day    | Full day name             | Saturday         |
| Dy     | Abbreviated day name      | Sat              |
| HH24   | Hour (00-23)              | 14               |
| HH12   | Hour (01-12)              | 02               |
| MI     | Minute (00-59)            | 30               |
| SS     | Second (00-59)            | 45               |
| AM/PM  | AM or PM indicator        | PM               |
+--------+---------------------------+------------------+
```

Formatting Timestamps

```sql
SELECT created_at,
       TO_CHAR(created_at, 'YYYY-MM-DD HH24:MI:SS') AS iso_timestamp,
       TO_CHAR(created_at, 'Mon DD, YYYY at HH12:MI AM') AS readable
FROM contacts;
```

Result:

```
+---------------------+---------------------+---------------------------+
| created_at          | iso_timestamp       | readable                  |
+---------------------+---------------------+---------------------------+
| 2023-01-15 09:30:00 | 2023-01-15 09:30:00 | Jan 15, 2023 at 09:30 AM  |
| 2023-03-20 14:15:00 | 2023-03-20 14:15:00 | Mar 20, 2023 at 02:15 PM  |
| 2023-06-10 11:00:00 | 2023-06-10 11:00:00 | Jun 10, 2023 at 11:00 AM  |
| 2024-01-05 16:45:00 | 2024-01-05 16:45:00 | Jan 05, 2024 at 04:45 PM  |
| 2024-02-14 08:00:00 | 2024-02-14 08:00:00 | Feb 14, 2024 at 08:00 AM  |
+---------------------+---------------------+---------------------------+
(5 rows)
```

Casting Strings to Dates

Sometimes you receive dates as strings and need to convert them to proper date types.

```sql
-- Using CAST
SELECT CAST('2024-06-15' AS DATE) AS cast_date;

-- Using :: shorthand (PostgreSQL-specific)
SELECT '2024-06-15'::DATE AS cast_date;

-- Using TO_DATE for custom formats
SELECT TO_DATE('15/06/2024', 'DD/MM/YYYY') AS parsed_date;
SELECT TO_DATE('June 15, 2024', 'Month DD, YYYY') AS parsed_date;
```

Result:

```
+------------+
| parsed_date|
+------------+
| 2024-06-15 |
+------------+
```

For timestamps:

```sql
SELECT TO_TIMESTAMP('2024-06-15 14:30:00', 'YYYY-MM-DD HH24:MI:SS')
    AS parsed_timestamp;
```

Practical Example: Contact Data Cleanup

Let us combine several string functions to clean up our contact data.

```sql
SELECT contact_id,
       INITCAP(TRIM(full_name)) AS clean_name,
       LOWER(email) AS clean_email,
       SPLIT_PART(LOWER(email), '@', 2) AS email_domain,
       REGEXP_REPLACE(phone, '[^0-9]', '', 'g') AS clean_phone,
       EXTRACT(YEAR FROM AGE(birth_date)) AS age,
       TO_CHAR(created_at, 'Mon DD, YYYY') AS joined
FROM contacts
ORDER BY clean_name;
```

Result:

```
+----+------------------+----------------------+--------------+-------------+-----+----...
| id | clean_name       | clean_email          | email_domain | clean_phone | age | joi...
```

```
+----+------------------+----------------------+--------------+-------------+-----+----...
|  1 | Alice Johnson    | alice@example.com    | example.com  | 5551234567  |  34 | Jan...
|  2 | Bob Smith        | bob@example.com      | example.com  | 5559876543  |  38 | Mar...
|  3 | Charlie Brown    | charlie@gmail.com    | gmail.com    | 5552468101  |  31 | Jun...
|  4 | Diana Lee        | diana@example.com    | example.com  | 5553692580  |  36 | Jan...
|  5 | Eve Davis-Wilson | eve.davis@work.co.uk | work.co.uk   | 5551112222  |  28 | Feb...
+----+------------------+----------------------+--------------+-------------+-----+----...
(5 rows)
```

This query:

- Trims whitespace and applies title case to names
- Lowercases all emails for consistency
- Extracts the email domain
- Strips non-numeric characters from phone numbers
- Calculates age from birth date
- Formats the sign-up date in a readable format

Practical Example: Monthly Signup Report

```sql
SELECT TO_CHAR(DATE_TRUNC('month', created_at), 'YYYY-MM') AS month,
       COUNT(*) AS new_contacts,
       STRING_AGG(INITCAP(TRIM(full_name)), ', ' ORDER BY full_name) AS names
FROM contacts
GROUP BY DATE_TRUNC('month', created_at)
ORDER BY month;
```

Result:

```
+---------+--------------+------------------------+
| month   | new_contacts | names                  |
+---------+--------------+------------------------+
| 2023-01 |            1 | Alice Johnson          |
| 2023-03 |            1 | Bob Smith              |
| 2023-06 |            1 | Charlie Brown          |
| 2024-01 |            1 | Diana Lee              |
| 2024-02 |            1 | Eve Davis-Wilson       |
+---------+--------------+------------------------+
(5 rows)
```

`STRING_AGG` concatenates values from multiple rows into a single string, separated by the delimiter you choose (`, ` in this case).

Date Functions Quick Reference

```
+--------------------+-----------------------------------+-------------------------+
| Function           | What It Does                      | Example                 |
+--------------------+-----------------------------------+-------------------------+
| CURRENT_DATE       | Today's date                      | 2024-06-15              |
| CURRENT_TIMESTAMP  | Current date + time               | 2024-06-15 14:30:45     |
| NOW()              | Same as CURRENT_TIMESTAMP         | 2024-06-15 14:30:45     |
| EXTRACT(part FROM) | Get year, month, day, etc.        | EXTRACT(YEAR FROM d)    |
| DATE_TRUNC(prec,d) | Truncate to precision             | DATE_TRUNC('month', d)  |
| AGE(date)          | Time elapsed since date           | 34 years 3 mons        |
| AGE(d1, d2)        | Difference between two dates      | 1 year 5 mons           |
| + INTERVAL         | Add time to a date                | d + INTERVAL '7 days'   |
| - INTERVAL         | Subtract time from a date         | d - INTERVAL '1 month'  |
| TO_CHAR(d, fmt)    | Format date as string             | TO_CHAR(d, 'MM/DD/YYYY')|
| TO_DATE(s, fmt)    | Parse string as date              | TO_DATE('06/15/24',fmt) |
| TO_TIMESTAMP       | Parse string as timestamp         | TO_TIMESTAMP(s, fmt)    |
| d::DATE            | Cast to date (PostgreSQL)         | '2024-06-15'::DATE      |
+--------------------+-----------------------------------+-------------------------+
```

Common Mistakes

Mistake 1: Forgetting That || Returns NULL with NULL Inputs

```sql
-- WRONG: Returns NULL if middle_name is NULL
SELECT first_name || ' ' || middle_name || ' ' || last_name AS full_name
FROM people;
-- If middle_name is NULL, the entire result is NULL!
```

Fix: Use CONCAT or COALESCE.

```sql
-- CORRECT: CONCAT ignores NULLs
SELECT CONCAT(first_name, ' ', COALESCE(middle_name, ''), ' ', last_name)
FROM people;
```

Mistake 2: Using = for Date Comparison with Timestamps

```sql
-- WRONG: This misses rows with non-zero times
SELECT * FROM contacts
WHERE created_at = '2023-01-15';
-- Only matches exactly '2023-01-15 00:00:00.000000'
```

Fix: Use a range or DATE_TRUNC.

```sql
-- CORRECT: Range comparison
SELECT * FROM contacts
WHERE created_at >= '2023-01-15'
  AND created_at <  '2023-01-16';

-- CORRECT: Using DATE_TRUNC
SELECT * FROM contacts
WHERE DATE_TRUNC('day', created_at) = '2023-01-15';
```

Mistake 3: Confusing POSITION Return Value

```sql
-- WRONG: POSITION returns 0 (not NULL) if not found
SELECT email FROM contacts
WHERE POSITION('@' IN email) IS NULL;
-- Returns nothing! POSITION returns 0, not NULL.
```

Fix: Compare with 0.

```sql
-- CORRECT
SELECT email FROM contacts
WHERE POSITION('@' IN email) = 0;
-- Finds emails without @ signs
```

Mistake 4: Using String Functions on NULL Values

```sql
-- Most string functions return NULL when given NULL input
SELECT UPPER(NULL);     -- Returns NULL
SELECT LENGTH(NULL);    -- Returns NULL
SELECT TRIM(NULL);      -- Returns NULL
```

Fix: Use COALESCE to provide a default value before applying the function.

Best Practices

1. Clean data at the boundary. Apply TRIM, LOWER, and other cleanup functions when inserting or updating data, not just when querying. This keeps your data consistently formatted.
2. Use DATE_TRUNC for time-based grouping. It is cleaner than extracting individual parts and easier to work with in ORDER BY.
3. Prefer CONCAT over || when NULLs are possible. CONCAT treats NULL as an empty string, while || returns NULL if any operand is NULL.
4. Use SPLIT_PART over SUBSTRING+POSITION when splitting by delimiters. It is more readable and less error-prone.
5. Store dates as DATE or TIMESTAMP types, not as strings. Use TO_DATE or CAST only when receiving string input from external sources.
6. Use ISO 8601 format (YYYY-MM-DD) for date literals in your SQL. It is unambiguous and works regardless of locale settings.
7. Be aware of time zones when working with TIMESTAMP. Use `TIMESTAMP WITH TIME ZONE` (or `TIMESTAMPTZ`) for applications that span multiple time zones.

Quick Summary

Category	Key Functions		
Case	UPPER, LOWER, INITCAP		
Length/Trim	LENGTH, TRIM, LTRIM, RTRIM		
Extract/Split	SUBSTRING, LEFT, RIGHT, SPLIT_PART, POSITION		
Replace/Combine	REPLACE, CONCAT, \	\	
Pattern	REGEXP_MATCHES, ~, ~*		
Current time	CURRENT_DATE, CURRENT_TIMESTAMP, NOW()		

Date parts	EXTRACT, DATE_TRUNC		
Date math	AGE, INTERVAL arithmetic		
Formatting	TO_CHAR, TO_DATE, TO_TIMESTAMP		

Key Points

- String functions let you clean, format, split, and search text data. TRIM, LOWER, and SPLIT_PART are among the most commonly used.
- CONCAT is NULL-safe, while `||` returns NULL if any operand is NULL.
- POSITION returns 0 (not NULL) if the substring is not found.
- SPLIT_PART is a clean way to parse delimited strings.
- EXTRACT pulls out specific parts of dates (year, month, day, hour, etc.).
- DATE_TRUNC rounds down to a time precision and is ideal for grouping by time periods.
- AGE calculates the difference between dates in a human-readable interval format.
- INTERVAL arithmetic lets you add or subtract time from dates (`+ INTERVAL '7 days'`).
- TO_CHAR formats dates and timestamps as strings for display.
- TO_DATE and `::DATE` convert strings to proper date types.

Practice Questions

Question 1: What is the difference between CONCAT and || in PostgreSQL?

<details> <summary>Answer</summary>

Both concatenate strings. The key difference is NULL handling: `||` returns NULL if any operand is NULL, while CONCAT treats NULL as an empty string and continues concatenation. Use CONCAT when NULLs are possible.

</details>

Question 2: How would you extract just the year from a DATE column called `hire_date`?

<details> <summary>Answer</summary>

Use `EXTRACT(YEAR FROM hire_date)`. This returns the year as a number. For example, if hire_date is '2023-06-15', EXTRACT returns 2023.

</details>

Question 3: What is the difference between EXTRACT and DATE_TRUNC?

<details> <summary>Answer</summary>

EXTRACT returns a number representing one part of the date (e.g., the month number 6). DATE_TRUNC returns a date/timestamp truncated to the specified precision (e.g., '2024-06-01' for month truncation). DATE_TRUNC is better for grouping and ranges; EXTRACT is better when you need the numeric value of a date part.

</details>

Question 4: How do you find records from the last 7 days?

<details> <summary>Answer</summary>

```sql
SELECT * FROM table_name
WHERE date_column >= CURRENT_DATE - INTERVAL '7 days';
```

For timestamps, use `CURRENT_TIMESTAMP` instead of `CURRENT_DATE`.

</details>

Question 5: What does SPLIT_PART('hello-world-test', '-', 2) return?

<details> <summary>Answer</summary>

It returns `'world'`. SPLIT_PART splits the string by the delimiter '-' into parts ('hello', 'world', 'test') and returns the 2nd part.

</details>

Exercises

Exercise 1: Email Analysis

Write a query that shows each contact's clean name (trimmed, title case), their email domain (lowercase), and whether the domain is a free email provider (gmail.com, yahoo.com, hotmail.com) or a business email.

<details> <summary>Solution</summary>

```sql
SELECT INITCAP(TRIM(full_name)) AS name,
       LOWER(SPLIT_PART(email, '@', 2)) AS domain,
       CASE
           WHEN LOWER(SPLIT_PART(email, '@', 2))
               IN ('gmail.com', 'yahoo.com', 'hotmail.com')
               THEN 'Personal'
           ELSE 'Business'
       END AS email_type
FROM contacts
ORDER BY email_type, name;
```

Expected output:

```
+------------------+--------------+------------+
| name             | domain       | email_type |
+------------------+--------------+------------+
| Alice Johnson    | example.com  | Business   |
| Bob Smith        | example.com  | Business   |
| Diana Lee        | example.com  | Business   |
| Eve Davis-Wilson | work.co.uk   | Business   |
| Charlie Brown    | gmail.com    | Personal   |
+------------------+--------------+------------+
```

</details>

Exercise 2: Age Groups

Write a query that categorizes contacts by age group (Under 30, 30-35, 36-40, Over 40), shows their name and exact age, and counts how many contacts are in each group.

<details> <summary>Solution</summary>

```sql
SELECT INITCAP(TRIM(full_name)) AS name,
       EXTRACT(YEAR FROM AGE(birth_date))::INTEGER AS age,
       CASE
           WHEN EXTRACT(YEAR FROM AGE(birth_date)) < 30 THEN 'Under 30'
           WHEN EXTRACT(YEAR FROM AGE(birth_date)) BETWEEN 30 AND 35 THEN '30-35'
           WHEN EXTRACT(YEAR FROM AGE(birth_date)) BETWEEN 36 AND 40 THEN '36-40'
           ELSE 'Over 40'
       END AS age_group
FROM contacts
ORDER BY age;
```

For the group summary:

```sql
SELECT CASE
           WHEN EXTRACT(YEAR FROM AGE(birth_date)) < 30 THEN 'Under 30'
           WHEN EXTRACT(YEAR FROM AGE(birth_date)) BETWEEN 30 AND 35 THEN '30-35'
           WHEN EXTRACT(YEAR FROM AGE(birth_date)) BETWEEN 36 AND 40 THEN '36-40'
           ELSE 'Over 40'
       END AS age_group,
       COUNT(*) AS count
FROM contacts
GROUP BY age_group
ORDER BY age_group;
```

</details>

Exercise 3: Date-Based Reporting

Write a query that shows for each contact:

- Their clean name
- How many days since they signed up
- Which quarter they signed up in (Q1, Q2, Q3, Q4)
- Their sign-up date formatted as "January 15, 2023"

<details> <summary>Solution</summary>

```sql
SELECT INITCAP(TRIM(full_name)) AS name,
       (CURRENT_DATE - created_at::DATE) AS days_since_signup,
       'Q' || EXTRACT(QUARTER FROM created_at) AS signup_quarter,
       TO_CHAR(created_at, 'Month DD, YYYY') AS formatted_date
FROM contacts
ORDER BY created_at;
```

Expected output (days_since_signup will vary):

```
+------------------+-------------------+----------------+--------------------+
| name             | days_since_signup | signup_quarter | formatted_date     |
+------------------+-------------------+----------------+--------------------+
| Alice Johnson    |               517 | Q1             | January   15, 2023 |
| Bob Smith        |               453 | Q1             | March     20, 2023 |
| Charlie Brown    |               371 | Q2             | June      10, 2023 |
| Diana Lee        |               162 | Q1             | January   05, 2024 |
| Eve Davis-Wilson |               122 | Q1             | February  14, 2024 |
+------------------+-------------------+----------------+--------------------+
```

</details>

What Is Next?

You now have a solid toolkit for manipulating strings and dates in your queries. In the next chapter, you will learn about CASE expressions — SQL's version of if/then/else logic. CASE lets you add conditional logic directly inside your queries, transforming values, creating categories, and building dynamic columns based on conditions.

Chapter 19: CASE Expressions

What You Will Learn

In this chapter, you will learn how to make decisions inside your SQL queries. You will discover how to use CASE expressions to transform data, categorize results, and build conditional logic directly within SELECT, WHERE, ORDER BY, and aggregate functions. You will also learn about COALESCE and NULLIF, two handy functions that deal with NULL values.

Why This Chapter Matters

Imagine you are a teacher looking at a list of student scores. You do not just want to see the numbers. You want to see labels like "Pass" or "Fail" next to each score. Or imagine you are a manager who wants to group employees into salary bands like "Low," "Medium," and "High."

Without CASE expressions, you would need to pull the raw data out of the database and then write code in another programming language to add these labels. With CASE, you can do it all inside a single SQL query. This saves time, reduces complexity, and keeps your logic close to your data.

CASE expressions are one of the most practical tools in SQL. You will use them constantly in real-world reporting, data analysis, and application development.

Setting Up Our Practice Data

Before we begin, let us create some tables to work with throughout this chapter.

```sql
CREATE TABLE students (
    student_id   SERIAL PRIMARY KEY,
    first_name   VARCHAR(50),
    last_name    VARCHAR(50),
    score        INTEGER,
    grade_letter VARCHAR(2)
);

INSERT INTO students (first_name, last_name, score, grade_letter) VALUES
('Alice',   'Johnson',  92,   NULL),
('Bob',     'Smith',    85,   NULL),
('Charlie', 'Brown',    67,   NULL),
('Diana',   'Ross',     45,   NULL),
('Edward',  'Wilson',   78,   NULL),
('Fiona',   'Clark',    55,   NULL),
('George',  'Taylor',   91,   NULL),
('Hannah',  'Lee',      73,   NULL);

CREATE TABLE employees (
    employee_id  SERIAL PRIMARY KEY,
    first_name   VARCHAR(50),
    last_name    VARCHAR(50),
    department   VARCHAR(50),
    salary       INTEGER,
    status       VARCHAR(20),
    manager_id   INTEGER
);

INSERT INTO employees (first_name, last_name, department, salary, status, manager_id) V...
('Alice',   'Johnson',  'Engineering', 95000,  'active',      NULL),
('Bob',     'Smith',    'Engineering', 78000,  'active',      1),
('Charlie', 'Brown',    'Marketing',   62000,  'active',      1),
('Diana',   'Ross',     'Marketing',   58000,  'on_leave',    3),
('Edward',  'Wilson',   'Sales',       72000,  'active',      1),
('Fiona',   'Clark',    'Sales',       45000,  'terminated',  5),
('George',  'Taylor',   'Engineering', 110000, 'active',      1),
('Hannah',  'Lee',      'HR',          65000,  'active',      1),
('Ivan',    'Petrov',   'HR',          NULL,   'active',      8);
```

Simple CASE: The Switch Statement

The Simple CASE expression works like a light switch panel. You give it a value, and it checks that value against a list of options. When it finds a match, it returns the corresponding result.

Think of it like a vending machine. You press button A, you get chips. You press button B, you get a candy bar. Each button maps to one specific item.

Syntax

```
CASE expression
    WHEN value1 THEN result1
    WHEN value2 THEN result2
    WHEN value3 THEN result3
    ELSE default_result
END
```

Example: Converting Status Codes to Readable Labels

```sql
SELECT
    first_name,
    last_name,
    status,
    CASE status
        WHEN 'active'     THEN 'Currently Working'
        WHEN 'on_leave'   THEN 'On Leave'
        WHEN 'terminated' THEN 'No Longer Employed'
        ELSE 'Unknown'
    END AS status_label
FROM employees;
```

Result:

```
 first_name | last_name | status     | status_label
------------+-----------+------------+---------------------
 Alice      | Johnson   | active     | Currently Working
 Bob        | Smith     | active     | Currently Working
 Charlie    | Brown     | active     | Currently Working
 Diana      | Ross      | on_leave   | On Leave
 Edward     | Wilson    | active     | Currently Working
 Fiona      | Clark     | terminated | No Longer Employed
 George     | Taylor    | active     | Currently Working
 Hannah     | Lee       | active     | Currently Working
 Ivan       | Petrov    | active     | Currently Working
(9 rows)
```

Line-by-Line Explanation:

- `CASE status` -- We are examining the `status` column. This is the value we will compare.
- `WHEN 'active' THEN 'Currently Working'` -- If `status` equals `'active'`, return `'Currently Working'`.
- `WHEN 'on_leave' THEN 'On Leave'` -- If `status` equals `'on_leave'`, return `'On Leave'`.

- `WHEN 'terminated' THEN 'No Longer Employed'` -- If `status` equals `'terminated'`, return `'No Longer Employed'`.
- `ELSE 'Unknown'` -- If none of the above match, return `'Unknown'`. The ELSE is optional but recommended.
- `END AS status_label` -- Close the CASE block and give the resulting column the alias `status_label`.

How Simple CASE Evaluates

```
+-------------------+
|  CASE status      |
+-------------------+
        |
        v
+-------------------+     YES    +----------------------+
| status = 'active'|---------->| 'Currently Working' |
+-------------------+            +----------------------+
        | NO
        v
+-------------------+     YES    +----------------------+
| status = 'on_leave'|-------->| 'On Leave'          |
+-------------------+            +----------------------+
        | NO
        v
+-------------------+     YES    +----------------------+
| status = 'terminated'|------>| 'No Longer Employed'|
+-------------------+            +----------------------+
        | NO
        v
+-------------------+
|    ELSE           |
|   'Unknown'       |
+-------------------+
```

The Simple CASE checks each WHEN from top to bottom. The moment it finds a match, it stops and returns that result. If nothing matches, it falls through to the ELSE.

Searched CASE: The If/Else If Statement

The Searched CASE is more powerful than the Simple CASE. Instead of comparing one value against a list of options, each WHEN clause contains its own independent condition. This means you can test different columns, use comparison operators, and combine conditions.

Think of it like a series of questions a doctor asks. "Is your temperature above 102? Then you have a high fever. Is it between 99 and 102? Then you have a low fever. Otherwise, you are normal." Each question is a separate test.

Syntax

```
CASE
    WHEN condition1 THEN result1
    WHEN condition2 THEN result2
    WHEN condition3 THEN result3
    ELSE default_result
END
```

Notice there is no expression after the word CASE. That is the key difference from Simple CASE.

Example: Categorizing Salaries

```sql
SELECT
    first_name,
    last_name,
    salary,
    CASE
        WHEN salary >= 90000 THEN 'High'
        WHEN salary >= 60000 THEN 'Medium'
        WHEN salary IS NOT NULL THEN 'Low'
        ELSE 'Not Set'
    END AS salary_band
FROM employees;
```

Result:

```
 first_name | last_name | salary  | salary_band
------------+-----------+---------+------------
 Alice      | Johnson   |  95000  | High
 Bob        | Smith     |  78000  | Medium
 Charlie    | Brown     |  62000  | Medium
 Diana      | Ross      |  58000  | Low
 Edward     | Wilson    |  72000  | Medium
 Fiona      | Clark     |  45000  | Low
 George     | Taylor    | 110000  | High
 Hannah     | Lee       |  65000  | Medium
 Ivan       | Petrov    |  (null) | Not Set
(9 rows)
```

Line-by-Line Explanation:

- `CASE` -- Start a Searched CASE. No expression follows the word CASE.
- `WHEN salary >= 90000 THEN 'High'` -- If salary is 90,000 or more, label it "High."
- `WHEN salary >= 60000 THEN 'Medium'` -- If salary is between 60,000 and 89,999, label it "Medium." Notice we do not need to say `salary < 90000` because CASE checks conditions in order. If salary were 95,000, it would have already matched the first WHEN and stopped.
- `WHEN salary IS NOT NULL THEN 'Low'` -- If salary has a value but did not match the previous conditions, label it "Low."
- `ELSE 'Not Set'` -- If salary is NULL (none of the above conditions were true because comparisons with NULL return false), label it "Not Set."
- `END AS salary_band` -- Close the CASE and name the column `salary_band`.

> Important: CASE evaluates conditions from top to bottom and stops at the first match. The order of your WHEN clauses matters. Put the most specific conditions first.

CASE in SELECT: Practical Categorization

We already saw CASE in SELECT above. Let us look at a more detailed example that grades students based on their scores.

Example: Grading Students

```sql
SELECT
    first_name,
    last_name,
    score,
    CASE
        WHEN score >= 90 THEN 'A'
        WHEN score >= 80 THEN 'B'
        WHEN score >= 70 THEN 'C'
        WHEN score >= 60 THEN 'D'
        ELSE 'F'
    END AS letter_grade,
    CASE
        WHEN score >= 60 THEN 'Pass'
        ELSE 'Fail'
    END AS result
FROM students
ORDER BY score DESC;
```

Result:

```
 first_name | last_name | score | letter_grade | result
------------+-----------+-------+--------------+-------
 Alice      | Johnson   |    92 | A            | Pass
 George     | Taylor    |    91 | A            | Pass
 Bob        | Smith     |    85 | B            | Pass
 Edward     | Wilson    |    78 | C            | Pass
 Hannah     | Lee       |    73 | C            | Pass
 Charlie    | Brown     |    67 | D            | Pass
 Fiona      | Clark     |    55 | F            | Fail
 Diana      | Ross      |    45 | F            | Fail
(8 rows)
```

You can use multiple CASE expressions in the same SELECT. Each one creates an independent column. Here, we created both a `letter_grade` and a `result` column in one query.

CASE in WHERE

You can use CASE inside a WHERE clause, though this is less common. It is useful when you need conditional filtering logic.

Example: Filter Based on Department Rules

Suppose you want to find employees who are "underpaid" based on department-specific thresholds. Engineering employees should earn at least 80,000, while other departments should earn at least 55,000.

```sql
SELECT
    first_name,
    last_name,
    department,
    salary
FROM employees
WHERE salary < CASE
    WHEN department = 'Engineering' THEN 80000
    WHEN department = 'Sales'       THEN 60000
    ELSE 55000
END
AND salary IS NOT NULL;
```

Result:

```
 first_name | last_name | department   | salary
------------+-----------+-------------+-------
 Bob        | Smith     | Engineering | 78000
 Fiona      | Clark     | Sales       | 45000
(2 rows)
```

Explanation:

- Bob earns 78,000 in Engineering, which is below the 80,000 threshold for that department.
- Fiona earns 45,000 in Sales, which is below the 60,000 threshold for Sales.
- Other employees either meet their department threshold or have NULL salaries (filtered out by `AND salary IS NOT NULL`).

CASE in ORDER BY

CASE in ORDER BY lets you create custom sorting orders that do not follow alphabetical or numerical rules.

Example: Custom Department Sort Order

Suppose you want Engineering first, then Sales, then Marketing, then everything else.

```sql
SELECT
    first_name,
    department,
    salary
FROM employees
WHERE salary IS NOT NULL
ORDER BY
    CASE department
        WHEN 'Engineering' THEN 1
        WHEN 'Sales'       THEN 2
        WHEN 'Marketing'   THEN 3
        ELSE 4
    END,
    salary DESC;
```

Result:

```
 first_name | department   | salary
------------+-------------+--------
```

```
 George     | Engineering | 110000
 Alice      | Engineering |  95000
 Bob        | Engineering |  78000
 Edward     | Sales       |  72000
 Fiona      | Sales       |  45000
 Charlie    | Marketing   |  62000
 Diana      | Marketing   |  58000
 Hannah     | HR          |  65000
(8 rows)
```

How it works:

The CASE assigns a sort number to each department. Engineering gets 1, so it comes first. Within each department, we sort by salary in descending order. This gives you complete control over the display order.

CASE with Aggregates: Pivot-Like Queries

One of the most powerful uses of CASE is combining it with aggregate functions to create pivot-table-like results. This lets you turn row values into columns.

Example: Count Employees by Status per Department

```sql
SELECT
    department,
    COUNT(*) AS total,
    COUNT(CASE WHEN status = 'active'     THEN 1 END) AS active,
    COUNT(CASE WHEN status = 'on_leave'   THEN 1 END) AS on_leave,
    COUNT(CASE WHEN status = 'terminated' THEN 1 END) AS terminated
FROM employees
GROUP BY department
ORDER BY department;
```

Result:

```
 department  | total | active | on_leave | terminated
-------------+-------+--------+----------+-----------
 Engineering |     3 |      3 |        0 |          0
 HR          |     2 |      2 |        0 |          0
 Marketing   |     2 |      1 |        1 |          0
 Sales       |     2 |      1 |        0 |          1
(4 rows)
```

Line-by-Line Explanation:

- `COUNT(*)` -- Counts all employees in each department.
- `COUNT(CASE WHEN status = 'active' THEN 1 END)` -- For each row, if the status is `'active'`, CASE returns 1. If not, CASE returns NULL (because there is no ELSE). COUNT ignores NULLs, so it only counts the active employees.
- The same pattern repeats for `'on_leave'` and `'terminated'`.

This technique effectively pivots the `status` values from rows into columns.

Example: Salary Summary by Department

```sql
SELECT
    department,
    SUM(salary) AS total_salary,
    SUM(CASE WHEN status = 'active' THEN salary ELSE 0 END) AS active_salary,
    ROUND(AVG(salary)) AS avg_salary,
    MAX(salary) AS max_salary,
    MIN(salary) AS min_salary
FROM employees
WHERE salary IS NOT NULL
GROUP BY department
ORDER BY total_salary DESC;
```

Result:

```
 department  | total_salary | active_salary | avg_salary | max_salary | min_salary
-------------+--------------+---------------+------------+------------+-----------
 Engineering |       283000 |        283000 |      94333 |     110000 |      78000
 Marketing   |       120000 |         62000 |      60000 |      62000 |      58000
 Sales       |       117000 |         72000 |      58500 |      72000 |      45000
 HR          |        65000 |         65000 |      65000 |      65000 |      65000
(4 rows)
```

Here, `active_salary` uses CASE inside SUM to only add up salaries of active employees. Diana is on leave, so her 58,000 salary is excluded from Marketing's `active_salary`.

COALESCE: First Non-NULL Value

COALESCE is a function that takes a list of values and returns the first one that is not NULL. Think of it like a backup plan. "Try this first. If it is NULL, try this. If that is also NULL, use this default."

Syntax

```
COALESCE(value1, value2, value3, ...)
```

Example: Displaying a Fallback for Missing Salaries

```sql
SELECT
    first_name,
    last_name,
    salary,
    COALESCE(salary, 0) AS salary_or_zero,
    COALESCE(CAST(salary AS VARCHAR), 'Not Assigned') AS salary_display
FROM employees
WHERE department = 'HR';
```

Result:

```
 first_name | last_name | salary | salary_or_zero | salary_display
------------+-----------+--------+----------------+---------------
 Hannah     | Lee       |  65000 |          65000 | 65000
 Ivan       | Petrov    | (null) |              0 | Not Assigned
(2 rows)
```

Explanation:

- For Hannah, `salary` is 65,000 (not NULL), so COALESCE returns 65,000.
- For Ivan, `salary` is NULL, so COALESCE moves to the next value: 0 for the numeric version, or `'Not Assigned'` for the text version.

COALESCE with Multiple Fallbacks

```sql
-- Imagine a contacts table with multiple phone fields
SELECT
    COALESCE(work_phone, mobile_phone, home_phone, 'No Phone') AS best_phone
FROM contacts;
```

This tries `work_phone` first. If that is NULL, it tries `mobile_phone`. If that is also NULL, it tries `home_phone`. If all three are NULL, it returns `'No Phone'`.

How COALESCE Works

```
COALESCE(NULL, NULL, 42, 99)
         |     |     |
         v     v     v
        NULL? NULL? 42!  --> Returns 42 (first non-NULL)

COALESCE('hello', NULL, 'world')
          |
          v
        'hello'!  --> Returns 'hello' (first non-NULL)

COALESCE(NULL, NULL, NULL)
         |     |     |
         v     v     v
        NULL? NULL? NULL?  --> Returns NULL (no non-NULL found)
```

NULLIF: Create NULL from a Specific Value

NULLIF does the opposite of COALESCE in a way. It takes two arguments. If they are equal, it returns NULL. If they are different, it returns the first argument.

Syntax

```
NULLIF(value1, value2)
```

- If `value1 = value2`, return NULL.
- If `value1 != value2`, return `value1`.

Example: Avoiding Division by Zero

NULLIF is most commonly used to prevent division-by-zero errors.

```sql
CREATE TABLE products (
    product_id   SERIAL PRIMARY KEY,
    product_name VARCHAR(100),
    list_price   NUMERIC(10,2),
    sale_price   NUMERIC(10,2)
);

INSERT INTO products (product_name, list_price, sale_price) VALUES
('Widget A', 25.00, 20.00),
('Widget B', 30.00, 30.00),
```

```
('Widget C', 15.00,  0.00),
('Widget D', 40.00, 35.00);
```

```sql
SELECT
    product_name,
    list_price,
    sale_price,
    ROUND(list_price / NULLIF(sale_price, 0), 2) AS price_ratio
FROM products;
```

Result:

```
 product_name | list_price | sale_price | price_ratio
--------------+------------+------------+------------
 Widget A     |      25.00 |      20.00 |        1.25
 Widget B     |      30.00 |      30.00 |        1.00
 Widget C     |      15.00 |       0.00 |      (null)
 Widget D     |      40.00 |      35.00 |        1.14
(4 rows)
```

Explanation:

- For Widget C, `sale_price` is 0. Without NULLIF, dividing by 0 would cause an error. `NULLIF(0, 0)` returns NULL, and dividing by NULL produces NULL instead of crashing.
- For all other products, `sale_price` is not 0, so NULLIF returns the original `sale_price` value, and division proceeds normally.

Another Use: Treating Empty Strings as NULL

```sql
SELECT
    NULLIF(email, '') AS email_or_null
FROM users;
```

If `email` is an empty string `''`, NULLIF converts it to NULL. This is useful when some systems store empty strings instead of NULL for missing data.

Practical Example: Complete Student Grading System

Let us bring everything together with a complete grading system.

```sql
SELECT
    first_name || ' ' || last_name AS student_name,
    score,
    CASE
        WHEN score >= 90 THEN 'A'
        WHEN score >= 80 THEN 'B'
        WHEN score >= 70 THEN 'C'
        WHEN score >= 60 THEN 'D'
        ELSE 'F'
    END AS letter_grade,
    CASE
        WHEN score >= 90 THEN 'Excellent'
        WHEN score >= 80 THEN 'Good'
        WHEN score >= 70 THEN 'Satisfactory'
        WHEN score >= 60 THEN 'Needs Improvement'
        ELSE 'Failing'
    END AS performance,
    CASE
        WHEN score >= 60 THEN 'Pass'
        ELSE 'Fail'
    END AS result,
    CASE
        WHEN score >= 90 THEN 4.0
        WHEN score >= 80 THEN 3.0
        WHEN score >= 70 THEN 2.0
        WHEN score >= 60 THEN 1.0
        ELSE 0.0
    END AS grade_points
FROM students
ORDER BY score DESC;
```

Result:

```
  student_name   | score | letter_grade | performance       | result | grade_points
-----------------+-------+--------------+-------------------+--------+-------------
 Alice Johnson   |    92 | A            | Excellent         | Pass   |          4.0
 George Taylor   |    91 | A            | Excellent         | Pass   |          4.0
 Bob Smith       |    85 | B            | Good              | Pass   |          3.0
 Edward Wilson   |    78 | C            | Satisfactory      | Pass   |          2.0
 Hannah Lee      |    73 | C            | Satisfactory      | Pass   |          2.0
 Charlie Brown   |    67 | D            | Needs Improvement | Pass   |          1.0
 Fiona Clark     |    55 | F            | Failing           | Fail   |          0.0
 Diana Ross      |    45 | F            | Failing           | Fail   |          0.0
(8 rows)
```

Grade Distribution Summary

```sql
SELECT
    CASE
        WHEN score >= 90 THEN 'A'
        WHEN score >= 80 THEN 'B'
        WHEN score >= 70 THEN 'C'
        WHEN score >= 60 THEN 'D'
        ELSE 'F'
    END AS letter_grade,
    COUNT(*) AS num_students,
    ROUND(AVG(score), 1) AS avg_score,
    MIN(score) AS lowest,
    MAX(score) AS highest
FROM students
GROUP BY
    CASE
        WHEN score >= 90 THEN 'A'
        WHEN score >= 80 THEN 'B'
        WHEN score >= 70 THEN 'C'
        WHEN score >= 60 THEN 'D'
        ELSE 'F'
    END
ORDER BY
    MIN(score) DESC;
```

Result:

```
 letter_grade | num_students | avg_score | lowest | highest
--------------+--------------+-----------+--------+--------
 A            |            2 |      91.5 |     91 |      92
 B            |            1 |      85.0 |     85 |      85
 C            |            2 |      75.5 |     73 |      78
 D            |            1 |      67.0 |     67 |      67
 F            |            2 |      50.0 |     45 |      55
(5 rows)
```

> Note: When you use a CASE expression in GROUP BY, you must repeat the entire CASE expression. You cannot use the alias (`letter_grade`) in GROUP BY in standard SQL. PostgreSQL does allow column aliases in GROUP BY, but repeating the expression is the portable approach.

Practical Example: Employee Status Dashboard

```sql
SELECT
    department,
    COUNT(*) AS headcount,
    SUM(COALESCE(salary, 0)) AS total_payroll,
    COUNT(CASE WHEN status = 'active' THEN 1 END) AS active_count,
    COUNT(CASE WHEN status = 'on_leave' THEN 1 END) AS leave_count,
    COUNT(CASE WHEN status = 'terminated' THEN 1 END) AS termed_count,
    ROUND(
        100.0 * COUNT(CASE WHEN status = 'active' THEN 1 END) / COUNT(*),
        1
    ) AS active_pct
FROM employees
GROUP BY department
ORDER BY headcount DESC;
```

Result:

```
 department  | headcount | total_payroll | active_count | leave_count | termed_count | ...
-------------+-----------+---------------+--------------+-------------+--------------+-...
 Engineering |         3 |        283000 |            3 |           0 |            0 | ...
 Marketing   |         2 |        120000 |            1 |           1 |            0 | ...
 Sales       |         2 |        117000 |            1 |           0 |            1 | ...
 HR          |         2 |         65000 |            2 |           0 |            0 | ...
(4 rows)
```

This query combines CASE with aggregates and COALESCE to build a department dashboard in a single query.

Common Mistakes

Mistake 1: Forgetting END

```sql
-- WRONG: Missing END
SELECT
    CASE status
        WHEN 'active' THEN 'Working'
        WHEN 'on_leave' THEN 'Away'
    AS label
FROM employees;

-- ERROR: syntax error at or near "AS"
```

```sql
-- CORRECT: Always close with END
SELECT
    CASE status
        WHEN 'active' THEN 'Working'
        WHEN 'on_leave' THEN 'Away'
    END AS label
FROM employees;
```

Mistake 2: Wrong Order of WHEN Clauses

```sql
-- WRONG: The second WHEN will never match
SELECT
    score,
    CASE
        WHEN score >= 60 THEN 'Pass'
        WHEN score >= 90 THEN 'Excellent'  -- Never reached!
        ELSE 'Fail'
    END AS result
FROM students;
```

A score of 95 matches `score >= 60` first and returns "Pass." It never gets to check `score >= 90`. Always put the most specific (highest threshold) condition first.

Mistake 3: Mixing Simple CASE and Searched CASE Syntax

```sql
-- WRONG: Cannot use conditions with Simple CASE
SELECT
    CASE department
        WHEN salary > 90000 THEN 'High Paid'  -- ERROR!
    END
FROM employees;
```

Simple CASE compares against exact values. If you need conditions with operators like `>`, `<`, or `BETWEEN`, use Searched CASE (no expression after CASE).

Mistake 4: Forgetting ELSE

```sql
-- Missing ELSE returns NULL for unmatched rows
SELECT
    CASE status
        WHEN 'active' THEN 'Working'
    END AS label
```

```
FROM employees;
-- 'on_leave' and 'terminated' employees get NULL
```

Always include ELSE unless you intentionally want NULL for unmatched cases.

Mistake 5: Using COALESCE with Mismatched Types

```sql
-- WRONG: Mixing integer and text
SELECT COALESCE(salary, 'Not Set') FROM employees;
-- ERROR: COALESCE types integer and text cannot be matched

-- CORRECT: Cast to the same type
SELECT COALESCE(CAST(salary AS VARCHAR), 'Not Set') FROM employees;
```

Best Practices

1. Always include ELSE. Even if you think all cases are covered, an ELSE clause acts as a safety net. It makes your intent clear and handles unexpected values.
2. Order WHEN clauses carefully. Put the most specific conditions first in Searched CASE. Remember that evaluation stops at the first match.
3. Use meaningful aliases. Always give your CASE expression an alias with `AS`. Without it, the column will have no useful name.
4. Keep CASE expressions readable. If a CASE has more than five or six WHEN clauses, consider whether a lookup table might be cleaner.
5. Use COALESCE for NULL defaults. It is cleaner than writing `CASE WHEN value IS NULL THEN default ELSE value END`.
6. Use NULLIF to prevent division by zero. Wrap the divisor in NULLIF(divisor, 0) to safely handle zero values.
7. Use CASE with aggregates for pivot-like reports. This is one of the most valuable patterns in real-world SQL.

Quick Summary

```
+-------------------+-----------------------------------------+
| Feature           | Description                             |
+-------------------+-----------------------------------------+
| Simple CASE       | Compare one value against a list        |
|                   | CASE expr WHEN val THEN result END      |
+-------------------+-----------------------------------------+
| Searched CASE     | Evaluate independent conditions         |
|                   | CASE WHEN cond THEN result END          |
+-------------------+-----------------------------------------+
| CASE in SELECT    | Create computed/categorized columns     |
+-------------------+-----------------------------------------+
| CASE in WHERE     | Conditional filtering                   |
+-------------------+-----------------------------------------+
| CASE in ORDER BY  | Custom sort orders                      |
+-------------------+-----------------------------------------+
| CASE + Aggregates | Pivot rows into columns                 |
+-------------------+-----------------------------------------+
| COALESCE          | Returns first non-NULL value            |
|                   | COALESCE(a, b, c, default)              |
+-------------------+-----------------------------------------+
| NULLIF            | Returns NULL if two values are equal    |
|                   | NULLIF(value, 0) prevents /0 errors     |
+-------------------+-----------------------------------------+
```

Key Points

- Simple CASE compares one expression against multiple values, like a switch statement.
- Searched CASE evaluates independent Boolean conditions, like an if/else if chain.
- CASE expressions work in SELECT, WHERE, ORDER BY, GROUP BY, and inside aggregate functions.
- Evaluation stops at the first matching WHEN. Order matters.
- COALESCE returns the first non-NULL value from a list of arguments.
- NULLIF returns NULL if two values are equal. It is most useful for preventing division-by-zero errors.
- Combining CASE with COUNT, SUM, and AVG lets you create pivot-table-like reports.
- Always include ELSE and always use AS to name your CASE columns.

Practice Questions

1. What is the difference between Simple CASE and Searched CASE? When would you use one over the other?
2. What does the following expression return: `COALESCE(NULL, NULL, 'hello', 'world')`? Explain why.
3. Write a CASE expression that labels employees as "Senior" if their salary is 80,000 or more, "Mid-Level" if 50,000 or more, and "Junior" otherwise.
4. Why does `NULLIF(0, 0)` return NULL, and how does this help with division?
5. You have a CASE expression with these WHEN clauses in this order: `WHEN score >= 50`, `WHEN score >= 70`, `WHEN score >= 90`. What is wrong with this order? What would a score of 95 return?

Exercises

Exercise 1: Product Price Categories

Using the `products` table, write a query that categorizes each product as "Premium" (list_price >= 30), "Standard" (list_price >= 15), or "Budget" (less than 15). Show the product name, list price, and category. Sort by list price descending.

Exercise 2: Employee Salary Report with COALESCE

Write a query that shows all employees with their names, departments, and salaries. Replace any NULL salary with 0 using COALESCE. Add a CASE column that shows "Above Average" if the salary is above 70,000, "Average" for 50,000 to 70,000, and "Below Average" or "No Salary" as appropriate.

Exercise 3: Department Pivot Report

Write a single query that shows each department with separate columns counting the number of employees earning "High" (>= 90,000), "Medium" (60,000-89,999), and "Low" (< 60,000) salaries. Use CASE inside COUNT. Include a total column.

What Is Next?

Now that you can add conditional logic to your queries with CASE expressions, the next chapter will introduce Views. Views let you save your complex queries (including those with CASE expressions) as reusable virtual tables. Instead of writing the same long query over and over, you can save it once as a view and then query it with a simple SELECT statement.

Chapter 20: Views

What You Will Learn

In this chapter, you will learn how to create views, which are saved queries that act like virtual tables. You will discover how to simplify complex queries, control data access, and build reusable abstractions. You will also learn about materialized views, which store cached results for faster performance.

Why This Chapter Matters

Imagine you live in a house with many windows. Each window gives you a different view of the outside world. The kitchen window shows the garden. The living room window shows the street. The bedroom window shows the backyard. The world outside does not change based on which window you look through, but each window frames a different perspective.

Views in SQL work the same way. Your data sits in tables (the outside world). A view is a window that shows a specific perspective of that data. You do not duplicate the data. You simply define a way to look at it.

In the real world, you will encounter situations where:

- A complex query joins five tables and gets used in ten different reports.
- You want to give a user access to employee names and departments but not salaries.
- You want to hide the complexity of your database schema from application developers.

Views solve all these problems. They are one of the most widely used features in production databases.

What Is a View?

A view is a saved SQL query that you can use as if it were a table. When you query a view, PostgreSQL runs the underlying query and returns the results. The view itself does not store any data (with one exception we will cover later).

```
+---------------------------+
|      Regular Tables       |
|  (Actually store data)    |
+---------------------------+
         |
         | SELECT queries
         v
+---------------------------+
|         View              |
|  (Saved query definition) |
|  (No data stored)         |
+---------------------------+
         |
         | You query the view
         v
+---------------------------+
|   Your Results            |
|  (Generated on the fly)   |
+---------------------------+
```

Think of a view like a recipe card. The recipe card does not contain the actual cake. It contains instructions for making the cake. Every time you follow the recipe, you get a fresh cake. Similarly, every time you query a view, PostgreSQL follows the saved query and produces fresh results from the current data.

Setting Up Our Practice Data

Let us create tables to work with throughout this chapter.

```sql
CREATE TABLE departments (
    department_id   SERIAL PRIMARY KEY,
    department_name VARCHAR(50) NOT NULL,
    location        VARCHAR(100)
);

CREATE TABLE employees (
    employee_id  SERIAL PRIMARY KEY,
    first_name   VARCHAR(50) NOT NULL,
    last_name    VARCHAR(50) NOT NULL,
```

```
    email        VARCHAR(100),
    department_id INTEGER REFERENCES departments(department_id),
    salary       NUMERIC(10,2),
    hire_date    DATE,
    is_active    BOOLEAN DEFAULT TRUE
);

CREATE TABLE sales (
    sale_id      SERIAL PRIMARY KEY,
    employee_id  INTEGER REFERENCES employees(employee_id),
    sale_date    DATE NOT NULL,
    amount       NUMERIC(10,2) NOT NULL,
    product_name VARCHAR(100)
);

INSERT INTO departments (department_name, location) VALUES
('Engineering', 'Building A'),
('Marketing',   'Building B'),
('Sales',       'Building C'),
('HR',          'Building A');

INSERT INTO employees (first_name, last_name, email, department_id, salary, hire_date) ...
('Alice',   'Johnson', 'alice@company.com',   1, 95000,  '2020-03-15'),
('Bob',     'Smith',   'bob@company.com',     1, 78000,  '2021-07-01'),
('Charlie', 'Brown',   'charlie@company.com', 2, 62000,  '2019-11-20'),
('Diana',   'Ross',    'diana@company.com',   2, 58000,  '2022-01-10'),
('Edward',  'Wilson',  'edward@company.com',  3, 72000,  '2020-08-05'),
('Fiona',   'Clark',   'fiona@company.com',   3, 45000,  '2023-02-14'),
('George',  'Taylor',  'george@company.com',  1, 110000, '2018-06-01'),
('Hannah',  'Lee',     'hannah@company.com',  4, 65000,  '2021-09-30');

INSERT INTO employees (first_name, last_name, email, department_id, salary, hire_date, ...
('Ivan',    'Petrov',  'ivan@company.com',    3, 55000,  '2022-05-15', FALSE);

INSERT INTO sales (employee_id, sale_date, amount, product_name) VALUES
(5, '2024-01-15', 15000, 'Enterprise License'),
(5, '2024-02-20', 8500,  'Team License'),
(5, '2024-03-10', 22000, 'Enterprise License'),
(6, '2024-01-22', 5000,  'Starter License'),
(6, '2024-02-28', 7500,  'Team License'),
(6, '2024-03-15', 3000,  'Starter License'),
(5, '2024-03-25', 18000, 'Enterprise License'),
(6, '2024-01-05', 4500,  'Starter License');
```

Creating a View: CREATE VIEW

The syntax for creating a view is straightforward.

Syntax

```sql
CREATE VIEW view_name AS
SELECT ...
FROM ...
WHERE ...;
```

Example: Employee Directory View

Let us create a simple employee directory that shows names, emails, and department names, but hides salaries and internal IDs.

```sql
CREATE VIEW employee_directory AS
SELECT
    e.first_name,
    e.last_name,
    e.email,
    d.department_name,
    e.hire_date
FROM employees e
JOIN departments d ON e.department_id = d.department_id
WHERE e.is_active = TRUE;
```

Line-by-Line Explanation:

- `CREATE VIEW employee_directory AS` -- Define a new view called `employee_directory`. Everything after `AS` is the query that defines what the view shows.
- `SELECT e.first_name, e.last_name, e.email, d.department_name, e.hire_date` -- Choose only the columns we want to expose. Notice we exclude `salary`, `employee_id`, and `department_id`.
- `FROM employees e JOIN departments d ON e.department_id = d.department_id` -- Join the two tables to get department names.
- `WHERE e.is_active = TRUE` -- Only include active employees. Terminated employees are hidden.

Using Views Like Tables

Once a view is created, you use it exactly like a table in SELECT statements.

```sql
SELECT * FROM employee_directory;
```

Result:

```
 first_name | last_name | email                | department_name | hire_date
------------+-----------+----------------------+-----------------+-----------
 Alice      | Johnson   | alice@company.com    | Engineering     | 2020-03-15
 Bob        | Smith     | bob@company.com      | Engineering     | 2021-07-01
 Charlie    | Brown     | charlie@company.com  | Marketing       | 2019-11-20
 Diana      | Ross      | diana@company.com    | Marketing       | 2022-01-10
 Edward     | Wilson    | edward@company.com   | Sales           | 2020-08-05
 Fiona      | Clark     | fiona@company.com    | Sales           | 2023-02-14
 George     | Taylor    | george@company.com   | Engineering     | 2018-06-01
 Hannah     | Lee       | hannah@company.com   | HR              | 2021-09-30
(8 rows)
```

Notice that Ivan (who has `is_active = FALSE`) does not appear. The view filters him out automatically.

Filtering a View

You can add WHERE clauses when querying a view, just like a table.

```sql
SELECT first_name, last_name, department_name
FROM employee_directory
WHERE department_name = 'Engineering'
ORDER BY hire_date;
```

Result:

```
 first_name | last_name | department_name
------------+-----------+----------------
 George     | Taylor    | Engineering
 Alice      | Johnson   | Engineering
 Bob        | Smith     | Engineering
(3 rows)
```

Joining a View with Other Tables

Views can be joined with tables or other views.

```sql
SELECT
    ed.first_name,
    ed.last_name,
    COUNT(s.sale_id) AS total_sales,
    COALESCE(SUM(s.amount), 0) AS total_revenue
FROM employee_directory ed
LEFT JOIN sales s ON ed.first_name = 'Edward' AND s.employee_id = 5
    OR ed.first_name = 'Fiona' AND s.employee_id = 6
WHERE ed.department_name = 'Sales'
GROUP BY ed.first_name, ed.last_name;
```

Views participate in queries just like regular tables. Behind the scenes, PostgreSQL merges the view's query with your outer query and optimizes them together.

How Views Work Behind the Scenes

When you query a view, PostgreSQL does not materialize the view first and then filter it. Instead, it merges the view's query definition into your query and optimizes the combined result.

```
Your Query:
  SELECT * FROM employee_directory WHERE department_name = 'Sales';

PostgreSQL Expands To:
  SELECT e.first_name, e.last_name, e.email, d.department_name, e.hire_date
  FROM employees e
  JOIN departments d ON e.department_id = d.department_id
  WHERE e.is_active = TRUE
    AND d.department_name = 'Sales';
```

This means views have no performance overhead compared to writing the query directly. They are purely a convenience feature.

CREATE OR REPLACE VIEW

If you need to modify an existing view, you can use `CREATE OR REPLACE VIEW`. This updates the view definition without dropping and recreating it.

```sql
CREATE OR REPLACE VIEW employee_directory AS
SELECT
    e.first_name,
    e.last_name,
    e.email,
    d.department_name,
    d.location,           -- Added this column
    e.hire_date
FROM employees e
JOIN departments d ON e.department_id = d.department_id
WHERE e.is_active = TRUE;
```

Important rules for CREATE OR REPLACE:

- You can add new columns to the end of the SELECT list.
- You cannot remove existing columns.
- You cannot change the data type of existing columns.
- You cannot reorder existing columns.

If you need to make those kinds of changes, you must DROP the view first and recreate it.

```sql
SELECT * FROM employee_directory;
```

Result:

```
 first_name | last_name | email                | department_name | location   | hire_date
------------+-----------+----------------------+-----------------+------------+-----------
 Alice      | Johnson   | alice@company.com    | Engineering     | Building A | 2020-03-15
 Bob        | Smith     | bob@company.com      | Engineering     | Building A | 2021-07-01
 Charlie    | Brown     | charlie@company.com  | Marketing       | Building B | 2019-11-20
 Diana      | Ross      | diana@company.com    | Marketing       | Building B | 2022-01-10
 Edward     | Wilson    | edward@company.com   | Sales           | Building C | 2020-08-05
 Fiona      | Clark     | fiona@company.com    | Sales           | Building C | 2023-02-14
 George     | Taylor    | george@company.com   | Engineering     | Building A | 2018-06-01
 Hannah     | Lee       | hannah@company.com   | HR              | Building A | 2021-09-30
(8 rows)
```

WITH CHECK OPTION

When a view has a WHERE clause, you might be able to INSERT or UPDATE data through the view (for simple views). But what happens if someone inserts a row through the view that violates the view's WHERE condition? The row would go into the underlying table but would

immediately become invisible through the view. This is confusing.

`WITH CHECK OPTION` prevents this. It ensures that any INSERT or UPDATE through the view must produce a row that is visible through the view.

Example

```sql
CREATE VIEW active_employees AS
SELECT
    employee_id,
    first_name,
    last_name,
    is_active
FROM employees
WHERE is_active = TRUE
WITH CHECK OPTION;
```

Now if someone tries to insert an inactive employee through this view:

```sql
INSERT INTO active_employees (first_name, last_name, is_active)
VALUES ('Test', 'User', FALSE);
```

```
ERROR: new row violates check option for view "active_employees"
DETAIL: Failing row contains (10, Test, User, ..., f).
```

The insert is rejected because `is_active = FALSE` violates the view's WHERE condition.

```
+-------------------------------------------+
|  View: active_employees                   |
|  WHERE is_active = TRUE                   |
|  WITH CHECK OPTION                        |
+-------------------------------------------+
        |                          |
        v                          v
  INSERT (is_active=TRUE)    INSERT (is_active=FALSE)
        |                          |
        v                          v
     Allowed                   REJECTED!
```

DROP VIEW

To remove a view, use the DROP VIEW statement.

```sql
DROP VIEW employee_directory;
```

If other views depend on the view you are trying to drop, you will get an error. Use `CASCADE` to drop the view and all dependent views.

```sql
DROP VIEW employee_directory CASCADE;
```

To avoid errors when the view might not exist:

```sql
DROP VIEW IF EXISTS employee_directory;
```

Materialized Views: Cached Results

A regular view runs its query every time you use it. For complex queries that join many tables or process millions of rows, this can be slow. A materialized view solves this by storing the query results physically on disk, like a snapshot.

Think of the difference like this:

- Regular view = A window you look through. You see the world as it is right now.
- Materialized view = A photograph taken through the window. It shows how the world looked when the photo was taken. To see the current state, you need to take a new photo (refresh).

Creating a Materialized View

```sql
CREATE MATERIALIZED VIEW sales_summary AS
SELECT
    e.first_name || ' ' || e.last_name AS salesperson,
    COUNT(s.sale_id) AS total_deals,
    SUM(s.amount) AS total_revenue,
    ROUND(AVG(s.amount), 2) AS avg_deal_size,
    MAX(s.amount) AS largest_deal,
    MIN(s.sale_date) AS first_sale,
    MAX(s.sale_date) AS last_sale
```

```
FROM employees e
JOIN sales s ON e.employee_id = s.employee_id
GROUP BY e.first_name, e.last_name;
```

```sql
SELECT * FROM sales_summary;
```

Result:

```
  salesperson   | total_deals | total_revenue | avg_deal_size | largest_deal | first_sa...
----------------+-------------+---------------+---------------+--------------+---------...
 Edward Wilson  |           4 |         63500 |      15875.00 |        22000 | 2024-01-...
 Fiona Clark    |           4 |         20000 |       5000.00 |         7500 | 2024-01-...
(2 rows)
```

Refreshing a Materialized View

When the underlying data changes (new sales are added, for example), the materialized view still shows the old data. You need to explicitly refresh it.

```sql
-- Add a new sale
INSERT INTO sales (employee_id, sale_date, amount, product_name)
VALUES (5, '2024-04-01', 30000, 'Enterprise License');

-- The materialized view still shows old data
SELECT total_deals, total_revenue FROM sales_summary
WHERE salesperson = 'Edward Wilson';
```

```
 total_deals | total_revenue
-------------+--------------
           4 |         63500
(1 row)
```

```sql
-- Refresh the materialized view
REFRESH MATERIALIZED VIEW sales_summary;

-- Now it shows updated data
SELECT total_deals, total_revenue FROM sales_summary
WHERE salesperson = 'Edward Wilson';
```

```
 total_deals | total_revenue
-------------+--------------
```

```
           5 |         93500
(1 row)
```

CONCURRENTLY Refresh

The basic `REFRESH` command locks the materialized view while refreshing. No one can query it during the refresh. For large views, use `CONCURRENTLY` to allow queries during the refresh.

```sql
-- First, create a unique index (required for CONCURRENTLY)
CREATE UNIQUE INDEX idx_sales_summary_person ON sales_summary(salesperson);

-- Now you can refresh without blocking readers
REFRESH MATERIALIZED VIEW CONCURRENTLY sales_summary;
```

Regular View vs. Materialized View

```
+--------------------+---------------------------+---------------------------+
| Feature            | Regular View              | Materialized View         |
+--------------------+---------------------------+---------------------------+
| Stores data?       | No (query only)           | Yes (cached on disk)      |
| Always up to date? | Yes                       | No (needs REFRESH)        |
| Query speed         | Same as running the query | Fast (reads cached data)  |
| Uses disk space?   | No                        | Yes                       |
| Can have indexes?  | No                        | Yes                       |
| Good for           | Simple queries, security  | Complex/slow queries      |
+--------------------+---------------------------+---------------------------+
```

When to Use Views

1. Security: Restricting Access to Data

Views let you control what data users can see without modifying the underlying tables.

```sql
-- HR can see everything
CREATE VIEW hr_employee_view AS
SELECT * FROM employees;
```

```
-- Managers see names and departments, but not salaries
CREATE VIEW manager_employee_view AS
SELECT employee_id, first_name, last_name, department_id, hire_date
FROM employees
WHERE is_active = TRUE;

-- Public directory shows only names and emails
CREATE VIEW public_directory AS
SELECT first_name, last_name, email
FROM employees
WHERE is_active = TRUE;
```

```
+-----------------+
|  employees table |
| (all columns,    |
|  all rows)       |
+-----------------+
   |       |       |
   v       v       v
+------+ +------+ +------+
| HR   | | Mgr  | | Pub  |
| View | | View | | View |
| All  | | No $ | | Name |
| cols | | only | | only |
+------+ +------+ +------+
```

2. Simplicity: Hiding Complex Queries

Save a complex query once, then use it with a simple SELECT.

```sql
CREATE VIEW monthly_sales_report AS
SELECT
    TO_CHAR(s.sale_date, 'YYYY-MM') AS month,
    d.department_name,
    e.first_name || ' ' || e.last_name AS salesperson,
    COUNT(s.sale_id) AS deals,
    SUM(s.amount) AS revenue,
    ROUND(AVG(s.amount), 2) AS avg_deal
FROM sales s
JOIN employees e ON s.employee_id = e.employee_id
JOIN departments d ON e.department_id = d.department_id
GROUP BY
    TO_CHAR(s.sale_date, 'YYYY-MM'),
    d.department_name,
    e.first_name, e.last_name
ORDER BY month, revenue DESC;
```

Now anyone can get the monthly sales report with:

```sql
SELECT * FROM monthly_sales_report;
```

Result:

```
  month   | department_name |  salesperson   | deals | revenue |  avg_deal
---------+-----------------+----------------+-------+---------+----------
 2024-01 | Sales           | Edward Wilson |     1 |   15000 | 15000.00
 2024-01 | Sales           | Fiona Clark   |     2 |    9500 |  4750.00
 2024-02 | Sales           | Edward Wilson |     1 |    8500 |  8500.00
 2024-02 | Sales           | Fiona Clark   |     1 |    7500 |  7500.00
 2024-03 | Sales           | Edward Wilson |     2 |   40000 | 20000.00
 2024-03 | Sales           | Fiona Clark   |     1 |    3000 |  3000.00
(6 rows)
```

3. Abstraction: Insulating from Schema Changes

If you rename a column or restructure a table, you can update the view definition without changing the application code that queries the view.

```
Before:                           After:
  App --> employees table           App --> employee_view --> employees table
                                           (view absorbs     (column renamed)
                                            the change)
```

Practical Example: Sales Summary View

Let us build a comprehensive sales dashboard view.

```sql
CREATE VIEW sales_dashboard AS
SELECT
    e.first_name || ' ' || e.last_name AS salesperson,
    d.department_name,
    COUNT(s.sale_id) AS total_deals,
    SUM(s.amount) AS total_revenue,
    ROUND(AVG(s.amount), 2) AS avg_deal_size,
    MAX(s.amount) AS best_deal,
    MIN(s.sale_date) AS first_sale_date,
    MAX(s.sale_date) AS most_recent_sale,
    CASE
        WHEN SUM(s.amount) >= 50000 THEN 'Gold'
```

```
        WHEN SUM(s.amount) >= 20000 THEN 'Silver'
        ELSE 'Bronze'
    END AS performance_tier
FROM employees e
JOIN departments d ON e.department_id = d.department_id
JOIN sales s ON e.employee_id = s.employee_id
GROUP BY e.first_name, e.last_name, d.department_name;
```

```sql
SELECT * FROM sales_dashboard;
```

Result:

```
  salesperson   | department_name | total_deals | total_revenue | avg_deal_size | best_...
----------------+-----------------+-------------+---------------+---------------+------...
 Edward Wilson  | Sales           |           4 |         63500 |      15875.00 |     2...
 Fiona Clark    | Sales           |           4 |         20000 |       5000.00 |      ...
(2 rows)
```

Now simple queries give you rich insights:

```sql
-- Who are the gold performers?
SELECT salesperson, total_revenue
FROM sales_dashboard
WHERE performance_tier = 'Gold';

-- What is the average deal size per tier?
SELECT performance_tier, ROUND(AVG(avg_deal_size), 2) AS avg_across_reps
FROM sales_dashboard
GROUP BY performance_tier;
```

Listing and Inspecting Views

To see all views in your schema:

```sql
SELECT table_name, table_type
FROM information_schema.tables
WHERE table_schema = 'public'
  AND table_type = 'VIEW';
```

To see the definition of a specific view:

```sql
SELECT pg_get_viewdef('employee_directory', TRUE);
```

Or using the psql shortcut:

```
\dv          -- List all views
\d+ view_name  -- Show view details
```

Common Mistakes

Mistake 1: Thinking Views Store Data

```
-- WRONG mental model:
-- "I created a view, so the data is now copied."

-- CORRECT mental model:
-- "I saved a query. It runs fresh every time I use it."
```

Regular views store only the query definition, not the data. Only materialized views store data.

Mistake 2: Not Refreshing Materialized Views

```sql
-- You add new data to the tables
INSERT INTO sales (employee_id, sale_date, amount, product_name)
VALUES (5, '2024-04-15', 25000, 'Enterprise License');

-- But forget to refresh
-- The materialized view still shows stale data!

-- Fix: Always refresh after data changes
REFRESH MATERIALIZED VIEW sales_summary;
```

Mistake 3: Creating Views for One-Time Queries

Not every query needs to be a view. If you only use a query once, just run it directly. Views are for queries you reuse repeatedly or need to share.

Mistake 4: Trying to CREATE OR REPLACE with Removed Columns

```sql
-- Original view has: first_name, last_name, email
CREATE OR REPLACE VIEW my_view AS
SELECT first_name, last_name  -- Removed email!
FROM employees;

-- ERROR: cannot drop columns from view
```

Use DROP VIEW and then CREATE VIEW instead.

Mistake 5: Forgetting CASCADE When Dropping Views with Dependencies

```sql
-- view_b depends on view_a
DROP VIEW view_a;
-- ERROR: cannot drop view view_a because other objects depend on it

-- Fix:
DROP VIEW view_a CASCADE;
-- This also drops view_b
```

Best Practices

1. Name views descriptively. Use names like `active_employee_directory` or `monthly_sales_summary` instead of `v1` or `temp_view`.
2. Use views for security. Grant users access to views instead of tables. This lets you control exactly which columns and rows they can see.
3. Use materialized views for expensive queries. If a query takes seconds or minutes and the data does not change every second, a materialized view can make it instant.
4. Schedule materialized view refreshes. Set up a cron job or scheduled task to refresh your materialized views at appropriate intervals (hourly, daily, etc.).
5. Do not nest views too deeply. A view that queries a view that queries another view becomes hard to debug and may have performance implications. Keep nesting to two or three levels at most.

6. Document your views. Use `COMMENT ON VIEW` to add descriptions that help other developers understand the purpose of each view.

```sql
COMMENT ON VIEW employee_directory IS
    'Public employee directory showing active employees with department info. Excludes ...
```

Quick Summary

```
+---------------------------+-----------------------------------------+
| Feature                   | Description                             |
+---------------------------+-----------------------------------------+
| CREATE VIEW               | Save a query as a virtual table         |
+---------------------------+-----------------------------------------+
| SELECT FROM view          | Query a view like a regular table       |
+---------------------------+-----------------------------------------+
| CREATE OR REPLACE VIEW    | Modify a view (can add columns only)    |
+---------------------------+-----------------------------------------+
| DROP VIEW                 | Remove a view                           |
+---------------------------+-----------------------------------------+
| WITH CHECK OPTION         | Prevent inserts/updates that violate    |
|                           | the view's WHERE clause                 |
+---------------------------+-----------------------------------------+
| CREATE MATERIALIZED VIEW  | Store query results on disk for speed   |
+---------------------------+-----------------------------------------+
| REFRESH MATERIALIZED VIEW | Update cached results in a mat. view    |
+---------------------------+-----------------------------------------+
```

Key Points

- A view is a saved query, not a stored copy of data. It runs fresh every time you query it.
- Views act like virtual tables. You can SELECT from them, filter them, and join them with other tables.
- Use views for security (hide sensitive columns), simplicity (save complex queries), and abstraction (insulate from schema changes).
- CREATE OR REPLACE VIEW lets you modify a view but only by adding columns. You cannot remove or reorder existing columns.
- WITH CHECK OPTION prevents data modifications through a view that would make the row invisible in the view.

- Materialized views physically store query results for faster access. They require explicit REFRESH to update.
- Use REFRESH MATERIALIZED VIEW CONCURRENTLY (with a unique index) to avoid locking during refresh.
- Do not create views for queries you only use once. Views are for reusable, shared queries.

Practice Questions

1. What is the difference between a regular view and a materialized view? When would you choose one over the other?
2. You create a view with `WHERE status = 'active'`. A user inserts a row through the view with `status = 'inactive'`. What happens without WITH CHECK OPTION? What happens with it?
3. Can you create an index on a regular view? What about a materialized view? Why?
4. You have a view that three other views depend on. What happens when you run `DROP VIEW my_view`? How do you force the drop?
5. A materialized view was refreshed yesterday. Since then, 1,000 new rows were added to the underlying table. When you query the materialized view, will you see the new rows? Why or why not?

Exercises

Exercise 1: Employee Directory View

Create a view called `department_roster` that shows each employee's full name (first and last combined), email, department name, and the number of years they have been with the company (calculated from hire_date). Only include active employees. Then write a query that uses this view to find all employees in Engineering who have been with the company for more than 3 years.

Exercise 2: Materialized Sales Summary

Create a materialized view called `quarterly_sales` that shows the year, quarter number, total number of sales, total revenue, and average deal size for each quarter. Then add a new sale

to the sales table and demonstrate the difference between querying before and after a REFRESH.

Exercise 3: Security Views

Create three views for different access levels:

1. `public_staff_list` - Shows only first names and department names of active employees.
2. `manager_view` - Shows full names, departments, hire dates, and years of service, but not salaries.
3. `hr_full_view` - Shows everything including salaries, with a CASE expression that labels salary bands as "Executive" (>= 100,000), "Senior" (>= 75,000), "Mid" (>= 55,000), and "Entry."

Query each view to verify they show the correct level of detail.

What Is Next?

Now that you know how to create views to simplify and secure your queries, the next chapter will introduce Indexes. Indexes are the key to making your queries fast. Just like a book index helps you find a topic without reading every page, a database index helps PostgreSQL find rows without scanning every record. You will learn when to create indexes, which types to use, and how to read query plans to understand performance.

Chapter 21: Indexes

What You Will Learn

In this chapter, you will learn how indexes work and why they make your queries faster. You will discover how to create, manage, and choose the right type of index for different situations. You will also learn how to use EXPLAIN and EXPLAIN ANALYZE to read query plans and verify that your indexes are actually being used.

Why This Chapter Matters

Imagine you have a thick textbook with 800 pages. You need to find every mention of "photosynthesis." You have two choices:

1. No index: Start at page 1 and read every single page, looking for the word "photosynthesis." This is slow and tedious.
2. With an index: Flip to the back of the book, find "photosynthesis" in the alphabetical index, and see "pages 45, 112, 234, 567." Jump directly to those pages.

Databases work the same way. Without an index, PostgreSQL must scan every row in the table (called a sequential scan or full table scan). With the right index, PostgreSQL can jump directly to the matching rows.

For small tables with a few hundred rows, this barely matters. But for tables with millions of rows, the difference between a query taking 5 seconds and 5 milliseconds is enormous. Indexes are one of the most important tools for database performance.

Setting Up Our Practice Data

Let us create a reasonably sized dataset to see indexes in action.

```sql
CREATE TABLE customers (
    customer_id  SERIAL PRIMARY KEY,
    first_name   VARCHAR(50) NOT NULL,
    last_name    VARCHAR(50) NOT NULL,
    email        VARCHAR(100) UNIQUE,
    city         VARCHAR(50),
    state        VARCHAR(2),
    created_at   TIMESTAMP DEFAULT NOW(),
    is_active    BOOLEAN DEFAULT TRUE
);

CREATE TABLE orders (
    order_id     SERIAL PRIMARY KEY,
    customer_id  INTEGER REFERENCES customers(customer_id),
    order_date   DATE NOT NULL,
    total_amount NUMERIC(10,2),
    status       VARCHAR(20) DEFAULT 'pending'
);

-- Insert sample data
INSERT INTO customers (first_name, last_name, email, city, state)
SELECT
    'First' || n,
    'Last' || n,
    'user' || n || '@example.com',
    CASE (n % 5)
        WHEN 0 THEN 'New York'
        WHEN 1 THEN 'Los Angeles'
        WHEN 2 THEN 'Chicago'
        WHEN 3 THEN 'Houston'
        WHEN 4 THEN 'Phoenix'
    END,
    CASE (n % 5)
        WHEN 0 THEN 'NY'
        WHEN 1 THEN 'CA'
        WHEN 2 THEN 'IL'
        WHEN 3 THEN 'TX'
        WHEN 4 THEN 'AZ'
    END
FROM generate_series(1, 100000) AS n;

INSERT INTO orders (customer_id, order_date, total_amount, status)
SELECT
    (random() * 99999 + 1)::INTEGER,
    '2023-01-01'::DATE + (random() * 730)::INTEGER,
    (random() * 500 + 10)::NUMERIC(10,2),
    CASE (random() * 3)::INTEGER
        WHEN 0 THEN 'pending'
        WHEN 1 THEN 'shipped'
        WHEN 2 THEN 'delivered'
        ELSE 'cancelled'
    END
FROM generate_series(1, 500000);
```

How Indexes Speed Up Queries

Without an index, PostgreSQL uses a Sequential Scan (Seq Scan). It reads every row in the table and checks whether it matches your condition. For a table with 100,000 rows, it must check all 100,000 rows.

With an index, PostgreSQL uses an Index Scan. It looks up the matching values in the index data structure (usually a B-tree) and then jumps directly to the relevant rows. For a query that matches 10 rows out of 100,000, it only needs to read about 10 rows instead of 100,000.

```
Without Index (Sequential Scan):
+------+------+------+------+------+------+------+------+
| Row1 | Row2 | Row3 | Row4 | Row5 | Row6 | Row7 | ...  |
+------+------+------+------+------+------+------+------+
  Check  Check  Check  MATCH  Check  Check  MATCH  ...

  Must check EVERY row. Slow for large tables.


With Index (Index Scan):
+------------------+
|   B-tree Index   |
|  (sorted values) |
+------------------+
       |
       v
  Jump directly to Row4 and Row7

  Checks only matching rows. Fast!
```

B-tree Index: The Default

The B-tree (Balanced tree) is the default index type in PostgreSQL. It works like a well-organized filing system.

Think of a library's card catalog. Cards are organized alphabetically. To find a book by "Smith," you do not start at "A." You jump to the "S" section, then narrow down to "Sm," then "Smi," and quickly find "Smith."

A B-tree works similarly. It organizes values in a tree structure where each level narrows the search.

```
B-tree for last_name index:

                  [Johnson]
                 /          \
          [Brown]            [Smith]
         /      \           /      \
    [Adams] [Clark]   [Miller] [Wilson]
      /\      /\        /\       /\
   rows..  rows..    rows..   rows..

To find "Miller":
  1. Start at root: "Miller" > "Johnson" -> go right
  2. Next level: "Miller" < "Smith" -> go left
  3. Found "Miller" -> follow pointer to actual rows

Only 3 comparisons instead of scanning all rows!
```

B-tree indexes support these operations efficiently:

- Equality: `WHERE last_name = 'Smith'`
- Range: `WHERE salary > 50000`
- Sorting: `ORDER BY last_name`
- BETWEEN: `WHERE created_at BETWEEN '2024-01-01' AND '2024-06-30'`
- IS NULL / IS NOT NULL

Creating Indexes

Syntax

```sql
CREATE INDEX index_name ON table_name (column_name);
```

Example: Index on city

```sql
CREATE INDEX idx_customers_city ON customers (city);
```

Line-by-Line Explanation:

- `CREATE INDEX` -- Tell PostgreSQL to create a new index.

- `idx_customers_city` -- The name of the index. A common convention is `idx_tablename_columnname`.
- `ON customers` -- The table this index belongs to.
- `(city)` -- The column to index.

Naming Convention

A good naming convention makes it easy to understand what an index does at a glance.

```
idx_<table>_<column(s)>

Examples:
  idx_customers_email
  idx_orders_customer_id
  idx_orders_date_status     (composite index)
```

EXPLAIN and EXPLAIN ANALYZE

How do you know if PostgreSQL is actually using your index? Use EXPLAIN and EXPLAIN ANALYZE to see the query plan.

EXPLAIN: See the Plan Without Running It

```sql
EXPLAIN SELECT * FROM customers WHERE city = 'Chicago';
```

Result (before creating index):

```
                          QUERY PLAN
-----------------------------------------------------------
 Seq Scan on customers  (cost=0.00..2137.00 rows=20000 width=67)
   Filter: ((city)::text = 'Chicago'::text)
(2 rows)
```

Reading the output:

- `Seq Scan on customers` -- PostgreSQL is doing a sequential scan (reading every row). No index is being used.

- `cost=0.00..2137.00` -- Estimated cost (in arbitrary units). The first number is startup cost, the second is total cost.
- `rows=20000` -- PostgreSQL estimates it will find about 20,000 matching rows.
- `Filter: ((city)::text = 'Chicago'::text)` -- The filter condition being applied to each row.

EXPLAIN ANALYZE: Run It and Show Actual Times

```sql
EXPLAIN ANALYZE SELECT * FROM customers WHERE city = 'Chicago';
```

Result (before index):

```
                          QUERY PLAN
-------------------------------------------------------------
 Seq Scan on customers  (cost=0.00..2137.00 rows=20000 width=67)
                        (actual time=0.015..12.345 rows=20000 loops=1)
   Filter: ((city)::text = 'Chicago'::text)
   Rows Removed by Filter: 80000
 Planning Time: 0.085 ms
 Execution Time: 13.456 ms
(5 rows)
```

Now let us create the index and see the difference:

```sql
CREATE INDEX idx_customers_city ON customers (city);

EXPLAIN ANALYZE SELECT * FROM customers WHERE city = 'Chicago';
```

Result (after index):

```
                          QUERY PLAN
-------------------------------------------------------------
 Bitmap Heap Scan on customers  (cost=236.00..1287.00 rows=20000 width=67)
                                (actual time=1.234..5.678 rows=20000 loops=1)
   Recheck Cond: ((city)::text = 'Chicago'::text)
   ->  Bitmap Index Scan on idx_customers_city  (cost=0.00..231.00 rows=20000 width=0)
                                                (actual time=0.987..0.987 rows=20000 lo...
         Index Cond: ((city)::text = 'Chicago'::text)
 Planning Time: 0.120 ms
 Execution Time: 6.789 ms
(6 rows)
```

Reading the output:

- `Bitmap Index Scan on idx_customers_city` -- PostgreSQL is using our index.
- The execution time dropped from ~13ms to ~7ms.

> Note: When a query returns a large portion of the table (like 20% of rows), PostgreSQL may still choose a sequential scan because reading lots of scattered index entries is not much faster than reading the whole table. Indexes shine most when finding a small number of rows in a large table.

A More Dramatic Example

```sql
-- Find a single customer by email (only 1 row out of 100,000)
EXPLAIN ANALYZE SELECT * FROM customers WHERE email = 'user50000@example.com';
```

```
                          QUERY PLAN
-------------------------------------------------------------
 Index Scan using customers_email_key on customers
   (cost=0.42..8.44 rows=1 width=67)
   (actual time=0.025..0.026 rows=1 loops=1)
   Index Cond: ((email)::text = 'user50000@example.com'::text)
 Planning Time: 0.095 ms
 Execution Time: 0.045 ms
(4 rows)
```

This query uses the unique index that was automatically created by the `UNIQUE` constraint on `email`. It finds one row out of 100,000 in 0.045 milliseconds. That is the power of indexes.

When Indexes Help

Indexes are most effective in these situations:

1. WHERE Clauses

```sql
-- Finding specific rows
CREATE INDEX idx_customers_state ON customers (state);

SELECT * FROM customers WHERE state = 'TX';
```

2. JOIN Conditions

```sql
-- Joining tables on foreign keys
CREATE INDEX idx_orders_customer_id ON orders (customer_id);

SELECT c.first_name, o.order_date, o.total_amount
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
WHERE c.city = 'Houston';
```

The index on `orders.customer_id` helps PostgreSQL quickly find all orders for each customer.

3. ORDER BY

```sql
-- Sorting results
CREATE INDEX idx_orders_order_date ON orders (order_date);

SELECT * FROM orders ORDER BY order_date DESC LIMIT 20;
```

With the index, PostgreSQL can read the rows in order directly from the index instead of sorting them afterward.

4. Aggregate Functions with GROUP BY

```sql
SELECT customer_id, COUNT(*), SUM(total_amount)
FROM orders
GROUP BY customer_id;
```

An index on `customer_id` can speed up grouping operations.

When Indexes Hurt

Indexes are not free. They come with costs.

Cost 1: Slower Writes

Every time you INSERT, UPDATE, or DELETE a row, PostgreSQL must also update every index on that table. More indexes mean slower write operations.

```
INSERT a new row:
  1. Write the row to the table          -- always happens
  2. Update index on customer_id         -- extra work
  3. Update index on email               -- extra work
  4. Update index on city                -- extra work
  5. Update index on state               -- extra work
  6. Update index on created_at          -- extra work

5 indexes = 5 extra operations per INSERT!
```

Cost 2: Disk Space

Each index takes up disk space. For a large table, indexes can use as much space as the table itself.

Cost 3: Maintenance Overhead

Indexes need to be maintained. Over time, as data changes, indexes can become bloated and need to be rebuilt.

Rule of Thumb

```
+-----------------------------------+-----------------------------------+
| DO create an index when:          | DO NOT create an index when:      |
+-----------------------------------+-----------------------------------+
| Column is used often in WHERE     | Column is rarely used in queries  |
| Column is used in JOIN            | Table is very small (< 1000 rows)|
| Column is used in ORDER BY       | Column has very few distinct      |
| High selectivity (few matches)    |   values (e.g., boolean)          |
| Table has many rows               | Table has heavy write workload    |
+-----------------------------------+-----------------------------------+
```

Composite Indexes

A composite (multi-column) index covers two or more columns. It is useful when you frequently query using multiple columns together.

Creating a Composite Index

```sql
CREATE INDEX idx_orders_date_status ON orders (order_date, status);
```

This index is efficient for queries that filter on:

- `order_date` alone (leftmost column)
- `order_date AND status` (both columns)

But it is not efficient for queries that filter on:

- `status` alone (not the leftmost column)

This is called the leftmost prefix rule. Think of it like a phone book sorted by last name, then first name. You can look up "Smith" (last name only) or "Smith, John" (both). But you cannot efficiently look up just "John" (first name only) because the book is not sorted that way.

```
Composite Index: (order_date, status)

+----+------------+-----------+
|    | order_date | status    |  <-- Sorted by order_date first,
+----+------------+-----------+      then by status within each date
|  1 | 2024-01-01 | delivered |
|  2 | 2024-01-01 | pending   |
|  3 | 2024-01-01 | shipped   |
|  4 | 2024-01-02 | cancelled |
|  5 | 2024-01-02 | delivered |
|  6 | 2024-01-02 | pending   |
+----+------------+-----------+

Can use index:                    Cannot use index efficiently:
  WHERE order_date = '2024-01-01'   WHERE status = 'pending'
  WHERE order_date = '2024-01-01'   (status is not the leftmost
    AND status = 'shipped'            column in the index)
```

Example

```sql
EXPLAIN ANALYZE
SELECT * FROM orders
WHERE order_date = '2024-06-15' AND status = 'shipped';
```

```
                          QUERY PLAN
-------------------------------------------------------------
 Index Scan using idx_orders_date_status on orders
   (cost=0.42..12.50 rows=5 width=39)
   (actual time=0.030..0.042 rows=3 loops=1)
   Index Cond: ((order_date = '2024-06-15') AND (status = 'shipped'))
 Planning Time: 0.150 ms
```

```
 Execution Time: 0.065 ms
(4 rows)
```

The composite index handles both conditions efficiently.

Partial Indexes

A partial index only indexes rows that meet a certain condition. It is smaller and more efficient than a full index when you only query a subset of the data.

Syntax

```sql
CREATE INDEX index_name ON table_name (column_name)
WHERE condition;
```

Example: Index Only Active Customers

If 95% of your queries filter on `is_active = TRUE`, there is no point indexing the inactive rows.

```sql
CREATE INDEX idx_customers_city_active
ON customers (city)
WHERE is_active = TRUE;
```

This index is smaller because it excludes inactive customers. Queries that filter on active customers will be faster, and the index takes up less space.

```sql
-- This query uses the partial index
EXPLAIN ANALYZE
SELECT * FROM customers
WHERE city = 'Phoenix' AND is_active = TRUE;

-- This query CANNOT use the partial index (missing is_active filter)
EXPLAIN ANALYZE
SELECT * FROM customers
WHERE city = 'Phoenix';
```

When to Use Partial Indexes

- When you frequently filter on a specific condition (e.g., `status = 'active'`, `is_deleted = FALSE`)
- When only a small portion of the table matches your usual queries
- When you want to reduce index size and maintenance cost

Unique Indexes

A unique index ensures that no two rows have the same value in the indexed column(s). PostgreSQL automatically creates a unique index when you define a `UNIQUE` constraint or `PRIMARY KEY`.

```sql
-- These two are equivalent:
ALTER TABLE customers ADD CONSTRAINT unique_email UNIQUE (email);
CREATE UNIQUE INDEX idx_customers_email_unique ON customers (email);
```

Unique indexes serve double duty: they enforce data integrity AND speed up lookups.

Index Types in PostgreSQL

PostgreSQL offers several index types, each optimized for different use cases.

B-tree (Default)

```sql
CREATE INDEX idx_name ON table_name (column);
-- Equivalent to:
CREATE INDEX idx_name ON table_name USING btree (column);
```

Best for: Equality (`=`), range (`<`, `>`, `BETWEEN`), sorting (`ORDER BY`), pattern matching (`LIKE 'abc%'`).

Hash

```sql
CREATE INDEX idx_name ON table_name USING hash (column);
```

Best for: Equality comparisons only (=). Slightly faster than B-tree for pure equality lookups, but cannot handle ranges or sorting.

GIN (Generalized Inverted Index)

```sql
CREATE INDEX idx_name ON table_name USING gin (column);
```

Best for: Full-text search, array columns, JSONB data. A GIN index maps each value (like each word in a text) to the rows that contain it.

GiST (Generalized Search Tree)

```sql
CREATE INDEX idx_name ON table_name USING gist (column);
```

Best for: Geometric data, range types, full-text search, nearest-neighbor queries. Often used with PostGIS for geographic data.

Summary

```
+--------+----------------------------------+----------------------------+
| Type   | Best For                         | Example Use Case           |
+--------+----------------------------------+----------------------------+
| B-tree | Equality, ranges, sorting        | WHERE price > 100          |
|        |                                  | ORDER BY date              |
+--------+----------------------------------+----------------------------+
| Hash   | Equality only                    | WHERE status = 'active'    |
+--------+----------------------------------+----------------------------+
| GIN    | Arrays, JSONB, full-text search   | WHERE tags @> '{sql}'      |
|        |                                  | Full-text search           |
+--------+----------------------------------+----------------------------+
| GiST   | Geometry, ranges, proximity       | Nearest restaurant query   |
|        |                                  | Overlapping date ranges    |
+--------+----------------------------------+----------------------------+
```

For most everyday work, B-tree is the right choice. You will use GIN and GiST for specialized scenarios.

DROP INDEX

To remove an index:

```sql
DROP INDEX idx_customers_city;
```

To avoid errors if the index does not exist:

```sql
DROP INDEX IF EXISTS idx_customers_city;
```

To drop an index even when used by views or other objects:

```sql
DROP INDEX idx_customers_city CASCADE;
```

> Note: Dropping an index does not delete any data. It only removes the index structure. Queries will still work, they just might be slower.

Reading Query Plans in Detail

Let us break down a more complex EXPLAIN ANALYZE output to understand what each part means.

```sql
EXPLAIN ANALYZE
SELECT c.first_name, c.last_name, COUNT(o.order_id) AS order_count
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
WHERE c.state = 'TX'
GROUP BY c.first_name, c.last_name
ORDER BY order_count DESC
LIMIT 10;
```

```
                          QUERY PLAN
-------------------------------------------------------------
 Limit  (cost=5000.00..5000.02 rows=10 width=23)
        (actual time=45.123..45.130 rows=10 loops=1)
   ->  Sort  (cost=5000.00..5050.00 rows=20000 width=23)
             (actual time=45.120..45.125 rows=10 loops=1)
         Sort Key: (count(o.order_id)) DESC
         Sort Method: top-N heapsort  Memory: 25kB
         ->  HashAggregate  (cost=4500.00..4700.00 rows=20000 width=23)
                            (actual time=40.000..43.000 rows=20000 loops=1)
```

```
                Group Key: c.first_name, c.last_name
                ->  Hash Join  (cost=700.00..3500.00 rows=100000 width=19)
                               (actual time=5.000..25.000 rows=100234 loops=1)
                      Hash Cond: (o.customer_id = c.customer_id)
                      ->  Seq Scan on orders o  (cost=0.00..2000.00 rows=500000 width=8)
                                                (actual time=0.010..8.000 rows=500000 lo...
                      ->  Hash  (cost=600.00..600.00 rows=20000 width=19)
                                (actual time=4.500..4.500 rows=20000 loops=1)
                            ->  Seq Scan on customers c  (cost=0.00..600.00 rows=20000 w...
                                                         (actual time=0.010..3.000 rows=...
                                  Filter: ((state)::text = 'TX'::text)
                                  Rows Removed by Filter: 80000
 Planning Time: 0.500 ms
 Execution Time: 45.250 ms
(15 rows)
```

Reading from bottom to top (that is how execution flows):

1. Seq Scan on customers -- Scans all 100,000 customers, filters to 20,000 where state = 'TX'. Removes 80,000 rows.
2. Hash -- Builds a hash table of those 20,000 Texas customers in memory.
3. Seq Scan on orders -- Scans all 500,000 orders.
4. Hash Join -- Joins orders to the hash table of Texas customers. Finds ~100,234 matching order-customer pairs.
5. HashAggregate -- Groups by first_name and last_name, counts orders per group. Produces 20,000 groups.
6. Sort -- Sorts by order count descending.
7. Limit -- Returns only the top 10.

Key numbers to watch:

- `actual time` -- Real wall-clock time in milliseconds. Look at the second number (total time).
- `rows` -- How many rows that step produced. Compare estimated (`rows=20000` in cost section) vs actual (`rows=20000` in actual section). Large mismatches suggest stale statistics.
- `Rows Removed by Filter` -- How many rows were read but did not match. High numbers may indicate a missing index.

Practical Example: Adding Indexes to Slow Queries

Let us walk through a real-world scenario of finding and fixing a slow query.

Step 1: Identify the Slow Query

```sql
-- Find all pending orders from Texas customers in the last month
EXPLAIN ANALYZE
SELECT
    c.first_name,
    c.last_name,
    c.email,
    o.order_date,
    o.total_amount
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
WHERE c.state = 'TX'
  AND o.status = 'pending'
  AND o.order_date >= '2024-06-01'
ORDER BY o.order_date DESC;
```

Step 2: Read the Query Plan

Look for:

- Seq Scan on large tables (indicates no index is being used)
- Rows Removed by Filter (high numbers mean we are reading many unnecessary rows)
- High execution time

Step 3: Add Targeted Indexes

```sql
-- Index for filtering customers by state
CREATE INDEX idx_customers_state ON customers (state);

-- Composite index for filtering orders by status and date
CREATE INDEX idx_orders_status_date ON orders (status, order_date);

-- Index for the join condition (if not already indexed)
CREATE INDEX idx_orders_customer_id ON orders (customer_id);
```

Step 4: Verify Improvement

```sql
EXPLAIN ANALYZE
SELECT
    c.first_name,
    c.last_name,
```

```
    c.email,
    o.order_date,
    o.total_amount
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
WHERE c.state = 'TX'
  AND o.status = 'pending'
  AND o.order_date >= '2024-06-01'
ORDER BY o.order_date DESC;
```

After adding indexes, you should see:

- Index Scans replacing Seq Scans
- Lower execution time
- Fewer rows removed by filter

Step 5: Clean Up Unused Indexes

You can check which indexes are not being used:

```sql
SELECT
    schemaname,
    tablename,
    indexname,
    idx_scan AS times_used
FROM pg_stat_user_indexes
WHERE idx_scan = 0
ORDER BY tablename, indexname;
```

If an index has `idx_scan = 0` after weeks of production use, consider dropping it to save space and improve write performance.

Common Mistakes

Mistake 1: Indexing Every Column

```sql
-- WRONG: Creating an index on every column
CREATE INDEX idx1 ON orders (order_id);       -- Already has PK index
CREATE INDEX idx2 ON orders (customer_id);
CREATE INDEX idx3 ON orders (order_date);
```

```
CREATE INDEX idx4 ON orders (total_amount);
CREATE INDEX idx5 ON orders (status);

-- This slows down every INSERT, UPDATE, and DELETE!
```

Only create indexes on columns that are frequently used in WHERE, JOIN, or ORDER BY clauses.

Mistake 2: Wrong Column Order in Composite Indexes

```sql
-- You query: WHERE status = 'pending' AND order_date > '2024-01-01'
-- But most of your rows are filtered by order_date, not status

-- LESS EFFECTIVE: status first (low selectivity)
CREATE INDEX idx_bad ON orders (status, order_date);

-- MORE EFFECTIVE: order_date first (high selectivity)
CREATE INDEX idx_good ON orders (order_date, status);
```

Put the most selective column (the one that eliminates the most rows) first in a composite index.

Mistake 3: Not Checking if Indexes Are Used

Always verify with EXPLAIN ANALYZE that your index is actually being used. PostgreSQL might choose not to use it if:

- The table is small enough that a sequential scan is faster
- The query returns too many rows (low selectivity)
- Statistics are outdated (run `ANALYZE table_name` to update)

Mistake 4: Indexing Low-Selectivity Columns

```sql
-- POOR: Boolean columns have only 2 values
CREATE INDEX idx_customers_active ON customers (is_active);
-- PostgreSQL will likely ignore this index

-- BETTER: Use a partial index
CREATE INDEX idx_customers_inactive ON customers (customer_id)
WHERE is_active = FALSE;
```

Mistake 5: Forgetting to ANALYZE After Bulk Loads

```sql
-- After inserting many rows, update statistics
ANALYZE customers;
ANALYZE orders;
```

PostgreSQL uses statistics to decide whether to use an index. Outdated statistics can lead to poor query plans.

Best Practices

1. Start without indexes, then add as needed. Create indexes to solve specific performance problems, not preemptively.
2. Always verify with EXPLAIN ANALYZE. Do not assume an index is helping. Check the query plan.
3. Index foreign key columns. Columns used in JOINs (especially foreign keys) are almost always worth indexing.
4. Use composite indexes for multi-column filters. If you always filter on `status AND date`, a composite index on `(status, date)` is better than two separate indexes.
5. Use partial indexes for common filters. If 95% of queries filter `WHERE is_active = TRUE`, a partial index saves space and improves performance.
6. Monitor index usage. Regularly check `pg_stat_user_indexes` to find and remove unused indexes.
7. Run ANALYZE after bulk operations. After large inserts, updates, or deletes, update table statistics so PostgreSQL can make good query planning decisions.
8. Consider CONCURRENTLY for production. Use `CREATE INDEX CONCURRENTLY` to avoid locking the table during index creation:

```sql
CREATE INDEX CONCURRENTLY idx_orders_date ON orders (order_date);
```

This takes longer but allows reads and writes to continue during index creation.

Quick Summary

```
+---------------------------+-------------------------------------------+
| Feature                   | Description                               |
+---------------------------+-------------------------------------------+
| CREATE INDEX              | Create an index on a column               |
+---------------------------+-------------------------------------------+
| CREATE UNIQUE INDEX       | Index that enforces uniqueness            |
+---------------------------+-------------------------------------------+
| Composite Index           | Index on multiple columns                 |
+---------------------------+-------------------------------------------+
| Partial Index             | Index with a WHERE condition              |
+---------------------------+-------------------------------------------+
| DROP INDEX                | Remove an index                           |
+---------------------------+-------------------------------------------+
| EXPLAIN                   | Show the query plan (without running)     |
+---------------------------+-------------------------------------------+
| EXPLAIN ANALYZE           | Show the plan with actual execution data  |
+---------------------------+-------------------------------------------+
| B-tree                    | Default. Equality, range, sorting         |
+---------------------------+-------------------------------------------+
| Hash                      | Equality only                             |
+---------------------------+-------------------------------------------+
| GIN                       | Arrays, JSONB, full-text search           |
+---------------------------+-------------------------------------------+
| GiST                      | Geometry, ranges, proximity               |
+---------------------------+-------------------------------------------+
```

Key Points

- An index is a data structure that speeds up data retrieval, similar to a book's index.
- The default index type is B-tree, which works well for equality, range, and sorting operations.
- Indexes speed up reads (SELECT) but slow down writes (INSERT, UPDATE, DELETE).
- Use EXPLAIN ANALYZE to verify that PostgreSQL is using your indexes and to measure actual performance.
- Composite indexes cover multiple columns. The leftmost prefix rule determines which queries can use them.
- Partial indexes only index rows matching a condition, making them smaller and faster.
- Do not over-index. Every index adds overhead to write operations. Only index columns that are frequently queried.
- Use CREATE INDEX CONCURRENTLY in production to avoid locking tables.
- Monitor index usage with `pg_stat_user_indexes` and remove unused indexes.

Practice Questions

1. Explain the book index analogy for database indexes. Why does a sequential scan become problematic as a table grows?
2. You have a composite index on `(city, state)`. Which of these queries can use the index: (a) `WHERE city = 'Houston'`, (b) `WHERE state = 'TX'`, (c) `WHERE city = 'Houston' AND state = 'TX'`? Explain why.
3. What is the difference between EXPLAIN and EXPLAIN ANALYZE? When would you use one over the other?
4. Why would you choose a partial index over a regular index? Give an example scenario.
5. A table has 10 indexes and experiences heavy INSERT traffic. The INSERT operations are becoming slow. What would you investigate and how might you fix the problem?

Exercises

Exercise 1: Index Investigation

Using the `orders` table, write a query that finds all delivered orders with a total amount greater than 400. First, run it with EXPLAIN ANALYZE to see the plan without any custom indexes. Then create an appropriate index and run EXPLAIN ANALYZE again. Compare the results.

Exercise 2: Composite Index Design

You frequently run this query:

```sql
SELECT * FROM orders
WHERE customer_id = 12345
  AND order_date >= '2024-01-01'
ORDER BY order_date DESC;
```

Design the best composite index for this query. Explain your choice of column order. Verify with EXPLAIN ANALYZE.

Exercise 3: Finding and Removing Unused Indexes

Write a query against `pg_stat_user_indexes` that lists all indexes on your tables along with how many times each has been scanned. Identify any indexes that have never been used and write the DROP statements for them (but do not execute them yet -- review first).

What Is Next?

Now that you understand how indexes make queries fast, the next chapter will explore Normalization. Normalization is the art of organizing your tables to reduce redundancy and prevent data anomalies. You will learn the normal forms (1NF, 2NF, 3NF) and how to transform a messy spreadsheet into a set of clean, well-structured tables.

Chapter 22: Normalization

What You Will Learn

In this chapter, you will learn how to organize your database tables to eliminate redundancy, prevent data anomalies, and maintain data integrity. You will discover the three main normal forms (1NF, 2NF, 3NF) and Boyce-Codd Normal Form (BCNF). You will also learn when it makes sense to break the rules through denormalization.

Why This Chapter Matters

Imagine you keep all your personal information -- contacts, appointments, shopping lists, bank records, medical history -- in one giant notebook. At first it seems convenient. Everything is in one place. But soon you run into problems:

- You wrote your doctor's phone number on three different pages. When the number changes, you update it on one page but forget the other two. Now you have conflicting information.
- You want to delete a canceled appointment, but the appointment entry also contains your dentist's address. Deleting the appointment means losing the address.
- You want to add a new contact, but the notebook requires you to also write an appointment date. You do not have an appointment yet, so what do you write?

These are exactly the problems that happen in poorly designed databases. Normalization is the process of organizing your data to prevent these issues. It is one of the most fundamental concepts in database design, and understanding it will make you a significantly better database developer.

The Messy Spreadsheet: Our Starting Point

Let us start with a real-world scenario. A small bookstore tracks orders in a single spreadsheet.

```
+---------+--------+----------+-----------+---------+--------+----------+--------+-------+
|order_id |cust_name|cust_email|cust_phone |book_title|author |author_email|price  |qty...
+---------+--------+----------+-----------+---------+--------+----------+--------+-------+
|1001     |Alice   |alice@    |555-0101   |SQL Made |Bob Lee |bob@pub   |29.99   |2      |
|         |        |mail.com  |           |Easy     |        |.com      |        |       |
+---------+--------+----------+-----------+---------+--------+----------+--------+-------+
|1001     |Alice   |alice@    |555-0101   |Data     |Carol   |carol@pub |39.99   |1      |
|         |        |mail.com  |           |Design   |King    |.com      |        |       |
+---------+--------+----------+-----------+---------+--------+----------+--------+-------+
|1002     |Dave    |dave@     |555-0202   |SQL Made |Bob Lee |bob@pub   |29.99   |1      |
|         |        |mail.com  |           |Easy     |        |.com      |        |       |
+---------+--------+----------+-----------+---------+--------+----------+--------+-------+
|1003     |Alice   |alice@    |555-0101   |NoSQL    |Bob Lee |bob@pub   |34.99   |3      |
|         |        |mail.com  |           |Basics   |        |.com      |        |       |
+---------+--------+----------+-----------+---------+--------+----------+--------+-------+
```

This spreadsheet has many problems. Let us explore them.

Data Anomalies: Why Bad Design Hurts

Update Anomaly

Alice's email appears in three rows. If she changes her email, you must update all three rows. If you miss even one, her data becomes inconsistent.

```
Before update:
  Row 1: Alice, alice@mail.com   <-- Updated
  Row 2: Alice, alice@mail.com   <-- FORGOT to update!
  Row 3: Alice, alice_new@mail.com  <-- Wait, which is correct?

This is the UPDATE ANOMALY.
```

Insert Anomaly

You want to add a new book to your catalog, but since all book information is stored alongside orders, you cannot add a book until someone orders it. What if you want to record a book that nobody has bought yet?

```
Trying to insert:
  order_id: ???       <-- No order yet!
  cust_name: ???      <-- No customer yet!
  book_title: "New SQL Book"
  author: "Jane Doe"
  price: 24.99

Cannot insert! The table requires order and customer data.
This is the INSERT ANOMALY.
```

Delete Anomaly

Dave only has one order. If you delete that order, you lose all record that Dave was ever a customer. You also lose the fact that "SQL Made Easy" costs 29.99, if Dave's row was the only one with that information.

```
Deleting order 1002:
  Lose Dave's name: Dave        -- Gone!
  Lose Dave's email: dave@mail.com  -- Gone!
  Lose Dave's phone: 555-0202     -- Gone!

This is the DELETE ANOMALY.
```

Summary of Anomalies

```
+------------------+-----------------------------------------------------+
| Anomaly          | Problem                                             |
+------------------+-----------------------------------------------------+
| Update Anomaly   | Must update the same fact in multiple rows.         |
|                  | Miss one and data becomes inconsistent.             |
+------------------+-----------------------------------------------------+
| Insert Anomaly   | Cannot add new data without unrelated data.         |
|                  | e.g., Cannot add a book without an order.           |
+------------------+-----------------------------------------------------+
| Delete Anomaly   | Deleting one fact accidentally removes other        |
|                  | unrelated facts.                                    |
+------------------+-----------------------------------------------------+
```

Normalization eliminates these anomalies by organizing data into separate, focused tables.

First Normal Form (1NF): Atomic Values

A table is in First Normal Form if:

1. Every column contains atomic (indivisible) values. No lists, no sets, no comma-separated values in a single cell.
2. Each row is unique (there is a way to identify each row).
3. There are no repeating groups of columns.

Violation: Non-Atomic Values

```
BAD: Comma-separated values in one column

+---------+-----------+---------------------------+
|order_id | cust_name | books_ordered             |
+---------+-----------+---------------------------+
| 1001    | Alice     | SQL Made Easy, Data Design |
| 1002    | Dave      | SQL Made Easy             |
+---------+-----------+---------------------------+

Problem: How do you query "find all orders containing SQL Made Easy"?
You would need string searching, which is slow and error-prone.
```

Violation: Repeating Groups

```
BAD: Multiple columns for the same kind of data

+---------+-----------+--------+--------+--------+--------+
|order_id | cust_name | book_1 | price_1| book_2 | price_2|
+---------+-----------+--------+--------+--------+--------+
| 1001    | Alice     | SQL    | 29.99  | Data   | 39.99  |
|         |           | Made   |        | Design |        |
| 1002    | Dave      | SQL    | 29.99  | (null) | (null) |
|         |           | Made   |        |        |        |
+---------+-----------+--------+--------+--------+--------+

Problem: What if someone orders 10 books? 50? You cannot keep
adding columns. And queries become nightmarish.
```

Fixed: 1NF Version

```
GOOD: Each cell has one value, each row is unique

+---------+-----------+-----------+-------+-----+
|order_id | cust_name | book_title| price | qty |
+---------+-----------+-----------+-------+-----+
```

```
| 1001    | Alice     | SQL Made  | 29.99 |  2  |
|         |           | Easy      |       |     |
| 1001    | Alice     | Data      | 39.99 |  1  |
|         |           | Design    |       |     |
| 1002    | Dave      | SQL Made  | 29.99 |  1  |
|         |           | Easy      |       |     |
+---------+-----------+-----------+-------+-----+

Each cell contains exactly one value.
No repeating column groups.
```

1NF Rule Summary

```
+-------------------------------+--------------------------------+
| Rule                          | Example of Violation           |
+-------------------------------+--------------------------------+
| Atomic values                 | "SQL, Data Design" in one cell |
| No repeating groups           | book_1, book_2, book_3 columns|
| Rows are uniquely identifiable| No primary key                 |
+-------------------------------+--------------------------------+
```

Second Normal Form (2NF): No Partial Dependencies

A table is in Second Normal Form if:

1. It is already in 1NF.
2. Every non-key column depends on the entire primary key, not just part of it.

2NF only applies to tables with composite primary keys (keys made of two or more columns). If your primary key is a single column, the table automatically satisfies 2NF.

Understanding Dependencies

A dependency means one column's value is determined by another. If you know the `order_id`, you can determine the `cust_name`. We say `cust_name` depends on `order_id`.

Violation: Partial Dependency

Let us look at our bookstore table where the composite key is `(order_id, book_title)`:

```
Composite Key: (order_id, book_title)

+---------+-----------+----------+-----------+-------+-----+
|order_id | book_title|cust_name |cust_email |price  | qty |
+---------+-----------+----------+-----------+-------+-----+
|(PK)     | (PK)      |          |           |       |     |
+---------+-----------+----------+-----------+-------+-----+

Dependencies:
  order_id --> cust_name, cust_email   (depends on PART of key)
  book_title --> price                  (depends on PART of key)
  (order_id, book_title) --> qty        (depends on FULL key)
```

`cust_name` depends only on `order_id`, not on `book_title`. This is a partial dependency. Similarly, `price` depends only on `book_title`, not on `order_id`.

Fixed: 2NF Version

Split the table so that each non-key column depends on the full key of its table.

```
Table: orders
+---------+----------+-----------+
|order_id | cust_name| cust_email|
|(PK)     |          |           |
+---------+----------+-----------+
| 1001    | Alice    | alice@    |
| 1002    | Dave     | dave@     |
+---------+----------+-----------+

Table: books
+-----------+-------+
|book_title | price |
|(PK)       |       |
+-----------+-------+
|SQL Made   | 29.99 |
|Data Design| 39.99 |
+-----------+-------+

Table: order_items
+---------+-----------+-----+
|order_id | book_title| qty |
|(PK)     | (PK)      |     |
+---------+-----------+-----+
| 1001    | SQL Made  |  2  |
| 1001    | Data Design|  1 |
| 1002    | SQL Made  |  1  |
+---------+-----------+-----+
```

Now:

- `cust_name` depends on `order_id` (the full key of the `orders` table).
- `price` depends on `book_title` (the full key of the `books` table).
- `qty` depends on `(order_id, book_title)` (the full key of the `order_items` table).

No partial dependencies remain.

Third Normal Form (3NF): No Transitive Dependencies

A table is in Third Normal Form if:

1. It is already in 2NF.
2. No non-key column depends on another non-key column. Every non-key column must depend directly on the primary key, not through another non-key column.

Understanding Transitive Dependencies

A transitive dependency is when column A determines column B, and column B determines column C. So A determines C, but indirectly, through B.

Violation: Transitive Dependency

Look at our `orders` table from the 2NF step:

```
Table: orders
+---------+----------+-----------+-----------+
|order_id | cust_name| cust_email| cust_phone|
|(PK)     |          |           |           |
+---------+----------+-----------+-----------+

Dependencies:
  order_id --> cust_name      (direct, OK)
  order_id --> cust_email     (but wait...)
  cust_name --> cust_email    (cust_email depends on cust_name,
                               not directly on order_id!)

  order_id --> cust_name --> cust_email
  This is a TRANSITIVE dependency.
```

The problem: `cust_email` is really a fact about the customer, not about the order. If Alice's email changes, we must update every order she has made.

Fixed: 3NF Version

Move customer information to its own table.

```
Table: customers
+-----------+----------+-----------+-----------+
|customer_id| cust_name| cust_email| cust_phone|
|(PK)       |          |           |           |
+-----------+----------+-----------+-----------+
| 1         | Alice    | alice@    | 555-0101  |
| 2         | Dave     | dave@     | 555-0202  |
+-----------+----------+-----------+-----------+

Table: orders
+---------+-----------+
|order_id | customer_id|
|(PK)     | (FK)       |
+---------+-----------+
| 1001    | 1          |
| 1002    | 2          |
| 1003    | 1          |
+---------+-----------+

Table: books
+---------+-----------+-------+-----------+-----------+
|book_id  | book_title| price | author    |author_email|
|(PK)     |           |       |           |            |
+---------+-----------+-------+-----------+-----------+
| 1       | SQL Made  | 29.99 | Bob Lee   | bob@pub   |
| 2       | Data Design| 39.99| Carol King| carol@pub |
| 3       | NoSQL     | 34.99 | Bob Lee   | bob@pub   |
+---------+-----------+-------+-----------+-----------+

Table: order_items
+---------+---------+-----+
|order_id | book_id | qty |
|(PK)     | (PK)    |     |
+---------+---------+-----+
| 1001    | 1       |  2  |
| 1001    | 2       |  1  |
| 1002    | 1       |  1  |
| 1003    | 3       |  3  |
+---------+---------+-----+
```

Wait, the `books` table still has a problem. `author_email` depends on `author`, not on `book_id`. Let us fix that too:

```
Table: authors
+-----------+-----------+-----------+
|author_id  | author_name| author_email|
|(PK)       |            |            |
```

```
+-----------+-----------+-----------+
| 1         | Bob Lee    | bob@pub    |
| 2         | Carol King | carol@pub  |
+-----------+-----------+-----------+

Table: books
+---------+-----------+-------+-----------+
|book_id  | book_title| price | author_id |
|(PK)     |           |       | (FK)      |
+---------+-----------+-------+-----------+
| 1       | SQL Made  | 29.99 | 1         |
| 2       | Data Design| 39.99| 2         |
| 3       | NoSQL     | 34.99 | 1         |
+---------+-----------+-------+-----------+
```

Now every non-key column depends directly on its table's primary key.

The Full Normalization Journey

Let us visualize the entire transformation from messy spreadsheet to clean 3NF design:

```
ORIGINAL MESSY SPREADSHEET (Not normalized)
+-------------------------------------------------------------+
| order_id, cust_name, cust_email, cust_phone,                |
| book_title, author, author_email, price, qty                |
+-------------------------------------------------------------+
                     |
                     | Apply 1NF: Atomic values, no repeating groups
                     v
+-------------------------------------------------------------+
| Each cell has one value, rows uniquely identifiable         |
| Still one big table                                         |
+-------------------------------------------------------------+
                     |
                     | Apply 2NF: Remove partial dependencies
                     v
+-------------------+  +-------------------+  +--------------+
| orders            |  | books             |  | order_items  |
| order_id (PK)     |  | book_title (PK)   |  | order_id(PK) |
| cust_name         |  | price             |  | book_title(PK)|
| cust_email        |  | author            |  | qty          |
| cust_phone        |  | author_email      |  |              |
+-------------------+  +-------------------+  +--------------+
                     |
                     | Apply 3NF: Remove transitive dependencies
                     v
+-----------+  +----------+  +----------+  +--------+  +------------+
```

```
| customers |  | orders   |  | books    |  | authors|  | order_items|
| cust_id   |  | order_id |  | book_id  |  | auth_id|  | order_id   |
| name      |  | cust_id  |  | title    |  | name   |  | book_id    |
| email     |  | date     |  | price    |  | email  |  | qty        |
| phone     |  |          |  | auth_id  |  |        |  |            |
+-----------+  +----------+  +----------+  +--------+  +------------+
```

BCNF (Boyce-Codd Normal Form)

BCNF is a stricter version of 3NF. A table is in BCNF if:

1. It is in 3NF.
2. For every dependency, the left side (determinant) is a candidate key (a column or set of columns that could serve as the primary key).

In practice, most tables that are in 3NF are also in BCNF. The difference only matters in rare cases involving overlapping composite candidate keys.

Simple Example

Consider a table tracking which teachers teach which subjects in which rooms:

```
+---------+---------+------+
| teacher | subject | room |
+---------+---------+------+
| Smith   | Math    | 101  |
| Smith   | Science | 102  |
| Jones   | Math    | 101  |
+---------+---------+------+

Rules:
  - Each room is used for only one subject.
  - A teacher can teach multiple subjects.
  - Multiple teachers can teach in the same room.

Dependencies:
  (teacher, subject) --> room    (composite key determines room)
  room --> subject               (room determines subject)

Problem: room --> subject, but room is NOT a candidate key.
This violates BCNF.
```

The fix is to split into two tables:

```
Table: room_subjects          Table: teacher_rooms
+------+---------+            +---------+------+
| room | subject |            | teacher | room |
+------+---------+            +---------+------+
| 101  | Math    |            | Smith   | 101  |
| 102  | Science |            | Smith   | 102  |
+------+---------+            | Jones   | 101  |
                              +---------+------+
```

For most real-world databases, achieving 3NF is sufficient. BCNF matters mainly in academic study and edge cases.

Denormalization: When to Break the Rules

Normalization is about correctness and data integrity. But sometimes, fully normalized databases can be slow for certain queries because they require many JOINs to reassemble the data.

Denormalization is the deliberate process of introducing some redundancy back into your database to improve read performance.

When to Consider Denormalization

```
+-----------------------------------+-----------------------------------+
| Normalize When:                   | Denormalize When:                 |
+-----------------------------------+-----------------------------------+
| Data integrity is critical        | Read performance is critical      |
| Data changes frequently           | Data rarely changes               |
| Storage space is limited          | JOINs are too slow                |
| You need to avoid anomalies       | You need reporting/analytics speed|
| It is an OLTP system (many        | It is an OLAP system (few writes, |
|   writes, small reads)            |   large reads)                    |
+-----------------------------------+-----------------------------------+
```

Example: Denormalization for Reporting

Suppose you have a fully normalized schema with customers, orders, order_items, products, and categories. A dashboard query joins all five tables to show sales by category.

```sql
-- Normalized: 5-table JOIN (slow for a dashboard)
SELECT
    c.category_name,
    SUM(oi.qty * p.price) AS revenue
FROM categories c
JOIN products p ON c.category_id = p.category_id
JOIN order_items oi ON p.product_id = oi.product_id
JOIN orders o ON oi.order_id = o.order_id
JOIN customers cu ON o.customer_id = cu.customer_id
WHERE o.order_date >= '2024-01-01'
GROUP BY c.category_name;
```

A denormalized approach might add `category_name` directly to the `order_items` table:

```sql
-- Denormalized: simpler, faster query
SELECT
    category_name,
    SUM(line_total) AS revenue
FROM order_items_denormalized
WHERE order_date >= '2024-01-01'
GROUP BY category_name;
```

The tradeoff: if a category name changes, you must update it in the `order_items_denormalized` table too. But for a reporting database that rarely sees category changes, this is often acceptable.

Common Denormalization Techniques

1. Add redundant columns. Store a frequently needed value from another table directly (e.g., store `customer_name` in the orders table).
2. Use materialized views. Create a pre-computed view of joined data (covered in Chapter 20).
3. Create summary tables. Maintain pre-aggregated data for dashboards:

```sql
CREATE TABLE daily_sales_summary (
    sale_date       DATE,
    category_name   VARCHAR(50),
    total_orders    INTEGER,
    total_revenue   NUMERIC(12,2)
);
```

4. Store computed values. Instead of calculating `total_amount` from line items every time, store it directly in the orders table.

Step-by-Step: Normalizing an Order Spreadsheet

Let us do a complete normalization exercise from start to finish.

Step 0: The Original Spreadsheet

A company tracks orders in a spreadsheet:

```
+--------+-------+--------+---------+--------+------+--------+-------+--------+
|OrderNo |ODate  |CustName|CustCity |Product |ProdCat|Supplier|SuppPh |Qty     |
+--------+-------+--------+---------+--------+------+--------+-------+--------+
|5001    |Jan 10 |Alice   |New York |Laptop  |Elec  |TechCo  |555-100|2       |
|5001    |Jan 10 |Alice   |New York |Mouse   |Elec  |TechCo  |555-100|5       |
|5001    |Jan 10 |Alice   |New York |Desk    |Furn  |OfficeCo|555-200|1       |
|5002    |Jan 12 |Bob     |Chicago  |Laptop  |Elec  |TechCo  |555-100|1       |
|5003    |Jan 15 |Alice   |New York |Chair   |Furn  |OfficeCo|555-200|4       |
+--------+-------+--------+---------+--------+------+--------+-------+--------+
```

Step 1: Verify 1NF

Check: Are all values atomic? Yes, each cell has one value. Check: Are there repeating groups? No repeating column patterns. Check: Can we identify each row? The combination of (OrderNo, Product) is unique.

The table is in 1NF.

Step 2: Achieve 2NF

Identify the composite key: `(OrderNo, Product)`

Find partial dependencies (columns that depend on only part of the key):

- `ODate, CustName, CustCity` depend only on `OrderNo`
- `ProdCat, Supplier, SuppPh` depend only on `Product`
- `Qty` depends on the full key `(OrderNo, Product)`

Split into tables:

```sql
-- Table: orders (depends on OrderNo)
CREATE TABLE orders (
    order_no    INTEGER PRIMARY KEY,
    order_date  DATE,
    cust_name   VARCHAR(50),
```

```
    cust_city   VARCHAR(50)
);

-- Table: products (depends on Product)
CREATE TABLE products (
    product_name VARCHAR(50) PRIMARY KEY,
    category     VARCHAR(50),
    supplier     VARCHAR(50),
    supplier_ph  VARCHAR(20)
);

-- Table: order_items (depends on full key)
CREATE TABLE order_items (
    order_no     INTEGER,
    product_name VARCHAR(50),
    qty          INTEGER,
    PRIMARY KEY (order_no, product_name)
);
```

Step 3: Achieve 3NF

Check for transitive dependencies in each table.

In `orders`:

- `order_no --> cust_name --> cust_city` (city depends on customer, not order)
- This is a transitive dependency. Split customers out.

In `products`:

- `product_name --> supplier --> supplier_ph` (phone depends on supplier, not product)
- This is a transitive dependency. Split suppliers out.

Final 3NF schema:

```sql
CREATE TABLE customers (
    customer_id  SERIAL PRIMARY KEY,
    cust_name    VARCHAR(50),
    cust_city    VARCHAR(50)
);

CREATE TABLE suppliers (
    supplier_id  SERIAL PRIMARY KEY,
    supplier_name VARCHAR(50),
    supplier_ph   VARCHAR(20)
);

CREATE TABLE orders (
    order_no     INTEGER PRIMARY KEY,
```

```
    order_date   DATE,
    customer_id  INTEGER REFERENCES customers(customer_id)
);

CREATE TABLE products (
    product_id   SERIAL PRIMARY KEY,
    product_name VARCHAR(50),
    category     VARCHAR(50),
    supplier_id  INTEGER REFERENCES suppliers(supplier_id)
);

CREATE TABLE order_items (
    order_no     INTEGER REFERENCES orders(order_no),
    product_id   INTEGER REFERENCES products(product_id),
    qty          INTEGER,
    PRIMARY KEY (order_no, product_id)
);
```

Final Schema Diagram

```
+------------+       +------------+       +------------+
| customers  |       | orders     |       | order_items|
+------------+       +------------+       +------------+
| customer_id|<------| order_no   |<------| order_no   |
| cust_name  |  (FK) | order_date |       | product_id |----+
| cust_city  |       | customer_id|       | qty        |    |
+------------+       +------------+       +------------+    |
                                                             |
+------------+       +------------+                         |
| suppliers  |       | products   |                         |
+------------+       +------------+                         |
| supplier_id|<------| product_id |<------------------------+
| supplier_name|  (FK)| product_name|
| supplier_ph|       | category   |
+------------+       | supplier_id|
                     +------------+
```

Verify: Anomalies Are Gone

- Update: Change Alice's city? Update one row in `customers`. All her orders still link to the correct customer record.
- Insert: Add a new product? Insert into `products`. No order needed.
- Delete: Delete order 5002? Bob's customer record remains in `customers`.

Normal Forms Summary

```
+------+----------------------------+----------------------------------+
| Form | Rule                       | What It Prevents                 |
+------+----------------------------+----------------------------------+
| 1NF  | Atomic values              | Comma-separated lists,           |
|      | No repeating groups        | repeating columns (book_1,       |
|      | Unique rows                | book_2, book_3)                  |
+------+----------------------------+----------------------------------+
| 2NF  | No partial dependencies    | Non-key column depending on      |
|      | (depends on full key)      | only part of a composite key     |
+------+----------------------------+----------------------------------+
| 3NF  | No transitive dependencies | Non-key column depending on      |
|      | (depends directly on key)  | another non-key column           |
+------+----------------------------+----------------------------------+
| BCNF | Every determinant is a     | Rare edge cases with             |
|      | candidate key              | overlapping candidate keys       |
+------+----------------------------+----------------------------------+
```

Common Mistakes

Mistake 1: Storing Comma-Separated Lists

```sql
-- WRONG: Violates 1NF
CREATE TABLE students (
    student_id   INTEGER PRIMARY KEY,
    student_name VARCHAR(50),
    courses      VARCHAR(200)  -- 'Math, Science, English'
);

-- CORRECT: Separate table for the many-to-many relationship
CREATE TABLE enrollments (
    student_id INTEGER REFERENCES students(student_id),
    course_id  INTEGER REFERENCES courses(course_id),
    PRIMARY KEY (student_id, course_id)
);
```

Mistake 2: Over-Normalizing

Not everything needs to be in a separate table. A country code inside an address table is fine. You do not need a separate table for every small piece of data.

```sql
-- OVER-NORMALIZED: Separate table for city? Overkill for most apps.
CREATE TABLE cities (city_id SERIAL PRIMARY KEY, city_name VARCHAR(50));

-- REASONABLE: City as a column in the address table.
CREATE TABLE addresses (
    address_id SERIAL PRIMARY KEY,
    street     VARCHAR(100),
    city       VARCHAR(50),
    state      VARCHAR(2)
);
```

Mistake 3: Confusing Normalization with Optimization

Normalization is about correctness, not speed. Sometimes a normalized schema is slower for queries. That is okay -- you can add indexes, materialized views, or targeted denormalization to improve speed without abandoning correctness everywhere.

Mistake 4: Skipping the Design Step

Do not jump straight into creating tables. Start with a list of all the data you need to store, identify the dependencies, and work through the normal forms systematically.

Best Practices

1. Aim for 3NF as your default. Most applications benefit from 3NF. Only denormalize when you have measured evidence of a performance problem.
2. Normalize first, denormalize later. It is much easier to denormalize a normalized schema than to normalize a denormalized one.
3. Identify entities clearly. Each "thing" in your system (customer, order, product, supplier) should have its own table.
4. Use surrogate keys. Instead of using `cust_name` as a primary key, add an auto-generated `customer_id`. Names change; IDs do not.
5. Document your design decisions. If you denormalize, document why. Future developers (including your future self) will thank you.
6. Think about how data will change. Normalization prevents anomalies. Ask yourself: "If this data changes, will I need to update it in multiple places?"
7. Use foreign keys. Always define foreign key relationships. They enforce referential integrity and make the relationships between tables explicit.

Quick Summary

```
+---------------------------+-----------------------------------------+
| Concept                   | Description                             |
+---------------------------+-----------------------------------------+
| Normalization             | Organizing tables to reduce redundancy  |
+---------------------------+-----------------------------------------+
| Update Anomaly            | Same fact in multiple rows, risk of     |
|                           | inconsistency when updating             |
+---------------------------+-----------------------------------------+
| Insert Anomaly            | Cannot add data without unrelated data  |
+---------------------------+-----------------------------------------+
| Delete Anomaly            | Removing a row loses unrelated facts    |
+---------------------------+-----------------------------------------+
| 1NF                       | Atomic values, no repeating groups      |
+---------------------------+-----------------------------------------+
| 2NF                       | No partial dependencies on composite key |
+---------------------------+-----------------------------------------+
| 3NF                       | No transitive dependencies              |
+---------------------------+-----------------------------------------+
| BCNF                      | Every determinant is a candidate key    |
+---------------------------+-----------------------------------------+
| Denormalization           | Adding controlled redundancy for speed  |
+---------------------------+-----------------------------------------+
```

Key Points

- Normalization is the process of organizing database tables to eliminate redundancy and prevent update, insert, and delete anomalies.
- 1NF requires atomic values (no lists or repeating groups in a single column).
- 2NF requires that all non-key columns depend on the entire primary key, not just part of it. Only relevant for composite keys.
- 3NF requires that non-key columns depend directly on the primary key, not through other non-key columns (no transitive dependencies).
- BCNF is a stricter 3NF where every determinant must be a candidate key.
- Denormalization deliberately reintroduces redundancy for performance, typically in reporting or analytics systems.
- Always normalize first, then denormalize specific areas only when you have measured evidence of performance problems.

- Use surrogate keys (auto-generated IDs) instead of natural keys when possible.

Practice Questions

1. Explain the three types of data anomalies (update, insert, delete) using a real-world example of your choosing.
2. A table has the columns `(student_id, course_id, student_name, course_name, grade)` with a composite primary key of `(student_id, course_id)`. Which normal form does this table violate? What are the partial dependencies? How would you fix it?
3. What is a transitive dependency? Give an example of a table that violates 3NF and show how to fix it.
4. When is denormalization appropriate? Give two specific scenarios where you might choose to denormalize.
5. Why is it recommended to "normalize first, denormalize later" rather than starting with a denormalized design?

Exercises

Exercise 1: Normalize a Library Spreadsheet

A library tracks loans in one spreadsheet:

```
| LoanID | MemberName | MemberPhone | BookTitle | Author | AuthorCountry | DueDate | Fi...
```

Identify all anomalies. Normalize this to 3NF. Show each step (1NF, 2NF, 3NF) and the final set of CREATE TABLE statements.

Exercise 2: Normalize an Order Spreadsheet

You receive this spreadsheet from a client:

```
| InvoiceNo | InvoiceDate | CustomerName | CustomerAddress | ProductCode | ProductDesc ...
```

The primary key is `(InvoiceNo, ProductCode)`. Normalize to 3NF. Write the final CREATE TABLE statements with appropriate primary keys and foreign keys.

Exercise 3: Identify the Normal Form

For each of the following tables, identify the highest normal form it satisfies (1NF, 2NF, 3NF, or unnormalized). Explain your reasoning.

Table A: `employee_id, employee_name, department_id, department_name, department_location`

Table B: `order_id, product_id, quantity` (composite PK: order_id, product_id)

Table C: `student_id, courses_enrolled` (where courses_enrolled = "Math, Science, English")

What Is Next?

Now that you understand how to organize your data through normalization, the next chapter will introduce ER Diagrams (Entity-Relationship Diagrams). ER diagrams are visual blueprints that help you plan and communicate your database design before writing a single line of SQL. They are the bridge between understanding your data (normalization) and building your schema (SQL).

Chapter 23: ER Diagrams (Entity-Relationship Diagrams)

What You Will Learn

In this chapter, you will learn how to design databases visually using Entity-Relationship (ER) diagrams. You will discover how to identify entities, define attributes, and draw relationships with proper cardinality. You will also learn crow's foot notation and practice converting ER diagrams into actual SQL tables.

Why This Chapter Matters

Imagine building a house without a blueprint. You start laying bricks, realize the kitchen is too small, knock down a wall, rebuild it, then discover the plumbing does not reach the bathroom. Every mistake costs time and money.

Database design works the same way. If you jump straight into writing CREATE TABLE statements without planning, you will inevitably discover missing relationships, redundant columns, and structural problems that are painful to fix once data is stored.

ER diagrams are your blueprint. They let you plan your entire database visually before writing any SQL. You can spot problems, discuss the design with teammates, and make changes on paper instead of in production. Every professional database project starts with an ER diagram.

What Is an ER Diagram?

An ER diagram is a visual representation of the data in your system and the relationships between different pieces of data. It was introduced by Peter Chen in 1976 and has become

the standard way to design relational databases.

An ER diagram has three main components:

```
+-------------------+
|   ENTITIES        |  =  The "things" you want to store data about
|   (rectangles)    |     (Customer, Order, Product)
+-------------------+

+-------------------+
|   ATTRIBUTES      |  =  The properties of each entity
|   (listed inside)|     (name, email, price)
+-------------------+

+-------------------+
|   RELATIONSHIPS   |  =  How entities are connected
|   (lines)         |     (Customer "places" Order)
+-------------------+
```

Think of it like planning a school:

- Entities = Students, Teachers, Courses, Classrooms
- Attributes = Student has name, age, grade. Course has title, credits.
- Relationships = Students enroll in Courses. Teachers teach Courses.

Entities

An entity represents a "thing" or concept that you want to track in your database. Each entity becomes a table in your final database.

How to Identify Entities

Ask yourself: "What are the main things I need to store information about?"

- In an e-commerce system: Customers, Products, Orders, Categories
- In a library: Members, Books, Authors, Loans
- In a hospital: Patients, Doctors, Appointments, Departments

Representing Entities

In an ER diagram, entities are drawn as rectangles with the entity name inside.

```
+------------+    +------------+    +------------+
|  CUSTOMER  |    |   ORDER    |    |  PRODUCT   |
+------------+    +------------+    +------------+
```

Each entity will have a primary key attribute that uniquely identifies each instance (each row in the eventual table).

Attributes

Attributes are the properties or characteristics of an entity. They become columns in your table.

Types of Attributes

```
+-------------------------+-------------------------------------------+
| Attribute Type          | Example                                   |
+-------------------------+-------------------------------------------+
| Simple                  | first_name, price, quantity                |
| Composite               | full_name = first_name + last_name        |
| Derived                 | age (calculated from date_of_birth)       |
| Multi-valued            | phone_numbers (a person may have several) |
| Key (Primary Key)       | customer_id, product_id                   |
+-------------------------+-------------------------------------------+
```

Representing Attributes

Attributes are listed inside the entity rectangle. The primary key is underlined or marked with (PK).

```
+---------------------+
|      CUSTOMER       |
+---------------------+
| customer_id  (PK)   |
| first_name           |
| last_name            |
| email                |
| phone                |
| created_at           |
+---------------------+
```

Relationships

Relationships describe how entities are connected to each other. They answer questions like:

- A Customer places how many Orders?
- An Order contains how many Products?
- A Doctor treats how many Patients?

Representing Relationships

Relationships are shown as lines connecting entities, with a label describing the relationship.

```
+------------+                    +------------+
|  CUSTOMER  |----< places >----|   ORDER    |
+------------+                    +------------+
```

Cardinality: How Many?

Cardinality defines the number of instances of one entity that can be associated with instances of another entity. There are three main types:

One-to-One (1:1)

Each instance of Entity A is associated with exactly one instance of Entity B, and vice versa.

Example: Each employee has exactly one company laptop. Each laptop is assigned to exactly one employee.

```
+------------+        1:1        +------------+
|  EMPLOYEE  |------------------| LAPTOP     |
+------------+                   +------------+

One Employee  <-->  One Laptop
```

Real-world examples:

- Person and Passport

- Employee and Parking Spot
- Country and Capital City

One-to-Many (1:N)

Each instance of Entity A can be associated with many instances of Entity B, but each instance of Entity B is associated with exactly one instance of Entity A.

Example: A customer can place many orders, but each order belongs to exactly one customer.

```
+------------+        1:N        +------------+
|  CUSTOMER  |---||-----------|<-|   ORDER    |
+------------+                   +------------+

One Customer  -->  Many Orders
One Order     -->  One Customer
```

Real-world examples:

- Department and Employees (one department has many employees)
- Author and Books (one author writes many books)
- Parent and Children

Many-to-Many (M:N)

Each instance of Entity A can be associated with many instances of Entity B, and each instance of Entity B can be associated with many instances of Entity A.

Example: A student can enroll in many courses. A course can have many students.

```
+------------+        M:N        +------------+
|  STUDENT   |------|<---->|-----|  COURSE    |
+------------+                   +------------+

One Student   -->  Many Courses
One Course    -->  Many Students
```

Real-world examples:

- Students and Courses
- Actors and Movies
- Products and Orders (an order has many products; a product appears in many orders)

> Important: Many-to-many relationships cannot be directly implemented in a relational database. They require a junction table (also called a bridge table or associative table). We will cover this in the conversion section.

Crow's Foot Notation

Crow's foot notation is the most widely used notation for ER diagrams in practice. It uses special symbols at the ends of relationship lines to indicate cardinality.

The Symbols

```
----||----    Exactly one (mandatory)
----O|----    Zero or one (optional)
----|<----    One or many (at least one)
----O<----    Zero or many (optional, could be none)

Symbol Key:
  ||   = One (and only one)
  O|   = Zero or one
  |<   = One or many     (the "crow's foot" or "chicken foot")
  O<   = Zero or many
```

Reading Crow's Foot Diagrams

Read the symbols from the perspective of each entity.

```
CUSTOMER  ||--------O<  ORDER

Read from left to right:
  "A Customer can have ZERO OR MANY (O<) Orders."

Read from right to left:
  "An Order belongs to exactly ONE (||) Customer."
```

Common Relationships in Crow's Foot

```
One-to-One (mandatory on both sides):
  EMPLOYEE  ||--------||  PARKING_SPOT
```

```
One-to-Many (customer must exist, orders are optional):
  CUSTOMER  ||--------O<  ORDER

Many-to-Many (through junction table):
  STUDENT  >O--------O<  ENROLLMENT  >O--------O<  COURSE
```

Full Crow's Foot Example

```
+------------+          +------------+          +--------------+
|  CUSTOMER  |          |   ORDER    |          | ORDER_ITEM   |
+------------+          +------------+          +--------------+
| customer_id|          | order_id   |          | item_id      |
| name       |---||--O<-| order_date |---||--O<-| quantity      |
| email      |          | customer_id|          | order_id     |
| phone      |          | total      |          | product_id   |
+------------+          +------------+          +--------------+
                                                       |
                                                    O| |
                                                    || |
                                                       |
                                                +--------------+
                                                |   PRODUCT    |
                                                +--------------+
                                                | product_id   |
                                                | name         |
                                                | price        |
                                                | category     |
                                                +--------------+

Reading:
  - One Customer has zero or many Orders
  - One Order belongs to exactly one Customer
  - One Order has zero or many Order Items
  - One Order Item belongs to exactly one Order
  - One Product appears in zero or many Order Items
  - One Order Item refers to exactly one Product
```

Drawing ER Diagrams: Library System

Let us design a library management system step by step.

Step 1: Identify Entities

What "things" does a library need to track?

- Members -- people who borrow books
- Books -- the physical books in the library
- Authors -- who wrote the books
- Loans -- the borrowing transactions
- Categories -- book categories (Fiction, Science, History)

Step 2: Define Attributes

```
+---------------------+    +---------------------+    +---------------------+
|       MEMBER        |    |        BOOK         |    |       AUTHOR        |
+---------------------+    +---------------------+    +---------------------+
| member_id    (PK)   |    | book_id      (PK)   |    | author_id    (PK)   |
| first_name          |    | title               |    | first_name          |
| last_name           |    | isbn                |    | last_name           |
| email               |    | publish_year        |    | birth_year          |
| phone               |    | category_id  (FK)   |    | nationality         |
| join_date            |    | copies_available    |    +---------------------+
+---------------------+    +---------------------+

+---------------------+    +---------------------+
|        LOAN         |    |      CATEGORY       |
+---------------------+    +---------------------+
| loan_id      (PK)   |    | category_id  (PK)   |
| member_id    (FK)   |    | category_name       |
| book_id      (FK)   |    | description         |
| loan_date           |    +---------------------+
| due_date            |
| return_date         |
+---------------------+
```

Step 3: Define Relationships

```
MEMBER  ||--------O<  LOAN
  "A member can have zero or many loans."
  "A loan belongs to exactly one member."

BOOK  ||--------O<  LOAN
  "A book can have zero or many loans."
  "A loan is for exactly one book."

CATEGORY  ||--------O<  BOOK
  "A category contains zero or many books."
  "A book belongs to exactly one category."

AUTHOR  >O--------O<  BOOK_AUTHOR  >O--------O<  BOOK
  "An author can write many books."
  "A book can have many authors."
```

```
  (Many-to-many, needs junction table BOOK_AUTHOR)
```

Step 4: Draw the Complete ER Diagram

```
+----------+                              +----------+
| CATEGORY |                              |  AUTHOR  |
+----------+                              +----------+
|cat_id(PK)|                              |auth_id(PK)|
|cat_name  |                              |first_name|
|descript. |                              |last_name |
+----------+                              |birth_year|
     |                                    +----------+
     | 1                                       |
     |                                         | M
     | N                                       |
+----------+       +----------+       +-------------+
|   BOOK   |       |   LOAN   |       | BOOK_AUTHOR |
+----------+       +----------+       +-------------+
|book_id(PK)|--||--O<|loan_id(PK)|    |book_id (FK) |
|title      |      |member_id(FK)|   |author_id(FK)|
|isbn       |      |book_id(FK) |    +-------------+
|pub_year   |      |loan_date   |          |
|cat_id(FK) |      |due_date    |          | N
|copies     |      |return_date |          |
+----------+       +----------+       +----------+
                        |              |   BOOK   |
                        | N            +----------+
                        |
                   +----------+
                   |  MEMBER  |
                   +----------+
                   |mem_id(PK)|--||--O< LOAN
                   |first_name|
                   |last_name |
                   |email     |
                   |phone     |
                   |join_date |
                   +----------+
```

Drawing ER Diagrams: E-Commerce System

Entities and Relationships

```
+-------------+       +-------------+       +---------------+
|  CUSTOMER   |       |    ORDER    |       |  ORDER_ITEM   |
+-------------+       +-------------+       +---------------+
|customer_id  |       |order_id     |       |item_id        |
|first_name   |-||--O<|order_date   |-||--O<|order_id   (FK)|
|last_name    |       |customer_id  |       |product_id (FK)|
|email        |       |ship_address |       |quantity       |
|password_hash|       |total_amount |       |unit_price     |
|created_at   |       |status       |       |line_total     |
+-------------+       +-------------+       +---------------+
                                                   |
                                                   |O<
                                                   |
                                                   |||
                                              +---------------+
                                              |   PRODUCT     |
+-------------+                               +---------------+
|  CATEGORY   |                               |product_id     |
+-------------+                               |product_name   |
|category_id  |-||-----------------------O<|description    |
|cat_name     |                               |price          |
|parent_cat_id|                               |stock_quantity |
+-------------+                               |category_id(FK)|
      |                                       +---------------+
      |
      +----> (self-referencing for sub-categories)


+-------------+       +-------------+
|   REVIEW    |       |   PAYMENT   |
+-------------+       +-------------+
|review_id    |       |payment_id   |
|product_id   |       |order_id (FK)|
|customer_id  |       |amount       |
|rating       |       |method       |
|comment      |       |paid_at      |
|created_at   |       |status       |
+-------------+       +-------------+
```

Relationships:

- Customer 1 --> N Orders (one customer places many orders)
- Order 1 --> N Order Items (one order contains many items)
- Product 1 --> N Order Items (one product appears in many order items)
- Category 1 --> N Products (one category has many products)
- Category 1 --> N Category (self-referencing for subcategories)
- Customer 1 --> N Reviews (one customer writes many reviews)
- Product 1 --> N Reviews (one product has many reviews)
- Order 1 --> 1 Payment (one order has one payment)

Drawing ER Diagrams: Hospital System

Entities and Relationships

```
+---------------+       +----------------+       +---------------+
|   PATIENT     |       |  APPOINTMENT   |       |    DOCTOR     |
+---------------+       +----------------+       +---------------+
|patient_id(PK)|       |appt_id    (PK) |       |doctor_id(PK) |
|first_name    |-||--O<|patient_id (FK) |>O--||+|first_name    |
|last_name     |       |doctor_id  (FK) |       |last_name     |
|dob           |       |appt_date       |       |specialty     |
|gender        |       |appt_time       |       |phone         |
|phone         |       |reason          |       |dept_id  (FK) |
|address       |       |status          |       +---------------+
|blood_type    |       +----------------+               |
+---------------+                                        | N
      |                                                  |
      | 1                                                | 1
      |                                           +---------------+
      | N                                         |  DEPARTMENT   |
+---------------+                                 +---------------+
| PRESCRIPTION |                                  |dept_id  (PK) |
+---------------+                                 |dept_name     |
|rx_id    (PK) |                                  |floor         |
|patient_id(FK)|                                  |phone         |
|doctor_id(FK) |                                  +---------------+
|medication    |
|dosage        |
|start_date    |
|end_date      |
+---------------+

+---------------+       +---------------+
|     ROOM      |       |  ADMISSION    |
+---------------+       +---------------+
|room_id  (PK) |       |admission_id   |
|room_number   |-||--O<|room_id  (FK) |>O--||--PATIENT
|room_type     |       |patient_id(FK)|
|floor         |       |admit_date    |
|is_available  |       |discharge_date|
+---------------+       |doctor_id(FK) |
                        +---------------+
```

Relationships:

- Patient 1 --> N Appointments
- Doctor 1 --> N Appointments
- Patient 1 --> N Prescriptions

- Doctor 1 --> N Prescriptions
- Department 1 --> N Doctors
- Room 1 --> N Admissions
- Patient 1 --> N Admissions

Converting ER Diagrams to Tables

Once you have an ER diagram, converting it to SQL tables follows clear rules.

Rule 1: Each Entity Becomes a Table

```
ER Entity:                      SQL Table:
+------------+                  CREATE TABLE customers (
|  CUSTOMER  |                      customer_id  SERIAL PRIMARY KEY,
+------------+                      first_name   VARCHAR(50),
| customer_id|                      last_name    VARCHAR(50),
| first_name |                      email        VARCHAR(100),
| last_name  |      =====>         phone        VARCHAR(20)
| email      |                  );
| phone      |
+------------+
```

Rule 2: 1:N Relationships Use Foreign Keys

The "many" side gets a foreign key pointing to the "one" side.

```
CUSTOMER  ||--------O<  ORDER

CREATE TABLE orders (
    order_id     SERIAL PRIMARY KEY,
    order_date   DATE,
    customer_id  INTEGER REFERENCES customers(customer_id),  -- FK
    total_amount NUMERIC(10,2)
);
```

Rule 3: M:N Relationships Need a Junction Table

Many-to-many relationships cannot be directly represented. You create a junction table with foreign keys to both entities.

```
STUDENT  >O--------O<  COURSE
      (Many-to-Many)

Becomes three tables:

CREATE TABLE students (
    student_id  SERIAL PRIMARY KEY,
    name        VARCHAR(100)
);

CREATE TABLE courses (
    course_id   SERIAL PRIMARY KEY,
    title       VARCHAR(100)
);

CREATE TABLE enrollments (                -- Junction table
    student_id  INTEGER REFERENCES students(student_id),
    course_id   INTEGER REFERENCES courses(course_id),
    enrolled_at DATE DEFAULT CURRENT_DATE,
    grade       VARCHAR(2),
    PRIMARY KEY (student_id, course_id)   -- Composite PK
);
```

Rule 4: 1:1 Relationships Use Foreign Key with UNIQUE

Either table can hold the foreign key. Add a UNIQUE constraint to enforce the one-to-one relationship.

```
EMPLOYEE  ||--------||  PARKING_SPOT

CREATE TABLE parking_spots (
    spot_id      SERIAL PRIMARY KEY,
    spot_number  VARCHAR(10),
    location     VARCHAR(50)
);

CREATE TABLE employees (
    employee_id  SERIAL PRIMARY KEY,
    name         VARCHAR(100),
    spot_id      INTEGER UNIQUE REFERENCES parking_spots(spot_id)  -- FK + UNIQUE
);
```

Conversion Summary

```
+-------------------+-------------------------------------------+
| ER Concept        | SQL Implementation                        |
+-------------------+-------------------------------------------+
```

```
| Entity            | CREATE TABLE                              |
| Attribute         | Column in the table                       |
| Primary Key       | PRIMARY KEY constraint                    |
| 1:1 Relationship  | FK with UNIQUE constraint                 |
| 1:N Relationship  | FK on the "many" side                     |
| M:N Relationship  | Junction table with two FKs               |
+-------------------+-------------------------------------------+
```

Complete Conversion Example: Library System

Let us convert our library ER diagram to actual SQL.

```sql
-- Independent tables first (no foreign keys needed)

CREATE TABLE categories (
    category_id    SERIAL PRIMARY KEY,
    category_name  VARCHAR(50) NOT NULL,
    description    TEXT
);

CREATE TABLE authors (
    author_id      SERIAL PRIMARY KEY,
    first_name     VARCHAR(50) NOT NULL,
    last_name      VARCHAR(50) NOT NULL,
    birth_year     INTEGER,
    nationality    VARCHAR(50)
);

CREATE TABLE members (
    member_id      SERIAL PRIMARY KEY,
    first_name     VARCHAR(50) NOT NULL,
    last_name      VARCHAR(50) NOT NULL,
    email          VARCHAR(100) UNIQUE,
    phone          VARCHAR(20),
    join_date      DATE DEFAULT CURRENT_DATE
);

-- Tables with foreign keys

CREATE TABLE books (
    book_id          SERIAL PRIMARY KEY,
    title            VARCHAR(200) NOT NULL,
    isbn             VARCHAR(20) UNIQUE,
    publish_year     INTEGER,
    category_id      INTEGER REFERENCES categories(category_id),
    copies_available INTEGER DEFAULT 1
```

```
);

-- Junction table for M:N relationship (Book <-> Author)

CREATE TABLE book_authors (
    book_id    INTEGER REFERENCES books(book_id),
    author_id  INTEGER REFERENCES authors(author_id),
    PRIMARY KEY (book_id, author_id)
);

-- Loans table (references both members and books)

CREATE TABLE loans (
    loan_id      SERIAL PRIMARY KEY,
    member_id    INTEGER NOT NULL REFERENCES members(member_id),
    book_id      INTEGER NOT NULL REFERENCES books(book_id),
    loan_date    DATE NOT NULL DEFAULT CURRENT_DATE,
    due_date     DATE NOT NULL,
    return_date  DATE
);
```

Tools for Drawing ER Diagrams

While ASCII diagrams are great for learning and quick sketches, professional projects benefit from dedicated tools.

Free Tools

- draw.io (diagrams.net) -- Free, browser-based diagramming tool. Supports ER diagram shapes and crow's foot notation. Can export to PNG, SVG, and PDF. Works offline. This is an excellent choice for beginners.
- dbdiagram.io -- Free online tool specifically designed for database diagrams. You write a simple text-based syntax and it generates the visual diagram. Very fast for creating schemas.

Example dbdiagram.io syntax:

```
Table customers {
  customer_id integer [pk, increment]
  first_name varchar
  last_name varchar
  email varchar [unique]
}
```

```
Table orders {
  order_id integer [pk, increment]
  customer_id integer [ref: > customers.customer_id]
  order_date date
  total numeric
}
```

- pgModeler -- Open-source tool specifically for PostgreSQL. Can generate SQL from diagrams and reverse-engineer diagrams from existing databases.

Tips for Drawing ER Diagrams

1. Start with entities. List all the "things" you need to track before thinking about relationships.
2. Add attributes. For each entity, list all the properties you need to store.
3. Identify primary keys. Every entity needs a unique identifier.
4. Draw relationships. Connect entities and label the relationships.
5. Determine cardinality. For each relationship, ask "how many?" on both sides.
6. Handle M:N relationships. Add junction tables where needed.
7. Review for normalization. Check that your design follows the normal forms from Chapter 22.

Common Mistakes

Mistake 1: Skipping the ER Diagram

Jumping straight into SQL without planning leads to redesigns later. Even a rough sketch on paper saves time.

Mistake 2: Confusing Entities with Attributes

```
WRONG: Making "phone_number" an entity
  +---------------+
  | PHONE_NUMBER  |   <-- This is an attribute, not an entity!
  +---------------+

RIGHT: Phone number is an attribute of Customer
  +------------+
  |  CUSTOMER  |
```

```
  +------------+
  | phone      |
  +------------+
```

An entity is something that has its own attributes and exists independently. "Phone number" is a property of a customer, not a thing on its own (unless you are building a phone system where phone numbers have their own attributes like carrier, type, etc.).

Mistake 3: Forgetting Junction Tables for M:N

```
WRONG: Trying to implement M:N directly
  students.course_id --> courses.course_id
  (But a student can have MANY courses! A single FK does not work.)

RIGHT: Use a junction table
  students --> enrollments <-- courses
```

Mistake 4: Wrong Cardinality

Think carefully about the business rules. "Can a book have zero authors?" (Probably not.) "Can a customer have zero orders?" (Yes, they just signed up.) Getting cardinality wrong leads to incorrect constraints in your database.

Mistake 5: Not Including All Necessary Attributes

Forgetting important attributes at the design stage means ALTER TABLE later. Take time to think through what data each entity needs.

Best Practices

1. Always start with an ER diagram. Even for small projects, spending 15 minutes on an ER diagram saves hours of rework.
2. Use consistent naming. Pick a convention (singular or plural table names, snake_case or camelCase) and stick with it throughout.
3. Include data types in your diagram. Knowing that `price` is NUMERIC(10,2) versus INTEGER affects your design decisions.

4. Review with stakeholders. Show your ER diagram to the people who will use the system. They can catch missing entities and incorrect relationships that you might miss.
5. Iterate. Your first ER diagram will not be perfect. Draw it, review it, improve it, and repeat.
6. Keep it readable. Avoid crossing lines when possible. Group related entities together. Use colors or sections for large diagrams.
7. Version control your diagrams. Save your ER diagram files alongside your code. When the schema changes, update the diagram.

Quick Summary

```
+---------------------------+-------------------------------------------+
| Concept                   | Description                               |
+---------------------------+-------------------------------------------+
| Entity                    | A thing to store data about (table)       |
+---------------------------+-------------------------------------------+
| Attribute                 | A property of an entity (column)          |
+---------------------------+-------------------------------------------+
| Relationship              | A connection between entities              |
+---------------------------+-------------------------------------------+
| 1:1 Cardinality           | One to one (FK + UNIQUE)                  |
+---------------------------+-------------------------------------------+
| 1:N Cardinality           | One to many (FK on the "many" side)       |
+---------------------------+-------------------------------------------+
| M:N Cardinality           | Many to many (junction table)             |
+---------------------------+-------------------------------------------+
| Crow's Foot Notation      | Industry-standard visual notation         |
+---------------------------+-------------------------------------------+
| Junction Table            | Bridge table for M:N relationships        |
+---------------------------+-------------------------------------------+
```

Key Points

- ER diagrams are visual blueprints for database design. Always create one before writing SQL.
- Entities are the things you store data about. They become tables.
- Attributes are properties of entities. They become columns.

- Relationships connect entities. They are implemented with foreign keys and junction tables.
- Cardinality defines how many instances relate to each other: 1:1, 1:N, or M:N.
- Crow's foot notation is the most common way to show cardinality in professional diagrams.
- M:N relationships always require a junction table with foreign keys to both entities.
- Convert ER to SQL by making entities into tables, attributes into columns, and relationships into foreign keys.
- Tools like draw.io and dbdiagram.io make creating ER diagrams fast and free.
- Review and iterate on your ER diagram before building the database.

Practice Questions

1. What are the three main components of an ER diagram? Give an example of each from a school system.
2. Explain the difference between 1:1, 1:N, and M:N relationships. For each, give a real-world example and explain how it would be implemented in SQL.
3. Why can many-to-many relationships not be directly implemented in a relational database? What solution do we use instead?
4. In crow's foot notation, what does `||--------0<` mean? Read it from both directions.
5. You are designing a music streaming service. Identify at least five entities and their relationships (with cardinality).

Exercises

Exercise 1: Design a Library System ER Diagram

Draw a complete ER diagram for a library system with these requirements:

- Members can borrow books. Track loan date and due date.
- Books can have multiple authors. Authors can write multiple books.
- Books belong to categories (Fiction, Non-Fiction, Science, etc.).
- Members can write reviews for books (rating 1-5 and comment).

Show all entities, attributes, relationships, and cardinality using crow's foot notation. Then write the CREATE TABLE statements to implement your design.

Exercise 2: E-Commerce ER Diagram

Design an ER diagram for an online store:

- Customers can place orders containing multiple products.
- Products belong to categories. Categories can have subcategories.
- Customers can leave reviews on products.
- Orders have a shipping address and a billing address.
- Track order status history (ordered, processing, shipped, delivered).

Draw the ER diagram and convert it to CREATE TABLE statements.

Exercise 3: Convert an Existing Design

Given these business rules, draw the ER diagram and write the SQL:

- A hospital has many departments.
- Each department has many doctors. A doctor belongs to one department.
- Patients can have appointments with doctors.
- Doctors can prescribe medications to patients during appointments.
- Track the medication name, dosage, and duration for each prescription.

What Is Next?

Now that you can design your database visually with ER diagrams, the next chapter will cover Constraints and Keys in depth. You have already seen PRIMARY KEY and FOREIGN KEY in this chapter. Next, you will learn about all the constraint types (UNIQUE, NOT NULL, CHECK, DEFAULT) and how to use CASCADE rules to control what happens when referenced data is deleted or updated. Constraints are the rules that keep your data clean and consistent.

Chapter 24: Constraints and Keys — Guarding Your Data

What You Will Learn

In this chapter, you will learn:

- What constraints are and why every table needs them
- How PRIMARY KEY creates a unique identifier for each row
- How FOREIGN KEY enforces relationships between tables
- How UNIQUE prevents duplicate values
- How NOT NULL stops empty fields
- How CHECK adds custom validation rules
- How DEFAULT provides automatic values
- What composite keys are and when to use them
- What CASCADE options do (ON DELETE CASCADE, ON UPDATE CASCADE, RESTRICT, SET NULL, SET DEFAULT)
- How to add and drop constraints with ALTER TABLE
- How to build a fully constrained e-commerce schema

Why This Chapter Matters

Imagine you run a library. You have a simple rule: every book must have an ISBN number, and no two books can share the same ISBN. You also have a rule: every loan must reference a real member — you cannot lend books to people who do not exist in your system.

Now imagine your library software had no way to enforce these rules. A volunteer could accidentally enter a book without an ISBN. Another volunteer could record a loan for member number 9999, even though that member does not exist. Over time, your data becomes a mess. You have ghost loans, duplicate books, and no way to trust your records.

Constraints are the rules you attach to your tables so the database itself refuses bad data. You do not rely on the application or the person typing queries to "remember the rules." The database enforces them automatically, every single time, no exceptions.

Every professional database uses constraints. They are not optional extras — they are the foundation of data integrity.

Setting Up Our Practice Tables

Let us start with simple tables and add constraints as we learn each type:

```sql
CREATE TABLE customers (
    customer_id SERIAL PRIMARY KEY,
    first_name  VARCHAR(50) NOT NULL,
    last_name   VARCHAR(50) NOT NULL,
    email       VARCHAR(100) UNIQUE NOT NULL,
    phone       VARCHAR(20),
    created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

INSERT INTO customers (first_name, last_name, email, phone) VALUES
('Alice', 'Johnson', 'alice@example.com', '555-0101'),
('Bob',   'Smith',   'bob@example.com',   '555-0102'),
('Carol', 'Davis',   'carol@example.com',  NULL);
```

Let us verify:

```sql
SELECT customer_id, first_name, last_name, email
FROM customers;
```

```
 customer_id | first_name | last_name |       email
-------------+------------+-----------+-------------------
           1 | Alice      | Johnson   | alice@example.com
           2 | Bob        | Smith     | bob@example.com
           3 | Carol      | Davis     | carol@example.com
(3 rows)
```

PRIMARY KEY — The Unique Identifier

A PRIMARY KEY is a column (or set of columns) that uniquely identifies every row in a table. No two rows can have the same primary key value, and it can never be NULL.

The House Address Analogy

Think of a primary key like a home address. Every house on a street has a unique address. You would never have two houses with the address "123 Main Street" in the same city. And you would never have a house with no address at all — the mail carrier would have no way to find it.

A primary key works the same way. Every row gets a unique "address" so the database can find it instantly.

```
+-------------------------------------------------------------+
|                     PRIMARY KEY Rules                        |
+-------------------------------------------------------------+
|                                                             |
|  1. Every value must be UNIQUE — no duplicates              |
|  2. The value can NEVER be NULL                             |
|  3. A table can have only ONE primary key                   |
|  4. PostgreSQL automatically creates an index on it         |
|                                                             |
+-------------------------------------------------------------+
```

Creating a Table with a Primary Key

```sql
CREATE TABLE products (
    product_id  SERIAL PRIMARY KEY,
    name        VARCHAR(100) NOT NULL,
    price       DECIMAL(10, 2) NOT NULL,
    stock       INT NOT NULL DEFAULT 0
);
```

Line-by-line breakdown:

- `product_id SERIAL PRIMARY KEY` — SERIAL auto-generates increasing numbers (1, 2, 3...). PRIMARY KEY makes this column the unique identifier.
- `name VARCHAR(100) NOT NULL` — product name, up to 100 characters, cannot be empty.
- `price DECIMAL(10, 2) NOT NULL` — price with 2 decimal places, required.
- `stock INT NOT NULL DEFAULT 0` — stock count, defaults to 0 if not specified.

What Happens When You Violate a Primary Key

```sql
INSERT INTO products (product_id, name, price) VALUES
(1, 'Laptop', 999.99);

INSERT INTO products (product_id, name, price) VALUES
(1, 'Mouse', 29.99);
```

```
ERROR:  duplicate key value violates unique constraint "products_pkey"
DETAIL:  Key (product_id)=(1) already exists.
```

PostgreSQL refuses the second insert because product_id 1 already exists. The database protects your data even if your application has a bug.

You Cannot Insert NULL into a Primary Key

```sql
INSERT INTO products (product_id, name, price) VALUES
(NULL, 'Keyboard', 79.99);
```

```
ERROR:  null value in column "product_id" of relation "products"
        violates not-null constraint
```

FOREIGN KEY — The Relationship Enforcer

A FOREIGN KEY is a column in one table that points to the primary key of another table. It creates a link between two tables and ensures that the reference is always valid.

The Library Card Analogy

Think of a foreign key like a library card number on a loan record. When you check out a book, the library writes your card number on the loan slip. That card number must match a real person in the members list. If you tried to write card number 9999 on the loan slip but no member has that number, the librarian would refuse — that loan record would point to nobody.

```
+---------------------+         +----------------------+
|     customers       |         |       orders         |
+---------------------+         +----------------------+
| customer_id (PK)  <----------| customer_id (FK)     |
```

```
| first_name          |         | order_id (PK)        |
| last_name           |         | order_date           |
| email               |         | total                |
+---------------------+         +----------------------+

The FOREIGN KEY in orders points back to the PRIMARY KEY in customers.
An order CANNOT reference a customer_id that does not exist.
```

Creating a Table with a Foreign Key

```sql
CREATE TABLE orders (
    order_id    SERIAL PRIMARY KEY,
    customer_id INT NOT NULL,
    order_date  DATE NOT NULL DEFAULT CURRENT_DATE,
    total       DECIMAL(10, 2) NOT NULL DEFAULT 0.00,
    status      VARCHAR(20) NOT NULL DEFAULT 'pending',
    CONSTRAINT fk_orders_customer
        FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
);
```

Line-by-line breakdown:

- `order_id SERIAL PRIMARY KEY` — unique identifier for each order.
- `customer_id INT NOT NULL` — stores which customer placed this order.
- `CONSTRAINT fk_orders_customer` — gives the foreign key a name. Naming constraints makes it easier to find and manage them later.
- `FOREIGN KEY (customer_id)` — declares customer_id as a foreign key.
- `REFERENCES customers(customer_id)` — this foreign key must match a value in the customers table's customer_id column.

Inserting Valid and Invalid Data

```sql
-- This works: customer_id 1 (Alice) exists
INSERT INTO orders (customer_id, total)
VALUES (1, 149.99);

-- This works: customer_id 2 (Bob) exists
INSERT INTO orders (customer_id, total)
VALUES (2, 59.99);
```

```sql
-- This FAILS: customer_id 999 does not exist
INSERT INTO orders (customer_id, total)
VALUES (999, 29.99);
```

```
ERROR:  insert or update on table "orders" violates foreign key
        constraint "fk_orders_customer"
DETAIL:  Key (customer_id)=(999) is not present in table "customers".
```

PostgreSQL checks the customers table and confirms that customer 999 does not exist. The insert is rejected. Your orders table will never contain a reference to a phantom customer.

You Cannot Delete a Referenced Parent

```sql
-- Alice (customer_id 1) has an order. Try to delete her:
DELETE FROM customers WHERE customer_id = 1;
```

```
ERROR:  update or delete on table "customers" violates foreign key
        constraint "fk_orders_customer" on table "orders"
DETAIL:  Key (customer_id)=(1) is still referenced from table "orders".
```

By default, PostgreSQL uses the RESTRICT behavior — it refuses to delete a parent row if any child rows reference it. This prevents orphan records.

UNIQUE — No Duplicates Allowed

A UNIQUE constraint ensures that no two rows have the same value in a column. Unlike PRIMARY KEY, a table can have multiple UNIQUE constraints, and UNIQUE columns can contain NULL values (but only one NULL per column in most cases).

When to Use UNIQUE

```
+-------------------------------------------------------+
|              PRIMARY KEY vs UNIQUE                    |
+-------------------------------------------------------+
| Feature          | PRIMARY KEY   | UNIQUE            |
+------------------+---------------+-------------------+
| Duplicates       | Not allowed   | Not allowed       |
| NULL values      | Not allowed   | Allowed (one NULL) |
| Per table limit  | Only ONE      | As many as needed |
| Creates index    | Yes           | Yes               |
+------------------+---------------+-------------------+
```

Example: Unique Email Addresses

We already defined email as UNIQUE in our customers table. Let us test it:

```sql
INSERT INTO customers (first_name, last_name, email)
VALUES ('Dave', 'Wilson', 'alice@example.com');
```

```
ERROR:  duplicate key value violates unique constraint
        "customers_email_key"
DETAIL:  Key (email)=(alice@example.com) already exists.
```

Alice already has that email address. Dave cannot use the same one.

UNIQUE on Multiple Columns

You can also make a combination of columns unique. For example, a student can enroll in a course only once:

```sql
CREATE TABLE enrollments (
    enrollment_id SERIAL PRIMARY KEY,
    student_id    INT NOT NULL,
    course_id     INT NOT NULL,
    enrolled_at   DATE DEFAULT CURRENT_DATE,
    UNIQUE (student_id, course_id)
);
```

This means: the same student_id and course_id combination cannot appear twice. But the same student can enroll in different courses, and different students can enroll in the same course.

```sql
INSERT INTO enrollments (student_id, course_id) VALUES (1, 101);
INSERT INTO enrollments (student_id, course_id) VALUES (1, 102);  -- OK, different course
INSERT INTO enrollments (student_id, course_id) VALUES (2, 101);  -- OK, different student
INSERT INTO enrollments (student_id, course_id) VALUES (1, 101);  -- FAILS: duplicate
```

```
ERROR:  duplicate key value violates unique constraint
        "enrollments_student_id_course_id_key"
DETAIL:  Key (student_id, course_id)=(1, 101) already exists.
```

NOT NULL — No Empty Fields

NOT NULL is the simplest constraint. It says: "This column must always have a value. You cannot leave it blank."

The Registration Form Analogy

Think of NOT NULL like a required field on a registration form. The form will not submit unless you fill in your name and email. The phone number field might be optional, but the name is mandatory.

```sql
-- This FAILS: first_name is NOT NULL
INSERT INTO customers (first_name, last_name, email)
VALUES (NULL, 'Brown', 'nobody@example.com');
```

```
ERROR:  null value in column "first_name" of relation "customers"
        violates not-null constraint
DETAIL:  Failing row contains (null, Brown, nobody@example.com, null, ...).
```

When to Use NOT NULL

Use NOT NULL on any column that should always have a value. In practice, most columns should be NOT NULL. Only leave a column nullable when a missing value genuinely makes sense (like a middle name or a phone number).

```
+-----------------------------------------------+
|       Should this column be NOT NULL?         |
+-----------------------------------------------+
| Column           | NOT NULL? | Why?           |
+------------------+-----------+----------------+
| customer_name    | YES       | Must have name  |
| email            | YES       | Must have email |
| phone            | NO        | Optional info   |
| order_total      | YES       | Must have value |
| discount_code    | NO        | May not apply   |
| created_at       | YES       | Use DEFAULT     |
+------------------+-----------+----------------+
```

CHECK — Custom Validation Rules

A CHECK constraint lets you write custom rules that data must satisfy. Think of it like a bouncer at a club who checks whether you meet specific criteria before letting you in.

Basic CHECK Constraints

```sql
CREATE TABLE products_v2 (
    product_id   SERIAL PRIMARY KEY,
    name         VARCHAR(100) NOT NULL,
    price        DECIMAL(10, 2) NOT NULL,
    stock        INT NOT NULL DEFAULT 0,
    weight_kg    DECIMAL(5, 2),
    CONSTRAINT chk_price_positive CHECK (price > 0),
    CONSTRAINT chk_stock_not_negative CHECK (stock >= 0),
    CONSTRAINT chk_weight_reasonable CHECK (weight_kg IS NULL OR weight_kg > 0)
);
```

Line-by-line breakdown:

- `CHECK (price > 0)` — price must be greater than zero. You cannot sell something for free or for a negative amount.
- `CHECK (stock >= 0)` — stock cannot go below zero. You cannot have negative inventory.
- `CHECK (weight_kg IS NULL OR weight_kg > 0)` — weight is optional (can be NULL), but if provided, it must be positive.

Testing CHECK Constraints

```sql
-- This FAILS: price must be positive
INSERT INTO products_v2 (name, price, stock)
VALUES ('Free Item', 0, 10);
```

```
ERROR:  new row for relation "products_v2" violates check constraint
        "chk_price_positive"
DETAIL:  Failing row contains (1, Free Item, 0, 10, null).
```

```sql
-- This FAILS: stock cannot be negative
INSERT INTO products_v2 (name, price, stock)
VALUES ('Laptop', 999.99, -5);
```

```
ERROR:  new row for relation "products_v2" violates check constraint
        "chk_stock_not_negative"
DETAIL:  Failing row contains (2, Laptop, 999.99, -5, null).
```

```sql
-- This WORKS: all rules are satisfied
INSERT INTO products_v2 (name, price, stock, weight_kg)
VALUES ('Laptop', 999.99, 50, 2.10);
```

CHECK with Multiple Conditions

You can combine conditions:

```sql
CREATE TABLE employees (
    employee_id  SERIAL PRIMARY KEY,
    name         VARCHAR(100) NOT NULL,
    salary       DECIMAL(10, 2) NOT NULL,
    hire_date    DATE NOT NULL DEFAULT CURRENT_DATE,
    age          INT NOT NULL,
    CONSTRAINT chk_salary_range CHECK (salary >= 30000 AND salary <= 500000),
    CONSTRAINT chk_age_valid CHECK (age >= 18 AND age <= 100)
);
```

This ensures salaries fall within a reasonable range and employees are between 18 and 100 years old.

DEFAULT — Automatic Values

A DEFAULT constraint provides a value automatically when you do not specify one during INSERT. Think of it like a form that pre-fills certain fields for you.

Common DEFAULT Patterns

```sql
CREATE TABLE articles (
    article_id   SERIAL PRIMARY KEY,
    title        VARCHAR(200) NOT NULL,
    content      TEXT NOT NULL,
    status       VARCHAR(20) NOT NULL DEFAULT 'draft',
    views        INT NOT NULL DEFAULT 0,
    is_published BOOLEAN NOT NULL DEFAULT FALSE,
    created_at   TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at   TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

Line-by-line breakdown:

- `DEFAULT 'draft'` — new articles start as drafts.
- `DEFAULT 0` — view count starts at zero.
- `DEFAULT FALSE` — articles are not published by default.
- `DEFAULT CURRENT_TIMESTAMP` — automatically records when the row was created.

Testing DEFAULT Values

```sql
INSERT INTO articles (title, content)
VALUES ('My First Post', 'Hello, world!');

SELECT article_id, title, status, views, is_published, created_at
FROM articles;
```

```
 article_id |     title     | status | views | is_published |         created_at
------------+---------------+--------+-------+--------------+----------------------------
          1 | My First Post | draft  |     0 | f            | 2025-01-15 10:30:00.000000
(1 row)
```

We only specified title and content. The database filled in status, views, is_published, and created_at automatically.

You Can Override DEFAULT Values

```sql
INSERT INTO articles (title, content, status, views)
VALUES ('Breaking News', 'Important story!', 'published', 500);
```

If you provide a value, the DEFAULT is ignored. DEFAULT only applies when you omit the column.

Composite Keys — Two Columns as One Key

Sometimes a single column is not enough to uniquely identify a row. You need two or more columns working together. This is called a composite key (or compound key).

The Concert Ticket Analogy

Think of concert seating. A seat is identified by its section AND seat number. Section "A" alone does not identify a seat. Seat number "12" alone does not identify a seat. But "Section A, Seat 12" uniquely identifies exactly one seat in the venue.

```
+-----------------------------------------------------+
|              Composite Primary Key                  |
+-----------------------------------------------------+
|                                                     |
| Section alone:  A, A, A, B, B, B  (not unique)    |
| Seat alone:     1, 2, 3, 1, 2, 3  (not unique)    |
| Section + Seat: A1, A2, A3, B1, B2, B3  (unique!) |
|                                                     |
+-----------------------------------------------------+
```

Creating a Table with a Composite Key

```sql
CREATE TABLE order_items (
    order_id   INT NOT NULL,
    product_id INT NOT NULL,
    quantity   INT NOT NULL,
    unit_price DECIMAL(10, 2) NOT NULL,
    PRIMARY KEY (order_id, product_id),
    CONSTRAINT fk_items_order
        FOREIGN KEY (order_id) REFERENCES orders(order_id),
    CONSTRAINT fk_items_product
        FOREIGN KEY (product_id) REFERENCES products(product_id),
    CONSTRAINT chk_quantity_positive CHECK (quantity > 0)
);
```

Line-by-line breakdown:

- `PRIMARY KEY (order_id, product_id)` — the combination of order and product is the unique identifier. One order can have many products, and one product can appear in many orders, but the same product cannot appear twice in the same order.
- Both columns also serve as foreign keys, linking to the orders and products tables.

Testing the Composite Key

```sql
-- Add products first
INSERT INTO products (name, price, stock) VALUES
('Laptop', 999.99, 50),
('Mouse', 29.99, 200),
('Keyboard', 79.99, 150);
```

```
-- Add items to order 1
INSERT INTO order_items (order_id, product_id, quantity, unit_price) VALUES
(1, 1, 1, 999.99),  -- 1 Laptop
(1, 2, 2, 29.99);   -- 2 Mice

-- This FAILS: same order_id + product_id combination
INSERT INTO order_items (order_id, product_id, quantity, unit_price) VALUES
(1, 1, 3, 999.99);  -- Laptop already in order 1
```

```
ERROR:  duplicate key value violates unique constraint "order_items_pkey"
DETAIL:  Key (order_id, product_id)=(1, 1) already exists.
```

```sql
-- This WORKS: same product, different order
INSERT INTO order_items (order_id, product_id, quantity, unit_price) VALUES
(2, 1, 1, 999.99);  -- Laptop in order 2 — perfectly fine
```

CASCADE Options — What Happens to Children?

When you delete or update a parent row, what should happen to the child rows that reference it? CASCADE options let you control this behavior.

The Family Tree Analogy

Imagine a company closes a department. What happens to the employees in that department?

- CASCADE: All employees are also removed (deleted automatically).
- RESTRICT: The company cannot close the department until all employees are reassigned.
- SET NULL: Employees stay but their department field becomes NULL (unassigned).
- SET DEFAULT: Employees are moved to a default department (like "General").

```
+-------------------------------------------------------------------+
|                   CASCADE Options Summary                         |
+-------------------------------------------------------------------+
| Option       | What Happens to Child Rows                        |
+--------------+----------------------------------------------------+
| CASCADE      | Automatically deleted or updated along with       |
|              | the parent                                         |
| RESTRICT     | Prevent the parent from being deleted or updated  |
```

```
|              | (this is the default)                              |
| SET NULL     | Set the foreign key column to NULL                 |
| SET DEFAULT  | Set the foreign key column to its DEFAULT value    |
| NO ACTION    | Similar to RESTRICT but checked at end of          |
|              | transaction                                        |
+--------------+----------------------------------------------------+
```

ON DELETE CASCADE

When a parent row is deleted, automatically delete all child rows that reference it.

```sql
CREATE TABLE departments (
    dept_id   SERIAL PRIMARY KEY,
    dept_name VARCHAR(100) NOT NULL
);

CREATE TABLE staff (
    staff_id  SERIAL PRIMARY KEY,
    name      VARCHAR(100) NOT NULL,
    dept_id   INT,
    CONSTRAINT fk_staff_dept
        FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
        ON DELETE CASCADE
);

INSERT INTO departments (dept_name) VALUES ('Engineering'), ('Marketing');
INSERT INTO staff (name, dept_id) VALUES
('Alice', 1), ('Bob', 1), ('Carol', 2);
```

Now watch what happens when we delete the Engineering department:

```sql
DELETE FROM departments WHERE dept_id = 1;

SELECT * FROM staff;
```

```
 staff_id | name  | dept_id
----------+-------+---------
        3 | Carol |       2
(1 row)
```

Alice and Bob were automatically deleted because they belonged to the Engineering department. The cascade happened automatically — no extra DELETE statement was needed.

ON DELETE SET NULL

When a parent row is deleted, set the foreign key to NULL instead of deleting the child rows.

```sql
CREATE TABLE staff_v2 (
    staff_id  SERIAL PRIMARY KEY,
    name      VARCHAR(100) NOT NULL,
    dept_id   INT,
    CONSTRAINT fk_staff_dept_v2
        FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
        ON DELETE SET NULL
);

INSERT INTO departments (dept_name) VALUES ('Engineering');
INSERT INTO staff_v2 (name, dept_id) VALUES
('Dave', 3), ('Eve', 3);

DELETE FROM departments WHERE dept_id = 3;

SELECT * FROM staff_v2;
```

```
 staff_id | name | dept_id
----------+------+---------
        1 | Dave |    NULL
        2 | Eve  |    NULL
(2 rows)
```

Dave and Eve still exist, but their department is now NULL. They are "unassigned" rather than deleted.

ON DELETE SET DEFAULT

When a parent row is deleted, set the foreign key to its DEFAULT value.

```sql
CREATE TABLE staff_v3 (
    staff_id  SERIAL PRIMARY KEY,
    name      VARCHAR(100) NOT NULL,
    dept_id   INT DEFAULT 1,
    CONSTRAINT fk_staff_dept_v3
        FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
        ON DELETE SET DEFAULT
);
```

If department 1 is the "General" department, then any employee whose department is deleted gets reassigned to "General."

ON UPDATE CASCADE

When a parent row's primary key is updated, automatically update all child rows to match.

```sql
CREATE TABLE categories (
    category_code VARCHAR(10) PRIMARY KEY,
    category_name VARCHAR(100) NOT NULL
);

CREATE TABLE items (
    item_id        SERIAL PRIMARY KEY,
    item_name      VARCHAR(100) NOT NULL,
    category_code  VARCHAR(10),
    CONSTRAINT fk_items_category
        FOREIGN KEY (category_code) REFERENCES categories(category_code)
        ON UPDATE CASCADE
);

INSERT INTO categories VALUES ('ELEC', 'Electronics');
INSERT INTO items (item_name, category_code) VALUES ('Laptop', 'ELEC');
```

Now update the category code:

```sql
UPDATE categories SET category_code = 'ELECTRONICS' WHERE category_code = 'ELEC';

SELECT * FROM items;
```

```
 item_id | item_name | category_code
---------+-----------+---------------
       1 | Laptop    | ELECTRONICS
(1 row)
```

The items table automatically updated to match the new category code.

ON DELETE RESTRICT (The Default)

RESTRICT prevents deletion if any child rows exist. This is the safest option and the default in PostgreSQL.

```sql
-- With the default RESTRICT behavior:
DELETE FROM customers WHERE customer_id = 1;
```

```
ERROR:  update or delete on table "customers" violates foreign key
        constraint "fk_orders_customer" on table "orders"
```

You must delete all orders for customer 1 before you can delete the customer. This is intentional — it forces you to handle the dependent data explicitly.

How to Choose the Right Option

```
+-----------------------------------------------------------------+
|            Choosing the Right CASCADE Option                    |
+-----------------------------------------------------------------+
| Scenario                           | Use                        |
+------------------------------------+----------------------------+
| Deleting a user should delete      | ON DELETE CASCADE          |
| their posts                        |                            |
|                                    |                            |
| Deleting a manager should NOT      | ON DELETE SET NULL         |
| delete employees, just unassign    |                            |
|                                    |                            |
| Renaming a category code should    | ON UPDATE CASCADE          |
| update all products                |                            |
|                                    |                            |
| Deleting a category should be      | ON DELETE RESTRICT         |
| blocked if products exist          | (default)                  |
|                                    |                            |
| Deleting a team should move        | ON DELETE SET DEFAULT      |
| members to a default team          |                            |
+------------------------------------+----------------------------+
```

Adding and Dropping Constraints with ALTER TABLE

You do not always know every constraint you need when you first create a table. ALTER TABLE lets you add, modify, or remove constraints later.

Adding a NOT NULL Constraint

```sql
-- Make the phone column required
ALTER TABLE customers
ALTER COLUMN phone SET NOT NULL;
```

This will fail if any existing rows have NULL in the phone column. You would need to update those rows first:

```sql
-- First, fill in missing values
UPDATE customers SET phone = 'unknown' WHERE phone IS NULL;
```

```
-- Now add the constraint
ALTER TABLE customers
ALTER COLUMN phone SET NOT NULL;
```

Dropping a NOT NULL Constraint

```sql
ALTER TABLE customers
ALTER COLUMN phone DROP NOT NULL;
```

Adding a UNIQUE Constraint

```sql
ALTER TABLE customers
ADD CONSTRAINT uq_customers_phone UNIQUE (phone);
```

Adding a CHECK Constraint

```sql
ALTER TABLE orders
ADD CONSTRAINT chk_total_not_negative CHECK (total >= 0);
```

Adding a FOREIGN KEY Constraint

```sql
ALTER TABLE orders
ADD CONSTRAINT fk_orders_customer
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
    ON DELETE RESTRICT;
```

Dropping Any Named Constraint

```sql
ALTER TABLE orders
DROP CONSTRAINT chk_total_not_negative;
```

Adding a DEFAULT Value

```sql
ALTER TABLE customers
ALTER COLUMN phone SET DEFAULT 'Not provided';
```

Dropping a DEFAULT Value

```sql
ALTER TABLE customers
ALTER COLUMN phone DROP DEFAULT;
```

Viewing All Constraints on a Table

In psql, you can see a table's constraints with:

```sql
\d customers
```

Or query the information schema:

```sql
SELECT constraint_name, constraint_type
FROM information_schema.table_constraints
WHERE table_name = 'customers';
```

```
       constraint_name       | constraint_type
-----------------------------+-----------------
 customers_pkey              | PRIMARY KEY
 customers_email_key         | UNIQUE
 fk_orders_customer          | FOREIGN KEY
(3 rows)
```

Practical: Building a Fully Constrained E-Commerce Schema

Let us put everything together and build a real e-commerce database with proper constraints. This is what a professional schema looks like.

```
+-------------------+       +-------------------+       +-------------------+
|    customers      |       |     orders        |       |   order_items     |
+-------------------+       +-------------------+       +-------------------+
| customer_id (PK)  |<------| customer_id (FK)  |       | order_id (FK) (PK)|
| first_name        |       | order_id (PK)     |<------| product_id(FK)(PK)|
| last_name         |       | order_date        |       | quantity          |
| email (UNIQUE)    |       | total             |       | unit_price        |
| phone             |       | status            |       +-------------------+
| created_at        |       | shipping_address  |              |
+-------------------+       +-------------------+              |
                                                                  |
+-------------------+       +-------------------+              |
|   categories      |       |    products       |--------------+
+-------------------+       +-------------------+
| category_id (PK)  |<------| category_id (FK)  |
| name (UNIQUE)     |       | product_id (PK)   |
| description       |       | name              |
+-------------------+       | price             |
                             | stock             |
                             | is_active         |
                             +-------------------+
```

The Complete Schema

```sql
-- Drop tables if they exist (in reverse dependency order)
DROP TABLE IF EXISTS ecom_order_items;
DROP TABLE IF EXISTS ecom_orders;
DROP TABLE IF EXISTS ecom_products;
DROP TABLE IF EXISTS ecom_categories;
DROP TABLE IF EXISTS ecom_customers;

-- 1. Customers table
CREATE TABLE ecom_customers (
    customer_id    SERIAL PRIMARY KEY,
    first_name     VARCHAR(50) NOT NULL,
    last_name      VARCHAR(50) NOT NULL,
    email          VARCHAR(100) NOT NULL,
    phone          VARCHAR(20),
    date_of_birth  DATE,
    created_at     TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,

    -- Email must be unique across all customers
    CONSTRAINT uq_ecom_email UNIQUE (email),

    -- Age must be at least 13 if provided
    CONSTRAINT chk_ecom_age CHECK (
        date_of_birth IS NULL
        OR date_of_birth <= CURRENT_DATE - INTERVAL '13 years'
    )
);
```

```
-- 2. Categories table
CREATE TABLE ecom_categories (
    category_id   SERIAL PRIMARY KEY,
    name          VARCHAR(100) NOT NULL,
    description   TEXT,

    CONSTRAINT uq_ecom_category_name UNIQUE (name)
);

-- 3. Products table
CREATE TABLE ecom_products (
    product_id    SERIAL PRIMARY KEY,
    name          VARCHAR(200) NOT NULL,
    description   TEXT,
    price         DECIMAL(10, 2) NOT NULL,
    stock         INT NOT NULL DEFAULT 0,
    category_id   INT,
    is_active     BOOLEAN NOT NULL DEFAULT TRUE,
    created_at    TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,

    -- Price must be positive
    CONSTRAINT chk_ecom_price CHECK (price > 0),

    -- Stock cannot be negative
    CONSTRAINT chk_ecom_stock CHECK (stock >= 0),

    -- Category must exist (SET NULL if category is deleted)
    CONSTRAINT fk_ecom_product_category
        FOREIGN KEY (category_id) REFERENCES ecom_categories(category_id)
        ON DELETE SET NULL
        ON UPDATE CASCADE
);

-- 4. Orders table
CREATE TABLE ecom_orders (
    order_id          SERIAL PRIMARY KEY,
    customer_id       INT NOT NULL,
    order_date        TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    total             DECIMAL(10, 2) NOT NULL DEFAULT 0.00,
    status            VARCHAR(20) NOT NULL DEFAULT 'pending',
    shipping_address  TEXT NOT NULL,

    -- Total cannot be negative
    CONSTRAINT chk_ecom_total CHECK (total >= 0),

    -- Status must be one of these values
    CONSTRAINT chk_ecom_status CHECK (
        status IN ('pending', 'processing', 'shipped', 'delivered', 'cancelled')
    ),

    -- Customer must exist (RESTRICT delete if they have orders)
    CONSTRAINT fk_ecom_order_customer
        FOREIGN KEY (customer_id) REFERENCES ecom_customers(customer_id)
        ON DELETE RESTRICT
```

```
);

-- 5. Order Items table (composite primary key)
CREATE TABLE ecom_order_items (
    order_id    INT NOT NULL,
    product_id  INT NOT NULL,
    quantity    INT NOT NULL,
    unit_price  DECIMAL(10, 2) NOT NULL,

    -- Composite primary key: one product per order line
    PRIMARY KEY (order_id, product_id),

    -- Quantity must be at least 1
    CONSTRAINT chk_ecom_item_qty CHECK (quantity >= 1),

    -- Unit price must be positive
    CONSTRAINT chk_ecom_item_price CHECK (unit_price > 0),

    -- Order must exist (CASCADE delete: if order is deleted, items go too)
    CONSTRAINT fk_ecom_item_order
        FOREIGN KEY (order_id) REFERENCES ecom_orders(order_id)
        ON DELETE CASCADE,

    -- Product must exist (RESTRICT: cannot delete product if it is in orders)
    CONSTRAINT fk_ecom_item_product
        FOREIGN KEY (product_id) REFERENCES ecom_products(product_id)
        ON DELETE RESTRICT
);
```

Inserting Sample Data

```sql
-- Insert categories
INSERT INTO ecom_categories (name, description) VALUES
('Electronics', 'Gadgets and devices'),
('Clothing',    'Apparel and accessories'),
('Books',       'Physical and digital books');

-- Insert products
INSERT INTO ecom_products (name, price, stock, category_id) VALUES
('Laptop Pro 15',    1299.99, 50, 1),
('Wireless Mouse',   29.99,  200, 1),
('SQL Textbook',     49.99,  100, 3),
('Winter Jacket',    89.99,  75,  2);

-- Insert customers
INSERT INTO ecom_customers (first_name, last_name, email, phone) VALUES
('Alice', 'Johnson', 'alice.j@example.com', '555-0101'),
('Bob',   'Smith',   'bob.s@example.com',   '555-0102');

-- Insert an order for Alice
```

```
INSERT INTO ecom_orders (customer_id, total, status, shipping_address) VALUES
(1, 1359.97, 'processing', '123 Main St, Springfield');

-- Insert order items
INSERT INTO ecom_order_items (order_id, product_id, quantity, unit_price) VALUES
(1, 1, 1, 1299.99),  -- 1 Laptop
(1, 2, 2, 29.99);    -- 2 Mice
```

Testing the Constraints

```sql
-- Test 1: Cannot add product with negative price
INSERT INTO ecom_products (name, price, stock, category_id)
VALUES ('Bad Product', -10.00, 5, 1);
-- ERROR: violates check constraint "chk_ecom_price"

-- Test 2: Cannot add order with invalid status
INSERT INTO ecom_orders (customer_id, total, status, shipping_address)
VALUES (1, 50.00, 'unknown', '456 Oak Ave');
-- ERROR: violates check constraint "chk_ecom_status"

-- Test 3: Cannot delete customer who has orders
DELETE FROM ecom_customers WHERE customer_id = 1;
-- ERROR: violates foreign key constraint "fk_ecom_order_customer"

-- Test 4: Deleting an order cascades to order items
DELETE FROM ecom_orders WHERE order_id = 1;

SELECT * FROM ecom_order_items WHERE order_id = 1;
```

```
 order_id | product_id | quantity | unit_price
----------+------------+----------+------------
(0 rows)
```

The order items were automatically deleted along with the order, thanks to ON DELETE CASCADE.

```sql
-- Test 5: Cannot delete a product that is referenced in order items
-- (First, let us re-create the order and items)
INSERT INTO ecom_orders (customer_id, total, status, shipping_address)
VALUES (1, 29.99, 'pending', '123 Main St, Springfield');

INSERT INTO ecom_order_items (order_id, product_id, quantity, unit_price)
VALUES (2, 2, 1, 29.99);

DELETE FROM ecom_products WHERE product_id = 2;
-- ERROR: violates foreign key constraint "fk_ecom_item_product"
```

```sql
-- Test 6: Deleting a category sets product's category_id to NULL
DELETE FROM ecom_categories WHERE category_id = 2;

SELECT product_id, name, category_id
FROM ecom_products
WHERE name = 'Winter Jacket';
```

```
 product_id |     name      | category_id
------------+---------------+-------------
          4 | Winter Jacket |        NULL
(1 row)
```

The Winter Jacket's category became NULL instead of being deleted, thanks to ON DELETE SET NULL.

Common Mistakes

Mistake 1: Forgetting NOT NULL on Foreign Key Columns

```sql
-- BAD: customer_id can be NULL
CREATE TABLE bad_orders (
    order_id    SERIAL PRIMARY KEY,
    customer_id INT,  -- Should be NOT NULL!
    total       DECIMAL(10,2)
);

-- This creates an order with no customer — an orphan!
INSERT INTO bad_orders (total) VALUES (99.99);
```

The foreign key only checks that the value matches a parent row if a value is provided. If the column allows NULL, you can have orders that belong to nobody. Always add NOT NULL to foreign key columns unless you specifically want to allow "unassigned" rows.

Mistake 2: Naming Constraints Poorly (or Not at All)

```sql
-- BAD: auto-generated name like "orders_check" — hard to find later
CREATE TABLE bad_table (
    age INT CHECK (age >= 0)
);
```

```
-- GOOD: descriptive name tells you what the constraint checks
CREATE TABLE good_table (
    age INT,
    CONSTRAINT chk_age_not_negative CHECK (age >= 0)
);
```

When a constraint violation error appears, a descriptive name tells you immediately what went wrong. Auto-generated names like "table_column_check" are cryptic.

Mistake 3: Using CASCADE Everywhere Without Thinking

```sql
-- DANGEROUS: Deleting a customer deletes all their orders
CONSTRAINT fk_order_customer
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
    ON DELETE CASCADE
```

CASCADE is powerful but dangerous. Accidentally deleting a customer would wipe out their entire order history. Use CASCADE only when the child data truly has no meaning without the parent. Order items without an order make no sense (use CASCADE). But orders without a customer might still be needed for accounting (use RESTRICT).

Mistake 4: Adding Constraints After Data Exists Without Checking

```sql
-- You have existing data with some NULL phone numbers
ALTER TABLE customers ALTER COLUMN phone SET NOT NULL;
-- ERROR: column "phone" of relation "customers" contains null values
```

Always check existing data before adding constraints:

```sql
-- Check for violations first
SELECT COUNT(*) FROM customers WHERE phone IS NULL;

-- Fix the data
UPDATE customers SET phone = 'Not provided' WHERE phone IS NULL;

-- Now add the constraint
ALTER TABLE customers ALTER COLUMN phone SET NOT NULL;
```

Best Practices

1. Always use PRIMARY KEY on every table. Every table needs a unique identifier. Use SERIAL or UUID for auto-generated keys.
2. Name your constraints explicitly. Use a consistent naming convention like `chk_tablename_rule`, `fk_tablename_reference`, `uq_tablename_column`. This makes errors and maintenance much easier.
3. Default to NOT NULL. Start with NOT NULL on every column, then only remove it if you have a genuine reason to allow NULL values.
4. Use CHECK constraints for business rules. Prices should be positive, quantities should be at least 1, status values should be from a defined list. Enforce these at the database level, not just in your application code.
5. Choose CASCADE options carefully. Use RESTRICT (the default) unless you have a specific reason for CASCADE, SET NULL, or SET DEFAULT. RESTRICT is the safest option.
6. Add constraints early. It is much easier to add constraints when you create the table than to retrofit them onto a table with existing data.
7. Use DEFAULT values for columns that have an obvious initial state. Status columns, timestamps, boolean flags, and counters are great candidates for DEFAULT.

Quick Summary

```
+-------------------+-----------------------------------------------------+
| Constraint        | What It Does                                        |
+-------------------+-----------------------------------------------------+
| PRIMARY KEY       | Unique identifier for each row (no NULLs,           |
|                   | no duplicates, one per table)                       |
| FOREIGN KEY       | Links to another table's primary key, ensuring      |
|                   | references are valid                                |
| UNIQUE            | Prevents duplicate values (allows one NULL)         |
| NOT NULL          | Column must always have a value                     |
| CHECK             | Custom validation rule (e.g., price > 0)            |
| DEFAULT           | Auto-fills a value when none is provided            |
| Composite Key     | Two or more columns together form the primary key   |
| ON DELETE CASCADE | Automatically deletes child rows                    |
| ON DELETE RESTRICT| Blocks deletion if child rows exist (default)       |
| ON DELETE SET NULL| Sets foreign key to NULL in child rows              |
| ON UPDATE CASCADE | Updates child rows when parent key changes          |
+-------------------+-----------------------------------------------------+
```

Key Points

- Constraints enforce rules at the database level so bad data can never sneak in, regardless of which application or user is writing data.
- PRIMARY KEY uniquely identifies each row. Every table should have one.
- FOREIGN KEY creates a relationship between two tables and prevents orphan records.
- UNIQUE prevents duplicate values in a column. A table can have multiple UNIQUE constraints.
- NOT NULL ensures a column always has a value. Most columns should be NOT NULL.
- CHECK lets you write custom rules like "price must be positive" or "status must be one of these values."
- DEFAULT provides automatic values for columns when no value is specified.
- Composite keys use two or more columns together as the primary key.
- CASCADE options control what happens to child rows when a parent is deleted or updated.
- Use ALTER TABLE to add or remove constraints after a table is created. Always check existing data before adding new constraints.

Practice Questions

1. What is the difference between PRIMARY KEY and UNIQUE? Can a table have more than one of each?
2. If you have a foreign key with ON DELETE SET NULL and the foreign key column is defined as NOT NULL, what happens when you try to delete the parent row?
3. You have a products table with a CHECK constraint `CHECK (price > 0)`. Can you INSERT a row where price is NULL? Why or why not?
4. What is the difference between ON DELETE RESTRICT and ON DELETE NO ACTION in PostgreSQL?
5. Why should you name your constraints explicitly instead of letting PostgreSQL auto-generate names?

Exercises

Exercise 1: Build a School Database

Create a schema with these tables and constraints:

- students: student_id (PK), first_name (NOT NULL), last_name (NOT NULL), email (UNIQUE, NOT NULL), enrollment_date (DEFAULT today), gpa (CHECK between 0.0 and 4.0)
- courses: course_id (PK), course_name (NOT NULL, UNIQUE), credits (CHECK between 1 and 6), department (NOT NULL)
- enrollments: student_id (FK), course_id (FK), composite PK, grade (CHECK: A, B, C, D, F, or NULL), enrolled_at (DEFAULT now)

Test your constraints by attempting to insert invalid data.

Exercise 2: Constraint Migration

You have an existing table with no constraints:

```sql
CREATE TABLE legacy_users (
    id INT,
    username VARCHAR(50),
    email VARCHAR(100),
    age INT,
    status VARCHAR(20)
);
```

Write ALTER TABLE statements to add: a PRIMARY KEY on id, NOT NULL on username and email, UNIQUE on email, a CHECK that age is between 13 and 150, a CHECK that status is one of 'active', 'inactive', or 'banned', and a DEFAULT of 'active' for status.

Exercise 3: Design a Booking System

Design a hotel booking schema with proper constraints and CASCADE options. You need tables for hotels, rooms, guests, and bookings. Think carefully about what should happen when a hotel is deleted (should rooms be deleted too?), when a guest is deleted (should bookings be kept for records?), and what CHECK constraints you need (check-out after check-in, price positive, room capacity at least 1).

What Is Next?

In Chapter 25, you will learn about Transactions — how to group multiple SQL statements into a single all-or-nothing operation. Transactions work hand-in-hand with constraints: constraints ensure individual rows are valid, while transactions ensure that multi-step operations either succeed completely or fail completely. Together, they form the backbone of reliable database applications.

Chapter 25: Transactions — All or Nothing

What You Will Learn

In this chapter, you will learn:

- What a transaction is and why it matters
- The ACID properties that keep your data safe
- How to use BEGIN, COMMIT, and ROLLBACK
- How to create SAVEPOINTs for partial rollbacks
- What isolation levels are and when to change them
- What deadlocks are and how to avoid them
- How to build a safe bank transfer and order placement system

Why This Chapter Matters

Imagine you are at an ATM. You want to transfer 500 dollars from your savings account to your checking account. The ATM deducts 500 from savings, but right at that moment the power goes out. Your savings account is down 500 dollars, but your checking account never got the money. You just lost 500 dollars.

This is the exact problem transactions solve. A transaction wraps multiple operations into a single unit of work. Either everything succeeds together, or everything fails together. There is no in-between state where your money vanishes.

Every real application needs transactions. Online stores processing orders, banks transferring money, hospitals updating patient records — any time you need multiple changes to happen as one atomic unit, you need a transaction.

Setting Up Our Practice Tables

Let us create tables we will use throughout this chapter:

```sql
CREATE TABLE accounts (
    account_id   SERIAL PRIMARY KEY,
    owner_name   VARCHAR(100) NOT NULL,
    balance      DECIMAL(12, 2) NOT NULL DEFAULT 0.00,
    account_type VARCHAR(20) NOT NULL
);

INSERT INTO accounts (owner_name, balance, account_type) VALUES
('Alice', 5000.00, 'checking'),
('Alice', 10000.00, 'savings'),
('Bob',   3000.00, 'checking'),
('Bob',   7500.00, 'savings');

CREATE TABLE orders (
    order_id    SERIAL PRIMARY KEY,
    customer_id INT NOT NULL,
    order_date  DATE NOT NULL DEFAULT CURRENT_DATE,
    total       DECIMAL(10, 2) NOT NULL,
    status      VARCHAR(20) NOT NULL DEFAULT 'pending'
);

CREATE TABLE order_items (
    item_id    SERIAL PRIMARY KEY,
    order_id   INT NOT NULL REFERENCES orders(order_id),
    product_id INT NOT NULL,
    quantity   INT NOT NULL,
    price      DECIMAL(10, 2) NOT NULL
);

CREATE TABLE inventory (
    product_id   SERIAL PRIMARY KEY,
    product_name VARCHAR(100) NOT NULL,
    stock        INT NOT NULL DEFAULT 0
);

INSERT INTO inventory (product_name, stock) VALUES
('Laptop', 50),
('Mouse', 200),
('Keyboard', 150);
```

What Is a Transaction?

A transaction is a group of SQL statements that execute as a single unit. Think of it like a package deal — you either get everything or nothing.

```
Without Transactions:              With Transactions:
+---------------------------+      +---------------------------+
| Step 1: Deduct from       |      | BEGIN                     |
|         savings   --> OK  |      |   Step 1: Deduct savings  |
| Step 2: Add to            |      |   Step 2: Add to checking |
|         checking --> FAIL |      | COMMIT (or ROLLBACK)      |
|                           |      |                           |
| Result: Money lost!       |      | Result: All or nothing!   |
+---------------------------+      +---------------------------+
```

The ATM Analogy

Think of a transaction like using an ATM:

1. You insert your card (BEGIN the transaction)
2. You request a withdrawal (execute SQL statements)
3. The ATM dispenses cash and updates your balance (COMMIT)
4. If anything goes wrong, the ATM cancels everything (ROLLBACK)

The ATM never gives you cash without updating your balance, and it never updates your balance without giving you cash. Both things happen together or neither happens.

ACID Properties

Every transaction follows four rules known as ACID. These rules guarantee your data stays consistent and safe.

```
+-----------------------------------------------------+
|                 ACID Properties                     |
+-----------------------------------------------------+
|                                                     |
|  A - Atomicity    : All or nothing                  |
|  C - Consistency  : Data stays valid                |
|  I - Isolation    : Transactions do not interfere   |
|  D - Durability   : Changes survive crashes         |
|                                                     |
+-----------------------------------------------------+
```

Atomicity — All or Nothing

Either every statement in the transaction succeeds, or none of them take effect. There is no partial completion.

Real-life analogy: Ordering a combo meal. You either get the burger, fries, and drink together, or you get nothing. The restaurant does not give you just the burger and forget the fries.

Consistency — Data Stays Valid

A transaction moves the database from one valid state to another valid state. It cannot violate any rules (constraints, foreign keys, check constraints).

Real-life analogy: A chess game. Every move must follow the rules. You cannot move a pawn backward. The board always stays in a valid state.

Isolation — Transactions Do Not Interfere

When two transactions run at the same time, they do not see each other's incomplete work. Each transaction acts as if it is the only one running.

Real-life analogy: Two people using different ATMs at the same time. Each person sees a consistent view of their account, not some half-updated version.

Durability — Changes Survive Crashes

Once a transaction is committed, the changes are permanent. Even if the server crashes one second later, the data is safe.

Real-life analogy: Once you sign a contract and both parties have copies, the agreement stands even if the office burns down.

BEGIN, COMMIT, and ROLLBACK

These three commands control transactions.

Starting a Transaction with BEGIN

```sql
BEGIN;
```

This tells PostgreSQL: "Everything I do from now on is part of one transaction. Do not make anything permanent until I say so."

Committing with COMMIT

```sql
BEGIN;

UPDATE accounts
SET balance = balance - 500
WHERE account_id = 2;

UPDATE accounts
SET balance = balance + 500
WHERE account_id = 1;

COMMIT;
```

COMMIT says: "Everything worked. Make all the changes permanent."

Let us verify:

```sql
SELECT account_id, owner_name, balance, account_type
FROM accounts
WHERE owner_name = 'Alice';
```

```
 account_id | owner_name | balance  | account_type
------------+------------+----------+--------------
          1 | Alice      |  5500.00 | checking
          2 | Alice      |  9500.00 | savings
(2 rows)
```

The 500 dollars moved from savings to checking. Both updates happened together.

Rolling Back with ROLLBACK

ROLLBACK says: "Something went wrong. Undo everything since BEGIN."

```sql
BEGIN;

UPDATE accounts
SET balance = balance - 100000
WHERE account_id = 3;

-- Oops! Bob only has 3000 in checking.
```

```
-- We should not allow this.

ROLLBACK;
```

After ROLLBACK, Bob's account is unchanged:

```sql
SELECT account_id, owner_name, balance, account_type
FROM accounts
WHERE account_id = 3;
```

```
 account_id | owner_name | balance | account_type
------------+------------+---------+--------------
          3 | Bob        | 3000.00 | checking
(1 row)
```

How It Flows

```
BEGIN
  |
  v
+------------------+
| Execute SQL      |
| statements       |
+------------------+
  |            |
  | Success    | Error
  v            v
COMMIT      ROLLBACK
(save)      (undo all)
```

A Practical Bank Transfer

Let us build a proper bank transfer that checks for sufficient funds:

```sql
BEGIN;

-- Step 1: Check if sender has enough money
-- (We read the balance first)
DO $$
```

```
DECLARE
    sender_balance DECIMAL(12,2);
BEGIN
    SELECT balance INTO sender_balance
    FROM accounts
    WHERE account_id = 2;  -- Alice's savings

    IF sender_balance < 2000.00 THEN
        RAISE EXCEPTION 'Insufficient funds. Balance: %', sender_balance;
    END IF;

    -- Step 2: Deduct from sender
    UPDATE accounts
    SET balance = balance - 2000.00
    WHERE account_id = 2;

    -- Step 3: Add to receiver
    UPDATE accounts
    SET balance = balance + 2000.00
    WHERE account_id = 3;

    RAISE NOTICE 'Transfer complete!';
END $$;

COMMIT;
```

Let us check the results:

```sql
SELECT account_id, owner_name, balance, account_type
FROM accounts
WHERE account_id IN (2, 3);
```

```
 account_id | owner_name | balance | account_type
------------+------------+---------+--------------
          2 | Alice      | 7500.00 | savings
          3 | Bob        | 5000.00 | checking
(2 rows)
```

If the balance check had failed, the RAISE EXCEPTION would have caused PostgreSQL to automatically rollback the entire transaction.

SAVEPOINT — Partial Rollbacks

Sometimes you want to undo part of a transaction without losing everything. SAVEPOINTs let you create checkpoints within a transaction.

```
BEGIN
  |
  v
Statement 1  -----> saved
  |
  v
SAVEPOINT sp1
  |
  v
Statement 2  -----> can be undone
  |
  v
ROLLBACK TO sp1    (undoes Statement 2 only)
  |
  v
Statement 3  -----> saved
  |
  v
COMMIT             (saves Statement 1 and Statement 3)
```

SAVEPOINT in Action

```sql
BEGIN;

-- Insert a valid order
INSERT INTO orders (customer_id, total, status)
VALUES (1, 150.00, 'confirmed');

SAVEPOINT before_items;

-- Try to insert order items
INSERT INTO order_items (order_id, product_id, quantity, price)
VALUES (1, 1, 1, 100.00);

INSERT INTO order_items (order_id, product_id, quantity, price)
VALUES (1, 2, 5, 10.00);

-- Oops, something went wrong with this batch of items
-- Roll back to the savepoint (keeps the order, removes items)
ROLLBACK TO before_items;

-- Insert corrected items instead
INSERT INTO order_items (order_id, product_id, quantity, price)
VALUES (1, 1, 1, 100.00);

INSERT INTO order_items (order_id, product_id, quantity, price)
```

```
VALUES (1, 3, 2, 25.00);

COMMIT;
```

After this transaction:

- The order exists (it was before the savepoint)
- Only the corrected items exist (the first batch was rolled back)

Nested SAVEPOINTs

You can create multiple savepoints:

```sql
BEGIN;

INSERT INTO orders (customer_id, total) VALUES (2, 200.00);
SAVEPOINT sp1;

INSERT INTO order_items (order_id, product_id, quantity, price)
VALUES (2, 1, 1, 150.00);
SAVEPOINT sp2;

INSERT INTO order_items (order_id, product_id, quantity, price)
VALUES (2, 2, 10, 5.00);

-- Undo only the last insert
ROLLBACK TO sp2;

-- The order and first item still exist
COMMIT;
```

Releasing SAVEPOINTs

When you no longer need a savepoint, you can release it to free resources:

```sql
BEGIN;
SAVEPOINT my_savepoint;
-- ... do some work ...
RELEASE SAVEPOINT my_savepoint;
-- Now my_savepoint cannot be rolled back to
COMMIT;
```

Isolation Levels

When multiple transactions run at the same time, isolation levels control how much they can see of each other's work.

The Problem: Concurrent Access

Imagine two bank tellers processing transactions on the same account at the same time:

```
Teller 1 (Transaction A)         Teller 2 (Transaction B)
------------------------         ------------------------
Reads balance: $1000
                                  Reads balance: $1000
Deducts $200
Balance = $800
                                  Deducts $300
                                  Balance = $700
Commits ($800)
                                  Commits ($700)

Final balance: $700
Should be: $500  (1000 - 200 - 300)
```

This is called a "lost update" problem. Isolation levels prevent these kinds of issues.

PostgreSQL Isolation Levels

PostgreSQL supports four isolation levels, but two are most commonly used:

```
+---------------------+------------------+----------------+---------------+
| Isolation Level     | Dirty Read       | Non-Repeatable | Phantom Read  |
|                     |                  | Read           |               |
+---------------------+------------------+----------------+---------------+
| READ UNCOMMITTED*   | Not possible     | Possible       | Possible      |
| READ COMMITTED      | Not possible     | Possible       | Possible      |
| REPEATABLE READ     | Not possible     | Not possible   | Not possible  |
| SERIALIZABLE        | Not possible     | Not possible   | Not possible  |
+---------------------+------------------+----------------+---------------+

* In PostgreSQL, READ UNCOMMITTED behaves like READ COMMITTED.
```

Dirty Read: Reading data that another transaction has changed but not yet committed.

Non-Repeatable Read: Reading the same row twice and getting different values because another transaction modified it in between.

Phantom Read: Running the same query twice and getting different rows because another transaction inserted or deleted rows.

READ COMMITTED (Default)

This is PostgreSQL's default level. Each statement in a transaction sees only data that was committed before that statement began.

```sql
-- This is the default, but you can set it explicitly:
BEGIN;
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;

SELECT balance FROM accounts WHERE account_id = 1;
-- Returns the latest committed balance

-- If another transaction commits a change here,
-- the next SELECT will see the new value

SELECT balance FROM accounts WHERE account_id = 1;
-- Might return a different value!

COMMIT;
```

REPEATABLE READ

Once a transaction reads data, it always sees the same data, even if other transactions modify it.

```sql
BEGIN;
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;

SELECT balance FROM accounts WHERE account_id = 1;
-- Returns 5500.00

-- Even if another transaction changes this balance
-- and commits, this transaction still sees 5500.00

SELECT balance FROM accounts WHERE account_id = 1;
-- Still returns 5500.00

COMMIT;
```

SERIALIZABLE

The strictest level. Transactions behave as if they ran one after another, not concurrently. If a conflict is detected, PostgreSQL will abort one of the transactions.

```sql
BEGIN;
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;

-- All operations here are fully isolated
-- If a conflict is detected, PostgreSQL will raise:
-- ERROR: could not serialize access

COMMIT;
```

When to Use Each Level

```
+--------------------+----------------------------------------+
| Level              | Use When                               |
+--------------------+----------------------------------------+
| READ COMMITTED     | Most everyday operations               |
|                    | Simple INSERT/UPDATE/DELETE             |
|                    | Default — use unless you need more     |
+--------------------+----------------------------------------+
| REPEATABLE READ    | Reports that need consistent data      |
|                    | Reading the same data multiple times   |
+--------------------+----------------------------------------+
| SERIALIZABLE       | Financial calculations                 |
|                    | Inventory management                   |
|                    | Any time correctness is critical       |
+--------------------+----------------------------------------+
```

Deadlocks

A deadlock happens when two transactions are each waiting for the other to release a lock. Neither can proceed, so they are stuck forever.

The Deadlock Analogy

Imagine two cars meeting on a narrow one-lane bridge from opposite sides. Neither can move forward, and neither will back up. They are deadlocked.

```
Transaction A                    Transaction B
-------------                    -------------
Locks Row 1                      Locks Row 2
  |                                |
  v                                v
```

```
Wants Row 2 (locked by B)       Wants Row 1 (locked by A)
  |                                 |
  v                                 v
WAITING...                         WAITING...

  ==> DEADLOCK! Neither can proceed.
```

Deadlock Example

```sql
-- Transaction A (in one session):
BEGIN;
UPDATE accounts SET balance = balance - 100 WHERE account_id = 1;
-- Now locks row with account_id = 1

-- Transaction B (in another session):
BEGIN;
UPDATE accounts SET balance = balance - 50 WHERE account_id = 2;
-- Now locks row with account_id = 2

-- Transaction A tries:
UPDATE accounts SET balance = balance + 100 WHERE account_id = 2;
-- WAITS... because B has locked account_id = 2

-- Transaction B tries:
UPDATE accounts SET balance = balance + 50 WHERE account_id = 1;
-- WAITS... because A has locked account_id = 1

-- DEADLOCK DETECTED!
-- PostgreSQL automatically kills one transaction:
-- ERROR: deadlock detected
```

Preventing Deadlocks

Rule 1: Always lock rows in the same order.

```sql
-- GOOD: Both transactions lock accounts in order (1, then 2)
-- Transaction A:
BEGIN;
UPDATE accounts SET balance = balance - 100 WHERE account_id = 1;
UPDATE accounts SET balance = balance + 100 WHERE account_id = 2;
COMMIT;

-- Transaction B:
BEGIN;
UPDATE accounts SET balance = balance + 50 WHERE account_id = 1;
UPDATE accounts SET balance = balance - 50 WHERE account_id = 2;
COMMIT;
```

Rule 2: Keep transactions short.

```sql
-- BAD: Long transaction holds locks for too long
BEGIN;
UPDATE accounts SET balance = balance - 100 WHERE account_id = 1;
-- ... lots of slow processing ...
-- ... calculating reports ...
-- ... sending emails ...
UPDATE accounts SET balance = balance + 100 WHERE account_id = 2;
COMMIT;

-- GOOD: Do processing outside the transaction
-- Calculate everything first, then:
BEGIN;
UPDATE accounts SET balance = balance - 100 WHERE account_id = 1;
UPDATE accounts SET balance = balance + 100 WHERE account_id = 2;
COMMIT;
```

Rule 3: Use SELECT ... FOR UPDATE to explicitly lock rows you plan to modify.

```sql
BEGIN;

-- Lock both rows upfront in a consistent order
SELECT * FROM accounts
WHERE account_id IN (1, 2)
ORDER BY account_id
FOR UPDATE;

-- Now safe to update both
UPDATE accounts SET balance = balance - 100 WHERE account_id = 1;
UPDATE accounts SET balance = balance + 100 WHERE account_id = 2;

COMMIT;
```

Practical Example: Order Placement

Let us build a complete order placement system that uses transactions properly:

```sql
BEGIN;

-- Step 1: Check inventory for all items
DO $$
DECLARE
```

```
    laptop_stock INT;
    mouse_stock INT;
BEGIN
    -- Lock inventory rows to prevent race conditions
    SELECT stock INTO laptop_stock
    FROM inventory
    WHERE product_id = 1
    FOR UPDATE;

    SELECT stock INTO mouse_stock
    FROM inventory
    WHERE product_id = 2
    FOR UPDATE;

    -- Step 2: Verify sufficient stock
    IF laptop_stock < 1 THEN
        RAISE EXCEPTION 'Not enough laptops in stock. Available: %',
                        laptop_stock;
    END IF;

    IF mouse_stock < 2 THEN
        RAISE EXCEPTION 'Not enough mice in stock. Available: %',
                        mouse_stock;
    END IF;

    -- Step 3: Create the order
    INSERT INTO orders (customer_id, total, status)
    VALUES (1, 1020.00, 'confirmed');

    -- Step 4: Add order items
    INSERT INTO order_items (order_id, product_id, quantity, price)
    VALUES
        (currval('orders_order_id_seq'), 1, 1, 1000.00),
        (currval('orders_order_id_seq'), 2, 2, 10.00);

    -- Step 5: Reduce inventory
    UPDATE inventory SET stock = stock - 1 WHERE product_id = 1;
    UPDATE inventory SET stock = stock - 2 WHERE product_id = 2;

    RAISE NOTICE 'Order placed successfully!';
END $$;

COMMIT;
```

Let us verify everything worked:

```sql
SELECT * FROM orders WHERE customer_id = 1;
```

```
 order_id | customer_id | order_date |  total  |  status
----------+-------------+------------+---------+-----------
        1 |           1 | 2024-01-15 | 1020.00 | confirmed
```

```
(1 row)
```

```sql
SELECT * FROM order_items WHERE order_id = 1;
```

```
 item_id | order_id | product_id | quantity |  price
---------+----------+------------+----------+---------
       1 |        1 |          1 |        1 | 1000.00
       2 |        1 |          2 |        2 |   10.00
(2 rows)
```

```sql
SELECT product_id, product_name, stock FROM inventory;
```

```
 product_id | product_name | stock
------------+--------------+-------
          1 | Laptop       |    49
          2 | Mouse        |   198
          3 | Keyboard     |   150
(3 rows)
```

All three tables were updated together. If any step had failed, everything would have been rolled back — no orphaned orders, no incorrect inventory.

Auto-Commit Mode

By default, PostgreSQL runs in auto-commit mode. Every single statement is its own transaction:

```sql
-- These are actually two separate transactions:
UPDATE accounts SET balance = balance - 100 WHERE account_id = 1;
-- ^ automatically committed

UPDATE accounts SET balance = balance + 100 WHERE account_id = 2;
-- ^ automatically committed
```

If the second statement fails, the first one has already been committed. That is why you need explicit BEGIN/COMMIT when multiple statements must succeed together.

```
Auto-commit (default):           Explicit transaction:
+--------+    +--------+        +----------------------------+
| UPDATE | -> | UPDATE |        | BEGIN                      |
| (auto  |    | (auto  |        |   UPDATE accounts ...      |
| commit)|    | commit)|        |   UPDATE accounts ...      |
+--------+    +--------+        | COMMIT                     |
                                +----------------------------+
If #2 fails,    #1 is           If anything fails,
#1 already      already         everything is
committed       gone            rolled back
```

Common Mistakes

Mistake 1: Forgetting BEGIN

```sql
-- BAD: These are two separate auto-committed transactions
UPDATE accounts SET balance = balance - 500 WHERE account_id = 1;
UPDATE accounts SET balance = balance + 500 WHERE account_id = 2;

-- GOOD: Wrapped in a transaction
BEGIN;
UPDATE accounts SET balance = balance - 500 WHERE account_id = 1;
UPDATE accounts SET balance = balance + 500 WHERE account_id = 2;
COMMIT;
```

Mistake 2: Long-Running Transactions

```sql
-- BAD: Transaction holds locks while waiting for user input
BEGIN;
UPDATE accounts SET balance = balance - 100 WHERE account_id = 1;
-- ... wait for user confirmation for 5 minutes ...
COMMIT;

-- GOOD: Keep transactions short
-- Validate and prepare data first, then:
BEGIN;
UPDATE accounts SET balance = balance - 100 WHERE account_id = 1;
UPDATE accounts SET balance = balance + 100 WHERE account_id = 2;
COMMIT;
```

Mistake 3: Not Handling Errors

```sql
-- BAD: If the second UPDATE fails, the first is already done
BEGIN;
UPDATE accounts SET balance = balance - 500 WHERE account_id = 1;
UPDATE accounts SET balance = balance + 500 WHERE account_id = 999;
-- account_id 999 does not exist — but no error raised!
COMMIT;

-- GOOD: Check that updates actually affected rows
BEGIN;

DO $$
DECLARE
    rows_affected INT;
BEGIN
    UPDATE accounts SET balance = balance - 500 WHERE account_id = 1;
    GET DIAGNOSTICS rows_affected = ROW_COUNT;
    IF rows_affected = 0 THEN
        RAISE EXCEPTION 'Source account not found';
    END IF;

    UPDATE accounts SET balance = balance + 500 WHERE account_id = 2;
    GET DIAGNOSTICS rows_affected = ROW_COUNT;
    IF rows_affected = 0 THEN
        RAISE EXCEPTION 'Destination account not found';
    END IF;
END $$;

COMMIT;
```

Mistake 4: Catching Exceptions and Continuing

```sql
-- BAD: Swallowing errors inside a transaction
BEGIN;
-- After an error in PostgreSQL, the transaction is "aborted"
-- You cannot run more statements — you must ROLLBACK
-- Trying to continue will give:
-- ERROR: current transaction is aborted, commands ignored until
--        end of transaction block
```

Best Practices

1. Keep transactions as short as possible. Do validation and calculations before BEGIN. Only put the actual data modifications inside the transaction.

2. Always use transactions for multi-table operations. Any time you modify more than one table, wrap it in a transaction.
3. Lock rows in a consistent order. If transaction A locks accounts in the order (1, 2, 3), then transaction B should also lock them in the order (1, 2, 3), not (3, 2, 1). This prevents deadlocks.
4. Use SELECT ... FOR UPDATE when you read data that you plan to modify. This prevents other transactions from changing the data between your read and your write.
5. Handle errors properly. Check row counts, use exception handling, and always have a plan for what happens when things go wrong.
6. Use the right isolation level. READ COMMITTED is fine for most operations. Use SERIALIZABLE for critical financial operations where correctness matters more than performance.
7. Monitor for deadlocks. Check PostgreSQL logs regularly. Deadlocks are a sign that your transactions need redesigning.

Quick Summary

```
+------------------+----------------------------------------------+
| Concept          | What It Does                                 |
+------------------+----------------------------------------------+
| BEGIN            | Starts a transaction                         |
| COMMIT           | Makes all changes permanent                  |
| ROLLBACK         | Undoes all changes since BEGIN                |
| SAVEPOINT        | Creates a checkpoint for partial rollback     |
| ROLLBACK TO      | Undoes changes back to a savepoint            |
| RELEASE SAVEPOINT| Removes a savepoint (frees resources)        |
| READ COMMITTED   | Default — sees only committed data           |
| REPEATABLE READ  | Same data throughout the transaction          |
| SERIALIZABLE     | Strictest — transactions run "one at a time" |
| FOR UPDATE       | Locks rows you plan to modify                |
+------------------+----------------------------------------------+
```

Key Points

- A transaction groups multiple SQL statements into a single all-or-nothing unit.
- ACID properties (Atomicity, Consistency, Isolation, Durability) guarantee data integrity.
- BEGIN starts a transaction. COMMIT saves changes. ROLLBACK undoes everything.

- SAVEPOINTs let you undo part of a transaction without losing all of it.
- READ COMMITTED is the default isolation level in PostgreSQL and is suitable for most operations.
- SERIALIZABLE provides the strongest guarantees but may cause transactions to fail and retry.
- Deadlocks happen when two transactions wait for each other. Prevent them by locking rows in a consistent order.
- Keep transactions short to minimize lock contention.
- Always use transactions when modifying multiple related tables.

Practice Questions

1. What happens if you run an UPDATE statement without BEGIN? Does it still use a transaction?
2. You have a transaction with three INSERT statements. The second INSERT violates a UNIQUE constraint. What happens to the first and third INSERTs?
3. What is the difference between ROLLBACK and ROLLBACK TO SAVEPOINT? When would you use each?
4. Two transactions are running at READ COMMITTED isolation level. Transaction A reads a row, then Transaction B updates and commits that row, then Transaction A reads the same row again. Will Transaction A see the change?
5. Why is it important to lock rows in a consistent order across all transactions?

Exercises

Exercise 1: Safe Transfer Function

Write a transaction that transfers money between two accounts. The transaction should:

- Check that both accounts exist
- Verify the sender has sufficient funds
- Perform the transfer
- Roll back if anything goes wrong

Hint: Use a DO block with exception handling and GET DIAGNOSTICS to check row counts.

Exercise 2: Order with Rollback

Write a transaction that:

1. Creates a new order
2. Adds three items to the order
3. Uses a SAVEPOINT after the second item
4. Attempts to add a third item that fails (simulate with a bad product_id)
5. Rolls back to the savepoint
6. Adds a valid third item instead
7. Commits the transaction

Exercise 3: Concurrent Simulation

Open two psql sessions. In each session, start a transaction and try to update the same row. Observe what happens:

- What does the second session see while the first transaction is still open?
- What happens when the first session commits?
- Try the same experiment with REPEATABLE READ isolation level.

What Is Next?

Now that you understand how to keep your data safe with transactions, you are ready to learn about window functions in Chapter 26. Window functions let you perform calculations across groups of rows without collapsing them — like running totals, rankings, and comparisons to previous rows. They are one of SQL's most powerful features for data analysis.

Chapter 26: Window Functions — Calculations Across Rows

What You Will Learn

In this chapter, you will learn:

- What window functions are and how they differ from GROUP BY
- How to use the OVER clause to define a window
- How to partition data with PARTITION BY
- How to rank rows with ROW_NUMBER, RANK, and DENSE_RANK
- How to access previous and next rows with LAG and LEAD
- How to calculate running totals with SUM OVER
- How to use NTILE to divide rows into buckets
- How to control the window frame with ROWS BETWEEN

Why This Chapter Matters

Imagine you have a spreadsheet of monthly sales. You want to add a column showing each month's rank, another column showing the running total, and another showing the difference from the previous month. In a spreadsheet, you would just add formulas in new columns — the original rows stay intact.

That is exactly what window functions do in SQL. They let you calculate across rows without collapsing them into groups. With GROUP BY, you lose the individual rows. With window functions, you keep every row and add the calculation alongside it.

Window functions are essential for reporting, analytics, and any time you need to compare rows to each other — rankings, running totals, moving averages, row-by-row comparisons, and more.

Setting Up Our Practice Tables

```sql
CREATE TABLE employees (
    employee_id  SERIAL PRIMARY KEY,
    name         VARCHAR(100) NOT NULL,
    department   VARCHAR(50) NOT NULL,
    salary       DECIMAL(10, 2) NOT NULL,
    hire_date    DATE NOT NULL
);

INSERT INTO employees (name, department, salary, hire_date) VALUES
('Alice',   'Engineering', 95000, '2020-03-15'),
('Bob',     'Engineering', 90000, '2019-07-01'),
('Charlie', 'Engineering', 90000, '2021-01-10'),
('Diana',   'Sales',       75000, '2020-06-20'),
('Eve',     'Sales',       80000, '2018-11-05'),
('Frank',   'Sales',       72000, '2022-02-14'),
('Grace',   'Marketing',   70000, '2021-05-01'),
('Hank',    'Marketing',   68000, '2020-09-15');

CREATE TABLE monthly_sales (
    sale_id    SERIAL PRIMARY KEY,
    salesperson VARCHAR(50) NOT NULL,
    month      DATE NOT NULL,
    amount     DECIMAL(10, 2) NOT NULL
);

INSERT INTO monthly_sales (salesperson, month, amount) VALUES
('Diana', '2024-01-01', 15000),
('Diana', '2024-02-01', 18000),
('Diana', '2024-03-01', 12000),
('Diana', '2024-04-01', 22000),
('Diana', '2024-05-01', 19000),
('Diana', '2024-06-01', 25000),
('Eve',   '2024-01-01', 20000),
('Eve',   '2024-02-01', 17000),
('Eve',   '2024-03-01', 23000),
('Eve',   '2024-04-01', 19000),
('Eve',   '2024-05-01', 21000),
('Eve',   '2024-06-01', 28000);
```

What Are Window Functions?

A window function performs a calculation across a set of rows that are related to the current row. Unlike GROUP BY, which collapses rows into groups, window functions keep every row and add the result as an extra column.

```
GROUP BY:                              Window Function:
+------------+----------+            +--------+------------+--------+----------+
| department | avg_sal  |            | name   | department | salary | avg_sal  |
+------------+----------+            +--------+------------+--------+----------+
| Engineering| 91666.67 |            | Alice  | Engineering| 95000  | 91666.67 |
| Sales      | 75666.67 |            | Bob    | Engineering| 90000  | 91666.67 |
| Marketing  | 69000.00 |            | Charlie| Engineering| 90000  | 91666.67 |
+------------+----------+            | Diana  | Sales      | 75000  | 75666.67 |
                                      | Eve    | Sales      | 80000  | 75666.67 |
3 rows (details lost)                 | Frank  | Sales      | 72000  | 75666.67 |
                                      | Grace  | Marketing  | 70000  | 69000.00 |
                                      | Hank   | Marketing  | 68000  | 69000.00 |
                                      +--------+------------+--------+----------+
                                      8 rows (all details kept!)
```

The OVER Clause

The OVER clause is what makes a function a window function. It defines the "window" of rows the function looks at.

Basic Syntax

```sql
function_name() OVER (
    [PARTITION BY column]
    [ORDER BY column]
    [frame_clause]
)
```

Simplest Window Function — OVER()

An empty OVER() means "use all rows as the window":

```sql
SELECT
    name,
    department,
```

```
    salary,
    AVG(salary) OVER () AS company_avg
FROM employees;
```

```
  name   | department  |  salary  | company_avg
---------+-------------+----------+------------
 Alice   | Engineering | 95000.00 |   80000.00
 Bob     | Engineering | 90000.00 |   80000.00
 Charlie | Engineering | 90000.00 |   80000.00
 Diana   | Sales       | 75000.00 |   80000.00
 Eve     | Sales       | 80000.00 |   80000.00
 Frank   | Sales       | 72000.00 |   80000.00
 Grace   | Marketing   | 70000.00 |   80000.00
 Hank    | Marketing   | 68000.00 |   80000.00
(8 rows)
```

Every row shows the same company-wide average. The key insight: every original row is preserved, and the average is added alongside.

Line-by-line explanation:

- `AVG(salary)` — the aggregate function we want to calculate
- `OVER ()` — an empty window means "calculate across all rows"
- `AS company_avg` — gives the computed column a name

PARTITION BY — Grouping Without Collapsing

PARTITION BY divides rows into groups (partitions), and the window function calculates separately within each group. It is like GROUP BY, but you keep all the rows.

```sql
SELECT
    name,
    department,
    salary,
    AVG(salary) OVER (PARTITION BY department) AS dept_avg
FROM employees;
```

```
  name   | department  |  salary  | dept_avg
---------+-------------+----------+----------
 Alice   | Engineering | 95000.00 | 91666.67
 Bob     | Engineering | 90000.00 | 91666.67
 Charlie | Engineering | 90000.00 | 91666.67
```

```
 Diana   | Sales       | 75000.00 | 75666.67
 Eve     | Sales       | 80000.00 | 75666.67
 Frank   | Sales       | 72000.00 | 75666.67
 Grace   | Marketing   | 70000.00 | 69000.00
 Hank    | Marketing   | 68000.00 | 69000.00
(8 rows)
```

```
How PARTITION BY works:

All Rows
+-------------------------------------------+
| Alice  | Bob | Charlie | Diana | Eve | ...|
+-------------------------------------------+
         |
    PARTITION BY department
         |
         v
+-------------+  +-----------+  +-----------+
| Engineering |  |   Sales   |  | Marketing |
| Alice       |  | Diana     |  | Grace     |
| Bob         |  | Eve       |  | Hank      |
| Charlie     |  | Frank     |  |           |
| AVG=91666   |  | AVG=75666 |  | AVG=69000 |
+-------------+  +-----------+  +-----------+

Each partition calculates independently.
All original rows are returned.
```

Comparing Each Employee to Their Department Average

```sql
SELECT
    name,
    department,
    salary,
    AVG(salary) OVER (PARTITION BY department) AS dept_avg,
    salary - AVG(salary) OVER (PARTITION BY department) AS diff_from_avg
FROM employees
ORDER BY department, salary DESC;
```

```
  name   | department  |  salary  | dept_avg  | diff_from_avg
---------+-------------+----------+-----------+--------------
 Alice   | Engineering | 95000.00 | 91666.67 |      3333.33
 Bob     | Engineering | 90000.00 | 91666.67 |     -1666.67
 Charlie | Engineering | 90000.00 | 91666.67 |     -1666.67
 Eve     | Sales       | 80000.00 | 75666.67 |      4333.33
 Diana   | Sales       | 75000.00 | 75666.67 |      -666.67
 Frank   | Sales       | 72000.00 | 75666.67 |     -3666.67
 Grace   | Marketing   | 70000.00 | 69000.00 |      1000.00
```

```
 Hank    | Marketing   | 68000.00 | 69000.00 |     -1000.00
(8 rows)
```

Now you can instantly see who earns above or below their department average.

ROW_NUMBER, RANK, and DENSE_RANK

These three functions assign a number to each row based on ordering. They differ in how they handle ties.

ROW_NUMBER

Assigns a unique sequential number to each row. Ties get different numbers (arbitrarily).

```sql
SELECT
    name,
    department,
    salary,
    ROW_NUMBER() OVER (ORDER BY salary DESC) AS row_num
FROM employees;
```

```
  name   | department  |  salary  | row_num
---------+-------------+----------+---------
 Alice   | Engineering | 95000.00 |       1
 Bob     | Engineering | 90000.00 |       2
 Charlie | Engineering | 90000.00 |       3
 Eve     | Sales       | 80000.00 |       4
 Diana   | Sales       | 75000.00 |       5
 Frank   | Sales       | 72000.00 |       6
 Grace   | Marketing   | 70000.00 |       7
 Hank    | Marketing   | 68000.00 |       8
(8 rows)
```

Notice Bob and Charlie both earn 90,000, but they get different row numbers (2 and 3).

RANK

Like ROW_NUMBER, but ties get the same rank. The next rank skips numbers.

```sql
SELECT
    name,
    department,
    salary,
    RANK() OVER (ORDER BY salary DESC) AS rank
FROM employees;
```

```
  name    | department   |  salary   | rank
---------+-------------+----------+------
 Alice   | Engineering | 95000.00 |    1
 Bob     | Engineering | 90000.00 |    2
 Charlie | Engineering | 90000.00 |    2
 Eve     | Sales       | 80000.00 |    4
 Diana   | Sales       | 75000.00 |    5
 Frank   | Sales       | 72000.00 |    6
 Grace   | Marketing   | 70000.00 |    7
 Hank    | Marketing   | 68000.00 |    8
(8 rows)
```

Bob and Charlie both get rank 2. The next rank is 4 (skipping 3).

DENSE_RANK

Like RANK, but no gaps. Ties get the same rank, and the next rank is the very next number.

```sql
SELECT
    name,
    department,
    salary,
    DENSE_RANK() OVER (ORDER BY salary DESC) AS dense_rank
FROM employees;
```

```
  name    | department   |  salary   | dense_rank
---------+-------------+----------+------------
 Alice   | Engineering | 95000.00 |          1
 Bob     | Engineering | 90000.00 |          2
 Charlie | Engineering | 90000.00 |          2
 Eve     | Sales       | 80000.00 |          3
 Diana   | Sales       | 75000.00 |          4
 Frank   | Sales       | 72000.00 |          5
 Grace   | Marketing   | 70000.00 |          6
 Hank    | Marketing   | 68000.00 |          7
(8 rows)
```

Bob and Charlie both get rank 2. The next rank is 3 (no gap).

Comparison of All Three

```
+--------+--------+------------+---------+------+------------+
| name   | salary | ROW_NUMBER | RANK    | DENSE_RANK        |
+--------+--------+------------+---------+-------------------+
| Alice  | 95000  |     1      |    1    |        1          |
| Bob    | 90000  |     2      |    2    |        2          |
| Charlie| 90000  |     3      |    2    |        2   <--tie |
| Eve    | 80000  |     4      |    4    |        3          |
+--------+--------+------------+---------+-------------------+
                        ^           ^            ^
                   Always       Skip after    No gaps
                   unique       ties          after ties
```

Ranking Within Departments

Combine ranking with PARTITION BY:

```sql
SELECT
    name,
    department,
    salary,
    RANK() OVER (
        PARTITION BY department
        ORDER BY salary DESC
    ) AS dept_rank
FROM employees;
```

```
  name   | department  |  salary  | dept_rank
---------+-------------+----------+-----------
 Alice   | Engineering | 95000.00 |         1
 Bob     | Engineering | 90000.00 |         2
 Charlie | Engineering | 90000.00 |         2
 Eve     | Sales       | 80000.00 |         1
 Diana   | Sales       | 75000.00 |         2
 Frank   | Sales       | 72000.00 |         3
 Grace   | Marketing   | 70000.00 |         1
 Hank    | Marketing   | 68000.00 |         2
(8 rows)
```

Each department has its own ranking starting from 1.

LAG and LEAD — Previous and Next Rows

LAG looks at the previous row. LEAD looks at the next row. They are perfect for comparing a row to its neighbors.

LAG — Looking Back

```sql
SELECT
    salesperson,
    month,
    amount,
    LAG(amount) OVER (
        PARTITION BY salesperson
        ORDER BY month
    ) AS prev_month,
    amount - LAG(amount) OVER (
        PARTITION BY salesperson
        ORDER BY month
    ) AS change
FROM monthly_sales
WHERE salesperson = 'Diana';
```

```
 salesperson |   month    |  amount   | prev_month |  change
-------------+------------+-----------+------------+---------
 Diana       | 2024-01-01 | 15000.00 |            |
 Diana       | 2024-02-01 | 18000.00 |   15000.00 | 3000.00
 Diana       | 2024-03-01 | 12000.00 |   18000.00 |-6000.00
 Diana       | 2024-04-01 | 22000.00 |   12000.00 |10000.00
 Diana       | 2024-05-01 | 19000.00 |   22000.00 |-3000.00
 Diana       | 2024-06-01 | 25000.00 |   19000.00 | 6000.00
(6 rows)
```

Line-by-line explanation:

- `LAG(amount)` — gets the value of `amount` from the previous row
- `PARTITION BY salesperson` — each salesperson has their own sequence
- `ORDER BY month` — defines what "previous" means (by month)
- The first row has no previous row, so LAG returns NULL

LAG with Default Value

You can specify a default value for when there is no previous row:

```sql
SELECT
    salesperson,
    month,
```

```
    amount,
    LAG(amount, 1, 0) OVER (
        PARTITION BY salesperson
        ORDER BY month
    ) AS prev_month
FROM monthly_sales
WHERE salesperson = 'Diana';
```

```
 salesperson |   month    |  amount   | prev_month
-------------+------------+----------+-----------
 Diana       | 2024-01-01 | 15000.00 |      0.00
 Diana       | 2024-02-01 | 18000.00 |  15000.00
 Diana       | 2024-03-01 | 12000.00 |  18000.00
 Diana       | 2024-04-01 | 22000.00 |  12000.00
 Diana       | 2024-05-01 | 19000.00 |  22000.00
 Diana       | 2024-06-01 | 25000.00 |  19000.00
(6 rows)
```

The three arguments to LAG are: `LAG(column, offset, default)`.

- `column` — which column to look at
- `offset` — how many rows back (default is 1)
- `default` — value to use when there is no previous row

LEAD — Looking Forward

```sql
SELECT
    salesperson,
    month,
    amount,
    LEAD(amount) OVER (
        PARTITION BY salesperson
        ORDER BY month
    ) AS next_month
FROM monthly_sales
WHERE salesperson = 'Eve';
```

```
 salesperson |   month    |  amount   | next_month
-------------+------------+----------+-----------
 Eve         | 2024-01-01 | 20000.00 |  17000.00
 Eve         | 2024-02-01 | 17000.00 |  23000.00
 Eve         | 2024-03-01 | 23000.00 |  19000.00
 Eve         | 2024-04-01 | 19000.00 |  21000.00
 Eve         | 2024-05-01 | 21000.00 |  28000.00
 Eve         | 2024-06-01 | 28000.00 |
(6 rows)
```

The last row has no next row, so LEAD returns NULL.

Running Totals with SUM OVER

When you combine SUM with ORDER BY in the OVER clause, you get a running total — the sum accumulates row by row.

```sql
SELECT
    salesperson,
    month,
    amount,
    SUM(amount) OVER (
        PARTITION BY salesperson
        ORDER BY month
    ) AS running_total
FROM monthly_sales
WHERE salesperson = 'Diana';
```

```
 salesperson |   month    |  amount   | running_total
-------------+------------+----------+--------------
 Diana       | 2024-01-01 | 15000.00 |     15000.00
 Diana       | 2024-02-01 | 18000.00 |     33000.00
 Diana       | 2024-03-01 | 12000.00 |     45000.00
 Diana       | 2024-04-01 | 22000.00 |     67000.00
 Diana       | 2024-05-01 | 19000.00 |     86000.00
 Diana       | 2024-06-01 | 25000.00 |    111000.00
(6 rows)
```

```
How running total works:

Month    Amount    Running Total
Jan      15000     15000            (15000)
Feb      18000     33000            (15000 + 18000)
Mar      12000     45000            (15000 + 18000 + 12000)
Apr      22000     67000            (15000 + 18000 + 12000 + 22000)
May      19000     86000            (... + 19000)
Jun      25000     111000           (... + 25000)
```

Running Average

```sql
SELECT
    salesperson,
    month,
    amount,
    ROUND(AVG(amount) OVER (
        PARTITION BY salesperson
        ORDER BY month
    ), 2) AS running_avg
FROM monthly_sales
WHERE salesperson = 'Diana';
```

```
 salesperson |   month    |  amount  | running_avg
-------------+------------+----------+------------
 Diana       | 2024-01-01 | 15000.00 |   15000.00
 Diana       | 2024-02-01 | 18000.00 |   16500.00
 Diana       | 2024-03-01 | 12000.00 |   15000.00
 Diana       | 2024-04-01 | 22000.00 |   16750.00
 Diana       | 2024-05-01 | 19000.00 |   17200.00
 Diana       | 2024-06-01 | 25000.00 |   18500.00
(6 rows)
```

NTILE — Dividing Rows into Buckets

NTILE divides rows into a specified number of roughly equal groups:

```sql
SELECT
    name,
    salary,
    NTILE(4) OVER (ORDER BY salary DESC) AS quartile
FROM employees;
```

```
  name   |  salary  | quartile
---------+----------+----------
 Alice   | 95000.00 |        1
 Bob     | 90000.00 |        1
 Charlie | 90000.00 |        2
 Eve     | 80000.00 |        2
 Diana   | 75000.00 |        3
 Frank   | 72000.00 |        3
 Grace   | 70000.00 |        4
 Hank    | 68000.00 |        4
(8 rows)
```

Line-by-line explanation:

- `NTILE(4)` — divide all rows into 4 groups
- `ORDER BY salary DESC` — highest salaries in group 1
- 8 employees / 4 groups = 2 per group

This is useful for percentile analysis. Quartile 1 is the top 25 percent, quartile 4 is the bottom 25 percent.

Frame Clause — ROWS BETWEEN

The frame clause gives you precise control over which rows the window function considers. This is how you create moving averages and sliding windows.

Default Frame

When you use ORDER BY in the OVER clause, the default frame is:

```
RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
```

This means "from the first row in the partition up to the current row" — which is why SUM with ORDER BY gives you a running total.

Moving Average (3-Month)

A 3-month moving average smooths out fluctuations by averaging the current row and the two rows before it:

```sql
SELECT
    salesperson,
    month,
    amount,
    ROUND(AVG(amount) OVER (
        PARTITION BY salesperson
        ORDER BY month
        ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
    ), 2) AS moving_avg_3m
FROM monthly_sales
WHERE salesperson = 'Diana';
```

```
 salesperson |   month    |  amount  | moving_avg_3m
-------------+------------+----------+---------------
 Diana       | 2024-01-01 | 15000.00 |     15000.00
 Diana       | 2024-02-01 | 18000.00 |     16500.00
 Diana       | 2024-03-01 | 12000.00 |     15000.00
 Diana       | 2024-04-01 | 22000.00 |     17333.33
 Diana       | 2024-05-01 | 19000.00 |     17666.67
 Diana       | 2024-06-01 | 25000.00 |     22000.00
(6 rows)
```

```
How ROWS BETWEEN 2 PRECEDING AND CURRENT ROW works:

Row 1 (Jan):  AVG(15000)                        = 15000
Row 2 (Feb):  AVG(15000, 18000)                  = 16500
Row 3 (Mar):  AVG(15000, 18000, 12000)           = 15000
Row 4 (Apr):  AVG(18000, 12000, 22000)           = 17333   <-- window slides
Row 5 (May):  AVG(12000, 22000, 19000)           = 17667
Row 6 (Jun):  AVG(22000, 19000, 25000)           = 22000

         +-----+
         | Jan |  Feb   Mar   Apr   May   Jun
         +-----+
               +-------+
          Jan  |  Feb  |  Mar   Apr   May   Jun
               +-------+
               +-------------+
          Jan  |  Feb    Mar  |  Apr   May   Jun
               +-------------+
                     +-------------+
          Jan   Feb  |  Mar    Apr  |  May   Jun
                     +-------------+
                              The window slides forward!
```

Frame Clause Options

```
+-----------------------------------+----------------------------------------+
| Frame Specification               | Meaning                                |
+-----------------------------------+----------------------------------------+
| ROWS BETWEEN UNBOUNDED PRECEDING | From first row to current row          |
| AND CURRENT ROW                   |                                        |
+-----------------------------------+----------------------------------------+
| ROWS BETWEEN 2 PRECEDING         | From 2 rows before to current row      |
| AND CURRENT ROW                   |                                        |
+-----------------------------------+----------------------------------------+
| ROWS BETWEEN 1 PRECEDING         | From 1 row before to 1 row after       |
| AND 1 FOLLOWING                   | (centered window)                      |
+-----------------------------------+----------------------------------------+
| ROWS BETWEEN CURRENT ROW         | From current row to last row           |
| AND UNBOUNDED FOLLOWING           |                                        |
```

```
+-----------------------------------+-----------------------------------------+
| ROWS BETWEEN UNBOUNDED PRECEDING | All rows in the partition               |
| AND UNBOUNDED FOLLOWING          |                                         |
+-----------------------------------+-----------------------------------------+
```

Centered Moving Average

```sql
SELECT
    salesperson,
    month,
    amount,
    ROUND(AVG(amount) OVER (
        PARTITION BY salesperson
        ORDER BY month
        ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING
    ), 2) AS centered_avg
FROM monthly_sales
WHERE salesperson = 'Diana';
```

```
 salesperson |   month    |  amount   | centered_avg
-------------+------------+----------+-------------
 Diana       | 2024-01-01 | 15000.00 |    16500.00
 Diana       | 2024-02-01 | 18000.00 |    15000.00
 Diana       | 2024-03-01 | 12000.00 |    17333.33
 Diana       | 2024-04-01 | 22000.00 |    17666.67
 Diana       | 2024-05-01 | 19000.00 |    22000.00
 Diana       | 2024-06-01 | 25000.00 |    22000.00
(6 rows)
```

This looks at one row before AND one row after the current row.

Practical: Ranking Employees by Department

Let us create a comprehensive employee ranking report:

```sql
SELECT
    name,
    department,
    salary,
    RANK() OVER (
```

```
        PARTITION BY department
        ORDER BY salary DESC
    ) AS dept_salary_rank,
    ROUND(salary / SUM(salary) OVER (PARTITION BY department) * 100, 1)
        AS pct_of_dept_total,
    CASE
        WHEN salary >= AVG(salary) OVER (PARTITION BY department)
        THEN 'Above Average'
        ELSE 'Below Average'
    END AS vs_dept_avg
FROM employees
ORDER BY department, salary DESC;
```

```
  name   | department  |  salary  | dept_salary_rank | pct_of_dept_total |   vs_dept_avg
---------+-------------+----------+------------------+-------------------+---------------
 Alice   | Engineering | 95000.00 |                1 |              34.5 | Above Average
 Bob     | Engineering | 90000.00 |                2 |              32.7 | Below Average
 Charlie | Engineering | 90000.00 |                2 |              32.7 | Below Average
 Grace   | Marketing   | 70000.00 |                1 |              50.7 | Above Average
 Hank    | Marketing   | 68000.00 |                2 |              49.3 | Below Average
 Eve     | Sales       | 80000.00 |                1 |              35.2 | Above Average
 Diana   | Sales       | 75000.00 |                2 |              33.0 | Below Average
 Frank   | Sales       | 72000.00 |                3 |              31.7 | Below Average
(8 rows)
```

Practical: Running Sales Total with Comparison

```sql
SELECT
    salesperson,
    month,
    amount,
    SUM(amount) OVER (
        PARTITION BY salesperson
        ORDER BY month
    ) AS ytd_total,
    LAG(amount) OVER (
        PARTITION BY salesperson
        ORDER BY month
    ) AS prev_month,
    CASE
        WHEN LAG(amount) OVER (
            PARTITION BY salesperson ORDER BY month
        ) IS NULL THEN 'N/A'
        WHEN amount > LAG(amount) OVER (
            PARTITION BY salesperson ORDER BY month
```

```
        ) THEN 'UP'
        WHEN amount < LAG(amount) OVER (
            PARTITION BY salesperson ORDER BY month
        ) THEN 'DOWN'
        ELSE 'FLAT'
    END AS trend
FROM monthly_sales
ORDER BY salesperson, month;
```

```
 salesperson |   month    |  amount   | ytd_total | prev_month | trend
-------------+------------+----------+-----------+------------+------
 Diana       | 2024-01-01 | 15000.00 |  15000.00 |            | N/A
 Diana       | 2024-02-01 | 18000.00 |  33000.00 |   15000.00 | UP
 Diana       | 2024-03-01 | 12000.00 |  45000.00 |   18000.00 | DOWN
 Diana       | 2024-04-01 | 22000.00 |  67000.00 |   12000.00 | UP
 Diana       | 2024-05-01 | 19000.00 |  86000.00 |   22000.00 | DOWN
 Diana       | 2024-06-01 | 25000.00 | 111000.00 |   19000.00 | UP
 Eve         | 2024-01-01 | 20000.00 |  20000.00 |            | N/A
 Eve         | 2024-02-01 | 17000.00 |  37000.00 |   20000.00 | DOWN
 Eve         | 2024-03-01 | 23000.00 |  60000.00 |   17000.00 | UP
 Eve         | 2024-04-01 | 19000.00 |  79000.00 |   23000.00 | DOWN
 Eve         | 2024-05-01 | 21000.00 | 100000.00 |   19000.00 | UP
 Eve         | 2024-06-01 | 28000.00 | 128000.00 |   21000.00 | UP
(12 rows)
```

This single query gives you the year-to-date total, the previous month's amount, and a trend indicator — all without any JOINs or subqueries.

Named Windows with WINDOW Clause

When you use the same window definition multiple times, you can name it to avoid repetition:

```sql
SELECT
    salesperson,
    month,
    amount,
    SUM(amount) OVER w AS running_total,
    AVG(amount) OVER w AS running_avg,
    COUNT(*)    OVER w AS months_so_far
FROM monthly_sales
WHERE salesperson = 'Diana'
WINDOW w AS (PARTITION BY salesperson ORDER BY month)
ORDER BY month;
```

```
 salesperson |   month    |  amount   | running_total | running_avg | months_so_far
-------------+------------+-----------+---------------+-------------+--------------
 Diana       | 2024-01-01 | 15000.00 |     15000.00  |   15000.00  |            1
 Diana       | 2024-02-01 | 18000.00 |     33000.00  |   16500.00  |            2
 Diana       | 2024-03-01 | 12000.00 |     45000.00  |   15000.00  |            3
 Diana       | 2024-04-01 | 22000.00 |     67000.00  |   16750.00  |            4
 Diana       | 2024-05-01 | 19000.00 |     86000.00  |   17200.00  |            5
 Diana       | 2024-06-01 | 25000.00 |    111000.00  |   18500.00  |            6
(6 rows)
```

The `WINDOW w AS (...)` at the bottom defines the window once, and all three functions reference it with `OVER w`.

Common Mistakes

Mistake 1: Using Window Functions in WHERE

```sql
-- BAD: You cannot filter by a window function in WHERE
SELECT name, salary, RANK() OVER (ORDER BY salary DESC) AS rnk
FROM employees
WHERE rnk <= 3;   -- ERROR!

-- GOOD: Use a subquery or CTE
SELECT * FROM (
    SELECT name, salary, RANK() OVER (ORDER BY salary DESC) AS rnk
    FROM employees
) ranked
WHERE rnk <= 3;
```

Window functions are computed after WHERE, so you must wrap the query.

Mistake 2: Confusing RANK, DENSE_RANK, and ROW_NUMBER

```sql
-- If you want exactly 3 results, use ROW_NUMBER:
-- RANK might return more than 3 if there are ties at position 3

-- ROW_NUMBER: Always exactly N results
-- RANK: Might return more than N (ties share a rank)
-- DENSE_RANK: No gaps, but might still return more than N
```

Mistake 3: Forgetting ORDER BY in Running Totals

```sql
-- BAD: Without ORDER BY, SUM gives the total for the whole partition
SELECT month, amount,
       SUM(amount) OVER (PARTITION BY salesperson) AS not_running
FROM monthly_sales;
-- Every row shows the same total!

-- GOOD: Add ORDER BY for a running total
SELECT month, amount,
       SUM(amount) OVER (
           PARTITION BY salesperson
           ORDER BY month
       ) AS running_total
FROM monthly_sales;
```

Mistake 4: Not Understanding the Default Frame

```sql
-- With ORDER BY, the default frame is:
-- RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
-- This means SUM gives a running total, not a full partition total.
-- Without ORDER BY, the default frame is the entire partition.
```

Best Practices

1. Use named windows (WINDOW clause) when reusing the same definition. It reduces code duplication and makes queries easier to maintain.
2. Choose the right ranking function. Use ROW_NUMBER for unique numbering, RANK for competition-style ranking (with gaps), and DENSE_RANK for ranking without gaps.
3. Always specify ORDER BY in the OVER clause when using LAG, LEAD, or running totals. The result is undefined without it.
4. Use window functions instead of self-joins. Comparing rows to previous rows with LAG is much cleaner and faster than joining a table to itself.
5. Be explicit about the frame clause when using aggregate window functions with ORDER BY. Do not rely on the default frame unless you are certain it does what you want.

Quick Summary

```
+-------------------+-------------------------------------------------+
| Function          | What It Does                                    |
+-------------------+-------------------------------------------------+
| ROW_NUMBER()      | Unique sequential number (no ties)              |
| RANK()            | Rank with gaps after ties                       |
| DENSE_RANK()      | Rank without gaps after ties                    |
| LAG(col, n)       | Value from n rows before (default n=1)          |
| LEAD(col, n)      | Value from n rows after (default n=1)           |
| SUM() OVER        | Running total (with ORDER BY)                   |
| AVG() OVER        | Running average (with ORDER BY)                 |
| NTILE(n)          | Divide rows into n equal buckets                |
| PARTITION BY      | Split rows into groups (like GROUP BY)           |
| ROWS BETWEEN      | Define exact frame of rows to consider           |
+-------------------+-------------------------------------------------+
```

Key Points

- Window functions calculate across related rows without collapsing them — every row is preserved.
- The OVER clause defines the window. An empty OVER() uses all rows.
- PARTITION BY splits rows into independent groups for calculation.
- ROW_NUMBER gives unique numbers; RANK and DENSE_RANK handle ties differently.
- LAG looks at previous rows; LEAD looks at next rows. Both are great for comparisons.
- SUM with ORDER BY in OVER gives you a running total.
- The frame clause (ROWS BETWEEN) controls exactly which rows the function considers.
- Named windows (WINDOW clause) reduce repetition when multiple functions share the same window.
- Window functions cannot be used in WHERE — wrap the query in a subquery instead.

Practice Questions

1. What is the difference between GROUP BY and a window function with PARTITION BY? When would you use each?
2. You have a table of exam scores. Three students scored 95. What rank would the next student (scoring 90) receive with RANK vs DENSE_RANK?
3. What does `LAG(amount, 2, 0)` return? What do the three arguments mean?
4. Why does `SUM(amount) OVER (PARTITION BY salesperson ORDER BY month)` give a running total, but `SUM(amount) OVER (PARTITION BY salesperson)` gives the same total for every row?
5. Can you use a window function in a WHERE clause? If not, how do you filter by a window function's result?

Exercises

Exercise 1: Top Earners per Department

Write a query that shows the top 2 highest-paid employees in each department. Include their name, department, salary, and rank within the department. Use ROW_NUMBER to ensure exactly 2 results per department.

Hint: Use ROW_NUMBER with PARTITION BY department, then wrap it in a subquery to filter.

Exercise 2: Month-over-Month Growth Rate

Using the monthly_sales table, write a query that shows each salesperson's monthly sales along with:

- The previous month's sales
- The percentage change from the previous month
- Whether sales went UP, DOWN, or stayed FLAT

Hint: Use LAG to get the previous month's value, then calculate the percentage.

Exercise 3: Moving Average Report

Create a report showing each salesperson's monthly sales with a 3-month moving average. Also include the cumulative year-to-date total. Use the WINDOW clause to avoid repeating the partition definition.

Hint: The moving average needs `ROWS BETWEEN 2 PRECEDING AND CURRENT ROW`. The running total uses the default frame.

What Is Next?

In Chapter 27, you will learn about Common Table Expressions (CTEs) and recursive queries. CTEs let you break complex queries into named, readable steps — like giving names to temporary results. Recursive CTEs take it further, letting you traverse hierarchies like organizational charts and category trees.

Chapter 27: Common Table Expressions and Recursive Queries

What You Will Learn

In this chapter, you will learn:

- What Common Table Expressions (CTEs) are and why they exist
- How to use the WITH clause to create named temporary results
- How to chain multiple CTEs together
- How CTEs compare to subqueries and views
- What recursive CTEs are and how they work
- How to traverse hierarchies like organizational charts and category trees
- How to generate number and date series with recursion

Why This Chapter Matters

Imagine you are writing a recipe. You could write it as one enormous paragraph: "Take the flour that you sifted and mixed with the baking powder that you measured from the container and combine it with the eggs that you cracked and whisked with the sugar that you measured..." That is unreadable.

Or you could break it into steps:

1. Sift the dry ingredients
2. Whisk the wet ingredients
3. Combine wet and dry ingredients

CTEs are the SQL equivalent of numbered recipe steps. Instead of nesting subqueries five levels deep, you break the query into named steps that are easy to read, debug, and maintain.

Recursive CTEs go even further. They let you answer questions that regular SQL cannot: "Who reports to whom in the entire company hierarchy?" or "What is the full path from a top-level category down to a subcategory?" These are problems that require traversing tree structures, and recursive CTEs handle them elegantly.

Setting Up Our Practice Tables

```sql
CREATE TABLE employees (
    employee_id  SERIAL PRIMARY KEY,
    name         VARCHAR(100) NOT NULL,
    manager_id   INT REFERENCES employees(employee_id),
    department   VARCHAR(50) NOT NULL,
    salary       DECIMAL(10, 2) NOT NULL
);

INSERT INTO employees (employee_id, name, manager_id, department, salary) VALUES
(1, 'Sarah CEO',       NULL, 'Executive',   200000),
(2, 'Tom VP Eng',      1,    'Engineering', 150000),
(3, 'Lisa VP Sales',   1,    'Sales',       145000),
(4, 'Mike Director',   2,    'Engineering', 120000),
(5, 'Anna Director',   2,    'Engineering', 118000),
(6, 'Jake Manager',    4,    'Engineering',  95000),
(7, 'Emma Manager',    4,    'Engineering',  92000),
(8, 'Chris Developer', 6,    'Engineering',  85000),
(9, 'Pat Developer',   6,    'Engineering',  82000),
(10,'Sam Sales Rep',   3,    'Sales',        70000),
(11,'Kim Sales Rep',   3,    'Sales',        72000);

CREATE TABLE categories (
    category_id   SERIAL PRIMARY KEY,
    category_name VARCHAR(100) NOT NULL,
    parent_id     INT REFERENCES categories(category_id)
);

INSERT INTO categories (category_id, category_name, parent_id) VALUES
(1, 'Electronics',     NULL),
(2, 'Computers',       1),
(3, 'Phones',          1),
(4, 'Laptops',         2),
(5, 'Desktops',        2),
(6, 'Smartphones',     3),
(7, 'Feature Phones',  3),
(8, 'Gaming Laptops',  4),
(9, 'Business Laptops',4),
(10,'Clothing',        NULL),
(11,'Men',             10),
```

```
(12,'Women',           10),
(13,'Shirts',          11),
(14,'Pants',           11);

CREATE TABLE orders (
    order_id    SERIAL PRIMARY KEY,
    customer_id INT NOT NULL,
    order_date  DATE NOT NULL,
    total       DECIMAL(10, 2) NOT NULL
);

INSERT INTO orders (customer_id, order_date, total) VALUES
(1, '2024-01-15', 250.00),
(2, '2024-01-20', 150.00),
(1, '2024-02-10', 300.00),
(3, '2024-02-15', 450.00),
(2, '2024-03-01', 200.00),
(1, '2024-03-15', 175.00),
(3, '2024-03-20', 500.00),
(4, '2024-04-01', 350.00);
```

What Is a CTE?

A Common Table Expression (CTE) is a named temporary result set that you define at the beginning of a query. It exists only for the duration of that single query.

Basic Syntax

```sql
WITH cte_name AS (
    SELECT ...
)
SELECT ...
FROM cte_name;
```

Think of it as saying: "First, calculate this result and call it `cte_name`. Then, use `cte_name` in the main query."

Your First CTE

Let us find employees who earn more than the average salary:

```sql
WITH avg_salary AS (
    SELECT AVG(salary) AS average
    FROM employees
)
SELECT
    e.name,
    e.department,
    e.salary,
    a.average,
    e.salary - a.average AS above_average
FROM employees e
CROSS JOIN avg_salary a
WHERE e.salary > a.average
ORDER BY e.salary DESC;
```

```
     name      | department  |  salary   |   average    | above_average
---------------+-------------+-----------+--------------+--------------
 Sarah CEO     | Executive   | 200000.00 | 111727.27    |     88272.73
 Tom VP Eng    | Engineering | 150000.00 | 111727.27    |     38272.73
 Lisa VP Sales | Sales       | 145000.00 | 111727.27    |     33272.73
 Mike Director | Engineering | 120000.00 | 111727.27    |      8272.73
 Anna Director | Engineering | 118000.00 | 111727.27    |      6272.73
(5 rows)
```

Line-by-line explanation:

- `WITH avg_salary AS (...)` — define a CTE named `avg_salary` that calculates the average
- The CTE produces a single row with a single column called `average`
- `CROSS JOIN avg_salary a` — join every employee row with the average value
- `WHERE e.salary > a.average` — filter to only above-average earners

Why Use CTEs Instead of Subqueries?

The Subquery Version (Harder to Read)

```sql
SELECT
    e.name,
    e.department,
    e.salary,
    (SELECT AVG(salary) FROM employees) AS average,
    e.salary - (SELECT AVG(salary) FROM employees) AS above_average
```

```
FROM employees e
WHERE e.salary > (SELECT AVG(salary) FROM employees)
ORDER BY e.salary DESC;
```

The subquery `(SELECT AVG(salary) FROM employees)` is repeated three times. With a CTE, you calculate it once and reference it by name.

Side-by-Side Comparison

```
Nested Subqueries:                  CTEs:
+---------------------------+      +---------------------------+
| SELECT ...                |      | WITH                      |
|   FROM (                  |      |   step1 AS (SELECT ...),  |
|     SELECT ...            |      |   step2 AS (SELECT ...),  |
|       FROM (              |      |   step3 AS (SELECT ...)   |
|         SELECT ...        |      | SELECT ...                |
|       ) sub2              |      | FROM step1                |
|   ) sub1                  |      | JOIN step2 ...            |
| WHERE ...                 |      | JOIN step3 ...            |
+---------------------------+      +---------------------------+

Read from inside out               Read from top to bottom
Hard to debug                      Easy to debug
Hard to maintain                   Easy to maintain
```

Multiple CTEs

You can chain multiple CTEs together, separated by commas. Each CTE can reference the ones defined before it.

```sql
WITH monthly_totals AS (
    SELECT
        DATE_TRUNC('month', order_date)::DATE AS month,
        SUM(total) AS month_total
    FROM orders
    GROUP BY DATE_TRUNC('month', order_date)
),
monthly_with_prev AS (
    SELECT
        month,
        month_total,
        LAG(month_total) OVER (ORDER BY month) AS prev_total
```

```
    FROM monthly_totals
),
monthly_growth AS (
    SELECT
        month,
        month_total,
        prev_total,
        CASE
            WHEN prev_total IS NULL THEN NULL
            ELSE ROUND(
                (month_total - prev_total) / prev_total * 100, 1
            )
        END AS growth_pct
    FROM monthly_with_prev
)
SELECT * FROM monthly_growth
ORDER BY month;
```

```
   month    | month_total | prev_total | growth_pct
------------+-------------+------------+-----------
 2024-01-01 |      400.00 |            |
 2024-02-01 |      750.00 |     400.00 |      87.5
 2024-03-01 |      875.00 |     750.00 |      16.7
 2024-04-01 |      350.00 |     875.00 |     -60.0
(4 rows)
```

Line-by-line explanation:

- `monthly_totals` — Step 1: aggregate orders by month
- `monthly_with_prev` — Step 2: add previous month's total using LAG (references step 1)
- `monthly_growth` — Step 3: calculate growth percentage (references step 2)
- The final SELECT reads from the last CTE

Each step is simple on its own. Together, they build a complex analysis.

```
Data Flow Through Multiple CTEs:

orders table
     |
     v
+----------------+     +------------------+     +---------------+
| monthly_totals | --> | monthly_with_prev| --> | monthly_growth|
| (GROUP BY)     |     | (LAG)            |     | (% calc)      |
+----------------+     +------------------+     +---------------+
                                                       |
                                                       v
                                                 Final SELECT
```

CTE vs Subquery vs View

```
+-----------------+-------------------+-----------------+-------------------+
| Feature         | Subquery          | CTE             | View              |
+-----------------+-------------------+-----------------+-------------------+
| Scope           | Inside one query  | Inside one query | Permanent object  |
| Reusable        | No (must repeat)  | Yes (within query) | Yes (any query)  |
| Readable        | Hard when nested  | Very readable   | Very readable     |
| Stored in DB    | No                | No              | Yes               |
| Can be recursive | No               | Yes             | No                |
| Performance     | Same as CTE*      | Same as subquery* | Same as CTE*      |
+-----------------+-------------------+-----------------+-------------------+

* In PostgreSQL 12+, CTEs are usually inlined (optimized the same as subqueries).
  In earlier versions, CTEs were always materialized (computed once).
```

When to Use Each

- Subquery: Simple, one-time use, not deeply nested
- CTE: Complex logic, multiple steps, need to reference the same result multiple times, or when you want readable code
- View: Need to reuse the query across multiple queries, or share the logic with other users

---

Recursive CTEs

A recursive CTE is a CTE that references itself. It is how SQL traverses trees and hierarchies.

The Basic Structure

```sql
WITH RECURSIVE cte_name AS (
    -- Base case (anchor): the starting point
    SELECT ...
    WHERE ... (starting condition)

    UNION ALL
```

```
    -- Recursive case: references the CTE itself
    SELECT ...
    FROM table_name
    JOIN cte_name ON ... (how to find the next level)
)
SELECT * FROM cte_name;
```

```
How Recursion Works:

Step 1: Run the base case (anchor query)
        Result: {Row A}

Step 2: Run recursive query using Step 1 results
        Result: {Row B, Row C}

Step 3: Run recursive query using Step 2 results
        Result: {Row D, Row E}

Step 4: Run recursive query using Step 3 results
        Result: {} (empty — stop!)

Final result: {Row A, Row B, Row C, Row D, Row E}
```

The Organizational Chart Analogy

Think of it like exploring a family tree. You start with the founder (base case). Then you find everyone who reports directly to the founder (first recursion). Then you find everyone who reports to those people (second recursion). You keep going until there are no more reports to find.

Practical: Organizational Chart

Let us build the complete organizational chart starting from the CEO:

```sql
WITH RECURSIVE org_chart AS (
    -- Base case: start with the CEO (no manager)
    SELECT
        employee_id,
        name,
        manager_id,
        department,
```

```
        salary,
        0 AS level,
        name AS path
    FROM employees
    WHERE manager_id IS NULL

    UNION ALL

    -- Recursive case: find employees who report to someone
    -- we already found
    SELECT
        e.employee_id,
        e.name,
        e.manager_id,
        e.department,
        e.salary,
        oc.level + 1,
        oc.path || ' > ' || e.name
    FROM employees e
    JOIN org_chart oc ON e.manager_id = oc.employee_id
)
SELECT
    REPEAT('  ', level) || name AS employee,
    department,
    salary,
    level,
    path
FROM org_chart
ORDER BY path;
```

```
       employee        | department  |  salary   | level |                path
-----------------------+-------------+-----------+-------+----------------------------...
 Sarah CEO             | Executive   | 200000.00 |     0 | Sarah CEO
   Lisa VP Sales       | Sales       | 145000.00 |     1 | Sarah CEO > Lisa VP Sales
     Kim Sales Rep     | Sales       |  72000.00 |     2 | Sarah CEO > Lisa VP Sales > ...
     Sam Sales Rep     | Sales       |  70000.00 |     2 | Sarah CEO > Lisa VP Sales > ...
   Tom VP Eng          | Engineering | 150000.00 |     1 | Sarah CEO > Tom VP Eng
     Anna Director     | Engineering | 118000.00 |     2 | Sarah CEO > Tom VP Eng > Ann...
     Mike Director     | Engineering | 120000.00 |     2 | Sarah CEO > Tom VP Eng > Mik...
       Emma Manager    | Engineering |  92000.00 |     3 | Sarah CEO > ... > Emma Manager
       Jake Manager    | Engineering |  95000.00 |     3 | Sarah CEO > ... > Jake Manager
         Chris Developer| Engineering |  85000.00 |     4 | Sarah CEO > ... > Chris Dev...
         Pat Developer | Engineering |  82000.00 |     4 | Sarah CEO > ... > Pat Developer
(11 rows)
```

Line-by-line explanation of the recursive CTE:

- `WHERE manager_id IS NULL` — the base case finds the CEO (who has no manager)
- `0 AS level` — the CEO is at level 0
- `name AS path` — start building the path with just the CEO's name

- `JOIN org_chart oc ON e.manager_id = oc.employee_id` — find employees whose manager is someone we already found
- `oc.level + 1` — each level down adds 1
- `oc.path || ' > ' || e.name` — append the current name to the path

Visualizing the Hierarchy

```
                    Sarah CEO (Level 0)
                    /              \
           Tom VP Eng          Lisa VP Sales (Level 1)
           /        \            /        \
    Mike Dir    Anna Dir    Sam Rep    Kim Rep (Level 2)
     /     \
Jake Mgr  Emma Mgr                              (Level 3)
  /    \
Chris  Pat                                       (Level 4)
```

Practical: Category Tree with Breadcrumb Paths

Let us build breadcrumb paths for a product category tree:

```sql
WITH RECURSIVE category_tree AS (
    -- Base case: top-level categories (no parent)
    SELECT
        category_id,
        category_name,
        parent_id,
        1 AS depth,
        category_name AS breadcrumb
    FROM categories
    WHERE parent_id IS NULL

    UNION ALL

    -- Recursive case: child categories
    SELECT
        c.category_id,
        c.category_name,
        c.parent_id,
        ct.depth + 1,
        ct.breadcrumb || ' > ' || c.category_name
    FROM categories c
    JOIN category_tree ct ON c.parent_id = ct.category_id
```

```
)
SELECT
    REPEAT('  ', depth - 1) || category_name AS category,
    depth,
    breadcrumb
FROM category_tree
ORDER BY breadcrumb;
```

```
        category       | depth |                breadcrumb
-----------------------+-------+-------------------------------------------
 Clothing              |     1 | Clothing
   Men                 |     2 | Clothing > Men
     Pants             |     3 | Clothing > Men > Pants
     Shirts            |     3 | Clothing > Men > Shirts
   Women               |     2 | Clothing > Women
 Electronics           |     1 | Electronics
   Computers           |     2 | Electronics > Computers
     Desktops          |     3 | Electronics > Computers > Desktops
     Laptops           |     3 | Electronics > Computers > Laptops
       Business Laptops|     4 | Electronics > Computers > Laptops > ...
       Gaming Laptops  |     4 | Electronics > Computers > Laptops > ...
   Phones              |     2 | Electronics > Phones
     Feature Phones    |     3 | Electronics > Phones > Feature Phones
     Smartphones       |     3 | Electronics > Phones > Smartphones
(14 rows)
```

The breadcrumb paths are exactly what you would see in an e-commerce navigation bar:

```
Electronics > Computers > Laptops > Gaming Laptops
```

Finding All Descendants

Find all employees who report to Tom VP Eng (directly or indirectly):

```sql
WITH RECURSIVE reports AS (
    -- Base case: start with Tom
    SELECT employee_id, name, manager_id, salary
    FROM employees
    WHERE employee_id = 2  -- Tom VP Eng

    UNION ALL
```

```
    -- Recursive case: find everyone who reports to someone
    -- we already found
    SELECT e.employee_id, e.name, e.manager_id, e.salary
    FROM employees e
    JOIN reports r ON e.manager_id = r.employee_id
)
SELECT name, salary
FROM reports
WHERE employee_id != 2  -- Exclude Tom himself
ORDER BY salary DESC;
```

```
      name       |  salary
-----------------+----------
 Mike Director   | 120000.00
 Anna Director   | 118000.00
 Jake Manager    |  95000.00
 Emma Manager    |  92000.00
 Chris Developer |  85000.00
 Pat Developer   |  82000.00
(6 rows)
```

Generating a Number Series

Recursive CTEs can generate sequences of numbers:

```sql
WITH RECURSIVE numbers AS (
    -- Base case: start with 1
    SELECT 1 AS n

    UNION ALL

    -- Recursive case: add 1 each time
    SELECT n + 1
    FROM numbers
    WHERE n < 10
)
SELECT n FROM numbers;
```

```
 n
----
  1
  2
  3
  4
```

```
  5
  6
  7
  8
  9
 10
(10 rows)
```

Important: The `WHERE n < 10` is the termination condition. Without it, the recursion would run forever (or until PostgreSQL's recursion limit is hit).

Generating a Date Series

```sql
WITH RECURSIVE dates AS (
    SELECT DATE '2024-01-01' AS d

    UNION ALL

    SELECT d + INTERVAL '1 month'
    FROM dates
    WHERE d < DATE '2024-06-01'
)
SELECT d::DATE AS month FROM dates;
```

```
   month
------------
 2024-01-01
 2024-02-01
 2024-03-01
 2024-04-01
 2024-05-01
 2024-06-01
(6 rows)
```

Note: PostgreSQL has a built-in `generate_series()` function that is usually better for simple sequences. Recursive CTEs are more useful when the logic at each step is complex.

Counting Levels in a Hierarchy

How many levels deep is each part of the organization?

```sql
WITH RECURSIVE org_depth AS (
    SELECT
        employee_id,
        name,
        manager_id,
        0 AS depth
    FROM employees
    WHERE manager_id IS NULL

    UNION ALL

    SELECT
        e.employee_id,
        e.name,
        e.manager_id,
        od.depth + 1
    FROM employees e
    JOIN org_depth od ON e.manager_id = od.employee_id
)
SELECT
    MAX(depth) AS max_depth,
    COUNT(*) FILTER (WHERE depth = 0) AS level_0,
    COUNT(*) FILTER (WHERE depth = 1) AS level_1,
    COUNT(*) FILTER (WHERE depth = 2) AS level_2,
    COUNT(*) FILTER (WHERE depth = 3) AS level_3,
    COUNT(*) FILTER (WHERE depth = 4) AS level_4
FROM org_depth;
```

```
 max_depth | level_0 | level_1 | level_2 | level_3 | level_4
-----------+---------+---------+---------+---------+---------
         4 |       1 |       2 |       4 |       2 |       2
(1 row)
```

The organization is 5 levels deep (0 through 4), with the most employees at level 2.

Safety: Preventing Infinite Recursion

If your data has a cycle (employee A reports to B, who reports to A), the recursive CTE will loop forever. PostgreSQL has a default recursion limit, but you can also protect yourself:

Method 1: Limit Depth

```sql
WITH RECURSIVE safe_tree AS (
    SELECT employee_id, name, manager_id, 0 AS depth
    FROM employees
    WHERE manager_id IS NULL

    UNION ALL

    SELECT e.employee_id, e.name, e.manager_id, st.depth + 1
    FROM employees e
    JOIN safe_tree st ON e.manager_id = st.employee_id
    WHERE st.depth < 20  -- Safety limit: stop at 20 levels
)
SELECT * FROM safe_tree;
```

Method 2: Track Visited Nodes

```sql
WITH RECURSIVE safe_tree AS (
    SELECT
        employee_id,
        name,
        manager_id,
        ARRAY[employee_id] AS visited
    FROM employees
    WHERE manager_id IS NULL

    UNION ALL

    SELECT
        e.employee_id,
        e.name,
        e.manager_id,
        st.visited || e.employee_id
    FROM employees e
    JOIN safe_tree st ON e.manager_id = st.employee_id
    WHERE e.employee_id != ALL(st.visited)  -- Skip if already visited
)
SELECT * FROM safe_tree;
```

Common Mistakes

Mistake 1: Forgetting RECURSIVE Keyword

```sql
-- BAD: Missing RECURSIVE keyword
WITH org AS (
    SELECT * FROM employees WHERE manager_id IS NULL
    UNION ALL
    SELECT e.* FROM employees e JOIN org ON e.manager_id = org.employee_id
    -- ERROR: relation "org" does not exist
)
SELECT * FROM org;

-- GOOD: Include RECURSIVE
WITH RECURSIVE org AS (
    ...
)
SELECT * FROM org;
```

Mistake 2: No Termination Condition

```sql
-- BAD: This will run forever!
WITH RECURSIVE nums AS (
    SELECT 1 AS n
    UNION ALL
    SELECT n + 1 FROM nums
    -- No WHERE clause to stop!
)
SELECT * FROM nums;

-- GOOD: Always have a stopping condition
WITH RECURSIVE nums AS (
    SELECT 1 AS n
    UNION ALL
    SELECT n + 1 FROM nums
    WHERE n < 100  -- Stop at 100
)
SELECT * FROM nums;
```

Mistake 3: Using UNION Instead of UNION ALL

```sql
-- CAUTION: UNION removes duplicates (slower, and may hide
-- legitimate duplicate paths in a graph)
-- Use UNION ALL unless you specifically need deduplication
WITH RECURSIVE tree AS (
    ...
    UNION ALL  -- Usually want UNION ALL for recursion
    ...
)
```

Mistake 4: Not Referencing the CTE in the Recursive Part

```sql
-- BAD: The recursive part must reference the CTE
WITH RECURSIVE tree AS (
    SELECT * FROM employees WHERE manager_id IS NULL
    UNION ALL
    SELECT * FROM employees WHERE manager_id IS NOT NULL
    -- This is not recursive! It just gets all rows with managers.
)
```

Best Practices

1. Use CTEs for readability. Any time you have more than two levels of nested subqueries, refactor into CTEs.
2. Give CTEs descriptive names. Use `monthly_totals` instead of `cte1`. Good names make the query self-documenting.
3. Always include a termination condition in recursive CTEs. Either filter by depth, use a WHERE clause, or track visited nodes.
4. Use UNION ALL (not UNION) in recursive CTEs unless you specifically need to eliminate duplicates. UNION ALL is faster and the correct choice for most tree traversals.
5. Test recursive CTEs on small data first. Infinite recursion on a large table can consume significant resources.
6. Consider PostgreSQL's `generate_series()` instead of recursive CTEs for simple number or date sequences.

Quick Summary

```
+---------------------+-----------------------------------------------+
| Concept             | What It Does                                  |
+---------------------+-----------------------------------------------+
| WITH ... AS         | Define a named temporary result (CTE)         |
| Multiple CTEs       | Chain CTEs separated by commas                |
| WITH RECURSIVE      | Enable self-referencing CTEs                  |
| Base case (anchor)  | Starting point of recursion                   |
| UNION ALL           | Combines base case with recursive results     |
| Recursive case      | Finds the next level by joining to the CTE    |
```

```
| Termination         | WHERE clause that stops the recursion        |
| Path tracking       | Build strings showing the path from root     |
| Depth tracking      | Count levels with a depth counter            |
+---------------------+----------------------------------------------+
```

Key Points

- A CTE is a named temporary result defined with WITH that exists only for one query.
- CTEs improve readability by turning nested subqueries into sequential named steps.
- Multiple CTEs can be chained with commas, each referencing previous ones.
- Recursive CTEs use WITH RECURSIVE and have two parts: a base case and a recursive case joined by UNION ALL.
- The base case provides the starting rows. The recursive case references the CTE itself to find the next level.
- Always include a termination condition to prevent infinite recursion.
- Recursive CTEs are ideal for hierarchies (org charts, category trees) and generating sequences.
- In PostgreSQL 12 and later, non-recursive CTEs are usually optimized the same as subqueries.

Practice Questions

1. What is the difference between a CTE and a subquery? When would you choose one over the other?
2. In a recursive CTE, what are the two required parts? What keyword connects them?
3. What happens if a recursive CTE has no termination condition? How can you protect against this?
4. Can one CTE reference another CTE defined before it in the same WITH clause? Can it reference one defined after it?
5. Why do recursive CTEs typically use UNION ALL instead of UNION?

Exercises

Exercise 1: Multi-Step Sales Analysis

Using CTEs, write a query that:

1. First CTE: Calculate the total sales per customer
2. Second CTE: Rank customers by their total sales
3. Final query: Show only the top 3 customers with their rank and total

Hint: Use two CTEs — one for aggregation and one for ranking.

Exercise 2: Full Category Breadcrumbs

Write a recursive CTE that generates the full breadcrumb path for every category in the categories table. The output should look like:

```
Electronics > Computers > Laptops > Gaming Laptops
```

Include only leaf categories (categories that have no children).

Hint: A leaf category is one whose category_id does not appear as any other category's parent_id.

Exercise 3: Chain of Command

Write a recursive CTE that, given a specific employee (like Chris Developer), shows the full chain of command from that employee up to the CEO. The output should list each person in the chain with their title and level.

Hint: This time, the recursion goes upward — start with the employee and follow manager_id up the tree.

What Is Next?

In Chapter 28, you will learn about stored procedures and functions. You will discover how to save programs directly in the database using PL/pgSQL — complete with variables, IF/ELSE logic, loops, and error handling. These are reusable programs that encapsulate business logic right where your data lives.

Chapter 28: Stored Procedures and Functions

What You Will Learn

In this chapter, you will learn:

- What stored procedures and functions are
- How to create functions in PostgreSQL using PL/pgSQL
- How to use parameters (IN, OUT, INOUT)
- How to declare and use variables
- How to write IF/ELSE conditional logic
- How to use loops (FOR, WHILE)
- How to debug with RAISE NOTICE
- How to call functions and handle errors
- How to build practical functions for business logic

Why This Chapter Matters

Imagine you run a restaurant. Every time a customer orders, the waiter has to remember a long sequence of steps: check if the dish is available, calculate the price with any discounts, update inventory, print the receipt. If you hire a new waiter, they have to learn all these steps from scratch.

Now imagine you create a recipe book that lives in the kitchen. Any waiter can follow it, and the steps are always done the same way. That is what a stored procedure is — a saved program that lives in the database. Instead of writing the same complex SQL over and over in your application code, you write it once as a function in the database.

Stored functions are essential for:

- Encapsulating business logic (like discount calculations)
- Ensuring consistency (everyone uses the same logic)
- Reducing network traffic (one function call instead of many queries)
- Security (grant access to the function, not the underlying tables)

Setting Up Our Practice Tables

```sql
CREATE TABLE products (
    product_id   SERIAL PRIMARY KEY,
    product_name VARCHAR(100) NOT NULL,
    price        DECIMAL(10, 2) NOT NULL,
    stock        INT NOT NULL DEFAULT 0
);

INSERT INTO products (product_name, price, stock) VALUES
('Laptop',   999.99, 50),
('Mouse',     29.99, 200),
('Keyboard',  79.99, 150),
('Monitor',  349.99, 75),
('Headphones', 149.99, 100);

CREATE TABLE accounts (
    account_id SERIAL PRIMARY KEY,
    owner_name VARCHAR(100) NOT NULL,
    balance    DECIMAL(12, 2) NOT NULL DEFAULT 0.00
);

INSERT INTO accounts (owner_name, balance) VALUES
('Alice', 5000.00),
('Bob',   3000.00),
('Charlie', 1500.00);

CREATE TABLE audit_log (
    log_id     SERIAL PRIMARY KEY,
    action     VARCHAR(50) NOT NULL,
    details    TEXT,
    created_at TIMESTAMP DEFAULT NOW()
);
```

What Is a Stored Function?

A stored function is a block of code that is saved in the database. You create it once, and then you can call it by name whenever you need it.

```
Without functions:                  With functions:
+---------------------------+       +---------------------------+
| Application Code          |       | Application Code          |
|                           |       |                           |
| query1 = "SELECT ..."     |       | CALL calculate_discount() |
| query2 = "UPDATE ..."     |       |                           |
| query3 = "INSERT ..."     |       | (one call does it all)    |
| query4 = "UPDATE ..."     |       +---------------------------+
| (repeat everywhere)       |
+---------------------------+       +---------------------------+
                                    | Database                  |
                                    | calculate_discount():     |
                                    |   SELECT ...              |
                                    |   UPDATE ...              |
                                    |   INSERT ...              |
                                    |   UPDATE ...              |
                                    +---------------------------+
```

PostgreSQL Note

In PostgreSQL, everything is a function (created with CREATE FUNCTION). PostgreSQL 11 added CREATE PROCEDURE for operations that do not return values, but functions are far more common and flexible. We will focus on CREATE FUNCTION in this chapter.

Your First Function

Let us create a simple function that calculates a discounted price:

```sql
CREATE OR REPLACE FUNCTION calculate_discount(
    original_price DECIMAL,
    discount_pct   DECIMAL
)
RETURNS DECIMAL
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN original_price * (1 - discount_pct / 100);
END;
$$;
```

Line-by-line explanation:

- `CREATE OR REPLACE FUNCTION` — creates the function (or updates it if it already exists)
- `calculate_discount` — the function name
- `original_price DECIMAL, discount_pct DECIMAL` — two input parameters
- `RETURNS DECIMAL` — the function returns a decimal number
- `LANGUAGE plpgsql` — we are using PL/pgSQL (PostgreSQL's procedural language)
- `AS $$ ... $$` — the dollar-quoted string contains the function body
- `BEGIN ... END` — the code block
- `RETURN` — sends the result back to the caller

Calling the Function

```sql
SELECT calculate_discount(100.00, 15);
```

```
 calculate_discount
--------------------
              85.00
(1 row)
```

Using It in Queries

```sql
SELECT
    product_name,
    price AS original_price,
    calculate_discount(price, 10) AS sale_price
FROM products;
```

```
 product_name | original_price | sale_price
--------------+----------------+-----------
 Laptop       |         999.99 |    899.99
 Mouse        |          29.99 |     26.99
 Keyboard     |          79.99 |     71.99
 Monitor      |         349.99 |    314.99
 Headphones   |         149.99 |    134.99
(5 rows)
```

Parameters: IN, OUT, and INOUT

IN Parameters (Default)

IN parameters pass values into the function. They are the default:

```sql
CREATE OR REPLACE FUNCTION get_product_info(
    p_product_id INT  -- IN is the default
)
RETURNS TABLE(name VARCHAR, price DECIMAL, stock INT)
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN QUERY
    SELECT product_name, p.price, p.stock
    FROM products p
    WHERE p.product_id = p_product_id;
END;
$$;
```

```sql
SELECT * FROM get_product_info(1);
```

```
  name   |  price   | stock
--------+---------+-------
 Laptop | 999.99  |    50
(1 row)
```

OUT Parameters

OUT parameters return values from the function. They are an alternative to RETURNS:

```sql
CREATE OR REPLACE FUNCTION get_account_summary(
    IN  p_account_id INT,
    OUT p_owner      VARCHAR,
    OUT p_balance    DECIMAL,
    OUT p_status     VARCHAR
)
LANGUAGE plpgsql
AS $$
BEGIN
    SELECT owner_name, balance
    INTO p_owner, p_balance
    FROM accounts
```

```
    WHERE account_id = p_account_id;

    -- Determine status based on balance
    IF p_balance >= 5000 THEN
        p_status := 'Gold';
    ELSIF p_balance >= 2000 THEN
        p_status := 'Silver';
    ELSE
        p_status := 'Bronze';
    END IF;
END;
$$;
```

```sql
SELECT * FROM get_account_summary(1);
```

```
 p_owner | p_balance | p_status
---------+-----------+----------
 Alice   |   5000.00 | Gold
(1 row)
```

Line-by-line explanation:

- `IN p_account_id INT` — input parameter (the account to look up)
- `OUT p_owner VARCHAR` — output parameter (will hold the owner's name)
- `OUT p_balance DECIMAL` — output parameter (will hold the balance)
- `OUT p_status VARCHAR` — output parameter (will hold the calculated status)
- No `RETURNS` clause needed — the OUT parameters define the return type
- `SELECT ... INTO p_owner, p_balance` — fills the OUT parameters from the query
- `p_status := 'Gold'` — the `:=` operator assigns a value

INOUT Parameters

INOUT parameters pass a value in and can be modified to pass a value back out:

```sql
CREATE OR REPLACE FUNCTION apply_tax(
    INOUT amount DECIMAL,
    IN    tax_rate DECIMAL DEFAULT 8.5
)
LANGUAGE plpgsql
AS $$
BEGIN
    amount := amount * (1 + tax_rate / 100);
END;
$$;
```

```sql
SELECT apply_tax(100.00);
```

```
 apply_tax
-----------
    108.50
(1 row)
```

```sql
SELECT apply_tax(100.00, 10);
```

```
 apply_tax
-----------
    110.00
(1 row)
```

Variables with DECLARE

You can declare local variables inside a function using the DECLARE section:

```sql
CREATE OR REPLACE FUNCTION calculate_order_total(
    p_product_id INT,
    p_quantity   INT
)
RETURNS DECIMAL
LANGUAGE plpgsql
AS $$
DECLARE
    v_price      DECIMAL;
    v_subtotal   DECIMAL;
    v_tax_rate   DECIMAL := 0.085;  -- 8.5% tax
    v_total      DECIMAL;
    v_product    VARCHAR;
BEGIN
    -- Get the product price
    SELECT price, product_name
    INTO v_price, v_product
    FROM products
    WHERE product_id = p_product_id;

    -- Check if product exists
    IF v_price IS NULL THEN
```

```
        RAISE EXCEPTION 'Product with ID % not found', p_product_id;
    END IF;

    -- Calculate totals
    v_subtotal := v_price * p_quantity;
    v_total    := v_subtotal * (1 + v_tax_rate);

    RAISE NOTICE 'Product: %, Qty: %, Subtotal: %, Total with tax: %',
                 v_product, p_quantity, v_subtotal, v_total;

    RETURN ROUND(v_total, 2);
END;
$$;
```

```sql
SELECT calculate_order_total(1, 2);
```

```
NOTICE:  Product: Laptop, Qty: 2, Subtotal: 1999.98, Total with tax: 2169.9783
 calculate_order_total
-----------------------
               2169.98
(1 row)
```

Line-by-line explanation of DECLARE:

- `DECLARE` — starts the variable declaration section
- `v_price DECIMAL` — declares a variable (no initial value, defaults to NULL)
- `v_tax_rate DECIMAL := 0.085` — declares and initializes a variable
- Variables are local to the function and exist only while it runs
- The `v_` prefix is a naming convention to distinguish variables from column names

IF/ELSE Conditional Logic

PL/pgSQL supports IF, ELSIF, and ELSE for branching:

```sql
CREATE OR REPLACE FUNCTION get_shipping_cost(
    p_order_total DECIMAL,
    p_is_member   BOOLEAN DEFAULT FALSE
)
RETURNS DECIMAL
LANGUAGE plpgsql
```

```
AS $$
DECLARE
    v_shipping DECIMAL;
BEGIN
    IF p_is_member THEN
        -- Members get free shipping on orders over $50
        IF p_order_total >= 50 THEN
            v_shipping := 0;
        ELSE
            v_shipping := 4.99;
        END IF;
    ELSE
        -- Non-members
        IF p_order_total >= 100 THEN
            v_shipping := 0;
        ELSIF p_order_total >= 50 THEN
            v_shipping := 7.99;
        ELSIF p_order_total >= 25 THEN
            v_shipping := 9.99;
        ELSE
            v_shipping := 12.99;
        END IF;
    END IF;

    RETURN v_shipping;
END;
$$;
```

```sql
SELECT
    get_shipping_cost(120, FALSE) AS non_member_120,
    get_shipping_cost(75, FALSE)  AS non_member_75,
    get_shipping_cost(30, TRUE)   AS member_30,
    get_shipping_cost(60, TRUE)   AS member_60;
```

```
 non_member_120 | non_member_75 | member_30 | member_60
----------------+---------------+-----------+-----------
           0.00 |          7.99 |      4.99 |      0.00
(1 row)
```

```
Decision Flow:

                   Is Member?
                  /          \
               Yes            No
              /                \
    Total >= 50?           Total >= 100?
    /        \             /          \
  Yes        No          Yes          No
  |           |           |            |
$0.00      $4.99       $0.00     Total >= 50?
```

```
                                  /        \
                                Yes        No
                                |           |
                              $7.99    Total >= 25?
                                        /       \
                                      Yes       No
                                       |         |
                                     $9.99    $12.99
```

Loops

FOR Loop — Iterating Over a Range

```sql
CREATE OR REPLACE FUNCTION generate_multiplication_table(
    p_number INT
)
RETURNS TABLE(expression TEXT, result INT)
LANGUAGE plpgsql
AS $$
BEGIN
    FOR i IN 1..10 LOOP
        expression := p_number || ' x ' || i;
        result := p_number * i;
        RETURN NEXT;  -- Add this row to the result set
    END LOOP;
END;
$$;
```

```sql
SELECT * FROM generate_multiplication_table(7);
```

```
 expression | result
------------+--------
 7 x 1      |      7
 7 x 2      |     14
 7 x 3      |     21
 7 x 4      |     28
 7 x 5      |     35
 7 x 6      |     42
 7 x 7      |     49
 7 x 8      |     56
 7 x 9      |     63
 7 x 10     |     70
(10 rows)
```

Line-by-line explanation:

- `FOR i IN 1..10 LOOP` — loop from 1 to 10, with `i` as the loop variable
- `RETURN NEXT` — adds the current row to the output (used with RETURNS TABLE)
- `END LOOP` — marks the end of the loop body

FOR Loop — Iterating Over Query Results

```sql
CREATE OR REPLACE FUNCTION list_low_stock(
    p_threshold INT DEFAULT 100
)
RETURNS TABLE(name VARCHAR, current_stock INT, needs INT)
LANGUAGE plpgsql
AS $$
DECLARE
    rec RECORD;
BEGIN
    FOR rec IN
        SELECT product_name, stock
        FROM products
        WHERE stock < p_threshold
        ORDER BY stock
    LOOP
        name := rec.product_name;
        current_stock := rec.stock;
        needs := p_threshold - rec.stock;
        RETURN NEXT;
    END LOOP;
END;
$$;
```

```sql
SELECT * FROM list_low_stock(100);
```

```
   name    | current_stock | needs
----------+---------------+-------
 Laptop   |            50 |    50
 Monitor  |            75 |    25
(2 rows)
```

WHILE Loop

```sql
CREATE OR REPLACE FUNCTION compound_interest(
    p_principal   DECIMAL,
```

```
    p_rate        DECIMAL,    -- Annual rate as percentage
    p_years       INT
)
RETURNS TABLE(year INT, balance DECIMAL)
LANGUAGE plpgsql
AS $$
DECLARE
    v_balance DECIMAL;
    v_year    INT := 0;
BEGIN
    v_balance := p_principal;

    WHILE v_year <= p_years LOOP
        year := v_year;
        balance := ROUND(v_balance, 2);
        RETURN NEXT;

        v_balance := v_balance * (1 + p_rate / 100);
        v_year := v_year + 1;
    END LOOP;
END;
$$;
```

```sql
SELECT * FROM compound_interest(1000, 5, 5);
```

```
 year | balance
------+---------
    0 | 1000.00
    1 | 1050.00
    2 | 1102.50
    3 | 1157.63
    4 | 1215.51
    5 | 1276.28
(6 rows)
```

RAISE NOTICE for Debugging

RAISE NOTICE prints messages to the console. It is your primary debugging tool:

```sql
CREATE OR REPLACE FUNCTION debug_example(p_value INT)
RETURNS INT
LANGUAGE plpgsql
AS $$
```

```
DECLARE
    v_result INT;
BEGIN
    RAISE NOTICE 'Function called with value: %', p_value;

    v_result := p_value * 2;
    RAISE NOTICE 'After doubling: %', v_result;

    v_result := v_result + 10;
    RAISE NOTICE 'After adding 10: %', v_result;

    RETURN v_result;
END;
$$;
```

```sql
SELECT debug_example(5);
```

```
NOTICE:  Function called with value: 5
NOTICE:  After doubling: 10
NOTICE:  After adding 10: 20
 debug_example
---------------
            20
(1 row)
```

RAISE Levels

```
+----------+-----------------------------------------------+
| Level    | Purpose                                       |
+----------+-----------------------------------------------+
| DEBUG    | Detailed information for debugging            |
| LOG      | Information logged to server log               |
| NOTICE   | Information sent to the client (most common)  |
| WARNING  | Warning message                               |
| EXCEPTION| Error — aborts the transaction!               |
+----------+-----------------------------------------------+
```

```sql
RAISE NOTICE 'This is informational';
RAISE WARNING 'This might be a problem';
RAISE EXCEPTION 'This stops execution and rolls back!';
```

Error Handling with EXCEPTION

You can catch and handle errors using the EXCEPTION block:

```sql
CREATE OR REPLACE FUNCTION safe_divide(
    p_numerator   DECIMAL,
    p_denominator DECIMAL
)
RETURNS DECIMAL
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN p_numerator / p_denominator;
EXCEPTION
    WHEN division_by_zero THEN
        RAISE NOTICE 'Division by zero attempted. Returning NULL.';
        RETURN NULL;
    WHEN OTHERS THEN
        RAISE NOTICE 'Unexpected error: %', SQLERRM;
        RETURN NULL;
END;
$$;
```

```sql
SELECT safe_divide(10, 3);
```

```
 safe_divide
--------------
 3.3333333333
(1 row)
```

```sql
SELECT safe_divide(10, 0);
```

```
NOTICE:  Division by zero attempted. Returning NULL.
 safe_divide
-------------

(1 row)
```

Line-by-line explanation:

- `EXCEPTION` — starts the error-handling section (like try/catch in other languages)
- `WHEN division_by_zero THEN` — catches a specific error type
- `WHEN OTHERS THEN` — catches any error not handled above
- `SQLERRM` — a built-in variable containing the error message

Practical: Transfer Funds Function

Let us build a complete, production-ready fund transfer function:

```sql
CREATE OR REPLACE FUNCTION transfer_funds(
    p_from_account INT,
    p_to_account   INT,
    p_amount       DECIMAL
)
RETURNS TEXT
LANGUAGE plpgsql
AS $$
DECLARE
    v_from_balance DECIMAL;
    v_from_owner   VARCHAR;
    v_to_owner     VARCHAR;
    v_rows         INT;
BEGIN
    -- Validate amount
    IF p_amount <= 0 THEN
        RAISE EXCEPTION 'Transfer amount must be positive. Got: %', p_amount;
    END IF;

    IF p_from_account = p_to_account THEN
        RAISE EXCEPTION 'Cannot transfer to the same account';
    END IF;

    -- Lock both accounts in consistent order to prevent deadlocks
    SELECT owner_name, balance
    INTO v_from_owner, v_from_balance
    FROM accounts
    WHERE account_id = p_from_account
    FOR UPDATE;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'Source account % not found', p_from_account;
    END IF;

    SELECT owner_name
    INTO v_to_owner
    FROM accounts
    WHERE account_id = p_to_account
    FOR UPDATE;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'Destination account % not found', p_to_account;
    END IF;
```

```
    -- Check sufficient funds
    IF v_from_balance < p_amount THEN
        RAISE EXCEPTION 'Insufficient funds. Account % has %, needs %',
                        p_from_account, v_from_balance, p_amount;
    END IF;

    -- Perform the transfer
    UPDATE accounts
    SET balance = balance - p_amount
    WHERE account_id = p_from_account;

    UPDATE accounts
    SET balance = balance + p_amount
    WHERE account_id = p_to_account;

    -- Log the transfer
    INSERT INTO audit_log (action, details)
    VALUES (
        'TRANSFER',
        FORMAT('$%s from %s (acct %s) to %s (acct %s)',
               p_amount, v_from_owner, p_from_account,
               v_to_owner, p_to_account)
    );

    RETURN FORMAT('Success: $%s transferred from %s to %s',
                  p_amount, v_from_owner, v_to_owner);

EXCEPTION
    WHEN OTHERS THEN
        -- Log the error
        INSERT INTO audit_log (action, details)
        VALUES ('TRANSFER_FAILED', SQLERRM);
        RAISE;  -- Re-raise the exception
END;
$$;
```

Using the Transfer Function

```sql
SELECT transfer_funds(1, 2, 500.00);
```

```
                   transfer_funds
--------------------------------------------------
 Success: $500.00 transferred from Alice to Bob
(1 row)
```

```sql
SELECT account_id, owner_name, balance FROM accounts;
```

```
 account_id | owner_name | balance
------------+------------+---------
          3 | Charlie    | 1500.00
          1 | Alice      | 4500.00
          2 | Bob        | 3500.00
(3 rows)
```

```sql
SELECT * FROM audit_log;
```

```
 log_id |  action  |                        details                        |         cre...
--------+----------+-------------------------------------------------------+------------...
      1 | TRANSFER | $500.00 from Alice (acct 1) to Bob (acct 2)           | 2024-01-15 ...
(1 row)
```

Testing Error Handling

```sql
SELECT transfer_funds(1, 2, 99999.00);
```

```
ERROR:  Insufficient funds. Account 1 has 4500.00, needs 99999.00
```

```sql
SELECT transfer_funds(1, 99, 100.00);
```

```
ERROR:  Destination account 99 not found
```

Practical: Generate Report Function

A function that generates a product inventory report:

```sql
CREATE OR REPLACE FUNCTION generate_inventory_report()
RETURNS TABLE(
    report_line TEXT
)
LANGUAGE plpgsql
AS $$
```

```
DECLARE
    v_total_products INT;
    v_total_value    DECIMAL;
    v_low_stock      INT;
    rec              RECORD;
BEGIN
    -- Header
    report_line := '=== INVENTORY REPORT ===';
    RETURN NEXT;
    report_line := 'Generated: ' || NOW()::DATE;
    RETURN NEXT;
    report_line := '';
    RETURN NEXT;

    -- Summary statistics
    SELECT COUNT(*), SUM(price * stock)
    INTO v_total_products, v_total_value
    FROM products;

    SELECT COUNT(*)
    INTO v_low_stock
    FROM products
    WHERE stock < 100;

    report_line := 'Total Products: ' || v_total_products;
    RETURN NEXT;
    report_line := 'Total Inventory Value: $' || TO_CHAR(v_total_value, 'FM999,999.99');
    RETURN NEXT;
    report_line := 'Low Stock Items: ' || v_low_stock;
    RETURN NEXT;
    report_line := '';
    RETURN NEXT;

    -- Detail section
    report_line := '--- PRODUCT DETAILS ---';
    RETURN NEXT;
    report_line := RPAD('Product', 15) || RPAD('Price', 12) ||
                   RPAD('Stock', 8) || 'Value';
    RETURN NEXT;
    report_line := REPEAT('-', 50);
    RETURN NEXT;

    FOR rec IN
        SELECT product_name, price, stock, (price * stock) AS value
        FROM products
        ORDER BY price * stock DESC
    LOOP
        report_line := RPAD(rec.product_name, 15) ||
                       RPAD('$' || rec.price, 12) ||
                       RPAD(rec.stock::TEXT, 8) ||
                       '$' || TO_CHAR(rec.value, 'FM999,999.99');
        RETURN NEXT;
    END LOOP;

    report_line := REPEAT('-', 50);
```

```
    RETURN NEXT;
    report_line := '=== END OF REPORT ===';
    RETURN NEXT;
END;
$$;
```

```sql
SELECT * FROM generate_inventory_report();
```

```
                report_line
----------------------------------------------------
 === INVENTORY REPORT ===
 Generated: 2024-01-15

 Total Products: 5
 Total Inventory Value: $107,645.50
 Low Stock Items: 2

 --- PRODUCT DETAILS ---
 Product        Price       Stock   Value
 ----------------------------------------------------
 Laptop         $999.99     50      $49,999.50
 Monitor        $349.99     75      $26,249.25
 Headphones     $149.99     100     $14,999.00
 Keyboard       $79.99      150     $11,998.50
 Mouse          $29.99      200     $5,998.00
 ----------------------------------------------------
 === END OF REPORT ===
(16 rows)
```

Dropping Functions

To remove a function:

```sql
-- Drop a function (must match the parameter types)
DROP FUNCTION calculate_discount(DECIMAL, DECIMAL);

-- Drop if it exists (no error if it does not)
DROP FUNCTION IF EXISTS calculate_discount(DECIMAL, DECIMAL);
```

Viewing Existing Functions

```sql
-- List all user-defined functions in the current schema
SELECT
    routine_name,
    routine_type,
    data_type AS return_type
FROM information_schema.routines
WHERE routine_schema = 'public'
ORDER BY routine_name;
```

```
       routine_name        | routine_type | return_type
---------------------------+--------------+------------
 apply_tax                 | FUNCTION     | numeric
 calculate_discount        | FUNCTION     | numeric
 calculate_order_total     | FUNCTION     | numeric
 compound_interest         | FUNCTION     | record
 generate_inventory_report | FUNCTION     | record
 get_account_summary       | FUNCTION     | record
 transfer_funds            | FUNCTION     | text
(7 rows)
```

Common Mistakes

Mistake 1: Variable Name Conflicts with Column Names

```sql
-- BAD: Variable 'name' conflicts with column 'name'
CREATE OR REPLACE FUNCTION bad_example(name VARCHAR)
RETURNS INT
LANGUAGE plpgsql
AS $$
BEGIN
    -- Does this compare to the parameter or the column?
    RETURN (SELECT COUNT(*) FROM products WHERE product_name = name);
END;
$$;

-- GOOD: Prefix parameters with p_ and variables with v_
CREATE OR REPLACE FUNCTION good_example(p_name VARCHAR)
RETURNS INT
LANGUAGE plpgsql
AS $$
```

```
BEGIN
    RETURN (SELECT COUNT(*) FROM products WHERE product_name = p_name);
END;
$$;
```

Mistake 2: Forgetting IF NOT FOUND

```sql
-- BAD: Does not check if the SELECT found anything
SELECT price INTO v_price FROM products WHERE product_id = p_id;
-- v_price is NULL if not found, which may cause silent bugs

-- GOOD: Always check
SELECT price INTO v_price FROM products WHERE product_id = p_id;
IF NOT FOUND THEN
    RAISE EXCEPTION 'Product % not found', p_id;
END IF;
```

Mistake 3: Using RETURN in a RETURNS TABLE Function

```sql
-- BAD: RETURN exits the function immediately
-- Use RETURN NEXT to add rows, then the function ends naturally

-- GOOD pattern for RETURNS TABLE functions:
--   Set column values, then call RETURN NEXT for each row
--   The function returns all accumulated rows when it ends
```

Mistake 4: Not Using CREATE OR REPLACE

```sql
-- BAD: CREATE FUNCTION fails if function already exists
CREATE FUNCTION my_func() ...

-- GOOD: CREATE OR REPLACE updates the function if it exists
CREATE OR REPLACE FUNCTION my_func() ...
```

Best Practices

1. Use naming conventions. Prefix parameters with `p_` and local variables with `v_`. This prevents ambiguity with column names.

2. Always validate inputs. Check for NULL values, negative numbers, and invalid IDs at the start of the function.
3. Use CREATE OR REPLACE. This makes it easy to update functions during development without dropping them first.
4. Keep functions focused. Each function should do one thing well. A function that calculates a discount should not also send an email.
5. Use RAISE NOTICE during development for debugging. Remove or change to DEBUG level before deploying to production.
6. Handle errors with EXCEPTION blocks. Catch specific exceptions when possible, and always provide helpful error messages.
7. Document your functions with comments explaining what they do, what parameters they expect, and what they return.
8. Use transactions within functions when multiple tables need to be updated together (functions run inside the caller's transaction by default).

Quick Summary

```
+-----------------------+-----------------------------------------------+
| Concept               | What It Does                                  |
+-----------------------+-----------------------------------------------+
| CREATE FUNCTION       | Creates a new stored function                 |
| CREATE OR REPLACE     | Creates or updates a function                 |
| RETURNS type          | Declares the return type                      |
| RETURNS TABLE(...)    | Returns multiple columns/rows                 |
| LANGUAGE plpgsql      | Uses PostgreSQL's procedural language         |
| DECLARE               | Section for declaring local variables         |
| :=                    | Assignment operator                           |
| IF/ELSIF/ELSE/END IF  | Conditional logic                             |
| FOR i IN 1..10 LOOP   | Loop over a range                             |
| FOR rec IN query LOOP | Loop over query results                       |
| WHILE condition LOOP  | Loop while condition is true                  |
| RETURN                | Return a single value and exit                |
| RETURN NEXT           | Add a row to the result set (RETURNS TABLE)   |
| RETURN QUERY          | Add query results to the result set           |
| RAISE NOTICE          | Print a debug message                         |
| RAISE EXCEPTION       | Throw an error (aborts transaction)           |
| EXCEPTION WHEN        | Catch and handle errors                       |
| NOT FOUND             | True when SELECT INTO finds no rows           |
| DROP FUNCTION         | Remove a function                             |
+-----------------------+-----------------------------------------------+
```

Key Points

- Stored functions are saved programs in the database created with CREATE FUNCTION.
- PL/pgSQL is PostgreSQL's procedural language for writing functions with variables, logic, and loops.
- Parameters can be IN (input), OUT (output), or INOUT (both).
- DECLARE lets you define local variables with types and optional default values.
- Use IF/ELSIF/ELSE for conditional logic and FOR/WHILE for loops.
- RAISE NOTICE is the primary debugging tool. RAISE EXCEPTION stops execution and rolls back.
- RETURNS TABLE lets functions return multiple rows and columns.
- RETURN NEXT adds one row at a time to the result set.
- Always prefix parameters with `p_` and variables with `v_` to avoid name conflicts with column names.
- Functions run inside the caller's transaction by default.

Practice Questions

1. What is the difference between RETURN and RETURN NEXT? When do you use each?
2. What happens when RAISE EXCEPTION is called inside a function? What happens to any changes the function already made?
3. Why should you prefix parameter names with `p_` and variable names with `v_`? What problem does this solve?
4. What does the NOT FOUND special variable tell you? When is it set?
5. How do you update an existing function without dropping it first?

Exercises

Exercise 1: Tiered Discount Function

Create a function called `tiered_discount` that takes a product price and quantity, then returns the discounted total:

- 1-9 items: no discount

- 10-49 items: 5% discount
- 50-99 items: 10% discount
- 100+ items: 15% discount

Include input validation (price must be positive, quantity must be at least 1).

Exercise 2: Account Statement Function

Create a function called `account_statement` that takes an account_id and returns a formatted statement showing:

- Account holder name
- Current balance
- Account status (Gold/Silver/Bronze based on balance)
- The last 5 transactions from the audit_log related to that account

Use RETURNS TABLE and RETURN NEXT to build the output.

Exercise 3: Stock Reorder Function

Create a function called `reorder_stock` that:

1. Takes a minimum stock threshold as input
2. Finds all products below that threshold
3. Calculates how many units to order to bring stock up to the threshold
4. Inserts a record into an `orders_placed` table for each product
5. Returns a summary of what was ordered
6. Uses RAISE NOTICE to show progress during execution

What Is Next?

In Chapter 29, you will learn about triggers — automatic actions that fire when data changes. Triggers use the same PL/pgSQL language you just learned, but they run automatically in response to INSERT, UPDATE, or DELETE operations. They are perfect for audit logging, automatic timestamps, and data validation.

Chapter 29: Triggers — Automatic Actions When Data Changes

What You Will Learn

In this chapter, you will learn:

- What triggers are and when to use them
- The difference between BEFORE and AFTER triggers
- How triggers respond to INSERT, UPDATE, and DELETE
- How to write trigger functions
- How to use the NEW and OLD records
- The difference between FOR EACH ROW and FOR EACH STATEMENT
- How to build practical triggers for audit logging, timestamps, validation, and deletion prevention

Why This Chapter Matters

Imagine a security camera system in a building. You do not have to remember to press "record" every time someone walks through the door. The camera detects motion and automatically starts recording. It captures who entered, when they entered, and through which door.

Triggers work the same way in a database. They are automatic actions that fire whenever data changes. When someone inserts a new row, updates an existing row, or deletes a row, a trigger can automatically log the change, update a timestamp, validate the data, or even prevent the operation entirely.

Without triggers, you would have to remember to add audit logging in every piece of application code that modifies data. Miss one spot, and you have a gap in your audit trail. With triggers, the database handles it automatically — no matter where the change comes from.

Setting Up Our Practice Tables

```sql
CREATE TABLE employees (
    employee_id SERIAL PRIMARY KEY,
    name        VARCHAR(100) NOT NULL,
    department  VARCHAR(50) NOT NULL,
    salary      DECIMAL(10, 2) NOT NULL,
    email       VARCHAR(150),
    created_at  TIMESTAMP DEFAULT NOW(),
    updated_at  TIMESTAMP DEFAULT NOW()
);

INSERT INTO employees (name, department, salary, email) VALUES
('Alice Johnson',  'Engineering', 95000, 'alice@company.com'),
('Bob Smith',      'Sales',       75000, 'bob@company.com'),
('Charlie Brown',  'Marketing',   70000, 'charlie@company.com');

CREATE TABLE products (
    product_id   SERIAL PRIMARY KEY,
    product_name VARCHAR(100) NOT NULL,
    price        DECIMAL(10, 2) NOT NULL,
    min_price    DECIMAL(10, 2) NOT NULL DEFAULT 0,
    created_at   TIMESTAMP DEFAULT NOW(),
    updated_at   TIMESTAMP DEFAULT NOW()
);

INSERT INTO products (product_name, price, min_price) VALUES
('Laptop',     999.99, 799.99),
('Mouse',       29.99,  19.99),
('Keyboard',    79.99,  59.99);

CREATE TABLE audit_log (
    log_id      SERIAL PRIMARY KEY,
    table_name  VARCHAR(50) NOT NULL,
    action      VARCHAR(10) NOT NULL,
    record_id   INT,
    old_values  JSONB,
    new_values  JSONB,
    changed_by  VARCHAR(100) DEFAULT CURRENT_USER,
    changed_at  TIMESTAMP DEFAULT NOW()
);

CREATE TABLE important_records (
    record_id  SERIAL PRIMARY KEY,
    title      VARCHAR(100) NOT NULL,
    content    TEXT,
    is_locked  BOOLEAN DEFAULT FALSE,
    created_at TIMESTAMP DEFAULT NOW()
```

```
);

INSERT INTO important_records (title, content, is_locked) VALUES
('Company Policy', 'No changes allowed to this document', TRUE),
('Draft Memo', 'Work in progress', FALSE),
('Annual Report', 'Final version - locked', TRUE);
```

What Is a Trigger?

A trigger is a function that runs automatically when a specific event (INSERT, UPDATE, or DELETE) happens on a table. You do not call it — the database calls it for you.

```
Application
    |
    | INSERT INTO employees ...
    v
+-------------------+
|  employees table |
+-------------------+
    |
    | (trigger fires automatically)
    v
+-------------------+
| Trigger Function |
|  - Log change    |
|  - Validate data |
|  - Update fields |
+-------------------+
```

Two Parts of a Trigger

A trigger has two components:

1. Trigger function — the code that runs (what to do)
2. Trigger definition — when and where to run it (when to do it)

```
Step 1: Create the function       Step 2: Attach it to a table
+----------------------------+     +----------------------------+
| CREATE FUNCTION            |     | CREATE TRIGGER             |
|   my_trigger_func()        |     |   BEFORE INSERT            |
| RETURNS TRIGGER            |     |   ON employees             |
| ...                        |     |   FOR EACH ROW             |
+----------------------------+     |   EXECUTE FUNCTION         |
```

```
                                    |   my_trigger_func();       |
                                    +----------------------------+
```

BEFORE vs AFTER Triggers

```
+--------+-----------------------------------------------------+
| Timing | What It Means                                       |
+--------+-----------------------------------------------------+
| BEFORE | Runs before the data change is applied              |
|        | Can modify the incoming data (NEW record)           |
|        | Can cancel the operation (return NULL)               |
+--------+-----------------------------------------------------+
| AFTER  | Runs after the data change is applied               |
|        | Cannot modify the data (it is already saved)        |
|        | Good for logging, notifications, cascade updates    |
+--------+-----------------------------------------------------+
```

```
Timeline of an INSERT:

  BEFORE trigger       INSERT happens       AFTER trigger
       |                    |                    |
       v                    v                    v
  +---------+         +-----------+         +---------+
  | Validate|         | Row saved |         | Log the |
  | or      |  --->   | to table  |  --->   | change  |
  | modify  |         |           |         |         |
  +---------+         +-----------+         +---------+

  Can change             Data is              Data is
  or cancel              committed            committed
  the data               to table             (read only)
```

NEW and OLD Records

Inside a trigger function, you have access to special variables that represent the data:

```
+-----------+--------+--------+--------+
| Operation | NEW    | OLD    | Notes  |
```

```
+-----------+--------+--------+--------+
| INSERT    | Yes    | No     | NEW = the row being inserted       |
| UPDATE    | Yes    | Yes    | OLD = before, NEW = after          |
| DELETE    | No     | Yes    | OLD = the row being deleted        |
+-----------+--------+--------+--------+
```

- NEW — the new version of the row (available in INSERT and UPDATE)
- OLD — the old version of the row (available in UPDATE and DELETE)

Practical 1: Automatic updated_at Timestamp

One of the most common triggers. Every time a row is updated, automatically set the `updated_at` column to the current time.

Step 1: Create the Trigger Function

```sql
CREATE OR REPLACE FUNCTION set_updated_at()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    NEW.updated_at := NOW();
    RETURN NEW;
END;
$$;
```

Line-by-line explanation:

- `RETURNS TRIGGER` — this function is specifically for use as a trigger
- `NEW.updated_at := NOW()` — modify the `updated_at` field of the incoming row
- `RETURN NEW` — return the modified row (required for BEFORE triggers)

Step 2: Create the Trigger

```sql
CREATE TRIGGER trg_employees_updated_at
    BEFORE UPDATE ON employees
    FOR EACH ROW
    EXECUTE FUNCTION set_updated_at();
```

Line-by-line explanation:

- `CREATE TRIGGER trg_employees_updated_at` — name the trigger (prefix with `trg_`)
- `BEFORE UPDATE` — fire before an UPDATE is applied
- `ON employees` — attach to the employees table
- `FOR EACH ROW` — fire once for each row being updated
- `EXECUTE FUNCTION set_updated_at()` — call this function

Testing It

```sql
-- Check current timestamps
SELECT name, updated_at FROM employees WHERE employee_id = 1;
```

```
     name      |         updated_at
---------------+----------------------------
 Alice Johnson | 2024-01-15 10:00:00.000000
(1 row)
```

```sql
-- Update Alice's salary
UPDATE employees SET salary = 100000 WHERE employee_id = 1;

-- Check again - updated_at changed automatically!
SELECT name, salary, updated_at FROM employees WHERE employee_id = 1;
```

```
     name      |  salary   |         updated_at
---------------+-----------+----------------------------
 Alice Johnson | 100000.00 | 2024-01-15 14:35:22.123456
(1 row)
```

The `updated_at` timestamp was automatically set to the current time, even though our UPDATE statement only changed the salary.

Reusing the Same Function on Multiple Tables

```sql
-- Same function works for any table with an updated_at column
CREATE TRIGGER trg_products_updated_at
    BEFORE UPDATE ON products
    FOR EACH ROW
    EXECUTE FUNCTION set_updated_at();
```

Practical 2: Audit Logging Trigger

Let us create a comprehensive audit log that records every change to the employees table:

The Trigger Function

```sql
CREATE OR REPLACE FUNCTION audit_employee_changes()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    IF TG_OP = 'INSERT' THEN
        INSERT INTO audit_log (table_name, action, record_id, new_values)
        VALUES (
            TG_TABLE_NAME,
            'INSERT',
            NEW.employee_id,
            jsonb_build_object(
                'name', NEW.name,
                'department', NEW.department,
                'salary', NEW.salary,
                'email', NEW.email
            )
        );
        RETURN NEW;

    ELSIF TG_OP = 'UPDATE' THEN
        INSERT INTO audit_log (table_name, action, record_id,
                               old_values, new_values)
        VALUES (
            TG_TABLE_NAME,
            'UPDATE',
            NEW.employee_id,
            jsonb_build_object(
                'name', OLD.name,
                'department', OLD.department,
                'salary', OLD.salary,
                'email', OLD.email
            ),
            jsonb_build_object(
                'name', NEW.name,
                'department', NEW.department,
                'salary', NEW.salary,
                'email', NEW.email
            )
        );
        RETURN NEW;

    ELSIF TG_OP = 'DELETE' THEN
        INSERT INTO audit_log (table_name, action, record_id, old_values)
        VALUES (
```

```
            TG_TABLE_NAME,
            'DELETE',
            OLD.employee_id,
            jsonb_build_object(
                'name', OLD.name,
                'department', OLD.department,
                'salary', OLD.salary,
                'email', OLD.email
            )
        );
        RETURN OLD;
    END IF;
END;
$$;
```

Key variables explained:

- `TG_OP` — the operation that fired the trigger ('INSERT', 'UPDATE', or 'DELETE')
- `TG_TABLE_NAME` — the name of the table the trigger is attached to
- `jsonb_build_object(...)` — creates a JSON object from key-value pairs

Attach the Trigger

```sql
CREATE TRIGGER trg_audit_employees
    AFTER INSERT OR UPDATE OR DELETE ON employees
    FOR EACH ROW
    EXECUTE FUNCTION audit_employee_changes();
```

Notice: `AFTER INSERT OR UPDATE OR DELETE` — this single trigger fires on all three operations.

Testing the Audit Log

```sql
-- Insert a new employee
INSERT INTO employees (name, department, salary, email)
VALUES ('Diana Prince', 'Engineering', 110000, 'diana@company.com');

-- Update an employee
UPDATE employees SET salary = 80000 WHERE employee_id = 2;

-- Delete an employee
DELETE FROM employees WHERE employee_id = 3;

-- Check the audit log
SELECT
    log_id,
```

```
    action,
    record_id,
    old_values,
    new_values,
    changed_at::DATE
FROM audit_log
ORDER BY log_id;
```

```
 log_id | action | record_id |              old_values              |              new_...
--------+--------+-----------+--------------------------------------+------------------...
      1 | INSERT |         4 |                                      | {"name": "Diana P...
      2 | UPDATE |         2 | {"name": "Bob Smith", "salary": ...} | {"name": "Bob Smi...
      3 | DELETE |         3 | {"name": "Charlie Brown", ...}       |                  ...
(3 rows)
```

Every change is automatically recorded with the old and new values, who made the change, and when.

Practical 3: Data Validation Trigger

A trigger that enforces business rules that cannot be expressed with simple CHECK constraints:

```sql
CREATE OR REPLACE FUNCTION validate_product_price()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    -- Rule 1: Price cannot be negative
    IF NEW.price < 0 THEN
        RAISE EXCEPTION 'Price cannot be negative. Got: %', NEW.price;
    END IF;

    -- Rule 2: Price cannot be below minimum price
    IF NEW.price < NEW.min_price THEN
        RAISE EXCEPTION 'Price (%) cannot be below minimum price (%)',
                        NEW.price, NEW.min_price;
    END IF;

    -- Rule 3: Price increases cannot exceed 50% at once
    IF TG_OP = 'UPDATE' AND OLD.price > 0 THEN
        IF NEW.price > OLD.price * 1.5 THEN
            RAISE EXCEPTION
```

```
                'Price increase too large. Old: %, New: %, Max allowed: %',
                OLD.price, NEW.price, OLD.price * 1.5;
        END IF;
    END IF;

    -- Rule 4: Normalize product name to title case
    NEW.product_name := INITCAP(TRIM(NEW.product_name));

    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_validate_product
    BEFORE INSERT OR UPDATE ON products
    FOR EACH ROW
    EXECUTE FUNCTION validate_product_price();
```

Testing Validation

```sql
-- Try to set a negative price
UPDATE products SET price = -10 WHERE product_id = 1;
```

```
ERROR:  Price cannot be negative. Got: -10
```

```sql
-- Try to set price below minimum
UPDATE products SET price = 500 WHERE product_id = 1;
```

```
ERROR:  Price (500) cannot be below minimum price (799.99)
```

Wait — that is not right. The min_price for the Laptop is 799.99, so 500 is indeed below minimum. The trigger caught it.

```sql
-- Try an excessive price increase (Laptop is currently 999.99)
UPDATE products SET price = 1800 WHERE product_id = 1;
```

```
ERROR:  Price increase too large. Old: 999.99, New: 1800, Max allowed: 1499.985
```

```sql
-- Valid price update
UPDATE products SET price = 1099.99 WHERE product_id = 1;

SELECT product_name, price FROM products WHERE product_id = 1;
```

```
 product_name |  price
--------------+---------
 Laptop       | 1099.99
(1 row)
```

```sql
-- Test name normalization
INSERT INTO products (product_name, price, min_price)
VALUES ('  WIRELESS   headphones  ', 149.99, 99.99);

SELECT product_name, price FROM products WHERE product_id = 4;
```

```
    product_name     |  price
---------------------+---------
 Wireless Headphones | 149.99
(1 row)
```

The trigger automatically cleaned up the product name.

Practical 4: Preventing Deletion

Sometimes you want to prevent certain rows from being deleted:

```sql
CREATE OR REPLACE FUNCTION prevent_locked_deletion()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    IF OLD.is_locked THEN
        RAISE EXCEPTION
            'Cannot delete locked record: "%" (ID: %). Unlock it first.',
            OLD.title, OLD.record_id;
    END IF;

    RETURN OLD;
END;
$$;
```

```
CREATE TRIGGER trg_prevent_delete_locked
    BEFORE DELETE ON important_records
    FOR EACH ROW
    EXECUTE FUNCTION prevent_locked_deletion();
```

Testing Deletion Prevention

```sql
-- Try to delete a locked record
DELETE FROM important_records WHERE record_id = 1;
```

```
ERROR:  Cannot delete locked record: "Company Policy" (ID: 1). Unlock it first.
```

```sql
-- Delete an unlocked record - works fine
DELETE FROM important_records WHERE record_id = 2;
```

```
DELETE 1
```

```sql
-- Unlock, then delete
UPDATE important_records SET is_locked = FALSE WHERE record_id = 1;
DELETE FROM important_records WHERE record_id = 1;
```

```
DELETE 1
```

FOR EACH ROW vs FOR EACH STATEMENT

```
+-------------------+-----------------------------------------------+
| Type              | When It Fires                                 |
+-------------------+-----------------------------------------------+
| FOR EACH ROW      | Once for every row affected                   |
|                   | Has access to NEW and OLD                     |
|                   | Most common choice                            |
+-------------------+-----------------------------------------------+
| FOR EACH STATEMENT| Once for the entire statement                 |
|                   | Even if zero rows are affected                |
```

```
|                   | No access to NEW and OLD                     |
|                   | Good for logging that an operation happened  |
+-------------------+----------------------------------------------+
```

Example

```sql
-- UPDATE employees SET salary = salary + 1000;
-- If this affects 5 rows:

-- FOR EACH ROW trigger fires:    5 times (once per row)
-- FOR EACH STATEMENT trigger:    1 time  (once total)
```

Statement-Level Trigger Example

```sql
CREATE OR REPLACE FUNCTION log_bulk_operation()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    INSERT INTO audit_log (table_name, action, record_id, new_values)
    VALUES (
        TG_TABLE_NAME,
        TG_OP || '_STATEMENT',
        NULL,
        jsonb_build_object('note', 'Bulk operation performed')
    );
    RETURN NULL;  -- Return value is ignored for AFTER triggers
END;
$$;

CREATE TRIGGER trg_log_bulk_employees
    AFTER INSERT OR UPDATE OR DELETE ON employees
    FOR EACH STATEMENT
    EXECUTE FUNCTION log_bulk_operation();
```

Conditional Triggers with WHEN

You can add a condition to a trigger so it only fires when certain criteria are met:

```sql
-- Only fire when salary actually changes
CREATE TRIGGER trg_salary_change_audit
    AFTER UPDATE ON employees
    FOR EACH ROW
    WHEN (OLD.salary IS DISTINCT FROM NEW.salary)
    EXECUTE FUNCTION audit_employee_changes();
```

This trigger will not fire if you update the employee's name or department — only when the salary changes.

Column-Specific Triggers

You can also limit triggers to specific columns:

```sql
-- Only fire when salary or department columns are updated
CREATE TRIGGER trg_compensation_change
    BEFORE UPDATE OF salary, department ON employees
    FOR EACH ROW
    EXECUTE FUNCTION audit_employee_changes();
```

Trigger Execution Order

When multiple triggers exist on the same table and event:

```
1. BEFORE statement triggers
2. BEFORE row triggers (for each affected row)
3. The actual data modification
4. AFTER row triggers (for each affected row)
5. AFTER statement triggers

Within the same timing/level, triggers fire in alphabetical order by name.
```

Managing Triggers

Viewing Triggers

```sql
SELECT
    trigger_name,
    event_manipulation AS event,
    action_timing AS timing,
    action_orientation AS level
FROM information_schema.triggers
WHERE trigger_schema = 'public'
ORDER BY event_object_table, trigger_name;
```

```
        trigger_name         |  event | timing | level
-----------------------------+--------+--------+-----------
 trg_audit_employees         | INSERT | AFTER  | ROW
 trg_audit_employees         | UPDATE | AFTER  | ROW
 trg_audit_employees         | DELETE | AFTER  | ROW
 trg_employees_updated_at    | UPDATE | BEFORE | ROW
 trg_prevent_delete_locked   | DELETE | BEFORE | ROW
 trg_validate_product        | INSERT | BEFORE | ROW
 trg_validate_product        | UPDATE | BEFORE | ROW
(7 rows)
```

Disabling and Enabling Triggers

```sql
-- Disable a specific trigger
ALTER TABLE employees DISABLE TRIGGER trg_audit_employees;

-- Re-enable it
ALTER TABLE employees ENABLE TRIGGER trg_audit_employees;

-- Disable ALL triggers on a table (useful for bulk data loads)
ALTER TABLE employees DISABLE TRIGGER ALL;

-- Re-enable ALL triggers
ALTER TABLE employees ENABLE TRIGGER ALL;
```

Dropping Triggers

```sql
-- Drop a trigger
DROP TRIGGER trg_audit_employees ON employees;

-- Drop if exists
DROP TRIGGER IF EXISTS trg_audit_employees ON employees;
```

Note: Dropping a trigger does not drop the trigger function. You must drop the function separately if you no longer need it.

A Complete Trigger Workflow

Here is a typical workflow showing BEFORE and AFTER triggers working together on an employee insert:

```
INSERT INTO employees (name, department, salary, email)
VALUES ('New Person', 'engineering', 85000, 'NEW@COMPANY.COM');

    |
    v
BEFORE INSERT trigger (set_updated_at):
    - Sets updated_at to NOW()
    |
    v
BEFORE INSERT trigger (validate_and_normalize):
    - Could normalize department to 'Engineering'
    - Could lowercase the email
    - Returns the modified NEW record
    |
    v
Actual INSERT happens (with modified data)
    |
    v
AFTER INSERT trigger (audit_employee_changes):
    - Records the insert in audit_log
    - NEW record has the final, saved values
    |
    v
Done! One INSERT triggered three automatic actions.
```

Common Mistakes

Mistake 1: Forgetting RETURN NEW in BEFORE Triggers

```sql
-- BAD: Forgetting to return NEW cancels the operation!
CREATE OR REPLACE FUNCTION bad_trigger()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
```

```
BEGIN
    NEW.updated_at := NOW();
    -- Forgot RETURN NEW!
    -- This returns NULL, which cancels the INSERT/UPDATE
END;
$$;

-- GOOD: Always return NEW for BEFORE INSERT/UPDATE triggers
CREATE OR REPLACE FUNCTION good_trigger()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    NEW.updated_at := NOW();
    RETURN NEW;  -- Required!
END;
$$;
```

Mistake 2: Infinite Trigger Loops

```sql
-- BAD: Trigger on employees that updates employees
-- causes the trigger to fire again, which updates again...
CREATE OR REPLACE FUNCTION infinite_loop_trigger()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    UPDATE employees SET updated_at = NOW()
    WHERE employee_id = NEW.employee_id;
    -- This UPDATE fires this trigger again!
    RETURN NEW;
END;
$$;

-- GOOD: Modify NEW directly instead of running another UPDATE
CREATE OR REPLACE FUNCTION no_loop_trigger()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    NEW.updated_at := NOW();  -- Modifies the row being inserted/updated
    RETURN NEW;               -- No additional UPDATE needed
END;
$$;
```

Mistake 3: Heavy Logic in Triggers

```sql
-- BAD: Sending emails, calling external APIs, or doing
-- heavy computation in triggers makes every INSERT/UPDATE slow

-- GOOD: Keep triggers lightweight. Use triggers for:
--   - Setting timestamps
--   - Simple validation
--   - Writing to audit tables
-- Use application code or background jobs for heavy work.
```

Mistake 4: Not Handling All Operations

```sql
-- BAD: Trigger function only handles INSERT but is attached to
-- INSERT OR UPDATE OR DELETE
-- The function crashes on UPDATE/DELETE because it tries to
-- access NEW on a DELETE (which has no NEW)

-- GOOD: Use IF TG_OP = ... to handle each operation correctly
```

Best Practices

1. Name triggers consistently. Use a prefix like `trg_` followed by the table name and purpose: `trg_employees_updated_at`, `trg_orders_audit`.
2. Keep triggers simple and fast. Triggers run synchronously — they block the original operation until they finish. Complex or slow triggers make every INSERT/UPDATE/DELETE slow.
3. Use BEFORE triggers for data modification. If you need to change the incoming data (normalize, validate, set defaults), use BEFORE.
4. Use AFTER triggers for side effects. If you need to log changes, update related tables, or send notifications, use AFTER.
5. Return NEW from BEFORE INSERT/UPDATE triggers. Returning NULL cancels the operation. Return OLD from BEFORE DELETE triggers.
6. Avoid modifying the same table in an AFTER trigger. This can cause infinite loops. If you need to modify the row, use a BEFORE trigger and change the NEW record directly.
7. Document your triggers. Triggers are invisible in application code. Other developers may not know they exist. Keep a clear record of what triggers are on each table.
8. Use WHEN clauses to limit trigger execution. Only fire when relevant columns change: `WHEN (OLD.salary IS DISTINCT FROM NEW.salary)`.

Quick Summary

```
+-------------------------+----------------------------------------------+
| Concept                 | What It Does                                 |
+-------------------------+----------------------------------------------+
| BEFORE trigger          | Fires before data change, can modify data    |
| AFTER trigger           | Fires after data change, for side effects    |
| INSERT trigger          | Fires when a new row is added                |
| UPDATE trigger          | Fires when an existing row is changed        |
| DELETE trigger          | Fires when a row is removed                  |
| NEW                     | The new version of the row                   |
| OLD                     | The old version of the row                   |
| TG_OP                   | The operation: INSERT, UPDATE, or DELETE      |
| TG_TABLE_NAME           | Name of the table that fired the trigger     |
| FOR EACH ROW            | Fire once per affected row                   |
| FOR EACH STATEMENT      | Fire once per SQL statement                  |
| RETURN NEW              | Allow the operation (BEFORE INSERT/UPDATE)   |
| RETURN NULL             | Cancel the operation (BEFORE trigger)        |
| WHEN clause             | Conditional trigger execution                |
| DISABLE TRIGGER         | Temporarily turn off a trigger               |
+-------------------------+----------------------------------------------+
```

Key Points

- Triggers are automatic functions that fire when data is inserted, updated, or deleted.
- A trigger consists of a trigger function (RETURNS TRIGGER) and a trigger definition (CREATE TRIGGER).
- BEFORE triggers can modify or cancel the operation. AFTER triggers cannot modify data but are good for logging.
- NEW holds the new row data (INSERT/UPDATE). OLD holds the old row data (UPDATE/DELETE).
- FOR EACH ROW triggers fire once per affected row. FOR EACH STATEMENT triggers fire once per statement.
- Always RETURN NEW from BEFORE INSERT/UPDATE triggers. Returning NULL cancels the operation.
- TG_OP and TG_TABLE_NAME are special variables that tell the trigger function what happened and where.

- Keep triggers lightweight — they run synchronously and slow down the original operation.
- Avoid infinite loops by modifying NEW directly in BEFORE triggers instead of running additional UPDATE statements.

Practice Questions

1. What is the difference between a BEFORE trigger and an AFTER trigger? When would you choose each?
2. In a trigger function, what are the NEW and OLD records? Which operations have access to each?
3. What happens if a BEFORE INSERT trigger returns NULL instead of returning NEW?
4. Why is it dangerous to have a trigger on the `employees` table that runs `UPDATE employees ...`? How do you avoid this problem?
5. What is the difference between FOR EACH ROW and FOR EACH STATEMENT? Which one has access to NEW and OLD?

Exercises

Exercise 1: Email Normalization Trigger

Create a BEFORE INSERT OR UPDATE trigger on the employees table that:

- Converts the email to lowercase
- Trims whitespace from the email
- Validates that the email contains an @ symbol (raise an exception if it does not)

Test with: `INSERT INTO employees (name, department, salary, email) VALUES ('Test', 'IT', 60000, ' TEST@Company.COM ');`

Exercise 2: Comprehensive Audit Trigger

Create a generic audit trigger function that works on any table. It should:

- Log the table name, operation, timestamp, and user
- For UPDATEs, only log columns that actually changed

- Store old and new values as JSONB

Attach it to both the employees and products tables.

Hint: Use `TG_TABLE_NAME` for the table name. For detecting changes, compare `OLD.column IS DISTINCT FROM NEW.column`.

Exercise 3: Salary Change Guardian

Create a trigger that:

- Prevents salary decreases of more than 10% in a single update
- Prevents salary increases of more than 25% in a single update
- Logs all salary changes (old salary, new salary, percentage change) to the audit_log table
- Allows unrestricted changes for employees in the "Executive" department

Hint: Calculate the percentage change with `(NEW.salary - OLD.salary) / OLD.salary * 100`.

What Is Next?

In Chapter 30, you will learn about performance tuning — how to make your queries run faster. You will learn to read query plans with EXPLAIN ANALYZE, choose the right indexes, rewrite slow queries, and use tools like VACUUM and ANALYZE to keep your database running smoothly. Performance tuning is where everything you have learned comes together.

Chapter 30: Performance Tuning — Making Queries Fast

What You Will Learn

In this chapter, you will learn:

- How to use EXPLAIN ANALYZE to understand query plans
- How to read query plan nodes (Seq Scan, Index Scan, Hash Join, etc.)
- How to identify slow queries with pg_stat_statements
- When and how to create indexes for maximum impact
- How to rewrite queries for better performance
- What connection pooling is and why PgBouncer matters
- How VACUUM and ANALYZE keep your database healthy
- What table partitioning is and when to use it
- How to optimize real slow queries step by step

Why This Chapter Matters

Imagine you own a library with a million books. A customer asks for a specific book. Without any organization, you would have to walk through every shelf and check every book one by one. That could take hours. But with a card catalog (an index), you look up the book's location in seconds and walk straight to it.

Database performance works the same way. A query that takes 5 seconds on a small development database might take 5 minutes on a production database with millions of rows. The difference between a fast application and a frustratingly slow one often comes down to how well your queries and indexes are designed.

Performance tuning is not about premature optimization — it is about knowing how to diagnose problems when they arise and having the tools to fix them. This chapter gives you

those tools.

Setting Up Our Practice Tables

We need a reasonably large dataset to see meaningful performance differences:

```sql
-- Create a large orders table
CREATE TABLE orders (
    order_id     SERIAL PRIMARY KEY,
    customer_id  INT NOT NULL,
    product_id   INT NOT NULL,
    order_date   DATE NOT NULL,
    amount       DECIMAL(10, 2) NOT NULL,
    status       VARCHAR(20) NOT NULL,
    shipping_city VARCHAR(100)
);

-- Insert 100,000 rows of sample data
INSERT INTO orders (customer_id, product_id, order_date, amount,
                    status, shipping_city)
SELECT
    (random() * 999 + 1)::INT,
    (random() * 49 + 1)::INT,
    DATE '2022-01-01' + (random() * 730)::INT,
    (random() * 500 + 10)::DECIMAL(10,2),
    CASE (random() * 3)::INT
        WHEN 0 THEN 'pending'
        WHEN 1 THEN 'shipped'
        WHEN 2 THEN 'delivered'
        ELSE 'cancelled'
    END,
    CASE (random() * 5)::INT
        WHEN 0 THEN 'New York'
        WHEN 1 THEN 'Los Angeles'
        WHEN 2 THEN 'Chicago'
        WHEN 3 THEN 'Houston'
        WHEN 4 THEN 'Phoenix'
        ELSE 'Seattle'
    END
FROM generate_series(1, 100000);

CREATE TABLE customers (
    customer_id  SERIAL PRIMARY KEY,
    name         VARCHAR(100) NOT NULL,
    email        VARCHAR(150) NOT NULL,
    city         VARCHAR(100),
    created_at   DATE NOT NULL DEFAULT CURRENT_DATE
```

```
);

INSERT INTO customers (name, email, city)
SELECT
    'Customer ' || i,
    'customer' || i || '@example.com',
    CASE (random() * 5)::INT
        WHEN 0 THEN 'New York'
        WHEN 1 THEN 'Los Angeles'
        WHEN 2 THEN 'Chicago'
        WHEN 3 THEN 'Houston'
        WHEN 4 THEN 'Phoenix'
        ELSE 'Seattle'
    END
FROM generate_series(1, 1000) AS i;

-- Run ANALYZE so PostgreSQL has accurate statistics
ANALYZE orders;
ANALYZE customers;
```

EXPLAIN ANALYZE — Your Diagnostic Tool

EXPLAIN ANALYZE is the single most important tool for performance tuning. It shows you exactly what PostgreSQL does to execute your query, how long each step takes, and how many rows it processes.

Basic Usage

```sql
EXPLAIN ANALYZE
SELECT * FROM orders WHERE customer_id = 42;
```

```
                                                    QUERY PLAN
-----------------------------------------------------------------------------------------...
 Seq Scan on orders  (cost=0.00..2137.00 rows=98 width=52) (actual time=0.025..12.345 l...
   Filter: (customer_id = 42)
   Rows Removed by Filter: 99902
 Planning Time: 0.085 ms
 Execution Time: 12.432 ms
(5 rows)
```

Reading the Query Plan

Let us break down every piece of that output:

```
Seq Scan on orders
     ^         ^
     |         |
     |         +-- Which table
     +-- The method used (Sequential Scan = read every row)

(cost=0.00..2137.00 rows=98 width=52)
       ^        ^      ^       ^
       |        |      |       +-- Average row size in bytes
       |        |      +-- Estimated number of rows returned
       |        +-- Estimated total cost
       +-- Estimated startup cost

(actual time=0.025..12.345 loops=1)
              ^        ^       ^
              |        |       +-- Number of times this step ran
              |        +-- Actual total time in milliseconds
              +-- Actual time to return first row

Filter: (customer_id = 42)
   +-- The condition being applied

Rows Removed by Filter: 99902
   +-- Rows that were read but did not match
```

```
What a Seq Scan looks like:

+-----------------------------------------------------+
| orders table (100,000 rows)                         |
| [x] [x] [x] [x] [x] [x] [x] [x] [x] [x] ...   |
|  ^                                                  |
|  Read EVERY row, check if customer_id = 42          |
|  Keep matches, discard the rest                     |
+-----------------------------------------------------+

99,902 rows read and discarded
98 rows matched and returned

This is slow! We read 100,000 rows to find 98.
```

EXPLAIN vs EXPLAIN ANALYZE

```
+------------------+-------------------------------------------+
| Command          | What It Shows                             |
+------------------+-------------------------------------------+
| EXPLAIN          | Estimated plan only (does not run query)  |
| EXPLAIN ANALYZE  | Estimated plan + actual execution times   |
```

```
|                  | (runs the query!)                          |
+------------------+--------------------------------------------+
```

Warning: EXPLAIN ANALYZE actually executes the query. For INSERT, UPDATE, or DELETE, wrap it in a transaction and roll back:

```sql
BEGIN;
EXPLAIN ANALYZE DELETE FROM orders WHERE status = 'cancelled';
ROLLBACK;  -- Undo the delete!
```

Scan Types — How PostgreSQL Reads Data

Sequential Scan (Seq Scan)

Reads every row in the table. Like reading every page of a book to find one paragraph.

```sql
EXPLAIN ANALYZE
SELECT * FROM orders WHERE amount > 400;
```

```
 Seq Scan on orders  (cost=0.00..2387.00 rows=20105 width=52)
                      (actual time=0.015..15.234 loops=1)
   Filter: (amount > 400.00)
```

A Seq Scan is not always bad. If you need most of the rows, scanning the whole table is actually faster than using an index.

Index Scan

Uses an index to go directly to the matching rows. Like using a book's index to jump to the right page.

```sql
-- First, create an index
CREATE INDEX idx_orders_customer_id ON orders(customer_id);

-- Now try the same query
EXPLAIN ANALYZE
SELECT * FROM orders WHERE customer_id = 42;
```

```
 Index Scan using idx_orders_customer_id on orders
   (cost=0.29..38.50 rows=98 width=52) (actual time=0.030..0.245 loops=1)
   Index Cond: (customer_id = 42)
```

```
What an Index Scan looks like:

Index: idx_orders_customer_id
+-----------------------------------+
| 1 -> row 523, row 8821          |
| 2 -> row 102, row 3344, ...     |
| ...                              |
| 42 -> row 55, row 1203, ...     |  <-- Jump directly here!
| ...                              |
+-----------------------------------+
          |
          v
+-----------------------------------------------------+
| orders table                                        |
| [...] [...] [row 55] [...] [...] [row 1203] [...] |
|              ^                    ^                |
|              Fetch only matching rows              |
+-----------------------------------------------------+

Only 98 rows read (vs 100,000 with Seq Scan)
```

The query went from 12.4ms to 0.25ms — about 50 times faster.

Index Only Scan

Even faster than an Index Scan. If all the columns you need are in the index, PostgreSQL does not even need to look at the table.

```sql
-- Create a covering index
CREATE INDEX idx_orders_customer_amount
ON orders(customer_id, amount);

EXPLAIN ANALYZE
SELECT customer_id, amount FROM orders WHERE customer_id = 42;
```

```
 Index Only Scan using idx_orders_customer_amount on orders
   (cost=0.29..4.50 rows=98 width=10) (actual time=0.025..0.089 loops=1)
   Index Cond: (customer_id = 42)
   Heap Fetches: 0
```

`Heap Fetches: 0` means PostgreSQL got everything from the index without touching the table at all.

Bitmap Index Scan

A middle ground between Seq Scan and Index Scan. PostgreSQL first scans the index to build a bitmap of matching pages, then reads those pages from the table.

```sql
CREATE INDEX idx_orders_status ON orders(status);

EXPLAIN ANALYZE
SELECT * FROM orders WHERE status = 'pending';
```

```
 Bitmap Heap Scan on orders  (cost=289.25..1534.50 rows=25000 width=52)
   Recheck Cond: (status = 'pending'::text)
   ->  Bitmap Index Scan on idx_orders_status
       (cost=0.00..283.00 rows=25000 width=0)
       Index Cond: (status = 'pending'::text)
```

PostgreSQL chose a Bitmap scan because about 25% of rows match — too many for an Index Scan to be efficient, but not enough to justify a full Seq Scan.

Join Types — How PostgreSQL Combines Tables

Nested Loop Join

For each row in the outer table, scan the inner table. Fast when the outer table is small and the inner table has an index.

```sql
EXPLAIN ANALYZE
SELECT c.name, o.order_date, o.amount
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
WHERE c.customer_id = 42;
```

```
 Nested Loop  (cost=0.58..42.85 rows=98 width=30)
              (actual time=0.045..0.312 loops=1)
   ->  Index Scan using customers_pkey on customers c
       (cost=0.28..8.30 rows=1 width=18)
       Index Cond: (customer_id = 42)
   ->  Index Scan using idx_orders_customer_id on orders o
       (cost=0.29..34.05 rows=98 width=16)
       Index Cond: (customer_id = 42)
```

```
Nested Loop:

customers (1 row: customer_id = 42)
    |
    +---> For this 1 customer, find all matching orders
          using the index on orders.customer_id
          (98 matches)

Result: 98 rows
```

Hash Join

Builds a hash table from the smaller table, then probes it with rows from the larger table. Good for joining large tables.

```sql
EXPLAIN ANALYZE
SELECT c.name, o.order_date, o.amount
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_date = '2023-06-15';
```

```
 Hash Join  (cost=28.50..2285.15 rows=137 width=30)
            (actual time=0.425..18.234 loops=1)
   Hash Cond: (o.customer_id = c.customer_id)
   ->  Seq Scan on orders o  (cost=0.00..2137.00 rows=137 width=16)
       Filter: (order_date = '2023-06-15')
   ->  Hash  (cost=16.00..16.00 rows=1000 width=18)
       ->  Seq Scan on customers c  (cost=0.00..16.00 rows=1000 width=18)
```

```
Hash Join:

Step 1: Build hash table from customers (smaller table)
+-----------------------------------+
| Hash Table                        |
| customer_id 1 -> "Customer 1"   |
| customer_id 2 -> "Customer 2"   |
| ...                               |
+-----------------------------------+

Step 2: Scan orders, look up each customer_id in hash table
+-----------------------------------+
| orders row (customer_id=42)       |
|   -> Look up 42 in hash table   |
|   -> Found! Join them.          |
+-----------------------------------+
```

Merge Join

Both tables are sorted on the join column, then PostgreSQL walks through them simultaneously. Efficient when both tables are large and sorted.

```sql
-- PostgreSQL may choose Merge Join when both sides are large and sorted
-- You can hint at it by ensuring indexes exist on both join columns
```

```
Merge Join:

Sorted customers:     Sorted orders:
[1] [2] [3] ...       [1] [1] [2] [3] [3] ...
 ^                     ^
 |                     |
 Walk through both lists simultaneously
 Match when values are equal
```

Comparison

```
+-------------+------------------+-----------------------------------+
| Join Type   | Best When        | How It Works                      |
+-------------+------------------+-----------------------------------+
| Nested Loop | Small outer,     | For each outer row, search inner |
|             | indexed inner    |                                   |
+-------------+------------------+-----------------------------------+
| Hash Join   | Medium/large     | Build hash table from smaller    |
|             | tables, no index | table, probe with larger         |
+-------------+------------------+-----------------------------------+
| Merge Join  | Both tables      | Walk through both sorted lists   |
|             | sorted on key    | simultaneously                   |
+-------------+------------------+-----------------------------------+
```

Identifying Slow Queries with pg_stat_statements

The pg_stat_statements extension tracks execution statistics for all queries. It is the best way to find which queries are actually slow in production.

Enabling pg_stat_statements

```sql
-- Add to postgresql.conf:
-- shared_preload_libraries = 'pg_stat_statements'
-- Then restart PostgreSQL and run:
CREATE EXTENSION IF NOT EXISTS pg_stat_statements;
```

Finding Your Slowest Queries

```sql
SELECT
    calls,
    ROUND(total_exec_time::NUMERIC, 2) AS total_ms,
    ROUND(mean_exec_time::NUMERIC, 2) AS avg_ms,
    ROUND((100 * total_exec_time /
           SUM(total_exec_time) OVER ())::NUMERIC, 2) AS pct_time,
    query
FROM pg_stat_statements
ORDER BY total_exec_time DESC
LIMIT 10;
```

```
 calls  | total_ms  | avg_ms  | pct_time |              query
--------+-----------+---------+----------+--------------------------------
  15234 | 45230.50  |   2.97  |    32.15 | SELECT * FROM orders WHERE ...
   8901 | 28103.20  |   3.16  |    19.97 | SELECT c.name, o.* FROM ...
   3456 | 15890.00  |   4.60  |    11.29 | UPDATE orders SET status = ...
    ...
```

This tells you:

- Which queries run most often (calls)
- Which queries take the most total time (total_ms)
- Which queries are slowest per execution (avg_ms)
- What percentage of total database time each query uses (pct_time)

Reset Statistics

```sql
SELECT pg_stat_statements_reset();
```

Index Strategies

When to Create an Index

```
+-----------------------------------------+-------------------+
| Scenario                                | Create Index?     |
+-----------------------------------------+-------------------+
| Column used in WHERE frequently         | Yes               |
| Column used in JOIN conditions          | Yes               |
| Column used in ORDER BY                 | Often yes         |
| Column with high cardinality (many      | Yes               |
|   unique values, like customer_id)      |                   |
| Column with low cardinality (few unique | Usually no        |
|   values, like status with 4 values)    |                   |
| Small table (< 1000 rows)              | Usually no        |
| Column rarely used in queries           | No                |
| Table with heavy INSERT/UPDATE          | Be selective      |
+-----------------------------------------+-------------------+
```

Composite Indexes

When queries filter on multiple columns, a composite index can help:

```sql
-- Query that filters on two columns
SELECT * FROM orders
WHERE customer_id = 42 AND status = 'shipped';

-- Composite index for this query
CREATE INDEX idx_orders_customer_status
ON orders(customer_id, status);
```

Column order matters! Put the most selective column first (the one that filters out the most rows).

```
Composite Index (customer_id, status):

customer_id = 42:
  status = 'pending'   -> [row 55, row 1203]
  status = 'shipped'   -> [row 892, row 3401]   <-- finds these directly
  status = 'delivered' -> [row 2100]

The index works for:
  WHERE customer_id = 42                    -- YES (uses first column)
  WHERE customer_id = 42 AND status = 'x'  -- YES (uses both columns)
  WHERE status = 'shipped'                  -- NO  (cannot skip first column)
```

Partial Indexes

Index only the rows that matter:

```sql
-- If you mostly query pending orders:
CREATE INDEX idx_orders_pending
ON orders(customer_id, order_date)
WHERE status = 'pending';
```

This index is smaller and faster because it only includes pending orders.

Expression Indexes

Index the result of an expression:

```sql
-- If you often search by lowercase email:
CREATE INDEX idx_customers_email_lower
ON customers(LOWER(email));

-- Now this query uses the index:
SELECT * FROM customers WHERE LOWER(email) = 'customer42@example.com';
```

Checking Index Usage

```sql
SELECT
    indexrelname AS index_name,
    idx_scan AS times_used,
    pg_size_pretty(pg_relation_size(indexrelid)) AS index_size
FROM pg_stat_user_indexes
WHERE schemaname = 'public'
ORDER BY idx_scan DESC;
```

```
         index_name          | times_used | index_size
-----------------------------+------------+-----------
 orders_pkey                 |      15234 | 2208 kB
 idx_orders_customer_id      |       8901 | 2208 kB
 idx_orders_status           |       3456 | 2208 kB
 idx_orders_customer_status  |        234 | 2208 kB
 idx_orders_pending          |         12 | 560 kB
(5 rows)
```

If an index has `times_used = 0`, it is wasting disk space and slowing down writes. Consider dropping it.

Query Rewriting Tips

Tip 1: Avoid SELECT *

```sql
-- BAD: Fetches all columns, even ones you do not need
SELECT * FROM orders WHERE customer_id = 42;

-- GOOD: Fetch only what you need
SELECT order_id, order_date, amount FROM orders WHERE customer_id = 42;
```

Fetching fewer columns means less data to read from disk, less data to send over the network, and potentially allows Index Only Scans.

Tip 2: Use EXISTS Instead of IN for Subqueries

```sql
-- SLOWER: IN with subquery
SELECT * FROM customers c
WHERE c.customer_id IN (
    SELECT customer_id FROM orders WHERE amount > 400
);

-- FASTER: EXISTS stops at the first match
SELECT * FROM customers c
WHERE EXISTS (
    SELECT 1 FROM orders o
    WHERE o.customer_id = c.customer_id
    AND o.amount > 400
);
```

IN must collect all matching customer_ids first. EXISTS can stop as soon as it finds one match for each customer.

Tip 3: Minimize Subqueries in SELECT

```sql
-- BAD: Subquery runs once per row in the outer query
SELECT
    c.name,
    (SELECT COUNT(*) FROM orders o WHERE o.customer_id = c.customer_id) AS order_count,
    (SELECT SUM(amount) FROM orders o WHERE o.customer_id = c.customer_id) AS total_spent
FROM customers c;

-- GOOD: Use a JOIN with aggregation
SELECT
```

```
    c.name,
    COUNT(o.order_id) AS order_count,
    COALESCE(SUM(o.amount), 0) AS total_spent
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.name;
```

Tip 4: Use LIMIT for Pagination

```sql
-- BAD: Fetch everything, then paginate in the application
SELECT * FROM orders ORDER BY order_date DESC;

-- GOOD: Fetch only the page you need
SELECT * FROM orders ORDER BY order_date DESC
LIMIT 20 OFFSET 0;  -- Page 1

SELECT * FROM orders ORDER BY order_date DESC
LIMIT 20 OFFSET 20;  -- Page 2
```

Tip 5: Avoid Functions on Indexed Columns in WHERE

```sql
-- BAD: The function prevents index usage
SELECT * FROM orders
WHERE EXTRACT(YEAR FROM order_date) = 2023;

-- GOOD: Rewrite to use the column directly
SELECT * FROM orders
WHERE order_date >= '2023-01-01'
  AND order_date < '2024-01-01';
```

The first query applies a function to every row's order_date, so the index cannot be used. The second query compares the column directly, which allows index usage.

Connection Pooling with PgBouncer

Every database connection consumes memory (about 5-10 MB in PostgreSQL). If your application opens 200 connections, that is 1-2 GB just for connections. Connection pooling solves this.

```
Without Pooling:                    With PgBouncer:

App Server (200 threads)            App Server (200 threads)
    |  |  |  |  |  |                    |  |  |  |  |  |
    |  |  |  |  |  |                    v  v  v  v  v  v
    v  v  v  v  v  v                +-------------------+
+----------------------+            | PgBouncer         |
| PostgreSQL           |            | (connection pool) |
| 200 connections      |            | 200 app conns ->  |
| (1-2 GB memory!)    |            |  20 DB conns      |
+----------------------+            +-------------------+
                                          |  |
                                          v  v
                                    +-------------------+
                                    | PostgreSQL        |
                                    | 20 connections    |
                                    | (100-200 MB)      |
                                    +-------------------+
```

PgBouncer sits between your application and PostgreSQL. It maintains a small pool of database connections and shares them among many application connections.

PgBouncer Modes

```
+--------------------+-----------------------------------------------+
| Mode               | How It Works                                  |
+--------------------+-----------------------------------------------+
| Session pooling    | Connection assigned for entire session        |
|                    | Safest, least efficient                       |
+--------------------+-----------------------------------------------+
| Transaction        | Connection assigned per transaction           |
| pooling            | Best balance of safety and efficiency         |
+--------------------+-----------------------------------------------+
| Statement pooling  | Connection assigned per statement             |
|                    | Most efficient, but no multi-statement        |
|                    | transactions                                  |
+--------------------+-----------------------------------------------+
```

Transaction pooling is the most commonly used mode. It gives each transaction a dedicated connection, then returns the connection to the pool when the transaction completes.

VACUUM and ANALYZE

Why VACUUM?

When you UPDATE or DELETE a row in PostgreSQL, the old version of the row is not immediately removed. It is marked as "dead" but still takes up space. VACUUM reclaims that space.

```
Before VACUUM:
+------+------+------+------+------+------+------+
| Live | Dead | Live | Dead | Dead | Live | Dead |
+------+------+------+------+------+------+------+

After VACUUM:
+------+------+------+------+
| Live | Live | Live | Free |
+------+------+------+------+

Space from dead rows is now available for reuse.
```

Running VACUUM

```sql
-- VACUUM a specific table
VACUUM orders;

-- VACUUM with verbose output
VACUUM VERBOSE orders;

-- VACUUM FULL - reclaims space and compacts the table
-- WARNING: Locks the table! Use during maintenance windows.
VACUUM FULL orders;
```

Autovacuum

PostgreSQL runs VACUUM automatically in the background. You can check its activity:

```sql
SELECT
    relname AS table_name,
    n_dead_tup AS dead_rows,
    n_live_tup AS live_rows,
    last_autovacuum
FROM pg_stat_user_tables
WHERE n_dead_tup > 0
ORDER BY n_dead_tup DESC;
```

```
 table_name | dead_rows | live_rows |     last_autovacuum
------------+-----------+-----------+-----------------------
 orders     |      1523 |    100000 | 2024-01-15 03:00:00
```

```
 customers  |        45 |      1000 | 2024-01-15 03:00:00
(2 rows)
```

Why ANALYZE?

ANALYZE collects statistics about the distribution of data in your tables. PostgreSQL uses these statistics to decide the best query plan.

```sql
-- Analyze a specific table
ANALYZE orders;

-- Analyze all tables in the database
ANALYZE;
```

When to run ANALYZE:

- After loading a large amount of data
- After major UPDATE or DELETE operations
- When query plans suddenly change for the worse

```
Without ANALYZE:                    With ANALYZE:
PostgreSQL guesses:                 PostgreSQL knows:
"status has ~4 unique values"       "status has 4 values:
                                     pending=25%, shipped=25%,
                                     delivered=25%, cancelled=25%"

Bad guess -> Bad query plan          Accurate stats -> Good query plan
```

Table Partitioning Basics

Partitioning splits a large table into smaller physical pieces (partitions) while treating them as one logical table. Queries that filter on the partition key only scan the relevant partition.

```
Single large table:                 Partitioned table:
+----------------------------+      +----------+----------+----------+
| orders (10 million rows)   |      | orders   | orders   | orders   |
| ALL data scanned           |      | 2022     | 2023     | 2024     |
+----------------------------+      | 3M rows  | 4M rows  | 3M rows  |
                                    +----------+----------+----------+
                                         ^
```

```
                                        |
                                  Query for 2023?
                                  Only scan this partition!
```

Creating a Partitioned Table

```sql
-- Create the parent table
CREATE TABLE orders_partitioned (
    order_id     SERIAL,
    customer_id  INT NOT NULL,
    order_date   DATE NOT NULL,
    amount       DECIMAL(10, 2) NOT NULL,
    status       VARCHAR(20) NOT NULL
) PARTITION BY RANGE (order_date);

-- Create partitions for each year
CREATE TABLE orders_2022 PARTITION OF orders_partitioned
    FOR VALUES FROM ('2022-01-01') TO ('2023-01-01');

CREATE TABLE orders_2023 PARTITION OF orders_partitioned
    FOR VALUES FROM ('2023-01-01') TO ('2024-01-01');

CREATE TABLE orders_2024 PARTITION OF orders_partitioned
    FOR VALUES FROM ('2024-01-01') TO ('2025-01-01');
```

How Partition Pruning Works

```sql
EXPLAIN ANALYZE
SELECT * FROM orders_partitioned
WHERE order_date = '2023-06-15';
```

```
 Append  (cost=0.00..15.00 rows=1 width=44)
   ->  Seq Scan on orders_2023 orders_partitioned_1
       (cost=0.00..15.00 rows=1 width=44)
       Filter: (order_date = '2023-06-15')
```

PostgreSQL only scanned the 2023 partition. The 2022 and 2024 partitions were completely skipped (pruned).

When to Use Partitioning

- Tables with millions of rows
- Queries that consistently filter on a date or category column

- When you need to efficiently drop old data (drop the partition instead of DELETE)
- When maintenance operations (VACUUM, REINDEX) need to work on smaller chunks

---

Practical: Optimizing 5 Slow Queries

Slow Query 1: Missing Index

```sql
-- BEFORE: Full table scan
EXPLAIN ANALYZE
SELECT * FROM orders WHERE customer_id = 42 AND status = 'shipped';
```

```
 Seq Scan on orders  (cost=0.00..2637.00 rows=25 width=52)
                      (actual time=0.045..14.567 loops=1)
   Filter: ((customer_id = 42) AND (status = 'shipped'))
   Rows Removed by Filter: 99975
 Execution Time: 14.623 ms
```

Diagnosis: Sequential scan reading 100,000 rows to find 25.

Fix: Create a composite index.

```sql
CREATE INDEX idx_orders_cust_status ON orders(customer_id, status);

-- AFTER:
EXPLAIN ANALYZE
SELECT * FROM orders WHERE customer_id = 42 AND status = 'shipped';
```

```
 Index Scan using idx_orders_cust_status on orders
   (cost=0.42..28.50 rows=25 width=52) (actual time=0.025..0.098 loops=1)
   Index Cond: ((customer_id = 42) AND (status = 'shipped'))
 Execution Time: 0.132 ms
```

Result: 14.6ms down to 0.13ms — 112 times faster.

Slow Query 2: Function on Indexed Column

```sql
-- BEFORE: Function prevents index usage
EXPLAIN ANALYZE
SELECT * FROM orders
WHERE EXTRACT(YEAR FROM order_date) = 2023
  AND EXTRACT(MONTH FROM order_date) = 6;
```

```
 Seq Scan on orders  (cost=0.00..3137.00 rows=500 width=52)
                      (actual time=0.035..22.456 loops=1)
   Filter: ((EXTRACT(YEAR FROM order_date) = 2023) AND ...)
   Rows Removed by Filter: 99500
 Execution Time: 22.512 ms
```

Diagnosis: Functions on columns prevent index usage.

Fix: Rewrite to use range comparison.

```sql
CREATE INDEX idx_orders_date ON orders(order_date);

-- AFTER: Range comparison uses the index
EXPLAIN ANALYZE
SELECT * FROM orders
WHERE order_date >= '2023-06-01'
  AND order_date < '2023-07-01';
```

```
 Index Scan using idx_orders_date on orders
   (cost=0.29..55.30 rows=500 width=52) (actual time=0.028..0.456 loops=1)
   Index Cond: ((order_date >= '2023-06-01') AND (order_date < '2023-07-01'))
 Execution Time: 0.523 ms
```

Result: 22.5ms down to 0.52ms — 43 times faster.

Slow Query 3: SELECT * with Unnecessary Columns

```sql
-- BEFORE: Fetching all columns when we only need two
EXPLAIN ANALYZE
SELECT * FROM orders WHERE customer_id = 42;
```

```
 Index Scan using idx_orders_customer_id on orders
   (cost=0.29..38.50 rows=98 width=52) (actual time=0.030..0.245 loops=1)
 Execution Time: 0.312 ms
```

Fix: Select only needed columns, use a covering index.

```sql
CREATE INDEX idx_orders_cust_date_amount
ON orders(customer_id) INCLUDE (order_date, amount);

-- AFTER: Index Only Scan
EXPLAIN ANALYZE
SELECT order_date, amount FROM orders WHERE customer_id = 42;
```

```
 Index Only Scan using idx_orders_cust_date_amount on orders
   (cost=0.42..4.85 rows=98 width=10) (actual time=0.020..0.065 loops=1)
   Heap Fetches: 0
 Execution Time: 0.089 ms
```

Result: 0.31ms down to 0.089ms — 3.5 times faster. Plus much less data transferred.

Slow Query 4: IN Subquery vs EXISTS

```sql
-- BEFORE: IN with correlated subquery
EXPLAIN ANALYZE
SELECT c.name, c.city
FROM customers c
WHERE c.customer_id IN (
    SELECT DISTINCT customer_id
    FROM orders
    WHERE amount > 400
);
```

```
 Hash Join  (cost=2637.50..2682.00 rows=500 width=24)
            (actual time=18.234..18.890 loops=1)
   ->  HashAggregate  (cost=2637.00..2642.00 rows=500 width=4)
       ->  Seq Scan on orders  (cost=0.00..2387.00 rows=20000 width=4)
           Filter: (amount > 400.00)
 Execution Time: 19.012 ms
```

Fix: Use EXISTS.

```sql
-- AFTER: EXISTS with index
EXPLAIN ANALYZE
SELECT c.name, c.city
FROM customers c
WHERE EXISTS (
```

```
    SELECT 1 FROM orders o
    WHERE o.customer_id = c.customer_id
    AND o.amount > 400
);
```

```
 Nested Loop Semi Join  (cost=0.29..1650.00 rows=500 width=24)
                         (actual time=0.045..8.234 loops=1)
   ->  Seq Scan on customers c  (cost=0.00..16.00 rows=1000 width=28)
   ->  Index Scan using idx_orders_customer_id on orders o
       (cost=0.29..1.50 rows=1 width=4)
       Index Cond: (customer_id = c.customer_id)
       Filter: (amount > 400.00)
 Execution Time: 8.345 ms
```

Result: 19ms down to 8.3ms — 2.3 times faster. The improvement is even larger on bigger tables.

Slow Query 5: Aggregate Without Proper Index

```sql
-- BEFORE: Aggregating over a large table
EXPLAIN ANALYZE
SELECT
    customer_id,
    COUNT(*) AS order_count,
    SUM(amount) AS total_spent
FROM orders
WHERE order_date >= '2023-01-01' AND order_date < '2024-01-01'
GROUP BY customer_id
ORDER BY total_spent DESC
LIMIT 10;
```

```
 Limit  (cost=2890.00..2890.03 rows=10 width=44)
        (actual time=25.678..25.689 loops=1)
   ->  Sort  (cost=2890.00..2892.50 rows=1000 width=44)
       ->  HashAggregate  (cost=2637.00..2867.00 rows=1000 width=44)
           ->  Seq Scan on orders  (cost=0.00..2387.00 rows=50000 width=10)
               Filter: ((order_date >= ...) AND (order_date < ...))
 Execution Time: 25.756 ms
```

Fix: Create an index that covers the filter and grouping.

```sql
CREATE INDEX idx_orders_date_customer
ON orders(order_date, customer_id) INCLUDE (amount);

-- AFTER:
EXPLAIN ANALYZE
SELECT
    customer_id,
    COUNT(*) AS order_count,
    SUM(amount) AS total_spent
FROM orders
WHERE order_date >= '2023-01-01' AND order_date < '2024-01-01'
GROUP BY customer_id
ORDER BY total_spent DESC
LIMIT 10;
```

```
 Limit  (cost=890.00..890.03 rows=10 width=44)
        (actual time=8.234..8.245 loops=1)
   ->  Sort  (cost=890.00..892.50 rows=1000 width=44)
       ->  HashAggregate  (cost=637.00..867.00 rows=1000 width=44)
           ->  Index Only Scan using idx_orders_date_customer on orders
               (cost=0.42..387.00 rows=50000 width=10)
               Index Cond: ((order_date >= ...) AND (order_date < ...))
               Heap Fetches: 0
 Execution Time: 8.312 ms
```

Result: 25.8ms down to 8.3ms — 3.1 times faster. The Index Only Scan avoids reading the table entirely.

Performance Tuning Checklist

```
+--+-------------------------------------------------------------+
|  | Step                                                        |
+--+-------------------------------------------------------------+
| 1| Run EXPLAIN ANALYZE on the slow query                       |
| 2| Check for Seq Scans on large tables — add indexes           |
| 3| Check for functions on columns in WHERE — rewrite           |
| 4| Check for missing indexes on JOIN columns                   |
| 5| Replace SELECT * with specific columns                      |
| 6| Replace IN subqueries with EXISTS                           |
| 7| Check if VACUUM and ANALYZE have run recently               |
| 8| Check for unused indexes (wasting write performance)        |
| 9| Consider partial indexes for common filters                 |
|10| Consider partitioning for very large tables (100M+ rows)    |
+--+-------------------------------------------------------------+
```

Common Mistakes

Mistake 1: Adding Indexes to Everything

```sql
-- BAD: Index on every column
CREATE INDEX idx1 ON orders(customer_id);
CREATE INDEX idx2 ON orders(product_id);
CREATE INDEX idx3 ON orders(order_date);
CREATE INDEX idx4 ON orders(amount);
CREATE INDEX idx5 ON orders(status);
CREATE INDEX idx6 ON orders(shipping_city);

-- Every INSERT/UPDATE must update ALL six indexes.
-- This slows down writes significantly.

-- GOOD: Only index columns that appear in WHERE, JOIN, or ORDER BY
-- of your actual slow queries.
```

Mistake 2: Not Running ANALYZE After Loading Data

```sql
-- After a bulk load, PostgreSQL's statistics are outdated.
-- The query planner may make bad decisions.

-- Always run:
ANALYZE orders;
-- After loading significant amounts of data.
```

Mistake 3: Optimizing Before Measuring

```sql
-- BAD: "I think this query might be slow, let me add indexes"
-- You might be adding indexes for queries that are not even slow.

-- GOOD: First measure with EXPLAIN ANALYZE or pg_stat_statements,
-- then optimize the queries that are actually slow.
```

Mistake 4: Ignoring the Execution Plan

```sql
-- BAD: Only looking at execution time
EXPLAIN ANALYZE SELECT ...;
-- "It took 500ms, that seems slow"

-- GOOD: Read the full plan to understand WHY it is slow
-- Look for:
--   Seq Scan on large tables (missing index?)
--   Rows Removed by Filter: 999,000 (index needed)
--   Sort (could ORDER BY use an index?)
--   Nested Loop with high loop count (missing index on inner table?)
```

Best Practices

1. Measure first, optimize second. Use EXPLAIN ANALYZE and pg_stat_statements to find the actual bottlenecks before making changes.
2. Create indexes based on your query patterns. Look at WHERE clauses, JOIN conditions, and ORDER BY columns in your most frequent and slowest queries.
3. Keep statistics up to date. Ensure autovacuum is running. Run ANALYZE manually after bulk data loads.
4. Use composite indexes wisely. Put the most selective column first. An index on (customer_id, status) works for queries filtering on customer_id alone, but not for status alone.
5. Avoid SELECT * in production queries. Fetch only the columns you need. This enables Index Only Scans and reduces data transfer.
6. Use EXISTS instead of IN for correlated subqueries. EXISTS stops at the first match, while IN must collect all values.
7. Keep functions out of WHERE clauses on indexed columns. Instead of `WHERE UPPER(email) = 'X'`, create a functional index or rewrite the query.
8. Use connection pooling in production. PgBouncer in transaction pooling mode is the standard approach.
9. Monitor dead tuples and vacuum activity. Too many dead tuples slow down queries because PostgreSQL must skip over them.
10. Consider partitioning for tables with hundreds of millions of rows and queries that consistently filter on a date or category column.

Quick Summary

```
+----------------------+----------------------------------------------+
| Tool / Concept       | What It Does                                 |
+----------------------+----------------------------------------------+
| EXPLAIN ANALYZE      | Shows actual query execution plan and times  |
| Seq Scan             | Reads every row (slow on large tables)       |
| Index Scan           | Jumps to matching rows via index (fast)      |
| Index Only Scan      | Gets data from index without touching table  |
| Bitmap Scan          | Builds a map of matching pages               |
| Nested Loop Join     | For each outer row, search inner table       |
| Hash Join            | Build hash from small table, probe with big  |
| pg_stat_statements   | Tracks query execution statistics            |
| Composite Index      | Index on multiple columns                    |
| Partial Index        | Index with a WHERE clause                    |
| INCLUDE              | Add columns to index for Index Only Scans    |
| VACUUM               | Reclaims space from dead rows                |
| ANALYZE              | Updates table statistics for query planner   |
| PgBouncer            | Connection pooler for PostgreSQL              |
| Partitioning         | Splits table into smaller physical pieces    |
+----------------------+----------------------------------------------+
```

Key Points

- EXPLAIN ANALYZE is your most important diagnostic tool. It shows what PostgreSQL actually does and how long each step takes.
- Sequential Scans on large tables are a common performance problem. Adding the right index often provides dramatic improvements.
- Composite indexes should put the most selective column first. They work for queries filtering on the leading columns but not for trailing columns alone.
- Functions on indexed columns in WHERE clauses prevent index usage. Rewrite queries to compare columns directly.
- SELECT * fetches unnecessary data. Specify only the columns you need to enable Index Only Scans.
- EXISTS is generally faster than IN for subqueries because it stops at the first match.
- VACUUM reclaims space from deleted rows. ANALYZE updates statistics. Both are essential for performance.
- Connection pooling (PgBouncer) reduces memory usage by sharing database connections among many application connections.

- Table partitioning helps with very large tables by allowing PostgreSQL to skip irrelevant partitions entirely.
- Always measure before optimizing. Fix the queries that actually cause problems, not the ones you think might be slow.

Practice Questions

1. What is the difference between EXPLAIN and EXPLAIN ANALYZE? Why should you be careful using EXPLAIN ANALYZE with DELETE or UPDATE statements?
2. A query shows `Seq Scan on orders` with `Rows Removed by Filter: 999,500` and only `rows=500` returned. What does this tell you, and how would you fix it?
3. You have an index on `orders(customer_id)`. Will it help this query: `WHERE LOWER(customer_id::TEXT) = '42'`? Why or why not?
4. What is the difference between `VACUUM` and `VACUUM FULL`? When would you use each?
5. You have a table with columns A, B, C and a composite index on (A, B). Which of these queries can use the index?
    - `WHERE A = 1`
    - `WHERE B = 2`
    - `WHERE A = 1 AND B = 2`
    - `WHERE A = 1 AND C = 3`

Exercises

Exercise 1: Diagnose and Fix

Run EXPLAIN ANALYZE on the following query and optimize it. Document the before and after execution times:

```sql
SELECT c.name, c.city, COUNT(*) AS order_count, SUM(o.amount) AS total
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
WHERE EXTRACT(YEAR FROM o.order_date) = 2023
GROUP BY c.name, c.city
HAVING COUNT(*) > 5
ORDER BY total DESC;
```

Hint: Rewrite the date filter and ensure proper indexes exist on the join and filter columns.

Exercise 2: Index Audit

Write a query that shows all indexes in the public schema along with:

- How many times each index has been used (scanned)
- The size of each index
- Whether the index has been used in the last 30 days

Identify indexes that should be dropped (never used, taking up space).

Hint: Use `pg_stat_user_indexes` and `pg_relation_size()`.

Exercise 3: Full Performance Report

Create a comprehensive performance report that:

1. Shows the 5 largest tables and their row counts
2. Shows tables with the most dead tuples (needing VACUUM)
3. Shows the most-used and least-used indexes
4. Identifies tables missing indexes on foreign key columns

Hint: Use `pg_stat_user_tables`, `pg_stat_user_indexes`, `pg_class`, and `information_schema`.

What Is Next?

Congratulations! You have completed the advanced SQL section of this book. You now have a solid foundation in transactions, window functions, CTEs, stored procedures, triggers, and performance tuning. These skills will serve you well as you build and maintain real-world database applications. In the next section, we will explore database administration and deployment topics to complete your journey from beginner to confident database practitioner.

Chapter 31: Database Security

What You Will Learn

In this chapter, you will learn how to protect your database from unauthorized access and attacks. You will understand the difference between authentication and authorization, learn how to create roles and manage privileges, implement row-level security, protect against SQL injection, encrypt your data, and create backup strategies. By the end, you will set up a complete security model for a real application.

Why This Chapter Matters

Imagine you own a bank. You would not leave the vault door open and hope that only honest people walk in. You would verify who enters the building (authentication), control which rooms they can access (authorization), install cameras (auditing), and keep copies of important records in a safe place (backups).

Your database works the same way. It holds your most valuable asset: data. Customer information, financial records, personal details -- all of it needs protection. A single security breach can destroy a business, damage trust, and violate privacy laws.

Whether you are building a small web application or managing enterprise systems, database security is not optional. It is essential.

Authentication: Who Are You?

Authentication answers one simple question: "Who are you?"

Think of it like showing your ID at the door of a building. Before you can do anything inside, you must prove your identity.

PostgreSQL supports several authentication methods:

```
+--------------------+----------------------------------------+
| Method             | How It Works                           |
+--------------------+----------------------------------------+
| password           | Plain text password (not recommended)  |
| md5                | Hashed password (legacy)               |
| scram-sha-256      | Strongly hashed password (recommended) |
| peer               | Uses OS username (local connections)   |
| cert               | Uses SSL certificates                  |
| ldap               | Delegates to LDAP directory            |
+--------------------+----------------------------------------+
```

The pg_hba.conf File

PostgreSQL controls authentication through a file called `pg_hba.conf` (Host-Based Authentication). Think of it as a guest list at the door.

Each line in this file says: "For this type of connection, from this address, to this database, for this user, use this authentication method."

```
# TYPE    DATABASE    USER       ADDRESS          METHOD

# Local connections (same machine)
local     all         postgres                    peer
local     all         all                         scram-sha-256

# Remote connections
host      all         all        127.0.0.1/32     scram-sha-256
host      all         all        192.168.1.0/24   scram-sha-256
host      myapp_db    app_user   10.0.0.0/8       scram-sha-256

# Reject everything else
host      all         all        0.0.0.0/0        reject
```

Let us break this down line by line:

- Line 1: Local connections from the `postgres` superuser use `peer` authentication (the OS verifies identity).
- Line 2: All other local connections require a password with `scram-sha-256` hashing.
- Line 3: Connections from localhost (127.0.0.1) require a hashed password.
- Line 4: Connections from the local network (192.168.1.x) require a hashed password.
- Line 5: Only the `app_user` can connect to `myapp_db` from the 10.x.x.x network.

- Line 6: Everything else is rejected.

The file is read from top to bottom, and the first matching rule wins. This is important. If you put a `reject` rule at the top, nobody can connect.

After editing `pg_hba.conf`, reload PostgreSQL to apply changes:

```sql
SELECT pg_reload_conf();
```

Authorization: What Can You Do?

Authorization answers a different question: "Now that I know who you are, what are you allowed to do?"

Think of a hospital. A doctor can view patient records, a nurse can update vital signs, and a receptionist can only see appointment schedules. Everyone is authenticated (they have ID badges), but they have different levels of authorization.

CREATE ROLE and CREATE USER

In PostgreSQL, users and roles are almost the same thing. The only difference is that `CREATE USER` automatically adds the `LOGIN` privilege.

```sql
-- Create a role (cannot log in by default)
CREATE ROLE read_only;

-- Create a user (can log in)
CREATE USER alice WITH PASSWORD 'secure_password_123';

-- These two are equivalent:
CREATE USER bob WITH PASSWORD 'bobs_password';
CREATE ROLE bob WITH LOGIN PASSWORD 'bobs_password';
```

Let us look at the output after creating these:

```
CREATE ROLE
CREATE ROLE
```

You can also set additional properties when creating a role:

```sql
CREATE ROLE admin_user WITH
    LOGIN
    PASSWORD 'admin_pass_456'
    CREATEDB
    CREATEROLE
    VALID UNTIL '2026-12-31';
```

Let us break this down:

- `LOGIN` -- This role can connect to the database.
- `PASSWORD` -- Sets the authentication password.
- `CREATEDB` -- This role can create new databases.
- `CREATEROLE` -- This role can create other roles.
- `VALID UNTIL` -- The password expires on this date.

Viewing Existing Roles

```sql
SELECT rolname, rolsuper, rolcreatedb, rolcanlogin
FROM pg_roles
WHERE rolname NOT LIKE 'pg_%'
ORDER BY rolname;
```

```
+--------------+----------+-------------+-------------+
| rolname      | rolsuper | rolcreatedb | rolcanlogin |
+--------------+----------+-------------+-------------+
| admin_user   | f        | t           | t           |
| alice        | f        | f           | t           |
| bob          | f        | f           | t           |
| postgres     | t        | t           | t           |
| read_only    | f        | f           | f           |
+--------------+----------+-------------+-------------+
(5 rows)
```

Notice that `read_only` has `rolcanlogin` set to `f` (false) because we created it with `CREATE ROLE` instead of `CREATE USER`. The `postgres` user is a superuser (`rolsuper = t`), which means it can do anything.

GRANT and REVOKE Privileges

Privileges control exactly what a role can do. Think of them as keys to different rooms.

Common Privileges

```
+-------------+-----------------------------------------+
| Privilege   | What It Allows                          |
+-------------+-----------------------------------------+
| SELECT      | Read data from a table                  |
| INSERT      | Add new rows to a table                 |
| UPDATE      | Modify existing rows                    |
| DELETE      | Remove rows from a table                |
| TRUNCATE    | Empty a table completely                |
| REFERENCES  | Create foreign key constraints          |
| TRIGGER     | Create triggers on a table              |
| CREATE      | Create objects in a schema              |
| CONNECT     | Connect to a database                   |
| USAGE       | Use a schema or sequence                |
| ALL         | All available privileges                |
+-------------+-----------------------------------------+
```

Granting Privileges

```sql
-- Grant SELECT on a specific table
GRANT SELECT ON employees TO alice;

-- Grant multiple privileges
GRANT SELECT, INSERT, UPDATE ON orders TO bob;

-- Grant all privileges on a table
GRANT ALL ON products TO admin_user;

-- Grant on all tables in a schema
GRANT SELECT ON ALL TABLES IN SCHEMA public TO read_only;

-- Grant usage on a schema
GRANT USAGE ON SCHEMA public TO read_only;
```

Let us understand what each line does:

- Line 1: Alice can now read data from the `employees` table, but she cannot insert, update, or delete.
- Line 2: Bob can read, add, and modify data in the `orders` table, but he cannot delete rows.
- Line 3: `admin_user` can do everything with the `products` table.
- Line 4: The `read_only` role can read all tables in the `public` schema.
- Line 5: The `read_only` role can access objects in the `public` schema. Without this, even `SELECT` would fail.

Revoking Privileges

```sql
-- Remove INSERT privilege from bob on orders
REVOKE INSERT ON orders FROM bob;

-- Remove all privileges
REVOKE ALL ON products FROM admin_user;

-- Remove from all tables in a schema
REVOKE ALL ON ALL TABLES IN SCHEMA public FROM alice;
```

Default Privileges

When new tables are created in the future, you want certain roles to automatically have access. Default privileges handle this:

```sql
-- Any new table created by postgres will be readable by read_only
ALTER DEFAULT PRIVILEGES IN SCHEMA public
    GRANT SELECT ON TABLES TO read_only;

-- Any new sequence will be usable by app_user
ALTER DEFAULT PRIVILEGES IN SCHEMA public
    GRANT USAGE ON SEQUENCES TO app_user;
```

Without default privileges, you would need to run `GRANT` every time a new table is created. That is tedious and easy to forget.

Role Hierarchy

Roles can be members of other roles. This creates a hierarchy, like an organizational chart.

```
            +-------------------+
            |   db_admin        |
            | (all privileges)  |
            +---------+---------+
                      |
          +-----------+-----------+
          |                       |
  +-------+-------+    +-------+-------+
  |  db_writer    |    |  db_reader    |
  | (read + write)|    | (read only)   |
```

```
  +-------+-------+    +-------+-------+
          |                    |
    +-----+-----+         +----+----+
    |     |     |         |         |
  alice  bob  carol     dave      eve
```

Here is how to set this up:

```sql
-- Step 1: Create the base roles
CREATE ROLE db_reader;
CREATE ROLE db_writer;
CREATE ROLE db_admin;

-- Step 2: Grant privileges to base roles
GRANT USAGE ON SCHEMA public TO db_reader;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO db_reader;

GRANT USAGE ON SCHEMA public TO db_writer;
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public TO db_writer;

GRANT ALL ON SCHEMA public TO db_admin;
GRANT ALL ON ALL TABLES IN SCHEMA public TO db_admin;

-- Step 3: Build the hierarchy
GRANT db_reader TO db_writer;   -- writers can also read
GRANT db_writer TO db_admin;    -- admins can also write

-- Step 4: Assign users to roles
GRANT db_reader TO dave;
GRANT db_reader TO eve;
GRANT db_writer TO alice;
GRANT db_writer TO bob;
GRANT db_writer TO carol;
GRANT db_admin TO admin_user;
```

Now dave and eve can only read data. Alice, bob, and carol can read and write. The admin_user can do everything.

The beauty of this approach is maintenance. If you need to change what "read access" means, you only update the `db_reader` role. All members automatically get the updated privileges.

Row-Level Security (RLS)

Regular privileges control access to entire tables. Row-Level Security goes further. It controls which specific rows a user can see or modify.

Think of an apartment building. Everyone has access to the lobby (the table), but each tenant can only enter their own apartment (their rows).

Enabling RLS

```sql
-- Create a table for a multi-tenant application
CREATE TABLE tenant_data (
    id          SERIAL PRIMARY KEY,
    tenant_id   INTEGER NOT NULL,
    data        TEXT NOT NULL,
    created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Insert sample data
INSERT INTO tenant_data (tenant_id, data) VALUES
    (1, 'Tenant 1 - Secret report'),
    (1, 'Tenant 1 - Budget plan'),
    (2, 'Tenant 2 - Strategy doc'),
    (2, 'Tenant 2 - Meeting notes'),
    (3, 'Tenant 3 - Product roadmap');

-- Enable Row-Level Security on the table
ALTER TABLE tenant_data ENABLE ROW LEVEL SECURITY;
```

Creating Policies

```sql
-- Each tenant can only see their own data
CREATE POLICY tenant_isolation ON tenant_data
    FOR ALL
    USING (tenant_id = current_setting('app.tenant_id')::INTEGER);
```

Let us break down this policy:

- `tenant_isolation` -- The name of the policy (you choose this).
- `ON tenant_data` -- The table this policy applies to.
- `FOR ALL` -- Applies to SELECT, INSERT, UPDATE, and DELETE.
- `USING (...)` -- The condition that must be true for a row to be visible.
- `current_setting('app.tenant_id')` -- Reads a session variable that your application sets.

How It Works

```sql
-- Application sets the tenant context when a user connects
SET app.tenant_id = '1';

-- Now queries only return Tenant 1's data
SELECT * FROM tenant_data;
```

```
+----+-----------+-------------------------+---------------------+
| id | tenant_id | data                    | created_at          |
+----+-----------+-------------------------+---------------------+
|  1 |         1 | Tenant 1 - Secret report | 2025-01-15 09:00:00 |
|  2 |         1 | Tenant 1 - Budget plan   | 2025-01-15 09:00:00 |
+----+-----------+-------------------------+---------------------+
(2 rows)
```

Even though there are five rows in the table, Tenant 1 only sees two. The other rows are completely invisible. It is as if they do not exist.

```sql
-- Switch to Tenant 2
SET app.tenant_id = '2';

SELECT * FROM tenant_data;
```

```
+----+-----------+-------------------------+---------------------+
| id | tenant_id | data                    | created_at          |
+----+-----------+-------------------------+---------------------+
|  3 |         2 | Tenant 2 - Strategy doc  | 2025-01-15 09:00:00 |
|  4 |         2 | Tenant 2 - Meeting notes | 2025-01-15 09:00:00 |
+----+-----------+-------------------------+---------------------+
(2 rows)
```

Separate Policies for Different Operations

You can create different policies for reading and writing:

```sql
-- Everyone can read public posts
CREATE POLICY read_public ON posts
    FOR SELECT
    USING (is_public = true);

-- Users can only edit their own posts
CREATE POLICY edit_own ON posts
    FOR UPDATE
    USING (author_id = current_setting('app.user_id')::INTEGER);
```

```
-- Users can only delete their own posts
CREATE POLICY delete_own ON posts
    FOR DELETE
    USING (author_id = current_setting('app.user_id')::INTEGER);
```

Important Note About Superusers

Row-Level Security does not apply to superusers or table owners by default. To force it on the table owner:

```sql
ALTER TABLE tenant_data FORCE ROW LEVEL SECURITY;
```

SQL Injection: The Most Dangerous Attack

SQL injection is one of the most common and devastating database attacks. Understanding it is critical for every developer.

What Is SQL Injection?

SQL injection happens when an attacker inserts malicious SQL code into a query through user input. It is like someone writing extra instructions on a form, and the system blindly follows them.

How It Works

Imagine a login form where a user enters their username and password. The application builds a query like this:

```
Dangerous code (DO NOT USE):

query = "SELECT * FROM users WHERE username = '" + username + "' AND password = '" + pa...
```

If a normal user types `alice` and `mypassword`, the query becomes:

```sql
SELECT * FROM users
WHERE username = 'alice' AND password = 'mypassword';
```

That works fine. But what if an attacker types this as the username:

```
' OR '1'='1' --
```

Now the query becomes:

```sql
SELECT * FROM users
WHERE username = '' OR '1'='1' --' AND password = 'anything';
```

Let us break down what happened:

```
Original:  WHERE username = '___INPUT___' AND password = '...'

Injected:  WHERE username = '' OR '1'='1' --' AND password = '...'
                                 ^^^^^^^^  ^^
                                 Always     Comment (ignores
                                 true!      the rest)
```

The `OR '1'='1'` is always true, so the query returns ALL users. The `--` is a comment that ignores the password check entirely. The attacker just logged in without knowing any password.

Even Worse: Destructive Injection

An attacker could type this as the username:

```
'; DROP TABLE users; --
```

The query becomes:

```sql
SELECT * FROM users WHERE username = ''; DROP TABLE users; --'...
```

This runs two commands: the SELECT query (which returns nothing), and then `DROP TABLE users`, which deletes the entire users table. Your application just lost all user data.

How SQL Injection Looks Step by Step

```
Step 1: Attacker finds a search box on your website
```

```
Step 2: Instead of typing a normal search term, they type:
        ' UNION SELECT username, password FROM users --

Step 3: Your vulnerable code builds:
        SELECT name, price FROM products
        WHERE name LIKE '' UNION SELECT username, password FROM users --'

Step 4: The database runs BOTH queries:
        - First: searches products (returns nothing)
        - Second: returns all usernames and passwords!

Step 5: The attacker sees everyone's credentials on the search results page
```

How to Prevent SQL Injection

The solution is simple: never put user input directly into SQL strings. Use parameterized queries instead.

Bad (vulnerable):

```python
# Python example - NEVER DO THIS
username = request.form['username']
query = "SELECT * FROM users WHERE username = '" + username + "'"
cursor.execute(query)
```

Good (safe):

```python
# Python example - ALWAYS DO THIS
username = request.form['username']
query = "SELECT * FROM users WHERE username = %s"
cursor.execute(query, (username,))
```

The difference is that `%s` is a placeholder. The database driver handles the input safely, escaping any special characters. The attacker's input is treated as data, not as SQL code.

Here is how it looks in different languages:

```
+------------------+----------------------------------+
| Language         | Parameterized Query Example      |
+------------------+----------------------------------+
| Python (psycopg) | cursor.execute(sql, (param,))    |
| Java (JDBC)      | stmt.setString(1, param)         |
| Node.js (pg)     | pool.query(sql, [param])         |
| PHP (PDO)        | $stmt->execute([$param])         |
| C# (Npgsql)      | cmd.Parameters.AddWithValue()    |
+------------------+----------------------------------+
```

Additional Prevention Measures

```sql
-- 1. Use least-privilege roles (limit what app user can do)
CREATE ROLE app_readonly WITH LOGIN PASSWORD 'readonly_pass';
GRANT SELECT ON ALL TABLES IN SCHEMA public TO app_readonly;

-- 2. Revoke unnecessary privileges
REVOKE CREATE ON SCHEMA public FROM PUBLIC;

-- 3. Use views to limit exposed data
CREATE VIEW safe_user_list AS
SELECT id, username, email, created_at
FROM users;
-- The view hides password_hash and other sensitive columns

GRANT SELECT ON safe_user_list TO app_readonly;
```

Even if an attacker manages to inject SQL, the damage is limited because the role does not have permission to drop tables or read password columns.

Encryption: Protecting Data at Rest and in Transit

Encryption converts readable data into an unreadable format that can only be decoded with the right key. Think of it as putting your data in a locked safe. Even if someone steals the safe, they cannot read what is inside without the key.

Encryption in Transit (SSL/TLS)

When data travels between your application and the database, it passes through networks. Without encryption, anyone who intercepts this traffic can read it. This is called a "man-in-the-middle" attack.

```
Without SSL:
  App  ----[plain text: "SELECT * FROM users"]----> Database
              ^
              |
          Attacker reads everything!

With SSL:
  App  ----[encrypted: "x8#kL9$mQ2..."]----> Database
              ^
```

```
                |
          Attacker sees gibberish
```

Configuring SSL in PostgreSQL

In `postgresql.conf`:

```
ssl = on
ssl_cert_file = '/path/to/server.crt'
ssl_key_file = '/path/to/server.key'
ssl_ca_file = '/path/to/ca.crt'
```

In `pg_hba.conf`, force SSL for remote connections:

```
# Require SSL for all remote connections
hostssl    all    all    0.0.0.0/0    scram-sha-256
```

Using `hostssl` instead of `host` means the connection must use SSL or it will be rejected.

Connecting with SSL from an Application

```python
# Python connection with SSL
import psycopg2

conn = psycopg2.connect(
    host='db.example.com',
    dbname='myapp',
    user='app_user',
    password='secure_password',
    sslmode='verify-full',
    sslrootcert='/path/to/ca.crt'
)
```

The `sslmode` options from least to most secure:

```
+---------------+-----------------------------------------+
| sslmode       | Behavior                                |
+---------------+-----------------------------------------+
| disable       | No SSL (not recommended)                |
| allow         | Try non-SSL first, then SSL             |
| prefer        | Try SSL first, then non-SSL (default)   |
| require       | Must use SSL, no certificate check      |
| verify-ca     | Must use SSL, verify CA certificate      |
| verify-full   | Must use SSL, verify CA + hostname       |
+---------------+-----------------------------------------+
```

For production, always use `verify-full`.

Encryption at Rest

Encryption at rest protects data stored on disk. If someone steals the hard drive, the data is unreadable.

Column-Level Encryption with pgcrypto

```sql
-- Enable the pgcrypto extension
CREATE EXTENSION IF NOT EXISTS pgcrypto;

-- Encrypt sensitive data before storing
INSERT INTO customers (name, email, ssn_encrypted)
VALUES (
    'Alice Smith',
    'alice@example.com',
    pgp_sym_encrypt('123-45-6789', 'my_encryption_key')
);

-- Decrypt when reading
SELECT
    name,
    email,
    pgp_sym_decrypt(ssn_encrypted, 'my_encryption_key') AS ssn
FROM customers
WHERE name = 'Alice Smith';
```

```
+-------------+-------------------+--------------+
| name        | email             | ssn          |
+-------------+-------------------+--------------+
| Alice Smith | alice@example.com | 123-45-6789  |
+-------------+-------------------+--------------+
(1 row)
```

The `ssn_encrypted` column stores the data in an unreadable format. Only someone with the encryption key can decrypt it.

Full-Disk Encryption

For complete protection, use full-disk encryption at the operating system level:

- Linux: LUKS (Linux Unified Key Setup)
- macOS: FileVault
- Windows: BitLocker

This encrypts everything on the disk, including the database files, logs, and temporary files.

Backup Strategies

Backups are your last line of defense. If everything else fails -- a hacker deletes your data, a disk crashes, or someone runs a bad query -- backups save you.

The rule of backups: "If you have not tested restoring from a backup, you do not have a backup."

pg_dump: Logical Backups

`pg_dump` creates a text file containing SQL commands that can recreate your database.

```bash
# Dump a single database to a SQL file
pg_dump -h localhost -U postgres myapp_db > backup_2025_01_15.sql

# Dump in custom format (compressed, more flexible)
pg_dump -h localhost -U postgres -Fc myapp_db > backup_2025_01_15.dump

# Dump only the schema (no data)
pg_dump -h localhost -U postgres --schema-only myapp_db > schema_only.sql

# Dump only the data (no schema)
pg_dump -h localhost -U postgres --data-only myapp_db > data_only.sql

# Dump a specific table
pg_dump -h localhost -U postgres -t orders myapp_db > orders_backup.sql
```

Let us understand the flags:

- `-h localhost` -- Connect to this host.
- `-U postgres` -- Connect as this user.
- `-Fc` -- Use custom format (compressed and allows selective restore).
- `-t orders` -- Only dump the `orders` table.
- `--schema-only` -- Structure without data.
- `--data-only` -- Data without structure.

pg_restore: Restoring Backups

```bash
# Restore from custom format dump
pg_restore -h localhost -U postgres -d myapp_db backup_2025_01_15.dump

# Restore only a specific table
pg_restore -h localhost -U postgres -d myapp_db -t orders backup_2025_01_15.dump

# Restore with clean (drop objects first)
pg_restore -h localhost -U postgres -d myapp_db --clean backup_2025_01_15.dump

# For plain SQL dumps, use psql instead
psql -h localhost -U postgres -d myapp_db < backup_2025_01_15.sql
```

pg_dumpall: Backing Up Everything

`pg_dumpall` backs up all databases on a server, plus global objects like roles and tablespaces:

```bash
# Backup everything including roles
pg_dumpall -h localhost -U postgres > full_backup.sql

# Backup only global objects (roles, tablespaces)
pg_dumpall -h localhost -U postgres --globals-only > globals.sql
```

Backup Strategy: The 3-2-1 Rule

A solid backup strategy follows the 3-2-1 rule:

```
+-------------------------------------------+
|          The 3-2-1 Backup Rule            |
+-------------------------------------------+
|                                           |
|  3 copies of your data                    |
|    - The original                         |
|    - A local backup                       |
|    - An offsite backup                    |
|                                           |
|  2 different types of storage             |
|    - Local disk                           |
|    - Cloud storage or tape                |
|                                           |
|  1 copy offsite                           |
|    - Different physical location          |
|    - Protects against fire, flood, theft  |
|                                           |
+-------------------------------------------+
```

Automated Backup Script

Here is a practical backup script you might use:

```bash
#!/bin/bash
# Daily PostgreSQL backup script

# Configuration
DB_NAME="myapp_db"
DB_USER="postgres"
BACKUP_DIR="/var/backups/postgresql"
DATE=$(date +%Y-%m-%d_%H%M%S)
RETENTION_DAYS=30

# Create backup directory if it does not exist
mkdir -p "$BACKUP_DIR"

# Create the backup
pg_dump -U "$DB_USER" -Fc "$DB_NAME" > "$BACKUP_DIR/${DB_NAME}_${DATE}.dump"

# Check if backup was successful
if [ $? -eq 0 ]; then
    echo "Backup successful: ${DB_NAME}_${DATE}.dump"
else
    echo "Backup FAILED!" >&2
    exit 1
fi

# Remove backups older than retention period
find "$BACKUP_DIR" -name "*.dump" -mtime +$RETENTION_DAYS -delete

echo "Old backups cleaned up (kept last $RETENTION_DAYS days)"
```

Practical Exercise: Setting Up Roles for an Application

Let us put everything together by setting up a complete security model for a web application.

The Scenario

You are building a blog application with three types of users:

1. Readers -- Can view published posts and comments.
2. Authors -- Can create and edit their own posts and comments.

3. Administrators -- Can manage everything, including users.

Step 1: Create the Database and Tables

```sql
-- Connect as superuser and create the database
CREATE DATABASE blog_app;

-- Connect to blog_app, then create tables
CREATE TABLE users (
    id          SERIAL PRIMARY KEY,
    username    VARCHAR(50) UNIQUE NOT NULL,
    email       VARCHAR(100) UNIQUE NOT NULL,
    role        VARCHAR(20) NOT NULL DEFAULT 'reader',
    created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE posts (
    id          SERIAL PRIMARY KEY,
    author_id   INTEGER REFERENCES users(id),
    title       VARCHAR(200) NOT NULL,
    body        TEXT NOT NULL,
    is_published BOOLEAN DEFAULT false,
    created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE comments (
    id          SERIAL PRIMARY KEY,
    post_id     INTEGER REFERENCES posts(id),
    user_id     INTEGER REFERENCES users(id),
    body        TEXT NOT NULL,
    created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Step 2: Create the Roles

```sql
-- Base roles (no login)
CREATE ROLE blog_reader;
CREATE ROLE blog_author;
CREATE ROLE blog_admin;

-- Build hierarchy: authors can read, admins can do everything
GRANT blog_reader TO blog_author;
GRANT blog_author TO blog_admin;
```

Step 3: Grant Privileges

```sql
-- Schema access for all roles
GRANT USAGE ON SCHEMA public TO blog_reader;
GRANT USAGE ON SCHEMA public TO blog_author;
GRANT USAGE ON SCHEMA public TO blog_admin;

-- Readers: can only SELECT published posts and all comments
GRANT SELECT ON posts TO blog_reader;
GRANT SELECT ON comments TO blog_reader;
GRANT SELECT ON users TO blog_reader;

-- Authors: can INSERT and UPDATE posts and comments
GRANT INSERT, UPDATE ON posts TO blog_author;
GRANT INSERT, UPDATE ON comments TO blog_author;
GRANT USAGE ON SEQUENCE posts_id_seq TO blog_author;
GRANT USAGE ON SEQUENCE comments_id_seq TO blog_author;

-- Admins: full control
GRANT ALL ON ALL TABLES IN SCHEMA public TO blog_admin;
GRANT ALL ON ALL SEQUENCES IN SCHEMA public TO blog_admin;
```

Step 4: Enable Row-Level Security

```sql
-- Enable RLS on posts
ALTER TABLE posts ENABLE ROW LEVEL SECURITY;

-- Readers can only see published posts
CREATE POLICY reader_see_published ON posts
    FOR SELECT
    TO blog_reader
    USING (is_published = true);

-- Authors can see published posts AND their own drafts
CREATE POLICY author_see_own ON posts
    FOR SELECT
    TO blog_author
    USING (
        is_published = true
        OR author_id = current_setting('app.user_id')::INTEGER
    );

-- Authors can only update their own posts
CREATE POLICY author_edit_own ON posts
    FOR UPDATE
    TO blog_author
    USING (author_id = current_setting('app.user_id')::INTEGER);

-- Authors can insert posts (as themselves)
CREATE POLICY author_insert ON posts
    FOR INSERT
    TO blog_author
```

```
    WITH CHECK (author_id = current_setting('app.user_id')::INTEGER);

-- Admins can do everything (bypass RLS)
CREATE POLICY admin_all ON posts
    FOR ALL
    TO blog_admin
    USING (true)
    WITH CHECK (true);
```

Step 5: Create Application Users

```sql
-- Create actual login users and assign roles
CREATE USER app_reader WITH PASSWORD 'reader_pass_789';
GRANT blog_reader TO app_reader;

CREATE USER app_author WITH PASSWORD 'author_pass_456';
GRANT blog_author TO app_author;

CREATE USER app_admin WITH PASSWORD 'admin_pass_123';
GRANT blog_admin TO app_admin;
```

Step 6: Test the Security

```sql
-- Insert test data as superuser
INSERT INTO users (username, email, role) VALUES
    ('alice', 'alice@blog.com', 'author'),
    ('bob', 'bob@blog.com', 'author'),
    ('admin', 'admin@blog.com', 'admin');

INSERT INTO posts (author_id, title, body, is_published) VALUES
    (1, 'Alice Published Post', 'Hello world!', true),
    (1, 'Alice Draft Post', 'Work in progress...', false),
    (2, 'Bob Published Post', 'Bob here!', true),
    (2, 'Bob Draft Post', 'Still writing...', false);
```

Now let us test as different users:

```sql
-- Test as reader
SET ROLE app_reader;
SELECT id, title, is_published FROM posts;
```

```
+----+----------------------+--------------+
| id | title                | is_published |
```

```
+----+----------------------+--------------+
|  1 | Alice Published Post | t            |
|  3 | Bob Published Post   | t            |
+----+----------------------+--------------+
(2 rows)
```

The reader only sees published posts. The drafts are invisible.

```sql
-- Test as Alice (author)
RESET ROLE;
SET ROLE app_author;
SET app.user_id = '1';

SELECT id, title, is_published FROM posts;
```

```
+----+----------------------+--------------+
| id | title                | is_published |
+----+----------------------+--------------+
|  1 | Alice Published Post | t            |
|  2 | Alice Draft Post     | f            |
|  3 | Bob Published Post   | t            |
+----+----------------------+--------------+
(3 rows)
```

Alice sees all published posts plus her own draft. She cannot see Bob's draft.

```sql
-- Alice tries to update Bob's post
UPDATE posts SET title = 'Hacked!' WHERE id = 3;
```

```
UPDATE 0
```

Zero rows updated. The RLS policy prevented Alice from modifying Bob's post.

The Complete Security Architecture

```
+-------------------------------------------------------------+
|                     Blog Application                         |
+-------------------------------------------------------------+
|                                                             |
|  Web Browser                                                |
|       |                                                     |
|       v                                                     |
|  Application Server (sets app.user_id per session)          |
```

```
|       |                                                        |
|       v (SSL/TLS encrypted connection)                         |
|  PostgreSQL                                                    |
|       |                                                        |
|       +-- pg_hba.conf (authentication)                         |
|       |       "Are you allowed to connect?"                    |
|       |                                                        |
|       +-- Roles & Privileges (authorization)                   |
|       |       "Can you access this table?"                     |
|       |                                                        |
|       +-- Row-Level Security (fine-grained control)            |
|       |       "Can you see this specific row?"                 |
|       |                                                        |
|       +-- Encryption at rest (pgcrypto / disk)                 |
|       |       "Data safe even if disk is stolen"               |
|       |                                                        |
|       +-- Backups (pg_dump)                                    |
|               "Recovery if all else fails"                     |
|                                                                |
+----------------------------------------------------------------+
```

Common Mistakes

1. Using the superuser for your application. The `postgres` superuser should only be used for administration. Create a dedicated role with minimal privileges for your application.
2. Forgetting to grant USAGE on schemas. Even if you grant `SELECT` on a table, the user cannot access it without `USAGE` on the schema.
3. Not setting default privileges. New tables created after your `GRANT` statements will not have the privileges you expect. Use `ALTER DEFAULT PRIVILEGES`.
4. Building SQL queries with string concatenation. This is the number one cause of SQL injection. Always use parameterized queries.
5. Storing passwords in plain text. Use PostgreSQL's `scram-sha-256` authentication and never store application passwords without hashing (use bcrypt or argon2 in your application).
6. Not testing backup restoration. A backup you have never tested might be corrupted or incomplete. Regularly test restoring to a separate server.
7. Leaving SSL disabled for remote connections. Without SSL, passwords and data travel across the network in plain text.

Best Practices

1. Follow the Principle of Least Privilege. Give each role only the minimum permissions it needs. Start with nothing and add privileges as required.
2. Use role hierarchy. Create base roles for common permission sets and assign users to these roles. This makes maintenance much easier.
3. Enable RLS for multi-tenant applications. Row-Level Security is the most reliable way to ensure data isolation between tenants.
4. Always use parameterized queries. There is no exception to this rule. Every query that includes user input must use parameters.
5. Encrypt sensitive data. Use SSL for connections and pgcrypto for sensitive columns like social security numbers or credit card data.
6. Automate backups. Set up a cron job to run backups daily. Test restoration monthly. Keep backups in multiple locations.
7. Rotate passwords regularly. Use the `VALID UNTIL` clause on roles and update passwords on a schedule.
8. Audit access. Enable PostgreSQL logging to track who accesses what data and when.

Quick Summary

Database security is built in layers:

- Authentication (pg_hba.conf) verifies identity.
- Authorization (GRANT/REVOKE) controls table-level access.
- Row-Level Security controls row-level access.
- Parameterized queries prevent SQL injection.
- Encryption protects data in transit (SSL) and at rest (pgcrypto).
- Backups (pg_dump/pg_restore) provide recovery from disasters.

Key Points

- Authentication verifies who you are. Authorization controls what you can do.
- `CREATE ROLE` creates a role without login. `CREATE USER` creates a role with login.
- `GRANT` gives privileges. `REVOKE` takes them away.

- Role hierarchy simplifies permission management by letting roles inherit from other roles.
- Row-Level Security (RLS) controls which rows a user can see or modify.
- SQL injection is prevented by using parameterized queries. Never concatenate user input into SQL.
- SSL/TLS encrypts data traveling between your application and the database.
- pgcrypto can encrypt sensitive columns stored in the database.
- pg_dump creates logical backups. pg_restore recovers from them.
- Follow the 3-2-1 rule: 3 copies, 2 storage types, 1 offsite.

Practice Questions

1. What is the difference between authentication and authorization? Give a real-life example of each.
2. You have a table called `invoices` and a role called `accountant`. Write the SQL to grant the accountant read access and the ability to insert new invoices, but not delete or update them.
3. Explain what SQL injection is and why this code is dangerous:

```python
query = "SELECT * FROM users WHERE id = " + user_input
```

How would you fix it?

4. You are building a SaaS application where each company should only see their own data. Which PostgreSQL feature would you use, and how does it work?
5. What is the difference between `host` and `hostssl` in `pg_hba.conf`? Which one should you use for production?

Exercises

Exercise 1: Set Up a Secure Role System

Create a role system for a hospital database with the following requirements:

- Doctors can read all patient records and update diagnosis fields.
- Nurses can read patient records and update vital signs.

- Receptionists can read patient names and appointment dates but cannot see medical details.
- Administrators can do everything.

Create the roles, build the hierarchy, and write the GRANT statements.

Exercise 2: Implement Row-Level Security

You have a `documents` table in a company where each department should only see its own documents. Create the table, enable RLS, and write policies so that:

- Users in the HR department only see HR documents.
- Users in the Engineering department only see Engineering documents.
- The CEO can see all documents.

Exercise 3: Create a Backup and Restore Plan

Write a shell script that:

1. Creates a compressed backup of a database called `production_db`.
2. Stores it in `/var/backups/` with a timestamp in the filename.
3. Removes backups older than 7 days.
4. Logs the result to a file.

Then write the command to restore from one of these backups to a new database called `production_db_restored`.

What Is Next?

You now know how to lock down your database like a vault. In the next chapter, you will put everything you have learned throughout this book into practice by building a complete e-commerce database from scratch. You will design the schema, create the tables, write queries, build views, add indexes, and implement triggers and functions -- a full, real-world project.

Chapter 32: Project -- Build a Complete E-Commerce Database

What You Will Learn

In this chapter, you will build a complete e-commerce database from scratch. You will walk through every step of a real database project: gathering requirements, designing an ER diagram, creating normalized tables, inserting sample data, writing essential queries, building views, adding indexes, implementing triggers, and creating functions. By the end, you will have a fully functional database that could power an online store.

Why This Chapter Matters

Throughout this book, you have learned SQL concepts one at a time. Now it is time to combine them all into a single, cohesive project. This is how real database work happens -- you do not just write one query in isolation. You design a system where tables, constraints, indexes, views, triggers, and functions all work together.

Building an e-commerce database is the perfect project because it touches almost every concept you have learned: data types, relationships, joins, aggregations, window functions, normalization, constraints, indexes, views, triggers, and stored functions.

Think of this chapter as your graduation project. When you finish it, you will have real, portfolio-worthy work.

Step 1: Gathering Requirements

Before writing any SQL, we need to understand what we are building. Let us talk to our imaginary client.

The client says:

"We are building an online store. Customers need to create accounts, browse products organized by categories, add items to their cart, place orders, leave reviews, and make payments. We need to track inventory, calculate totals, and generate reports."

From this conversation, we extract the main entities:

```
+------------------+----------------------------------------------+
| Entity           | What It Represents                           |
+------------------+----------------------------------------------+
| Users            | Customer accounts                            |
| Categories       | Product groupings (Electronics, Books, etc.) |
| Products         | Items for sale                               |
| Orders           | Customer purchases                           |
| Order Items      | Individual items within an order             |
| Reviews          | Customer feedback on products                |
| Payments         | Payment records for orders                   |
| Addresses        | Shipping and billing addresses               |
+------------------+----------------------------------------------+
```

Key Business Rules

Before designing tables, we document the rules that govern the data:

1. A user can have multiple addresses (home, work, etc.).
2. A product belongs to exactly one category.
3. A product can have many reviews, but a user can review a product only once.
4. An order contains one or more items.
5. Each order item refers to a specific product with a quantity and price at the time of purchase.
6. An order has exactly one payment.
7. Products have a stock quantity that decreases when ordered.
8. We store the price on the order item (not just the product) because prices change over time.

Step 2: ER Diagram

Here is the Entity-Relationship diagram for our e-commerce database:

```
+----------------+          +----------------+
|   categories   |          |    users       |
+----------------+          +----------------+
| id        (PK) |          | id        (PK) |
| name           |          | email          |
| description    |          | username       |
| created_at     |          | password_hash  |
+-------+--------+          | first_name     |
        |                   | last_name      |
        | 1                 | phone          |
        |                   | created_at     |
        | M                 | updated_at     |
+-------+--------+          +---+-----+------+
|   products     |              |     |
+----------------+              |     |
| id        (PK) |              |     |
| category_id(FK)|              |     |
| name           |         1    |     | 1
| description    |         |    |     |
| price          |         M    |     M
| stock_quantity |   +-----+----+-+  ++------------+
| image_url      |   |   addresses |  |   orders    |
| is_active      |   +-----------+   +-------------+
| created_at     |   | id    (PK)|   | id      (PK)|
| updated_at     |   | user_id(FK)|   | user_id (FK)|
+--+----------+--+   | type      |   | address_id  |
   |          |       | street    |   | status      |
   |          |       | city      |   | total_amount|
   |          |       | state     |   | created_at  |
   |          |       | zip_code  |   | updated_at  |
   | 1        | 1     | country   |   +---+-----+---+
   |          |       +-----------+       |     |
   | M        | M                         |     |
   |    +-----+------+             1     |     | 1
   |    |   reviews   |             |     |     |
   |    +-------------+             M     |     M
   |    | id      (PK)|    +--------+--+  |  +--+----------+
   |    | product_id  |    | order_items|  |  |  payments   |
   |    | user_id (FK)|    +--------+--+  |  +-------------+
   |    | rating      |    | id    (PK)|  |  | id      (PK)|
   |    | comment     |    | order_id  +--+  | order_id(FK)|
   |    | created_at  |    | product_id+-----+ amount      |
   |    +-------------+    | quantity  |     | method      |
   |                       | unit_price|     | status      |
   +----------+            | subtotal  |     | paid_at     |
              |            +-----------+     +-------------+
              |                  |
              +------------------+
                  (FK to products)
```

Relationships Summary

```
+-------------------+---------------+---------------------------+
| Relationship      | Type          | Description               |
+-------------------+---------------+---------------------------+
| users-addresses   | One-to-Many   | User has many addresses   |
| users-orders      | One-to-Many   | User places many orders   |
| users-reviews     | One-to-Many   | User writes many reviews  |
| categories-products| One-to-Many  | Category has many products|
| products-reviews  | One-to-Many   | Product has many reviews  |
| orders-order_items| One-to-Many   | Order has many items      |
| products-order_items| One-to-Many | Product in many order items|
| orders-payments   | One-to-One    | Order has one payment     |
+-------------------+---------------+---------------------------+
```

Step 3: Schema Design

Now let us create the actual tables. Each table is carefully designed with proper data types, constraints, and relationships.

Users Table

```sql
CREATE TABLE users (
    id              SERIAL PRIMARY KEY,
    email           VARCHAR(255) UNIQUE NOT NULL,
    username        VARCHAR(50) UNIQUE NOT NULL,
    password_hash   VARCHAR(255) NOT NULL,
    first_name      VARCHAR(100) NOT NULL,
    last_name       VARCHAR(100) NOT NULL,
    phone           VARCHAR(20),
    is_active       BOOLEAN DEFAULT true,
    created_at      TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at      TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Line-by-line explanation:

- `id SERIAL PRIMARY KEY` -- Auto-incrementing unique identifier for each user.
- `email VARCHAR(255) UNIQUE NOT NULL` -- Email must be unique and cannot be empty. 255 characters covers any valid email.
- `username VARCHAR(50) UNIQUE NOT NULL` -- Short, unique display name.
- `password_hash VARCHAR(255) NOT NULL` -- We never store plain passwords. The application hashes them before insertion.
- `first_name`, `last_name` -- Real name for shipping and display.

- `phone VARCHAR(20)` -- Optional phone number. We use VARCHAR because phone numbers can have dashes, spaces, or country codes.
- `is_active BOOLEAN DEFAULT true` -- Allows soft-deleting users instead of removing their data.
- `created_at`, `updated_at` -- Track when the record was created and last modified.

Categories Table

```sql
CREATE TABLE categories (
    id              SERIAL PRIMARY KEY,
    name            VARCHAR(100) UNIQUE NOT NULL,
    description     TEXT,
    parent_id       INTEGER REFERENCES categories(id),
    created_at      TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

- `parent_id INTEGER REFERENCES categories(id)` -- This is a self-referencing foreign key. It allows categories to have subcategories. For example, "Laptops" could be a child of "Electronics."

Products Table

```sql
CREATE TABLE products (
    id              SERIAL PRIMARY KEY,
    category_id     INTEGER NOT NULL REFERENCES categories(id),
    name            VARCHAR(200) NOT NULL,
    description     TEXT,
    price           NUMERIC(10, 2) NOT NULL CHECK (price > 0),
    stock_quantity  INTEGER NOT NULL DEFAULT 0 CHECK (stock_quantity >= 0),
    image_url       VARCHAR(500),
    is_active       BOOLEAN DEFAULT true,
    created_at      TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at      TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

- `price NUMERIC(10, 2) NOT NULL CHECK (price > 0)` -- Stores prices with exactly 2 decimal places. The CHECK constraint ensures prices are always positive. `NUMERIC(10,2)` supports values up to 99,999,999.99.
- `stock_quantity INTEGER NOT NULL DEFAULT 0 CHECK (stock_quantity >= 0)` -- Stock cannot go negative. This is a critical business rule enforced at the database level.

Addresses Table

```sql
CREATE TABLE addresses (
    id              SERIAL PRIMARY KEY,
    user_id         INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    address_type    VARCHAR(20) NOT NULL CHECK (address_type IN ('shipping', 'billing')),
    street          VARCHAR(255) NOT NULL,
    city            VARCHAR(100) NOT NULL,
    state           VARCHAR(100) NOT NULL,
    zip_code        VARCHAR(20) NOT NULL,
    country         VARCHAR(100) NOT NULL DEFAULT 'United States',
    is_default      BOOLEAN DEFAULT false,
    created_at      TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

- `ON DELETE CASCADE` -- If a user is deleted, their addresses are automatically removed.
- `address_type ... CHECK (...)` -- Limits address types to 'shipping' or 'billing'. This prevents invalid data.
- `is_default BOOLEAN DEFAULT false` -- Marks the user's preferred address.

Orders Table

```sql
CREATE TABLE orders (
    id              SERIAL PRIMARY KEY,
    user_id         INTEGER NOT NULL REFERENCES users(id),
    address_id      INTEGER REFERENCES addresses(id),
    status          VARCHAR(20) NOT NULL DEFAULT 'pending'
                    CHECK (status IN ('pending', 'confirmed', 'shipped',
                                      'delivered', 'cancelled', 'refunded')),
    total_amount    NUMERIC(12, 2) NOT NULL DEFAULT 0,
    notes           TEXT,
    created_at      TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at      TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

- `status ... CHECK (...)` -- Only allows valid order statuses. This prevents typos and invalid states.
- `total_amount NUMERIC(12, 2)` -- Supports order totals up to $9,999,999,999.99.

Order Items Table

```sql
CREATE TABLE order_items (
    id              SERIAL PRIMARY KEY,
    order_id        INTEGER NOT NULL REFERENCES orders(id) ON DELETE CASCADE,
    product_id      INTEGER NOT NULL REFERENCES products(id),
    quantity        INTEGER NOT NULL CHECK (quantity > 0),
    unit_price      NUMERIC(10, 2) NOT NULL CHECK (unit_price > 0),
```

```
    subtotal        NUMERIC(12, 2) NOT NULL,
    created_at      TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    UNIQUE(order_id, product_id)
);
```

- `unit_price` -- We store the price at the time of purchase, not a reference to the product's current price. Prices change, but order history should not.
- `subtotal` -- Pre-calculated as `quantity * unit_price` for performance.
- `UNIQUE(order_id, product_id)` -- A product can appear only once per order (use quantity for multiples).

Reviews Table

```sql
CREATE TABLE reviews (
    id              SERIAL PRIMARY KEY,
    product_id      INTEGER NOT NULL REFERENCES products(id) ON DELETE CASCADE,
    user_id         INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    rating          INTEGER NOT NULL CHECK (rating BETWEEN 1 AND 5),
    title           VARCHAR(200),
    comment         TEXT,
    created_at      TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    UNIQUE(product_id, user_id)
);
```

- `rating INTEGER NOT NULL CHECK (rating BETWEEN 1 AND 5)` -- Ratings must be 1 through 5 stars.
- `UNIQUE(product_id, user_id)` -- A user can only review a product once.

Payments Table

```sql
CREATE TABLE payments (
    id              SERIAL PRIMARY KEY,
    order_id        INTEGER UNIQUE NOT NULL REFERENCES orders(id),
    amount          NUMERIC(12, 2) NOT NULL CHECK (amount > 0),
    payment_method  VARCHAR(50) NOT NULL
                    CHECK (payment_method IN ('credit_card', 'debit_card',
                                              'paypal', 'bank_transfer')),
    status          VARCHAR(20) NOT NULL DEFAULT 'pending'
                    CHECK (status IN ('pending', 'completed', 'failed', 'refunded')),
    transaction_id  VARCHAR(100),
    paid_at         TIMESTAMP,
    created_at      TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

- `order_id INTEGER UNIQUE NOT NULL` -- The UNIQUE constraint ensures each order has at most one payment, creating a one-to-one relationship.
- `transaction_id VARCHAR(100)` -- Stores the external payment processor's reference number.

Step 4: Seed Data

Let us populate the database with realistic sample data.

Insert Categories

```sql
INSERT INTO categories (name, description, parent_id) VALUES
    ('Electronics',    'Electronic devices and accessories',         NULL),
    ('Books',          'Physical and digital books',                 NULL),
    ('Clothing',       'Apparel and fashion items',                  NULL),
    ('Home & Garden',  'Home improvement and garden supplies',       NULL),
    ('Laptops',        'Portable computers',                         1),
    ('Smartphones',    'Mobile phones',                              1),
    ('Fiction',        'Fiction books and novels',                   2),
    ('Non-Fiction',    'Educational and informational books',        2),
    ('Men',            'Men clothing',                               3),
    ('Women',          'Women clothing',                             3);
```

```
+----+----------------+-------------------------------------------+-----------+
| id | name           | description                               | parent_id |
+----+----------------+-------------------------------------------+-----------+
|  1 | Electronics    | Electronic devices and accessories        |      NULL |
|  2 | Books          | Physical and digital books                |      NULL |
|  3 | Clothing       | Apparel and fashion items                 |      NULL |
|  4 | Home & Garden  | Home improvement and garden supplies      |      NULL |
|  5 | Laptops        | Portable computers                        |         1 |
|  6 | Smartphones    | Mobile phones                             |         1 |
|  7 | Fiction        | Fiction books and novels                  |         2 |
|  8 | Non-Fiction    | Educational and informational books       |         2 |
|  9 | Men            | Men clothing                              |         3 |
| 10 | Women          | Women clothing                            |         3 |
+----+----------------+-------------------------------------------+-----------+
(10 rows)
```

Insert Users

```sql
INSERT INTO users (email, username, password_hash, first_name, last_name, phone) VALUES
    ('alice@example.com',   'alice_j',   '$2b$12$hash1...', 'Alice',   'Johnson',  '555...
    ('bob@example.com',     'bob_smith',  '$2b$12$hash2...', 'Bob',     'Smith',    '55...
    ('carol@example.com',   'carol_w',    '$2b$12$hash3...', 'Carol',   'Williams', '55...
    ('david@example.com',   'david_b',    '$2b$12$hash4...', 'David',   'Brown',    '55...
    ('eve@example.com',     'eve_d',      '$2b$12$hash5...', 'Eve',     'Davis',    '55...
    ('frank@example.com',   'frank_m',    '$2b$12$hash6...', 'Frank',   'Miller',   '55...
    ('grace@example.com',   'grace_t',    '$2b$12$hash7...', 'Grace',   'Taylor',   NULL),
    ('henry@example.com',   'henry_a',    '$2b$12$hash8...', 'Henry',   'Anderson', '55...
```

Insert Addresses

```sql
INSERT INTO addresses (user_id, address_type, street, city, state, zip_code, country, i...
    (1, 'shipping', '123 Oak Street',    'Portland',    'Oregon',       '97201', 'Unite...
    (1, 'billing',  '123 Oak Street',    'Portland',    'Oregon',       '97201', 'Unite...
    (2, 'shipping', '456 Maple Avenue',  'Seattle',     'Washington',   '98101', 'Unite...
    (3, 'shipping', '789 Pine Road',     'San Francisco','California',  '94102', 'Unite...
    (4, 'shipping', '321 Elm Drive',     'Denver',      'Colorado',     '80201', 'Unite...
    (5, 'shipping', '654 Cedar Lane',    'Austin',      'Texas',        '73301', 'Unite...
    (6, 'shipping', '987 Birch Court',   'Chicago',     'Illinois',     '60601', 'Unite...
    (7, 'shipping', '147 Willow Way',    'Miami',       'Florida',      '33101', 'Unite...
    (8, 'shipping', '258 Spruce Place',  'New York',    'New York',     '10001', 'Unite...
```

Insert Products

```sql
INSERT INTO products (category_id, name, description, price, stock_quantity, is_active)...
    (5,  'ProBook Laptop 15',       'High-performance 15-inch laptop with 16GB RAM',   ...
    (5,  'UltraSlim Notebook',      'Lightweight 13-inch notebook for everyday use',   ...
    (6,  'SmartPhone X200',         'Latest smartphone with 128GB storage',            ...
    (6,  'SmartPhone Lite',         'Budget-friendly smartphone with great camera',    ...
    (7,  'The Great Adventure',     'A thrilling fiction novel about exploration',     ...
    (7,  'Mystery at Midnight',     'A page-turning mystery thriller',                 ...
    (8,  'Learn SQL in 30 Days',    'Complete guide to SQL for beginners',             ...
    (8,  'Data Science Handbook',   'Comprehensive reference for data professionals',  ...
    (9,  'Classic Cotton T-Shirt',  'Comfortable cotton t-shirt in multiple colors',   ...
    (9,  'Slim Fit Jeans',          'Modern slim fit denim jeans',                     ...
    (10, 'Summer Dress',            'Light and breezy summer dress with floral pattern'...
    (10, 'Wool Blend Sweater',      'Warm wool blend sweater for cold weather',        ...
    (4,  'Garden Tool Set',         '5-piece stainless steel garden tool set',         ...
    (4,  'Indoor Plant Pot',        'Ceramic indoor plant pot with drainage',          ...
    (1,  'Wireless Earbuds',        'Bluetooth wireless earbuds with noise cancellation...
```

Insert Orders

```sql
INSERT INTO orders (user_id, address_id, status, total_amount, created_at) VALUES
    (1, 1, 'delivered',  1049.98, '2025-01-05 10:30:00'),
    (1, 1, 'delivered',    29.99, '2025-02-14 14:20:00'),
    (2, 3, 'delivered',   849.98, '2025-01-10 09:15:00'),
    (2, 3, 'shipped',     349.99, '2025-03-01 16:45:00'),
    (3, 4, 'delivered',   114.97, '2025-01-20 11:00:00'),
    (3, 4, 'delivered',    59.99, '2025-02-28 13:30:00'),
    (4, 5, 'confirmed',  1029.98, '2025-03-10 08:00:00'),
    (5, 6, 'delivered',    64.98, '2025-01-15 15:00:00'),
    (5, 6, 'pending',     799.99, '2025-03-15 10:00:00'),
    (6, 7, 'delivered',   159.98, '2025-02-01 12:00:00'),
    (7, 8, 'delivered',    39.99, '2025-02-10 09:30:00'),
    (8, 9, 'cancelled',   699.99, '2025-03-05 14:00:00'),
    (1, 1, 'delivered',    79.98, '2025-03-20 11:15:00'),
    (3, 4, 'pending',     129.99, '2025-03-25 16:00:00');
```

Insert Order Items

```sql
INSERT INTO order_items (order_id, product_id, quantity, unit_price, subtotal) VALUES
    (1,  1,  1,  999.99,   999.99),
    (1,  9,  1,   49.99,    49.99),
    (2,  7,  1,   29.99,    29.99),
    (3,  3,  1,  799.99,   799.99),
    (3,  9,  1,   49.99,    49.99),
    (4,  4,  1,  349.99,   349.99),
    (5,  5,  3,   14.99,    44.97),
    (5,  6,  2,   12.99,    25.98),
    (5, 13,  1,   34.99,    34.99),
    (5, 14,  1,    9.03,     9.03),
    (6, 11,  1,   59.99,    59.99),
    (7,  1,  1,  999.99,   999.99),
    (7,  7,  1,   29.99,    29.99),
    (8,  9,  2,   19.99,    39.98),
    (8, 14,  1,   24.99,    24.99),
    (9,  3,  1,  799.99,   799.99),
    (10, 12, 2,   79.99,   159.98),
    (11,  8, 1,   39.99,    39.99),
    (12,  2, 1,  699.99,   699.99),
    (13,  9, 2,   19.99,    39.98),
    (13, 10, 1,   49.99,    49.99),
    (13, 14, 1,   24.99,    24.99),
    (14, 15, 1,  129.99,   129.99);
```

Insert Reviews

```sql
INSERT INTO reviews (product_id, user_id, rating, title, comment, created_at) VALUES
    (1,  1, 5, 'Excellent laptop',          'Fast, reliable, great battery life.',     ...
    (1,  3, 4, 'Good but heavy',            'Great performance but a bit heavy to carry...
    (3,  2, 5, 'Best phone ever',           'Amazing camera and smooth performance.',  ...
    (3,  5, 4, 'Great phone',               'Love the camera. Battery could be better.'...
    (7,  1, 5, 'Must read!',                'Learned more than I expected. Highly recom...
    (7,  3, 4, 'Very helpful',              'Good for beginners. Could use more example...
    (5,  3, 5, 'Couldn''t put it down',     'Read it in one sitting. Amazing story.',  ...
    (9,  5, 4, 'Comfortable',               'Fits well, comfortable material.',        ...
    (9,  1, 3, 'Decent quality',            'OK for the price, but shrinks after washin...
    (11, 3, 5, 'Beautiful dress',           'Perfect for summer. Love the pattern.',   ...
    (12, 6, 4, 'Warm and cozy',             'Perfect for winter. Slight pilling after w...
    (15, 5, 5, 'Amazing sound quality',     'Great noise cancellation and battery life....
    (8,  7, 4, 'Comprehensive reference',   'Covers everything you need. Well organized...
    (4,  2, 4, 'Good budget phone',         'Great value for money. Does everything I n...
    (13, 5, 5, 'Built to last',             'Sturdy tools. Great for weekend gardening....
```

Insert Payments

```sql
INSERT INTO payments (order_id, amount, payment_method, status, transaction_id, paid_at...
    (1,  1049.98, 'credit_card',   'completed', 'TXN-001-2025', '2025-01-05 10:31:00'),
    (2,    29.99, 'credit_card',   'completed', 'TXN-002-2025', '2025-02-14 14:21:00'),
    (3,   849.98, 'paypal',        'completed', 'TXN-003-2025', '2025-01-10 09:16:00'),
    (4,   349.99, 'debit_card',    'completed', 'TXN-004-2025', '2025-03-01 16:46:00'),
    (5,   114.97, 'credit_card',   'completed', 'TXN-005-2025', '2025-01-20 11:01:00'),
    (6,    59.99, 'paypal',        'completed', 'TXN-006-2025', '2025-02-28 13:31:00'),
    (7,  1029.98, 'credit_card',   'pending',    NULL,           NULL),
    (8,    64.98, 'bank_transfer', 'completed', 'TXN-008-2025', '2025-01-15 15:01:00'),
    (9,   799.99, 'credit_card',   'pending',    NULL,           NULL),
    (10,  159.98, 'credit_card',   'completed', 'TXN-010-2025', '2025-02-01 12:01:00'),
    (11,   39.99, 'debit_card',    'completed', 'TXN-011-2025', '2025-02-10 09:31:00'),
    (12,  699.99, 'credit_card',   'refunded',  'TXN-012-2025', '2025-03-05 14:01:00'),
    (13,   79.98, 'paypal',        'completed', 'TXN-013-2025', '2025-03-20 11:16:00'),
    (14,  129.99, 'credit_card',   'pending',    NULL,           NULL);
```

Step 5: Essential Queries

Now that we have data, let us write the queries that a real e-commerce application would need.

Query 1: Top-Selling Products

Which products sell the most?

```sql
SELECT
    p.name                          AS product_name,
    c.name                          AS category,
    SUM(oi.quantity)                AS total_sold,
    SUM(oi.subtotal)                AS total_revenue,
    ROUND(AVG(r.rating), 1)        AS avg_rating
FROM products p
JOIN categories c ON c.id = p.category_id
JOIN order_items oi ON oi.product_id = p.id
JOIN orders o ON o.id = oi.order_id AND o.status != 'cancelled'
LEFT JOIN reviews r ON r.product_id = p.id
GROUP BY p.id, p.name, c.name
ORDER BY total_sold DESC
LIMIT 10;
```

```
+------------------------+--------------+------------+---------------+------------+
| product_name           | category     | total_sold | total_revenue | avg_rating |
+------------------------+--------------+------------+---------------+------------+
| Classic Cotton T-Shirt | Men          |          5 |        149.95 |        3.5 |
| Indoor Plant Pot       | Home & Garden|          3 |         59.01 |       NULL |
| Slim Fit Jeans         | Men          |          2 |         99.98 |       NULL |
| ProBook Laptop 15      | Laptops      |          2 |       1999.98 |        4.5 |
| SmartPhone X200        | Smartphones  |          2 |       1599.98 |        4.5 |
| Learn SQL in 30 Days   | Non-Fiction  |          2 |         59.98 |        4.5 |
| The Great Adventure    | Fiction      |          3 |         44.97 |        5.0 |
| Mystery at Midnight    | Fiction      |          2 |         25.98 |       NULL |
| SmartPhone Lite        | Smartphones  |          1 |        349.99 |        4.0 |
| Summer Dress           | Women        |          1 |         59.99 |        5.0 |
+------------------------+--------------+------------+---------------+------------+
(10 rows)
```

Let us break down this query:

- We join `products` with `categories` to get category names.
- We join with `order_items` to count sales and revenue.
- We filter out cancelled orders with `o.status != 'cancelled'`.
- We LEFT JOIN with `reviews` because not all products have reviews.
- We group by product to aggregate the totals.
- We sort by `total_sold` descending to see top sellers first.

Query 2: Customer Order History

Show a customer's complete order history:

```sql
SELECT
    o.id                            AS order_id,
    o.created_at::DATE              AS order_date,
    o.status,
    COUNT(oi.id)                    AS item_count,
    o.total_amount,
    p.status                        AS payment_status,
    p.payment_method
FROM orders o
JOIN order_items oi ON oi.order_id = o.id
JOIN payments p ON p.order_id = o.id
WHERE o.user_id = 1
GROUP BY o.id, o.created_at, o.status, o.total_amount, p.status, p.payment_method
ORDER BY o.created_at DESC;
```

```
+----------+------------+-----------+------------+--------------+----------------+-----...
| order_id | order_date | status    | item_count | total_amount | payment_status | paym...
+----------+------------+-----------+------------+--------------+----------------+-----...
|       13 | 2025-03-20 | delivered |          3 |        79.98 | completed      | payp...
|        2 | 2025-02-14 | delivered |          1 |        29.99 | completed      | cred...
|        1 | 2025-01-05 | delivered |          2 |      1049.98 | completed      | cred...
+----------+------------+-----------+------------+--------------+----------------+-----...
(3 rows)
```

Query 3: Revenue by Month

How much revenue are we making each month?

```sql
SELECT
    TO_CHAR(o.created_at, 'YYYY-MM')    AS month,
    COUNT(DISTINCT o.id)                 AS order_count,
    SUM(o.total_amount)                  AS gross_revenue,
    SUM(CASE WHEN o.status = 'cancelled' THEN o.total_amount ELSE 0 END)
                                         AS cancelled_amount,
    SUM(CASE WHEN o.status != 'cancelled' THEN o.total_amount ELSE 0 END)
                                         AS net_revenue
FROM orders o
GROUP BY TO_CHAR(o.created_at, 'YYYY-MM')
ORDER BY month;
```

```
+---------+-------------+---------------+------------------+-------------+
| month   | order_count | gross_revenue | cancelled_amount | net_revenue |
+---------+-------------+---------------+------------------+-------------+
| 2025-01 | 4           |       2079.91 |             0.00 |     2079.91 |
| 2025-02 | 3           |        229.96 |             0.00 |      229.96 |
| 2025-03 | 7           |       3169.91 |           699.99 |     2469.92 |
+---------+-------------+---------------+------------------+-------------+
(3 rows)
```

This query uses `CASE` expressions to separate cancelled orders from net revenue.

Query 4: Inventory Check -- Low Stock Products

Which products are running low on stock?

```sql
SELECT
    p.name,
    p.stock_quantity,
    c.name AS category,
    CASE
        WHEN p.stock_quantity = 0 THEN 'OUT OF STOCK'
        WHEN p.stock_quantity < 20 THEN 'CRITICAL'
        WHEN p.stock_quantity < 50 THEN 'LOW'
        WHEN p.stock_quantity < 100 THEN 'MODERATE'
        ELSE 'GOOD'
    END AS stock_status
FROM products p
JOIN categories c ON c.id = p.category_id
WHERE p.is_active = true
ORDER BY p.stock_quantity ASC;
```

```
+------------------------+----------------+---------------+--------------+
| name                   | stock_quantity | category      | stock_status |
+------------------------+----------------+---------------+--------------+
| ProBook Laptop 15      |             50 | Laptops       | MODERATE     |
| UltraSlim Notebook     |             75 | Laptops       | MODERATE     |
| Garden Tool Set        |             80 | Home & Garden | MODERATE     |
| SmartPhone X200        |            100 | Smartphones   | GOOD         |
| Indoor Plant Pot       |            120 | Home & Garden | GOOD         |
| Data Science Handbook  |            150 | Non-Fiction   | GOOD         |
| Wool Blend Sweater     |            150 | Women         | GOOD         |
| SmartPhone Lite        |            200 | Smartphones   | GOOD         |
| Summer Dress           |            200 | Women         | GOOD         |
| Learn SQL in 30 Days   |            250 | Non-Fiction   | GOOD         |
| Wireless Earbuds       |            300 | Electronics   | GOOD         |
| Mystery at Midnight    |            300 | Fiction       | GOOD         |
| Slim Fit Jeans         |            400 | Men           | GOOD         |
| The Great Adventure    |            500 | Fiction       | GOOD         |
| Classic Cotton T-Shirt |           1000 | Men           | GOOD         |
+------------------------+----------------+---------------+--------------+
(15 rows)
```

Query 5: Product Search with Reviews

A query that powers the product search page with average ratings:

```sql
SELECT
    p.id,
    p.name,
    p.price,
    p.stock_quantity,
    c.name                                      AS category,
    COALESCE(ROUND(AVG(r.rating), 1), 0)        AS avg_rating,
    COUNT(r.id)                                  AS review_count
FROM products p
JOIN categories c ON c.id = p.category_id
LEFT JOIN reviews r ON r.product_id = p.id
WHERE p.is_active = true
  AND p.name ILIKE '%phone%'
GROUP BY p.id, p.name, p.price, p.stock_quantity, c.name
ORDER BY avg_rating DESC, review_count DESC;
```

```
+----+------------------+--------+----------------+-------------+------------+---------...
| id | name             | price  | stock_quantity | category    | avg_rating | review_c...
+----+------------------+--------+----------------+-------------+------------+---------...
|  3 | SmartPhone X200  | 799.99 |            100 | Smartphones |        4.5 |         ...
|  4 | SmartPhone Lite  | 349.99 |            200 | Smartphones |        4.0 |         ...
+----+------------------+--------+----------------+-------------+------------+---------...
(2 rows)
```

Notice the use of `ILIKE` for case-insensitive search and `COALESCE` to show 0 instead of NULL for products with no reviews.

Step 6: Views

Views simplify complex queries and provide a clean interface for your application.

Product Catalog View

This view powers the main product listing page:

```sql
CREATE VIEW product_catalog AS
SELECT
    p.id                                        AS product_id,
    p.name                                      AS product_name,
    p.description,
    p.price,
    p.stock_quantity,
```

```
    p.image_url,
    c.name                                      AS category_name,
    parent_cat.name                             AS parent_category,
    CASE
        WHEN p.stock_quantity > 0 THEN true
        ELSE false
    END                                         AS in_stock,
    COALESCE(ROUND(AVG(r.rating), 1), 0)        AS avg_rating,
    COUNT(r.id)                                 AS review_count
FROM products p
JOIN categories c ON c.id = p.category_id
LEFT JOIN categories parent_cat ON parent_cat.id = c.parent_id
LEFT JOIN reviews r ON r.product_id = p.id
WHERE p.is_active = true
GROUP BY p.id, p.name, p.description, p.price, p.stock_quantity,
         p.image_url, c.name, parent_cat.name;
```

Now the application can simply query:

```sql
SELECT * FROM product_catalog
WHERE category_name = 'Laptops'
ORDER BY avg_rating DESC;
```

```
+------------+---------------------+-------------------------------+--------+----------...
| product_id | product_name        | description                   | price  | stock_qty...
+------------+---------------------+-------------------------------+--------+----------...
|          1 | ProBook Laptop 15   | High-performance 15-inch...   | 999.99 |       50 ...
|          2 | UltraSlim Notebook  | Lightweight 13-inch...        | 699.99 |       75 ...
+------------+---------------------+-------------------------------+--------+----------...
(2 rows)
```

Order Summary View

This view provides a complete order overview:

```sql
CREATE VIEW order_summary AS
SELECT
    o.id                            AS order_id,
    o.created_at                    AS order_date,
    o.status                        AS order_status,
    u.id                            AS customer_id,
    u.first_name || ' ' || u.last_name AS customer_name,
    u.email                         AS customer_email,
    COUNT(oi.id)                    AS item_count,
    SUM(oi.quantity)                AS total_items,
    o.total_amount,
```

```
    p.payment_method,
    p.status                        AS payment_status,
    a.city || ', ' || a.state       AS shipping_location
FROM orders o
JOIN users u ON u.id = o.user_id
JOIN order_items oi ON oi.order_id = o.id
LEFT JOIN payments p ON p.order_id = o.id
LEFT JOIN addresses a ON a.id = o.address_id
GROUP BY o.id, o.created_at, o.status, u.id, u.first_name, u.last_name,
         u.email, o.total_amount, p.payment_method, p.status, a.city, a.state;
```

```sql
SELECT order_id, order_date::DATE, order_status, customer_name,
       item_count, total_amount, payment_status
FROM order_summary
ORDER BY order_date DESC
LIMIT 5;
```

```
+----------+------------+--------------+-----------------+------------+--------------+-...
| order_id | order_date | order_status | customer_name   | item_count | total_amount | ...
+----------+------------+--------------+-----------------+------------+--------------+-...
|       14 | 2025-03-25 | pending      | Carol Williams  |          1 |       129.99 | ...
|       13 | 2025-03-20 | delivered    | Alice Johnson   |          3 |        79.98 | ...
|        9 | 2025-03-15 | pending      | Eve Davis       |          1 |       799.99 | ...
|        7 | 2025-03-10 | confirmed    | David Brown     |          2 |      1029.98 | ...
|       12 | 2025-03-05 | cancelled    | Henry Anderson  |          1 |       699.99 | ...
+----------+------------+--------------+-----------------+------------+--------------+-...
(5 rows)
```

Customer Dashboard View

Everything a customer needs to see on their dashboard:

```sql
CREATE VIEW customer_dashboard AS
SELECT
    u.id                                AS user_id,
    u.first_name,
    u.last_name,
    u.email,
    u.created_at                        AS member_since,
    COUNT(DISTINCT o.id)                AS total_orders,
    COALESCE(SUM(
        CASE WHEN o.status != 'cancelled' THEN o.total_amount ELSE 0 END
    ), 0)                               AS total_spent,
    COALESCE(AVG(
        CASE WHEN o.status != 'cancelled' THEN o.total_amount END
    ), 0)                               AS avg_order_value,
```

```
    MAX(o.created_at)                   AS last_order_date,
    COUNT(DISTINCT rv.id)               AS reviews_written
FROM users u
LEFT JOIN orders o ON o.user_id = u.id
LEFT JOIN reviews rv ON rv.user_id = u.id
GROUP BY u.id, u.first_name, u.last_name, u.email, u.created_at;
```

```sql
SELECT first_name, last_name, total_orders,
       ROUND(total_spent, 2) AS total_spent,
       ROUND(avg_order_value, 2) AS avg_order,
       reviews_written
FROM customer_dashboard
ORDER BY total_spent DESC;
```

```
+------------+-----------+--------------+-------------+-----------+-----------------+
| first_name | last_name | total_orders | total_spent | avg_order | reviews_written |
+------------+-----------+--------------+-------------+-----------+-----------------+
| Alice      | Johnson   |            3 |     1159.95 |    386.65 |               3 |
| Carol      | Williams  |            3 |      304.95 |    101.65 |               3 |
| David      | Brown     |            1 |     1029.98 |   1029.98 |               0 |
| Bob        | Smith     |            2 |     1199.97 |    599.99 |               2 |
| Eve        | Davis     |            2 |      864.97 |    432.49 |               3 |
| Frank      | Miller    |            1 |      159.98 |    159.98 |               1 |
| Grace      | Taylor    |            1 |       39.99 |     39.99 |               1 |
| Henry      | Anderson  |            1 |        0.00 |      0.00 |               0 |
+------------+-----------+--------------+-------------+-----------+-----------------+
(8 rows)
```

Step 7: Indexes for Performance

Without indexes, the database scans every row in a table to find matches. As your data grows, this becomes slow. Indexes are like the table of contents in a book -- they help the database jump directly to the right page.

Primary Key and Unique Indexes

PostgreSQL automatically creates indexes for primary keys and unique constraints. We already have those. Let us add indexes for the columns we frequently search and join on.

```sql
-- Users: search by email (login), username
CREATE INDEX idx_users_email ON users (email);
-- Already covered by UNIQUE constraint, but explicit for clarity

-- Products: filter by category, search by name, filter active products
CREATE INDEX idx_products_category ON products (category_id);
CREATE INDEX idx_products_name ON products USING gin (to_tsvector('english', name));
CREATE INDEX idx_products_active ON products (is_active) WHERE is_active = true;

-- Orders: find orders by user, filter by status, sort by date
CREATE INDEX idx_orders_user ON orders (user_id);
CREATE INDEX idx_orders_status ON orders (status);
CREATE INDEX idx_orders_created ON orders (created_at DESC);

-- Order Items: lookup by order and product
CREATE INDEX idx_order_items_order ON order_items (order_id);
CREATE INDEX idx_order_items_product ON order_items (product_id);

-- Reviews: find reviews for a product, reviews by a user
CREATE INDEX idx_reviews_product ON reviews (product_id);
CREATE INDEX idx_reviews_user ON reviews (user_id);

-- Payments: lookup by order
CREATE INDEX idx_payments_order ON payments (order_id);

-- Addresses: find addresses for a user
CREATE INDEX idx_addresses_user ON addresses (user_id);
```

Let us explain a few interesting ones:

- `USING gin (to_tsvector('english', name))` -- This is a full-text search index. It allows fast searches like "find products with 'laptop' in the name" using PostgreSQL's text search features.
- `WHERE is_active = true` -- This is a partial index. It only indexes active products, making it smaller and faster because we rarely query inactive products.
- `(created_at DESC)` -- The index is sorted in descending order, which matches our most common query pattern (newest orders first).

Verifying Indexes Are Used

```sql
EXPLAIN ANALYZE
SELECT * FROM orders
WHERE user_id = 1
ORDER BY created_at DESC;
```

```
                                                    QUERY PLAN
-----------------------------------------------------------------------------------------...
```

```
Sort  (cost=8.18..8.19 rows=3 width=52) (actual time=0.045..0.046 rows=3 loops=1)
  Sort Key: created_at DESC
  Sort Method: quicksort  Memory: 25kB
  ->  Index Scan using idx_orders_user on orders  (cost=0.14..8.16 rows=3 width=52)
        (actual time=0.020..0.025 rows=3 loops=1)
        Index Cond: (user_id = 1)
Planning Time: 0.112 ms
Execution Time: 0.072 ms
```

The output shows "Index Scan using idx_orders_user" which means PostgreSQL used our index instead of scanning every row.

Step 8: Triggers

Triggers automatically run code when certain events happen. They enforce business rules that cannot be expressed with simple constraints.

Trigger 1: Auto-Update updated_at Timestamp

Every time a row is modified, we want the `updated_at` column to reflect the current time:

```sql
-- Step 1: Create the trigger function
CREATE OR REPLACE FUNCTION update_updated_at()
RETURNS TRIGGER AS $$
BEGIN
    NEW.updated_at = CURRENT_TIMESTAMP;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

Line-by-line explanation:

- `CREATE OR REPLACE FUNCTION` -- Creates the function or replaces it if it exists.
- `RETURNS TRIGGER` -- This function is designed to be called by a trigger.
- `NEW.updated_at = CURRENT_TIMESTAMP` -- `NEW` refers to the row being inserted or updated. We set its `updated_at` to the current time.
- `RETURN NEW` -- Returns the modified row. If we returned NULL, the operation would be cancelled.

```sql
-- Step 2: Attach the trigger to tables
CREATE TRIGGER trigger_users_updated_at
    BEFORE UPDATE ON users
    FOR EACH ROW
    EXECUTE FUNCTION update_updated_at();

CREATE TRIGGER trigger_products_updated_at
    BEFORE UPDATE ON products
    FOR EACH ROW
    EXECUTE FUNCTION update_updated_at();

CREATE TRIGGER trigger_orders_updated_at
    BEFORE UPDATE ON orders
    FOR EACH ROW
    EXECUTE FUNCTION update_updated_at();
```

Now let us test it:

```sql
-- Check current timestamp
SELECT id, name, updated_at FROM products WHERE id = 1;
```

```
+----+-------------------+---------------------+
| id | name              | updated_at          |
+----+-------------------+---------------------+
|  1 | ProBook Laptop 15 | 2025-01-01 00:00:00 |
+----+-------------------+---------------------+
```

```sql
-- Update the product
UPDATE products SET price = 949.99 WHERE id = 1;

-- Check again
SELECT id, name, price, updated_at FROM products WHERE id = 1;
```

```
+----+-------------------+--------+---------------------+
| id | name              | price  | updated_at          |
+----+-------------------+--------+---------------------+
|  1 | ProBook Laptop 15 | 949.99 | 2025-03-29 14:35:22 |
+----+-------------------+--------+---------------------+
```

The `updated_at` column was automatically set to the current timestamp without us explicitly updating it.

Trigger 2: Reduce Stock When Order Is Placed

When an order item is inserted, reduce the product's stock quantity:

```sql
CREATE OR REPLACE FUNCTION reduce_stock_on_order()
RETURNS TRIGGER AS $$
BEGIN
    -- Check if enough stock is available
    IF (SELECT stock_quantity FROM products WHERE id = NEW.product_id) < NEW.quantity THEN
        RAISE EXCEPTION 'Insufficient stock for product ID %. Requested: %, Available: %',
            NEW.product_id,
            NEW.quantity,
            (SELECT stock_quantity FROM products WHERE id = NEW.product_id);
    END IF;

    -- Reduce the stock
    UPDATE products
    SET stock_quantity = stock_quantity - NEW.quantity
    WHERE id = NEW.product_id;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trigger_reduce_stock
    AFTER INSERT ON order_items
    FOR EACH ROW
    EXECUTE FUNCTION reduce_stock_on_order();
```

Let us break down the logic:

1. First, we check if the product has enough stock. If not, we raise an exception, which cancels the INSERT and rolls back the transaction.
2. If stock is available, we subtract the ordered quantity from the product's `stock_quantity`.
3. We use `AFTER INSERT` because we want the order item to be saved first, then reduce stock.

Let us test it:

```sql
-- Check current stock
SELECT id, name, stock_quantity FROM products WHERE id = 1;
```

```
+----+-------------------+----------------+
| id | name              | stock_quantity |
+----+-------------------+----------------+
|  1 | ProBook Laptop 15 |             50 |
+----+-------------------+----------------+
```

```sql
-- Insert a new order item (this triggers stock reduction)
INSERT INTO orders (user_id, address_id, status, total_amount)
```

```
VALUES (1, 1, 'pending', 949.99);

INSERT INTO order_items (order_id, product_id, quantity, unit_price, subtotal)
VALUES (15, 1, 2, 949.99, 1899.98);

-- Check stock again
SELECT id, name, stock_quantity FROM products WHERE id = 1;
```

```
+----+-------------------+----------------+
| id | name              | stock_quantity |
+----+-------------------+----------------+
|  1 | ProBook Laptop 15 |             48 |
+----+-------------------+----------------+
```

The stock dropped from 50 to 48 automatically.

Trigger 3: Restore Stock When Order Is Cancelled

```sql
CREATE OR REPLACE FUNCTION restore_stock_on_cancel()
RETURNS TRIGGER AS $$
BEGIN
    -- Only restore if status changes to 'cancelled'
    IF NEW.status = 'cancelled' AND OLD.status != 'cancelled' THEN
        UPDATE products p
        SET stock_quantity = p.stock_quantity + oi.quantity
        FROM order_items oi
        WHERE oi.order_id = NEW.id
          AND p.id = oi.product_id;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trigger_restore_stock_on_cancel
    AFTER UPDATE ON orders
    FOR EACH ROW
    EXECUTE FUNCTION restore_stock_on_cancel();
```

This trigger fires when an order is updated. It checks if the status changed to 'cancelled' (and was not already cancelled), then adds the quantities back to the products.

Step 9: Functions

Functions encapsulate complex logic that you use repeatedly. They keep your application code clean and ensure consistency.

Function 1: Calculate Order Total

This function recalculates the total for an order based on its items:

```sql
CREATE OR REPLACE FUNCTION calculate_order_total(p_order_id INTEGER)
RETURNS NUMERIC AS $$
DECLARE
    v_total NUMERIC(12, 2);
BEGIN
    -- Sum all order item subtotals
    SELECT COALESCE(SUM(subtotal), 0)
    INTO v_total
    FROM order_items
    WHERE order_id = p_order_id;

    -- Update the order's total_amount
    UPDATE orders
    SET total_amount = v_total
    WHERE id = p_order_id;

    RETURN v_total;
END;
$$ LANGUAGE plpgsql;
```

Line-by-line explanation:

- `p_order_id INTEGER` -- The function takes an order ID as input.
- `RETURNS NUMERIC` -- It returns the calculated total.
- `DECLARE v_total NUMERIC(12, 2)` -- We declare a variable to hold the total.
- `SELECT ... INTO v_total` -- The query result is stored in our variable.
- `COALESCE(SUM(subtotal), 0)` -- If there are no items, return 0 instead of NULL.
- `UPDATE orders SET total_amount = v_total` -- Also update the order record.
- `RETURN v_total` -- Return the calculated total to the caller.

Using it:

```sql
SELECT calculate_order_total(1);
```

```
+--------------------------+
| calculate_order_total    |
+--------------------------+
|                  1049.98 |
```

```
+-------------------------+
(1 row)
```

Function 2: Apply Discount

This function applies a percentage discount to an order:

```sql
CREATE OR REPLACE FUNCTION apply_discount(
    p_order_id      INTEGER,
    p_discount_pct  NUMERIC
)
RETURNS TABLE (
    original_total  NUMERIC,
    discount_amount NUMERIC,
    final_total     NUMERIC
) AS $$
DECLARE
    v_original  NUMERIC(12, 2);
    v_discount  NUMERIC(12, 2);
    v_final     NUMERIC(12, 2);
BEGIN
    -- Validate discount percentage
    IF p_discount_pct < 0 OR p_discount_pct > 100 THEN
        RAISE EXCEPTION 'Discount must be between 0 and 100. Got: %', p_discount_pct;
    END IF;

    -- Get current total
    SELECT total_amount INTO v_original
    FROM orders
    WHERE id = p_order_id;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'Order % not found', p_order_id;
    END IF;

    -- Calculate discount
    v_discount := ROUND(v_original * p_discount_pct / 100, 2);
    v_final := v_original - v_discount;

    -- Update the order
    UPDATE orders
    SET total_amount = v_final
    WHERE id = p_order_id;

    -- Return the details
    RETURN QUERY SELECT v_original, v_discount, v_final;
END;
$$ LANGUAGE plpgsql;
```

This function:

1. Validates that the discount percentage is between 0 and 100.
2. Looks up the current order total.
3. Raises an error if the order does not exist.
4. Calculates the discount amount and new total.
5. Updates the order record.
6. Returns a table with the original total, discount amount, and final total.

Using it:

```sql
SELECT * FROM apply_discount(5, 10);
```

```
+----------------+-----------------+-------------+
| original_total | discount_amount | final_total |
+----------------+-----------------+-------------+
|         114.97 |           11.50 |      103.47 |
+----------------+-----------------+-------------+
(1 row)
```

Function 3: Get Product Recommendations

Find products frequently bought together:

```sql
CREATE OR REPLACE FUNCTION get_recommendations(p_product_id INTEGER, p_limit INTEGER DE...
RETURNS TABLE (
    recommended_product_id   INTEGER,
    product_name             VARCHAR,
    category                 VARCHAR,
    price                    NUMERIC,
    times_bought_together    BIGINT
) AS $$
BEGIN
    RETURN QUERY
    SELECT
        p.id,
        p.name,
        c.name,
        p.price,
        COUNT(*) AS times_together
    FROM order_items oi1
    JOIN order_items oi2 ON oi2.order_id = oi1.order_id
                        AND oi2.product_id != oi1.product_id
    JOIN products p ON p.id = oi2.product_id
    JOIN categories c ON c.id = p.category_id
    WHERE oi1.product_id = p_product_id
      AND p.is_active = true
```

```
    GROUP BY p.id, p.name, c.name, p.price
    ORDER BY times_together DESC
    LIMIT p_limit;
END;
$$ LANGUAGE plpgsql;
```

Using it:

```sql
SELECT * FROM get_recommendations(1);
```

```
+------------------------+-----------------------+-----------+--------+----------------...
| recommended_product_id | product_name          | category  | price  | times_bought_to...
+------------------------+-----------------------+-----------+--------+----------------...
|                      9 | Classic Cotton T-Shirt | Men      |  19.99 |                ...
|                      7 | Learn SQL in 30 Days  | Non-Fic. |  29.99 |                ...
+------------------------+-----------------------+-----------+--------+----------------...
(2 rows)
```

This query finds products that appear in the same orders as the given product. This is the "Customers who bought this also bought..." feature you see on e-commerce sites.

Complete Code Reference

Here is the complete setup script for the entire e-commerce database. You can run this from start to finish to create everything:

```sql
-- ============================================================
-- E-COMMERCE DATABASE - COMPLETE SETUP SCRIPT
-- ============================================================

-- 1. CREATE TABLES
-- (All CREATE TABLE statements from Step 3 above)

-- 2. INSERT SEED DATA
-- (All INSERT statements from Step 4 above)

-- 3. CREATE VIEWS
-- (All CREATE VIEW statements from Step 6 above)

-- 4. CREATE INDEXES
-- (All CREATE INDEX statements from Step 7 above)
```

```
-- 5. CREATE TRIGGER FUNCTIONS AND TRIGGERS
-- (All trigger code from Step 8 above)

-- 6. CREATE FUNCTIONS
-- (All function code from Step 9 above)

-- 7. VERIFY EVERYTHING WORKS
SELECT 'Users:      ' || COUNT(*) FROM users
UNION ALL
SELECT 'Categories: ' || COUNT(*) FROM categories
UNION ALL
SELECT 'Products:   ' || COUNT(*) FROM products
UNION ALL
SELECT 'Orders:     ' || COUNT(*) FROM orders
UNION ALL
SELECT 'Order Items:' || COUNT(*) FROM order_items
UNION ALL
SELECT 'Reviews:    ' || COUNT(*) FROM reviews
UNION ALL
SELECT 'Payments:   ' || COUNT(*) FROM payments
UNION ALL
SELECT 'Addresses:  ' || COUNT(*) FROM addresses;
```

```
+-----------------------+
| ?column?              |
+-----------------------+
| Users:      8         |
| Categories: 10        |
| Products:   15        |
| Orders:     14        |
| Order Items:23        |
| Reviews:    15        |
| Payments:   14        |
| Addresses:  9         |
+-----------------------+
(8 rows)
```

Common Mistakes

1. Storing current product price in order items. Always store the price at the time of purchase. Prices change, but order history should not.
2. Not using CHECK constraints for enums. Without `CHECK (status IN (...))`, someone could insert an order with status 'banana'. Use constraints to enforce valid values.

3. Forgetting ON DELETE CASCADE or ON DELETE SET NULL. Without these, deleting a user would fail if they have orders. Plan your delete behavior during design.
4. Using FLOAT for money. Always use `NUMERIC(precision, scale)` for financial data. FLOAT introduces rounding errors (try `0.1 + 0.2` in floating point).
5. Not indexing foreign key columns. Joins are the most common queries. Without indexes on foreign key columns, joins become slow as data grows.
6. Creating too many indexes. Each index speeds up reads but slows down writes. Only index columns you actually query on.

Best Practices

1. Design before coding. Create the ER diagram and document business rules before writing any SQL.
2. Use NUMERIC for money. Never use FLOAT or REAL for financial values. NUMERIC provides exact decimal arithmetic.
3. Add constraints at the database level. Do not rely only on application code to validate data. Use CHECK, NOT NULL, UNIQUE, and FOREIGN KEY constraints.
4. Store prices on order items. Always capture the price at the time of purchase, not a reference to the product table.
5. Use views for complex queries. If your application frequently runs the same complex query, create a view. It simplifies code and centralizes logic.
6. Index foreign keys and frequently filtered columns. Focus on columns used in WHERE clauses, JOIN conditions, and ORDER BY.
7. Use triggers for automatic updates. The `updated_at` trigger pattern is standard practice. It ensures timestamps are always accurate.
8. Test with realistic data. Seed your database with enough data to test performance and edge cases.

Quick Summary

You built a complete e-commerce database from scratch:

- Requirements defined 8 entities and their business rules.
- ER diagram mapped the relationships between entities.

- Schema created 8 normalized tables with proper constraints.
- Seed data populated the database with realistic sample data.
- Queries answered real business questions (top products, revenue, inventory).
- Views simplified complex queries for the application layer.
- Indexes optimized performance for common query patterns.
- Triggers automated stock management and timestamp updates.
- Functions encapsulated reusable business logic.

Key Points

- Always start a database project by gathering requirements and documenting business rules.
- Draw an ER diagram before creating tables. It catches design problems early.
- Use proper data types and constraints to enforce data integrity at the database level.
- Store historical values (like prices) on transactional records, not just references.
- Views abstract complex queries and provide a clean API for applications.
- Index columns that appear in WHERE, JOIN, and ORDER BY clauses.
- Use triggers to enforce business rules that constraints cannot express.
- Functions encapsulate reusable logic and keep application code clean.

Practice Questions

1. A customer wants to see all products they have ever purchased with quantities and prices. Write the query.
2. The business team asks: "Which category generates the most revenue?" Write a query that shows revenue by category, sorted from highest to lowest.
3. Why do we store `unit_price` on the `order_items` table instead of just joining to the `products` table for the price?
4. What would happen if we used `ON DELETE CASCADE` on the `orders.user_id` foreign key? Is that a good idea? Why or why not?
5. How would you modify the schema to support products that belong to multiple categories? What changes would be needed?

Exercises

Exercise 1: Add a Wishlist Feature

Design and implement a wishlist feature:

1. Create a `wishlists` table where users can save products they want to buy later.
2. Each user can have only one entry per product in their wishlist.
3. Write a query to show a user's wishlist with product details, current price, and stock status.
4. Write a query to find the most-wishlisted products.

Exercise 2: Add a Coupon System

Extend the database with a coupon system:

1. Create a `coupons` table with fields for code, discount type (percentage or fixed amount), discount value, minimum order amount, expiry date, and usage limit.
2. Create an `order_coupons` junction table to track which coupons were applied to which orders.
3. Write a function that validates and applies a coupon to an order.

Exercise 3: Build a Reporting Dashboard

Write these dashboard queries:

1. Daily revenue for the last 30 days (including days with zero revenue).
2. Customer lifetime value ranking (total spent by each customer, with rank).
3. Product performance report: for each product, show total sold, revenue, average rating, return rate (cancelled orders / total orders), and a performance score.

What Is Next?

Congratulations! You have just built a production-quality database. In the next chapter, you will use similar data to write advanced analytics queries: revenue trends, year-over-year growth, customer cohort analysis, RFM segmentation, and more. These are the queries that

turn raw data into business insights.

Chapter 33: Project -- Analytics Queries for Business Intelligence

What You Will Learn

In this chapter, you will write the kind of SQL queries that data analysts and business intelligence teams use every day. You will calculate daily, weekly, and monthly revenue, measure year-over-year growth, compute running totals and moving averages, perform customer cohort analysis, segment customers using RFM analysis, find top products per category, and build a conversion funnel. By the end, you will have a complete set of dashboard queries ready for any reporting tool.

Why This Chapter Matters

Raw data sitting in a database is like crude oil. It has tremendous value, but only after it is refined. Analytics queries are the refinery. They transform rows of transactions into answers: "Is revenue growing?" "Which customers are we losing?" "Which products drive the most profit?"

Every modern business runs on data-driven decisions. The person who can write these queries is invaluable. Whether you become a developer, analyst, or data engineer, these skills will serve you throughout your career.

Setting Up the Dataset

For this chapter, we will work with a sales dataset. If you completed the e-commerce project in Chapter 32, you can use that database. Otherwise, here is a focused dataset for analytics

work.

Create the Tables

```sql
CREATE TABLE customers (
    id              SERIAL PRIMARY KEY,
    name            VARCHAR(100) NOT NULL,
    email           VARCHAR(255) UNIQUE NOT NULL,
    region          VARCHAR(50) NOT NULL,
    signup_date     DATE NOT NULL,
    created_at      TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE products (
    id              SERIAL PRIMARY KEY,
    name            VARCHAR(200) NOT NULL,
    category        VARCHAR(100) NOT NULL,
    price           NUMERIC(10, 2) NOT NULL,
    cost            NUMERIC(10, 2) NOT NULL
);

CREATE TABLE orders (
    id              SERIAL PRIMARY KEY,
    customer_id     INTEGER REFERENCES customers(id),
    order_date      DATE NOT NULL,
    total_amount    NUMERIC(12, 2) NOT NULL,
    status          VARCHAR(20) NOT NULL DEFAULT 'completed'
);

CREATE TABLE order_items (
    id              SERIAL PRIMARY KEY,
    order_id        INTEGER REFERENCES orders(id),
    product_id      INTEGER REFERENCES products(id),
    quantity        INTEGER NOT NULL,
    unit_price      NUMERIC(10, 2) NOT NULL,
    subtotal        NUMERIC(12, 2) NOT NULL
);

-- Funnel tracking table
CREATE TABLE user_events (
    id              SERIAL PRIMARY KEY,
    customer_id     INTEGER REFERENCES customers(id),
    event_type      VARCHAR(50) NOT NULL,
    event_date      TIMESTAMP NOT NULL,
    metadata        JSONB
);
```

Insert Sample Data

```sql
-- Customers across different regions
INSERT INTO customers (name, email, region, signup_date) VALUES
    ('Alice Johnson',   'alice@example.com',    'West',     '2023-01-15'),
    ('Bob Smith',       'bob@example.com',      'East',     '2023-02-20'),
    ('Carol Williams',  'carol@example.com',    'West',     '2023-03-10'),
    ('David Brown',     'david@example.com',    'Central',  '2023-04-05'),
    ('Eve Davis',       'eve@example.com',      'East',     '2023-05-12'),
    ('Frank Miller',    'frank@example.com',    'West',     '2023-06-18'),
    ('Grace Taylor',    'grace@example.com',    'Central',  '2023-07-22'),
    ('Henry Anderson',  'henry@example.com',    'East',     '2023-08-30'),
    ('Ivy Chen',        'ivy@example.com',      'West',     '2023-09-15'),
    ('Jack Wilson',     'jack@example.com',     'Central',  '2023-10-01'),
    ('Karen Lee',       'karen@example.com',    'East',     '2024-01-10'),
    ('Leo Martinez',    'leo@example.com',      'West',     '2024-02-14'),
    ('Mia Robinson',    'mia@example.com',      'Central',  '2024-03-20'),
    ('Noah Clark',      'noah@example.com',     'East',     '2024-04-15'),
    ('Olivia Hall',     'olivia@example.com',   'West',     '2024-05-25');

-- Products with cost data for profit analysis
INSERT INTO products (name, category, price, cost) VALUES
    ('ProBook Laptop',       'Electronics',  999.99,  650.00),
    ('SmartPhone X',         'Electronics',  799.99,  480.00),
    ('Wireless Earbuds',     'Electronics',  129.99,   55.00),
    ('Running Shoes',        'Sports',        89.99,   35.00),
    ('Yoga Mat',             'Sports',        29.99,   10.00),
    ('Coffee Maker',         'Home',         149.99,   70.00),
    ('Blender Pro',          'Home',          79.99,   30.00),
    ('SQL Mastery Book',     'Books',         39.99,   12.00),
    ('Data Science Guide',   'Books',         49.99,   15.00),
    ('Winter Jacket',        'Clothing',     199.99,   85.00);

-- Orders spanning 2023 and 2024
INSERT INTO orders (customer_id, order_date, total_amount, status) VALUES
    -- 2023 Q1
    (1,  '2023-01-20',  999.99, 'completed'),
    (1,  '2023-02-15',  129.99, 'completed'),
    (2,  '2023-03-01',  799.99, 'completed'),
    (3,  '2023-03-15',   89.99, 'completed'),
    -- 2023 Q2
    (1,  '2023-04-10',   39.99, 'completed'),
    (2,  '2023-04-22',  149.99, 'completed'),
    (4,  '2023-05-08',  129.99, 'completed'),
    (3,  '2023-05-20',   79.99, 'completed'),
    (5,  '2023-06-05',  999.99, 'completed'),
    (5,  '2023-06-25',   29.99, 'completed'),
    -- 2023 Q3
    (1,  '2023-07-12',  199.99, 'completed'),
    (6,  '2023-07-18',  799.99, 'completed'),
    (2,  '2023-08-05',   89.99, 'completed'),
    (7,  '2023-08-14',  129.99, 'completed'),
    (3,  '2023-09-01',   49.99, 'completed'),
    (8,  '2023-09-20',  149.99, 'completed'),
    -- 2023 Q4
    (1,  '2023-10-15',  129.99, 'completed'),
```

```
    (4,  '2023-10-28',  799.99, 'completed'),
    (9,  '2023-11-05',   39.99, 'completed'),
    (5,  '2023-11-18',  199.99, 'completed'),
    (2,  '2023-12-01',  149.99, 'completed'),
    (10, '2023-12-15',  999.99, 'completed'),
    (6,  '2023-12-22',   49.99, 'completed'),
    -- 2024 Q1
    (1,  '2024-01-08',   89.99, 'completed'),
    (11, '2024-01-15',  799.99, 'completed'),
    (3,  '2024-01-22',  129.99, 'completed'),
    (2,  '2024-02-10',  999.99, 'completed'),
    (4,  '2024-02-18',   79.99, 'completed'),
    (12, '2024-02-25',  149.99, 'completed'),
    (5,  '2024-03-05',   39.99, 'completed'),
    (7,  '2024-03-12',  199.99, 'completed'),
    (8,  '2024-03-28',   49.99, 'completed'),
    -- 2024 Q2
    (1,  '2024-04-02',  799.99, 'completed'),
    (13, '2024-04-10',  129.99, 'completed'),
    (6,  '2024-04-20',   89.99, 'completed'),
    (3,  '2024-05-08',  999.99, 'completed'),
    (14, '2024-05-15',   29.99, 'completed'),
    (9,  '2024-05-22',  149.99, 'completed'),
    (2,  '2024-06-01',  129.99, 'completed'),
    (10, '2024-06-12',  199.99, 'completed'),
    (4,  '2024-06-25',   49.99, 'completed');

-- Order items (simplified - one item per order for clarity)
INSERT INTO order_items (order_id, product_id, quantity, unit_price, subtotal) VALUES
    (1,  1, 1, 999.99, 999.99), (2,  3, 1, 129.99, 129.99),
    (3,  2, 1, 799.99, 799.99), (4,  4, 1,  89.99,  89.99),
    (5,  8, 1,  39.99,  39.99), (6,  6, 1, 149.99, 149.99),
    (7,  3, 1, 129.99, 129.99), (8,  7, 1,  79.99,  79.99),
    (9,  1, 1, 999.99, 999.99), (10, 5, 1,  29.99,  29.99),
    (11,10, 1, 199.99, 199.99), (12, 2, 1, 799.99, 799.99),
    (13, 4, 1,  89.99,  89.99), (14, 3, 1, 129.99, 129.99),
    (15, 9, 1,  49.99,  49.99), (16, 6, 1, 149.99, 149.99),
    (17, 3, 1, 129.99, 129.99), (18, 2, 1, 799.99, 799.99),
    (19, 8, 1,  39.99,  39.99), (20,10, 1, 199.99, 199.99),
    (21, 6, 1, 149.99, 149.99), (22, 1, 1, 999.99, 999.99),
    (23, 9, 1,  49.99,  49.99), (24, 4, 1,  89.99,  89.99),
    (25, 2, 1, 799.99, 799.99), (26, 3, 1, 129.99, 129.99),
    (27, 1, 1, 999.99, 999.99), (28, 7, 1,  79.99,  79.99),
    (29, 6, 1, 149.99, 149.99), (30, 8, 1,  39.99,  39.99),
    (31,10, 1, 199.99, 199.99), (32, 9, 1,  49.99,  49.99),
    (33, 2, 1, 799.99, 799.99), (34, 3, 1, 129.99, 129.99),
    (35, 4, 1,  89.99,  89.99), (36, 1, 1, 999.99, 999.99),
    (37, 5, 1,  29.99,  29.99), (38, 6, 1, 149.99, 149.99),
    (39, 3, 1, 129.99, 129.99), (40,10, 1, 199.99, 199.99),
    (41, 9, 1,  49.99,  49.99);

-- Funnel events
INSERT INTO user_events (customer_id, event_type, event_date) VALUES
    -- Full funnel users
    (1,  'signup',    '2023-01-15 10:00:00'),
```

```
    (1,  'browse',    '2023-01-15 10:05:00'),
    (1,  'add_to_cart','2023-01-18 14:30:00'),
    (1,  'purchase',  '2023-01-20 09:00:00'),
    (2,  'signup',    '2023-02-20 11:00:00'),
    (2,  'browse',    '2023-02-20 11:10:00'),
    (2,  'add_to_cart','2023-02-25 16:00:00'),
    (2,  'purchase',  '2023-03-01 10:00:00'),
    (3,  'signup',    '2023-03-10 09:00:00'),
    (3,  'browse',    '2023-03-10 09:15:00'),
    (3,  'add_to_cart','2023-03-12 13:00:00'),
    (3,  'purchase',  '2023-03-15 11:00:00'),
    -- Dropped at cart
    (4,  'signup',    '2023-04-05 14:00:00'),
    (4,  'browse',    '2023-04-05 14:20:00'),
    (4,  'add_to_cart','2023-04-06 10:00:00'),
    (4,  'purchase',  '2023-05-08 15:00:00'),
    -- Browse but no cart
    (5,  'signup',    '2023-05-12 08:00:00'),
    (5,  'browse',    '2023-05-12 08:30:00'),
    (5,  'add_to_cart','2023-05-15 12:00:00'),
    (5,  'purchase',  '2023-06-05 14:00:00'),
    -- Signup only
    (6,  'signup',    '2023-06-18 16:00:00'),
    (6,  'browse',    '2023-06-20 10:00:00'),
    (6,  'add_to_cart','2023-07-10 09:00:00'),
    (6,  'purchase',  '2023-07-18 11:00:00'),
    (7,  'signup',    '2023-07-22 13:00:00'),
    (7,  'browse',    '2023-07-22 13:30:00'),
    (7,  'add_to_cart','2023-08-01 15:00:00'),
    (7,  'purchase',  '2023-08-14 09:00:00'),
    (8,  'signup',    '2023-08-30 10:00:00'),
    (8,  'browse',    '2023-08-30 10:15:00'),
    (8,  'add_to_cart','2023-09-10 14:00:00'),
    (8,  'purchase',  '2023-09-20 16:00:00'),
    -- Additional signups with partial funnels
    (9,  'signup',    '2023-09-15 11:00:00'),
    (9,  'browse',    '2023-09-16 09:00:00'),
    (9,  'add_to_cart','2023-10-20 14:00:00'),
    (9,  'purchase',  '2023-11-05 10:00:00'),
    (10, 'signup',    '2023-10-01 15:00:00'),
    (10, 'browse',    '2023-10-02 10:00:00'),
    (10, 'add_to_cart','2023-11-28 09:00:00'),
    (10, 'purchase',  '2023-12-15 14:00:00'),
    (11, 'signup',    '2024-01-10 09:00:00'),
    (11, 'browse',    '2024-01-10 09:20:00'),
    (11, 'add_to_cart','2024-01-12 11:00:00'),
    (11, 'purchase',  '2024-01-15 13:00:00'),
    (12, 'signup',    '2024-02-14 10:00:00'),
    (12, 'browse',    '2024-02-14 10:30:00'),
    (12, 'add_to_cart','2024-02-20 16:00:00'),
    (12, 'purchase',  '2024-02-25 09:00:00'),
    (13, 'signup',    '2024-03-20 14:00:00'),
    (13, 'browse',    '2024-03-21 10:00:00'),
    (13, 'add_to_cart','2024-03-25 15:00:00'),
    (13, 'purchase',  '2024-04-10 11:00:00'),
```

```
    (14, 'signup',    '2024-04-15 08:00:00'),
    (14, 'browse',    '2024-04-15 08:45:00'),
    (14, 'add_to_cart','2024-04-20 13:00:00'),
    (14, 'purchase',  '2024-05-15 10:00:00'),
    (15, 'signup',    '2024-05-25 12:00:00'),
    (15, 'browse',    '2024-05-25 12:30:00'),
    (15, 'add_to_cart','2024-06-01 14:00:00');
    -- Customer 15 has not purchased yet (dropped at add_to_cart)
```

Daily, Weekly, and Monthly Revenue

Daily Revenue

```sql
SELECT
    order_date,
    COUNT(*)                AS order_count,
    SUM(total_amount)       AS daily_revenue
FROM orders
WHERE status = 'completed'
  AND order_date >= '2024-01-01'
GROUP BY order_date
ORDER BY order_date;
```

```
+------------+-------------+---------------+
| order_date | order_count | daily_revenue |
+------------+-------------+---------------+
| 2024-01-08 |           1 |         89.99 |
| 2024-01-15 |           1 |        799.99 |
| 2024-01-22 |           1 |        129.99 |
| 2024-02-10 |           1 |        999.99 |
| 2024-02-18 |           1 |         79.99 |
| 2024-02-25 |           1 |        149.99 |
| 2024-03-05 |           1 |         39.99 |
| 2024-03-12 |           1 |        199.99 |
| 2024-03-28 |           1 |         49.99 |
| ...        |         ... |           ... |
+------------+-------------+---------------+
```

Weekly Revenue with DATE_TRUNC

`DATE_TRUNC` is the key function for time-based aggregations. It rounds a date down to the start of a period.

```sql
SELECT
    DATE_TRUNC('week', order_date)::DATE    AS week_starting,
    COUNT(*)                                 AS order_count,
    SUM(total_amount)                        AS weekly_revenue,
    ROUND(AVG(total_amount), 2)              AS avg_order_value
FROM orders
WHERE status = 'completed'
  AND order_date >= '2024-01-01'
GROUP BY DATE_TRUNC('week', order_date)
ORDER BY week_starting;
```

```
+---------------+-------------+----------------+-----------------+
| week_starting | order_count | weekly_revenue | avg_order_value |
+---------------+-------------+----------------+-----------------+
| 2024-01-08    |           2 |         889.98 |          444.99 |
| 2024-01-22    |           1 |         129.99 |          129.99 |
| 2024-02-05    |           1 |         999.99 |          999.99 |
| 2024-02-12    |           1 |          79.99 |           79.99 |
| 2024-02-19    |           1 |         149.99 |          149.99 |
| 2024-03-04    |           1 |          39.99 |           39.99 |
| 2024-03-11    |           1 |         199.99 |          199.99 |
| 2024-03-25    |           1 |          49.99 |           49.99 |
| ...           |         ... |            ... |             ... |
+---------------+-------------+----------------+-----------------+
```

Let us break down the key parts:

- `DATE_TRUNC('week', order_date)` -- Rounds each date to the Monday of its week. For example, a Wednesday on January 10th becomes Monday January 8th.
- `::DATE` -- Casts the result to a date (removes the time portion).
- We group by the truncated date, so all orders within the same week are combined.

Monthly Revenue

```sql
SELECT
    TO_CHAR(DATE_TRUNC('month', order_date), 'YYYY-MM')  AS month,
    COUNT(*)                                              AS order_count,
    SUM(total_amount)                                     AS monthly_revenue,
    ROUND(AVG(total_amount), 2)                           AS avg_order_value
FROM orders
WHERE status = 'completed'
GROUP BY DATE_TRUNC('month', order_date)
ORDER BY DATE_TRUNC('month', order_date);
```

```
+---------+-------------+----------------+-----------------+
| month   | order_count | monthly_revenue | avg_order_value |
```

```
+---------+-------------+-----------------+-----------------+
| 2023-01 |           1 |          999.99 |          999.99 |
| 2023-02 |           1 |          129.99 |          129.99 |
| 2023-03 |           2 |          889.98 |          444.99 |
| 2023-04 |           2 |          189.98 |           94.99 |
| 2023-05 |           2 |          209.98 |          104.99 |
| 2023-06 |           2 |         1029.98 |          514.99 |
| 2023-07 |           2 |          999.98 |          499.99 |
| 2023-08 |           2 |          219.98 |          109.99 |
| 2023-09 |           2 |          199.98 |           99.99 |
| 2023-10 |           2 |          929.98 |          464.99 |
| 2023-11 |           2 |          239.98 |          119.99 |
| 2023-12 |           3 |         1199.97 |          399.99 |
| 2024-01 |           3 |         1019.97 |          339.99 |
| 2024-02 |           3 |         1229.97 |          409.99 |
| 2024-03 |           3 |          289.97 |           96.66 |
| 2024-04 |           3 |         1019.97 |          339.99 |
| 2024-05 |           3 |         1059.97 |          353.32 |
| 2024-06 |           3 |          379.97 |          126.66 |
+---------+-------------+-----------------+-----------------+
(18 rows)
```

Year-over-Year Growth

Year-over-year (YoY) growth compares each month to the same month in the previous year. This is one of the most important business metrics because it removes seasonal effects.

```sql
WITH monthly_revenue AS (
    SELECT
        DATE_TRUNC('month', order_date)::DATE   AS month,
        SUM(total_amount)                        AS revenue
    FROM orders
    WHERE status = 'completed'
    GROUP BY DATE_TRUNC('month', order_date)
)
SELECT
    TO_CHAR(month, 'YYYY-MM')                   AS month,
    revenue                                      AS current_revenue,
    LAG(revenue, 12) OVER (ORDER BY month)       AS prior_year_revenue,
    CASE
        WHEN LAG(revenue, 12) OVER (ORDER BY month) IS NOT NULL
        THEN ROUND(
            (revenue - LAG(revenue, 12) OVER (ORDER BY month))
            / LAG(revenue, 12) OVER (ORDER BY month) * 100,
            1
        )
```

```
    END                                          AS yoy_growth_pct
FROM monthly_revenue
ORDER BY month;
```

```
+---------+-----------------+--------------------+-----------------+
| month   | current_revenue | prior_year_revenue | yoy_growth_pct  |
+---------+-----------------+--------------------+-----------------+
| 2023-01 |          999.99 |               NULL |            NULL |
| 2023-02 |          129.99 |               NULL |            NULL |
| 2023-03 |          889.98 |               NULL |            NULL |
| 2023-04 |          189.98 |               NULL |            NULL |
| 2023-05 |          209.98 |               NULL |            NULL |
| 2023-06 |         1029.98 |               NULL |            NULL |
| 2023-07 |          999.98 |               NULL |            NULL |
| 2023-08 |          219.98 |               NULL |            NULL |
| 2023-09 |          199.98 |               NULL |            NULL |
| 2023-10 |          929.98 |               NULL |            NULL |
| 2023-11 |          239.98 |               NULL |            NULL |
| 2023-12 |         1199.97 |               NULL |            NULL |
| 2024-01 |         1019.97 |             999.99 |             2.0 |
| 2024-02 |         1229.97 |             129.99 |           846.2 |
| 2024-03 |          289.97 |             889.98 |           -67.4 |
| 2024-04 |         1019.97 |             189.98 |           436.9 |
| 2024-05 |         1059.97 |             209.98 |           404.8 |
| 2024-06 |          379.97 |            1029.98 |           -63.1 |
+---------+-----------------+--------------------+-----------------+
(18 rows)
```

Let us break down the key function:

- `LAG(revenue, 12) OVER (ORDER BY month)` -- The `LAG` window function looks back 12 rows (12 months = 1 year) to find the same month's revenue from last year.
- The growth percentage formula is: `(current - previous) / previous * 100`.
- We wrap it in a `CASE` to handle the first year where there is no prior data (NULL).

This query tells us, for example, that January 2024 revenue grew 2.0% compared to January 2023, while February 2024 saw a massive 846.2% increase over February 2023.

Running Totals and Moving Averages

Running Total (Cumulative Revenue)

A running total shows how revenue accumulates over time. Think of it as a score that only goes up.

```sql
SELECT
    TO_CHAR(DATE_TRUNC('month', order_date), 'YYYY-MM')  AS month,
    SUM(total_amount)                                     AS monthly_revenue,
    SUM(SUM(total_amount)) OVER (
        ORDER BY DATE_TRUNC('month', order_date)
    )                                                     AS cumulative_revenue
FROM orders
WHERE status = 'completed'
GROUP BY DATE_TRUNC('month', order_date)
ORDER BY DATE_TRUNC('month', order_date);
```

```
+---------+-----------------+--------------------+
| month   | monthly_revenue | cumulative_revenue |
+---------+-----------------+--------------------+
| 2023-01 |          999.99 |             999.99 |
| 2023-02 |          129.99 |            1129.98 |
| 2023-03 |          889.98 |            2019.96 |
| 2023-04 |          189.98 |            2209.94 |
| 2023-05 |          209.98 |            2419.92 |
| 2023-06 |         1029.98 |            3449.90 |
| 2023-07 |          999.98 |            4449.88 |
| 2023-08 |          219.98 |            4669.86 |
| 2023-09 |          199.98 |            4869.84 |
| 2023-10 |          929.98 |            5799.82 |
| 2023-11 |          239.98 |            6039.80 |
| 2023-12 |         1199.97 |            7239.77 |
| 2024-01 |         1019.97 |            8259.74 |
| ...     |             ... |                ... |
+---------+-----------------+--------------------+
```

The trick here is `SUM(SUM(total_amount)) OVER (...)`. The inner `SUM` groups by month. The outer `SUM ... OVER` creates a running total across all months.

3-Month Moving Average

A moving average smooths out spikes and dips, revealing the underlying trend. It is like averaging your last three test scores instead of looking at each one individually.

```sql
WITH monthly AS (
    SELECT
        DATE_TRUNC('month', order_date)::DATE   AS month,
        SUM(total_amount)                        AS revenue
    FROM orders
    WHERE status = 'completed'
    GROUP BY DATE_TRUNC('month', order_date)
)
SELECT
```

```
    TO_CHAR(month, 'YYYY-MM')       AS month,
    revenue                          AS monthly_revenue,
    ROUND(AVG(revenue) OVER (
        ORDER BY month
        ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
    ), 2)                            AS moving_avg_3m
FROM monthly
ORDER BY month;
```

```
+---------+-----------------+----------------+
| month   | monthly_revenue | moving_avg_3m  |
+---------+-----------------+----------------+
| 2023-01 |          999.99 |         999.99 |
| 2023-02 |          129.99 |         564.99 |
| 2023-03 |          889.98 |         673.32 |
| 2023-04 |          189.98 |         403.32 |
| 2023-05 |          209.98 |         429.98 |
| 2023-06 |         1029.98 |         476.65 |
| 2023-07 |          999.98 |         746.65 |
| 2023-08 |          219.98 |         749.98 |
| 2023-09 |          199.98 |         473.31 |
| 2023-10 |          929.98 |         449.98 |
| 2023-11 |          239.98 |         456.65 |
| 2023-12 |         1199.97 |         789.98 |
| 2024-01 |         1019.97 |         819.97 |
| 2024-02 |         1229.97 |        1149.97 |
| 2024-03 |          289.97 |         846.64 |
| 2024-04 |         1019.97 |         846.64 |
| 2024-05 |         1059.97 |         789.97 |
| 2024-06 |          379.97 |         819.97 |
+---------+-----------------+----------------+
(18 rows)
```

The key is `ROWS BETWEEN 2 PRECEDING AND CURRENT ROW`. This tells PostgreSQL to average the current row plus the two rows before it -- a 3-month window.

The moving average reveals the trend more clearly. While monthly revenue jumps around from 129.99 to 1029.98, the moving average moves more smoothly, making it easier to see whether the business is growing or shrinking.

Customer Cohort Analysis

Cohort analysis groups customers by when they first joined (their "cohort") and tracks their behavior over time. It answers the question: "Are we retaining our customers?"

Step 1: Identify Each Customer's Cohort

A customer's cohort is the month they made their first purchase:

```sql
WITH customer_cohort AS (
    SELECT
        customer_id,
        DATE_TRUNC('month', MIN(order_date))::DATE AS cohort_month
    FROM orders
    WHERE status = 'completed'
    GROUP BY customer_id
),
customer_orders AS (
    SELECT
        o.customer_id,
        cc.cohort_month,
        DATE_TRUNC('month', o.order_date)::DATE AS order_month,
        -- Months since first purchase
        EXTRACT(YEAR FROM AGE(
            DATE_TRUNC('month', o.order_date),
            cc.cohort_month
        )) * 12 +
        EXTRACT(MONTH FROM AGE(
            DATE_TRUNC('month', o.order_date),
            cc.cohort_month
        )) AS months_since_first
    FROM orders o
    JOIN customer_cohort cc ON cc.customer_id = o.customer_id
    WHERE o.status = 'completed'
)
SELECT
    TO_CHAR(cohort_month, 'YYYY-MM')            AS cohort,
    months_since_first                           AS month_number,
    COUNT(DISTINCT customer_id)                  AS active_customers
FROM customer_orders
GROUP BY cohort_month, months_since_first
ORDER BY cohort_month, months_since_first;
```

```
+---------+--------------+------------------+
| cohort  | month_number | active_customers |
+---------+--------------+------------------+
| 2023-01 |            0 |                1 |
| 2023-01 |            1 |                1 |
| 2023-01 |            3 |                1 |
| 2023-01 |            6 |                1 |
| 2023-01 |            9 |                1 |
| 2023-01 |           12 |                1 |
| 2023-01 |           15 |                1 |
| 2023-03 |            0 |                2 |
| 2023-03 |            2 |                1 |
| 2023-03 |            6 |                1 |
| 2023-03 |           10 |                1 |
```

```
| 2023-03 |           14 |                1 |
| ...     |          ... |              ... |
+---------+--------------+------------------+
```

Step 2: Cohort Retention Table

The classic cohort retention table shows what percentage of each cohort returns in subsequent months:

```sql
WITH customer_cohort AS (
    SELECT
        customer_id,
        DATE_TRUNC('month', MIN(order_date))::DATE AS cohort_month
    FROM orders
    WHERE status = 'completed'
    GROUP BY customer_id
),
cohort_sizes AS (
    SELECT
        cohort_month,
        COUNT(DISTINCT customer_id) AS cohort_size
    FROM customer_cohort
    GROUP BY cohort_month
),
customer_activity AS (
    SELECT DISTINCT
        cc.cohort_month,
        o.customer_id,
        EXTRACT(YEAR FROM AGE(
            DATE_TRUNC('month', o.order_date),
            cc.cohort_month
        )) * 12 +
        EXTRACT(MONTH FROM AGE(
            DATE_TRUNC('month', o.order_date),
            cc.cohort_month
        )) AS month_number
    FROM orders o
    JOIN customer_cohort cc ON cc.customer_id = o.customer_id
    WHERE o.status = 'completed'
)
SELECT
    TO_CHAR(ca.cohort_month, 'YYYY-MM')         AS cohort,
    cs.cohort_size,
    ROUND(COUNT(DISTINCT CASE WHEN month_number = 0  THEN customer_id END)::NUMERIC
        / cs.cohort_size * 100)                  AS "M0",
    ROUND(COUNT(DISTINCT CASE WHEN month_number = 1  THEN customer_id END)::NUMERIC
        / cs.cohort_size * 100)                  AS "M1",
    ROUND(COUNT(DISTINCT CASE WHEN month_number = 2  THEN customer_id END)::NUMERIC
        / cs.cohort_size * 100)                  AS "M2",
```

```
    ROUND(COUNT(DISTINCT CASE WHEN month_number = 3  THEN customer_id END)::NUMERIC
        / cs.cohort_size * 100)                  AS "M3",
    ROUND(COUNT(DISTINCT CASE WHEN month_number = 6  THEN customer_id END)::NUMERIC
        / cs.cohort_size * 100)                  AS "M6"
FROM customer_activity ca
JOIN cohort_sizes cs ON cs.cohort_month = ca.cohort_month
GROUP BY ca.cohort_month, cs.cohort_size
ORDER BY ca.cohort_month;
```

```
+---------+-------------+------+------+------+------+------+
| cohort  | cohort_size | M0   | M1   | M2   | M3   | M6   |
+---------+-------------+------+------+------+------+------+
| 2023-01 |           1 |  100 |  100 |    0 |  100 |  100 |
| 2023-03 |           2 |  100 |    0 |   50 |    0 |   50 |
| 2023-04 |           1 |  100 |    0 |    0 |    0 |    0 |
| 2023-05 |           2 |  100 |   50 |    0 |    0 |   50 |
| 2023-06 |           1 |  100 |    0 |    0 |    0 |    0 |
| 2023-07 |           1 |  100 |  100 |    0 |    0 |    0 |
| 2023-08 |           1 |  100 |    0 |    0 |    0 |    0 |
| 2023-09 |           1 |  100 |    0 |    0 |    0 |    0 |
| 2023-10 |           1 |  100 |    0 |  100 |    0 |    0 |
| 2023-12 |           1 |  100 |    0 |    0 |    0 |  100 |
| 2024-01 |           1 |  100 |    0 |    0 |    0 |    0 |
| 2024-02 |           1 |  100 |    0 |    0 |    0 |    0 |
| 2024-04 |           2 |  100 |   50 |    0 |    0 |    0 |
+---------+-------------+------+------+------+------+------+
```

How to read this table:

- The 2023-01 cohort (1 customer) had 100% active in month 0 (by definition), 100% in month 1, then returned in months 3 and 6.
- The 2023-03 cohort (2 customers) had 100% in month 0, but only 50% (1 of 2) returned in month 2.

This is one of the most powerful analytical tools in business. High retention means customers keep coming back. Low retention means you are losing people.

RFM Analysis

RFM stands for Recency, Frequency, and Monetary value. It is a customer segmentation technique that divides customers into groups based on:

- Recency: How recently did they buy? (Recent buyers are more engaged)
- Frequency: How often do they buy? (Frequent buyers are more loyal)

- Monetary: How much do they spend? (High spenders are more valuable)

```sql
WITH rfm_raw AS (
    SELECT
        c.id                                AS customer_id,
        c.name                              AS customer_name,
        -- Recency: days since last purchase
        CURRENT_DATE - MAX(o.order_date)    AS recency_days,
        -- Frequency: number of orders
        COUNT(DISTINCT o.id)                AS frequency,
        -- Monetary: total amount spent
        SUM(o.total_amount)                 AS monetary
    FROM customers c
    JOIN orders o ON o.customer_id = c.id
    WHERE o.status = 'completed'
    GROUP BY c.id, c.name
),
rfm_scored AS (
    SELECT
        customer_id,
        customer_name,
        recency_days,
        frequency,
        ROUND(monetary, 2)                  AS monetary,
        -- Score each dimension from 1-4 using NTILE
        NTILE(4) OVER (ORDER BY recency_days DESC)     AS r_score,
        NTILE(4) OVER (ORDER BY frequency ASC)          AS f_score,
        NTILE(4) OVER (ORDER BY monetary ASC)           AS m_score
    FROM rfm_raw
)
SELECT
    customer_name,
    recency_days,
    frequency,
    monetary,
    r_score,
    f_score,
    m_score,
    r_score + f_score + m_score     AS rfm_total,
    CASE
        WHEN r_score >= 3 AND f_score >= 3 AND m_score >= 3 THEN 'Champions'
        WHEN r_score >= 3 AND f_score >= 2                  THEN 'Loyal Customers'
        WHEN r_score >= 3 AND f_score = 1                   THEN 'New Customers'
        WHEN r_score = 2  AND f_score >= 2                  THEN 'At Risk'
        WHEN r_score = 1  AND f_score >= 2                  THEN 'Losing Them'
        WHEN r_score = 1  AND f_score = 1                   THEN 'Lost'
        ELSE 'Other'
    END                             AS segment
FROM rfm_scored
ORDER BY rfm_total DESC;
```

```
+-----------------+--------------+-----------+----------+---------+---------+---------+...
| customer_name   | recency_days | frequency | monetary | r_score | f_score | m_score |...
```

```
+-----------------+--------------+-----------+----------+---------+---------+---------+...
| Alice Johnson   |          362 |         7 |  2389.92 |       4 |       4 |       4 |...
| Bob Smith       |          301 |         5 |  2119.95 |       4 |       4 |       4 |...
| Carol Williams  |          326 |         5 |  1349.94 |       4 |       4 |       3 |...
| David Brown     |          277 |         4 |  1059.96 |       4 |       3 |       3 |...
| Eve Davis       |          390 |         4 |  1269.96 |       3 |       3 |       3 |...
| Jack Wilson     |          291 |         3 |  1249.97 |       4 |       2 |       3 |...
| Frank Miller    |          343 |         3 |   939.97 |       3 |       2 |       2 |...
| Grace Taylor    |          383 |         2 |   329.98 |       2 |       1 |       1 |...
| Ivy Chen        |          311 |         2 |   189.98 |       3 |       1 |       1 |...
| Henry Anderson  |          367 |         2 |   199.98 |       2 |       1 |       1 |...
| Leo Martinez    |          397 |         1 |   149.99 |       1 |       1 |       1 |...
| Karen Lee       |          439 |         1 |   799.99 |       1 |       1 |       2 |...
| Mia Robinson    |          354 |         1 |   129.99 |       2 |       1 |       1 |...
| Noah Clark      |          318 |         1 |    29.99 |       3 |       1 |       1 |...
+-----------------+--------------+-----------+----------+---------+---------+---------+...
```

Let us break down the scoring:

- `NTILE(4) OVER (ORDER BY recency_days DESC)` -- Divides customers into 4 equal groups. The most recent buyers (fewest days) get a score of 4. We use `DESC` because low recency_days is better.
- `NTILE(4) OVER (ORDER BY frequency ASC)` -- Most frequent buyers get a 4.
- `NTILE(4) OVER (ORDER BY monetary ASC)` -- Highest spenders get a 4.

The `CASE` statement then assigns human-readable segments:

- Champions: Recent, frequent, high-spending. Your best customers.
- Loyal Customers: Regular buyers who come back often.
- New Customers: Recent but low frequency. Just getting started.
- At Risk: Used to buy but have not recently.
- Losing Them: Were once active but falling away.
- Lost: Have not bought in a long time and were infrequent.

This analysis tells you exactly where to focus your marketing efforts. Champions get reward programs. At-risk customers get re-engagement campaigns. Lost customers might get a "We miss you" email.

Top N Products per Category

Finding the top-selling products within each category requires the `ROW_NUMBER` window function with `PARTITION BY`.

```sql
WITH product_sales AS (
    SELECT
        p.id                            AS product_id,
        p.name                          AS product_name,
        p.category,
        p.price,
        p.cost,
        SUM(oi.quantity)                AS total_sold,
        SUM(oi.subtotal)               AS total_revenue,
        SUM(oi.subtotal) - SUM(oi.quantity * p.cost)
                                        AS total_profit,
        ROW_NUMBER() OVER (
            PARTITION BY p.category
            ORDER BY SUM(oi.subtotal) DESC
        )                               AS rank_in_category
    FROM products p
    JOIN order_items oi ON oi.product_id = p.id
    JOIN orders o ON o.id = oi.order_id
    WHERE o.status = 'completed'
    GROUP BY p.id, p.name, p.category, p.price, p.cost
)
SELECT
    category,
    rank_in_category    AS rank,
    product_name,
    total_sold,
    total_revenue,
    ROUND(total_profit, 2) AS total_profit,
    ROUND(total_profit / NULLIF(total_revenue, 0) * 100, 1)
                        AS profit_margin_pct
FROM product_sales
WHERE rank_in_category <= 3
ORDER BY category, rank_in_category;
```

```
+-------------+------+---------------------+------------+---------------+--------------...
| category    | rank | product_name        | total_sold | total_revenue | total_profit ...
+-------------+------+---------------------+------------+---------------+--------------...
| Books       | 1    | Data Science Guide  |          4 |        199.96 |       139.96 ...
| Books       | 2    | SQL Mastery Book    |          3 |        119.97 |        83.97 ...
| Clothing    | 1    | Winter Jacket       |          3 |        599.97 |       344.97 ...
| Electronics | 1    | ProBook Laptop      |          4 |       3999.96 |      1399.96 ...
| Electronics | 2    | SmartPhone X        |          4 |       3199.96 |      1279.96 ...
| Electronics | 3    | Wireless Earbuds    |          5 |        649.95 |       374.95 ...
| Home        | 1    | Coffee Maker        |          4 |        599.96 |       319.96 ...
| Home        | 2    | Blender Pro         |          2 |        159.98 |        99.98 ...
| Sports      | 1    | Running Shoes       |          3 |        269.97 |       164.97 ...
| Sports      | 2    | Yoga Mat            |          2 |         59.98 |        39.98 ...
+-------------+------+---------------------+------------+---------------+--------------...
```

The key technique here:

- `ROW_NUMBER() OVER (PARTITION BY p.category ORDER BY SUM(oi.subtotal) DESC)` -- This assigns rank 1, 2, 3... within each category based on total revenue.
- `WHERE rank_in_category <= 3` -- We filter to keep only the top 3 per category.

This is sometimes called a "Top N per group" query. It is one of the most common analytical patterns.

Funnel Analysis

Funnel analysis tracks how users move through a conversion process. Each step narrows the funnel, and we want to know where people drop off.

```
+----------------------------+
|         Signup             |     100%
|        (15 users)          |
+-------------+--------------+
              |
+-------------v--------------+
|         Browse             |     100%
|        (15 users)          |
+-------------+--------------+
              |
+-------------v--------------+
|      Add to Cart           |     100%
|        (15 users)          |
+-------------+--------------+
              |
+-------------v--------------+
|        Purchase            |     93%
|        (14 users)          |
+----------------------------+
```

Building the Funnel Query

```sql
WITH funnel AS (
    SELECT
        customer_id,
        MAX(CASE WHEN event_type = 'signup'      THEN 1 ELSE 0 END) AS signed_up,
        MAX(CASE WHEN event_type = 'browse'      THEN 1 ELSE 0 END) AS browsed,
        MAX(CASE WHEN event_type = 'add_to_cart' THEN 1 ELSE 0 END) AS added_to_cart,
        MAX(CASE WHEN event_type = 'purchase'    THEN 1 ELSE 0 END) AS purchased
    FROM user_events
```

```
    GROUP BY customer_id
)
SELECT
    'Signup'        AS stage,
    SUM(signed_up)  AS users,
    ROUND(SUM(signed_up)::NUMERIC / SUM(signed_up) * 100, 1)
                    AS pct_of_total,
    100.0           AS conversion_rate
FROM funnel
UNION ALL
SELECT
    'Browse',
    SUM(browsed),
    ROUND(SUM(browsed)::NUMERIC / SUM(signed_up) * 100, 1),
    ROUND(SUM(browsed)::NUMERIC / SUM(signed_up) * 100, 1)
FROM funnel
UNION ALL
SELECT
    'Add to Cart',
    SUM(added_to_cart),
    ROUND(SUM(added_to_cart)::NUMERIC / SUM(signed_up) * 100, 1),
    ROUND(SUM(added_to_cart)::NUMERIC / SUM(browsed) * 100, 1)
FROM funnel
UNION ALL
SELECT
    'Purchase',
    SUM(purchased),
    ROUND(SUM(purchased)::NUMERIC / SUM(signed_up) * 100, 1),
    ROUND(SUM(purchased)::NUMERIC / SUM(added_to_cart) * 100, 1)
FROM funnel;
```

```
+--------------+-------+--------------+-----------------+
| stage        | users | pct_of_total | conversion_rate |
+--------------+-------+--------------+-----------------+
| Signup       |    15 |        100.0 |           100.0 |
| Browse       |    15 |        100.0 |           100.0 |
| Add to Cart  |    15 |        100.0 |           100.0 |
| Purchase     |    14 |         93.3 |            93.3 |
+--------------+-------+--------------+-----------------+
(4 rows)
```

The `conversion_rate` column shows the step-by-step conversion. The `pct_of_total` shows cumulative drop-off from the first step.

Funnel with Time Analysis

How long does each step take?

```sql
WITH step_times AS (
    SELECT
        customer_id,
        MIN(CASE WHEN event_type = 'signup'      THEN event_date END) AS signup_time,
        MIN(CASE WHEN event_type = 'browse'      THEN event_date END) AS browse_time,
        MIN(CASE WHEN event_type = 'add_to_cart' THEN event_date END) AS cart_time,
        MIN(CASE WHEN event_type = 'purchase'    THEN event_date END) AS purchase_time
    FROM user_events
    GROUP BY customer_id
)
SELECT
    ROUND(AVG(EXTRACT(EPOCH FROM (browse_time - signup_time)) / 60), 0)
        AS avg_signup_to_browse_min,
    ROUND(AVG(EXTRACT(EPOCH FROM (cart_time - browse_time)) / 3600 / 24), 1)
        AS avg_browse_to_cart_days,
    ROUND(AVG(EXTRACT(EPOCH FROM (purchase_time - cart_time)) / 3600 / 24), 1)
        AS avg_cart_to_purchase_days,
    ROUND(AVG(EXTRACT(EPOCH FROM (purchase_time - signup_time)) / 3600 / 24), 1)
        AS avg_total_journey_days
FROM step_times
WHERE purchase_time IS NOT NULL;
```

```
+-------------------------+------------------------+---------------------------+------...
| avg_signup_to_browse_min | avg_browse_to_cart_days | avg_cart_to_purchase_days  | avg...
+-------------------------+------------------------+---------------------------+------...
|                       17 |                   13.5 |                      14.8 |      ...
+-------------------------+------------------------+---------------------------+------...
(1 row)
```

This tells us that on average:

- Users browse within 17 minutes of signing up.
- It takes about 13.5 days from browsing to adding something to the cart.
- It takes about 14.8 more days to go from cart to purchase.
- The total journey from signup to first purchase is about 29.1 days.

This information helps the marketing team decide when to send reminder emails and follow-up offers.

Complete Dashboard Queries

Here is a set of queries you might use to power an executive dashboard.

Dashboard Query 1: Revenue Overview

```sql
WITH current_period AS (
    SELECT
        SUM(total_amount)                       AS current_month_revenue,
        COUNT(DISTINCT id)                      AS current_month_orders,
        COUNT(DISTINCT customer_id)             AS current_month_customers
    FROM orders
    WHERE status = 'completed'
      AND DATE_TRUNC('month', order_date) = DATE_TRUNC('month', CURRENT_DATE)
),
prior_period AS (
    SELECT
        SUM(total_amount)                       AS prior_month_revenue,
        COUNT(DISTINCT id)                      AS prior_month_orders,
        COUNT(DISTINCT customer_id)             AS prior_month_customers
    FROM orders
    WHERE status = 'completed'
      AND DATE_TRUNC('month', order_date) = DATE_TRUNC('month', CURRENT_DATE - INTERVAL...
)
SELECT
    COALESCE(cp.current_month_revenue, 0)       AS this_month_revenue,
    COALESCE(pp.prior_month_revenue, 0)         AS last_month_revenue,
    CASE
        WHEN pp.prior_month_revenue > 0
        THEN ROUND((cp.current_month_revenue - pp.prior_month_revenue)
             / pp.prior_month_revenue * 100, 1)
        ELSE NULL
    END                                          AS revenue_change_pct,
    COALESCE(cp.current_month_orders, 0)        AS this_month_orders,
    COALESCE(cp.current_month_customers, 0)     AS this_month_customers
FROM current_period cp
CROSS JOIN prior_period pp;
```

Dashboard Query 2: Revenue by Region

```sql
SELECT
    c.region,
    COUNT(DISTINCT o.id)                AS order_count,
    COUNT(DISTINCT o.customer_id)       AS customer_count,
    SUM(o.total_amount)                 AS total_revenue,
    ROUND(AVG(o.total_amount), 2)       AS avg_order_value,
    ROUND(SUM(o.total_amount) / SUM(SUM(o.total_amount)) OVER () * 100, 1)
                                        AS revenue_share_pct
FROM orders o
JOIN customers c ON c.id = o.customer_id
WHERE o.status = 'completed'
GROUP BY c.region
ORDER BY total_revenue DESC;
```

```
+----------+-------------+----------------+---------------+-----------------+----------...
| region   | order_count | customer_count | total_revenue | avg_order_value | revenue_s...
+----------+-------------+----------------+---------------+-----------------+----------...
| West     |          17 |              5 |       5349.83 |          314.70 |          ...
| East     |          13 |              4 |       4419.87 |          339.99 |          ...
| Central  |          11 |              4 |       3769.90 |          342.72 |          ...
+----------+-------------+----------------+---------------+-----------------+----------...
(3 rows)
```

The clever part is `SUM(o.total_amount) / SUM(SUM(o.total_amount)) OVER ()`. The inner `SUM` groups by region. The outer `SUM ... OVER ()` with an empty window calculates the grand total. Dividing gives us each region's share.

Dashboard Query 3: Customer Acquisition Trend

```sql
SELECT
    TO_CHAR(signup_date, 'YYYY-MM')     AS month,
    COUNT(*)                             AS new_customers,
    SUM(COUNT(*)) OVER (
        ORDER BY TO_CHAR(signup_date, 'YYYY-MM')
    )                                    AS cumulative_customers
FROM customers
GROUP BY TO_CHAR(signup_date, 'YYYY-MM')
ORDER BY month;
```

```
+---------+----------------+-----------------------+
| month   | new_customers  | cumulative_customers  |
+---------+----------------+-----------------------+
| 2023-01 |              1 |                     1 |
| 2023-02 |              1 |                     2 |
| 2023-03 |              1 |                     3 |
| 2023-04 |              1 |                     4 |
| 2023-05 |              1 |                     5 |
| 2023-06 |              1 |                     6 |
| 2023-07 |              1 |                     7 |
| 2023-08 |              1 |                     8 |
| 2023-09 |              1 |                     9 |
| 2023-10 |              1 |                    10 |
| 2024-01 |              1 |                    11 |
| 2024-02 |              1 |                    12 |
| 2024-03 |              1 |                    13 |
| 2024-04 |              1 |                    14 |
| 2024-05 |              1 |                    15 |
+---------+----------------+-----------------------+
(15 rows)
```

Dashboard Query 4: Product Category Performance

```sql
SELECT
    p.category,
    COUNT(DISTINCT oi.order_id)         AS orders,
    SUM(oi.quantity)                     AS units_sold,
    SUM(oi.subtotal)                    AS revenue,
    SUM(oi.subtotal - (oi.quantity * p.cost))
                                        AS gross_profit,
    ROUND(
        SUM(oi.subtotal - (oi.quantity * p.cost))
        / NULLIF(SUM(oi.subtotal), 0) * 100,
        1
    )                                    AS margin_pct
FROM products p
JOIN order_items oi ON oi.product_id = p.id
JOIN orders o ON o.id = oi.order_id
WHERE o.status = 'completed'
GROUP BY p.category
ORDER BY revenue DESC;
```

```
+-------------+--------+------------+----------+--------------+------------+
| category    | orders | units_sold | revenue  | gross_profit | margin_pct |
+-------------+--------+------------+----------+--------------+------------+
| Electronics |     13 |         13 | 7849.87  |     3054.87  |       38.9 |
| Home        |      6 |          6 |  759.94  |      419.94  |       55.3 |
| Clothing    |      3 |          3 |  599.97  |      344.97  |       57.5 |
| Sports      |      5 |          5 |  329.95  |      204.95  |       62.1 |
| Books       |      7 |          7 |  319.93  |      223.93  |       70.0 |
+-------------+--------+------------+----------+--------------+------------+
(5 rows)
```

Notice that Electronics generates the most revenue but has the lowest margin (38.9%), while Books has the highest margin (70.0%) but the least revenue. This is a common business insight: high-ticket items often have lower margins.

Common Mistakes

1. Forgetting to handle NULL in LAG/LEAD. When there is no prior row, `LAG` returns NULL. Always wrap growth calculations in a `CASE` or `COALESCE` to avoid dividing by NULL.
2. Using the wrong date function. `DATE_TRUNC('month', date)` rounds down to the first of the month. `EXTRACT(MONTH FROM date)` gives just the month number. Know which one you need.
3. Not filtering cancelled orders. Analytics should typically exclude cancelled orders. Always include `WHERE status = 'completed'` (or your equivalent) to avoid inflating

numbers.

4. Confusing ROW_NUMBER, RANK, and DENSE_RANK. `ROW_NUMBER` always gives unique numbers. `RANK` skips numbers for ties. `DENSE_RANK` does not skip. For "Top N" queries, `ROW_NUMBER` is usually what you want.
5. Mixing aggregation levels. Be careful when joining tables at different granularities. Joining orders with order_items before aggregating can multiply your counts. Use CTEs to aggregate first, then join.

Best Practices

1. Use CTEs for readability. Break complex analytics queries into named steps with `WITH`. Each CTE should do one thing. This makes queries easier to understand, debug, and modify.
2. Always use DATE_TRUNC for time-based grouping. It handles all the edge cases (different month lengths, leap years, etc.) correctly.
3. Name your columns clearly. Use aliases like `monthly_revenue` instead of `sum`. Dashboard tools and downstream users need to understand what each column represents.
4. Test with known data. Before running analytics on production data, test your queries on a small dataset where you can manually verify the results.
5. Consider materialized views for heavy queries. If a dashboard query scans millions of rows and runs frequently, create a materialized view that refreshes on a schedule.
6. Document your business rules. "Revenue" can mean gross, net, or adjusted. Make sure everyone agrees on definitions before writing queries.

Quick Summary

You built a complete analytics toolkit:

- Time-based aggregation with `DATE_TRUNC` and `GROUP BY` for daily, weekly, and monthly revenue.
- Year-over-year growth using the `LAG` window function to compare months across years.
- Running totals with `SUM() OVER (ORDER BY ...)` for cumulative metrics.
- Moving averages with `AVG() OVER (ROWS BETWEEN ... AND ...)` for trend analysis.
- Cohort analysis to track customer retention over time.

- RFM segmentation using `NTILE` to classify customers.
- Top N per group with `ROW_NUMBER` and `PARTITION BY`.
- Funnel analysis to measure conversion rates at each step.
- Dashboard queries combining all techniques for real-time business intelligence.

Key Points

- `DATE_TRUNC` is the essential function for grouping data by time periods.
- `LAG` and `LEAD` window functions compare rows to previous or next rows.
- Running totals use `SUM() OVER (ORDER BY ...)` without a frame clause.
- Moving averages use `AVG() OVER (ROWS BETWEEN N PRECEDING AND CURRENT ROW)`.
- Cohort analysis identifies when customers first appeared and tracks them over time.
- RFM analysis uses `NTILE` to score customers on Recency, Frequency, and Monetary value.
- `ROW_NUMBER() OVER (PARTITION BY ... ORDER BY ...)` is the standard pattern for Top N per group.
- Funnel analysis uses `CASE` expressions to flag whether each user reached each stage.
- Always exclude cancelled or invalid orders from analytics calculations.

Practice Questions

1. How would you modify the YoY growth query to show quarter-over-quarter growth instead of month-over-month? What changes are needed?
2. In the RFM analysis, why do we use `NTILE(4)` instead of manually setting score thresholds? What are the advantages and disadvantages of each approach?
3. The funnel analysis shows a 93.3% conversion from "Add to Cart" to "Purchase." What query would you write to find the specific customers who dropped off at the cart stage?
4. How is a running total different from a moving average? When would you use each one?
5. In the cohort retention table, what does a 0% value in month 3 mean? Is this necessarily bad? What additional data might help explain it?

Exercises

Exercise 1: Revenue Forecast

Using the monthly revenue data, write a query that:

1. Calculates the average monthly growth rate over the last 6 months.
2. Projects the next 3 months of revenue using that growth rate.
3. Shows both actual and projected values in the same result set.

Hint: Use a CTE for the historical data and a `generate_series` for future months.

Exercise 2: Customer Lifetime Value

Write a query that calculates the Customer Lifetime Value (CLV) for each customer:

1. Calculate their average order value.
2. Calculate their average time between orders (in days).
3. Calculate their estimated yearly order frequency.
4. Estimate CLV as: `avg_order_value * estimated_yearly_orders * avg_customer_lifespan_years` (assume 3-year lifespan).
5. Rank customers by CLV and identify the top 5.

Exercise 3: Build a Complete Dashboard

Create a single query (or set of queries) that produces a complete monthly business report including:

1. Total revenue, order count, and unique customers.
2. New vs. returning customer breakdown.
3. Top 3 products by revenue.
4. Revenue by region with percentage share.
5. Comparison to the previous month (with percentage change).

Congratulations!

You have reached the end of the practical chapters in this book. Take a moment to appreciate how far you have come.

You started with the basics: what a database is, how to create tables, and how to write your first SELECT query. Along the way you learned joins, subqueries, window functions, normalization, indexes, triggers, transactions, and security. Then you put it all together by building a complete e-commerce database and writing the analytical queries that power real business decisions.

These are not theoretical skills. They are the same techniques used by data analysts at every major company, by backend developers building applications, and by data engineers designing data pipelines. You now have a solid foundation in SQL and database design.

What to Learn Next

Your SQL journey does not end here. Here are paths to explore:

- Advanced PostgreSQL features: Partitioning, materialized views, JSONB operations, full-text search, and PostGIS for geospatial data.
- Database administration: Replication, high availability, connection pooling with PgBouncer, and monitoring with pg_stat_statements.
- Data warehousing: Star schemas, fact and dimension tables, slowly changing dimensions, and ETL pipelines.
- Other databases: MySQL, SQLite, Microsoft SQL Server, and cloud databases like Amazon RDS and Google Cloud SQL.
- NoSQL databases: MongoDB, Redis, Cassandra, and understanding when to use them instead of (or alongside) relational databases.
- ORMs and frameworks: SQLAlchemy (Python), Prisma (JavaScript), ActiveRecord (Ruby), and how they generate SQL under the hood.
- Data tools: dbt for data transformation, Apache Airflow for orchestration, and Metabase or Grafana for visualization.

Whatever path you choose, remember: SQL has been around since the 1970s and is not going anywhere. The skills you have built in this book will serve you for your entire career.

Good luck, and happy querying.

Chapter 34: Glossary — Every Term You Need to Know

What You Will Learn

In this chapter, you will find:

- Definitions for 80+ SQL and database terms
- All terms organized alphabetically for quick reference
- Plain-English explanations with brief examples where helpful
- Cross-references to related terms

Why This Chapter Matters

Every field has its own vocabulary, and databases are no exception. When you read documentation, attend a job interview, or join a team discussion, you will hear terms like "cardinality," "referential integrity," and "materialized view." If these terms are unfamiliar, the conversation becomes confusing fast.

This glossary gives you a single place to look up any term you encounter. Bookmark this chapter. Come back to it whenever you need a quick refresher. Over time, these terms will become second nature.

The Glossary

ACID

A set of four properties that guarantee database transactions are reliable. ACID stands for Atomicity (all or nothing), Consistency (data stays valid), Isolation (transactions do not interfere with each other), and Durability (committed changes survive crashes). Every modern relational database, including PostgreSQL, follows ACID rules. See Chapter 25.

Aggregate Function

A function that takes multiple rows as input and returns a single value. Common aggregate functions include `COUNT()`, `SUM()`, `AVG()`, `MIN()`, and `MAX()`. For example, `SELECT AVG(price) FROM products` returns the average price across all products. Aggregate functions are often used with GROUP BY. See Chapter 10.

Alias

A temporary name you give to a table or column in a query. Aliases make queries easier to read. You create them with the `AS` keyword (or just a space). For example, `SELECT first_name AS name FROM customers` renames the output column to "name." Table aliases like `FROM orders AS o` let you write shorter JOIN conditions. See Chapter 7.

ALTER

A DDL command used to modify an existing database object. `ALTER TABLE` can add or drop columns, add or remove constraints, rename columns, and change data types. For example, `ALTER TABLE customers ADD COLUMN phone VARCHAR(20)`. See Chapters 4 and 24.

B-tree

The default index type in PostgreSQL. B-tree stands for "balanced tree." It organizes data in a sorted tree structure that allows fast lookups, range scans, and ordering. B-tree indexes work well with equality (`=`) and range (`<`, `>`, `BETWEEN`) comparisons. Think of it like a phone book — you can quickly jump to the right section without reading every page. See Chapter 21.

BEGIN

A command that starts a new transaction. Everything between `BEGIN` and `COMMIT` (or `ROLLBACK`) is treated as a single unit of work. If anything fails, you can roll back all changes. For example: `BEGIN; UPDATE accounts SET balance = 0; COMMIT;`. See Chapter 25.

Candidate Key

A column or set of columns that could serve as the primary key because it uniquely identifies every row. A table might have several candidate keys, but only one is chosen as the primary key. The others can be enforced with UNIQUE constraints. For example, both `email` and `social_security_number` might uniquely identify a person — both are candidate keys.

Cardinality

In database design, cardinality describes the numerical relationship between two tables. The three types are: one-to-one (one person has one passport), one-to-many (one customer has many orders), and many-to-many (many students enroll in many courses). In performance context, cardinality refers to the number of distinct values in a column — high cardinality means many unique values. See Chapter 23.

CASCADE

An action that automatically propagates changes from a parent table to child tables. `ON DELETE CASCADE` deletes child rows when the parent is deleted. `ON UPDATE CASCADE` updates child foreign keys when the parent's primary key changes. Use CASCADE carefully — it can delete more data than you expect. See Chapter 24.

CHECK

A constraint that validates data against a custom condition before allowing it into the table. For example, `CHECK (price > 0)` ensures prices are always positive. `CHECK (status IN ('active', 'inactive'))` limits status to specific values. The condition must evaluate to TRUE or NULL for the row to be accepted. See Chapter 24.

COALESCE

A function that returns the first non-NULL value from a list of arguments. `COALESCE(phone, email, 'No contact')` returns the phone number if it exists, otherwise the email, otherwise the text "No contact." It is commonly used to provide fallback values when dealing with NULL data. See Chapter 18.

Composite Key

A primary key made up of two or more columns. The combination of values must be unique, but individual columns can have duplicates. For example, an order_items table might use `(order_id, product_id)` as its composite key — the same product can appear in different orders, but not twice in the same order. See Chapter 24.

Constraint

A rule attached to a table or column that restricts what data can be stored. Constraints include PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, CHECK, and DEFAULT. Constraints enforce data integrity at the database level so bad data is rejected before it is stored. See Chapter 24.

Correlated Subquery

A subquery that references a column from the outer query. Unlike a regular subquery that runs once, a correlated subquery runs once for each row in the outer query. For example: `SELECT * FROM employees e WHERE salary > (SELECT AVG(salary) FROM employees WHERE dept_id = e.dept_id)`. This finds employees who earn more than their department's average. See Chapter 16.

CTE (Common Table Expression)

A named temporary result set defined within a query using the `WITH` keyword. CTEs make complex queries easier to read by breaking them into logical steps. For example: `WITH top_customers AS (SELECT ... FROM ...) SELECT * FROM top_customers`. CTEs exist only for the duration of the query. See Chapter 27.

DDL (Data Definition Language)

The subset of SQL commands that define and modify database structure. DDL commands include `CREATE`, `ALTER`, `DROP`, and `TRUNCATE`. These commands change the schema (tables, indexes, views) rather than the data inside the tables. DDL commands are auto-committed in many databases, but PostgreSQL supports transactional DDL.

Deadlock

A situation where two or more transactions are waiting for each other to release locks, creating a circular dependency. Neither transaction can proceed. PostgreSQL automatically detects deadlocks and kills one of the transactions so the other can continue. Prevent deadlocks by always locking rows in a consistent order. See Chapter 25.

DEFAULT

A constraint that provides an automatic value for a column when no value is specified during INSERT. For example, `created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP` automatically records the current time. `status VARCHAR(20) DEFAULT 'active'` sets new rows to "active"

unless you specify otherwise. See Chapter 24.

DELETE

A DML command that removes rows from a table. `DELETE FROM customers WHERE customer_id = 5` removes one specific customer. `DELETE FROM logs WHERE created_at < '2024-01-01'` removes old log entries. DELETE without a WHERE clause removes all rows but keeps the table structure. Always use a WHERE clause unless you intend to delete everything. See Chapter 6.

Denormalization

The deliberate process of adding redundant data to a normalized database to improve read performance. For example, storing a customer's name directly in the orders table (instead of joining to the customers table every time) is denormalization. It speeds up reads but makes updates harder because you must keep the redundant copies in sync. See Chapter 22.

DML (Data Manipulation Language)

The subset of SQL commands that work with the data inside tables. DML commands include `SELECT`, `INSERT`, `UPDATE`, and `DELETE`. Unlike DDL (which changes structure), DML changes the actual data stored in tables.

Entity

A real-world object or concept that is represented as a table in a database. A customer, a product, an order, and an employee are all entities. Each entity has attributes (columns) that describe it and instances (rows) that represent individual records. See Chapter 23.

ER Diagram (Entity-Relationship Diagram)

A visual representation of the tables in a database and the relationships between them. Tables are shown as boxes, columns as lists within the boxes, and relationships as lines connecting the boxes. ER diagrams help you design and communicate database structure. See Chapter 23.

EXCEPT

A set operation that returns rows from the first query that do not appear in the second query. `SELECT city FROM customers EXCEPT SELECT city FROM suppliers` returns cities where you have customers but no suppliers. EXCEPT removes duplicates from the result. Use EXCEPT

ALL to keep duplicates. See Chapter 17.

EXPLAIN

A command that shows the query plan — how PostgreSQL intends to execute a query. `EXPLAIN SELECT * FROM orders WHERE total > 100` shows whether the database will use an index or a sequential scan. `EXPLAIN ANALYZE` actually runs the query and shows real execution times. Essential for performance tuning. See Chapter 30.

FETCH

A command used with cursors to retrieve a specific number of rows from a result set. In standard SQL, `FETCH FIRST 10 ROWS ONLY` limits results (similar to LIMIT). In PL/pgSQL, `FETCH` retrieves rows from an open cursor one at a time. See Chapter 28.

FOREIGN KEY

A constraint that creates a link between two tables by requiring that a column's value must match an existing value in another table's primary key (or unique column). Foreign keys enforce referential integrity — they prevent orphan records. For example, an order's customer_id must reference a real customer. See Chapter 24.

FROM

The clause in a SELECT statement that specifies which table or tables to read data from. `SELECT * FROM customers` reads from the customers table. FROM can include multiple tables (for joins), subqueries, CTEs, or functions. See Chapter 7.

FULL JOIN (FULL OUTER JOIN)

A join that returns all rows from both tables. Rows that have a match are combined. Rows from either table that have no match are included with NULL values for the missing side. FULL JOIN is useful when you want to see all data from both tables, whether or not there is a match. See Chapter 15.

Function

A reusable piece of code that accepts input parameters, performs a computation, and returns a result. PostgreSQL has built-in functions (like `NOW()`, `LENGTH()`, `UPPER()`) and lets you create custom functions using PL/pgSQL or other languages. Functions can return a single value, a row, or a table. See Chapters 18 and 28.

GIN (Generalized Inverted Index)

An index type in PostgreSQL designed for values that contain multiple elements, such as arrays, JSONB documents, and full-text search vectors. GIN indexes store a mapping from each element to the rows that contain it. Think of it like a book index — you look up a word and find all the pages that mention it. See Chapter 21.

GiST (Generalized Search Tree)

An index type in PostgreSQL that supports complex data types like geometric shapes, ranges, and full-text search. GiST is more flexible than B-tree and can handle operations like "find all points within this circle" or "find all ranges that overlap." See Chapter 21.

GRANT

A command that gives a user or role specific permissions on a database object. `GRANT SELECT ON customers TO analyst_role` allows the analyst to read the customers table. `GRANT ALL ON products TO admin_role` gives full access. See Chapter 31.

GROUP BY

A clause that groups rows with the same values into summary rows. Used with aggregate functions to compute per-group results. `SELECT department, COUNT(*) FROM employees GROUP BY department` counts employees in each department. Every column in SELECT must be in GROUP BY or inside an aggregate function. See Chapter 11.

HAVING

A clause that filters groups after GROUP BY has been applied. HAVING is like WHERE, but for groups instead of individual rows. `SELECT department, AVG(salary) FROM employees GROUP BY department HAVING AVG(salary) > 50000` shows only departments with an average salary above 50,000. See Chapter 11.

Index

A data structure that speeds up data retrieval at the cost of extra storage and slower writes. Think of an index like a book's index — instead of reading every page to find a topic, you look it up in the index and jump directly to the right page. PostgreSQL supports several index types: B-tree, Hash, GIN, GiST, and others. See Chapter 21.

INNER JOIN

A join that returns only rows where there is a match in both tables. If a customer has no orders, that customer does not appear in the result. If an order references a non-existent customer, that order does not appear either. INNER JOIN is the most common join type. See Chapter 13.

INSERT

A DML command that adds new rows to a table. `INSERT INTO customers (name, email) VALUES ('Alice', 'alice@example.com')` adds one row. You can insert multiple rows at once and use RETURNING to see the inserted data. See Chapter 6.

INTERSECT

A set operation that returns only rows that appear in both the first and second query results. `SELECT city FROM customers INTERSECT SELECT city FROM suppliers` returns cities where you have both customers and suppliers. INTERSECT removes duplicates. See Chapter 17.

Isolation Level

A setting that controls how much one transaction can see of another transaction's uncommitted changes. PostgreSQL supports three levels: READ COMMITTED (default — sees only committed data), REPEATABLE READ (consistent snapshot throughout the transaction), and SERIALIZABLE (strictest — transactions behave as if they run one at a time). See Chapter 25.

JOIN

An operation that combines rows from two or more tables based on a related column. The main types are INNER JOIN, LEFT JOIN, RIGHT JOIN, FULL JOIN, and CROSS JOIN. JOINs are fundamental to relational databases because data is split across multiple tables by design. See Chapters 12 through 15.

JSON / JSONB

Data types for storing JSON (JavaScript Object Notation) data. `JSON` stores an exact copy of the input text. `JSONB` stores a parsed binary representation that is faster to query and supports indexing. Use JSONB in most cases. PostgreSQL provides operators like `->`, `->>`, and `@>` for querying JSON data.

Key

A column or set of columns used to identify rows or establish relationships. Types of keys include: primary key (unique identifier), foreign key (references another table), candidate key (could be a primary key), composite key (multiple columns), and surrogate key (artificial identifier like SERIAL).

LAG

A window function that accesses a value from a previous row without using a self-join. `LAG(sales, 1) OVER (ORDER BY month)` returns the sales value from the previous month. Useful for calculating differences between consecutive rows, like month-over-month growth. See Chapter 26.

LEAD

A window function that accesses a value from a subsequent row. `LEAD(sales, 1) OVER (ORDER BY month)` returns the sales value from the next month. LEAD is the opposite of LAG. See Chapter 26.

LEFT JOIN (LEFT OUTER JOIN)

A join that returns all rows from the left table and matching rows from the right table. If a left table row has no match in the right table, the right table's columns are filled with NULL. `SELECT * FROM customers LEFT JOIN orders ON customers.id = orders.customer_id` includes customers even if they have no orders. See Chapter 14.

LIKE

A pattern-matching operator used in WHERE clauses. `%` matches any sequence of characters, _ matches any single character. `WHERE name LIKE 'A%'` finds names starting with "A." `WHERE code LIKE '__-___'` finds codes in the format "XX-XXX." ILIKE is the case-insensitive version in PostgreSQL. See Chapter 8.

LIMIT

A clause that restricts the number of rows returned by a query. `SELECT * FROM products LIMIT 10` returns only the first 10 rows. Often used with ORDER BY and OFFSET for pagination. See Chapter 9.

Lock

A mechanism that prevents multiple transactions from modifying the same data simultaneously. When a transaction updates a row, it places a lock on that row so other transactions must wait. Locks prevent data corruption but can cause blocking and deadlocks if not managed carefully. See Chapter 25.

Materialized View

A view that stores its query results physically on disk, like a cached snapshot. Unlike a regular view (which re-executes the query each time), a materialized view returns pre-computed results instantly. You refresh it manually with `REFRESH MATERIALIZED VIEW`. Useful for expensive queries that do not need real-time data. See Chapter 20.

Normalization

The process of organizing a database to reduce redundancy and improve data integrity. Normalization splits data into multiple related tables. The main normal forms are 1NF (no repeating groups), 2NF (no partial dependencies), and 3NF (no transitive dependencies). Normalization trades some read performance for better data consistency. See Chapter 22.

NOT NULL

A constraint that prevents a column from containing NULL values. Every INSERT and UPDATE must provide a value for a NOT NULL column. `CREATE TABLE users (name VARCHAR(100) NOT NULL)` ensures every user has a name. Most columns in a well-designed database should be NOT NULL. See Chapter 24.

NULL

A special marker that represents a missing or unknown value. NULL is not the same as zero, an empty string, or false. Comparisons with NULL use `IS NULL` or `IS NOT NULL` instead of `=` or `!=`. NULL has special behavior in aggregates, sorting, and boolean logic. See Chapter 8.

OFFSET

A clause that skips a specified number of rows before returning results. `SELECT * FROM products ORDER BY name LIMIT 10 OFFSET 20` skips the first 20 rows and returns rows 21 through 30. Used with LIMIT for pagination. Large OFFSET values can be slow because the database must still process the skipped rows. See Chapter 9.

ON DELETE CASCADE

A foreign key option that automatically deletes child rows when the parent row is deleted. For example, if an order is deleted and its order_items have ON DELETE CASCADE, all items for that order are automatically removed. See Chapter 24.

ORDER BY

A clause that sorts query results by one or more columns. `ORDER BY price ASC` sorts cheapest first. `ORDER BY created_at DESC` sorts newest first. You can sort by multiple columns: `ORDER BY department ASC, salary DESC`. Without ORDER BY, row order is unpredictable. See Chapter 9.

OUTER JOIN

A general term for joins that include rows even when there is no match in the other table. LEFT OUTER JOIN, RIGHT OUTER JOIN, and FULL OUTER JOIN are all outer joins. The "OUTER" keyword is optional — `LEFT JOIN` and `LEFT OUTER JOIN` are identical. See Chapters 14 and 15.

Partition

A method of dividing a large table into smaller, more manageable pieces called partitions. Each partition holds a subset of the data based on a partition key (like date ranges or regions). Queries that filter by the partition key only scan the relevant partition, improving performance on very large tables. See Chapter 30.

PL/pgSQL

PostgreSQL's built-in procedural language for writing functions, procedures, and triggers. PL/pgSQL extends SQL with variables, loops, conditionals, and exception handling. It runs inside the database server, avoiding the overhead of sending queries back and forth from an application. See Chapter 28.

PRIMARY KEY

A constraint that uniquely identifies each row in a table. A primary key column (or combination of columns) must be unique and cannot contain NULL. Each table can have only one primary key. PostgreSQL automatically creates a B-tree index on the primary key. See Chapter 24.

Privilege

A permission granted to a user or role that controls what actions they can perform. Common privileges include SELECT (read), INSERT (add), UPDATE (modify), DELETE (remove), and ALL (everything). Privileges are granted with GRANT and removed with REVOKE. See Chapter 31.

Query Plan

The step-by-step strategy PostgreSQL uses to execute a query. The query planner considers available indexes, table sizes, and statistics to choose the most efficient approach. Use `EXPLAIN` or `EXPLAIN ANALYZE` to view the query plan. Understanding query plans is essential for performance tuning. See Chapter 30.

RANK

A window function that assigns a rank to each row within a partition based on the ORDER BY values. Rows with equal values receive the same rank, and the next rank is skipped. `RANK() OVER (ORDER BY score DESC)` might produce ranks 1, 2, 2, 4 (rank 3 is skipped because two rows tied for rank 2). See Chapter 26.

Recursive CTE

A Common Table Expression that references itself, allowing you to traverse hierarchical or graph-like data. For example, you can find all employees in a management chain by starting with the CEO and recursively following the manager_id relationship. Recursive CTEs use `WITH RECURSIVE`. See Chapter 27.

Referential Integrity

The guarantee that every foreign key value in a child table corresponds to an existing primary key value in the parent table. Referential integrity prevents orphan records — rows that reference data that does not exist. Foreign key constraints enforce referential integrity automatically. See Chapter 24.

Relation

In relational database theory, a relation is a table. It consists of a set of tuples (rows) with the same attributes (columns). The term comes from the mathematical concept of a relation, not from "relationships between tables." The relational model was proposed by Edgar F. Codd in 1970. See Chapter 2.

RETURNING

A PostgreSQL clause that returns data from rows affected by INSERT, UPDATE, or DELETE. `INSERT INTO customers (name) VALUES ('Alice') RETURNING customer_id` returns the newly generated ID. `DELETE FROM logs WHERE age > 30 RETURNING *` shows which rows were deleted. RETURNING saves you from needing a separate SELECT query. See Chapter 6.

REVOKE

A command that removes previously granted permissions from a user or role. `REVOKE INSERT ON customers FROM intern_role` takes away the ability to add data. REVOKE is the opposite of GRANT. See Chapter 31.

Role

A named collection of database permissions in PostgreSQL. Roles can represent individual users or groups. A role can own database objects, have specific privileges, and inherit permissions from other roles. PostgreSQL uses roles instead of separate "user" and "group" concepts. See Chapter 31.

ROLLBACK

A command that undoes all changes made during the current transaction. After ROLLBACK, the database returns to the state it was in before BEGIN. Use ROLLBACK when an error occurs or when you decide not to proceed with the changes. See Chapter 25.

ROW_NUMBER

A window function that assigns a unique sequential number to each row within a partition. Unlike RANK, ROW_NUMBER never produces duplicates — even rows with identical values get different numbers. `ROW_NUMBER() OVER (ORDER BY created_at)` numbers rows 1, 2, 3, 4 regardless of ties. See Chapter 26.

Schema

A named container within a database that organizes tables, views, functions, and other objects. Think of a schema like a folder. The default schema in PostgreSQL is `public`. You can create additional schemas to organize objects by purpose: `CREATE SCHEMA sales`. Refer to objects as `schema_name.table_name`. See Chapter 4.

SELECT

The most used SQL command. SELECT retrieves data from one or more tables. A basic SELECT includes columns to return and a FROM clause specifying the table. Additional clauses like WHERE, JOIN, GROUP BY, ORDER BY, and LIMIT refine the results. See Chapter 7.

Self Join

A join where a table is joined to itself. Useful for comparing rows within the same table or traversing hierarchical data. For example, finding employees and their managers from the same employees table: `SELECT e.name, m.name AS manager FROM employees e JOIN employees m ON e.manager_id = m.employee_id`. See Chapter 15.

Sequence

A database object that generates a series of unique numbers. Sequences are the engine behind SERIAL and BIGSERIAL columns. You can create custom sequences with `CREATE SEQUENCE`, get the next value with `nextval('sequence_name')`, and see the current value with `currval('sequence_name')`. Sequences are guaranteed to be unique even with concurrent access.

SERIAL

A pseudo data type in PostgreSQL that creates an auto-incrementing integer column. `SERIAL` is shorthand for creating a sequence and setting the column's default to `nextval()`. SERIAL produces integers from 1 to about 2.1 billion. Use BIGSERIAL for larger ranges. In newer PostgreSQL versions, `GENERATED ALWAYS AS IDENTITY` is the preferred alternative. See Chapter 5.

Subquery

A query nested inside another query. Subqueries can appear in SELECT, FROM, WHERE, and HAVING clauses. A subquery in WHERE acts as a filter: `SELECT * FROM products WHERE price > (SELECT AVG(price) FROM products)`. A subquery in FROM acts as a temporary table. See Chapter 16.

TABLE

A database object that stores data in rows and columns. Tables are the fundamental building blocks of a relational database. Each table represents an entity (like customers, orders, or products). Tables are created with `CREATE TABLE` and removed with `DROP TABLE`. See Chapter 4.

Transaction

A sequence of SQL statements that are executed as a single unit of work. Either all statements succeed (COMMIT) or all fail (ROLLBACK). Transactions follow ACID properties to ensure data integrity. In PostgreSQL, every single statement runs inside an implicit transaction even if you do not use BEGIN. See Chapter 25.

Trigger

A function that automatically runs in response to a specific event on a table (INSERT, UPDATE, DELETE, or TRUNCATE). Triggers can fire BEFORE or AFTER the event. For example, a trigger can automatically update an `updated_at` timestamp whenever a row is modified. See Chapter 29.

TRUNCATE

A DDL command that removes all rows from a table quickly. `TRUNCATE TABLE logs` is faster than `DELETE FROM logs` because it does not scan individual rows. TRUNCATE resets auto-increment sequences and cannot be used with a WHERE clause. In PostgreSQL, TRUNCATE is transactional. See Chapter 4.

UNION

A set operation that combines the results of two or more SELECT queries into one result set. `UNION` removes duplicates. `UNION ALL` keeps duplicates and is faster. Both queries must have the same number of columns with compatible data types. See Chapter 17.

UNIQUE

A constraint that ensures no two rows have the same value in a column (or combination of columns). Unlike PRIMARY KEY, a table can have multiple UNIQUE constraints, and UNIQUE columns can contain NULL values. PostgreSQL automatically creates an index on UNIQUE columns. See Chapter 24.

UPDATE

A DML command that modifies existing rows in a table. `UPDATE products SET price = 49.99 WHERE product_id = 5` changes one row. Always use a WHERE clause unless you intend to update every row. UPDATE can use JOINs, subqueries, and RETURNING. See Chapter 6.

Upsert

An operation that inserts a row if it does not exist or updates it if it does. PostgreSQL implements upsert with `INSERT ... ON CONFLICT DO UPDATE`. For example: `INSERT INTO products (id, name, price) VALUES (1, 'Laptop', 999) ON CONFLICT (id) DO UPDATE SET price = EXCLUDED.price`. This avoids separate "check then insert or update" logic.

UUID (Universally Unique Identifier)

A 128-bit value used as a unique identifier. UUIDs look like `550e8400-e29b-41d4-a716-446655440000`. Unlike SERIAL (which depends on a sequence), UUIDs can be generated independently by any system without risk of collision. PostgreSQL provides the `gen_random_uuid()` function. UUIDs are useful for distributed systems where multiple servers create records.

VACUUM

A PostgreSQL maintenance command that reclaims storage occupied by dead tuples (rows that have been deleted or updated). PostgreSQL uses MVCC, which means old row versions are not immediately removed. VACUUM cleans them up. `VACUUM ANALYZE` also updates statistics used by the query planner. PostgreSQL runs autovacuum automatically, but heavily updated tables may need manual vacuuming. See Chapter 30.

View

A virtual table defined by a stored SELECT query. A view does not store data — it runs the query every time you select from it. Views simplify complex queries, provide a layer of abstraction, and can restrict which columns or rows a user can see. Create with `CREATE VIEW view_name AS SELECT ...`. See Chapter 20.

WHERE

A clause that filters rows based on a condition. `SELECT * FROM orders WHERE total > 100` returns only orders with a total above 100. WHERE supports comparison operators (`=`, `>`, `<`, `!=`), logical operators (`AND`, `OR`, `NOT`), pattern matching (`LIKE`), and more. WHERE filters individual rows before grouping — use HAVING to filter groups. See Chapter 8.

Window Function

A function that performs a calculation across a set of rows related to the current row, without collapsing them into a single output row. Unlike aggregate functions with GROUP BY, window functions keep all rows in the result. They use the `OVER()` clause to define the window. Common window functions include `ROW_NUMBER()`, `RANK()`, `LAG()`, `LEAD()`, `SUM() OVER()`, and

`AVG() OVER()`. See Chapter 26.

WITH

The keyword that introduces a Common Table Expression (CTE). `WITH monthly_sales AS (SELECT ...) SELECT * FROM monthly_sales` defines a temporary named result set. `WITH RECURSIVE` creates a recursive CTE for hierarchical queries. The WITH clause appears at the beginning of a query, before the main SELECT. See Chapter 27.

Quick Reference: Constraint Types

```
+-------------------+-------------------------------------------------+
| Constraint        | Purpose                                         |
+-------------------+-------------------------------------------------+
| PRIMARY KEY       | Unique row identifier, no NULLs                 |
| FOREIGN KEY       | References another table, enforces relationships |
| UNIQUE            | No duplicate values (NULLs allowed)             |
| NOT NULL          | Value required, no NULLs                        |
| CHECK             | Custom validation rule                          |
| DEFAULT           | Auto-fill value when none given                 |
+-------------------+-------------------------------------------------+
```

Quick Reference: JOIN Types

```
+-------------------+-------------------------------------------------+
| JOIN Type         | Returns                                         |
+-------------------+-------------------------------------------------+
| INNER JOIN        | Only matching rows from both tables             |
| LEFT JOIN         | All left rows + matching right rows             |
| RIGHT JOIN        | All right rows + matching left rows             |
| FULL JOIN         | All rows from both tables                       |
| CROSS JOIN        | Every combination of rows (Cartesian product)   |
| SELF JOIN         | Table joined to itself                          |
+-------------------+-------------------------------------------------+
```

Quick Reference: Common Commands

```
+-------------------+-----------+-------------------------------------+
| Command           | Category  | Purpose                             |
+-------------------+-----------+-------------------------------------+
| SELECT            | DML       | Read data                           |
| INSERT            | DML       | Add rows                            |
| UPDATE            | DML       | Modify rows                         |
| DELETE            | DML       | Remove rows                         |
| CREATE TABLE      | DDL       | Create a table                      |
| ALTER TABLE       | DDL       | Modify table structure              |
| DROP TABLE        | DDL       | Remove a table                      |
| TRUNCATE          | DDL       | Remove all rows quickly             |
| BEGIN             | TCL       | Start a transaction                 |
| COMMIT            | TCL       | Save transaction changes            |
| ROLLBACK          | TCL       | Undo transaction changes            |
| GRANT             | DCL       | Give permissions                    |
| REVOKE            | DCL       | Remove permissions                  |
+-------------------+-----------+-------------------------------------+
```

Congratulations

You have reached the end of this book. Take a moment to appreciate how far you have come.

When you started, a database was a mysterious black box. Now you can:

- Design tables with proper data types and constraints
- Write queries that filter, sort, group, and join data across multiple tables
- Use subqueries, CTEs, and window functions for advanced analysis
- Create views and indexes to simplify and speed up your queries
- Normalize your data to eliminate redundancy
- Draw ER diagrams to plan and communicate your designs
- Write transactions that keep your data safe and consistent
- Build stored procedures and triggers for automation
- Tune queries for performance using EXPLAIN
- Manage security with roles and privileges
- Build complete database-backed applications from scratch

That is a substantial skill set. Many working professionals do not know half of what you have learned.

What to Learn Next

Your SQL and database journey does not end here. Here are paths you can explore:

Deepen your PostgreSQL knowledge. Explore advanced features like table inheritance, custom data types, extensions (PostGIS for geographic data, pg_trgm for fuzzy text search), and logical replication.

Learn database administration. Understand backup and restore strategies, monitoring tools, connection pooling, high availability with streaming replication, and disaster recovery planning.

Explore other databases. Try MySQL, SQLite, or SQL Server to see how they differ from PostgreSQL. Learn a NoSQL database like MongoDB or Redis to understand when relational databases are not the best fit.

Build real applications. Connect PostgreSQL to a programming language you know — Python with psycopg2 or SQLAlchemy, JavaScript with node-postgres or Prisma, Java with JDBC or Hibernate. Build a project that solves a real problem for you.

Study data engineering. Learn about ETL pipelines, data warehousing, columnar databases, and tools like dbt for data transformation.

Practice with real data. Download public datasets (government data, Kaggle datasets, public APIs) and load them into PostgreSQL. Ask questions of the data. Build dashboards. The best way to solidify your skills is to use them on problems you care about.

The most important thing is to keep writing SQL. Like any skill, database work gets better with practice. Every query you write, every schema you design, and every bug you debug makes you stronger.

Good luck, and happy querying.